

Structured Computer Architecture: An Order-Based Approach

John Glossner and Gheorghe M. Ștefan

January 1, 2025

Preliminary Release v0.1

FRONT MATTER

Copyright

Copyright Notice

Part I: Number Systems and Representations is copyright © 2025 by John Glossner and Gheorghe M. Ștefan and licensed under a Creative Commons Attribution 4.0 International License (CC BY).

Parts II through IV and all other content is copyright © 2025 by John Glossner and Gheorghe M. Ștefan. and licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License (CC BY-NC-ND 4.0).

Some portions of this document contain edited text originally generated by ChatGPT. Primarily Part 1 and the code appendices.

Code License

Chisel Scala code is licensed under the BSD 3-Clause License. See <https://opensource.org/licenses/BSD-3-Clause> for details.

Hand written SystemVerilog and Verilog code is licensed under CERN-OHL-S v2. See <https://cern.ch/cern-ohl> for details.

Typesetting

This document was prepared with $\text{\LaTeX}2_{\epsilon}$

Question: *Can we have a procedure that, receiving a program and the data it processes, decides if the program will stop executing in a finite time?*

This (halting) problem was formulated in computational terms in 1936 by Alonzo Church, Stephen Kleene, Emil Post, and Alan Turing as a reaction to the incompleteness theorem published by Kurt Gödel in 1931.

Answer: *No!*

And thus, the most important negative result in the history of mathematics underpins the science of computation.

Preface

This textbook arose from an introductory course on Computer Architecture taught at Rivier University in the Fall of 2024. It draws on Gheorghe M. Ștefan's years of teaching Computer Architecture at the University of Bucharest.

Unlike other Computer Architecture textbooks, this book approaches the subject from a Computer Science theoretical perspective. Using an ordered approach, we start with the most basic 0-order computing system - one with stateless operations. We then describe computing systems in terms of their feedback loop orders progressing order-by-order (Stefan, 2024). Table 1 shows the progression of computing systems feedback loops.

Order	Definition	Components	Example
0-Order	No loops. Stateless systems where output is determined by current input only.	Logic gates (AND, OR, NOT).	Simple combinational circuits, e.g., Multiplexer, adders, ALU.
1st-Order	One loop. Memory Circuits	Flip-flops, latches, registers, RAM, SRAM, ROM.	D flip-flop.
2nd-Order	Two loops. Automata.	Mealy, Moore automaton	Counters, FSMs, ALUs with registers.
3rd-Order	Three loops. Processing units executing instructions.	ALU, control units, registers.	Simple microprocessor, e.g., CPU.
4th-Order	Four loops. Processor plus one Memory	Program and Data in single memory.	von Neumann abstract machine.
5th-Order	Five loops. Processor with two memories.	Separate Program and Data memories.	Harvard abstract machine.
nth-Order	N loops. Multi-processor systems.	SCAN and REDUCE	TBD

Table 1: Orders of Computing Systems

Part I of this book provides a mathematical foundation for the number systems used in computing systems. Part II covers orders 0 through 3 - basic combinational logic through a processor. Part III covers the theory and implementation of computing including parallel heterogeneous systems.

In our opinion, an advantage of the order-based approach is that it roughly follows the historical

development of computing systems. We also ground these developments in the theoretical models that provided for their development.

While primarily written for our students, we both have industry experience and have shipped multiple machines in volume. The code we provide bridges the gap between theoretical models and commercial implementations. The code starts very simple using behavioral constructs but progresses towards code that can be used in commercial machines. Over time, it is our hope that these examples continue to expand.

The descriptions of the structures and the simulation of their behavior are written in both SystemVerilog and Chisel HDL. The Chisel code has unit tests and has been tested using Chisel 6.6.0. Some of the SystemVerilog code has unit tests. All SystemVerilog code has been validated using the AMD/Xilinx Vivado Design Suite ¹. It should also execute using online tools such as EDA Playground ².

Because this book's code is executable, it should not contain any syntax errors. The same files that are used for design descriptions are directly imported into the book.

This preliminary release we know is quite rough. However, it will become polished with time. The best way to help this effort is to send pull requests to https://github.com/glossner/Structured_Computer_Architecture.

John Glossner and Gheorghe M. Ștefan

January 1st, 2025

¹<https://www.xilinx.com/content/xilinx/en/support/download.html/>

²<https://www.edaplayground.com>

Contents

Copyright	iii
Part 1 is copyright © 2025 CC-BY by John Glossner and Gheorghe M. Ștefan	iii
All other content is copyright © 2025 CC-BY-NC-ND 4.0 by John Glossner and Gheorghe M. Ștefan	iii
Chisel Scala Code License - BSD 3-clause	iii
Hand Written Verilog Code License - CERN-OHL-S v2	iii
Preface	v
Contents (detailed)	xi
List of Figures	xxix
List of Tables	xxxiv
 I NUMBER SYSTEMS AND REPRESENTATIONS	 1
1 Number Systems	3
1.1 A Brief History of Number Systems	4
1.2 Decimal Arithmetic	6
1.3 Positional Notation	13
1.4 Binary Arithmetic	15
2 Numerical Sets	19
2.1 Formal Notation for Numerical Sets	20
2.2 The Set of Integers ($\mathbb{Z}$)	20
2.3 The Set of Real Numbers ($\mathbb{R}$)	21
2.4 Cauchy Sequences	22
2.5 $\mathbb{R}$ as a Metric Space	23
2.6 Comparing the Properties of $\mathbb{Z}$ and $\mathbb{R}$	25
2.7 The Set of Rational Numbers ($\mathbb{Q}$)	27
2.8 The Set of Complex Numbers ($\mathbb{C}$)	28
3 Finite Unsigned Integer Binary Representations in $\mathbb{N}$	31
3.1 Sets of Numbers and Representations of Numbers	32
3.2 Unsigned Binary Numbers	33

3.3	Range of Unsigned Binary Numbers	33
3.4	Example Ranges for Different Bit Lengths	34
3.5	B_n with Overflow and Underflow (Modulo Arithmetic)	35
3.6	Definition of B_n with Clipping (Saturation Arithmetic)	37
4	Finite Signed Binary Integer Representations in $\mathbb{Z}$	39
4.1	Signed-Magnitude Representation	40
4.2	One's Complement Representation	43
4.3	Two's Complement Representation	47
4.4	Excess- k Representation	52
4.5	Summary Table of Common Signed Number Representations	57
4.6	Binary Coded Decimal (BCD) Representation	59
4.7	Other Representations for Signed Numbers	63
5	Finite Real Number Representations in $\mathbb{R}$	65
5.1	Definition and Characteristics	66
5.2	Floating Point Representation of Real Numbers	66
5.3	Fixed-Point Representation	73
5.4	Examples of Finite Real Numbers	78
5.5	Challenges in Finite Real Representations	78
5.6	Alternative Finite Number Representations.	79
6	Character Representations	81
6.1	Encoding Alphabetic Characters in Computers	82
6.2	Early Character Encodings: EBCDIC and ASCII	82
6.3	Modern Character Encodings	83
7	Summary of Number Systems and Representations	85
II	FROM GATES TO PROCESSORS	87
8	Combinational Circuits: No-Loop, Zero-Order Systems (0-OS)	89
8.1	Boolean Algebra	91
8.2	Truth Tables	93
8.3	Digital Domain	95
8.4	One-Input Combinational Circuits	103
8.5	Two-input gates	104
8.6	Many-Input Gates	107
8.7	Generic combinational circuits	108
8.8	Behavioral vs. Structural Descriptions	131
8.9	Adders	132
8.10	Multipliers	139
8.11	Multi-Function Arithmetic Units	143
8.12	<i>Arithmetic & Logic Unit</i>	155
8.13	Problems	161
8.14	Exercises	166

9	Memory Circuits: One-Loop, First-Order Systems (1-OS)	169
9.1	Latch	170
9.2	Serial extension: Master-Slave Principle	173
9.3	Serial-parallel extension: Register	175
9.4	Parallel extension: Random-Access Memory	177
9.5	Problems	185
10	Automata: Two-Loop, Second-Order Systems (2-OS)	189
10.1	Definitions	190
10.2	Finite (complex) Automata	195
10.3	Simple Automata	202
10.4	Problems	211
11	Processors: Three-Loop, Third-Order Systems (3-OS)	215
11.1	Counter-Expanded Automata	216
11.2	Processor	218
11.3	<i>ToyRISC Processor</i>	221
11.4	How is Designed an Instruction Set Architecture	233
11.5	Problems	234
III	COMPUTING	237
12	Computers: Four-Loop, Fourth-Order Systems (4-OS)	239
12.1	The mathematical model of the Turing Machine	240
12.2	von Neumann Abstract Model	244
12.3	System Organization	244
12.4	Memory Management	246
12.5	I/O	253
13	Computers: Five-Loop, Fifth-Order Systems (5-OS)	265
13.1	Harvard Computer Version	266
13.2	Cache Memories	266
14	Parallel Computation: N-Loop, N-Order systems (N-OS)	267
14.1	Cellular Automata (CA)	268
14.2	Controlled Cellular Automaton	270
14.3	First Global Loop in CA	274
14.4	Second Global Loop in CA	277
14.5	The mathematical model of partially recursive functions	279
14.6	Heterogeneous Computing	281
14.7	Parallel Computing Patterns	288
IV	PRACTICAL DESIGNS	297
15	Performance Optimized Digital Logic	299
15.1	Comparison of when and MuxCase in Chisel	300

15.2	Carry Lookahead Adder	301
15.3	Carry Select Adder	301
15.4	Carry Save Adder	302
15.5	Kogge-Stone Adder	302
15.6	Multipliers	302
16	Instruction-Level Parallelism	303
16.1	Pipelining	304
16.2	Hazards Generated by Dependencies	309
16.3	Programming Pipelined Processors	321
16.4	Superscalar Processor	323
16.5	Problems	327
17	ToyRISC Design	329
17.1	Code	329
17.2	Assembly	340
17.3	Simulation	346
17.4	toyRISC Structural Implementation	349
17.5	Pipelined toyRISC	367
17.6	Forwarding toyRISC	386
18	RISC-V TBD	409
19	RISC-V Multithreaded TBD	411
20	Using the Heterogeneous System hRISC	413
20.1	Micro-architectures	417
20.2	Heterogeneous architecture	425
20.3	Programming heterogeneous system	430
	APPENDIXES	444
A	Hardware Description Languages	447
A.1	Chisel	447
A.2	System Verilog	473
A.3	Chisel Compared with System Verilog	478
B	Physical Implementations of Logic Gates	483
B.1	Physical Implementation of NOT Circuit	483
B.2	Physical Implementation of NAND & NOR Gates	488
C	Simulations	491
C.1	Combo Simulations	491
C.2	Memory Simulations	492
D	FIFO memory	495

E Complete Code Listing	499
E.1 SystemVerilog	500
E.2 Chisel	503
Bibliography	521
Index	527

Contents (detailed)

Copyright	iii
Part 1 is copyright © 2025 CC-BY by John Glossner and Gheorghe M. Ștefan	iii
All other content is copyright © 2025 CC-BY-NC-ND 4.0 by John Glossner and Gheorghe M. Ștefan	iii
Chisel Scala Code License - BSD 3-clause	iii
Hand Written Verilog Code License - CERN-OHL-S v2	iii
Preface	v
Contents (detailed)	xi
List of Figures	xxix
List of Tables	xxxiv

I NUMBER SYSTEMS AND REPRESENTATIONS	1
1 Number Systems	3
1.1 A Brief History of Number Systems	4
1.1.1 Early Development	4
1.1.2 Evolution of Mathematical Sets	4
Integers ($\mathbb{Z}$).	4
Rational Numbers ($\mathbb{Q}$).	4
Real Numbers ($\mathbb{R}$).	4
Complex Numbers ($\mathbb{C}$).	4
1.1.3 Pre-Computer Number Representations	5
Decimal Positional Systems.	5
10's Complement for Subtraction.	5
Logarithmic Representations.	5
1.1.4 Number Representations in Computing	5
Binary and Two's Complement.	5

	Excess- k Representations.	5
	Fixed-Point Arithmetic.	6
	Floating-Point Systems.	6
	Binary Coded Decimal (BCD).	6
1.1.5	Modern Trends and Challenges	6
1.2	Decimal Arithmetic	6
1.2.1	Decimal Addition	6
1.2.2	Decimal Multiplication	7
1.2.3	Decimal Subtraction	9
1.2.4	Decimal Division	10
1.2.5	Alternative Decimal Arithmetic Algorithms	11
1.2.6	10's Complement	12
1.3	Positional Notation	13
1.3.1	Positional Notation in Base 10	13
1.3.2	Example of Positional Notation in Base 10	13
1.3.3	Importance of Positional Notation	13
1.3.4	Positional Notation for Arithmetic Operations	14
1.3.5	Base/Radix Concepts	14
1.3.6	Generalized Positional Notation	14
1.3.7	Positional Notation for Different Bases	14
1.4	Binary Arithmetic	15
1.4.1	Binary Addition	15
1.4.2	Binary Subtraction	16
1.4.3	Binary Multiplication - TBD	17
1.4.4	Binary Division - TBD	17
2	Numerical Sets	19
2.1	Formal Notation for Numerical Sets	20
2.2	The Set of Integers ($\mathbb{Z}$)	20
2.2.1	Properties of $\mathbb{Z}$	20
2.2.2	Subsets of $\mathbb{Z}$	21
2.3	The Set of Real Numbers ($\mathbb{R}$)	21
2.3.1	Properties of $\mathbb{R}$	21
2.3.2	Subsets of $\mathbb{R}$	22
2.4	Cauchy Sequences	22
2.4.1	Cauchy Sequence Definition	22
2.4.2	Properties of Cauchy Sequence	23
2.4.3	Importance	23
2.5	$\mathbb{R}$ as a Metric Space	23
2.5.1	Properties	23
2.5.1.1	$\mathbb{R}$ with the Standard Metric	24
2.5.2	Standard Metric Properties:	24
2.5.3	Completeness of $\mathbb{R}$	24
2.5.4	Alternative Metrics on $\mathbb{R}$	24
2.5.5	Historical Context	25
2.5.6	Applications	25

2.6	Comparing the Properties of $\mathbb{Z}$ and $\mathbb{R}$	25
2.6.1	Properties of $\mathbb{Z}$ Not Shared by $\mathbb{R}$	25
2.6.2	Properties of $\mathbb{R}$ Not Shared by $\mathbb{Z}$	26
2.6.3	Summary Table of Differences between $\mathbb{Z}$ and $\mathbb{R}$	27
2.7	The Set of Rational Numbers ($\mathbb{Q}$)	27
2.7.1	Properties of $\mathbb{Q}$	27
2.7.2	Subsets of $\mathbb{Q}$	28
2.8	The Set of Complex Numbers ($\mathbb{C}$)	28
2.8.1	Properties of $\mathbb{C}$	29
3	Finite Unsigned Integer Binary Representations in $\mathbb{N}$	31
3.1	Sets of Numbers and Representations of Numbers	32
3.1.1	Sets of Numbers	32
3.1.2	Representations of Numbers	32
3.2	Unsigned Binary Numbers	33
3.3	Range of Unsigned Binary Numbers	33
3.4	Example Ranges for Different Bit Lengths	34
3.5	B_n with Overflow and Underflow (Modulo Arithmetic)	35
3.5.1	Overflow and Underflow Behavior	35
3.5.2	Computer Arithmetic Example Overflow	36
3.5.3	Computer Arithmetic Example Underflow	36
3.6	Definition of B_n with Clipping (Saturation Arithmetic)	37
3.6.1	Behavior of Arithmetic Operations with Clipping	37
3.6.2	Computer Arithmetic Example: Overflow with Clipping	37
3.6.3	Computer Arithmetic Example: Underflow with Clipping	38
4	Finite Signed Binary Integer Representations in $\mathbb{Z}$	39
4.1	Signed-Magnitude Representation	40
4.1.1	Definition	40
4.1.2	Properties of Signed-Magnitude Representation	41
4.1.3	Arithmetic with Signed-Magnitude Representation	42
	Addition	42
	Subtraction	43
4.1.4	Trade-offs of Signed-Magnitude Representation	43
4.2	One's Complement Representation	43
4.2.1	Definition	44
4.2.2	Properties of One's Complement Representation	44
4.2.3	Arithmetic with One's Complement Representation	45
	Addition	45
	Subtraction	46
4.2.4	Trade-offs of One's Complement Representation	46
4.3	Two's Complement Representation	47
4.3.1	Definition	47
4.3.2	Properties of Two's Complement Representation	47
4.3.3	Arithmetic with Two's Complement Representation	49
	Addition	49

	Subtraction	49
	Multiplication	50
4.3.4	Intermediate Overflow in 2's Complement Arithmetic	50
	Example of Intermediate Overflow.	50
	Key Property of 2's Complement Arithmetic.	51
	Practical Implications.	52
4.3.5	Trade-offs of Two's Complement Representation	52
4.4	Excess- k Representation	52
4.4.1	Definition	52
4.4.2	Properties of Excess- k Representation	53
4.4.3	Arithmetic with Excess- k Representation	54
	Comparisons	54
	Addition	55
	Subtraction	56
4.4.4	Trade-offs of Excess- k Representation	57
4.5	Summary Table of Common Signed Number Representations	57
4.6	Binary Coded Decimal (BCD) Representation	59
4.6.1	Definition of BCD	59
4.6.2	Properties of Binary Coded Decimal (BCD)	59
4.6.3	Arithmetic using BCD	60
	Addition	60
	Subtraction	60
4.6.4	Trade-offs of BCD Representation	61
4.6.5	Applications of BCD	61
4.6.6	Example: Floating-Point vs. BCD in Financial Applications	62
	Floating-Point Representation:	62
	BCD Representation:	62
4.7	Other Representations for Signed Numbers	63
	Sign-and-Logarithm Representation:	63
	Gray Code:	63
	Offset Binary (Biased Binary:)	63
	Base-2 Signed-Digit (SD) Representation:	64
	Balanced Ternary:	64
	Segmented Representations:	64
	Residue Number System (RNS):	64
	Summary:	64
5	Finite Real Number Representations in $\mathbb{R}$	65
5.1	Definition and Characteristics	66
5.2	Floating Point Representation of Real Numbers	66
5.2.1	General Floating-Point Format	66
5.2.2	Rounding in Floating-Point Arithmetic	67
	IEEE 754 Rounding Modes.	67
	Examples of Rounding Modes.	68
	Summary of Rounding Modes.	68
5.2.3	The Role of Denormalized Numbers in Floating-Point Arithmetic	68

	Discrete Spacing in Standard Floating-Point.	69
	Limitations Near Zero.	70
	Denormalized Numbers Improve the Metric Space.	70
5.2.4	IEEE Supported Formats	71
5.2.5	Half-Precision Floating-Point (f16)	72
	Value Calculation:	72
	Special Cases in f16:	72
5.2.6	Example: Representing 6.25 in f16	73
5.2.7	Applications and Trade-Offs	73
5.3	Fixed-Point Representation	73
5.3.1	Structure of Fixed-Point Numbers	74
5.3.2	Range of Signed Fixed-Point Representation	74
5.3.3	Properties of Fixed-Point Arithmetic	74
	Fixed Precision and Scaling.	75
	Deterministic Arithmetic.	75
	Overflow and Underflow Behavior.	76
	Rounding and Truncation.	77
5.3.4	Fixed Point Considerations	77
	Trade-offs Between Precision and Range.	77
	Efficient Hardware Implementation.	77
5.3.5	Comparison with Floating-Point Representation	77
5.4	Examples of Finite Real Numbers	78
5.5	Challenges in Finite Real Representations	78
	Rounding Errors.	78
	Overflow and Underflow.	78
	Loss of Associativity.	78
5.6	Alternative Finite Number Representations.	79
	Interval arithmetic	79
	Logarithmic number systems (LNS)	79
	Tapered floating-point	79
	Brain Floating Point	79
6	Character Representations	81
6.1	Encoding Alphabetic Characters in Computers	82
6.2	Early Character Encodings: EBCDIC and ASCII	82
6.2.1	EBCDIC (Extended Binary Coded Decimal Interchange Code)	82
6.2.2	ASCII (American Standard Code for Information Interchange).	82
6.3	Modern Character Encodings	83
6.3.1	Unicode	83
6.3.2	UTF Encodings	83
	UTF-8	83
	UTF-16.	83
	UTF-32.	83
6.3.3	Additional Encodings	83
	Base64.	83
	MIME Encodings.	83

Custom Encodings.	83
6.3.4 Comparison of Encoding Schemes	84
7 Summary of Number Systems and Representations	85
 II FROM GATES TO PROCESSORS	 87
8 Combinational Circuits: No-Loop, Zero-Order Systems (0-OS)	89
8.1 Boolean Algebra	91
8.1.1 Historical Background	91
8.1.2 Basic Concepts	91
8.1.3 Operations	91
AND ($\wedge$).	91
OR ($\vee$).	91
NOT ($\neg$).	91
8.1.4 Laws	91
Commutative Law:	91
Associative Law:	92
Distributive Law:	92
Identity Law:	92
Complement Law:	92
8.1.5 DeMorgan's Laws	92
8.1.5.1 Applications of DeMorgan's Laws	93
8.1.6 Applications in Digital Logic	93
8.1.6.1 Simplification of Logic Expressions	93
8.2 Truth Tables	93
8.2.1 Definition and Structure of Truth Tables	93
8.2.2 Logical Operators and Their Truth Tables	94
8.2.3 Applications of Truth Tables	94
8.2.4 Constructing Truth Tables for Complex Expressions	95
8.2.5 Limitations of Truth Tables	95
8.3 Digital Domain	95
8.3.1 Signals in the Digital Domain	95
8.3.1.1 SystemVerilog Implementation of Waveforms	96
8.3.1.2 Chisel Implementation of Waveforms	97
Trait - Scala	98
8.3.1.3 Comparison of Chisel and SystemVerilog Implementation of Waveforms	102
8.4 One-Input Combinational Circuits	103
8.4.1 Formal Description	103
8.4.2 Physical Implementation of NOT Circuit	103
8.5 Two-input gates	104
8.5.1 Formal Description	104
8.6 Many-Input Gates	107
8.7 Generic combinational circuits	108
8.7.1 Decoder	108

8.7.1.1	Decoder (SystemVerilog)	109
8.7.1.2	Decoder (Chisel)	110
8.7.1.3	Decoder Language Comparison	113
8.7.2	Multiplexer	114
8.7.3	Elementary Multiplexer	114
8.7.4	Many-input Multiplexer	115
8.7.4.1	Multiplexer (SystemVerilog)	115
8.7.4.2	Parameterized Bus Multiplexer (Chisel)	116
	gen vs dataType - Chisel	116
8.7.5	Demultiplexer	119
8.7.5.1	Demultiplexer (SystemVerilog)	120
8.7.5.2	Parameterized Demultiplexer (Chisel)	120
8.7.6	Seven-Segment Display	122
8.7.6.1	Seven Segment Display (SystemVerilog)	123
8.7.6.2	Seven Segment Display (Chisel)	124
8.7.6.3	Seven Segment Display Using MuxSelect	127
8.7.6.4	Seven Segment Display Language Comparison	130
8.8	Behavioral vs. Structural Descriptions	131
8.9	Adders	132
8.9.1	Behavioral Adders	132
8.9.1.1	Parameterized Abstract Adder (Chisel)	132
8.9.1.2	Precision in Addition	133
8.9.1.3	2-input 4-bit Behavioral Adder (SystemVerilog)	134
8.9.1.4	Parameterized Behavioral Adder (Chisel)	134
8.9.2	4-bit Behavioral Adder	134
8.9.3	Structural Adders	136
8.9.3.1	Half Adder	137
8.9.3.2	Increment	137
8.9.3.3	Full Adder (FA)	138
8.9.3.4	Ripple-Carry Adder	138
8.10	Multipliers	139
8.10.1	Behavioral Multiplier	139
8.10.1.1	Parameterized Abstract Multiplier (Chisel)	139
8.10.1.2	Parameterized Behavioral Multiplier (Chisel)	139
8.10.2	Precision in Multiplication	140
8.10.3	Two's Complement Multiplication and Intermediate Overflows	142
8.10.4	Handling Extended Results	142
8.11	Multi-Function Arithmetic Units	143
8.11.1	Subtraction	143
8.11.1.1	Behavioral Adder/Subtractor	143
8.11.1.2	Structural Subtractor	144
8.11.1.3	Multifunction 8/16/32/64-bit Signed/Unsigned Behavioral Adder/Subtractor (Chisel)	146
8.11.1.4	Multifunction Parameterized Signed/Unsigned Ripple-Carry Adder/Subtractor (Chisel)	153
8.12	<i>Arithmetic & Logic Unit</i>	155

8.13	Problems	161
8.13.1	Waveforms	161
8.13.2	Combinatorial Circuits	163
8.14	Exercises	166
9	Memory Circuits: One-Loop, First-Order Systems (1-OS)	169
9.1	Latch	170
9.1.1	Closing the first loop	170
9.1.2	Clocked latch	172
9.2	Serial extension: Master-Slave Principle	173
9.3	Serial-parallel extension: Register	175
9.3.1	Structure	175
9.3.2	Applications	176
9.3.2.1	Storing	176
9.3.2.2	Buffering	176
9.3.2.3	Synchronizing	176
9.3.2.4	Delaying	176
9.3.2.5	Looping	176
9.3.2.6	Pipelining	176
9.4	Parallel extension: Random-Access Memory	177
9.4.1	Generic structure	177
9.4.2	Synchronous RAM	180
9.4.3	Synchronous pipelined RAM	181
9.4.4	Register file	181
9.4.5	Representation of B_n in MSB and LSB Forms with Endianness	183
9.4.5.1	MSB Representation (Big-Endian Order)	183
9.4.5.2	LSB Representation (Little-Endian Order)	183
9.4.5.3	Commentary on Endianness	184
9.5	Problems	185
9.5.1	Registers	185
9.5.2	Memories	187
10	Automata: Two-Loop, Second-Order Systems (2-OS)	189
10.1	Definitions	190
10.1.1	Generic Definition	190
10.1.2	Size vs. complexity	191
10.1.3	Taxonomy	193
10.2	Finite (complex) Automata	195
10.2.1	Recognizing automata	195
10.2.2	Control automata	201
10.3	Simple Automata	202
10.3.1	Counters	202
10.3.1.1	T Flip-Flop	202
10.3.1.2	Generic Counter	203
10.3.2	Program Counter	205
10.3.3	Registers with Arithmetic & Logic Unit (RALU)	206

10.4 Problems	211
10.4.1 Finite Complex Automata	211
10.4.2 Simple Functional Automata	213
11 Processors: Three-Loop, Third-Order Systems (3-OS)	215
11.1 Counter-Expanded Automata	216
11.2 Processor	218
11.2.1 Architecture vs. Organization	218
11.2.2 Interpreting Processor (CISC processor)	219
11.2.3 Executing Processor (RISC processor)	220
11.3 <i>ToyRISC Processor</i>	221
11.3.1 Organization	221
11.3.1.1 PC: a control automaton	222
11.3.1.2 RALU: a functional automaton	222
11.3.1.3 DCD: interrupt automaton & decoder	222
11.3.2 Instruction Set Architecture	223
11.3.3 Assembly Code	225
11.3.3.1 Toy Assembler	226
11.3.3.2 Simulator	229
11.3.3.3 Assembly Programs	229
11.3.4 Time performance	232
11.4 How is Designed an Instruction Set Architecture	233
11.5 Problems	234
III COMPUTING	237
12 Computers: Four-Loop, Fourth-Order Systems (4-OS)	239
12.1 The mathematical model of the Turing Machine	240
12.1.1 The Decision Problem	240
12.1.2 Turing Machine	241
12.1.3 Theoretical Foundations of Number Representations in Turing's Work	241
12.1.3.1 Definition of Computable Numbers	242
12.1.4 Representation of Numbers Using Symbols and States	242
12.1.4.1 Finite State Automaton for Number Manipulation	242
12.1.4.2 Concept of Universal Representation	243
12.1.5 Summary of Key Theoretical Points in Turing's Paper	243
12.2 von Neumann Abstract Model	244
12.3 System Organization	244
12.4 Memory Management	246
12.4.1 Memory Gap	246
Locality principle	247
12.4.2 Memory Hierarchy	247
12.4.3 Virtual Memory Mechanism	248
12.4.4 Associative Memory-Based Page Translator	249
12.4.4.1 Content-Addressable Memory	249

12.4.4.2	Associative Memory	251
12.4.4.3	Translation Lookaside Buffer (TLB)	252
12.5	I/O	253
12.5.1	Bus	253
12.5.2	FIFO	254
12.5.2.1	Definition	254
12.5.2.2	Structure	256
12.5.3	DMA	257
12.5.3.1	Definition	257
12.5.3.2	Structure	258
12.5.4	I/O Devices	262
12.5.4.1	Hard Disk	262
12.5.4.2	Optical Disks	263
12.5.4.3	Flash Memory	263
12.5.4.4	Tapes	263
13	Computers: Five-Loop, Fifth-Order Systems (5-OS)	265
13.1	Harvard Computer Version	266
13.2	Cache Memories	266
14	Parallel Computation: N-Loop, N-Order systems (N-OS)	267
14.1	Cellular Automata (CA)	268
14.2	Controlled Cellular Automaton	270
14.2.1	Structure	270
14.2.2	Mapping	271
14.2.3	Architecture	272
14.2.3.1	Spatial selection	272
14.2.3.2	Arithmetic & Logic vector operations	273
14.2.3.3	Global shift/rotate operations	274
14.3	First Global Loop in CA	274
14.3.1	Structure	274
14.3.2	Architectural additions	276
14.4	Second Global Loop in CA	277
14.4.1	Structure	277
14.4.2	Architectural additions	279
14.5	The mathematical model of partially recursive functions	279
14.6	Heterogeneous Computing	281
14.6.1	Complexity vs. intensity in computation	281
14.6.2	Heterogeneous system	282
14.6.3	Programming a heterogeneous system	283
14.6.3.1	Accelerator-level programming	283
14.6.3.2	Library of primitives: an example	284
	Identity matrix	284
	Inner (dot or scalar) product	284
	Matrix-vector multiply	285
	Matrix multiplication	285

Matrix transpose	285
14.7 Parallel Computing Patterns	288
14.7.1 Patterns with Direct Correspondence	289
14.7.1.1 Map Pattern	289
14.7.1.2 Reduction Pattern	289
14.7.1.3 Scan Pattern	290
14.7.2 Patterns Executable on Map-Only	290
14.7.2.1 Stencil Pattern	290
14.7.2.2 Geometric Decomposition Pattern	290
14.7.2.3 Partition Pattern	291
14.7.2.4 Speculative Execution Pattern	291
14.7.2.5 Recurrence Pattern	291
14.7.2.6 Pipeline Pattern	292
14.7.3 Pattern Executable on MapReduce Only	293
14.7.3.1 Fork-Join Pattern	293
14.7.4 Patterns Executable on MapScanReduce	293
14.7.4.1 Category Reduction Pattern	293
14.7.4.2 Pack Pattern	294
14.7.4.3 Gather Pattern	294
14.7.4.4 Scatter Pattern	295
14.7.4.5 Split Pattern	295
14.7.4.6 Expand Pattern	295
14.7.5 Super-scalar Sequence Pattern	295
 IV PRACTICAL DESIGNS	 297
 15 Performance Optimized Digital Logic	 299
15.1 Comparison of <code>when</code> and <code>MuxCase</code> in Chisel	300
15.1.1 Behavioral and Structural Implications	300
15.1.2 Synthesis Implications	300
15.1.3 Specific Considerations	300
15.1.4 Switch Synthesis in Chisel	301
15.1.5 Comparison with <code>MuxCase</code> and <code>when</code>	301
15.1.5.1 Limitations of <code>switch</code>	301
15.2 Carry Lookahead Adder	301
15.3 Carry Select Adder	301
15.4 Carry Save Adder	302
15.5 Kogge-Stone Adder	302
15.6 <i>Multipliers</i>	302
 16 Instruction-Level Parallelism	 303
16.1 Pipelining	304
16.1.1 Pipeline Acceleration	304
16.1.2 <i>Pipelined Version of toyRISC</i>	305
16.1.2.1 Structure	305

16.1.2.2	Micro-architecture	306
16.1.2.3	Architecture	308
16.1.3	Latency	309
16.2	Hazards Generated by Dependencies	309
16.2.1	Data dependency	310
16.2.1.1	Stalling	312
16.2.1.2	Reordering	312
16.2.1.3	Forwarding	313
16.2.2	Control Dependency	315
16.2.2.1	Stalling	317
16.2.2.2	Reordering	318
16.2.2.3	Static Branch Prediction	319
16.2.2.4	Dynamic Branch Prediction	320
	Last-time, one-bit predictor	320
	Two-Bit Counter Based Predictor	321
16.3	Programming Pipelined Processors	321
16.4	Superscalar Processor	323
16.4.1	Register renaming	323
16.4.2	Out-of-Order Execution	324
16.4.2.1	Floating-point representation of real numbers	325
16.4.2.2	Tomasulo's Algorithm	326
16.5	Problems	327
16.5.1		327
16.5.2		327
17	ToyRISC Design	329
17.1	Code	329
17.1.1	First Level	329
17.1.2	Second Level	331
17.1.3	Third Level	333
17.1.4	Fourth Level	337
17.2	Assembly	340
17.3	Simulation	346
17.4	toyRISC Structural Implementation	349
17.4.1	Structure	349
17.4.2	Code generator	355
17.4.3	Simulator	364
17.4.4	Testing	365
17.5	Pipelined toyRISC	367
17.5.1	Structure	367
17.5.2	Code generator	376
17.5.3	Simulator	383
17.5.4	Testing	385
17.6	Forwarding toyRISC	386
17.6.1	Structure	386
17.6.2	Code generator	397

17.6.3	Simulator	405
17.6.4	Testing	406
18	RISC-V TBD	409
19	RISC-V Multithreaded TBD	411
20	Using the Heterogeneous System hRISC	413
20.1	Micro-architectures	417
20.1.1	Host micro-architecture	417
20.1.2	Accelerator micro-architecture	419
20.1.2.1	Controller micro-architecture	419
	Right operand select	421
20.1.2.2	Map micro-architecture	421
	Right operand select	424
20.1.3	Putting all together	424
20.2	Heterogeneous architecture	425
20.2.1	Host architecture	425
20.2.1.1	Instruction set	426
20.2.1.2	Low-level library of functions (FLA 0.1)	426
20.2.2	Accelerator's dual architecture	427
20.2.2.1	Controller's architecture	427
20.2.2.2	Map's architecture	428
20.3	Programming heterogeneous system	430
20.3.1	Direct programming using accelerator's assembly	431
20.3.2	FLA 0.1	435
20.3.3	Programming using FLA 0.1	439
APPENDIXES		444
A	Hardware Description Languages	447
A.1	Chisel	447
A.1.1	Introduction	447
A.1.2	Key Features of Chisel	447
	Hardware Construction Language.	447
	Parameterization.	447
	Strong Typing.	447
	Functional Programming.	447
A.1.3	Syntax	447
A.1.4	Scala Basics	447
	Immutable Variables (val).	447
	Mutable Variables (var).	448
	Functions (def).	448
	Classes and Objects.	448
A.1.5	Import Statements	448
A.1.6	Data Types	448

A.1.7	Base Types	448
	Bool.	448
	UInt.	448
	SInt.	448
	FixedPoint.	448
	Clock.	448
	Reset.	448
A.1.8	Aggregate Types	448
	Bundle.	448
	Vec.	448
A.1.9	Literals	448
	Unsigned Integer.	448
	Signed Integer.	448
	Boolean.	449
A.1.10	Operators	449
	Arithmetic (+, -, *, /, %).	449
	Logical (&, , ^, !).	449
	Comparison (==, !=, <, <=, >, >=).	449
	Bitwise Reduction (&, , ^).	449
	Shift (<<, >>).	449
A.1.11	Modules and Hierarchy	449
A.1.12	Defining Modules	449
A.1.13	Input/Output Ports	449
A.1.14	Instantiating Modules	449
A.1.15	Parameterized Modules	450
A.1.16	Wires and Registers	450
A.1.17	Control Structures	450
A.1.18	when Statements	450
A.1.19	switch Statements	451
A.1.20	Mux Function	451
A.1.21	Loops	451
A.1.22	Functions and Procedures	451
A.1.23	Defining Functions	451
A.1.24	Invoking Functions	451
A.1.25	Parameterized Types	452
A.1.26	Generic Modules	452
A.1.27	Hardware Construction in Chisel	452
A.1.28	Vectors (Vec)	452
A.1.29	Memories (Mem)	453
A.1.30	Bundles	453
A.1.31	Testing and Verification	453
A.1.32	ChiselTest Framework	453
A.1.33	Assertions	453
A.1.34	Peculiarities of Chisel	454
A.1.35	Chisel is Embedded in Scala	454
A.1.36	Delayed Execution Model	454

A.1.37	Assignment Operators	454
A.1.38	Type System	454
A.1.39	Combining Hardware and Software Constructs	454
A.1.40	Hardware vs. Software Mindset	454
A.1.41	Installing Chisel on Ubuntu	454
A.1.42	Install a Java Development Kit	454
A.1.42.1	Install the Default JDK	454
A.1.42.2	SDKMan	455
A.1.43	Install Scala Build Tool (sbt)	456
Clone this book		456
Build and Run Your Chisel Project		456
A.1.44	Install Verilator	456
A.1.45	Install gtkwave	456
A.1.46	Install Vivado	456
A.1.47	Install OpenLane	456
A.1.48	Install OSS CAD Suite	456
A.1.49	Install cirt (firtool)	456
A.1.50	Example Walkthrough	456
A.1.51	Key Directories	457
A.1.52	Run Chisel Test	457
A.1.53	Run the Testbench	458
A.1.54	Generate Clean System Verilog	460
A.1.55	Generate Annotated/Debug System Verilog	464
A.1.56	Verilator C++ Test Driver	467
A.1.57	Vivado Execution	469
A.1.58	Steps to Use Vivado	469
A.1.59	Key Features of Vivado	470
A.1.60	Comparison of Vivado and Verilator	470
A.2	System Verilog	473
A.2.1	Introduction	473
A.2.2	Overview of SystemVerilog	473
A.2.3	Key Enhancements over Verilog	473
A.2.4	Syntax	473
A.2.5	Data Types	473
A.2.5.1	Built-in Data Types	473
logic		473
bit		473
byte		474
shortint		474
int		474
longint		474
integer		474
time		474
A.2.5.2	User-Defined Types	474
typedef		474
enum		474

	struct	474
	union	474
A.2.6	Arrays	474
	A.2.6.1 Packed and Unpacked Arrays	474
	Packed Arrays	474
	Unpacked Arrays	474
	A.2.6.2 Dynamic and Associative Arrays	474
	dynamic array	474
	associative array	474
	queue	474
A.2.7	Operators	474
	Streaming Operators	474
	Type Casting	475
	Conditional Operator	475
A.2.8	Procedural Blocks and Control Flow	475
A.2.9	Enhanced Procedural Blocks	475
	always_comb	475
	always_ff	475
	always_latch	475
A.2.10	Control Flow Statements	475
	A.2.10.1 Looping Constructs	475
	foreach	475
	do...while	475
	break, continue	475
	return	475
	A.2.10.2 Conditional Compilation	475
	`\$ifdef, `\$ifndef, `\$elsif, `\$else, `\$endif	475
A.2.11	Interfaces and Modports	475
A.2.12	Tasks and Functions	476
A.2.13	Packages	476
A.2.14	Object-Oriented Programming Features	476
A.2.15	Classes	476
A.2.16	Randomization	476
A.2.17	Assertions and Coverage	477
A.2.18	Assertions	477
A.2.19	Coverage	477
A.2.20	Peculiarities of SystemVerilog	477
A.2.21	Multiple Procedural Blocks	477
A.2.22	Data Type Confusion	477
A.2.23	Implicit Net Declarations	477
A.2.24	Use of Interfaces	477
A.2.25	Enhanced For Loops	477
A.2.26	Randomization and Constraints	478
A.2.27	Mixing Verification and Design Constructs	478
A.3	Chisel Compared with System Verilog	478
	A.3.1 Introduction	478

A.3.2	Overview of SystemVerilog and Chisel	478
A.3.3	SystemVerilog	478
A.3.4	Chisel	478
A.3.5	Directly Mapped Constructs	478
A.3.6	Data Types	478
A.3.7	Modules and Hierarchy	478
A.3.8	Assignments	479
A.3.9	Control Structures	479
A.3.10	Loops	480
A.3.11	Memory and Arrays	480
A.3.12	Features Unique to Chisel	480
A.3.13	Parameterization and Generics	480
A.3.14	Object-Oriented Programming	480
A.3.15	Type Safety and Strong Typing	481
A.3.16	Hardware Generation	481
A.3.17	Testing with ChiselTest	481
A.3.18	Features Unique to SystemVerilog	481
A.3.19	Verification Constructs	481
A.3.20	Interfaces and Modports	481
A.3.21	Object-Oriented Verification	481
A.3.22	Time Control and Simulation	481
A.3.23	Explicit Event Control	481
A.3.24	Implementing Designs: A Cross-Language Guide	481
A.3.25	From SystemVerilog to Chisel	482
A.3.26	From Chisel to SystemVerilog	482
A.3.27	Common Pitfalls and Tips	482
B	Physical Implementations of Logic Gates	483
B.1	Physical Implementation of NOT Circuit	483
B.1.1	CMOS switches	483
B.1.2	The Inverter	485
B.1.2.1	The static behavior	485
B.1.2.2	Dynamic behavior	485
B.2	Physical Implementation of NAND & NOR Gates	488
B.2.1	The static behavior of gates	488
B.2.2	Propagation time	489
B.2.2.1	Propagation time for NAND gate	489
B.2.2.2	Propagation time for NOR gate	489
C	Simulations	491
C.1	Combo Simulations	491
C.1.1	ALU Simulation	491
C.2	Memory Simulations	492
C.2.1	Pipelined Three Number Adder	492
C.2.2	Read-During-Write Register File	493

D	FIFO memory	495
E	Complete Code Listing	499
E.1	SystemVerilog	500
E.1.1	Waveform Generator	500
E.1.2	Decoder	500
E.1.2.1	Parameterized Decoder	500
E.1.2.2	4-to-16 bit Behavioral Decoder	501
E.1.3	Multiplexer	501
E.1.3.1	16-to-1 Single Bit Multiplexer	501
E.1.4	Demultiplexer	502
E.1.4.1	1-to-16 bit Demultiplexer	502
E.1.5	Seven Segment Display	502
E.1.6	Adders	503
E.1.6.1	4-bit Behavioral Adder	503
E.2	Chisel	503
E.2.1	Waveform Generator	503
E.2.2	Decoder	504
E.2.2.1	Parameterized Decoder	504
E.2.2.2	2-to-4 Decoder	505
E.2.3	Multiplexer	505
E.2.3.1	Parameterized Bus Multiplexer	505
E.2.3.2	4-to-1 8-bit Bus Multiplexer	506
E.2.4	Demultiplexer	506
E.2.4.1	Parameterized Bus Demultiplexer	506
E.2.4.2	1-to-16 64-bit Bus Demultiplexer	507
E.2.5	Seven Segment Display	508
E.2.5.1	when Version	508
E.2.6	Mux Version	509
E.2.6.1	Flat Version	510
E.2.7	Adders	512
E.2.7.1	Parameterized Abstract Adder	512
E.2.7.2	Parameterized Behavioral Adder	512
E.2.7.3	4-bit Behavioral Adder	513
E.2.8	Multi-Function Arithmetic Units	513
E.2.8.1	Behavioral Subtractor	513
E.2.8.2	4-bit Behavioral Subtractor	514
E.2.8.3	Structural Subtractor	515
E.2.8.4	Multifunction Parameterized Ripple-Carry Adder/Subtractor	515
E.2.9	ALUs	517
	Bibliography	521
	Index	527

List of Figures

8.1	The two levels of the signal in the digital domain. Low level (0 Volt) for 0 or <code>false</code> , and high level (V_{DD} Volt) for 1 or <code>true</code>	96
8.2	Waveforms generated by Vivado for hand written SystemVerilog simulation	97
8.3	Waveforms for generated SystemVerilog Verilator simulation	102
8.4	Graphic representation of the NOT and BUFFER circuits.	103
8.5	Graphical representation of the most used two-input gates.	106
8.6	Wave forms for the non-inverting gates.	106
8.7	Decoder with n -bit input (DCDn).	108
8.8	Truth table for a 2-to-4 decoder. Each input combination activates one unique output in a one-hot encoding scheme.	109
8.9	Recursively defined 3-input decoder (DCD). a. Elementary decoder (eDCD). b. Two-input decoder (DCD2). c. Three-input DCD (DCD3).	109
8.10	Generated Schematic of a yosys synthesized Chisel 2-to-4 Decoder.	113
8.11	Elementary multiplexer (eMUX). a. The circuit structure: an eDCD serially connected with an AND-OR circuit. b. The logic symbol.	114
8.12	The general definition of the multiplexer circuit (MUXn). a. The structure of MUXn: a DCDn serially connected with a $m = 2^n$ ANDs on the first level of an AND-OR structure. b. The logic symbol for MUXn.	115
8.13	Generated Schematic of a yosys Elaborated Chisel 4-to-1 Multiplexer.	118
8.14	Generated Schematic of a yosys Elaborated and Flattened Chisel 4-to-1 Multiplexer.	119
8.15	Demultiplexer as a DCD whose outputs are ANDed with the one bit signal to be distributed according to the n -bit input code.	120
8.16	The version of digital circuits. a. Logic circuit: trans-coder for seven-segment display. b. Numeric circuit: four-bit numbers adder.	123
8.17	Generated Schematic of SevenSegmentDisplay Using <code>when</code> statements.	127
8.18	Generated Schematic of SevenSegmentDisplay Using <code>MuxCase</code>	128
8.19	Generated Schematic of SevenSegmentDisplay Using <code>MuxCase</code> and Skywater 130nm ASIC Technology files.	129
8.20	Generated Schematic of SevenSegmentDisplay Using <code>MuxCase</code> Showing Exact Standard Cell Skywater 130nm ASIC Cells.	130
8.21	Hierarchical Decomposition Adder Modules. a. Top-level 2-input 4-bit adder. b. The structure of a 3-input 4-bit adder.	131
8.22	4-bit Behavioral Adder Partial Truth Table	133
8.23	High-level Schematic of a Behavioral 4-bit Adder.	135
8.24	Yosys Synthesized and Flattend Schematic of a Behavioral 4-bit Adder.	136

8.25	Half adder (HA). The XOR circuit compute the <i>modulo-2</i> sum while the AND gate compute the carry value.	137
8.26	Increment circuit for 4-bit numbers (INC4) implemented by serially connected 4 HAs.	137
8.27	Adder. a. Full Adder (FA). b. 4-bit adder (ADD4).	138
8.28	4-bit Behavioral Multiplier Partial Truth Table for Unsigned Numbers	140
8.29	4-bit Behavioral Multiplier Partial Truth Table for Signed Numbers	141
8.30	Adder/Subtractor	145
8.31	Yosys ASIC Synthesized and Flattened Schematic of a Two's Complement 4-bit Adder/-Subtractor.	146
8.32	ALU.	155
8.33	The generic form of a 8-function Arithmetic & Logic Unit for n -bit words (ALUn).	155
8.34	A simple Arithmetic & Logic Unit for n -bit words (ALUn).	157
8.35	Output for waveform1.sv	162
8.36		163
8.37		164
8.38		165
8.39		165
9.1	The elementary latches. Using the loop, closed from the output to one input, elementary storage elements are built. a. <i>AND loop</i> provides a <i>reset-only</i> latch. b. <i>OR loop</i> provides the <i>set-only</i> version of a storage element. c. The heterogeneous elementary <i>set-reset</i> latch results combining the <i>reset-only</i> latch with the <i>set-only</i> latch. d. The wave forms describing the behavior of the previous three latch circuits.	171
9.2	Symmetric elementary latches. a. Symmetric elementary <i>NAND latch</i> with low-active commands S' and R' . b. Symmetric elementary <i>NOR latch</i> with high-active commands S and R	172
9.3	Elementary clocked latch. The transparent RS clocked latch is sensitive (transparent) to the input signals during the active level of the clock (the high level in this example). a. The internal structure. b. The logic symbol.	172
9.4	The data latch. Imposing the restriction $R = S'$ to an RS latch results the D latch without non-predictable transitions ($R = S = 1$ is not anymore possible). a. The structure. b. The logic symbol. c. An improved version for the data latch internal structure.	173
9.5	The master-slave principle. Serially connecting two RS latches, activated with different levels of the clock signal, results a non-transparent storage element. a. The structure of a RS master-slave flip-flop, active on the falling edge of the clock signal. b. The logic symbol of the RS flip-flop triggered by the <i>negative edge</i> of clock. c. The logic symbol of the RS flip-flop triggered by the <i>positive edge</i> of clock.	174
9.6	The delay (D) flip-flop. Restricting the two inputs of an RS flip-flop to $D = S = R'$, results an FF with predictable transitions. a. The structure. b. The logic symbol. c. The wave forms proving the delay effect of the D flip-flop.	174
9.7	The n-bit register. a. The structure: a bunch of DF-F connected in parallel. b. The logic symbol.	175
9.8	Register operation. At each active edge of clock (in this example it is the positive edge) the register's output takes the value applied on its inputs if <code>reset = 0</code> and <code>enable = 1</code>	175

9.9	The principle of the random access memory (RAM). The clock is distributed by a DMUX to one of $m = 2^p$ DLs, and the data is selected by a MUX from one of the m DLs. Both, DMUX and MUX use as selection code a p -bit address. The one-bit data DIN can be stored in the clocked DL.	178
9.10	Read and write cycles for an asynchronous RAM. Reading is a combinational process of selecting. The access time, t_{ACC} , is given by the propagation through a big MUX. The write enable signal must be strictly included in the time interval when the address is stable (see t_{ASU} and t_{AH}). Data must be stable related to the positive transition of WE' (see t_{DSU} and t_{DH}).	178
9.11	The asynchronous m-bit word RAM. Expanding the number of bits per word means to connect in parallel one-bit word memories which share the same decoder. Each COLUMN contains storing latches and AND-OR circuits for one bit.	179
9.12	Read and write cycles for Synchronous RAM (SRAM). For the <i>flow-through</i> version of a SRAM the time behavior is similar to a register. The set-up and hold time are defined related to the active edge of clock for all the input connections: <i>data</i> , <i>write-enable</i> , and <i>address</i> . The data output is also related to the same edge.	180
9.13	Register file. In this example it contains 2^n m -bit registers. In each clock cycle any two registers can be read and writing can be performed in anyone.	182
10.1	Automata types. a. The structure of the half-automaton ($A_{1/2}$), the no-output function automaton: the state is generated by the previous state and the previous input. b. The logic symbol of half-automaton. c. Immediate Mealy automaton: the output is generated by the current state and the current input. d. Immediate Moore automaton: the output is generated by the current state. e. Delayed Mealy automaton: the output is generated by the previous state and the previous input. f. Delayed Moore automaton: the output is generated by the previous state.	194
10.2	Transition diagram. The transition diagram for the half-automaton which recognizes strings of form $1^n 0^m$, for $n \geq 1$ and $m \geq 1$. Each circle represent a state, each (marked) arrow represent a (conditioned) transition.	196
10.3	A 4-state finite automaton. The structure of the finite automaton used to recognize binary string belonging to the $1^n 0^m$ set of strings.	197
10.4	Control Automaton. The functional definition of control automaton. Control means to issue commands and to receive back signals (flags) characterizing the effect of the command.	201
10.5	The simplest Controller. The Moore version of a control automaton is optimized by: (1) using a multiplexor to select, using MOD and T combined by <i>clc</i> , the next state from different sources, (2) using an increment circuit (INC) to compute the most frequent next address for CLC, and (3) using a multiplexor to select, using TEST, for each cycle the appropriate flag.	202
10.6	The T flip-flop. a. It is the simplest automaton because: has 1-bit state register (a DF-F), a 2-input loop circuit (one as automaton input and another to close the loop), and direct output from the state register. b. The structure of the T flip-flop: the XOR_2 circuits complements the state if $T = 1$. c. The logic symbol.	203
10.7	Counter.	203
10.8	Program Counter.	205
10.9	32-bit RALU.	207
10.10		213
11.1	Counter-expanded automaton.	216

11.2	The graph describing the behavior of the finite automaton used to build the counter-expanded automaton for the $(a^n b^n n > 0)$ language.	217
11.3	Generic Processor	218
11.4	Interpreting Processor	219
11.5	Executing Processor	220
11.6	The organization of the <i>toyRISC</i> processor in its simulation environment where the program memory and data memory are included.	221
12.1	Turing Machine. a. Original description: Tape & Head & FA. b. Digital representation: Memory & upDownCounter & FA.	241
12.2	The abstract model of computer proposed by John von Neumann.	244
12.3	The three-partite functionality of a computing system.	245
12.4	The organization of a computing system.	245
12.5	Processor-Memory Performance Gap (Patterson et al., 1997).	246
12.6	Memory hierarchy from fast and small register files to slow and large auxiliary storage memories such as HDD (Hard Disk Drive), SSD (Solid-State Drive), optical disk, tape.	247
12.7	Virtual memory mechanism. a. Page correspondence. b. Translation mechanism.	248
12.8	Content Addressable Cell. a. The structure: data latches whose content is compared against the input data using 4 XORs and one NAND. Write is performed applying the clock with stable data input. b. The logic symbol.	249
12.9	The Content Addressable Memory (CAM). a. A 4-word CAM is built using 4 content addressable cells, a demultiplexor to distribute the write enable (WE) signal, and a $NAND_4$ to generate the match signal (M). b. The logic symbol.	250
12.10	An associative memory (AM). The structure of an AM can be seen as a RAM with a programmable decoder implemented with a CAM. The decoder is programmed loading CAM with the considered addresses.	252
12.11	The waveforms for a read cycle followed by a write cycle.	254
12.12	a. Block schematic of FIFO. b. write A operation. c. write D. d. write F. e. read.	255
12.13	FIFO memory. Two pointers, evolving in the same direction, and a two-port RAM implement a LIFO (queue) memory. The limit flags are computed combinational from the addresses used to write and to read the memory.	256
12.14	Initiating the transfer between I/O and Memory.	259
12.15	Block schematic for a minimal DMA.	259
12.16	Hard Disk Drive Structure [Source: Google-Images]	262
13.1	The Harvard abstract model of computer.	266
14.1	An example of cellular structure as a Nth order system, N-OS. In this example, each cell is a second order system, 2-OS.	268
14.2	One-dimensional cellular automaton.	268
14.3	Controlled Cellular Automaton (CCA).	270
14.4	Distribution network for a circuit with $p = 16$	271
14.5	The <i>MapReduce</i> structure obtained by closing the first global loop over a controlled cellular automaton.	275
14.6	Reduce network for $p = 16$, with the size in $O(p)$ and the depth in $O(\log p)$ exemplified for the add function.	275

14.7	MapScanReduce structure obtained by closing the second global loop over a controlled cellular automaton.	277
14.8	An 8-input scan circuit adapted for an 8-input prefix operation Antonescu and Stefan (2020) Antonescu and Stefan (2024).	278
14.9	The circuit version of composition. It is a two-layer construct: the parallel expanded <i>map</i> layer serially connected with the <i>log</i> -dept <i>reduction</i> layer.	280
14.10	The circuit version of the enhanced composition. In the map composition right connections are added considering specific map compositions. The scan circuit, a <i>log</i> -layer map compositions, is connected with the generic map composition.	281
14.11		282
14.12	Parallel patterns McCool et al. (2012).	288
14.13	Recurrence pattern implementation McCool et al. (2012).	292
16.1	Pipelining. a. The initial circuit, without pipeline register. b. The pipelined version of the circuit.	304
16.2	The pipelined version of toyRISC.	306
16.3	The change used to reduce hazards imposed by data dependencies.	314
16.4	The saturated one-bit up/down counter as last-time branch predictor.	320
16.5	Two-bit saturated counter based predictor.	321
17.1	01_theSystem	330
17.2	02_toyRISC	331
17.3	02_memories	332
17.4		333
17.5		334
17.6		335
17.7		335
17.8		336
17.9		337
17.10		338
17.11		339
17.12		339
17.13		348
17.14		355
20.1	Heterogeneous System (1_hetSys.sv): HOST computer and many-cell ACCELERATOR interconnected through INTERFACES (three FIFOs: one for commands and programming and two for in and out of data).	414
20.2	Accelerator (2_accelerator.sv): the many-cell engine ARRAY and the sequencer CONTROLLER (a mono-core computer).	416
20.3	Array (3_array.sv): the array of p cells MAP loop connected with two <i>lor</i> -depth networks (REDUCE for reduce functions amd SCAN for scan functions).	417
A.1	Waveforms generated for a Verilator SystemVerilog simulation	469
B.1	Basic switches. a. Open switch connected to V_{DD} . b. Closed switch connected to V_{DD} . c. Open switch connected to <i>ground</i> . d. Closed switch connected to <i>ground</i>	483

B.2	The MOSFET switch. The <i>switch-resistor-capacitor</i> model consists in the two states: OF ($V_{GS} < V_T$), and ON ($V_{GS} \geq V_T$). In both states the input is defined by the capacitor C_{GS} .	484
B.3	Building an inverter. a. The inverter circuit. b. The logic symbol for the inverter circuit.	485
B.4	The propagation time.	486
B.5	The output characteristic of the nMOS transistor.	487
B.6	The NAND gate. a. The internal structure of a NAND gate: the output is 1 when at least one input is 0. b. The logic symbol for NAND.	488
B.7	The NOR gate. a. The internal structure of a NOR gate: the output is 1 only when both inputs are 0. b. The logic symbol for NOR.	489
C.1		492
C.2		493
C.3		494
D.1	FIFO memory. Two pointers, evolving in the same direction, and a two-port RAM implement a LIFO (queue) memory. The limit flags are computed combinational from the addresses used to write and to read the memory.	496

List of Tables

1	Orders of Computing Systems	v
3.1	Binary Representations for Different Bit Lengths	34
4.1	Comparison of signed number representations with fixed column widths and wrapped text.	58
5.1	IEEE 754 Rounding Modes	68
5.2	Denormalized Spacing in IEEE 754 Floating-Point Formats	71
5.3	Summary of IEEE 754 Supported Floating-Point Precisions	72
6.1	Comparison of Character Encoding Schemes, including Base64 and MIME.	84
8.1	One-input circuits.	103
8.2	Two-input Combinational Gates Truth Table.	104
8.3	Summary of Logical Gates and their Interpretations	105
8.4	Truth table proof of $A + A'B = A + B$.	108
8.5	Seven-segment display truth table.	123
10.1	Transition state table.	196
10.2	Sequence of commands applied to RALU in Example 10.6	209
10.3	Sequence of commands applied to RALU described in Section 10.3.3	214

11.1 Architecture vs. Organization.	218
11.2 Assembly language mnemonics, their meaning and binary form.	225
14.1 Accelerator-level operations	284
14.2 Library of primitives	288
16.1 Pipelined execution.	309
16.2 Data dependency in the pipelined execution	310
16.3 Stalling	312
16.4 Hazard generated by the control dependency	316
20.1 Host's Instruction Set Architecture.	426
20.2 Host's Function Library Architecture (FLA) defined in file: 00_theKernel.sv.	427
20.3 Instructions with operands from the controller's Instruction Set Architecture.	427
20.4 Sequencer's Instruction Set Architecture with no operand.	428
20.5 Instructions with operands from the Instruction Set Architecture.	429
20.6 Map's Instruction Set Architecture with no operand.	430
20.7 Matrix-vector multiplication evaluation depending on the size of the square matrix.	441
20.8 Multiplication evaluation depending on the size of the square matrices.	442
A.1 Comparison of Vivado and Verilator	471
A.2 Directly Mapped Data Types	479

Listings

8.1	Waveform Generator Code (Hand Written SystemVerilog)	96
8.2	Waveform Generator Code (Chisel: WaveFormGenerator.scala)	97
8.3	Waveform Generator Code with Emit SystemVerilog (Chisel)	98
8.4	fir.mlir file generation from Chisel	98
8.5	Waveform Generator SystemVerilog generated by Chisel	99
8.6	Parameterized Decoder (SystemVerilog: dcdParameterized.sv)	109
8.7	4-to-16 Behavioral Decoder (SystemVerilog:dcd.sv)	110
8.8	Parameterized Decoder (Chisel: Decoder.scala)	110
8.9	Instantiated 2-to-4 Decoder (Chisel: Decoder2to4.scala)	111
8.10	SystemVerilog Generated for Chisel firtool Decoder(4)	111
8.11	Verilog Generated by sv2v for Decoder(4)	112
8.12	Verilog Elaborated by yosys for Decoder(4)	112
8.13	16-to-1 single bit Multiplexer (SystemVerilog: mux.sv)	115
8.14	Parameterized Bus Type Multiplexer (Chisel: MultiplexerGenericType.scala)	116
8.15	Parameterized Bus Width Multiplexer (Chisel: Multiplexer.scala)	116
8.16	Parameterized Bus Width Multiplexer Companion Object (Chisel: Multiplexer.scala)	117
8.17	4-to-1 8-bit Bus Multiplexer (Chisel: Mux4.scala)	117
8.18	1-to-16 bit Demultiplexer (SystemVerilog: dMux.sv)	120
8.19	Parameterized Bus Demultiplexer (Chisel: Demultiplexer.scala)	120
8.20	1-to-16 64-bit Bus Demultiplexer (Chisel: DeMux16.scala)	121
8.21	Seven Segment Display (SystemVerilog: SevenSegmentDisplay.sv)	124
8.22	Seven Segment Display (Chisel: SevenSegmentDisplay.scala)	124
8.23	Propagation of suggestName() after yosys Elaboration (Generated SystemVerilog: yosys/SevenSegmentDisplay.v)	126
8.24	Seven Segment Display Using MuxCase (Chisel: SevenSegmentDisplayMuxCase.scala)	127
8.25	Parameterized Abstract Adder (Chisel: Adder.scala)	132
8.26	Behavioral 2-input 4-bit Adder (SystemVerilog: adder.sv)	134
8.27	Parameterized Behavioral Adder (Chisel)	134
8.28	4-bit Behavioral Adder (Chisel: BehavioralAdder4.scala)	135
8.29	Parameterized Abstract Multiplier (Chisel: Multiplier.scala)	139
8.30	Parameterized Behavioral Multiplier (Chisel: BehavioralMultiplier.scala)	140
8.31	Behavioral Adder/Subtractor (Chisel: BehavioralAdderSubtractor.scala)	143
8.32	4-bit Behavioral Adder/Subtractor (Chisel: BehavioralAdderSubtractor4.scala)	144
8.33	Parameterized Two's Complement Adder/Subtractor (Chisel: BehavioralAdderSubtrac- torHS.scala)	144
8.34	Abstract Multifunction Adder/Subtractor (Chisel: MultifunctionAdderSubtractor.scala)	146

8.35 Multifunction 64-bit Adder/Subtractor (Chisel)	148
8.36 Parameterized Ripple-Carry Adder (Chisel)	153
8.37 Arithmetic and Logic Unit. File: A08_alu.sv (SystemVerilog)	155
8.38 64-bit ALU (Chisel: ALU64.scala)	157
8.39 Waveforms. File: A08_waveform1.sv (SystemVerilog)	162
8.40 Half adder. File: A08_halfAdder.sv (SystemVerilog)	163
8.41 Full adder. File: A08_fullAdder.sv (SystemVerilog)	164
8.42 Hierarchical Full-Adder. File: A08_hFullAdder.sv (SystemVerilog)	164
8.43 Hierarchical Full-Adder. File: A08_testFullAdder.sv (SystemVerilog)	165
9.1 The module 'threeAdder' has 3 4-bit inputs and one 4-bit output. The circuit adds modulo 16 three numbers; do not provide carry output. File: A09_pipelinedThreeAdder.sv (SystemVerilog)	176
9.2 4096 32-bit words synchronous RAM; asynchronous read. File: A09_syncRAM.sv (SystemVerilog)	181
9.3 4096 32-bit words synchronous RAM; asynchronous read. File: A09_syncPipeRAM.sv (SystemVerilog)	181
9.4 Register file. File: A09_regFile.sv (SystemVerilog)	182
9.5 Circuit that compute the mean value from the last component of a received stream of numbers. File: A09_mean.sv (SystemVerilog)	186
9.6 Tester for circuit mean. File: A09_testMean.sv (SystemVerilog)	186
9.7 Define the parameterized synchronous RAM. File: A09_defineParamSyncPipeRAM.vh (SystemVerilog)	188
9.8 Parameterized synchronous RAM. File: A09_paramSyncPipeRAM.sv (SystemVerilog)	188
10.1 Example of a complex circuit. File: A10_randomCircuit.sv (SystemVerilog)	192
10.2 Example of a simple circuit. File: A10_orPrefixes.sv (SystemVerilog)	192
10.3 Defines binary codes for input, output and state variables. File: A10_defines.vh (SystemVerilog)	198
10.4 Structural description of the immediate Mealy automaton designed to recognize the regular language $L = (a^n b^m n, m > 0)$. File: A10_regLang.sv (SystemVerilog)	198
10.5 Combinational circuit used to compute the loop for the immediate Mealy automaton used to recognize the language $L = (a^n b^m n, m > 0)$. File: A10_loopCLC.sv (SystemVerilog)	199
10.6 Combinational circuit used to compute the output for immediate Mealy automaton which recognizes the language $L = (a^n b^m n, m > 0)$. File: A10_outCLC.sv (SystemVerilog)	199
10.7 Test module for the immediate Mealy automaton which recognizes the language $L = (a^n b^m n, m > 0)$. File: A10_testRegLang.sv (SystemVerilog)	200
10.8 Defines the command for the counter circuit. File: A10_counterDefines.vh (SystemVerilog)	204
10.9 A presetable up-down counter. File: A10_counter.sv (SystemVerilog)	204
10.10 Test module for the counter. File: A10_testCounter.sv (SystemVerilog)	204
10.11 Program counter. File: A10_programCounter.sv (SystemVerilog)	206
10.12 Defining the functions for a RALU. File: A10_defineRALU.vh (SystemVerilog)	207
10.13 RALU. File: A10_RALU.sv (SystemVerilog)	208
10.14 ALU. File: A10_alu.sv (SystemVerilog)	208
10.15 Testing RALU. File: A10_testRALU.sv (SystemVerilog)	209
10.16 Binary codes for input, output and state variables. File: A10_defines1.vh (SystemVerilog)	211
10.17 Binary codes for input, output and state variables. File: A10_defines2.vh (SystemVerilog)	212
10.18 Binary codes for input, output and state variables. File: A10_defines3.vh (SystemVerilog)	212

11.1 Counter expanded automaton with a 32-bit counter for recognizing the language $a^n b^n$. File: 0_CEA.vh (SystemVerilog)	217
11.2 toyRISC ISA. File: 0_DEFINES.vh (SystemVerilog)	224
11.3 A sample of the binary code generator for toyRISC processor. File: RISCcodeGenerator.sv (SystemVerilog)	226
11.4 toyRISC program: initializes the first 5 registers and adds them in register 0. File: A11_RISCprogram1.sv (SystemVerilog)	229
11.5 toyRISC program: tests the interrupt action. File: A11_RISCprogram2.sv (SystemVerilog)	230
11.6 toyRISC program: an unending running program. File: A11_RISCprogram3.sv (SystemVerilog)	231
11.7 toyRISC program: tests the access to the data memory. File: A11_RISCprogram4.sv (SystemVerilog)	232
11.8 toyRISC program used to validate the simulator. File: A11_RISCprogram5.sv (SystemVerilog)	234
20.1 Heterogeneous system. File: 1_hetSys.sv. (SystemVerilog)	415
20.2 Host micro-architecture. (SystemVerilog)	418
20.3 Controller micro-architecture. (SystemVerilog)	420
20.4 Map micro-architecture. (SystemVerilog)	422
20.5 Micro-architectures. File: 0_DEFINES.vh. (SystemVerilog)	424
20.6 The framework in which the host program must be included. File: 0_hProgram.sv (SystemVerilog)	431
20.7 The framework in which the accelerator program must be included. File: 0_aProgram.sv (SystemVerilog)	431
20.8 Program which adds in acc the indexes from a 16-cell Map. File: 0_aProgram1.sv (SystemVerilog)	431
20.9 Program which adds in acc the indexes from a p -cell Map. File: 0_aProgram2.sv (SystemVerilog)	432
20.10 Program executed by controller which adds in acc p numbers. File: 0_aProgram3.sv (SystemVerilog)	433
20.11 Program illustrates the use of the spatial selection mechanism. File: 0_aProgram4.sv (SystemVerilog)	434
20.12 Program generates in acc the biggest value generated pseudo-randomly in ACC. File: 0_aProgram5.sv (SystemVerilog)	435
20.13 FLA 0.1. File: 00_theKernel.sv (SystemVerilog)	435
20.14 The program on host for matrix-vector multiply. File: 0_hProgram3.sv (SystemVerilog) .	439
20.15 The program on accelerator which includes the kernel library FLA 0.1 used by host. File: 0_aProgram6.sv (SystemVerilog)	440
20.16 The program on host for multiplying matrices. File: 0_hProgram1.sv (SystemVerilog) .	441
20.17 The program on host for multiply-add matrices. File: 0_hProgram2.sv (SystemVerilog) .	443
A.1 Waveform Generator Code (Chisel)	457
A.2 Waveform Generator Testbench Code (Chisel)	458
A.3 Waveform Generator Testbench Code (Chisel)	458
A.4 Clean SystemVerilog Code Produced by Chisel	461
A.5 Annotated SystemVerilog Code Produced by Chisel	464
A.6 Verilator Testbench for WaveFormGenerator	467
RTL/Verilog/src/WaveFormGenerator.sv	470

A.7	SystemVerilog Testbench for waveFormGenerator	472
E.1	Waveform Generator Code (Hand Written SystemVerilog)	500
E.2	Parameterized Decoder (SystemVerilog: dcdParameterized.sv)	500
E.3	4-to-16 Behavioral Decoder (SystemVerilog:dcd.sv)	501
E.4	16-to-1 single bit Multiplexer (SystemVerilog: mux.sv)	501
E.5	1-to-16 bit Demultiplexer (SystemVerilog: dMux.sv)	502
E.6	Seven Segment Display (SystemVerilog: SevenSegmentDisplay.sv)	502
E.7	Behavioral 2-input 4-bit Adder (SystemVerilog: adder.sv)	503
E.8	Waveform Generator Code (Chisel: WaveFormGenerator.scala)	503
E.9	Parameterized Decoder (Chisel: Decoder.scala)	504
E.10	Instantiated 2-to-4 Decoder (Chisel: Decoder2to4.scala)	505
E.11	Parameterized Bus Width Multiplexer (Chisel: Multiplexer.scala)	505
E.12	4-to-1 8-bit Bus Multiplexer (Chisel: Mux4.scala)	506
E.13	Parameterized Bus Demultiplexer (Chisel: Demultiplexer.scala)	506
E.14	1-to-16 64-bit Bus Demultiplexer (Chisel: DeMux16.scala)	507
E.15	Seven Segment Display using Bus and when (Chisel: SevenSegmentDisplay.scala)	508
E.16	Seven Segment Display using Bus and Mux (Chisel: SevenSegmentDisplay.scala)	509
E.17	Seven Segment Display using Named Inputs (Chisel: SevenSegmentDisplayFlat.scala)	510
E.18	Parameterized Abstract Adder (Chisel: Adder.scala)	512
E.19	Parameterized Behavioral Adder (Chisel)	512
E.20	4-bit Behavioral Adder (Chisel: BehavioralAdder4.scala)	513
E.21	Behavioral Adder/Subtractor (Chisel: BehavioralAdderSubtractor.scala)	513
E.22	4-bit Behavioral Adder/Subtractor (Chisel: BehavioralAdderSubtractor4.scala)	514
E.23	Parameterized Two's Complement Adder/Subtractor (Chisel: BehavioralAdderSubtractorHS.scala)	515
E.24	Parameterized Ripple-Carry Adder (Chisel)	516
E.25	64-bit ALU (Chisel: ALU64.scala)	517

Part I

NUMBER SYSTEMS AND REPRESENTATIONS

Chapter 1

Number Systems

1.1	A Brief History of Number Systems	4
1.1.1	Early Development	4
1.1.2	Evolution of Mathematical Sets	4
1.1.3	Pre-Computer Number Representations	5
1.1.4	Number Representations in Computing	5
1.1.5	Modern Trends and Challenges	6
1.2	Decimal Arithmetic	6
1.2.1	Decimal Addition	6
1.2.2	Decimal Multiplication	7
1.2.3	Decimal Subtraction	9
1.2.4	Decimal Division	10
1.2.5	Alternative Decimal Arithmetic Algorithms	11
1.2.6	10's Complement	12
1.3	Positional Notation	13
1.3.1	Positional Notation in Base 10	13
1.3.2	Example of Positional Notation in Base 10	13
1.3.3	Importance of Positional Notation	13
1.3.4	Positional Notation for Arithmetic Operations	14
1.3.5	Base/Radix Concepts	14
1.3.6	Generalized Positional Notation	14
1.3.7	Positional Notation for Different Bases	14
1.4	Binary Arithmetic	15
1.4.1	Binary Addition	15
1.4.2	Binary Subtraction	16
1.4.3	Binary Multiplication - TBD	17
1.4.4	Binary Division - TBD	17

1.1 A Brief History of Number Systems

1.1.1 Early Development

Early humans used tally marks to record quantities, such as notches on bones or stones, dating back to approximately 30,000 BCE (Ifrah, 2000a). More structured systems emerged with the Babylonians, who developed a base-60 system around 2000 BCE, and the Egyptians, who used hieroglyphic numerals around 3000 BCE (Ore, 1990). The base-12 system, or duodecimal system, emerged due to its practicality in trade, measurement, and division of goods. Its divisibility by 2, 3, 4, and 6 made it ideal for splitting quantities evenly (Wells, 2010). These systems laid the foundation for positional notation and arithmetic, with remnants of base-60 and base-12 still evident today in the division of time into 60 minutes and 60 seconds, the 12 months of a year, the 12 inches in a foot, and the grouping of items into dozens in commerce.

The Hindu-Arabic numeral system, developed between 500–1200 CE, introduced the concept of zero and a fully positional decimal system. This innovation transformed arithmetic and facilitated the development of more advanced mathematical concepts, forming the basis of the modern number system (Kline, 1990).

The binary number system, foundational to digital computing, was first described by the Indian mathematician Pingala in the 2nd century BCE. Later, in the 17th century, Gottfried Wilhelm Leibniz formalized binary arithmetic, recognizing its potential for simplifying calculations (Kline, 1990). This system's alignment with the on-off states of electrical circuits made it the natural choice for modern computers.

1.1.2 Evolution of Mathematical Sets

The evolution of mathematical sets reflects humanity's growing understanding of quantity, ratios, and continuity.

Integers ($\mathbb{Z}$). The Babylonians and Egyptians used whole numbers for trade and astronomy. However, the concept of negative numbers was not widely accepted until Indian mathematician Brahmagupta introduced them in the 7th century CE to represent debts (Burton, 2010a).

Rational Numbers ($\mathbb{Q}$). Rational numbers were extensively studied by the Greeks, particularly in their exploration of ratios in geometry. The discovery of irrational numbers, such as $\sqrt{2}$, challenged their worldview, leading to the distinction between rational and irrational quantities (Maor, 2007).

Real Numbers ($\mathbb{R}$). Real numbers evolved from the need to describe continuous quantities. Euclid's work on incommensurable lengths hinted at irrational numbers, but rigorous definitions came in the 19th century through the works of Dedekind (via Dedekind cuts) and Cantor (via set theory) (Rosen, 2018).

Complex Numbers ($\mathbb{C}$). Complex numbers arose in the 16th century as mathematicians like Cardano and Bombelli sought solutions to polynomial equations, particularly cubic and quartic equations (Boyer and Merzbach, 1991). Initially dismissed as "imaginary," they were formalized in the 18th and 19th centuries by mathematicians such as Euler, Argand, and Gauss. Euler introduced the formula $e^{i\theta} = \cos \theta + i \sin \theta$, Argand developed the geometric interpretation of complex numbers on the complex plane, and Gauss expanded their application to number theory and algebra (Kline, 1990; Stewart

and Tall, 2015). Today, complex numbers are essential in fields like quantum mechanics, engineering, and signal processing (Maor, 2007).

1.1.3 Pre-Computer Number Representations

The representation of numbers has evolved alongside advances in mathematics, science, and technology. While early systems like the Babylonian base-60 and Hindu-Arabic decimal notation provided the foundation for positional number systems, more specialized representations emerged to address the computational challenges of different eras. From manual calculations to the computer age, each system reflects the requirements and constraints of its time.

Before the advent of digital computers, numerical representations were primarily designed for manual or mechanical computation. These early methods focused on simplifying arithmetic while addressing the limitations of the tools available.

Decimal Positional Systems. The Hindu-Arabic numeral system, which became dominant during the medieval period, introduced positional notation and the concept of zero. This innovation enabled efficient manual computation and set the stage for future representations in mechanical and digital systems (Ifrah, 2000b; Burton, 2010b).

10's Complement for Subtraction. The concept of complements, such as 10's complement, predates the computer era and was used in mechanical calculators like the 19th-century Arithmometer. By representing negative numbers as the difference between a base (e.g., 10^n) and a positive value, subtraction could be performed as addition with a small adjustment. This method inspired later binary complement systems used in digital computers (Randell, 1982).

Logarithmic Representations. In the early 17th century, John Napier introduced logarithms, enabling the multiplication of large numbers by adding their logarithms. Logarithmic tables and slide rules became essential tools for computation, representing a precursor to modern computational methods (Stewart, 2008).

1.1.4 Number Representations in Computing

The transition from mechanical to digital systems necessitated new numerical representations tailored to the constraints and capabilities of electronic computers.

Binary and Two's Complement. With the advent of binary systems, representing numbers using only two states (0 and 1) became standard in computing. Negative numbers posed a challenge, initially addressed with signed magnitude representation. However, this method proved inefficient for arithmetic operations, leading to the adoption of two's complement (Kline, 1990). In two's complement, the negative of a number is represented as its binary complement plus one, simplifying subtraction and enabling efficient hardware implementation.

Excess- k Representations. First used in analog computing systems, excess- k representations became a cornerstone of digital computing with the IEEE 754 floating-point standard. By biasing the representation of numbers, excess- k allows for simplified comparisons and normalization, making it ideal for scientific computations (Goldberg, 1991).

Fixed-Point Arithmetic. Fixed-point representation emerged as a practical solution for embedded systems and early computers with limited hardware capabilities. By dedicating a fixed number of bits to the fractional and integer parts of a number, fixed-point arithmetic provided a simpler alternative to floating-point for real-time applications (Smith and Doe, 1997).

Floating-Point Systems. The need for a dynamic range in numerical representation led to the development of floating-point systems. Introduced formally in the mid-20th century and standardized with IEEE 754 in 1985, floating-point representation encodes numbers as a sign, exponent, and mantissa. This system balances precision, range, and computational efficiency, making it indispensable for modern scientific computing (Goldberg, 1991).

Binary Coded Decimal (BCD). BCD, introduced in the early computer era, represents decimal digits directly in binary form. This representation avoids rounding errors associated with binary floating-point arithmetic, making it ideal for financial and accounting systems (Randell, 1982). Despite its storage inefficiency, BCD remains in use for applications requiring exact decimal precision.

1.1.5 Modern Trends and Challenges

Arbitrary precision and energy-efficient representations are current areas of continued research and development.

- **Arbitrary Precision Arithmetic:** Modern software libraries support representations that exceed hardware constraints, enabling computations with arbitrarily high precision for cryptography and scientific research (Smith and Doe, 1997).
- **Energy-Efficient Representations:** In the era of low-power devices and AI accelerators, representations like fixed-point and logarithmic encoding are experiencing a resurgence due to their energy efficiency.
- **Approximate Computing:** To reduce power consumption and improve efficiency, approximate computing techniques trade precision for energy savings in tasks like image processing, machine learning, and data analytics. These methods employ representations that prioritize computational simplicity over exactness, reflecting the growing importance of resource-constrained systems (Han and Orshansky, 2013).

1.2 Decimal Arithmetic

1.2.1 Decimal Addition

In the decimal system, addition is performed by aligning numbers according to their place values (units, tens, hundreds, etc.), adding each column from right to left. If the sum in a column exceeds 9, the digit in the "tens" place of the sum becomes a "carry," which is added to the next column on the left.

For example, let's add 456 and 378 step-by-step:

Initial Setup:

- Start by arranging the numbers vertically by place value.

$$\begin{array}{r} 4 \ 5 \ 6 \\ + 3 \ 7 \ 8 \\ \hline \end{array}$$

Step 1: Add the Units Column:

- Add the units column: $6 + 8 = 14$, which is greater than 9.
- Write down 4 in the units place and carry over 1 to the tens column.

$$\begin{array}{r} 1 \\ 4 \ 5 \ 6 \\ + 3 \ 7 \ 8 \\ \hline 4 \end{array}$$

Step 2: Add the Tens Column:

- Now, move to the tens column.
- Add $5 + 7 = 12$, and add the carry of 1, which gives $12 + 1 = 13$.
- Write down 3 in the tens place and carry over 1 to the hundreds column.

$$\begin{array}{r} 1 \ 1 \\ 4 \ 5 \ 6 \\ + 3 \ 7 \ 8 \\ \hline 3 \ 4 \end{array}$$

Step 3: Add the Hundreds Column:

- Finally, add the hundreds column.
- Add $4 + 3 = 7$, then add the carry of 1, which gives $7 + 1 = 8$.
- Write down 8 in the hundreds place.

$$\begin{array}{r} 1 \ 1 \\ 4 \ 5 \ 6 \\ + 3 \ 7 \ 8 \\ \hline 8 \ 3 \ 4 \end{array}$$

Final Result:

- The sum of $456 + 378$ is 834.

1.2.2 Decimal Multiplication

Decimal multiplication involves breaking down the operation into several smaller multiplications and then adding the results. Here's a step-by-step example multiplying 23 by 45:

Set Up the Multiplication:

- Arrange 23 and 45 in a vertical multiplication layout.

$$\begin{array}{r} 23 \\ \times 45 \\ \hline \end{array}$$

Step 1: Multiply by the Units Digit (5):

- Multiply each digit in 23 by 5, starting from the rightmost digit (units).
 - ◊ Units Column: $3 \times 5 = 15$. Write down 5 and carry over 1.
 - ◊ Tens Column: $2 \times 5 = 10$, plus the carry of 1 gives 11. Write down 1 and carry over 1 to the next column.
- Result after multiplying 23 by 5: 115.

$$\begin{array}{r} 11 \\ 23 \\ \times 5 \\ \hline 115 \end{array}$$

Step 2: Multiply by the Tens Digit (4):

- Write a 0 in the units place
- Multiply each digit in 23 by 4
- Tens column: $3 \times 4 = 12$. Write down 2 and carry over 1.
- Hundreds column: $2 \times 4 = 8$, plus the carry of 1 gives 9. Write down 9.

$$\begin{array}{r} 11 \\ 23 \\ \times 4 \\ \hline 115 \\ 920 \end{array}$$

Step 3: Add the Results:

- $5 + 0 = 5$. Write a 5 in the units place
- $1 + 2 = 3$. Write a 3 in the tens place
- $9 + 1 = 10$. Write a 0 in the hundreds place. Carry the 1.
- $1 + 0 = 1$. Write a 1 in the thousands place.

$$\begin{array}{r}
 1 \\
 1 \ 1 \ 5 \\
 9 \ 2 \ 0 \\
 \hline
 1 \ 0 \ 3 \ 5
 \end{array}$$

Thus, $23 \times 45 = 1035$.

1.2.3 Decimal Subtraction

The concepts of carrying (in addition) and borrowing (in subtraction) are important in basic arithmetic:

- Carrying occurs in addition when a sum exceeds the base value (10 in decimal). The extra value is "carried" to the next column.
- Borrowing occurs in subtraction when the top digit in a column is smaller than the bottom digit. We "borrow" 1 from the next column to the left.

For example, let's subtract 456 from 812:

Initial Setup:

- Write the numbers vertically, aligning them by place value.

$$\begin{array}{r}
 8 \ 1 \ 2 \\
 - \ 4 \ 5 \ 6 \\
 \hline
 \end{array}$$

Step 1. Right Column (Units Place):

- Start with the units column. $2 - 6$ is not possible because 2 is less than 6, so we need to borrow.
- Borrow 1 from the tens column. This turns the 2 into 12 and reduces the tens place from 1 to 0.
- Now, $12 - 6 = 6$, so we write down 6 in the units place.

$$\begin{array}{r}
 8 \ 0 \ 12 \\
 - \ 4 \ 5 \ 6 \\
 \hline
 6
 \end{array}$$

Step 2. Middle Column (Tens Place):

- Move to the tens column, where we now have $0 - 5$.
- Since 0 is less than 5, we need to borrow again.
- Borrow 1 from the hundreds column, turning the 0 into 10 and reducing the hundreds column from 8 to 7.
- Now, $10 - 5 = 5$, so we write down 5 in the tens place.

$$\begin{array}{r}
 7 \ 10 \ 12 \\
 - \ 4 \ 5 \ 6 \\
 \hline
 5 \ 6
 \end{array}$$

Step 3. Left Column (Hundreds Place):

- Finally, move to the hundreds column.
- $7 - 4 = 3$, so we write down 3 in the hundreds place.

$$\begin{array}{r} 7 \ 10 \ 12 \\ - \ 4 \ 5 \ 6 \\ \hline 3 \ 5 \ 6 \end{array}$$

This yields $812 - 456 = 356$.

1.2.4 Decimal Division

Long division is a method for dividing two numbers to find how many times one number fits into another. We'll illustrate this by dividing 853 by 4, showing each step in detail.

Set Up the Division Problem:

- We write 853 as the dividend (the number being divided) and 4 as the divisor (the number we're dividing by).

$$4 \overline{) 853}$$

Step 1. Divide the Hundreds Place:

- First, see how many times 4 fits into the hundreds digit of 853, which is 8.
- 4 fits into 8 exactly 2 times. Write 2 above the 8 in the quotient position.
- Multiply $2 \times 4 = 8$ and subtract from 8, leaving 0.

$$\begin{array}{r} 2 \\ 4 \overline{) 8 \ 5 \ 3} \\ \underline{-8} \\ 0 \end{array}$$

Step 2. Bring Down the Next Digit (Tens Place):

- Bring down the next digit in 853, which is 5.
- Now, determine how many times 4 fits into 5. It fits 1 time. Write 1 above the 5 in the quotient.
- Multiply $1 \times 4 = 4$ and subtract from 5, leaving 1.

$$\begin{array}{r} 2 \ 1 \\ 4 \overline{) 8 \ 5 \ 3} \\ \underline{-4} \\ 1 \end{array}$$

Step 3. Bring Down the Last Digit (Units Place):

- Bring down the last digit in 853, which is 3.
- Determine how many times 4 fits into 13. It fits 3 times. Write 3 in the quotient.
- Multiply $3 \times 4 = 12$ and subtract from 13, leaving 1.

$$\begin{array}{r} 213 \\ 4 \overline{) 853} \\ \underline{8} \\ 5 \\ \underline{12} \\ 1 \end{array}$$

Final Quotient: Since there are no more digits to bring down and the remainder is 1, the division is complete. The result of $853 \div 4$ is 213 with a remainder of 1.

$$853 \div 4 = 213 \text{ with a remainder of } 1$$

1.2.5 Alternative Decimal Arithmetic Algorithms

Alternative algorithms for addition and subtraction provide efficient strategies that are often different from standard grade-school methods. For example, left-to-right addition, also known as column-wise addition, allows numbers to be added starting from the leftmost digit, making it easier to track partial sums and perform mental calculations (Maharaj, 2010). Another technique, the breaking apart method, or partial sums addition, involves adding corresponding place values individually before summing them together, which is particularly effective for simplifying multi-digit addition (Ashlock, 1998). The compensation method, widely used in mental arithmetic, adjusts one number to a nearby round number, simplifying the addition or subtraction process and then compensating for the change afterward (Bobis, 2004).

Subtraction techniques also include counting up, where the user counts up from the subtrahend to the minuend, eliminating the need for borrowing (Neill, 1993). The equal additions method, often called European subtraction, adds the same amount to both numbers to avoid borrowing entirely, making subtraction easier (Martin, 1992). Swedish subtraction, or direct subtraction, allows for direct subtraction of digits without borrowing, even if intermediate negative values occur; these values are later adjusted with a single borrowing step (Nelson, 2002). Finally, the complements method for subtraction, often used in digital systems, involves converting the subtrahend to its 10's complement and adding it to the minuend, simplifying the subtraction process in certain contexts (Niven, 1991).

Alternative algorithms for multiplication and division offer different approaches that can often be more intuitive or efficient than standard methods. The Russian Peasant Multiplication algorithm, also known as the doubling and halving method, involves doubling one factor and halving the other, discarding rows where the halved value is even, and summing the remaining doubled values to reach the final product (Ifrah, 2000b). Lattice multiplication, or the grid method, simplifies multi-digit multiplication by arranging the partial products within a lattice and summing along diagonals, providing a highly visual way to handle carrying and alignment (Smith, 2009). Egyptian multiplication, similar to the Russian method, involves doubling but skips halving, using only sums of doubles that match the multiplicand to achieve the product (Koch, 1970).

For division, the chunking or partial quotients method involves repeated subtraction of multiples of the divisor from the dividend, allowing for a flexible approach that does not require exact division at each step (Reys, 1995). Another approach is cross multiplication for division, which leverages factors to

simplify division tasks through a series of multiplications and subtractions (Parker, 2014). Finally, Vedic mathematics techniques, such as vertically and crosswise multiplication, enable complex multiplications and divisions through specific rules that streamline steps by focusing on place values and relationships between numbers (Tirthaji, 1992). Each of these methods provides unique strategies for simplifying multiplication and division, and they are especially helpful for mental calculations or visual learners.

1.2.6 10's Complement

Because 2's complement is used in nearly all modern computers, we show an example of 10's complement to illustrate the technique. The 10's complement is a method for simplifying subtraction by converting it into addition.

For any n -digit number:

- The 10's complement of a number is obtained by subtracting each digit from 9 and then adding 1 to the least significant digit of the result.
- If the result of the addition results in a leading carry (a "1" at the beginning of the number), it can be dropped, which effectively gives the correct result for the subtraction.

In essence:

$$\text{10's complement of a number } x = (10^n - x)$$

where n is the number of digits in the number.

To subtract a number B from A using 10's complement:

- Find the 10's complement of B .
- Add this complement to A .
- Drop the leading 1 if there's an overflow (carry) in the result, which finalizes the subtraction.

Example: Subtract 275 from 500 using 10's complement

Step 1. Find the 10's Complement of 275:

- Subtract each digit of 275 from 9

$$999 - 275 = 724$$

- Add 1 to the result:

$$724 + 1 = 725$$

So, the 10's complement of 275 is 725.

Step 2. Add the 10's Complement of 275 to 500:

$$500 + 725 = 1225$$

Step 3. Drop the Leading 1: - Since 1225 has a leading 1, we drop it to get the final result:

$$1225 \rightarrow 225$$

Thus, $500 - 275 = 225$.

1.3 Positional Notation

Positional notation is a method for representing numbers in which the position of each digit in a number determines its value. This is a foundational concept in arithmetic and computer science, as it enables efficient representation and manipulation of numbers. In positional notation, each position is associated with a power of a given base (or radix), allowing numbers to be represented compactly and with minimal symbols (Ross, 2008).

1.3.1 Positional Notation in Base 10

In the decimal system, which is base 10, each digit in a number corresponds to a specific power of 10, depending on its position. For any integer N composed of n digits $d_n, d_{n-1}, \dots, d_1, d_0$, we can write N as:

$$N = d_n \cdot 10^n + d_{n-1} \cdot 10^{n-1} + \dots + d_1 \cdot 10^1 + d_0 \cdot 10^0$$

where each d_i is a digit contained in the set $d \in D = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$.

10^n represents the highest place value, where d_n is the coefficient. Each subsequent term decreases the power of 10 by one, moving from left to right in the number.

1.3.2 Example of Positional Notation in Base 10

To illustrate, let's examine the number 357. We can expand this number based on its positional values as follows:

$$357 = 3 \cdot 10^2 + 5 \cdot 10^1 + 7 \cdot 10^0$$

Breaking down each part:

- The leftmost digit, 3, is in the hundreds place. Its positional value is $3 \cdot 10^2 = 300$.
- The middle digit, 5, is in the tens place, contributing $5 \cdot 10^1 = 50$.
- The rightmost digit, 7, is in the units place, contributing $7 \cdot 10^0 = 7$.

Adding these values gives:

$$357 = 300 + 50 + 7$$

which confirms that the positional notation accurately represents the number 357.

1.3.3 Importance of Positional Notation

The power of positional notation lies in its ability to represent large numbers compactly. Unlike systems that use tally marks or other non-positional symbols, positional notation can handle any magnitude of numbers with a finite set of symbols, scaling efficiently as numbers grow. For instance, to represent

a number like 357, only three digits are required in base 10. In contrast, a tally mark system would require writing out 357 individual tally marks, which could be grouped for readability but remains far less efficient than positional notation.

1.3.4 Positional Notation for Arithmetic Operations

Positional notation also enables straightforward arithmetic operations such as addition, subtraction, multiplication, and division by aligning numbers by their place values. For example, adding two numbers in positional notation involves summing each pair of digits in the same place value and carrying over if the sum exceeds 9. This place-based structure makes arithmetic systematic and algorithmic, lending itself well to both manual and computational methods of calculation (Burton, 2010a).

1.3.5 Base/Radix Concepts

In mathematics and computer science, a base (also called a radix) is the number of unique symbols, including zero, used to represent numbers in a positional numeral system (Knuth, 1997a; Rosen, 2018). The base of a number system is denoted by r , and the unique symbols, or digits, are contained in the set $D = \{0, 1, 2, \dots, r-1\}$. In the decimal (base 10) system, for example, $r = 10$, and the set of digits D is $\{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$. However, in other bases, such as binary (base 2) or hexadecimal (base 16), the set D and the value of r would differ.

1.3.6 Generalized Positional Notation

For a number in any base r , the representation follows a similar positional notation structure to base 10. A number N in base r with $n+1$ digits can be written as:

$$N = d_n \cdot r^n + d_{n-1} \cdot r^{n-1} + \dots + d_1 \cdot r^1 + d_0 \cdot r^0$$

where each $d_i \in D$ is an integer from 0 to $r-1$ and represents the coefficient of each power of r . This means that each position in the number corresponds to a specific power of r , with r^n representing the highest place value (determined by the leftmost digit, d_n) and each subsequent term decreasing by a power of r .

1.3.7 Positional Notation for Different Bases

The decimal system (base 10) is just one example of a positional numeral system. Other bases follow the same general rules but use different sets of digits. For instance:

- Binary (base 2): Uses $D = \{0, 1\}$, with each position representing powers of 2. This system is fundamental in digital computing, as all binary values can be represented by combinations of 0 and 1. A number N in binary can be represented as:

$$N = d_n \cdot 2^n + d_{n-1} \cdot 2^{n-1} + \dots + d_1 \cdot 2^1 + d_0 \cdot 2^0$$

- Octal (base 8): Uses $D = \{0, 1, 2, 3, 4, 5, 6, 7\}$, where each position represents powers of 8. A number N in octal can be represented as:

$$N = d_n \cdot 8^n + d_{n-1} \cdot 8^{n-1} + \dots + d_1 \cdot 8^1 + d_0 \cdot 8^0$$

- Hexadecimal (base 16): Uses $D = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F\}$, where each position represents powers of 16. A number N in hexadecimal can be represented as:

$$N = d_n \cdot 16^n + d_{n-1} \cdot 16^{n-1} + \dots + d_1 \cdot 16^1 + d_0 \cdot 16^0$$

- Duodecimal (base 12): Uses $D = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B\}$, where each position represents powers of 12. A number N in duodecimal can be represented as:

$$N = d_n \cdot 12^n + d_{n-1} \cdot 12^{n-1} + \dots + d_1 \cdot 12^1 + d_0 \cdot 12^0$$

- Sexagesimal (base 60): Uses $D = \{0, 1, 2, \dots, 59\}$, with each position representing powers of 60. A number N in sexagesimal can be represented as:

$$N = d_n \cdot 60^n + d_{n-1} \cdot 60^{n-1} + \dots + d_1 \cdot 60^1 + d_0 \cdot 60^0$$

Bases 2, 8, and 16 are often used in computing and digital electronics because of their implementation properties. Historically, base 60, or the sexagesimal system, was used by the ancient Babylonians for mathematical and astronomical calculations, and its influence persists today in the way we measure time (e.g., 60 seconds in a minute, 60 minutes in an hour) (Robson, 2008). Base 12, known as the duodecimal system, has also been significant, with roots in ancient cultures where it was valued for its divisibility by multiple factors (2, 3, 4, and 6), making it useful for trade, measurements, and time-keeping systems. Traces of base 12 are still evident in modern units, such as a dozen (12 items) and the division of hours into sets of 12 on a clock face (Ifrah, 2000b).

1.4 Binary Arithmetic

Binary arithmetic is the foundation of all digital computer systems because binary numbers, composed of only two digits (0 and 1), correspond directly to the on-off states of transistors, which are the basic building blocks of digital circuits. These two states are convenient for digital electronics because they can easily represent high and low voltage levels, aligning with binary logic (Knuth, 1997a). Despite its simplicity, binary arithmetic follows the same general rules as decimal arithmetic. This reduction to two possible states significantly simplifies hardware design and enables efficient processing (Rosen, 2018).

1.4.1 Binary Addition

Binary addition operates similarly to decimal addition but involves only the digits 0 and 1, with carry rules adapted to binary logic. Binary addition requires only four basic cases:

$$\begin{aligned} 0 + 0 &= 0 \\ 0 + 1 &= 1 \\ 1 + 0 &= 1 \\ 1 + 1 &= 10 \quad (\text{where 1 is carried to the next higher bit}) \end{aligned}$$

When the sum of two bits equals 2 (in decimal), this triggers a carry to the next higher bit, as shown in the last case. This carry-over behavior is fundamental to binary addition and is analogous to the carry

mechanism in decimal addition. For instance, adding the binary numbers 1011 and 1101 proceeds as follows:

$$\begin{array}{r}
 \text{ (Carry)} \\
 \\
 + \\
 \hline
 1
 \end{array} = 11000_2$$

This example demonstrates that binary addition, although straightforward, can require multiple carry operations when the bit values sum to 1 in successive columns (Knuth, 1997b).

1.4.2 Binary Subtraction

Binary subtraction is similar to decimal subtraction, with borrowing rules adapted to the binary system. Binary subtraction also involves four basic cases:

$$\begin{aligned}
 0 - 0 &= 0 \\
 1 - 0 &= 1 \\
 1 - 1 &= 0 \\
 0 - 1 &= 1 \quad (\text{borrow 1 from the next higher bit})
 \end{aligned}$$

When subtracting a larger bit from a smaller bit (such as $0 - 1$), we need to "borrow" from the next higher bit, just as in decimal subtraction. This process is slightly simplified in binary because the only possible values are 0 and 1, reducing the complexity of borrowing. For instance, consider subtracting 101 from 1100:

Step 1: Initial Setup. We add a 0 to the subtrahend to align the digits.

$$\begin{array}{r}
 \\
 - \\
 \hline

 \end{array}$$

Step 2: Subtract the rightmost column.

Here, $0 - 1$ requires borrowing from the second column. Since the second column also has 0, we need to move leftward until we find a 1 to borrow. The third column has a 1. The minuend is set to 0 and the borrow row is set to two.

$$\begin{array}{r}
 \text{Borrow:} \\
 \\
 - \\
 \hline

 \end{array}$$

Step 3. Move the 2 to the first column subtracting 1 from the second column.

$$\begin{array}{r}
 \text{Borrow:} \\
 \\
 - \\
 \hline

 \end{array}$$

Step 4. We can subtract $1 - 0 = 1$ and place the result in the second column.

$$\begin{array}{r}
 \text{Borrow:} \quad \quad \quad 1 \ 2 \\
 \quad \quad \quad 1 \ 0 \ 0 \ 0 \\
 - \quad \quad 0 \ 1 \ 0 \ 1 \\
 \hline
 \quad \quad \quad 1 \ 1
 \end{array}$$

Step 5. The third column has $0 - 1$ so we need to borrow again. We set the fourth column minuend to **0** and place a 2 in the borrow row. We can then complete the subtraction $2 - 1 = 1$ and place it in the third row of the result.

$$\begin{array}{r}
 \text{Borrow:} \quad \quad \quad 2 \ 1 \ 2 \\
 \quad \quad \quad \mathbf{0} \ 0 \ 0 \ 0 \\
 - \quad \quad 0 \ 1 \ 0 \ 1 \\
 \hline
 \quad \quad \quad 1 \ 1 \ 1
 \end{array}$$

Step 7. We complete the result by subtracting $0 - 0 = 0$ and place a 0 in the fourth column of the result.

$$\begin{array}{r}
 \text{Borrow:} \quad 0 \ 2 \ 1 \ 2 \\
 \quad \quad \quad 0 \ 0 \ 0 \ 0 \\
 - \quad \quad 0 \ 1 \ 0 \ 1 \\
 \hline
 \quad \quad \quad 0 \ 1 \ 1 \ 1
 \end{array} = 0111_2$$

This example illustrates that borrowing in binary works similarly to decimal borrowing.

1.4.3 Binary Multiplication - TBD

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

1.4.4 Binary Division - TBD

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Chapter 2

Numerical Sets

2.1	Formal Notation for Numerical Sets	20
2.2	The Set of Integers ($\mathbb{Z}$)	20
2.2.1	Properties of $\mathbb{Z}$	20
2.2.2	Subsets of $\mathbb{Z}$	21
2.3	The Set of Real Numbers ($\mathbb{R}$)	21
2.3.1	Properties of $\mathbb{R}$	21
2.3.2	Subsets of $\mathbb{R}$	22
2.4	Cauchy Sequences	22
2.4.1	Cauchy Sequence Definition	22
2.4.2	Properties of Cauchy Sequence	23
2.4.3	Importance	23
2.5	$\mathbb{R}$ as a Metric Space	23
2.5.1	Properties	23
2.5.2	Standard Metric Properties:	24
2.5.3	Completeness of $\mathbb{R}$	24
2.5.4	Alternative Metrics on $\mathbb{R}$	24
2.5.5	Historical Context	25
2.5.6	Applications	25
2.6	Comparing the Properties of $\mathbb{Z}$ and $\mathbb{R}$	25
2.6.1	Properties of $\mathbb{Z}$ Not Shared by $\mathbb{R}$	25
2.6.2	Properties of $\mathbb{R}$ Not Shared by $\mathbb{Z}$	26
2.6.3	Summary Table of Differences between $\mathbb{Z}$ and $\mathbb{R}$	27
2.7	The Set of Rational Numbers ($\mathbb{Q}$)	27
2.7.1	Properties of $\mathbb{Q}$	27
2.7.2	Subsets of $\mathbb{Q}$	28
2.8	The Set of Complex Numbers ($\mathbb{C}$)	28
2.8.1	Properties of $\mathbb{C}$	29

2.1 Formal Notation for Numerical Sets

Our description of number systems and algorithms has been informal. We now formalize what sets of numbers we will be computing with. The development of mathematical sets, including integers, rational numbers, real numbers, and complex numbers, define what numbers a processor can compute. Integers are used for memory addressing, while processors use real and complex numbers to solve problems in many applications, including calculus and physics.

2.2 The Set of Integers ($\mathbb{Z}$)

The set of integers, denoted by $\mathbb{Z}$ (from the German word "Zahlen," meaning "numbers"), consists of positive and negative whole numbers, as well as zero (Knuth, 1997a). Integers are foundational concepts in arithmetic, memory addressing, and computer architecture.

Formally:

$$\mathbb{Z} = \{\dots, -3, -2, -1, 0, 1, 2, 3, \dots\}$$

2.2.1 Properties of $\mathbb{Z}$

- **Closure:** $\mathbb{Z}$ is closed under addition, subtraction, and multiplication.

$$\text{For any } a, b \in \mathbb{Z}, a + b, a - b, \text{ and } a \cdot b \text{ are also in } \mathbb{Z}$$

- **Associativity:** Addition and multiplication in $\mathbb{Z}$ are associative:

$$(a + b) + c = a + (b + c) \quad \text{and} \quad (a \cdot b) \cdot c = a \cdot (b \cdot c).$$

- **Commutativity:** Addition and multiplication are commutative:

$$a + b = b + a \quad \text{and} \quad a \cdot b = b \cdot a.$$

- **Existence of Identity Elements:** The additive identity is 0, and the multiplicative identity is 1:

$$a + 0 = a \quad \text{and} \quad a \cdot 1 = a \quad \forall a \in \mathbb{Z}.$$

- **Existence of Inverses:** Every integer a has an additive inverse $-a$ such that:

$$a + (-a) = 0.$$

However, $\mathbb{Z}$ does not contain multiplicative inverses for nonzero elements, as division does not always yield an integer.

- **Distributive Property:** Multiplication distributes over addition:

$$a \cdot (b + c) = (a \cdot b) + (a \cdot c).$$

- **Ordered Set:** $\mathbb{Z}$ is an ordered set with a natural less-than relation ($<$) satisfying:

$$a < b \implies a + c < b + c \quad \text{and} \quad a < b, c > 0 \implies a \cdot c < b \cdot c.$$

2.2.2 Subsets of $\mathbb{Z}$

Various subsets of integers are frequently used:

- **Positive integers:** Denoted as $\mathbb{Z}^+$ or $\mathbb{N}^+$, representing all integers greater than zero:

$$\mathbb{Z}^+ = \mathbb{N}^+ = \{1, 2, 3, \dots\}$$

- **Negative integers:** Denoted as $\mathbb{Z}^-$, representing all integers less than zero:

$$\mathbb{Z}^- = \{-1, -2, -3, \dots\}$$

- **Natural numbers including zero:** Denoted as $\mathbb{N}$, representing non-negative integers:

$$\mathbb{N} = \{0, 1, 2, 3, \dots\}$$

2.3 The Set of Real Numbers ($\mathbb{R}$)

The set of real numbers, denoted as $\mathbb{R}$, represents a continuous spectrum of numbers along the number line. This set includes both rational and irrational numbers and is fundamental in mathematics and science.

2.3.1 Properties of $\mathbb{R}$

- **Closure:** $\mathbb{R}$ is closed under addition, subtraction, multiplication, and division (except by zero).
For all $a, b \in \mathbb{R}$:

$$a + b \in \mathbb{R}, \quad a - b \in \mathbb{R}, \quad a \cdot b \in \mathbb{R}, \quad \text{and, if } b \neq 0, \quad \frac{a}{b} \in \mathbb{R}.$$

- **Associativity:** Addition and multiplication in $\mathbb{R}$ are associative:

$$(a + b) + c = a + (b + c), \quad (a \cdot b) \cdot c = a \cdot (b \cdot c).$$

- **Commutativity:** Addition and multiplication are commutative:

$$a + b = b + a, \quad a \cdot b = b \cdot a.$$

- **Existence of Identity Elements:** $\mathbb{R}$ has both additive and multiplicative identity elements:

$$a + 0 = a \quad (\text{additive identity}), \quad a \cdot 1 = a \quad (\text{multiplicative identity}).$$

- **Existence of Inverses:**

◊ For every $a \in \mathbb{R}$, there exists an additive inverse $-a$ such that:

$$a + (-a) = 0.$$

◇ For every $a \in \mathbb{R}$, $a \neq 0$, there exists a multiplicative inverse a^{-1} such that:

$$a \cdot a^{-1} = 1.$$

- **Distributive Property:** Multiplication distributes over addition:

$$a \cdot (b + c) = (a \cdot b) + (a \cdot c).$$

- **Density:** $\mathbb{R}$ is dense, meaning that between any two distinct real numbers $a, b \in \mathbb{R}$, there exists another real number $c \in \mathbb{R}$ such that:

$$a < c < b.$$

- **Completeness:** $\mathbb{R}$ is a complete metric space. This means that every Cauchy sequence (see Section 2.4) of real numbers converges to a limit that is also in $\mathbb{R}$.

- **Ordered Field:** $\mathbb{R}$ is an ordered field, meaning it is equipped with a total order $\leq$ that satisfies:

$$a \leq b \implies a + c \leq b + c, \quad a \leq b \text{ and } c > 0 \implies a \cdot c \leq b \cdot c.$$

- **Continuity:** The real numbers form a continuum with no gaps, enabling the definition of limits, derivatives, and integrals in calculus.

2.3.2 Subsets of $\mathbb{R}$

- Rational numbers ($\mathbb{Q}$): Numbers expressible as fractions of integers.
- Irrational numbers: Numbers not expressible as fractions (e.g., $\sqrt{2}$, π).

2.4 Cauchy Sequences

A Cauchy sequence is a mathematical concept used to describe sequences of numbers (or elements in a metric space) that become arbitrarily close to each other as the sequence progresses (Rudin, 1976). It plays a fundamental role in real analysis and the study of completeness in metric spaces (see Section 2.5).

2.4.1 Cauchy Sequence Definition

A sequence $\{x_n\}$ in a metric space (X, d) is called a Cauchy sequence if, for every $\varepsilon > 0$, there exists a positive integer N such that for all $m, n \geq N$:

$$d(x_m, x_n) < \varepsilon.$$

Where:

- $d(x_m, x_n)$: The distance between the elements x_m and x_n .
- $\varepsilon > 0$: An arbitrarily small positive number.

In simpler terms, a Cauchy sequence is one in which the terms get "closer and closer" to each other as the sequence progresses. This closeness is defined without reference to a specific limit point, relying instead on the internal behavior of the sequence.

2.4.2 Properties of Cauchy Sequence

1) Cauchy Sequences in $\mathbb{R}$:

- In the set of real numbers ($\mathbb{R}$), every Cauchy sequence converges to a limit within $\mathbb{R}$.
- This property arises because $\mathbb{R}$ is a complete metric space.

2) Cauchy Sequences in General Metric Spaces:

- Not all Cauchy sequences converge in general metric spaces.
- A metric space is called a complete metric space if every Cauchy sequence in the space converges to a limit within the space.

3) Examples:

- The sequence $\{1, 1/2, 1/3, 1/4, \dots\}$ is a Cauchy sequence in $\mathbb{R}$, as the terms get arbitrarily close to each other as $n \rightarrow \infty$.
- In $\mathbb{Q}$ (the set of rational numbers), the sequence $\{3, 3.1, 3.14, 3.141, \dots\}$, which approaches π , is a Cauchy sequence. However, it does not converge in $\mathbb{Q}$ because $\pi \notin \mathbb{Q}$.

2.4.3 Importance

Cauchy sequences:

- Define completeness: A space is complete if all Cauchy sequences converge within the space.
- Analyze convergence: They allow studying the behavior of sequences independently of a known limit.
- Construct the real numbers: The real numbers ($\mathbb{R}$) can be constructed by "completing" the rational numbers ($\mathbb{Q}$) using equivalence classes of Cauchy sequences.

2.5 $\mathbb{R}$ as a Metric Space

$\mathbb{R}$, the set of real numbers, is a metric space when equipped with the standard metric (distance function) (Rudin, 1976). A metric space is defined as a set X along with a metric $d : X \times X \rightarrow \mathbb{R}$ that satisfies the following properties for all $x, y, z \in X$:

2.5.1 Properties

- 1) **Non-negativity:** $d(x, y) \geq 0$, and $d(x, y) = 0$ if and only if $x = y$.
- 2) **Symmetry:** $d(x, y) = d(y, x)$.

- 3) **Triangle inequality:** $d(x, z) \leq d(x, y) + d(y, z)$.
- 4) **Definiteness:** $d(x, y) > 0$ if $x \neq y$.

2.5.1.1 $\mathbb{R}$ with the Standard Metric

The real numbers form a metric space under the *standard metric*, defined as:

$$d(x, y) = |x - y|,$$

where $|x - y|$ is the absolute value of the difference between x and y .

2.5.2 Standard Metric Properties:

- **Non-negativity:** $|x - y| \geq 0$, and $|x - y| = 0$ if and only if $x = y$.
- **Symmetry:** $|x - y| = |y - x|$.
- **Triangle inequality:** For any $x, y, z \in \mathbb{R}$,

$$|x - z| \leq |x - y| + |y - z|.$$

- **Definiteness:** $|x - y| > 0$ when $x \neq y$.

Since the standard metric satisfies these properties, $(\mathbb{R}, d)$ is a metric space.

2.5.3 Completeness of $\mathbb{R}$

In addition, $\mathbb{R}$ is a *complete metric space*, meaning every Cauchy sequence in $\mathbb{R}$ converges to a limit that is also in $\mathbb{R}$. This property distinguishes $\mathbb{R}$ from other metric spaces such as $\mathbb{Q}$ (the rationals), where Cauchy sequences may not converge within $\mathbb{Q}$.

2.5.4 Alternative Metrics on $\mathbb{R}$

While $d(x, y) = |x - y|$ is the most common, other metrics can also be defined on $\mathbb{R}$. Examples include:

- The maximum norm:

$$d_{\infty}(x, y) = \max\{|x|, |y|\}.$$

- The generalized p -metric for $p > 0$:

$$d_p(x, y) = (|x - y|^p)^{1/p}.$$

The metric choice determines the metric space's topology and properties, making $\mathbb{R}$ a versatile mathematical structure in different contexts.

2.5.5 Historical Context

The need for $\mathbb{R}$ arose from limitations in rational numbers for describing geometric concepts. For example:

- The diagonal of a square ($\sqrt{2}$) is irrational.
- The ratio of a circle's circumference to its diameter (π) is irrational.

Rigorous definitions of $\mathbb{R}$ emerged in the 19th century through Dedekind cuts and Cauchy sequences, establishing its completeness (Rosen, 2018; Stewart and Tall, 2015).

2.5.6 Applications

Real numbers underpin analysis, continuity, and the infinite processes essential in mathematics and physics. They enable the definition of functions, derivatives, and integrals as well as modeling real-world phenomena.

2.6 Comparing the Properties of $\mathbb{Z}$ and $\mathbb{R}$

Since most processors compute using $\mathbb{Z}$ and $\mathbb{R}$, we compare the differences between these sets. The sets $\mathbb{Z}$ (integers) and $\mathbb{R}$ (real numbers) have distinct properties that reflect their differing algebraic and topological structures. This section highlights the properties unique to $\mathbb{Z}$, those unique to $\mathbb{R}$, and their implications.

2.6.1 Properties of $\mathbb{Z}$ Not Shared by $\mathbb{R}$

1) Discrete Structure:

- $\mathbb{Z}$ is a discrete set, meaning there is a "gap" between consecutive elements. For any $a, b \in \mathbb{Z}$, if $a \neq b$, then $|a - b| \geq 1$.
- In contrast, $\mathbb{R}$ is a continuous set, with no gaps between elements.

2) No Multiplicative Inverses (Non-Divisible):

- Nonzero integers in $\mathbb{Z}$ do not have multiplicative inverses within $\mathbb{Z}$. For example, $\frac{1}{2} \notin \mathbb{Z}$.
- In $\mathbb{R}$, every nonzero element has a multiplicative inverse.

3) Subring:

- $\mathbb{Z}$ is a subring of $\mathbb{R}$, meaning it is closed under addition, subtraction, and multiplication but not division.
- $\mathbb{R}$ is a field that supports division.

4) Discrete Order:

- $\mathbb{Z}$ is ordered such that the "next" integer is uniquely defined (e.g., $a + 1$ for any $a \in \mathbb{Z}$).
- In $\mathbb{R}$, there is no "next" real number due to its continuity.

5) Unique Factorization:

- $\mathbb{Z}$ has the property of unique factorization, where every integer can be expressed uniquely (up to order and sign) as a product of prime numbers.
- $\mathbb{R}$, as a field, does not have a concept of factorization into primes.

2.6.2 Properties of $\mathbb{R}$ Not Shared by $\mathbb{Z}$

1) Completeness:

- $\mathbb{R}$ is a complete metric space, meaning every Cauchy sequence in $\mathbb{R}$ converges to a limit within $\mathbb{R}$ (Rudin, 1976).
- $\mathbb{Z}$ is not complete because sequences that seem to converge (e.g., $\{1, 2, 3, \dots\}$) do not have limits within $\mathbb{Z}$.

2) Field Structure:

- $\mathbb{R}$ is a field, allowing addition, subtraction, multiplication, and division (except by zero) (Sutherland, 2002).
- $\mathbb{Z}$ is only a ring, lacking division as a closed operation.

3) Density:

- $\mathbb{R}$ is dense, meaning that between any two real numbers, there exists another real number.
- $\mathbb{Z}$ is not dense; there are no integers between n and $n + 1$ for any $n \in \mathbb{Z}$.

4) Infinite Subdivisibility:

- $\mathbb{R}$ includes fractions, irrationals, and transcendental numbers, allowing infinite precision and continuity.
- $\mathbb{Z}$ only contains whole numbers.

5) Metric Space with Infinitesimal Distances:

- $\mathbb{R}$ has a natural metric $d(x, y) = |x - y|$ with arbitrarily small distances.

- $\mathbb{Z}$ has a discrete metric $d(x, y) = 1$ for $x \neq y$, with no intermediate values (Munkres, 2000).

2.6.3 Summary Table of Differences between $\mathbb{Z}$ and $\mathbb{R}$

Property	$\mathbb{Z}$	$\mathbb{R}$
Structure	Discrete set	Continuous set
Division	Not closed under division	Closed under division (except by zero)
Density	Not dense (gaps between elements)	Dense (no gaps)
Completeness	Not complete	Complete (Cauchy sequences converge)
Factorization	Unique factorization into primes	No concept of factorization
Metric	Discrete metric: $d(x, y) = 1$	Standard metric: $d(x, y) = x - y $

2.7 The Set of Rational Numbers ($\mathbb{Q}$)

The set of rational numbers, denoted $\mathbb{Q}$, consists of all numbers that can be expressed as the ratio of two integers:

$$\mathbb{Q} = \left\{ \frac{p}{q} \mid p, q \in \mathbb{Z}, q \neq 0 \right\}.$$

2.7.1 Properties of $\mathbb{Q}$

- **Closure:** $\mathbb{Q}$ is closed under addition, subtraction, multiplication, and division (except by zero). For any $a, b \in \mathbb{Q}$:

$$a + b \in \mathbb{Q}, \quad a - b \in \mathbb{Q}, \quad a \cdot b \in \mathbb{Q}, \quad \text{and, if } b \neq 0, \quad \frac{a}{b} \in \mathbb{Q}.$$

- **Associativity:** Addition and multiplication in $\mathbb{Q}$ are associative:

$$(a + b) + c = a + (b + c), \quad (a \cdot b) \cdot c = a \cdot (b \cdot c).$$

- **Commutativity:** Addition and multiplication in $\mathbb{Q}$ are commutative:

$$a + b = b + a, \quad a \cdot b = b \cdot a.$$

- **Existence of Identity Elements:** $\mathbb{Q}$ has additive and multiplicative identity elements:

$$a + 0 = a \quad (\text{additive identity}), \quad a \cdot 1 = a \quad (\text{multiplicative identity}).$$

- **Existence of Inverses:**

◊ Every $a \in \mathbb{Q}$ has an additive inverse $-a$ such that:

$$a + (-a) = 0.$$

◇ Every nonzero $a \in \mathbb{Q}$ has a multiplicative inverse a^{-1} such that:

$$a \cdot a^{-1} = 1.$$

- **Distributive Property:** Multiplication distributes over addition:

$$a \cdot (b + c) = (a \cdot b) + (a \cdot c).$$

- **Density:** $\mathbb{Q}$ is dense in $\mathbb{R}$, meaning that between any two distinct real numbers $a, b \in \mathbb{R}$, there exists a rational number $c \in \mathbb{Q}$ such that:

$$a < c < b.$$

- **Countability:** $\mathbb{Q}$ is countable, meaning its elements can be arranged in a one-to-one correspondence with the natural numbers $\mathbb{N}$. Unlike $\mathbb{R}$, which is uncountable, $\mathbb{Q}$ is a smaller infinity.
- **Incompleteness:** $\mathbb{Q}$ is not complete. There exist Cauchy sequences in $\mathbb{Q}$ that do not converge within $\mathbb{Q}$. For example, the sequence $\{1, 1.4, 1.41, 1.414, \dots\}$ (approaching $\sqrt{2}$) has no limit in $\mathbb{Q}$ because $\sqrt{2} \notin \mathbb{Q}$.

2.7.2 Subsets of $\mathbb{Q}$

- **Proper Fractions:** Rational numbers whose absolute value is less than 1, excluding 0:

$$\text{Proper fractions} = \{q \in \mathbb{Q} \mid 0 < |q| < 1\}.$$

- **Improper Fractions:** Rational numbers whose absolute value is greater than or equal to 1:

$$\text{Improper fractions} = \{q \in \mathbb{Q} \mid |q| \geq 1\}.$$

2.8 The Set of Complex Numbers ($\mathbb{C}$)

Most processors do not directly support operations on complex numbers. However, special purpose processors including some Digital Signal Processors have provided limited support particularly for Fast Fourier Transform operations.

The set of complex numbers, denoted $\mathbb{C}$, consists of all numbers that can be expressed in the form:

$$z = a + bi, \quad \text{where } a, b \in \mathbb{R} \text{ and } i^2 = -1.$$

Here, a is called the real part of z , denoted $\text{Re}(z)$, and b is called the imaginary part, denoted $\text{Im}(z)$. Complex numbers extend the real number system to include solutions to polynomial equations that have no real roots, such as $x^2 + 1 = 0$.

2.8.1 Properties of $\mathbb{C}$

- **Closure:** $\mathbb{C}$ is closed under addition, subtraction, multiplication, and division (except division by zero). For all $z_1, z_2 \in \mathbb{C}$:

$$z_1 + z_2 \in \mathbb{C}, \quad z_1 - z_2 \in \mathbb{C}, \quad z_1 \cdot z_2 \in \mathbb{C}, \quad \text{and, if } z_2 \neq 0, \quad \frac{z_1}{z_2} \in \mathbb{C}.$$

- **Associativity:** Addition and multiplication in $\mathbb{C}$ are associative:

$$(z_1 + z_2) + z_3 = z_1 + (z_2 + z_3), \quad (z_1 \cdot z_2) \cdot z_3 = z_1 \cdot (z_2 \cdot z_3).$$

- **Commutativity:** Addition and multiplication in $\mathbb{C}$ are commutative:

$$z_1 + z_2 = z_2 + z_1, \quad z_1 \cdot z_2 = z_2 \cdot z_1.$$

- **Existence of Identity Elements:** $\mathbb{C}$ has additive and multiplicative identity elements:

$$z + 0 = z \quad (\text{additive identity}), \quad z \cdot 1 = z \quad (\text{multiplicative identity}).$$

- **Existence of Inverses:**

- ◊ Every $z \in \mathbb{C}$ has an additive inverse $-z$ such that:

$$z + (-z) = 0.$$

- ◊ Every nonzero $z \in \mathbb{C}$ has a multiplicative inverse z^{-1} such that:

$$z \cdot z^{-1} = 1.$$

- **Distributive Property:** Multiplication distributes over addition:

$$z_1 \cdot (z_2 + z_3) = (z_1 \cdot z_2) + (z_1 \cdot z_3).$$

- **Field Structure:** $\mathbb{C}$ is a field, satisfying all the axioms of a field under addition and multiplication.

- **Algebraic Closure:** $\mathbb{C}$ is **algebraically closed**, meaning that every non-constant polynomial with coefficients in $\mathbb{C}$ has a root in $\mathbb{C}$. For example:

$$z^n + c_{n-1}z^{n-1} + \cdots + c_1z + c_0 = 0, \quad c_i \in \mathbb{C}, \quad \exists z \in \mathbb{C}.$$

- **Non-Orderable:** Unlike $\mathbb{R}$, $\mathbb{C}$ cannot be totally ordered in a way that is compatible with its field operations. There is no relation $\leq$ such that $a \leq b \implies a + c \leq b + c$ and $a \cdot c \leq b \cdot c$.

- **Complex Conjugate:** Every $z = a + bi \in \mathbb{C}$ has a complex conjugate $\bar{z} = a - bi$, which satisfies:

$$z \cdot \bar{z} = a^2 + b^2 \quad (\text{a non-negative real number}).$$

- **Norm (Magnitude):** The norm or magnitude of $z = a + bi$ is given by:

$$|z| = \sqrt{a^2 + b^2}.$$

This satisfies the triangle inequality:

$$|z_1 + z_2| \leq |z_1| + |z_2|.$$

These properties make $\mathbb{C}$ an essential mathematical system for solving equations, analyzing systems, and modeling phenomena in engineering, physics, signal processing, and quantum mechanics.

Chapter 3

Finite Unsigned Integer Binary Representations in $\mathbb{N}$

3.1	Sets of Numbers and Representations of Numbers	32
3.1.1	Sets of Numbers	32
3.1.2	Representations of Numbers	32
3.2	Unsigned Binary Numbers	33
3.3	Range of Unsigned Binary Numbers	33
3.4	Example Ranges for Different Bit Lengths	34
3.5	B_n with Overflow and Underflow (Modulo Arithmetic)	35
3.5.1	Overflow and Underflow Behavior	35
3.5.2	Computer Arithmetic Example Overflow	36
3.5.3	Computer Arithmetic Example Underflow	36
3.6	Definition of B_n with Clipping (Saturation Arithmetic)	37
3.6.1	Behavior of Arithmetic Operations with Clipping	37
3.6.2	Computer Arithmetic Example: Overflow with Clipping	37
3.6.3	Computer Arithmetic Example: Underflow with Clipping	38

Unlike traditional positional notation, where the number of digits can extend indefinitely, digital systems must represent both integers and non-integer values within a constrained bit-width. This finite bit-width limits the precision and range of values that can be represented, necessitating specialized schemes to approximate an infinite set of values.

Complement systems, Binary-Coded Decimal (BCD), and floating-point formats, are examples of number representations that have been developed to accommodate specific needs. These alternative representations extend beyond traditional positional/base systems by providing advantages for particular operations, memory usage, or display accuracy, for both integer and non-integer data (Knuth, 1997b; Rosen, 2018; Tanenbaum and Austin, 2013).

Depending upon the representation chosen, the same value may have more than one representation or no representation.

3.1 Sets of Numbers and Representations of Numbers

In mathematics and computer science, it is essential to distinguish between sets of numbers, such as $\mathbb{Z}$ (the set of integers), and representations of numbers, such as two's complement, excess- k , or Binary Coded Decimal (BCD). While representations are methods for encoding and manipulating numbers in practical systems, the sets themselves are abstract mathematical constructs that define properties and operations independent of specific implementations. This distinction has significant implications for understanding the properties, limitations, and behaviors of numerical systems.

3.1.1 Sets of Numbers

A set of numbers is a mathematical construct that defines a collection of elements, operations, and properties. Common sets of numbers include:

- $\mathbb{N}$ (Natural Numbers): The set of non-negative integers $\{0, 1, 2, \dots\}$.
- $\mathbb{Z}$ (Integers): The set of all integers $\{\dots, -2, -1, 0, 1, 2, \dots\}$.
- $\mathbb{R}$ (Real Numbers): The set of all numbers on the continuous number line, including both rational and irrational numbers.

Each set is defined by specific properties:

- Closure: Operations such as addition, subtraction, or multiplication remain within the set.
- Associativity and Commutativity: Mathematical operations are consistent and well-defined.
- Existence of Identity and Inverses: Sets may include elements like 0 (additive identity) or $-x$ (additive inverse).

These sets are abstract and independent of how numbers are represented in physical or computational systems.

3.1.2 Representations of Numbers

Representations of numbers refer to the ways in which numbers are encoded, stored, and manipulated in practical systems, such as computers or calculators. Examples of representations include:

- Two's Complement: A binary representation for signed integers.
- Excess- k : A biased representation commonly used in floating-point exponents.
- Binary Coded Decimal (BCD): Encodes each decimal digit as a separate binary sequence.
- IEEE 754 Floating-Point: A standardized representation for real numbers.

Unlike sets, representations are constrained by practical factors such as finite memory, bit-width, and computational efficiency. This can result in behaviors and limitations that deviate from the properties of the corresponding mathematical sets.

3.2 Unsigned Binary Numbers

Unsigned binary numbers are a subset of the natural numbers $\mathbb{N}$, where each number is represented in binary form using a finite sequence of binary digits, or bits. Formally, let $B = \{0, 1\}$ be the set of binary digits. For an n -bit representation, a natural number $x \in \mathbb{N}$ can be expressed as:

$$x = \sum_{i=0}^{n-1} b_i \cdot 2^i, \quad \text{where } b_i \in B \text{ for all } i.$$

The set of n -bit unsigned binary numbers can be defined as:

$$B_n = \{x \in \mathbb{N} \mid 0 \leq x < 2^n\}.$$

Here:

- n is the number of bits used to represent the binary number.
- 2^n is the total number of unique binary values representable with n bits.
- Note that 2^n is not part of B_n .
- b_i is a binary digit (0 or 1) representing the coefficient for 2^i , the positional value at bit i .

3.3 Range of Unsigned Binary Numbers

Since digital systems are finite, the number of bits, n , limits the range of values that can be represented. For n -bit unsigned binary numbers, the range is:

$$B_n = \{0, 1, 2, \dots, 2^n - 1\}.$$

Each number in B_n can be uniquely represented in binary form as a sequence of n bits:

$$x = b_{n-1} \cdot 2^{n-1} + b_{n-2} \cdot 2^{n-2} + \dots + b_1 \cdot 2^1 + b_0 \cdot 2^0, \quad b_i \in \{0, 1\}, \quad i = 0, 1, \dots, n-1.$$

- b_{n-1} is the most significant bit (MSB), corresponding to 2^{n-1} .
- b_0 is the least significant bit (LSB), corresponding to 2^0 .

- Each bit b_i can take the value 0 or 1.

Example for $n = 3$

For $B_3 = \{0, 1, 2, 3, 4, 5, 6, 7\}$, the binary representation and expansion for each number are:

$$x = 0 \Rightarrow 0 \cdot 2^2 + 0 \cdot 2^1 + 0 \cdot 2^0 = 000,$$

$$x = 1 \Rightarrow 0 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 = 001,$$

$$x = 2 \Rightarrow 0 \cdot 2^2 + 1 \cdot 2^1 + 0 \cdot 2^0 = 010,$$

$$x = 3 \Rightarrow 0 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0 = 011,$$

$$x = 4 \Rightarrow 1 \cdot 2^2 + 0 \cdot 2^1 + 0 \cdot 2^0 = 100,$$

$$x = 5 \Rightarrow 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 = 101,$$

$$x = 6 \Rightarrow 1 \cdot 2^2 + 1 \cdot 2^1 + 0 \cdot 2^0 = 110,$$

$$x = 7 \Rightarrow 1 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0 = 111.$$

For an unsigned integer using n bits, the smallest representable value is 0 (all bits set to 0), and the largest representable value is when all bits are set to 1, which corresponds to $2^n - 1$. Thus, the range of an n -bit unsigned integer is:

$$[0, 2^n - 1] \text{ sometimes written as } [0, 2^n)$$

This means that increasing n by one bit doubles the range of values that can be represented, as each additional bit allows for twice as many combinations.

Mathematically, $< 2^n$ ensures clarity in interval definitions. In engineering, the range is often expressed as $[0, 2^n - 1]$, explicitly naming the smallest and largest values. Mathematics focuses on the formal definition of the set and its properties, whereas engineering prioritizes practical application, particularly the endpoints of the range. The maximum value $2^n - 1$ is a natural consequence of the interval $[0, 2^n)$. Engineers often emphasize $2^n - 1$ because it corresponds to the largest representable value in an n -bit binary system. The mathematical definition $0 \leq x < 2^n$ and the engineering range 0 to $2^n - 1$ describe the same set B_n , with $|B_n| = 2^n$.

3.4 Example Ranges for Different Bit Lengths

Table 3.1 shows the minimum and maximum values for different bit lengths, along with the representation of the number 5 in each range.

Table 3.1: Binary Representations for Different Bit Lengths

Bit Length	Range	Minimum Value	Maximum Value	Example 5
4-bit	$[0, 15]$	0000 ₂	1111 ₂	0101 ₂
8-bit	$[0, 255]$	00000000 ₂	11111111 ₂	00000101 ₂
16-bit	$[0, 65535]$	0000000000000000 ₂	1111111111111111 ₂	0000000000000101 ₂

3.5 B_n with Overflow and Underflow (Modulo Arithmetic)

In digital systems, the representation of unsigned binary numbers is constrained by a fixed number of bits, n . When computations exceed the representable range of n -bit numbers, an *overflow* occurs, and when computations fall below zero, an *underflow* occurs. In both cases, numbers are typically handled using *modulo arithmetic*, where the result "wraps around" to stay within the range defined by n bits. This behavior ensures that computations remain bounded within the cyclic range $[0, 2^n - 1]$ and is fundamental in computer arithmetic, particularly for operations in finite fields, cryptography, and low-level programming.

The set of n -bit unsigned binary numbers with modulo arithmetic can be defined as:

$$B_n = \{x \in \mathbb{N} \mid x \equiv y \pmod{2^n}, y \in \{0, 1, \dots, 2^n - 1\}\}.$$

This equation defines the set B_n , which represents the set of n -bit unsigned binary numbers under modulo arithmetic:

- $x \in \mathbb{N}$: x is a natural number.
- $x \equiv y \pmod{2^n}$: x is congruent to y modulo 2^n .
 - ◊ In mathematical terms: $x - y$ is divisible by 2^n , or equivalently, $x \bmod 2^n = y$.
- $y \in \{0, 1, \dots, 2^n - 1\}$: y is within the valid range for n -bit unsigned binary numbers.

This representation ensures that any $x \in \mathbb{N}$ maps to an equivalent value y in the range $[0, 2^n - 1]$ using the modulo 2^n operation.

3.5.1 Overflow and Underflow Behavior

In this system, the arithmetic operations are defined modulo 2^n , which handles both *overflow* and *underflow* by wrapping around within the range $[0, 2^n - 1]$. For any two numbers $a, b \in B_n$:

- Addition: $(a + b) \bmod 2^n$
- Subtraction: $(a - b) \bmod 2^n$
- Multiplication: $(a \cdot b) \bmod 2^n$

For example, in a 3-bit system ($n = 3$):

$$B_3 = \{0, 1, 2, 3, 4, 5, 6, 7\}.$$

Overflow Example

If $a = 6$ and $b = 5$, then:

$$(a + b) \bmod 2^3 = (6 + 5) \bmod 8 = 11 \bmod 8 = 3.$$

Underflow Example

If $a = 2$ and $b = 5$, then:

$$(a - b) \bmod 2^3 = (2 - 5) \bmod 8 = -3 \bmod 8 = 5.$$

This modulo behavior ensures that results always wrap around into the valid range $[0, 2^n - 1]$, regardless of whether the computation results in overflow (values exceeding $2^n - 1$) or underflow (negative values).

3.5.2 Computer Arithmetic Example Overflow

In a 4-bit binary system, the maximum representable unsigned integer is 1111_2 (or 15 in decimal). If we add 1 to this value, it results in an overflow, as the sum exceeds the 4-bit limit and requires a 5th bit (carry bit) to fully represent the result. However, since we're limited to 4 bits, this overflow is often discarded, causing the result to wrap around to 0.

$$\begin{array}{r} \\ \\ + \\ \hline \text{Carry: } 1 \end{array}$$

Since the result exceeds 4 bits, we end up with a number that can't be represented with 4 bits:

$$10000_2 = 16_{10}$$

If we limit ourselves to only 4 bits, we discard the carry (the leftmost bit), and the result wraps around to:

$$0000_2 = 0_{10}$$

In systems that do not handle overflow, such operations may produce unintended results, particularly when only the lower 4 bits are retained, causing the system to interpret the result as 0.

3.5.3 Computer Arithmetic Example Underflow

In a 4-bit binary system, the minimum representable unsigned integer is 0000_2 (or 0 in decimal). If we subtract 1 from this value, it results in an underflow, as the difference falls below the representable range for unsigned numbers. In modulo arithmetic, this causes the result to wrap around to the maximum value for a 4-bit system.

$$\begin{array}{r} \\ \\ - \\ \hline \text{Borrow: } 1 \end{array}$$

The computation results in -1 , which cannot be represented in an unsigned 4-bit system. Instead, the value wraps around modulo $2^4 = 16$:

$$-1 \equiv 15 \pmod{16}$$

This means the result of $0000_2 - 1$ in a 4-bit system is:

$$1111_2 = 15_{10}$$

In systems that use modulo arithmetic, this behavior ensures that all results remain within the valid range $[0, 15]$. However, underflow can lead to unintended results if the system does not explicitly handle such cases or if signed arithmetic is required but unsigned arithmetic is used.

3.6 Definition of B_n with Clipping (Saturation Arithmetic)

In certain digital systems, particularly in Digital Signal Processors (DSPs), overflow and underflow are handled differently from the standard modulo arithmetic approach. Instead of wrapping around, values that exceed the representable range are clipped to the largest representable value. Values that are less than the representable range are clipped to 0. This behavior ensures that computations remain within bounds without introducing wraparound artifacts, which can be undesirable in signal processing applications. This is sometimes called saturation arithmetic or saturating arithmetic.

When clipping (saturation arithmetic) is used to handle overflow and underflow, the set B_n of n -bit unsigned binary numbers is defined as follows:

$$B_n = \begin{cases} x, & \text{if } 0 \leq x < 2^n, \\ 0, & \text{if } x < 0, \\ 2^n - 1, & \text{if } x \geq 2^n. \end{cases}$$

This definition ensures that:

- Values within the range $[0, 2^n - 1]$ remain unchanged.
- Values below 0 (underflow) are clipped to 0.
- Values above $2^n - 1$ (overflow) are clipped to $2^n - 1$.

3.6.1 Behavior of Arithmetic Operations with Clipping

The clipping operation effectively maps $x \geq 2^n$ to the maximum value $2^n - 1$, providing a bounded and monotonic representation of results. For any operation (addition, subtraction, multiplication) resulting in r :

$$r_{\text{clipped}} = \min(\max(r, 0), 2^n - 1).$$

Example for $n = 3$ ($B_3 = \{0, 1, 2, 3, 4, 5, 6, 7\}$):

- Addition: $r = 6 + 5 = 11$ exceeds 7 $r_{\text{clipped}} = 7$.
- Subtraction: $r = 2 - 4 = -2$ below 0 $r_{\text{clipped}} = 0$.
- Multiplication: $r = 3 \cdot 3 = 9$ exceeds 7 $r_{\text{clipped}} = 7$.

3.6.2 Computer Arithmetic Example: Overflow with Clipping

In a 4-bit binary system with clipping, the maximum representable unsigned integer is 1111_2 (or 15 in decimal). If we add 1 to this value, instead of wrapping around, the result is clipped to the maximum value 1111_2 .

$$\begin{array}{r} 1 1 1 1 \\ + 1 \\ \hline \text{Carry: } 1 0 0 0 0 \end{array}$$

Since the result exceeds the 4-bit limit, it is clipped to the maximum value:

$$\text{Clipped result: } 1111_2 = 15_{10}$$

In this system, clipping prevents the value from wrapping around to 0 and ensures that the result remains within the representable range $[0, 15]$.

3.6.3 Computer Arithmetic Example: Underflow with Clipping

In a 4-bit binary system with clipping, the minimum representable unsigned integer is 0000_2 (or 0 in decimal). If we subtract 1 from this value, instead of wrapping around to the maximum value, the result is clipped to the minimum value 0000_2 .

$$\begin{array}{r} \\ \\ \hline \text{Borrow: } 1 1 1 1 \end{array}$$

The computation results in -1 , which is not representable in an unsigned 4-bit system. Instead of wrapping around, the value is clipped to the minimum value:

$$\text{Clipped result: } 0000_2 = 0_{10}$$

Clipping ensures that results remain bounded within the valid range $[0, 15]$, preventing unintended wraparound behavior in cases of underflow.

Chapter 4

Finite Signed Binary Integer Representations in $\mathbb{Z}$

4.1	Signed-Magnitude Representation	40
4.1.1	Definition	40
4.1.2	Properties of Signed-Magnitude Representation	41
4.1.3	Arithmetic with Signed-Magnitude Representation	42
4.1.4	Trade-offs of Signed-Magnitude Representation	43
4.2	One's Complement Representation	43
4.2.1	Definition	44
4.2.2	Properties of One's Complement Representation	44
4.2.3	Arithmetic with One's Complement Representation	45
4.2.4	Trade-offs of One's Complement Representation	46
4.3	Two's Complement Representation	47
4.3.1	Definition	47
4.3.2	Properties of Two's Complement Representation	47
4.3.3	Arithmetic with Two's Complement Representation	49
4.3.4	Intermediate Overflow in 2's Complement Arithmetic	50
4.3.5	Trade-offs of Two's Complement Representation	52
4.4	Excess- k Representation	52
4.4.1	Definition	52
4.4.2	Properties of Excess- k Representation	53
4.4.3	Arithmetic with Excess- k Representation	54
4.4.4	Trade-offs of Excess- k Representation	57
4.5	Summary Table of Common Signed Number Representations	57
4.6	Binary Coded Decimal (BCD) Representation	59
4.6.1	Definition of BCD	59
4.6.2	Properties of Binary Coded Decimal (BCD)	59
4.6.3	Arithmetic using BCD	60
4.6.4	Trade-offs of BCD Representation	61
4.6.5	Applications of BCD	61
4.6.6	Example: Floating-Point vs. BCD in Financial Applications	62
4.7	Other Representations for Signed Numbers	63

Signed implementations implement a finite subset of $\mathbb{Z}$ and thus overlap operations with unsigned natural number $\mathbb{N}$. In a computer, the equivalent of placing a minus sign in front of a number led to the sign-magnitude representation. Other representations turn out to be more computationally efficient. We saw in Section 1.2.6 that a 10's complement representation turns a decimal subtraction into an addition.

We now generalize binary representations to include negative numbers. In unsigned binary systems, numbers are limited to non-negative values within the range $[0, 2^n - 1]$, where n is the number of bits. To represent negative numbers, several methods are employed, each with unique properties and use cases. These methods extend the binary representation system to include both positive and negative integers, enabling a broader range of applications such as arithmetic operations, signed comparisons, and signed data storage.

The most common methods for representing negative numbers in binary include:

- Signed-Magnitude Representation
- One's Complement
- Two's Complement
- Excess- k Representation

Each method modifies the interpretation of the most significant bit (MSB) to indicate the sign of the number, allowing the binary system to represent both positive and negative values. In the following sections, we explore each of these methods in detail, highlighting their advantages, disadvantages, and applications.

4.1 Signed-Magnitude Representation

The signed-magnitude representation is one of the simplest methods for representing negative numbers in binary. It extends the unsigned binary system by reserving the most significant bit (MSB) as a *sign bit*, while the remaining bits represent the magnitude of the number. This approach allows for both positive and negative integers to be encoded within the same binary system.

4.1.1 Definition

In an n -bit signed-magnitude system:

- The MSB (bit $n - 1$) indicates the sign:
 - 0: Positive number
 - 1: Negative number
- The remaining $n - 1$ bits represent the magnitude of the number, interpreted as an unsigned binary value.

Formally, a number x in signed-magnitude representation can be expressed as:

$$x = (-1)^s \cdot M,$$

where:

- s is the MSB (sign bit: 0 for positive, 1 for negative),
- M is the magnitude represented by the remaining $n - 1$ bits.

4.1.2 Properties of Signed-Magnitude Representation

The following are the mathematical properties of this representation:

1) Bit Allocation:

- For an n -bit signed-magnitude representation:
- The Most Significant Bit (MSB) is the *sign bit*, denoted s :

$$s = \begin{cases} 0, & \text{if the number is non-negative} \\ 1, & \text{if the number is negative.} \end{cases}$$

- The remaining $n - 1$ bits represent the *magnitude*, denoted m , where $m \in \{0, 1, \dots, 2^{n-1} - 1\}$.

2) Range:

- The representable range of numbers is:

$$-(2^{n-1} - 1) \leq x \leq 2^{n-1} - 1.$$

For example, in a 4-bit system:

$$x \in \{-7, -6, -5, -4, -3, -2, -1, +0, +1, +2, +3, +4, +5, +6, +7\}.$$

3) Dual Representation of Zero:

$$0000_2 \text{ (positive zero, } +0) \quad \text{and} \quad 1000_2 \text{ (negative zero, } -0).$$

4) Arithmetic Operations:

- Arithmetic operations must account for the separation of sign and magnitude:
- Addition: Requires determining the signs of both operands and performing magnitude-based addition or subtraction based on the signs.
- Subtraction: Requires determining the relative magnitudes of the operands and adjusting the sign of the result accordingly.

5) Symmetry:

- The representation is symmetric about zero. For any x with $s = 0$, the negative counterpart is obtained by setting $s = 1$ while keeping the same magnitude m .

6) Sign Bit Interpretation:

- The MSB (sign bit) does not contribute to the magnitude of the number but determines the polarity:

$$x = (-1)^s \cdot m.$$

7) Magnitude Interpretation:

- The magnitude is interpreted as a standard unsigned integer using the remaining $n - 1$ bits:

$$m = \sum_{i=0}^{n-2} b_i \cdot 2^i, \quad b_i \in \{0, 1\}.$$

For a 4-bit signed-magnitude representation:

$$x = (-1)^s \cdot m, \quad s = \text{MSB}, m = \text{binary value of remaining bits}.$$

Positive Numbers ($s = 0$)	Negative Numbers ($s = 1$)
0000 = 0	1000 = -0
0001 = 1	1001 = -1
0010 = 2	1010 = -2
0011 = 3	1011 = -3
0100 = 4	1100 = -4
0101 = 5	1101 = -5
0110 = 6	1110 = -6
0111 = 7	1111 = -7

Note that -0 is often treated as equivalent to $+0$

4.1.3 Arithmetic with Signed-Magnitude Representation

Addition Example (4-bit signed-magnitude):

$$3 + (-2)$$

In signed-magnitude:

$$3 = 0011, \quad -2 = 1010$$

Step-by-step addition:

- Determine the larger magnitude ($3 > 2$).
- Subtract the smaller magnitude from the larger:

$$3 - 2 = 1$$

- Assign the sign of the larger magnitude (+):

$$\text{Result: } 0001 = 1.$$

Subtraction Example (4-bit signed-magnitude):

$$-3 - 2$$

In signed-magnitude:

$$-3 = 1011, \quad 2 = 0010$$

Step-by-step subtraction:

- Determine the larger magnitude ($3 > 2$).
- Add the magnitudes:

$$3 + 2 = 5$$

- Assign the sign of the first operand ($-$):

$$\text{Result: } 1101 = -5.$$

4.1.4 Trade-offs of Signed-Magnitude Representation

Advantages:

- Simple to Understand: The use of a sign bit and magnitude closely mirrors the way humans write signed decimal numbers.
- Clear Separation of Sign and Magnitude: The MSB clearly indicates the sign, while the remaining bits represent the magnitude.

Disadvantages:

- Two Representations for Zero: Both 0000 (+0) and 1000 (−0) exist, which can lead to inconsistencies and inefficiencies.
- Complex Arithmetic: Arithmetic operations, such as addition and subtraction, require extra steps to manage the sign bit and may involve separate logic for handling positive and negative values.
- Limited Practical Use: Except for IEEE-754 floating point, signed-magnitude is rarely used in modern digital systems due to its inefficiencies.

4.2 One's Complement Representation

The one's complement representation is a method for representing signed integers in binary. It improves upon the signed-magnitude representation by eliminating the dual representation of zero, while still maintaining a straightforward method for negating numbers. In one's complement, negative numbers are represented by inverting (complementing) all the bits of their positive counterparts.

4.2.1 Definition

In an n -bit one's complement system:

- ****Positive Numbers****: Represented the same way as in unsigned binary, with the MSB (most significant bit) set to 0 for non-negative values.
- ****Negative Numbers****: Represented by inverting all the bits (one's complement) of the positive value.

Formally, for an integer x in one's complement representation:

$$x = \begin{cases} \text{Binary representation of } x, & \text{if } x \geq 0, \\ \sim (\text{Binary representation of } |x|), & \text{if } x < 0, \end{cases}$$

where $\sim$ represents the bitwise NOT operation (inversion of all bits).

4.2.2 Properties of One's Complement Representation

The following are the mathematical properties of this representation:

1) Bit Allocation:

- For an n -bit one's complement representation:
- The Most Significant Bit (MSB) is the *sign bit*, denoted s :

$$s = \begin{cases} 0, & \text{if the number is non-negative} \\ 1, & \text{if the number is negative.} \end{cases}$$

- The remaining $n - 1$ bits represent the *magnitude*, with negative numbers represented as the bitwise complement of their positive counterparts.

2) Range:

- The representable range of numbers is:

$$-(2^{n-1} - 1) \leq x \leq 2^{n-1} - 1.$$

For example, in a 4-bit system:

$$x \in \{-7, -6, -5, -4, -3, -2, -1, +0, -0, +1, +2, +3, +4, +5, +6, +7\}.$$

3) Dual Representation of Zero:

$$0000_2 \text{ (positive zero, } +0) \quad \text{and} \quad 1111_2 \text{ (negative zero, } -0).$$

4) Arithmetic Operations:

- Arithmetic operations use standard binary addition, with carries propagated as needed.
- Addition and subtraction are consistent for positive and negative numbers, but special care must be taken to handle the dual representation of zero.

5) Symmetry:

- The representation is symmetric about zero. For any x with $s = 0$, the negative counterpart is obtained by flipping all bits in the binary representation.

6) Sign Bit Interpretation:

- The MSB (sign bit) determines the polarity:

$$x = (-1)^s \cdot \text{Magnitude}.$$

7) Magnitude Interpretation:

- Negative numbers are interpreted as the bitwise complement of their positive counterpart.

$$\text{Magnitude} = \begin{cases} \text{Unsigned value of bits,} & \text{if } s = 0, \\ \text{Bitwise complement of unsigned value,} & \text{if } s = 1. \end{cases}$$

Example for a 4-bit one's complement representation:

$$x = (-1)^s \cdot \text{Magnitude}, \quad s = \text{MSB}.$$

Positive Numbers ($s = 0$)	Negative Numbers ($s = 1$)
0000 = +0	1111 = -0
0001 = +1	1110 = -1
0010 = +2	1101 = -2
0011 = +3	1100 = -3
0100 = +4	1011 = -4
0101 = +5	1010 = -5
0110 = +6	1001 = -6
0111 = +7	1000 = -7

Note that -0 is often treated as equivalent to $+0$.

4.2.3 Arithmetic with One's Complement Representation

Addition Example (4-bit one's complement):

$$3 + (-2)$$

In one's complement:

$$3 = 0011, \quad -2 = 1101$$

Step-by-step addition:

- Add the binary numbers:

$$0011 + 1101 = 10000$$

- Discard the carry bit:

$$\text{Result: } 0000 = +0 \text{ (or } -0\text{)}.$$

Subtraction Example (4-bit one's complement):

$$-3 - 2$$

In one's complement:

$$-3 = 1100, \quad 2 = 0010$$

Step-by-step subtraction:

- Add the first operand and the complement of the second operand:

$$1100 + 1101 = 11001$$

- Discard the carry bit:

$$\text{Result: } 1001 = -6.$$

4.2.4 Trade-offs of One's Complement Representation

Advantages:

- Simple Hardware: Arithmetic operations can be performed using standard binary addition and subtraction logic.
- Symmetry: The representation is symmetric, simplifying certain mathematical operations.

Disadvantages:

- Dual Representation of Zero: Both 0000 (+0) and 1111 (−0) exist, which can cause ambiguity.
- Carry Handling: Addition requires special handling for the end-around carry, complicating hardware design.
- Limited Practical Use: Due to its inefficiencies, one's complement is rarely used in modern systems, having been largely replaced by two's complement representation.

4.3 Two's Complement Representation

The two's complement representation is the most widely used method for representing signed integers in modern computing systems. It resolves the inefficiencies of signed-magnitude and one's complement, providing a simple and consistent way to represent both positive and negative integers, while ensuring efficient arithmetic operations. In two's complement, negative numbers are represented by the bitwise complement of their positive counterparts, plus one.

4.3.1 Definition

In an n -bit two's complement system:

- Positive Numbers: Represented the same way as in unsigned binary, with the most significant bit (MSB) set to 0.
- Negative Numbers: Represented by the two's complement of their positive counterpart, calculated as:

$$\text{Two's complement of } x = \sim x + 1,$$

where $\sim x$ denotes the bitwise complement (inversion) of x .

Formally, the value x in two's complement representation is:

$$x = \begin{cases} \text{Binary representation of } x, & \text{if } x \geq 0, \\ 2^n + x, & \text{if } x < 0. \end{cases}$$

This method simplifies arithmetic operations and eliminates the ambiguity of multiple representations for zero.

4.3.2 Properties of Two's Complement Representation

The following are the mathematical properties of this representation:

1) Bit Allocation:

- For an n -bit two's complement representation:
- The Most Significant Bit (MSB) is the *sign bit*, denoted s :

$$s = \begin{cases} 0, & \text{if the number is non-negative} \\ 1, & \text{if the number is negative.} \end{cases}$$

- The remaining $n - 1$ bits represent the magnitude, with negative numbers represented as the two's complement (bitwise complement plus 1) of their positive counterparts.

2) Range:

- The representable range of numbers is:

$$-2^{n-1} \leq x \leq 2^{n-1} - 1.$$

For example, in a 4-bit system:

$$x \in \{-8, -7, -6, -5, -4, -3, -2, -1, 0, +1, +2, +3, +4, +5, +6, +7\}.$$

3) Unique Representation of Zero:

$$0000_2 = 0.$$

4) Arithmetic Operations:

- Arithmetic operations use standard binary addition and subtraction, including the sign bit, without special handling for the sign.
- Overflow detection is necessary when performing operations near the boundaries of the representable range.

5) Symmetry:

- The representation is nearly symmetric, except for the most negative number (-2^{n-1}), which does not have a positive counterpart in the range.

6) Sign Bit Interpretation:

- The MSB (sign bit) determines the polarity and contributes to the magnitude:

$$x = -s \cdot 2^{n-1} + \sum_{i=0}^{n-2} b_i \cdot 2^i, \quad b_i \in \{0, 1\}.$$

7) Magnitude Interpretation:

- The magnitude of a negative number is obtained by taking its two's complement:

$$\text{Two's complement} = \text{Bitwise complement} + 1.$$

Example for a 4-bit two's complement representation:

$$x = -s \cdot 2^{n-1} + \text{Unsigned value of remaining bits}, \quad s = \text{MSB}.$$

Positive Numbers ($s = 0$)	Negative Numbers ($s = 1$)
0000 = 0	1000 = -8
0001 = 1	1001 = -7
0010 = 2	1010 = -6
0011 = 3	1011 = -5
0100 = 4	1100 = -4
0101 = 5	1101 = -3
0110 = 6	1110 = -2
0111 = 7	1111 = -1

Note that there is a unique representation for zero (0000).

4.3.3 Arithmetic with Two's Complement Representation

Addition Example (4-bit two's complement): $3 + (-2)$

1) Representation in 2's Complement

$$3 = 0011, \quad -2 = 1110$$

2) Addition

- Add the binary numbers

$$0011 + 1110 = 10001$$

- Discard the carry bit:

$$\text{Result: } 0001 = 1.$$

Subtraction Example (4-bit two's complement): $-3 - 2$

1) Representation in 2's Complement

$$-3 = 1101, \quad 2 = 0010$$

2) Subtraction

- Add the first operand and the two's complement of the second operand:

$$1101 + 1110 = 11011$$

- Discard the carry bit:

$$\text{Result: } 1011 = -5.$$

Multiplication 4-bit 2's Complement Multiplication: -3×2

1) Representation in 2's Complement:

- Decimal -3 : Represented in 4-bit 2's complement as 1101_2 .
- Decimal 2 : Represented in 4-bit 2's complement as 0010_2 .

2) Perform Unsigned Multiplication

- To preserve precision, it can be shown that multiplication require $2n$ bits for n – bit operands.
- Treat the binary numbers as unsigned values and multiply them. This requires extending to 8 bits to avoid truncation during intermediate calculations.

$$1101_2 \times 0010_2 = 11010_2.$$

3) Interpreting the Result in 2's Complement:

- Since this is a signed multiplication in 4-bit 2's complement, the result needs to fit within 4 bits, and overflow is possible.
- Truncate the result to the lower 4 bits: 1010_2 .
- 1010_2 in 4-bit 2's complement represents -6 (calculated as $-2^3 + 2^1 = -8 + 2 = -6$).

4.3.4 Intermediate Overflow in 2's Complement Arithmetic

In 2's complement arithmetic, it is permissible for intermediate results to overflow during a sequence of arithmetic operations, as long as the final result falls within the representable range. This is because 2's complement arithmetic operates modulo 2^n , where n is the number of bits, and modular arithmetic ensures correctness under these conditions.

In a system with n -bit 2's complement representation:

- Arithmetic operations (e.g., addition and subtraction) are performed modulo 2^n .
- Overflow occurs when intermediate results exceed the maximum representable value ($2^{n-1} - 1$) or go below the minimum representable value (-2^{n-1}).
- Intermediate overflow wraps around modulo 2^n , and as long as the final result is within the representable range, the computation is valid.

Example of Intermediate Overflow. Consider an 8-bit 2's complement system ($n = 8$), where the representable range is -128 to 127 .

1) Compute $100 + 50 - 90$:

a) Step 1: Compute $100 + 50$:

- Decimal representation: $100 + 50 = 150$, which exceeds the maximum representable value (127).
- Binary representation of 100: 01100100_2 .
- Binary representation of 50: 00110010_2 .
- Binary result (9-bit, showing overflow): 10010110_2 .
- Truncated to 8 bits: 10010110_2 , which represents -106 in 2's complement.

Intermediate result wraps around modulo 2^8 :

$$150 \bmod 256 = -106 \quad (\text{in 2's complement: } 150 - 256 = -106).$$

b) Step 2: Subtract 90:

- Decimal representation: $-106 - 90 = -196$, which overflows again.
- Binary representation of -106 : 10010110_2 .
- Binary representation of 90: 01011010_2 .
- Binary result (9-bit, showing overflow): 10111100_2 .
- Truncated to 8 bits: 00111100_2 , which represents 60 in 2's complement.

Final result wraps around modulo 2^8 :

$$-196 \bmod 256 = 60 \quad (\text{in 2's complement: } -196 + 256 = 60).$$

- 2) The true sum in $\mathbb{Z}$ is $100 + 50 - 90 = 60$, which is correctly represented within the range of 2's complement. The intermediate overflows did not affect the correctness of the final result due to modular arithmetic.

Key Property of 2's Complement Arithmetic. 2's complement arithmetic is closed under modular addition and subtraction, which ensures that the wrapping behavior of overflow does not affect the correctness of the final result. As long as the final sum or difference falls within the representable range, intermediate overflows are irrelevant.

- Intermediate overflow results in wrapping around, but this is consistent with the properties of 2's complement arithmetic.
- Binary truncation ensures that only the least significant 8 bits are retained, preserving correctness modulo 2^8 .
- The final result (60) lies within the representable range of -128 to 127 .

Practical Implications.

- **Hardware Arithmetic:** CPUs use fixed-width registers, allowing intermediate results to overflow without error detection unless explicitly checked.
- **Algorithms:** Algorithms relying on modular arithmetic, such as checksums or cryptographic methods, leverage this property for correctness.
- **Multiplication:** Intermediate overflow in multiplication may invalidate the result unless carefully managed, as modular behavior affects multiplication differently.

4.3.5 Trade-offs of Two's Complement Representation

Advantages:

- **Closed Under Addition and Subtraction:** ensures that the wrapping behavior of overflow does not affect the correctness of the final result.
- **Simplified Arithmetic:** Addition and subtraction use the same logic as unsigned integers, without requiring separate handling for the sign.
- **Unique Representation of Zero:** Avoids ambiguity and inefficiency caused by dual-zero representations.
- **Widely Used:** Two's complement is the standard representation for signed integers in modern computing systems.

Disadvantages:

- **Asymmetric Range:** The most negative number (-2^{n-1}) has no positive counterpart, which can lead to edge-case issues in arithmetic.
- **Overflow Detection:** Additional logic is needed to detect overflow in operations near the boundaries of the range.

4.4 Excess- k Representation

The excess- k (or biased) representation is a method for representing signed integers in binary by introducing a bias (offset) k to the actual value. This approach effectively maps both positive and negative integers to non-negative binary representations, simplifying certain arithmetic and logical operations. Excess- k is widely used in applications such as floating-point exponent representation, where a purely positive range of values is advantageous.

4.4.1 Definition

In an n -bit system using excess- k representation:

- $k = 2^{n-1}$ (the midpoint of the representable range) is the bias applied to the actual value.

- A signed integer x is encoded as:

$$\text{Excess-}k \text{ value} = x + k,$$

where x is the actual signed integer, and k is the bias.

The binary representation of the excess- k value corresponds to its unsigned binary encoding.

4.4.2 Properties of Excess- k Representation

The following are the mathematical properties of this representation:

1) Bit Allocation:

- For an n -bit excess- k representation, the value of k is:

$$k = 2^{n-1} - 1.$$

- The encoded value E is calculated as:

$$E = x + k, \quad \text{where } x \text{ is the actual signed integer.}$$

- All n bits represent the biased (offset) value, with no explicit sign bit.

2) Range:

- The representable range of numbers is:

$$-k \leq x \leq 2^n - 1 - k.$$

For example, in a 4-bit system ($k = 7$):

$$x \in \{-7, -6, -5, -4, -3, -2, -1, 0, +1, +2, +3, +4, +5, +6, +7\}.$$

3) Unique Representation of Zero:

$$E = k, \quad \text{or } 0111_2 \text{ in a 4-bit system.}$$

4) Arithmetic Operations:

- Arithmetic requires converting the biased value back to its true signed integer form:

$$x = E - k.$$

- After performing arithmetic on x , the result is re-encoded as:

$$E = x + k.$$

5) Symmetry:

- The representation is symmetric about zero, as the bias k ensures equal spacing for positive and negative values.

6) Interpretation:

- The value is interpreted as:

$$x = E - k, \quad E \in \{0, 1, \dots, 2^n - 1\}.$$

- The excess- k representation shifts the range of representable integers so that the minimum value corresponds to 0, and zero corresponds to k .

Example for a 4-bit excess-7 representation ($k = 7$):

$$x = E - 7, \quad E = \text{binary value.}$$

Encoded Value (E)	Actual Value (x)
0000 = 0	-7
0001 = 1	-6
0010 = 2	-5
0011 = 3	-4
0100 = 4	-3
0101 = 5	-2
0110 = 6	-1
0111 = 7	0
1000 = 8	+1
1001 = 9	+2
1010 = 10	+3
1011 = 11	+4
1100 = 12	+5
1101 = 13	+6
1110 = 14	+7
1111 = 15	+8

4.4.3 Arithmetic with Excess- k Representation

Comparisons One of the key advantages of excess- k representation is that it allows direct comparison of encoded values without needing to decode them into their actual signed values. This simplifies hardware design and improves computational efficiency for certain applications.

Example: Comparing -2 and 1 in Excess-7

$$E = x + k, \quad \text{where } x \text{ is the actual value.}$$

Step 1: Encode the values for -2 and 1:

$$E = -2 + 7 = 5, \quad \text{or } 0101_2.$$

$$E = 1 + 7 = 8, \quad \text{or } 1000_2.$$

Step 2: Compare the encoded values. In excess-7, the comparison of E values directly determines the result:

$$0101_2 < 1000_2 \Rightarrow -2 < 1.$$

There is no need to decode E back into x , as the ordering of E values mirrors the ordering of the actual values x . This property of excess- k representation makes it particularly useful in systems where frequent comparisons are required, such as in sorting algorithms or conditional logic in floating-point exponent comparisons. By avoiding decoding, the computational overhead is significantly reduced.

Addition In excess- k notation, addition can always be performed directly on the encoded values without converting to their actual values. This is because the bias k is consistent across all numbers, and the addition operation preserves this bias. Below is a detailed explanation of why this works.

In excess- k , a number x is encoded as:

$$E = x + k$$

where k is the bias.

When adding two numbers x_1 and x_2 :

1) Their encoded values are:

$$E_1 = x_1 + k, \quad E_2 = x_2 + k$$

2) Adding these encoded values directly:

$$E_{\text{sum}} = E_1 + E_2 = (x_1 + k) + (x_2 + k) = (x_1 + x_2) + 2k$$

3) The result is encoded with an excess of $2k$, not k . To interpret the result in excess- k , subtract k from the result:

$$E_{\text{corrected}} = E_{\text{sum}} - k = (x_1 + x_2) + k$$

Thus, the encoded values can be added directly, and the result will remain valid in the excess- k system after interpreting it correctly (i.e., accounting for the excess k).

Addition Example (4-bit excess-7):

$$3 + (-2)$$

In excess-7:

$$3 = 1010, \quad -2 = 0101$$

Step-by-step addition:

- Add the encoded values directly:

$$1010 + 0101 = 1111.$$

- Subtract the excess (7) to interpret the result:

$$1111 - 0111 = 1000.$$

- The result in excess-7 is 1000, corresponding to the actual value 1.

Subtraction Attempting to perform subtraction directly in excess-7 notation using binary arithmetic can lead to incorrect results due to the way the bias affects the encoded values. Subtraction is not symmetric in excess- k , as directly subtracting encoded values E_1 and E_2 includes the bias k in both values. When subtracting:

$$E_{\text{diff}} = E_1 - E_2 = (x_1 + k) - (x_2 + k) = (x_1 - x_2)$$

The bias cancels out, so the result is correct in terms of the actual value. However, handling negative results in binary may require re-encoding or adjustments to ensure the result stays within the bounds of the representation.

Assume (4-bit excess-7):

$$-3 - 2$$

In excess-7:

$$-3 = 0100, \quad 2 = 1001$$

Step-by-step subtraction:

- Compute the two's complement of 1001 (the encoded value of 2):

◊ Find the one's complement of 1001:

$$\text{One's complement of } 1001 = 0110$$

◊ Add 1 to obtain the two's complement:

$$0110 + 0001 = 0111$$

- Add the two's complement of 1001 to 0100:

$$0100 + 0111 = 1011$$

- Since there is no carry out, the result is 1011.
- Subtract the excess (0111) to interpret the result:

$$1011 - 0111 = 0100$$

- The result in excess-7 is 0100, corresponding to:

$$x = E - k = 0100_2 - 0111_2 = 4 - 7 = -3$$

- However, this does not match the expected result of -5 .

To accurately perform the subtraction $-3 - 2$ in excess-7 notation, it's necessary to convert the encoded values back to their actual decimal values:

4.4.4 Trade-offs of Excess- k Representation

Advantages:

- **Simplified Comparison:** Excess- k enables direct comparison of encoded values without decoding. Since the bias is uniform across all numbers, their relative magnitudes are preserved, allowing comparisons like $E_1 < E_2$ to directly reflect $x_1 < x_2$.
- **Simplified Addition:** Addition can be performed directly on the encoded values without decoding, as the bias k does not affect the sum. The result remains valid in excess- k form as long as the interpretation accounts for the bias.
- **Symmetry:** The representation ensures equal spacing between positive and negative values, making it suitable for balanced encoding systems.
- **Common in Floating Point:** Widely used in the exponent field of IEEE-754 floating-point numbers, where excess- k simplifies comparisons and operations on exponents.

Disadvantages:

- **Conversion Overhead for Subtraction:** Subtraction often requires decoding the values into their actual numerical equivalents to handle differences correctly, especially when the result is negative. Re-encoding is needed after the operation.
- **Ambiguity:** The value of k (the bias) must be known to interpret the representation correctly, which can introduce complexity in systems where multiple biases are used.
- **Limited Use in General Arithmetic:** Excess- k is not commonly used for general-purpose integer arithmetic because of its dependence on decoding and re-encoding for operations like subtraction.
- **Overflow/Underflow Handling:** Care must be taken to ensure that results of arithmetic operations stay within the valid range of encoded values, as excess- k systems do not naturally handle overflow or underflow.

4.5 Summary Table of Common Signed Number Representations

Table 4.1 summarizes the key properties of signed-magnitude, one's complement, two's complement, and excess- k representations:

Table 4.1: Comparison of signed number representations with fixed column widths and wrapped text.

Representation	Key Features	Range	Advantages/Disadvantages
Signed Magnitude	MSB indicates the sign (0 for positive, 1 for negative). Remaining bits represent magnitude. Dual representation of zero (+0, -0).	$-2^{n-1} + 1$ to $2^{n-1} - 1$	Simple to understand. Complex arithmetic due to sign handling. Inefficient for general computation.
One's Complement	MSB indicates the sign (0 for positive, 1 for negative). Negative numbers are bitwise complements of positive numbers. Negative zero exists but treated as equivalent to positive zero.	$-2^{n-1} + 1$ to $2^{n-1} - 1$	Simplified negation by bit inversion. Slightly improved arithmetic efficiency. Requires end-around carry for addition/subtraction. Negative zero can cause inefficiencies.
Two's Complement	MSB indicates the sign (0 for positive, 1 for negative). Negative numbers are represented as bitwise complements plus one. Single representation for zero.	-2^{n-1} to $2^{n-1} - 1$	Efficient arithmetic without special handling. Eliminates ambiguity of zero. Standard representation in modern computing. Asymmetric range (one extra negative value).
Excess-k	Encodes values by adding a bias k ($k = 2^{n-1}$). Values are shifted to ensure all are non-negative. Common in floating-point exponent representation.	-2^{n-1} to $2^{n-1} - 1$	Uniform binary range simplifies comparisons. Ideal for exponent encoding in floating-point systems. Requires bias adjustments for arithmetic. Less intuitive for human interpretation.

4.6 Binary Coded Decimal (BCD) Representation

Binary Coded Decimal (BCD) is a class of binary encoding for decimal numbers where each decimal digit is represented by a fixed number of binary bits, typically four. This encoding allows decimal digits to be stored and manipulated directly in a binary system, avoiding conversion between decimal and binary representations. BCD is widely used in financial applications, calculators, and digital systems where precision and human-readability of decimal values are critical (Hennessy and Patterson, 2019; Stallings, 2020).

4.6.1 Definition of BCD

In BCD, each decimal digit (0 through 9) is encoded as a 4-bit binary value. For example:

Decimal: 0 $\rightarrow$ BCD: 0000, Decimal: 9 $\rightarrow$ BCD: 1001.

A decimal number is represented by concatenating the BCD codes of its individual digits. For example:

Decimal: 47 $\rightarrow$ BCD: 0100 0111.

4.6.2 Properties of Binary Coded Decimal (BCD)

1) Bit Allocation

- Each decimal digit is represented by a 4-bit binary code.
- Invalid combinations (e.g., 1010 through 1111) are unused and may trigger errors in systems that strictly enforce BCD.

2) Range:

- Each 4-bit group represents a decimal digit from 0 to 9.
- For an n -digit decimal number, BCD requires $4n$ bits.
- Example: A 3-digit decimal number (0 to 999) requires 12 bits in BCD.

3) Unique Representation of Numbers:

- Each decimal digit has a unique 4-bit representation, ensuring that conversions between decimal and BCD are exact.

4) Arithmetic Operations:

- Addition and subtraction require special handling to ensure results remain in valid BCD.
- Overflow is corrected by adding 6 (binary 0110) to the result when a sum exceeds 9.

5) Human-Readable Decimal Encoding:

- BCD aligns closely with human-readable decimal notation, making it useful in systems requiring precise representation of decimal numbers, such as financial and accounting systems.

4.6.3 Arithmetic using BCD**Addition** Encoding Example:Decimal: 259 $\rightarrow$ BCD: 0010 0101 1001

Addition Example (in BCD):

$$7 + 8$$

Step-by-step addition:

- Add the binary representations:

$$0111 + 1000 = 1111 \text{ (invalid BCD code).}$$

- Correct the result by adding 0110:

$$1111 + 0110 = 0001\ 0101.$$

- The result is 15, encoded as 0001 0101 in BCD.

Subtraction We want to compute: $75 - 46 = 29$ using Binary Coded Decimal arithmetic.

BCD Representation:

- 75 in BCD: 0111 0101
- 46 in BCD: 0100 0110

Step-by-Step Subtraction:

1) Subtract Each Digit Individually:

- Perform the subtraction digit by digit, starting from the least significant digit (LSD):
- The subtraction in the LSD results in $5 - 6 = -1$, which is invalid in BCD.

$$\begin{array}{r} 0111\ 0101 \quad (75 \text{ in BCD}) \\ - 0100\ 0110 \quad (46 \text{ in BCD}) \\ \hline 0011\ -0001 \end{array}$$

2) Apply Borrow Correction:

- Since $5 - 6 = -1$, borrow 1 from the next higher digit.
- In BCD, borrowing subtracts 1 from the higher digit and adds 10 (binary 1010) to the lower digit.

Corrected subtraction:

$$\begin{array}{r} 0110\ 1111 \quad (\text{After Borrowing}) \\ - 0100\ 0110 \\ \hline 0010\ 1001 \end{array}$$

3) The final result in BCD is:

0010 1001 (BCD for 29)

4.6.4 Trade-offs of BCD Representation

Advantages:

- Precise representation of decimal numbers.
- Avoids errors due to floating-point approximations in decimal systems.
- Simplifies display formatting for human-readable outputs.

Disadvantages:

- Inefficient use of storage, as only 10 out of 16 possible 4-bit values are used.
- Arithmetic operations are slower compared to binary due to correction steps.
- Subtraction in BCD requires handling invalid results (e.g., negative digits) through borrow correction. By borrowing and adding 10 to the LSD, we ensure the result remains valid in BCD.
- Less common in modern general-purpose computing due to inefficiencies.

4.6.5 Applications of BCD

BCD is commonly used in applications requiring exact decimal representation and arithmetic, such as:

- Financial systems (e.g., accounting and banking software).
- Digital clocks and calculators.
- Embedded systems where human-readable output is necessary.

4.6.6 Example: Floating-Point vs. BCD in Financial Applications

This example shows why BCD is used in financial applications. A financial application needs to add \$1.10 and \$0.30, which should result in \$1.40. The values are represented in both binary floating-point and BCD to illustrate the differences in accuracy (Goldberg, 1991).

Floating-Point Representation: Floating-point uses binary fractions to approximate decimal values. We encode \$1.10 and \$0.30 as 32-bit IEEE 754 single-precision floating-point numbers.

1) Representation:

- For \$1.10: $1.10_{10} = 1.00011001100110011\dots_2$ (recurring in binary)
 - ◊ Approximated in floating-point as: $1.10 \approx 1.099999904632568359375$
- For \$0.30: $0.30_{10} = 0.010011001100110011\dots_2$ (recurring in binary)
 - ◊ Approximated in floating-point as: $0.30 \approx 0.2999999821186065673828125$

2) Add approximations

$$1.099999904632568359375 + 0.2999999821186065673828125 = 1.3999998867511749267578125$$

3) The floating-point result is approximately:

\$1.3999999 (not exactly \$1.40).

BCD Representation: In BCD, each decimal digit is encoded precisely without approximation. We encode \$1.10 and \$0.30 in BCD.

1) Representation:

- a) For \$1.10: BCD: 0001 0001 0000
- b) For \$0.30: BCD: 0000 0011 0000

2) Addition:

- Adding the BCD values:

$$\begin{array}{r}
 0001 \ 0001 \ 0000 \ (\$1.10) \\
 + \ 0000 \ 0011 \ 0000 \ (\$0.30) \\
 \hline
 0001 \ 0100 \ 0000 \ (\$1.40)
 \end{array}$$

3) The BCD result is exactly:

\$1.40 (no rounding error).

This example highlights why BCD is preferred in financial applications. It guarantees exact decimal arithmetic, avoiding the rounding errors and inaccuracies inherent in binary floating-point representation. While floating-point is more efficient for general-purpose computation, its limitations make it unsuitable for contexts like financial systems, where precision is critical.

Floating-Point:

- Binary fractions cannot precisely represent many decimal values (e.g., 0.1, 0.3).
- Arithmetic introduces small inaccuracies, such as \$1.3999999 instead of \$1.40.
- Errors accumulate over multiple operations, making floating-point unsuitable for precise financial calculations.

BCD:

- Decimal digits are encoded exactly, avoiding conversion errors.
- Arithmetic is performed directly on decimal representations, ensuring precision.
- The result is always exact for decimal numbers, as seen with the correct sum of \$1.40.

4.7 Other Representations for Signed Numbers

In addition to the widely known methods of signed-magnitude, one's complement, two's complement, and excess- k , other binary representations for signed numbers have been used historically or in specialized systems. These alternative representations often address specific challenges, such as reducing error, improving computational efficiency, or handling broader dynamic ranges.

Sign-and-Logarithm Representation: Numbers are represented using a sign bit and the logarithm of their absolute value. This method is particularly useful for encoding a wide dynamic range of values with fewer bits. Sign-and-logarithm representations have been used in early scientific computations and audio processing systems where efficiency in representation outweighed complexity in arithmetic. However, addition and subtraction operations are computationally expensive, as they require converting numbers back to linear scale. This method is conceptually similar to floating-point representations but lacks their generality (Hamming, 1980).

Gray Code: Gray code is a binary numeral system where consecutive numbers differ by only one bit. Although Gray code is typically used in signal encoding and rotary encoders, it has been adapted for signed number representations in niche applications. This representation reduces errors in digital communication systems by minimizing bit transitions. However, arithmetic operations in Gray code are nontrivial and require conversion to standard binary (Gray, 1953).

Offset Binary (Biased Binary:) Offset binary, also known as biased binary, is similar to excess- k but applies a fixed bias directly to the binary representation of both positive and negative numbers. This representation is often used in digital signal processing (DSP) systems to encode signals symmetrically around zero. Applications include Analog-to-Digital Converters (ADCs) and Digital-to-Analog Convert-

ers (DACs). The primary drawback is that arithmetic operations require normalization to adjust for the bias, similar to excess- k (Oppenheim and Schaffer, 2014).

Base-2 Signed-Digit (SD) Representation: A signed-digit representation allows each digit to independently represent a value of -1 , 0 , or $+1$. This system enables carry-free addition, making it highly efficient for certain high-speed arithmetic systems and redundant number systems. However, it requires special encoding and decoding processes, which complicates implementation. This method is primarily used in specific high-performance computing applications (Koren, 2002).

Balanced Ternary: Balanced ternary is a non-binary numeral system that uses three digits: -1 , 0 , and $+1$. While not strictly binary, it has been considered for signed representations in early computing systems. One notable example is the Soviet Setun computer, which used balanced ternary due to its efficiency in certain calculations. However, its lack of compatibility with modern binary-based digital systems limits its applicability today (Lukyanov, 1961).

Segmented Representations: In segmented representations, numbers are divided into multiple segments, with some segments used to encode the sign and others for magnitude or auxiliary components. These representations are rarely used in general-purpose systems but have been employed in specialized hardware architectures for non-standard data types. Segmented representations are complex to decode and implement for arithmetic operations, which has limited their adoption (Smith and Sohi, 1991).

Residue Number System (RNS): The residue number system represents numbers as a set of residues modulo several pairwise coprime integers. This system supports signed numbers by choosing appropriate moduli. RNS is advantageous for high-speed computation in specialized domains such as cryptography and signal processing. While addition, subtraction, and multiplication are efficient in RNS, comparison and division operations are significantly more complex, which limits its use in general computing (Heydari and Abdel-Hamid, 2016).

Summary: While signed-magnitude, one's complement, two's complement, and excess- k dominate practical use cases for signed number representations, alternative methods like sign-and-logarithm, Gray code, and residue number systems have found applications in specialized domains. These representations are often chosen to address specific challenges, such as reducing error, enhancing computational speed, or simplifying logical comparisons. However, their complexity and limited hardware compatibility have restricted their adoption in general-purpose computing.

Chapter 5

Finite Real Number Representations in $\mathbb{R}$

5.1	Definition and Characteristics	66
5.2	Floating Point Representation of Real Numbers	66
5.2.1	General Floating-Point Format	66
5.2.2	Rounding in Floating-Point Arithmetic	67
5.2.3	The Role of Denormalized Numbers in Floating-Point Arithmetic	68
5.2.4	IEEE Supported Formats	71
5.2.5	Half-Precision Floating-Point (f16)	72
5.2.6	Example: Representing 6.25 in f16	73
5.2.7	Applications and Trade-Offs	73
5.3	Fixed-Point Representation	73
5.3.1	Structure of Fixed-Point Numbers	74
5.3.2	Range of Signed Fixed-Point Representation	74
5.3.3	Properties of Fixed-Point Arithmetic	74
5.3.4	Fixed Point Considerations	77
5.3.5	Comparison with Floating-Point Representation	77
5.4	Examples of Finite Real Numbers	78
5.5	Challenges in Finite Real Representations	78
5.6	Alternative Finite Number Representations.	79

The set of real numbers, denoted $\mathbb{R}$, includes all rational and irrational numbers and forms a continuous, uncountable set. While theoretically infinite, practical computations in digital systems can only represent a finite subset of $\mathbb{R}$. Finite representations of real numbers are constrained by the limitations of hardware and software, leading to approximations in many cases.

5.1 Definition and Characteristics

Finite real numbers are those real numbers that can be precisely represented within a given computational framework or mathematical system. For instance, within the context of floating-point arithmetic, a finite real number must satisfy:

$$x \in \mathbb{R} \quad \text{and} \quad \text{finite precision representation exists.}$$

Unlike integers, which are discrete, real numbers encompass rational values (e.g., $\frac{1}{2}$, 0.75) and irrational values (e.g., $\sqrt{2}$, π). Finite real numbers are typically confined to the subset of $\mathbb{R}$ that can be approximated by rational representations due to hardware precision constraints.

In computation, real numbers are often represented using floating-point or fixed-point formats:

5.2 Floating Point Representation of Real Numbers

The IEEE 754 standard defines floating-point formats, which provide an efficient and scalable way to approximate real numbers using a finite number of bits. These formats are widely used in scientific computations, graphics, and machine learning due to their ability to balance precision, range, and efficiency.

5.2.1 General Floating-Point Format

A floating-point number under the IEEE 754 standard (IEEE, 2008) is expressed as:

$$x = (-1)^s \cdot m \cdot 2^e,$$

where:

- s : The sign bit, which determines whether the number is positive ($s = 0$) or negative ($s = 1$).
- m : The significand (or mantissa), which represents the precision of the number. It is normalized to the form $1.f$, where f is the fractional part in binary.
- e : The exponent, which determines the scale or range of the number. The exponent is stored in a biased format to allow both positive and negative exponents.

For example, the decimal number 6.25 is represented as:

$$6.25 = 1.5625 \cdot 2^2 \quad (\text{binary: } 1.101 \cdot 2^2).$$

In a floating-point format, $s = 0$, $m = 1.101$, and $e = 2$ (adjusted with the exponent bias).

5.2.2 Rounding in Floating-Point Arithmetic

Finite precision requires approximating real numbers ($\mathbb{R}$) with a subset of representable values. Floating-point numbers have a finite number of bits, limiting the precision with which real numbers can be represented. When a computation produces a result that cannot be represented exactly (e.g., $1/3$ in binary), rounding is applied to map the result to the nearest representable value.

Without consistent rounding rules, numerical computations could yield unpredictable results, especially in iterative algorithms or when combining multiple operations. The IEEE 754 standard specifies several rounding modes to handle this approximation, each designed to suit specific computational needs and numerical properties.

IEEE 754 Rounding Modes. The IEEE 754 standard defines the following five rounding modes:

- 1) Round to Nearest, Ties to Even (Default):
 - The value is rounded to the nearest representable number.
 - If the result is exactly halfway between two representable values, it is rounded to the nearest even value.
 - Use Case: This is the default rounding mode and minimizes overall rounding error in repeated computations.
- 2) Round to Nearest, Ties Away from Zero:
 - The value is rounded to the nearest representable number.
 - If the result is exactly halfway between two representable values, it is rounded away from zero.
 - Use Case: Useful in financial applications where avoiding bias toward zero is important.
- 3) Round Toward Zero (Truncation):
 - The fractional part is discarded, effectively truncating the value toward zero.
 - Use Case: Often used in integer arithmetic conversions to avoid exceeding bounds.
- 4) Round Toward Positive Infinity ($+\infty$):
 - The value is rounded up to the nearest representable number.
 - Use Case: Useful in applications requiring upper bounds, such as interval arithmetic or error estimation.
- 5) Round Toward Negative Infinity ($-\infty$):
 - The value is rounded down to the nearest representable number.

- Use Case: Useful in applications requiring lower bounds or ensuring conservative estimates.

Examples of Rounding Modes. Consider rounding the number $x = 2.625$ in binary (10.101_2) to 3 bits of precision in the fractional part.

- 1) Round to Nearest, Ties to Even: x rounds to 10.10_2 (2.5 in decimal) because it is closer to 10.10_2 than 10.11_2 .
- 2) Round to Nearest, Ties Away from Zero: x rounds to 10.11_2 (2.75 in decimal).
- 3) Round Toward Zero: x rounds to 10.10_2 (2.5 in decimal) by truncating the fractional part.
- 4) Round Toward Positive Infinity: x rounds to 10.11_2 (2.75 in decimal).
- 5) Round Toward Negative Infinity: x rounds to 10.10_2 (2.5 in decimal).

Summary of Rounding Modes. Table 5.1 shows the IEEE-754 rounding modes.

Table 5.1: IEEE 754 Rounding Modes

Rounding Mode	Description and Use Case
Round to Nearest, Ties to Even	Rounds to the nearest representable number; ties go to the nearest even number. Default mode for minimizing bias.
Round to Nearest, Ties Away from Zero	Rounds to the nearest representable number; ties go away from zero. Used in financial and accounting applications.
Round Toward Zero	Discards the fractional part, effectively truncating the value. Useful in integer conversions.
Round Toward Positive Infinity ($+\infty$)	Rounds up to the nearest representable number. Useful for bounding errors or interval arithmetic.
Round Toward Negative Infinity ($-\infty$)	Rounds down to the nearest representable number. Useful for conservative lower bounds.

5.2.3 The Role of Denormalized Numbers in Floating-Point Arithmetic

Floating-point systems, such as those defined by the IEEE 754 standard, provide an approximation of the real number line ($\mathbb{R}$) within a finite precision. This approximation forms a discrete metric space, where the representable numbers are distributed non-uniformly. Denormalized numbers (also called subnormal numbers) are introduced to address the issue of spacing near zero, improving the system's ability to approximate $\mathbb{R}$.

Discrete Spacing in Standard Floating-Point. In normalized floating-point representation, the spacing between consecutive representable numbers is determined by the precision (significand bits) and the magnitude of the exponent. For a normalized floating-point number:

$$x = (-1)^s \cdot (1.f) \cdot 2^e,$$

The discrete spacing between successive numbers doubles as the magnitude of x increases. For example, near zero (for small exponents), the numbers are closely packed, but as e grows, the spacing becomes increasingly larger. This doubling of spacing is a direct result of the binary scaling factor 2^e applied in the IEEE 754 standard. This spacing leaves a gap near zero, creating a discontinuity in the metric space.

Consider a simplified 4-bit floating-point system with the following structure:

- Sign bit (s): 1 bit.
- Exponent (e): 2 bits, biased by 1 (bias = 1).
- Fraction (mantissa) (f): 1 bit.

The value of a normalized number is given by:

$$x = (-1)^s \cdot (1.f) \cdot 2^{e-\text{bias}}.$$

In this system:

- The exponent range is $e \in [0, 3]$, corresponding to actual exponents $e - 1 \in [-1, 2]$.
- The significand range is 1.0 or 1.5 (depending on f).

We now show how the spacing changes. For each exponent value, the spacing doubles as e increases:

- Exponent $e = 0$ (2^{-1}):

$$x \in \{1.0 \cdot 2^{-1} = 0.5, 1.5 \cdot 2^{-1} = 0.75\}.$$

Spacing: $0.75 - 0.5 = 0.25$.

- Exponent $e = 1$ (2^0):

$$x \in \{1.0 \cdot 2^0 = 1.0, 1.5 \cdot 2^0 = 1.5\}.$$

Spacing: $1.5 - 1.0 = 0.5$.

- Exponent $e = 2$ (2^1):

$$x \in \{1.0 \cdot 2^1 = 2.0, 1.5 \cdot 2^1 = 3.0\}.$$

Spacing: $3.0 - 2.0 = 1.0$.

As the exponent increases, the spacing between consecutive representable numbers doubles:

- For $e = 0$, spacing is 0.25.
- For $e = 1$, spacing is 0.5.
- For $e = 2$, spacing is 1.0.

This behavior reflects the structure of normalized floating-point numbers, where the range of the significand remains fixed, but the scaling factor 2^e determines the magnitude and spacing.

The doubling of spacing as the exponent increases reduces the precision of floating-point representation for larger magnitudes. While this allows for a wide dynamic range, it introduces rounding errors in computations involving large and small numbers.

Limitations Near Zero. When the exponent e reaches its minimum value (e.g., $e_{\min} = -126$ for single precision), the smallest normalized floating-point number is:

$$x_{\min} = 1.0 \cdot 2^{e_{\min}}.$$

Numbers smaller than $x_{\min}$ cannot be represented in the normalized format, leaving a gap between 0 and $x_{\min}$. This discontinuity can cause issues in numerical computations, such as abrupt underflow to zero, which leads to a loss of precision in operations involving small values.

Denormalized Numbers Improve the Metric Space. Denormalized numbers are introduced to fill the gap near zero by allowing a leading significand bit of 0, instead of the implicit 1 required for normalized numbers. A denormalized number is represented as:

$$x = (-1)^s \cdot (0.f) \cdot 2^{e_{\min}}.$$

This representation permits a gradual decrease in the magnitude of x as the fractional part f decreases. For single precision, the range of denormalized numbers is:

$$x \in [2^{-149}, 2^{-126}].$$

By introducing smaller spacings near zero, denormalized numbers extend the metric space of floating-point numbers, enabling a more continuous approximation of $\mathbb{R}$.

Consider a simplified 4-bit floating-point system with:

- Sign bit (s): 1 bit.
- Exponent (e): 2 bits, biased by 1 (bias = 1).
- Fraction (mantissa) (f): 1 bit.

In this system, denormalized numbers allow the leading bit of the significand to be 0, enabling the representation of smaller values close to zero. The value of a denormalized number is:

$$x = (-1)^s \cdot (0.f) \cdot 2^{e_{\min}}.$$

Representation of Denormalized Numbers For $e = 0$ (the smallest exponent, used for denormalized numbers), the representable values are:

- Fraction $f = 0.0$: $x = 0.0 \cdot 2^{-1} = 0$.
- Fraction $f = 0.25$: $x = 0.25 \cdot 2^{-1} = 0.125$.
- Fraction $f = 0.5$: $x = 0.5 \cdot 2^{-1} = 0.25$.

- Fraction $f = 0.75$: $x = 0.75 \cdot 2^{-1} = 0.375$.

The spacing between consecutive denormalized numbers remains constant:

- Between 0.0 and 0.125: Spacing is 0.125.
- Between 0.125 and 0.25: Spacing is 0.125.
- Between 0.25 and 0.375: Spacing is 0.125.

Table 5.2 shows the denormalized spacing for IEEE formats. The inclusion of denormalized numbers affects the spacing within the floating-point metric space:

- Better Approximation Near Zero: Denormalized numbers significantly improve the precision of calculations involving very small values, reducing the abrupt transition to zero caused by underflow.
- Smooth Gradation: The gradual decrease in representable values leads to smoother behavior in numerical algorithms, particularly in iterative methods that rely on small updates.
- Support for Gradual Underflow: Instead of truncating to zero, operations involving small values can produce denormalized results, preserving information that would otherwise be lost.

Table 5.2: Denormalized Spacing in IEEE 754 Floating-Point Formats

Format	Minimum Denormalized Value	Next Larger Denormalized Value	Spacing
Half-Precision (f16)	$2^{-24} \approx 5.96 \times 10^{-8}$	$2^{-23} \approx 1.19 \times 10^{-7}$	2^{-24}
Single-Precision (f32)	$2^{-149} \approx 1.4 \times 10^{-45}$	$2^{-148} \approx 2.8 \times 10^{-45}$	2^{-149}
Double-Precision (f64)	$2^{-1074} \approx 4.9 \times 10^{-324}$	$2^{-1073} \approx 9.9 \times 10^{-324}$	2^{-1074}
Quadruple-Precision (f128)	$2^{-16494} \approx 6.5 \times 10^{-4966}$	$2^{-16493} \approx 1.3 \times 10^{-4965}$	2^{-16494}

Consider a single-precision system ($e_{\min} = -126$):

- Without Denormalized Numbers: The smallest representable positive number is 2^{-126} , with a large gap to zero.
- With Denormalized Numbers: Values like $2^{-127}, 2^{-128}, \dots, 2^{-149}$ are introduced, reducing the spacing and providing a closer approximation to $\mathbb{R}$.

5.2.4 IEEE Supported Formats

IEEE-754 and later editions support many different formats. We look at f16 in detail due to the ease of displaying bit widths. However, all formats work similarly. 5.3 shows the IEEE format definitions. Note that Decimal representations use Densely Packed Decimal (DPD) representations which require $n \times \frac{10}{3}$ bits for n -digits rather than standard BCD which would require $n \times 4$ bits per digit.

Table 5.3: Summary of IEEE 754 Supported Floating-Point Precisions

Format	Total Bits	Exponent Bits	Significand Bits	Applications
Half-Precision (f16)	16	5	10	Graphics, machine learning
Single-Precision (f32)	32	8	23	Scientific computing, gaming
Double-Precision (f64)	64	11	52	High-precision engineering, financial work
Quadruple-Precision (f128)	128	15	112	Cryptography, scientific research
Decimal32	32	8	7 digits	Financial applications
Decimal64	64	11	16 digits	Financial applications
Decimal128	128	15	34 digits	Precision-critical financial applications

5.2.5 Half-Precision Floating-Point (f16)

The IEEE 754 half-precision (f16) format is a compact 16-bit representation, commonly used in graphics, machine learning, and resource-constrained environments. It is defined as follows:

- Bit Layout: A 16-bit number is divided into three components:

Sign (1 bit) | Exponent (5 bits) | Fraction (10 bits)

- Sign Bit (s): The first bit determines the sign of the number (0 for positive, 1 for negative).
- Exponent (e): The next 5 bits represent the exponent in a biased format. The bias for f16 is $2^{k-1} - 1 = 15$ (where $k = 5$).
- Fraction (f): The final 10 bits store the fractional part of the significand. The leading bit of the significand is implicitly 1 for normalized numbers.

Value Calculation: The value of an f16 floating-point number is calculated as:

$$x = (-1)^s \cdot (1.f) \cdot 2^{e-15},$$

where s is the sign bit, f is the fraction, and e is the biased exponent.

Special Cases in f16:

- Normalized Numbers: For $1 \leq e \leq 30$, the number is normalized, and the leading bit of the significand is 1. The value is:

$$x = (-1)^s \cdot (1.f) \cdot 2^{e-15}.$$

- **Denormalized Numbers:** For $e = 0$, the number is denormalized (used to represent values close to zero). The leading bit of the significand is 0, and the value is:

$$x = (-1)^s \cdot (0.f) \cdot 2^{-14}.$$

- **Zero:** When $e = 0$ and $f = 0$, the value is $x = 0$ (with a sign determined by s).
- **Infinity:** When $e = 31$ and $f = 0$, the value is $x = \pm\infty$ (depending on the sign bit s).
- **NaN (Not a Number):** When $e = 31$ and $f \neq 0$, the value is NaN, used for undefined operations (e.g., $0/0$).

5.2.6 Example: Representing 6.25 in f16

To represent 6.25 in f16:

- 1) Convert 6.25 to binary: $6.25 = 1.101 \cdot 2^2$.
- 2) Determine the components:
 - $s = 0$ (positive number).
 - $e = 2 + 15 = 17$ (add the bias of 15).
 - $f = 1010000000_2$ (fractional part padded to 10 bits).

- 3) Write the bit layout:

$$0 \mid 10001 \mid 1010000000.$$

- 4) Verify:

$$x = (-1)^0 \cdot (1.101) \cdot 2^{17-15} = 6.25.$$

5.2.7 Applications and Trade-Offs

The f16 format is widely used in applications where memory and computational efficiency are important:

- **Graphics:** In rendering pipelines, f16 provides sufficient precision for colors, lighting, and textures.
- **Machine Learning:** Many AI accelerators and frameworks (e.g., TensorFlow, PyTorch) use half-precision floating-point formats like IEEE f16 or bfloat16 for faster training and inference.
- **Embedded Systems:** The compact size of f16 makes it ideal for low-power devices with limited memory.

However, the limited precision of f16 can lead to significant rounding errors in calculations, particularly for values requiring high precision or a large dynamic range (Goldberg, 1991; IEEE, 2008).

5.3 Fixed-Point Representation

Fixed-point representation is a method for storing and processing numbers by dividing a fixed number of bits into integer and fractional parts. This approach enables efficient computations, particularly in

resource-constrained environments such as embedded systems, digital signal processing (DSP), and real-time control systems (Lathi, 2005).

Compared to floating-point representation, fixed-point arithmetic offers simplicity, lower computational overhead, and predictable precision. However, it comes with limitations in dynamic range and precision.

5.3.1 Structure of Fixed-Point Numbers

In fixed-point representation, a number is divided into two parts:

- Integer Part: Specifies the whole number portion.
- Fractional Part: Specifies the fractional portion using a fixed scaling factor.

The scaling factor 2^q determines how the binary digits in the fractional part are interpreted. For an n -bit representation with p bits for the integer part and q bits for the fractional part:

$$\text{Value} = (-1)^s \cdot \left(\text{Integer Part} + \frac{\text{Fractional Part}}{2^q} \right),$$

where s is the sign bit for signed numbers.

For example, in an 8-bit fixed-point format with 4 integer bits and 4 fractional bits ($q = 4$):

- 6.25 is represented as 0110.0100₂, with 0110₂ as the integer part and 0100₂ as the fractional part.
- The scaling factor is $2^4 = 16$, so 6.25 is stored as $6 + \frac{4}{16} = 6.25$.

5.3.2 Range of Signed Fixed-Point Representation

Signed fixed-point numbers include a sign bit to represent negative values. The sign bit occupies the most significant bit (MSB), and the remaining bits are divided into integer and fractional parts. Negative numbers are typically represented using two's complement because it allows the same hardware to be used as with 2's complement integers.

For example, in an 8-bit signed fixed-point system with 4 fractional bits:

- 6.25: 0110.0100₂ (same as the unsigned case).
- -6.25: Two's complement of 0110.0100₂ is 1001.1100₂.

This system supports a range of values:

$$\text{Range} = \left[-2^{p-1}, 2^{p-1} - \frac{1}{2^q} \right].$$

For the 8-bit system with $p = 4$ and $q = 4$, the range is:

$$[-8.000, +7.9375].$$

5.3.3 Properties of Fixed-Point Arithmetic

Fixed-point arithmetic has distinct properties that make it well-suited for certain applications. These properties arise from its representation, where numbers are stored as integers with an implied binary point.

Fixed Precision and Scaling. Fixed-point arithmetic divides a number into two parts: an integer part $X \in \mathbb{Z}$ and a fractional part, allowing it to approximate real numbers $x \in \mathbb{R}$. A predetermined scaling factor 2^q , where q is the number of fractional bits, determines the granularity of the representation. Each fixed-point number is expressed as:

$$x = \frac{X}{2^q}, \quad \text{where } X \in \mathbb{Z} \text{ and } x \in \mathbb{R}.$$

This representation allows precise mapping between integer values and their scaled equivalents in the real number domain:

- The integer part determines the range of representable values.
- The fractional part determines the smallest difference (2^{-q}) between two consecutive representable values, ensuring a fixed level of precision.
- Precision remains constant for all values within the representable range, providing predictable granularity.
- Numbers that cannot be exactly represented within the fixed precision must be rounded or truncated, introducing small errors in computation.

Deterministic Arithmetic. Fixed-point arithmetic performs operations deterministically:

- **Shift Operations:** A left shift multiplies the number by 2, while a right shift divides it by 2, effectively adjusting the scaling factor.
- **Addition and Subtraction:** Operations are performed like integer arithmetic, and the results remain within the fixed-point format as long as overflow does not occur.
- **Multiplication:** The product of two fixed-point numbers requires adjustment to account for the implied binary point. For two numbers a and b with q fractional bits:

$$a \cdot b = \frac{A \cdot B}{2^q},$$

where A and B are the integer representations of a and b .

Example of 6.25×0.5 with $q = 4$ fractional bits,

- 1) Encode the operands in fixed point

$$6.25_{10} = 0110.0100_2 \quad \text{and} \quad 0.5_{10} = 0000.1000_2$$

- 2) Perform 2's complement multiplication:

$$01100100_2 \cdot 00001000_2 = 0000001100100000_2.$$

- 3) Shift the result right by $q = 4$ bits and retain the lower $n = 8$ bits:

$$0000001100100000_2 \rightarrow 000000110010xxxx_2 \rightarrow 00110010_2$$

- 4) Interpret the number with radix point at $q = 4$:

$$0011.0010_2$$

This corresponds to 3.125 in decimal.

The same example with ($q = 3$)

- 1) Encode the operands in fixed-point representation: Represent each operand with $q = 3$ fractional bits:

$$6.25_{10} = 00110.010_2 \quad \text{and} \quad 0.5_{10} = 00000.100_2.$$

- 2) Perform 2's complement multiplication:

$$00110010_2 \cdot 00000.100_2 = 0000000011001000_2.$$

- 3) Shift the result right by $q = 3$ bits and retain the lower $n = 8$ bits:

$$0000000011001000_2 \rightarrow 0000000011001_{xxx}_2 \rightarrow 00011001_2$$

- 4) Interpret the number with radix point at $q = 3$:

$$00011.001_2$$

This corresponds to 3.125 in decimal.

- **Division:** Division requires rescaling to maintain precision, often introducing rounding errors:

$$a/b = \frac{A \cdot 2^q}{B}.$$

Overflow and Underflow Behavior. Unlike floating-point arithmetic, fixed-point numbers cannot dynamically adjust their range, resulting in strict overflow and underflow behavior:

- **Overflow:** Values exceeding the maximum representable range wrap around (in unsigned systems) or saturate (in signed systems with clipping).

For example, in a signed 8-bit system with 4 fractional bits:

+8.0 is unrepresentable and results in overflow.

- **Underflow:** Values closer to zero than the smallest representable value are rounded to zero, losing precision.

For $q = 4$, the smallest representable nonzero value is $\pm \frac{1}{16}$.

Rounding and Truncation. Fixed-point arithmetic introduces rounding or truncation when numbers cannot be exactly represented. For example, 0.1 in decimal is approximately 0000.1100_2 in binary, resulting in a small rounding error. Common methods to deal with this include:

- Truncation: Simply removes excess fractional bits, leading to predictable but potentially biased errors.
- Rounding: Rounds to the nearest representable value, reducing bias but requiring additional computation.

5.3.4 Fixed Point Considerations

Trade-offs Between Precision and Range. Fixed-point representation inherently balances precision and range:

- Increasing the number of fractional bits (q) improves precision but reduces the representable range.
- Increasing the number of integer bits (p) extends the range but reduces precision.

For example a 16-bit fixed-point number with 12 fractional bits has a higher precision but a smaller range than one with 8 fractional bits.

Efficient Hardware Implementation. Fixed-point arithmetic often uses the same hardware as two's complement integer arithmetic, with the distinction lying primarily in how the bits are interpreted. This close relationship allows fixed-point arithmetic to reuse existing hardware while supporting fractional precision. Arithmetic operations such as addition, subtraction, and shifts directly map to standard integer operations, reducing computational complexity.

Fixed-point numbers are essentially integers with an *implied binary point* that is not physically stored in the hardware. For example, in a 16-bit fixed-point number with 4 fractional bits, the value 0110.0100_2 is treated as 6.25 but stored and manipulated as the integer 100. Arithmetic operations like addition and subtraction can use the same integer-based hardware because the binary point's location is an interpretation applied at the software or algorithm level.

Fixed-point numbers require consistent scaling factors. When performing operations, both operands must be in the same fixed-point format (same number of fractional bits). Multiplication results may need to be scaled back to the original format, which involves shifting the binary point. This introduces precision management, as the fractional part can lead to truncation or rounding errors.

In two's complement, the integer result is directly usable. In fixed-point arithmetic, the integer result must be interpreted based on the scaling factor to obtain the actual numerical value.

5.3.5 Comparison with Floating-Point Representation

While fixed-point arithmetic is simpler and more efficient, it lacks the flexibility and dynamic range of floating-point numbers. Floating-point systems dynamically adjust the scaling factor using an exponent, making them suitable for applications requiring high precision or a wide dynamic range.

Fixed-Point Advantages:

- Lower computational cost.

- Predictable rounding behavior.
- Simpler hardware implementation.

Fixed-Point Disadvantages:

- Limited dynamic range.
- Susceptible to overflow and underflow.
- Loss of precision in arithmetic operations.

5.4 Examples of Finite Real Numbers

Practical examples of finite real numbers include:

- Rational Numbers: Numbers like 0.75 or 0.333... are approximated with a finite number of decimal or binary digits, depending on the precision.
- Irrational Numbers: Irrationals like π or $\sqrt{2}$ are represented by truncated or rounded approximations. For instance, π may be approximated as 3.14159265358979 in double precision.
- Large or Small Magnitudes: Real numbers with large or small magnitudes, such as 6.022×10^{23} (Avogadro's number) or 1.602×10^{-19} (charge of an electron), are stored in scientific notation using floating-point systems.

5.5 Challenges in Finite Real Representations

The finite representation of real numbers introduces challenges in computation:

Rounding Errors. Due to limited precision, some numbers cannot be represented exactly, leading to rounding errors. For instance, the decimal 0.1 has no exact binary representation and is approximated, resulting in errors in calculations.

Overflow and Underflow. Numbers exceeding the range of the floating-point format result in overflow, while numbers too close to zero result in underflow, both leading to inaccuracies or loss of data.

Loss of Associativity. Finite precision arithmetic breaks mathematical properties like associativity. For example:

$$(a + b) + c \neq a + (b + c),$$

when $a, b, c \in \mathbb{R}$ are subject to rounding.

5.6 Alternative Finite Number Representations.

Beyond traditional floating-point formats, alternative finite number representations have been developed to address specific computational challenges.

Interval arithmetic represents numbers as intervals $[a, b]$ rather than single values, ensuring that rounding errors and uncertainties are contained within computations (Moore and Lodwick, 2009). For example, adding two intervals produces a new interval that encloses all possible results. This method is important in numerical analysis, optimization, and scientific computing but requires careful management to prevent interval overgrowth.

Logarithmic number systems (LNS) represent numbers as logarithms of their absolute values, offering efficient multiplication and division but complicating addition and subtraction (Naylor and Jones, 2013).

Tapered floating-point formats dynamically adjust the precision of the significand based on the exponent, improving adaptability for applications requiring variable precision (Gustafson, 2015).

Brain Floating Point In machine learning, bfloat16 balances the range of ‘float32’ with reduced precision, optimizing performance on large-scale AI workloads (Wang and Goyal, 2019).

Chapter 6

Character Representations

6.1	Encoding Alphabetic Characters in Computers	82
6.2	Early Character Encodings: EBCDIC and ASCII	82
6.2.1	EBCDIC (Extended Binary Coded Decimal Interchange Code)	82
6.2.2	ASCII (American Standard Code for Information Interchange).	82
6.3	Modern Character Encodings	83
6.3.1	Unicode	83
6.3.2	UTF Encodings	83
6.3.3	Additional Encodings	83
6.3.4	Comparison of Encoding Schemes	84

6.1 Encoding Alphabetic Characters in Computers

Modern computers operate using binary data, and to process text, they require a mechanism to encode alphabetic characters, digits, symbols, and control commands into binary representations. Character encoding is fundamental to storing, transmitting, and processing text data across systems, enabling interoperability between hardware and software platforms.

Computers represent all data using binary values (0s and 1s). However, human languages rely on diverse sets of characters and symbols. To bridge this gap, encoding systems map characters to unique binary values. The requirements for character encoding include:

- **Storage and Transmission:** Characters must be converted to binary for storage on disk, transmission over networks, and display on screens.
- **Standardization:** Encoding standards ensure that data can be correctly interpreted across different systems and applications.
- **Compatibility:** Encodings allow older systems to communicate with newer ones while supporting a range of languages and symbols.

6.2 Early Character Encodings: EBCDIC and ASCII

6.2.1 EBCDIC (Extended Binary Coded Decimal Interchange Code)

Developed by IBM in the 1960s for mainframe computers, EBCDIC was designed to encode text for punch cards and early data processing systems (IBM, 1964). EBCDIC uses 8 bits per character, allowing for 256 possible values. Key features include:

- A focus on backward compatibility with IBM's earlier systems.
- Separate blocks of values for uppercase letters, lowercase letters, digits, and control characters.
- Limited adoption outside IBM systems.

Example EBCDIC values:

A: 11000001₂, B: 11000010₂, 0: 11110000₂.

6.2.2 ASCII (American Standard Code for Information Interchange).

Introduced in 1963, ASCII quickly became the dominant character encoding standard for text. ASCII uses 7 bits per character, providing 128 unique values (Mackenzie, 1980). Key features of ASCII include:

- Standardized mappings for English alphabets (both uppercase and lowercase), digits, punctuation, and control characters.
- Compatibility with early printers and telecommunication systems.
- Limited to English, making it unsuitable for internationalization.

Example ASCII values:

A: 1000001₂, B: 1000010₂, a: 1100001₂, 0: 0110000₂.

6.3 Modern Character Encodings

6.3.1 Unicode

ASCII has limitations. Many languages require more characters than ASCII's 128 values. To address this, the Unicode standard was developed in the 1990s to support nearly all written languages (Mackenzie, 1980; Consortium, 2020).

Unicode assigns a unique numeric code (code point) to every character, independent of the underlying binary representation. For example:

A: U+0041, Ω: U+03A9, □□: U+4F60 U+597D.

While Unicode defines code points, it does not specify how these points are stored in binary. This led to the development of various Unicode Transformation Formats (UTFs).

6.3.2 UTF Encodings

UTF-8 is the most widely used Unicode encoding today (Consortium, 2020). It is a variable-length encoding that uses 1 to 4 bytes per character:

- ASCII characters (U+0000 to U+007F) are encoded in 1 byte, preserving backward compatibility.
- Characters outside the ASCII range use 2 to 4 bytes, depending on their code point.

Example UTF-8 encodings:

A: 01000001₂ (1 byte), Ω: 11001110 10101001₂ (2 bytes), □□: 11100100 10111100 10000000₂ 11100101 10100111

UTF-16. UTF-16 uses 2 or 4 bytes per character. Characters in the Basic Multilingual Plane (BMP) are stored in 2 bytes, while supplementary characters require 4 bytes. UTF-16 is popular in Windows systems and programming environments like Java.

UTF-32. UTF-32 uses a fixed 4 bytes per character, simplifying indexing and processing at the cost of memory efficiency. It is rarely used due to its high storage requirements.

6.3.3 Additional Encodings

In addition to Unicode-based encodings, various modern encodings have been developed for specific use cases:

Base64. Encodes binary data as printable ASCII characters, widely used in email and web applications.

MIME Encodings. Designed for encoding text and attachments in email protocols.

Custom Encodings. Certain systems, such as legacy hardware or proprietary software, use custom encoding schemes optimized for their requirements.

6.3.4 Comparison of Encoding Schemes

Table 6.1 summarizes the key differences between encoding schemes:

Encoding Scheme	Bits per Character	Characters Supported	Key Features
EBCDIC	8	256	IBM mainframes, legacy compatibility
ASCII	7	128	English text, limited international support
UTF-8	1–4 bytes	All Unicode characters	Backward compatible with ASCII
UTF-16	2–4 bytes	All Unicode characters	Efficient for BMP, used in Windows
UTF-32	4 bytes	All Unicode characters	Simplifies indexing, high storage cost
Base64	6 bits	64 printable characters	Encodes binary data into text for email, URLs, and web use
MIME (Multipurpose Internet Mail Extensions)	Variable	Text, images, and other media types	Standard for encoding email attachments and multimedia

Table 6.1: Comparison of Character Encoding Schemes, including Base64 and MIME.

Chapter 7

Summary of Number Systems and Representations

While representations encode numbers from a specific set, they may not fully exhibit the properties of that set. For example:

- Closure: In two's complement, the result of adding two numbers can overflow and exceed the representable range, even though addition within $\mathbb{Z}$ is closed.
- Symmetry: The set $\mathbb{Z}$ is symmetric around 0, but representations like two's complement have an asymmetric range due to the extra negative value (-2^{n-1}).
- Arithmetic Operations: Subtraction in excess- k may require decoding and re-encoding, as direct subtraction of encoded values can produce incorrect results.

These deviations arise because representations are designed for efficiency and practicality rather than mathematical purity.

Representations often have a limited range and precision due to finite storage:

- In $\mathbb{Z}$, there is no upper or lower bound on integers. However, in two's complement, the range is limited to $[-2^{n-1}, 2^{n-1} - 1]$ for n -bit systems.
- In $\mathbb{R}$, real numbers are continuous and infinite. In IEEE 754, real numbers are approximated using finite precision, leading to rounding errors.

Errors can propagate in numerical representations, even when the underlying set is exact. For example:

- Floating-point arithmetic introduces rounding errors due to the inability to represent certain numbers exactly (e.g., 0.1).
- BCD ensures exact decimal representation but requires correction during arithmetic operations like subtraction, which can be slower.

Representations often prioritize specific tasks or applications:

- Excess- k simplifies comparisons in floating-point systems, but it is not closed under subtraction without decoding.
- BCD facilitates precise decimal arithmetic for financial systems but is less efficient for general-purpose computation.

While sets like $\mathbb{Z}$, $\mathbb{Q}$, and $\mathbb{R}$ define the abstract mathematical properties of numbers, representations are practical implementations tailored to computational systems. A representation may encode elements of a set, but it might not preserve all the properties of the set. Understanding these differences is critical in applications where precision, range, or specific properties (e.g., symmetry or closure) are required. Representations must be chosen carefully based on the demands of the task, as their limitations can impact accuracy, efficiency, and correctness in practical systems.

Part II

FROM GATES TO PROCESSORS

Chapter 8

Combinational Circuits: No-Loop, Zero-Order Systems (0-OS)

8.1	Boolean Algebra	91
8.1.1	Historical Background	91
8.1.2	Basic Concepts	91
8.1.3	Operations	91
8.1.4	Laws	91
8.1.5	DeMorgan's Laws	92
8.1.6	Applications in Digital Logic	93
8.2	Truth Tables	93
8.2.1	Definition and Structure of Truth Tables	93
8.2.2	Logical Operators and Their Truth Tables	94
8.2.3	Applications of Truth Tables	94
8.2.4	Constructing Truth Tables for Complex Expressions	95
8.2.5	Limitations of Truth Tables	95
8.3	Digital Domain	95
8.3.1	Signals in the Digital Domain	95
8.4	One-Input Combinational Circuits	103
8.4.1	Formal Description	103
8.4.2	Physical Implementation of NOT Circuit	103
8.5	Two-input gates	104
8.5.1	Formal Description	104
8.6	Many-Input Gates	107
8.7	Generic combinational circuits	108
8.7.1	Decoder	108
8.7.2	Multiplexer	114
8.7.3	Elementary Multiplexer	114
8.7.4	Many-input Multiplexer	115
8.7.5	Demultiplexer	119
8.7.6	Seven-Segment Display	122
8.8	Behavioral vs. Structural Descriptions	131
8.9	Adders	132

8.9.1	Behavioral Adders	132
8.9.2	4-bit Behavioral Adder	134
8.9.3	Structural Adders	136
8.10	Multipliers	139
8.10.1	Behavioral Multiplier	139
8.10.2	Precision in Multiplication	140
8.10.3	Two's Complement Multiplication and Intermediate Overflows	142
8.10.4	Handling Extended Results	142
8.11	Multi-Function Arithmetic Units	143
8.11.1	Subtraction	143
8.12	Arithmetic & Logic Unit	155
8.13	Problems	161
8.13.1	Waveforms	161
8.13.2	Combinatorial Circuits	163
8.14	Exercises	166

In this chapter, we make a compromise and provide an introduction to the field of digital circuits that introduces only the necessary concepts to understand the principles on which the organization and architecture of computers are based. We generally present functional or behavioral designs in preference to highly optimized structural components.

8.1 Boolean Algebra

Boolean algebra is a branch of mathematics that deals with binary variables and logical operations. It serves as the foundation for modern digital logic and computer science, enabling the design and analysis of digital circuits and computational algorithms. Developed in the mid-19th century, Boolean algebra forms the basis of everything from simple electronic gates to complex processors.

8.1.1 Historical Background

Boolean algebra was introduced by George Boole in *An Investigation of the Laws of Thought*, published in 1854 (Boole, 1854). Boole sought to formalize the process of human reasoning using a symbolic language, representing logical relationships through algebraic structures. Although initially developed as a tool for philosophy and logic, Boolean algebra found practical applications in the early 20th century when Claude Shannon demonstrated its utility in designing electrical circuits (Shannon, 1938).

Shannon's master's thesis, *A Symbolic Analysis of Relay and Switching Circuits*, linked Boolean algebra with electrical switches, showing how logical operations could be implemented using relay circuits. This work laid the foundation for digital circuit design and the development of modern computers (Shannon, 1938).

8.1.2 Basic Concepts

Boolean algebra operates on a binary set $B = \{0, 1\}$, where:

- 0 represents **false**.
- 1 represents **true**.

8.1.3 Operations

AND ($\wedge$). A binary operation where $A \wedge B = 1$ if and only if both $A = 1$ and $B = 1$.

OR ($\vee$). A binary operation where $A \vee B = 1$ if at least one of A or B equals 1.

NOT ($\neg$). A unary operation where $\neg A = 1$ if $A = 0$, and $\neg A = 0$ if $A = 1$.

8.1.4 Laws

These operations obey the following basic laws:

Commutative Law:

$$A \vee B = B \vee A, \quad A \wedge B = B \wedge A$$

Associative Law:

$$A \vee (B \vee C) = (A \vee B) \vee C, \quad A \wedge (B \wedge C) = (A \wedge B) \wedge C$$

Distributive Law:

$$A \wedge (B \vee C) = (A \wedge B) \vee (A \wedge C)$$

Identity Law:

$$A \vee 0 = A, \quad A \wedge 1 = A$$

Complement Law:

$$A \vee \neg A = 1, \quad A \wedge \neg A = 0$$

8.1.5 DeMorgan's Laws

Augustus De Morgan (1806–1871) was a British mathematician and logician whose work played an important role in the development of formal logic and algebra. Born in India and educated in England, De Morgan became the first professor of mathematics at University College London (Grattan-Guinness, 1997).

De Morgan's Laws were introduced in his 1847 book, *Formal Logic: or, The Calculus of Inference, Necessary and Probable* (Morgan, 1847). In this work, De Morgan formalized rules for distributing negations over logical operators, providing a foundation for algebraic reasoning in logic. While the concepts underlying these laws can be traced back to Aristotelian logic, De Morgan was the first to formalize them using algebraic symbols.

His work paralleled that of George Boole, whose algebraic system, now known as Boolean algebra, was published in the same period. Together, De Morgan and Boole changed the field of logic, moving it from a purely philosophical discipline to a mathematical one (Grattan-Guinness, 1997).

DeMorgan's Laws are fundamental principles in Boolean algebra, describing the relationships between logical conjunction (AND), disjunction (OR), and negation (NOT). They provide a way to express the complement of a compound expression in terms of the complements of its components. These laws are especially useful in digital logic design and simplification, as they allow for the transformation of logic expressions to equivalent forms using fewer gates or alternate gate configurations (Rosen, 2018; Tanenbaum and Austin, 2013).

Mathematically, DeMorgan's Laws state:

$$\neg(A \vee B) = (\neg A) \wedge (\neg B), \quad \neg(A \wedge B) = (\neg A) \vee (\neg B)$$

These laws can be proven using truth tables or algebraic manipulation and hold for any Boolean variables A and B . In words:

- The complement of a disjunction ($A \vee B$) is equivalent to the conjunction of the complements ($\neg A \wedge \neg B$).
- The complement of a conjunction ($A \wedge B$) is equivalent to the disjunction of the complements ($\neg A \vee \neg B$).

8.1.5.1 Applications of DeMorgan's Laws

DeMorgan's Laws are widely applied in the simplification and design of digital circuits. For example, consider the simplification of the following expression that enables the implementation of the logic using AND gates and inverters rather than a more complex combination of gates (Rosen, 2018; Karnaugh, 1953).

$$\neg(A \vee B \vee C) = \neg A \wedge \neg B \wedge \neg C$$

8.1.6 Applications in Digital Logic

Boolean algebra provides the mathematical framework for designing and analyzing digital circuits. Logical gates such as AND, OR, and NOT are implemented in hardware to perform the basic operations of Boolean algebra. More complex circuits, such as multiplexers, decoders, and arithmetic logic units (ALUs), are built using combinations of these gates.

8.1.6.1 Simplification of Logic Expressions

One of the uses of Boolean algebra in digital logic is the simplification of logic expressions to minimize the number of gates required in a circuit. Simplification methods include:

- **Boolean identities:** Using the laws of Boolean algebra to rewrite expressions to convert between AND-OR logic and NAND-NOR logic (Rosen, 2018).
- **Karnaugh maps:** A graphical method for simplifying logic functions (Karnaugh, 1953).
- **Quine-McCluskey algorithm:** A tabular approach to minimize logic expressions (Quine, 1952).

8.2 Truth Tables

Truth tables are a fundamental tool in Boolean algebra, logic, and digital circuit design. They provide a systematic way to represent and analyze the behavior of logical expressions and digital circuits by listing all possible input combinations and their corresponding output values (Rosen, 2018; Tanenbaum and Austin, 2013).

8.2.1 Definition and Structure of Truth Tables

A truth table is a tabular representation of a logical expression or circuit. It includes the following components:

- **Input Variables:** Columns representing the logical variables ($A, B, \dots, N$).
- **Output Values:** Columns representing the result of applying a logical operation or expression to the input variables.
- **Rows:** Each row corresponds to a unique combination of input variable values, covering all 2^n possible combinations for n variables.

For example, the truth table for a simple AND operation ($A \wedge B$) is as follows:

A	B	$A \wedge B$
0	0	0
0	1	0
1	0	0
1	1	1

8.2.2 Logical Operators and Their Truth Tables

Truth tables are used to describe the behavior of basic logical operators. The primary logical operations include:

- NOT ($\neg$): Negation inverts the value of a single variable.

A	$\neg A$
0	1
1	0

- AND ($\wedge$): Returns true if both inputs are true.

A	B	$A \wedge B$
0	0	0
0	1	0
1	0	0
1	1	1

- OR ($\vee$): Returns true if at least one input is true.

A	B	$A \vee B$
0	0	0
0	1	1
1	0	1
1	1	1

- XOR ($\oplus$): Returns true if exactly one input is true.

A	B	$A \oplus B$
0	0	0
0	1	1
1	0	1
1	1	0

8.2.3 Applications of Truth Tables

Truth tables are used in various domains of computer science, mathematics, and engineering:

- 1) Expression Simplification: Truth tables help simplify logical expressions by identifying equivalent forms.

- 2) Circuit Design: In digital electronics, truth tables describe the behavior of combinational circuits such as multiplexers, decoders, and adders.
- 3) Validation and Verification: Logical correctness of algorithms and circuits can be validated using truth tables.
- 4) Set Theory and Predicate Logic: Truth tables are used to analyze logical relationships and implications in formal proofs.

8.2.4 Constructing Truth Tables for Complex Expressions

For complex expressions involving multiple variables and operators, truth tables are constructed by evaluating the expression step-by-step:

- 1) List all possible input combinations (2^n rows for n variables).
- 2) Break the expression into subexpressions and calculate their values row by row.
- 3) Combine subexpression results to compute the final output.

For example, consider $\neg(A \vee B) \wedge C$. The truth table is as follows:

A	B	C	$A \vee B$	$\neg(A \vee B)$	$\neg(A \vee B) \wedge C$
0	0	0	0	1	0
0	0	1	0	1	1
0	1	0	1	0	0
0	1	1	1	0	0
1	0	0	1	0	0
1	0	1	1	0	0
1	1	0	1	0	0
1	1	1	1	0	0

8.2.5 Limitations of Truth Tables

While truth tables are highly effective for small-scale problems, their size grows exponentially with the number of variables (2^n rows for n variables). This makes them impractical for analyzing complex systems with a large number of inputs. In such cases, alternative methods such as Karnaugh maps, Quine-McCluskey algorithms, or software tools are employed (Karnaugh, 1953; Rosen, 2018).

8.3 Digital Domain

8.3.1 Signals in the Digital Domain

In the electronic digital domain, we work with only two values (see Figure 8.1):

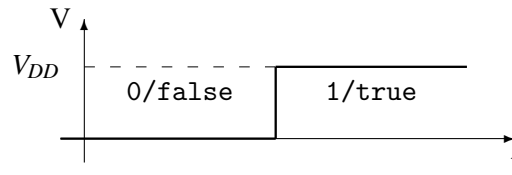


Figure 8.1: The two levels of the signal in the digital domain. Low level (0 Volt) for 0 or `false`, and high level (V_{DD} Volt) for 1 or `true`.

- 0, represented by the electrical value 0 V, having two meanings:
 - ◊ the numerical value 0
 - ◊ the logic value `false`
- 1, represented by the electrical value V_{DD} V, having two meanings:
 - ◊ the numerical value 1
 - ◊ the logic value `true`

The signals used in a digital system are of two types:

- non-periodic, with a certain structure ("random") adapted to the process of simulating the operation of a digital system
- periods, usually with a strictly alternating structure, used as clock signals by which the operation of a digital system is synchronized.

Any simulation is based on the generation of waveforms with which the simulated circuit is stimulated on its inputs. We describe two generated waveforms: one "random" and one usable as a periodic signal.

8.3.1.1 SystemVerilog Implementation of Waveforms

```

8  module waveFormGenerator();
9      logic randomWave;
10     logic clock      ;
11     initial begin      randomWave = 0  ;
12                         #2 randomWave = 1  ;
13                         #6 randomWave = 0  ;
14                         #4 randomWave = 1  ;
15                         #8 randomWave = 0  ;
16                         #5 $stop          ;
17     end
18     initial begin      clock = 0          ;
19                         forever #2 clock = ~clock  ;
20     end

```

```
21 endmodule
```

Listing 8.1: Waveform Generator Code (Hand Written SystemVerilog)

Listing 8.1 shows the SystemVerilog module, `waveFormGenerator`. It generates two output waveforms: a `randomWave` signal and a `clock` signal. The `randomWave` signal transitions between predefined states based on explicit delays specified using the `#` operator. It begins at 0, transitions to 1 after 2 time units, back to 0 after 6 more time units, to 1 after 4 additional time units, and finally to 0 after 8 time units. The simulation halts 5 time units later using the `$stop` command. The `clock` signal is a periodic waveform toggling its state every 2 time units, implemented with a `forever` loop.

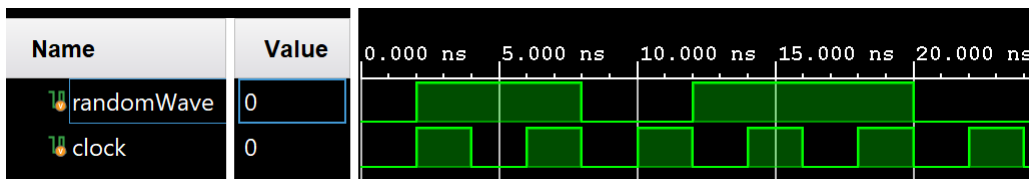


Figure 8.2: Waveforms generated by Vivado for hand written SystemVerilog simulation

Figure 8.2 shows the results of simulating the `waveFormGenerator.sv` module using Vivado.

8.3.1.2 Chisel Implementation of Waveforms

```

8 class WaveFormGenerator extends Module {
9   val io = IO(new Bundle {
10     val randomWave = Output(Bool())
11     val clockWave = Output(Bool())
12   })
13
14   // Random wave pattern based on a sequence of values
15   val randomWaveSeq = VecInit(false.B, true.B, false.B, true.B, false.B)
16   val randomIndex = RegInit(0.U(3.W))
17   io.randomWave := randomWaveSeq(randomIndex)
18   randomIndex := Mux(randomIndex === 4.U, 0.U, randomIndex + 1.U)
19
20   // Clock waveform toggles every 2 cycles
21   val clockToggle = RegInit(false.B)
22   clockToggle := ~clockToggle
23   io.clockWave := clockToggle
24 }

```

Listing 8.2: Waveform Generator Code (Chisel: WaveFormGenerator.scala)

Listing 8.2 shows a Chisel version of a `WaveFormGenerator` module. The `WaveFormGenerator` class extends the `Module` trait. In Chisel, a trait is a feature of Scala that allows for the composition of behavior and reusable code. Traits are similar to interfaces in Java but are more powerful because they

can contain concrete methods and fields, as well as abstract ones. The `Module` trait is a base building block for defining hardware modules. It encapsulates the logic for defining a hardware component and managing its inputs, outputs, and internal connections.

When the `WaveFormGenerator` class extends the `Module` trait, it inherits the functionality provided by the `Module`, such as the ability to define an IO bundle for interfacing with the module, encapsulate logic within the hardware design, and interact with other modules. The `Module` trait ensures that the `WaveFormGenerator` class behaves as a hardware component within the Chisel design hierarchy, supporting clean composition and integration into larger systems.

The `WaveFormGenerator` class has two distinct output waveforms: `randomWave` and `clockWave`. Both signals are driven by internal logic that determines their behavior over time.

The `randomWave` signal is defined as a repeating sequence of Boolean values: `false`, `true`, `false`, `true`, and `false`. A register, `randomIndex`, is used to track the current position in this sequence. The module cycles through the sequence in order, and when the last value is reached, the index resets to zero, causing the pattern to loop indefinitely.

The `clockWave` signal generates a toggling clock with a period of two cycles. This is achieved using a register, `clockToggle`, which alternates between `true` and `false` on each clock cycle. The result is a square wave output that can be used as a clock signal for other hardware modules or for synchronization purposes.

The `WaveFormGenerator` module demonstrates the generation of simple waveforms using high-level constructs in Chisel. By abstracting away low-level timing details, it provides a clean and maintainable implementation suitable for hardware design and simulation.

```

26 object WaveFormGenerator extends App {
27   ChiselStage.emitSystemVerilogFile(
28     new WaveFormGenerator,
29     Array("--target-dir", "generated_verilog_annotated")

```

Listing 8.3: Waveform Generator Code with Emit SystemVerilog (Chisel)

```

79   synthesizeNetlist(module, moduleName, optimizeForASIC)
80   generateSVG(module, moduleName)
81   convertSVGtoPNG(module, moduleName, convertSVGcommand)
82   if(shouldDeleteIntermediateFiles) deleteIntermediateFiles(module,
83     moduleName)
84   //printHelp()
85 } catch {
86   case e: Exception =>
87     println(s"Error generating hardware ${e.getMessage}")
88     throw e
89 }
90 }
91
92 def generateFIRRTLAndSystemVerilog(chiselModule: () => chisel3.Module
93   , moduleName: String): Unit = {
94   println(s"${moduleName}: generateFIRRTL")

```

```

94      // Stage1: Dump fir file with all annotations. Will also generate
          stipped fir.mlir file
95      val firrtlBaseArgs = Array("--target", "firrtl",
96                                "--dump-fir", // Contains all
          suggestNames()
97                                "--preserve-aggregate", "all",
98                                "--target-dir", generatedFirRTLPath)
99
100     if(isCmdInstalled("firtool")) {
101         val firtoolPath = getCmdPath("firtool")
102         println(s"firtoolPath=$firtoolPath")

```

Listing 8.4: fir.mlir file generation from Chisel

Listing 8.3 show that Chisel has the capability to emit SystemVerilog. However, the tools that ship natively with Chisel are typically behind the latest releases. Our process for generating hardware is documented in Appendix A.1. In short, we execute a scala file that invokes command line versions of the latest tools or pass in the path to the latest tools if the option exists. Listing 8.4 line 89 shows the command line argument for firtool mlir generation. We then run firtool again to generate SystemVerilog.

```

1  // Generated by CIRCT firtool-1.99.1
2
3  // Include register initializers in init blocks unless synthesis is set
4  `ifndef RANDOMIZE
5      `ifdef RANDOMIZE_REG_INIT
6          `define RANDOMIZE
7      `endif // RANDOMIZE_REG_INIT
8  `endif // not def RANDOMIZE
9  `ifndef SYNTHESIS
10     `ifndef ENABLE_INITIAL_REG_
11         `define ENABLE_INITIAL_REG_
12     `endif // not def ENABLE_INITIAL_REG_
13 `endif // not def SYNTHESIS
14
15 // Standard header to adapt well known macros for register
    randomization.
16
17 // RANDOM may be set to an expression that produces a 32-bit random
    unsigned value.
18 `ifndef RANDOM
19     `define RANDOM $random
20 `endif // not def RANDOM
21
22 // Users can define INIT_RANDOM as general code that gets injected into
    the
23 // initializer block for modules with registers.
24 `ifndef INIT_RANDOM
25     `define INIT_RANDOM
26 `endif // not def INIT_RANDOM
27

```

```

28 // If using random initialization, you can also define RANDOMIZE_DELAY
    to
29 // customize the delay used, otherwise 0.002 is used.
30 `ifndef RANDOMIZE_DELAY
31     `define RANDOMIZE_DELAY 0.002
32 `endif // not def RANDOMIZE_DELAY
33
34 // Define INIT_RANDOM_PROLOG_ for use in our modules below.
35 `ifndef INIT_RANDOM_PROLOG_
36     `ifdef RANDOMIZE
37         `ifdef VERILATOR
38             `define INIT_RANDOM_PROLOG_ `INIT_RANDOM
39         `else // VERILATOR
40             `define INIT_RANDOM_PROLOG_ `INIT_RANDOM #`RANDOMIZE_DELAY begin
                end
41         `endif // VERILATOR
42     `else // RANDOMIZE
43         `define INIT_RANDOM_PROLOG_
44     `endif // RANDOMIZE
45 `endif // not def INIT_RANDOM_PROLOG_
46 module WaveFormGenerator( // /home/jglossner/GitRepos/
    Structured_Computer_Architecture/RTL/Chisel/generators/generated/
    firrtl/WaveFormGenerator.fir.mlir:3:5
47 input  clock, // /home/jglossner/GitRepos/
    Structured_Computer_Architecture/RTL/Chisel/generators/generated/
    firrtl/WaveFormGenerator.fir.mlir:3:41
48     reset, // /home/jglossner/GitRepos/
    Structured_Computer_Architecture/RTL/Chisel/generators/
    generated/firrtl/WaveFormGenerator.fir.mlir:3:67
49 output io_randomWave, // /home/jglossner/GitRepos/
    Structured_Computer_Architecture/RTL/Chisel/generators/generated/
    firrtl/WaveFormGenerator.fir.mlir:3:96
50     io_clockWave // /home/jglossner/GitRepos/
    Structured_Computer_Architecture/RTL/Chisel/generators/
    generated/firrtl/WaveFormGenerator.fir.mlir:3:133
51 );
52
53 wire [7:0] _GEN = '{1'h0, 1'h0, 1'h0, 1'h0, 1'h1, 1'h0, 1'h1, 1'h0};
54 reg  [2:0] randomIndex; // /home/jglossner/GitRepos/
    Structured_Computer_Architecture/RTL/Chisel/generators/generated/
    firrtl/WaveFormGenerator.fir.mlir:12:22
55 reg      clockToggle; // /home/jglossner/GitRepos/
    Structured_Computer_Architecture/RTL/Chisel/generators/generated/
    firrtl/WaveFormGenerator.fir.mlir:20:22
56 always @(posedge clock) begin // /home/jglossner/GitRepos/
    Structured_Computer_Architecture/RTL/Chisel/generators/generated/
    firrtl/WaveFormGenerator.fir.mlir:3:41
57 if (reset) begin // /home/jglossner/GitRepos/
    Structured_Computer_Architecture/RTL/Chisel/generators/generated
    /firrtl/WaveFormGenerator.fir.mlir:3:41
58     randomIndex <= 3'h0; // /home/jglossner/GitRepos/

```

```

        Structured_Computer_Architecture/RTL/Chisel/generators/
        generated/firrtl/WaveFormGenerator.fir.mlir:7:17, :12:22
59 clockToggle <= 1'h0;          // /home/jglossner/GitRepos/
        Structured_Computer_Architecture/RTL/Chisel/generators/
        generated/firrtl/WaveFormGenerator.fir.mlir:3:5, :20:22
60 end
61 else begin // /home/jglossner/GitRepos/
        Structured_Computer_Architecture/RTL/Chisel/generators/generated
        /firrtl/WaveFormGenerator.fir.mlir:3:41
62 randomIndex <= randomIndex == 3'h4 ? 3'h0 : randomIndex + 3'h1;
        // /home/jglossner/GitRepos/
        Structured_Computer_Architecture/RTL/Chisel/generators/
        generated/firrtl/WaveFormGenerator.fir.mlir:6:17, :7:17,
        :12:22, :15:25, :16:27, :18:27
63 clockToggle <= ~clockToggle; // /home/jglossner/GitRepos/
        Structured_Computer_Architecture/RTL/Chisel/generators/
        generated/firrtl/WaveFormGenerator.fir.mlir:20:22, :21:25
64 end
65 end // always @(posedge)
66 `ifdef ENABLE_INITIAL_REG_ // /home/jglossner/GitRepos/
        Structured_Computer_Architecture/RTL/Chisel/generators/generated/
        firrtl/WaveFormGenerator.fir.mlir:3:5
67 `ifdef FIRRTL_BEFORE_INITIAL // /home/jglossner/GitRepos/
        Structured_Computer_Architecture/RTL/Chisel/generators/generated
        /firrtl/WaveFormGenerator.fir.mlir:3:5
68 `FIRRTL_BEFORE_INITIAL // /home/jglossner/GitRepos/
        Structured_Computer_Architecture/RTL/Chisel/generators/
        generated/firrtl/WaveFormGenerator.fir.mlir:3:5
69 `endif // FIRRTL_BEFORE_INITIAL
70 initial begin // /home/jglossner/GitRepos/
        Structured_Computer_Architecture/RTL/Chisel/generators/generated
        /firrtl/WaveFormGenerator.fir.mlir:3:5
71 automatic logic [31:0] _RANDOM[0:0]; // /home/jglossner/
        GitRepos/Structured_Computer_Architecture/RTL/Chisel/
        generators/generated/firrtl/WaveFormGenerator.fir.mlir:3:5
72 `ifdef INIT_RANDOM_PROLOG_ // /home/jglossner/GitRepos/
        Structured_Computer_Architecture/RTL/Chisel/generators/
        generated/firrtl/WaveFormGenerator.fir.mlir:3:5
73 `INIT_RANDOM_PROLOG_ // /home/jglossner/GitRepos/
        Structured_Computer_Architecture/RTL/Chisel/generators/
        generated/firrtl/WaveFormGenerator.fir.mlir:3:5
74 `endif // INIT_RANDOM_PROLOG_
75 `ifdef RANDOMIZE_REG_INIT // /home/jglossner/GitRepos/
        Structured_Computer_Architecture/RTL/Chisel/generators/
        generated/firrtl/WaveFormGenerator.fir.mlir:3:5
76 _RANDOM[/*Zero width*/ 1'b0] = `RANDOM; // /home/jglossner/
        GitRepos/Structured_Computer_Architecture/RTL/Chisel/
        generators/generated/firrtl/WaveFormGenerator.fir.mlir:3:5
77 randomIndex = _RANDOM[/*Zero width*/ 1'b0][2:0]; // /
        home/jglossner/GitRepos/Structured_Computer_Architecture/RTL
        /Chisel/generators/generated/firrtl/WaveFormGenerator.fir.

```

```

    mlir:3:5, :12:22
78     clockToggle = _RANDOM[/*Zero width*/ 1'b0][3]; // /home/
        jglossner/GitRepos/Structured_Computer_Architecture/RTL/
        Chisel/generators/generated/firrtl/WaveFormGenerator.fir.
        mlir:3:5, :12:22, :20:22
79     `endif // RANDOMIZE_REG_INIT
80 end // initial
81 `ifdef FIRRTL_AFTER_INITIAL // /home/jglossner/GitRepos/
    Structured_Computer_Architecture/RTL/Chisel/generators/generated
    /firrtl/WaveFormGenerator.fir.mlir:3:5
82     `FIRRTL_AFTER_INITIAL // /home/jglossner/GitRepos/
        Structured_Computer_Architecture/RTL/Chisel/generators/
        generated/firrtl/WaveFormGenerator.fir.mlir:3:5
83     `endif // FIRRTL_AFTER_INITIAL
84 `endif // ENABLE_INITIAL_REG_
85 assign io_randomWave = _GEN[randomIndex]; // /home/jglossner/
    GitRepos/Structured_Computer_Architecture/RTL/Chisel/generators/
    generated/firrtl/WaveFormGenerator.fir.mlir:3:5, :12:22, :13:12
86 assign io_clockWave = clockToggle; // /home/jglossner/GitRepos/
    Structured_Computer_Architecture/RTL/Chisel/generators/generated/
    firrtl/WaveFormGenerator.fir.mlir:3:5, :20:22
87 endmodule

```

Listing 8.5: Waveform Generator SystemVerilog generated by Chisel

Listing 8.5 shows the system verilog code generated by firtool.

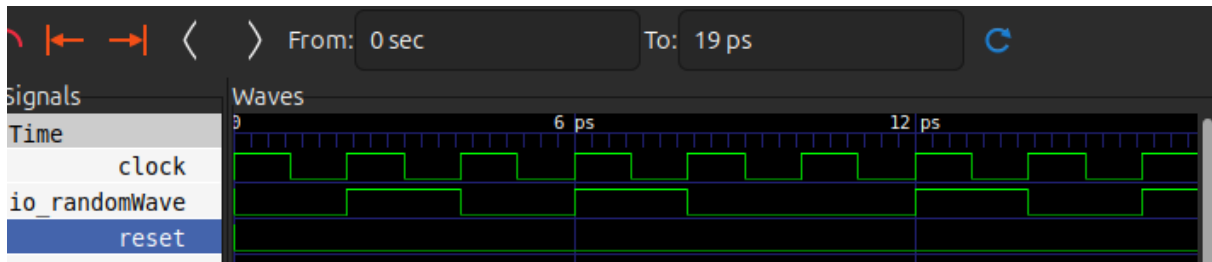


Figure 8.3: Waveforms for generated SystemVerilog Verilog simulation

Figure 8.3 shows the waveform produced by simulating the clock generator using verilator. Section A.1.56 shows the process for generating the waveform shown in Figure 8.3.

8.3.1.3 Comparison of Chisel and SystemVerilog Implementation of Waveforms

In the Chisel implementation of the waveFormGenerator, the functionality is expressed using high-level abstractions. The randomWave is generated as a cyclic sequence of Boolean values, and its transitions are determined by a counter that increments on every clock cycle. Unlike the Verilog version, the Chisel implementation does not explicitly emulate timing delays but instead assumes uniform timing for each state in the sequence.

While both implementations generate a randomWave and a clock-like signal, they differ in their timing behavior and functionality. The Verilog version explicitly specifies time delays for signal transitions and

provides a clear stop condition using `$stop`. The Chisel version is higher-level, more abstract, and assumes regular timing intervals without emulating the exact delays. This makes the Chisel version more suitable for hardware synthesis, where timing is managed by the clock, while the Verilog version offers precise control for simulation purposes.

8.4 One-Input Combinational Circuits

Combinational circuits are a fundamental type of digital circuit where the outputs are determined solely by the current inputs, without any memory or feedback elements. They perform logical and arithmetic operations based on Boolean algebra. Examples of combinational circuits include logic gates (AND, OR, NOT), arithmetic circuits (adders, subtractors), and data routing components (multiplexers, encoders). These circuits are deterministic, meaning their outputs are uniquely determined by the given inputs (Mano and Ciletti, 2017).

8.4.1 Formal Description

A one-input combinational circuit is a digital logic circuit where the output depends solely on the value of a single binary input A . Such circuits are governed by a Boolean function $f(A)$, where $f: \{0, 1\} \rightarrow \{0, 1\}$. The output is computed instantaneously based on the input value, with no memory or feedback elements involved. The behavior of the circuit can be described by a truth table, which enumerates the two possible input values ($A = 0$ and $A = 1$) and their corresponding outputs ($f(0)$ and $f(1)$).

Table 8.1: One-input circuits.

A	NOT	BUFFER
0	1	0
1	0	1

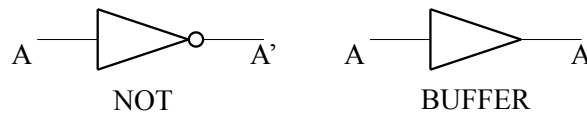


Figure 8.4: Graphic representation of the NOT and BUFFER circuits.

Table 8.1 shows the truth table for a NOT gate, which inverts the input ($f(A) = \neg A$), and BUFFER (constant) circuit, which outputs a fixed value regardless of the input ($f(A) = 0$ or $f(A) = 1$). Figure 8.4 shows the logic circuit symbol of the NOT and BUFFER combinational circuits. This is also called a gate symbol.

8.4.2 Physical Implementation of NOT Circuit

The physical implementation of a NOT circuit, or inverter, bridges the abstract concept of Boolean algebra with real-world digital hardware. A NOT gate takes a single input and produces an output that is its

logical complement. In practice, this is typically achieved using transistor-based designs, such as CMOS (Complementary Metal-Oxide-Semiconductor) technology.

The delay inherent in a NOT circuit, commonly referred to as the *gate delay*, arises from the physical properties of the transistors and the interconnects. When the input changes state, the transistors must charge or discharge the capacitive load at the output. This charging and discharging process is influenced by factors such as the capacitance of the output node, the resistance of the transistors, and the speed of the power supply. As a result, the propagation delay, defined as the time taken for the output to reach a certain percentage of its final value after a change in input, becomes an important metric in digital circuit performance.

Understanding where gate delays originate is important for designing circuits that meet timing requirements in high-speed applications. The interested reader can consult Appendix B for more information.

8.5 Two-input gates

Two-input gates depend on two binary inputs. Common two-input gates include AND, OR, XOR, NAND, and NOR.

8.5.1 Formal Description

Two-input gates are fundamental components in digital logic that perform Boolean operations on two binary variables, A and B , to produce a single output $f(A, B)$. Mathematically, these gates implement functions $f : \{0, 1\} \times \{0, 1\} \rightarrow \{0, 1\}$, where the domain consists of all possible ordered pairs of binary inputs, and the range is restricted to binary outputs.

Each type of two-input gate corresponds to a specific Boolean operation, such as AND ($f(A, B) = A \wedge B$). The behavior of these gates can be fully specified using truth tables, which enumerate all possible input combinations and their corresponding outputs.

Table 8.2: Two-input Combinational Gates Truth Table.

A	B	OR $A \vee B$	NOR $\neg(A \vee B)$	AND $A \wedge B$	NAND $\neg(A \wedge B)$	XOR $A \oplus B$	XNOR $\neg(A \oplus B)$
0	0	0	1	0	1	0	1
0	1	1	0	0	1	1	0
1	0	1	0	0	1	1	0
1	1	1	0	1	0	0	1

Table 8.2 shows the truth table for combinational 2-input gates and Table 8.3 shows the summary of their interpretations.

AND : $A \cdot B = AB$, where $AB = 1$ only if both A and B are 1, otherwise $AB = 0$.

Numerical interpretation: Arithmetic product.

Symbolic interpretation: Gate, because $A = 1$ opens the way for B .

Alternative names: Conjunction, multiply gate, all-or-nothing gate (Boole, 1854; Rosen, 2018).

NAND : $\neg(A \cdot B)$, where the output is 0 only if both A and B are 1; otherwise, the output is 1.

Symbolic interpretation: Universal gate.

Alternative names: Sheffer stroke, negated AND gate (Sheffer, 1913; Rosen, 2018).

OR : $A + B$, where $A + B = 1$ if at least one of A or B is 1; otherwise, $A + B = 0$.

Symbolic interpretation: Inclusive OR.

Alternative names: Disjunction, any-or-all gate (Rosen, 2018).

NOR : $\neg(A + B)$, where the output is 0 if at least one of A or B is 1; otherwise, the output is 1.

Symbolic interpretation: Universal gate.

Alternative names: Peirce arrow, negated OR gate (Peirce, 1885; Rosen, 2018).

XOR : Exclusive OR, because it excludes the case $A = B = 1$ in determining 1 on the output.

$A \oplus B = 1$ when only one of the inputs A or B is 1.

Logic interpretation: Conditioned inverter, i.e., if $A = 0$ then $A \oplus B = B$, else $A \oplus B = B'$.

Numerical interpretation: Modulo-2 sum.

Symbolic interpretation: Anticoincidence.

Alternative names: Either-or gate, modulo-2 adder (Shannon, 1948; Rosen, 2018).

XNOR : $\neg(A \oplus B)$.

Symbolic interpretation: Coincidence.

Alternative names: Equality gate, NOT XOR (Mano and Ciletti, 2017; Rosen, 2018).

Logic Gate	Numerical	Symbolic	Alternative Names
OR	Logical addition (no carry) $A + B$	$A \vee B$	Disjunction, Any-or-All Gate, Inclusive OR
NOR	Complement of OR: $1 - (A + B)$	$\neg(A \vee B)$	Peirce Arrow, Negated OR Gate, Universal Gate
AND	Logical multiplication $A \cdot B$	$A \wedge B$	Conjunction, Multiply Gate, All-or-Nothing Gate
NAND	Complement of AND: $1 - (A \cdot B)$	$\neg(A \wedge B)$	Sheffer Stroke, Negated AND Gate, Universal Gate
XOR	Modulo-2 addition $A \oplus B$	$A \oplus B$	Exclusive OR, Modulo-2 Adder, Anticoincidence
XNOR	Complement of XOR: $1 - (A \oplus B)$	$\neg(A \oplus B)$	Exclusive NOR, Equality Gate, Coincidence

Table 8.3: Summary of Logical Gates and their Interpretations

Many of the historical names in 8.3 stem from different disciplines. Terms like conjunction, disjunction, and negation originate from symbolic logic. Coincidence and anticoincidence were used in early telecommunications signal processing contexts. Practical descriptions like multiply gate and modulo-2 adder reflect the role of these gates in arithmetic and data processing. Modern digital logic has largely

standardized the terms AND, OR, XOR, etc., as defined by IEEE and other standard bodies (IEEE Standards Association, 1984).

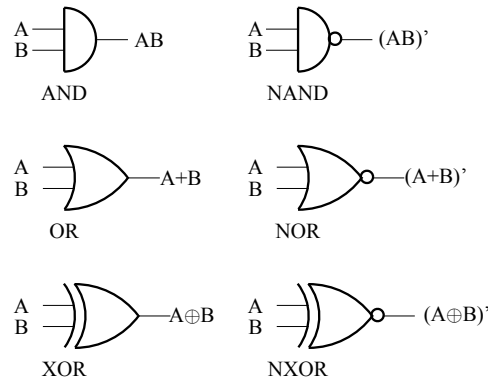


Figure 8.5: Graphical representation of the most used two-input gates.

Figure 8.5 shows the graphical representations of logic gates. These originated from the need to visually depict logical operations in a way that facilitated the design and analysis of digital systems. These representations were inspired by symbolic logic and Boolean algebra, where logic functions were expressed using abstract symbols. Early pioneers, such as Claude Shannon, formalized the connection between Boolean algebra and electronic circuits, enabling the translation of logical expressions into circuit designs (Shannon, 1948). Over time, distinct shapes were assigned to each gate for clarity—such as the triangular symbol for NOT gates and the curved line for OR gates. Standardization efforts by organizations like the IEEE and IEC later codified these symbols, ensuring universal recognition and consistency in circuit diagrams (IEEE Standards Association, 1984).

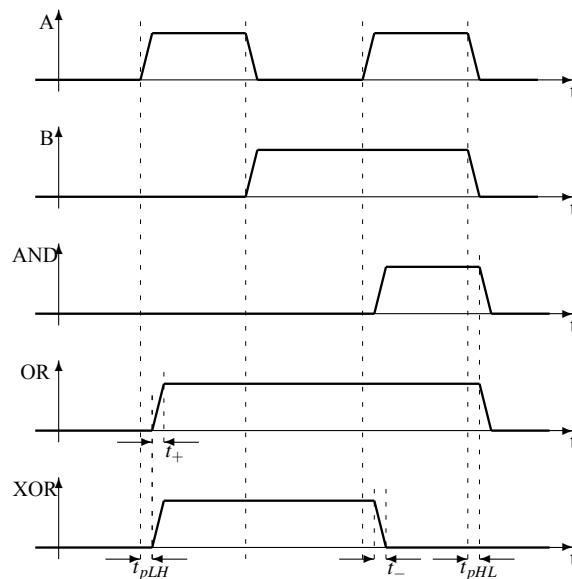


Figure 8.6: Wave forms for the non-inverting gates.

Waveforms are tools used in digital logic for visualizing and analyzing the behavior of signals over

time. They represent the changes in voltage levels for inputs, outputs, and intermediate nodes in a circuit, making it easier to understand timing relationships, transitions, and delays. The use of waveforms emerged alongside the development of oscilloscopes and other signal measurement tools in the early 20th century, as engineers needed a method to observe electrical signals in real-time. With the advent of digital systems, waveforms became a standard way to represent binary signals, where high and low voltage levels correspond to logical 1s and 0s, respectively. Waveforms allow engineers to verify the correct operation of logic circuits and diagnose timing-related issues such as glitches, race conditions, and setup/hold violations (Horowitz and Hill, 1993; IEEE Standards Association, 2011). Their graphical nature complements schematic diagrams by providing temporal information that is not inherently visible in static circuit representations.

Figure 8.6 shows the behavior of AND, OR and XOR gates. They are driven by the signals A and B represented in the first two wave forms. Besides the logic response to the inputs, timing characteristics are also shown.

- t_+ : the positive transition time (the duration of the positive edge)
- t_- : the negative transition time (the duration of the negative edge)
- t_{pLH} : the propagation time of the signal through the gate for the transition from Low to High
- t_{pHL} : the propagation time of the signal through the gate for the transition from High to Low

We leave the treatment of timing to a course on digital logic design. It is a critical component in building working systems but is abstracted away in this book.

8.6 Many-Input Gates

Many-input gates are a generalization of 2-input gates, allowing the combination of more than two inputs into a single logical operation. For instance, an n -input AND gate outputs a logical 1 only when all n inputs are 1, while an n -input OR gate outputs 1 when at least one of the inputs is 1. As the number of inputs increases, the complexity of the gate's behavior grows exponentially, based on the number of possible input configurations.

Mathematically, the total number of distinct n -input gates is N^{2^n} , where N represents the number of possible output values (typically $N = 2$ for binary logic), and 2^n is the total number of unique input configurations for n inputs. This exponential growth means that even for relatively small n , the number of possible logic gates becomes vast and impractical to enumerate. For example, with $n = 3$, there are $2^{2^3} = 256$ possible gates. For $n > 2$, Boolean algebra rules are necessary for simplifying and managing these functions, enabling the design of practical circuits without explicitly listing all possible gates.

In modern digital design, many-input gates are often implemented using a combination of smaller gates, as direct implementation can become physically and computationally impractical for large n . For example, a 16-input AND gate might be constructed hierarchically by combining multiple 2-input AND gates.

Let's consider some significant examples using verilog's ' (tick) symbol to denote negation:

- $(A')' = A$
- $AB + AB' = A(B + B') = A \cdot 1 = A$

When $B=1$ the expression takes the value A , and when $B=0$ the expression still takes the value

A, we conclude that the value of B does not matter and the expression takes the value A.

- $A + A'B = A + B$

This can be proved by the truth table in Table 8.4.

Table 8.4: Truth table proof of $A + A'B = A + B$.

A	B	A'	A'B	A+A'B	A+B
0	0	1	0	0	0
0	1	1	1	1	1
1	0	0	0	1	1
1	1	0	0	1	1

- $A \oplus B = (A' \oplus B)' = (A \oplus B')' = A' \oplus B'$

- $A + B = (A'B')'$

From de Morgan's rule, A or B is 1 is equivalent to the fact that it is not true that A is 0 and B is 0.

- $AB = (A' + B')'$

From de Morgan rule, A and B are 1 is equivalent to the fact that it is not true that A is 0 or B is 0.

8.7 Generic combinational circuits

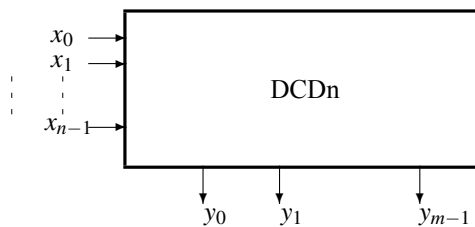


Figure 8.7: Decoder with n -bit input (DCDn).

8.7.1 Decoder

Figure 8.7 shows a generic n -bit decoder. A decoder is a combinational logic circuit that translates a binary-encoded input signal into a unique one-hot output. The term "one-hot" refers to a representation in which only one bit is set to '1', while all other bits are '0'. For example, in a 4-bit 1-hot code, valid outputs include 0001, 0010, 0100, and 1000. This encoding is commonly used in digital systems in control logic. It eliminates the need for decoding multiple bits simultaneously.

A decoder takes an n -bit binary input and generates 2^n output lines, with exactly one line active (logic high) for any given input combination. The structure of a decoder includes an array of logic gates that process the input bits and their complements to generate the appropriate output signals. For example,

Input	Input	Output	Output	Output	Output
x_1	x_0	y_3	y_2	y_1	y_0
0	0	0	0	0	1
0	1	0	0	1	0
1	0	0	1	0	0
1	1	1	0	0	0

Figure 8.8: Truth table for a 2-to-4 decoder. Each input combination activates one unique output in a one-hot encoding scheme.

Figure 8.8 shows the truth table for a 2-to-4 decoder. Figure 8.9a shows an elementary decoder that output both x_n and $\bar{x}_n$. Figure 8.9a shows the two input bits (x_1 and x_0) and their negations ($\bar{x}_0$ and $\bar{x}_1$) are used to drive four output signals (y_3, y_2, y_1, y_0).

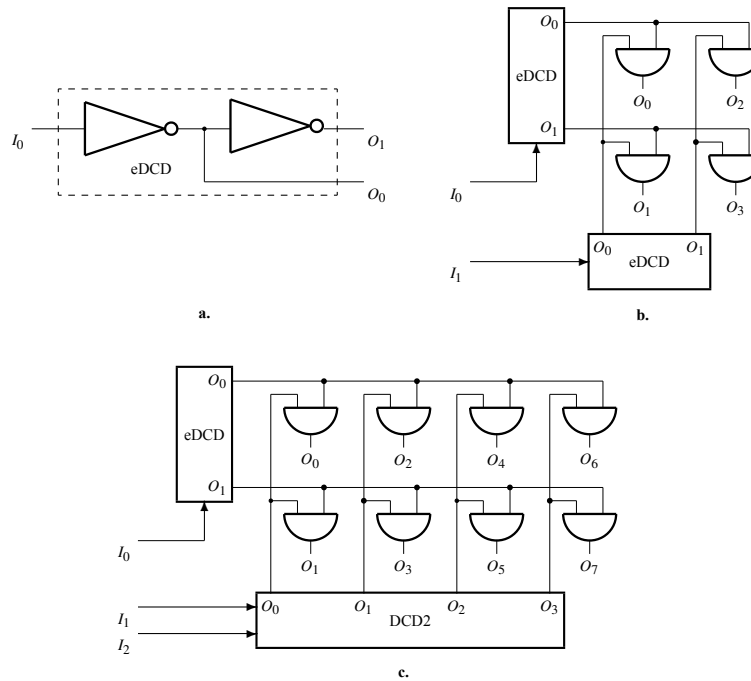


Figure 8.9: Recursively defined 3-input decoder (DCD). **a.** Elementary decoder (eDCD). **b.** Two-input decoder (DCD2). **c.** Three-input DCD (DCD3).

Decoders are widely used in digital systems for applications such as memory addressing, instruction decoding in processors, and enabling control signals in multiplexers.

8.7.1.1 Decoder (SystemVerilog)

```
8 module Decoder #(
```

```

9     parameter INPUT_WIDTH = 4,                // Number of input bits
10    parameter OUTPUT_WIDTH = (1 << INPUT_WIDTH) // Number of output
        bits (2^INPUT_WIDTH)
11 ) (
12     input  logic [INPUT_WIDTH-1:0] in,        // Binary input
13     output logic [OUTPUT_WIDTH-1:0] out       // Decoded outputs
14 );
15
16     always_comb begin
17         out = 1'b1 << in; // Shift the bit based on the input value
18     end
19
20 endmodule

```

Listing 8.6: Parameterized Decoder (SystemVerilog: dcdParameterized.sv)

Listing 8.6 shows a parameterized decoder implemented in SystemVerilog, which provides a scalable solution for decoding binary inputs to a one-hot encoded output. The design uses the shift operation to map the binary input to a specific bit position in the output. By using the parameter `INPUT_WIDTH`, the module can dynamically adjust the size of the binary input and its corresponding one-hot output, with the output width automatically calculated as $2^{\text{INPUT_WIDTH}}$. This ensures that the decoder is flexible and reusable across various designs.

The key operation in the decoder is the left shift, expressed as `out = 1'b1 << in`, where the binary input `in` determines how many positions the single high bit is shifted. This eliminates the need for case statements or nested conditional logic. The shift operation is synthesis-friendly, allowing hardware tools to map the logic to a simple decoder circuit directly.

```

8 module dcd( output logic [15:0]    dcdOut  ,
9             input  logic [3:0]    dcdIn   );
10
11     assign dcdOut = 1 << dcdIn ;
12 endmodule

```

Listing 8.7: 4-to-16 Behavioral Decoder (SystemVerilog:dcd.sv)

Listing 8.7 shows a directly implemented 4-to-16 decoder.

8.7.1.2 Decoder (Chisel)

```

9 class Decoder(n: Int) extends Module {
10     val io = IO(new Bundle {
11         val input = Input(UInt(log2Ceil(n).W)) // Input signal, width
            depends on n
12         val output = Output(UInt(n.W))         // Output is n bits wide
13     })
14
15     // Default: all outputs are 0
16     io.output := 0.U

```

```

17
18 // Decode input to one-hot output
19 io.output := 1.U << io.input
20 }

```

Listing 8.8: Parameterized Decoder (Chisel: Decoder.scala)

Listing 8.8 shows a Chisel parameterized decoder that can generate decoder logic based on a specified width. The decoder accepts a parameter, `n` the output width. This is also used to determine the number of input bits using the function `log2Ceil(n)`. This parameter dynamically adjusts the size of the output vector to `1 << inputWidth` (i.e., $2^{\text{inputWidth}}$), allowing the decoder to handle a variable number of input combinations. The use of a shift operation, `1.U << io.in`, implements the decoding logic by setting a single output bit corresponding to the binary input value.

```

8 class Decoder2to4 extends Decoder(4) {
9 }

```

Listing 8.9: Instantiated 2-to-4 Decoder (Chisel: Decoder2to4.scala)

Listing 8.9 line 8 shows how simple it is to instantiate a decoder of any output width.

```

1 // Generated by CIRCT firtool-1.99.1
2 module Decoder2to4( // /home/jglossner/GitRepos/
   Structured_Computer_Architecture/RTL/Chisel/generators/generated/
   firrtl/Decoder2to4.fir.mlir:3:5
3 input clock, // /home/jglossner/GitRepos/
   Structured_Computer_Architecture/RTL/Chisel/generators/generated/
   firrtl/Decoder2to4.fir.mlir:3:35
4 reset, // /home/jglossner/GitRepos/
   Structured_Computer_Architecture/RTL/Chisel/
   generators/generated/firrtl/Decoder2to4.fir.mlir:3:61
5 input [1:0] io_input, // /home/jglossner/GitRepos/
   Structured_Computer_Architecture/RTL/Chisel/generators/generated/
   firrtl/Decoder2to4.fir.mlir:3:89
6 output [3:0] io_output // /home/jglossner/GitRepos/
   Structured_Computer_Architecture/RTL/Chisel/generators/generated/
   firrtl/Decoder2to4.fir.mlir:3:121
7 );
8
9 assign io_output = 4'h1 << io_input; // /home/jglossner/GitRepos/
   Structured_Computer_Architecture/RTL/Chisel/generators/generated/
   firrtl/Decoder2to4.fir.mlir:3:5, :7:23
10 endmodule

```

Listing 8.10: SystemVerilog Generated for Chisel firtool Decoder(4)

Chisel (or more precisely, firtool) automatically generates SystemVerilog for parameterized modules, while guaranteeing combinational logic. Listing 8.10 shows the generated verilog code for a 2-to-4 decoder `Decoder(4)`.


```

1  module Decoder2to4 (
2      clock,
3      reset,
4      io_input,
5      io_output
6  );
7      input clock;
8      input reset;
9      input [1:0] io_input;
10     output wire [3:0] io_output;
11     assign io_output = 4'h1 << io_input;
12 endmodule

```

Listing 8.11: Verilog Generated by sv2v for Decoder(4)

Commercial tools such as Vivado can directly elaborate and synthesize SystemVerilog. yosys, an open-source tool, can elaborate and synthesize some SystemVerilog statements but not all of them. We convert SystemVerilog to pure verilog using the open-source sv2v tool. Listing 8.11 line 11 shows that the shift semantics of the decoder are preserved in the generated verilog.

```

1  /* Generated by Yosys 0.48+5 (git sha1 7a362f1f7, clang++ 18.1.8 -fPIC
   -O3) */
2
3  (* top = 1 *)
4  (* src = "generators/generated/verilog_sv2v_clean/Decoder2to4.v
   :1.1-12.10" *)
5  module Decoder2to4(clock, reset, io_input, io_output);
6      (* src = "generators/generated/verilog_sv2v_clean/Decoder2to4.v
   :7.8-7.13" *)
7      input clock;
8      wire clock;
9      (* src = "generators/generated/verilog_sv2v_clean/Decoder2to4.v
   :9.14-9.22" *)
10     input [1:0] io_input;
11     wire [1:0] io_input;
12     (* src = "generators/generated/verilog_sv2v_clean/Decoder2to4.v
   :10.20-10.29" *)
13     output [3:0] io_output;
14     wire [3:0] io_output;
15     (* src = "generators/generated/verilog_sv2v_clean/Decoder2to4.v
   :8.8-8.13" *)
16     input reset;
17     wire reset;
18     assign io_output[0] = ~(io_input[1] | io_input[0]);
19     assign io_output[1] = io_input[0] & ~(io_input[1]);
20     assign io_output[2] = io_input[1] & ~(io_input[0]);
21     assign io_output[3] = io_input[1] & io_input[0];

```

```
22  endmodule
```

Listing 8.12: Verilog Elaborated by yosys for Decoder(4)

Listing 8.12 shows the elaborated verilog produced by yosys. The shift operation has been transformed into logical operations. Lines 18 through 21 show the elaboration.

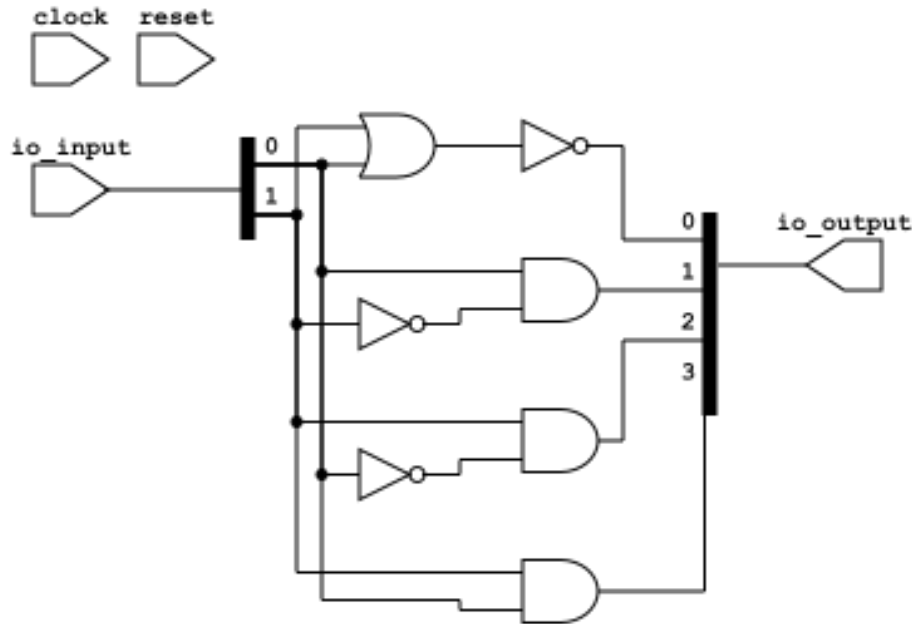


Figure 8.10: Generated Schematic of a yosys synthesized Chisel 2-to-4 Decoder.

After being elaborated by yosys, the schematic of 8.10 is automatically generated by netlistsvg.

8.7.1.3 Decoder Language Comparison

Chisel is a hardware construction language that generates hardware descriptions (e.g., Verilog) through the execution of Scala code. Its parameterized decoder uses functional programming paradigms to adjust the number of output bits based on the input width. The parameterized `Decoder` in Chisel is evaluated at **generation time**, which corresponds to the compile time of the Scala program, not the hardware synthesis. When the Chisel code is executed, the Scala runtime evaluates the parameterization and generates the corresponding Verilog/SystemVerilog code. This generated Verilog/SystemVerilog is then synthesized into hardware, meaning the parameterization is resolved before hardware runtime and does not exist dynamically in the final design.

Verilog lacks native parameterization and higher-level constructs, making it less flexible for scalable designs. A typical Verilog decoder implementation relies on manual enumeration or explicit assignment of output bits using case statements or combinational logic.

SystemVerilog, as an enhancement of Verilog, introduces parameterization capabilities, which allow developers to create reusable and configurable hardware modules. A parameterized module in SystemVerilog uses `parameter` and `generate` constructs. Additionally, SystemVerilog supports advanced

constructs like `logic` and `always_comb`, which enforce combinational behavior. SystemVerilog parameterized designs are evaluated at **synthesis time**, when the hardware description is compiled and mapped into a physical implementation. These constructs allow for flexible and static parameterization, which the synthesis tool resolves before producing the final hardware netlist.

8.7.2 Multiplexer

A **multiplexer** (often abbreviated as *mux*) is a combinational logic circuit that selects one of several input signals and forwards the selected input to a single output line. The input selection is controlled by a set of control signals or *select lines*, which determine which input to pass to the output. A multiplexer with n select lines can control 2^n inputs, making it a highly versatile and compact solution for signal routing in digital systems.

Multiplexers are widely used in digital design due to their ability to manage multiple signals within a system efficiently. In arithmetic circuits, multiplexers are often used for operations like selecting between different data sources or routing data to various processing units. They are also important components in larger systems such as CPUs, where they help implement control logic, ALU operation selection, and pipeline data forwarding. For example, in a microprocessor, a multiplexer might be used to select between input operands for an arithmetic operation based on the current instruction.

Multiplexers are used because they minimize the number of physical connections and wires required in a circuit. By reducing the need for separate connections for each input-output pair, a multiplexer allows for more efficient circuit designs. Additionally, they enhance the scalability of systems, because additional inputs can be accommodated by increasing the number of select lines.

8.7.3 Elementary Multiplexer

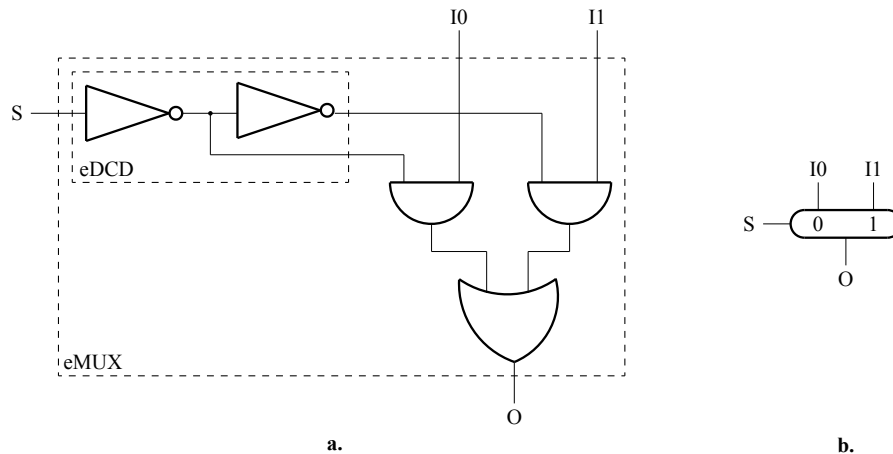


Figure 8.11: Elementary multiplexer (eMUX). **a.** The circuit structure: an eDCD serially connected with an AND-OR circuit. **b.** The logic symbol.

Any combinational circuit can be expressed using only NAND functions. But a more interesting "fundamental brick" could be a circuit corresponding to the ubiquitous *if-then-else* statement. It is the elementary multiplexer (eMUX) represented in Figure 8.11, and shows:

$$O = \text{if } (S) \text{ then } I_1 \text{ else } I_0$$

8.7.4 Many-input Multiplexer

Figure 8.12 shows a many-input multiplexer. It takes bits from $m = 2^n$ inputs selected by a n -bit code.

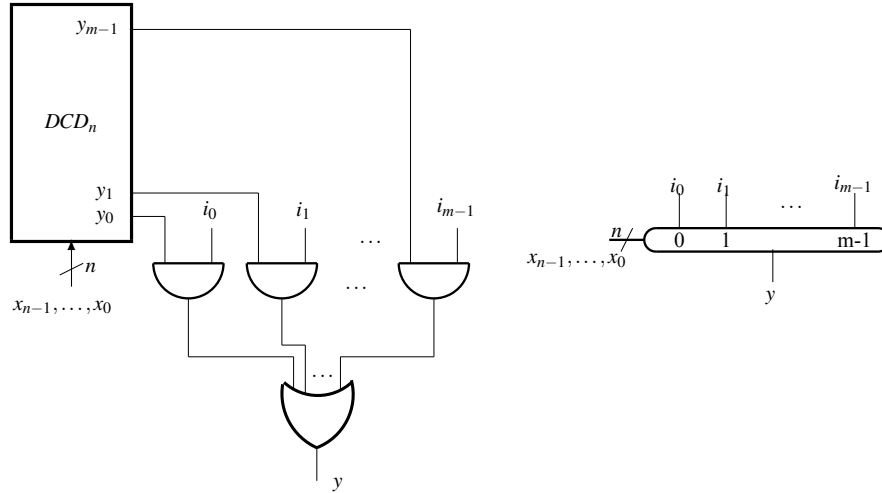


Figure 8.12: The general definition^a of the multiplexer circuit (MUXn). **a.** The structure of MUXn: a DCD_n serially connected with a $m = 2^n$ ANDs on the first level of an AND-OR structure. **b.** The logic symbol for MUXn.

8.7.4.1 Multiplexer (SystemVerilog)

```

8  module mux( output logic      muxOut  ,
9              input  logic [15:0] muxIn  ,
10             input  logic [3:0]  muxSel );
11
12      assign muxOut = muxIn[muxSel] ;
13  endmodule

```

Listing 8.13: 16-to-1 single bit Multiplexer (SystemVerilog: mux.sv)

Listing 8.13 implements a 16-to-1 single bit multiplexer. The module takes three inputs: a 16-bit vector `muxIn`, a 4-bit selector `muxSel`, and produces a single 1-bit output `muxOut`. The selector `muxSel` determines which bit of the 16-bit input vector is routed to the output. This is achieved through the indexing operation `muxIn[muxSel]`, which dynamically selects the bit of `muxIn` corresponding to the value of `muxSel`.

This design is both compact and efficient, leveraging Verilog's built-in array indexing capabilities. It allows the multiplexer to be easily scaled by adjusting the widths of `muxIn` and `muxSel`. Such multiplexers are fundamental building blocks in digital systems, often used to select between multiple data sources or routes in logic circuits, enabling dynamic and efficient data flow management.

8.7.4.2 Parameterized Bus Multiplexer (Chisel)

A *bus* in digital systems is a collection of parallel lines or pathways used to transfer data between components within a system. Each line in a bus corresponds to a bit in the data, allowing the bus to carry multi-bit data simultaneously. Buses are parameterized by their width, which determines how many bits can be transmitted at once, and are used to connect various hardware components, such as processors, memory, and peripherals. For example, a 32-bit bus can carry 32 bits of data in parallel.

```
9 class MultiplexerGenericType[T <: Data](gen: T, val n: Int) extends
  Module {
```

Listing 8.14: Parameterized Bus Type Multiplexer (Chisel: MultiplexerGenericType.scala)

Listing 8.14 shows a parameterized bus multiplexer in Chisel where the type of the bus is also parameterized using `[T <: Data](gen: T)`. In Chisel, parameterizing modules with type parameters is a common practice to create reusable and modular designs. The choice between using `gen` and `dataType` depends on the context and the semantics of the module being designed. Each serves a slightly different purpose depending upon the intended hardware abstraction.

The `gen` parameter is well-suited when the type represents a *blueprint* for generating multiple hardware elements. This is particularly relevant in cases where the module processes or instantiates multiple instances of the same type, such as multiplexers or reusable modules like FIFO queues and bus interfaces. For example, in Listing 8.14, a multiplexer uses `gen` to define the type of data inputs and outputs, emphasizing that multiple identical instances are being handled within the hardware structure.

Given scala's generic types and type checking capabilities, it would be desirable to parameterize the bus type. Unfortunately, when generating hardware, this can sometimes appear as an invalid type conversion even if all interfaces are instantiated with `UInt<>`. Therefore, ***in this book, we have specified all classes to use UInt***. ***This more closely aligns to verilog wires***. Where possible, inside implementations, we still strive to ensure type safety using conversions such as `.asBool` and `.asSInt`, etc.

```
9 class Multiplexer(val n: Int, val width: Int) extends Module {
10   require(n > 0, "The number of inputs must be greater than zero.")
11   require(isPow2(n), "The number of inputs must be a power of two.")
12
13   val io = IO(new Bundle {
14     val inputs = Input(Vec(n, UInt(width.W))) // `n` inputs of type `
15     val select = Input(UInt(log2Ceil(n).W)) // Select line
16     val output = Output(UInt(width.W)) // Output of type `UInt`
17   })
18
19   io.output := io.inputs(io.select) // Select which input
20 }
```

Listing 8.15: Parameterized Bus Width Multiplexer (Chisel: Multiplexer.scala)

Listing 8.15 shows a bus multiplexer implemented as a class named `class Multiplexer`. This class is parameterized by the number of inputs (`n`) and the width of the bus (`width`). As shown on lines 14 through 16, all inputs and outputs are of type `UInt`. The class ensures through `require` statements

that the number of inputs is both greater than zero and a power of two.

The I/O bundle defines the interface for the multiplexer. It includes a vector (bus) of inputs, a selection signal, and the output. The size of the selection signal is calculated using `log2Ceil`, ensuring that it has enough bits to address all the inputs. The core logic of the multiplexer is implemented in a single line `io.output := io.inputs(io.select)`, where the output is assigned the value of the selected input. This is performed using Chisel's vector indexing.

In Scala, a companion object is an object that shares the same name as a class and is defined in the same file. It acts as a singleton instance and provides a convenient way to group static-like methods and values associated with the class. Companion objects are particularly useful for factory methods, constants, or utility functions related to the class. Since they are in the same scope, companion objects and their corresponding classes can access each other's private members. This feature allows Scala developers to maintain clean and organized code, while still supporting the functional programming paradigm by avoiding the need for static members like in Java.

```

22 // Companion object to simplify instantiation
23 object Multiplexer {
24   def apply(inputs: Vec[UInt], select: UInt): UInt = {
25     require(inputs.nonEmpty, "Inputs cannot be empty.")
26     val mux = Module(new Multiplexer(inputs.length, inputs(0).getWidth)
27                       )
28     mux.io.inputs := inputs
29     mux.io.select := select
30     mux.io.output
31   }

```

Listing 8.16: Parameterized Bus Width Multiplexer Companion Object (Chisel: Multiplexer.scala)

Listing 8.16 shows the companion object. The `Multiplexer object` provides a factory method for creating and connecting instances of a parameterized multiplexer. The method takes a `Vec[UInt]` of inputs, and a `UInt` that specifies the selection signal for the multiplexer. After the module is instantiated, its I/O ports are connected. The `mux.io.inputs` port is connected to the `inputs` vector, and the `mux.io.select` port is connected to the `select` signal. Finally, the `apply` method returns the `mux.io.output`, providing the result of the multiplexer to the caller.

```

8 class Mux4 extends Module {
9   val io = IO(new Bundle {
10     val inputs = Input(Vec(4, UInt(8.W))) // Four 8-bit inputs
11     val select = Input(UInt(2.W))         // 2-bit selector
12     val output = Output(UInt(8.W))        // Single 8-bit output
13   })
14
15   // Use the Multiplexer companion object for simplicity
16   io.output := Multiplexer(io.inputs, io.select)
17 }

```

Listing 8.17: 4-to-1 8-bit Bus Multiplexer (Chisel: Mux4.scala)

Listing 8.17 shows an instantiation of a multiplexer with four 8-bit inputs. It uses the companion

object to connect the signals. If the I/O signals retain the same names as the Multiplexer class, the 4-to-1 8-bit bus multiplexer could have been instantiated the same way as the 2-to-4 decoder shown in Listing 8.9 - class `Mux4` extends `Multiplexer(4)`. However, it is often more convenient for users of hardware libraries to see the I/O definitions in the leaf class. Generally, we try to follow this design principle for instantiated Chisel hardware and so Listing 8.17 fully specifies the I/O signals.

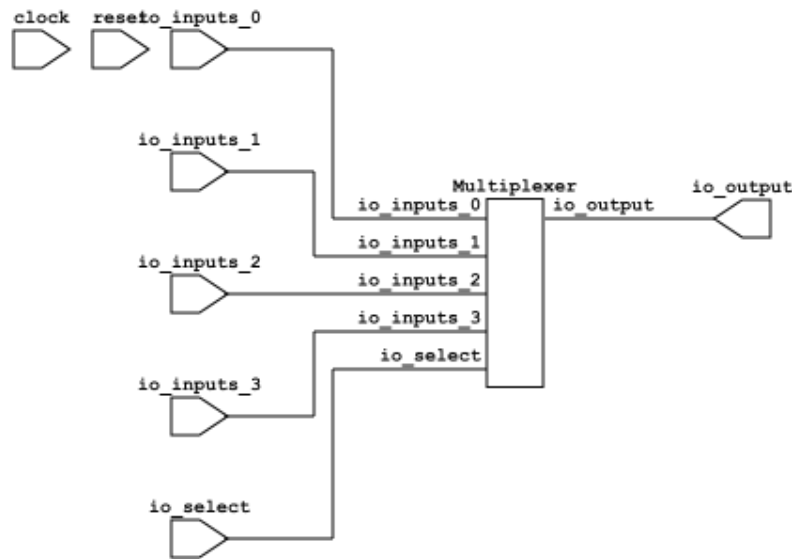


Figure 8.13: Generated Schematic of a Yosys Elaborated Chisel 4-to-1 Multiplexer.

Figure 8.13 shows the hierarchical schematic after Yosys elaboration.

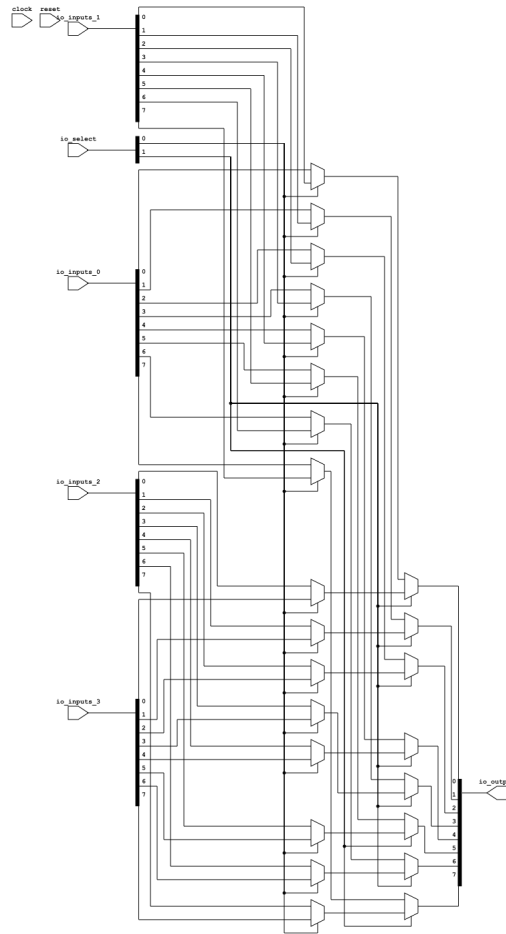


Figure 8.14: Generated Schematic of a Yosys Elaborated and Flattened Chisel 4-to-1 Multiplexer.

Figure 8.14 shows the schematic after the elaborated design is flattened. It is built from multiple 2-to-1 multiplexers.

8.7.5 Demultiplexer

A demultiplexer, often abbreviated as "demux," is a combinational circuit that takes a single input and channels it into one of several output lines based on the value of a selection signal. It is effectively the reverse operation of a multiplexer, consolidating multiple inputs into a single output. A demultiplexer is characterized by its ability to control the routing of the input data to the desired output line, ensuring that only one output is active at a time. The selection signal's binary value governs the active output selection.

Demultiplexers are useful in digital systems for tasks requiring data distribution. For instance, they are employed in scenarios where a single source of information needs to be directed to one of many destinations. In data routing applications, demultiplexers help in reducing the complexity of data paths and enable efficient communication between components. Moreover, they are integral to memory addressing systems, where they are used to select specific memory locations for reading or writing operations.

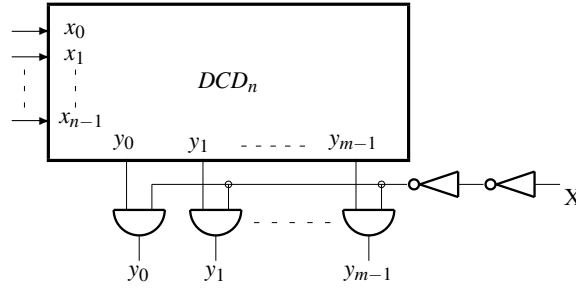


Figure 8.15: Demultiplexer as a DCD whose outputs are ANDed with the one bit signal to be distributed according to the n -bit input code.

Figure 8.15 shows the demultiplexer transfers the input signal X to one of the $m = 2^n$ outputs, $y_0, y_1, \dots, y_{m-1}$, indicated by the n -bit selection input, $x_0, x_1, \dots, x_{n-1}$. In Figure 8.15 a demultiplexer is built using a decoder which open only one of the m AND gate sending the X input to the selected output.

For example: if

$$\{x_{n-1}x_{n-2}, \dots, x_1, x_0\} = 00\dots 011$$

then

$$\{y_{m-1}y_{m-2} \dots y_0 y_1\} = 00\dots 0X000$$

8.7.5.1 Demultiplexer (SystemVerilog)

```

8  module dMux(output logic [15:0]    dMuxOut    ,
9         input  logic [3:0]         dMuxIn      ,
10        input  logic               dMuxEnable );
11
12        assign dMuxOut = dMuxEnable << dMuxIn ;
13  endmodule

```

Listing 8.18: 1-to-16 bit Demultiplexer (SystemVerilog: dMux.sv)

Listing 8.18 shows SystemVerilog code that implements a simple demultiplexer module, named `dMux`. This demultiplexer takes a input signal `dMuxEnable` and routes one-bit of it to one of the 16 output lines, represented by the vector `dMuxOut`, based on the 4-bit selection signal `dMuxIn`. Instead of left-shifting the number 1 as in the decoder, the one-bit value X is left-positioned. The operation is performed using a left-shift operation, where the input signal `dMuxEnable` is shifted by the value of `dMuxIn`, effectively enabling only one bit in the 16-bit output vector.

The `dMuxEnable` input acts as a control signal, ensuring that no outputs are active when it is low.

8.7.5.2 Parameterized Demultiplexer (Chisel)

```

9  class Demultiplexer(numOutputs: Int, width: Int) extends Module {
10    require(isPow2(numOutputs), "Number of outputs must be a power of 2")

```

```

11
12   val io = IO(new Bundle {
13       val input = Input(UInt(width.W))           // Single `
14       val select = Input(UInt(log2Ceil(numOutputs).W)) // Select
15       val outputs = Output(Vec(numOutputs, UInt(width.W))) // Vec of `
16   })
17
18   // Default all outputs to zero
19   io.outputs.foreach(_ := 0.U)
20
21   // Drive the selected output
22   io.outputs(io.select) := io.input
23 }
24
25 // Companion object for easier instantiation
26 object Demultiplexer {
27     def apply(input: UInt, select: UInt, numOutputs: Int): Vec[UInt] = {
28         require(numOutputs > 0, "Number of outputs must be greater than
29             zero.")
30         require(isPow2(numOutputs), "Number of outputs must be a power of
31             2.")
32         val demux = Module(new Demultiplexer(numOutputs, input.getWidth))
33         demux.io.input := input
34         demux.io.select := select
35         demux.io.outputs
36     }
37 }

```

Listing 8.19: Parameterized Bus Demultiplexer (Chisel: Demultiplexer.scala)

Listing 8.19 shows a parameterized demultiplexer (demux) that can handle an arbitrary number of outputs of type `Vec[UInt]`. The `Demultiplexer` class that extends Chisel’s `Module` class to create a hardware module. The module includes a requirement that the number of outputs must be a power of two, enforced by the `require` statement.

The `io` bundle defines the interface for the module, consisting of an input signal of type `UInt(width.W)`, a selection signal of width $\log_2(\text{numOutputs})$, and a vector of `numOutputs` output signals. The implementation defaults all output signals to zero using `foreach`, ensuring that only the selected output is driven by the input signal. This is achieved by indexing the output vector with the `select` signal, which determines the active output line.

Similar to the `Multiplexer`, a companion object provides a convenient factory method for instantiating the demultiplexer. The `apply` method creates an instance of the `Demultiplexer` class and connects the provided input and selection signals to the instance’s IO ports.

```

9   class DeMux16 extends Module {
10       val io = IO(new Bundle {
11           val input = Input(UInt(64.W))           // 64-bit input bus

```

```
12     val select = Input(UInt(4.W))      // 4-bit selector for 16 outputs
13     val outputs = Output(Vec(16, UInt(64.W))) // 16 output buses, each
        64-bit
14   })
15
16   // Instantiate the 1-to-16 demultiplexer
17   io.outputs := Demultiplexer(
18     input = io.input,
19     select = io.select,
20     numOutputs = 16
21   )
22 }
```

Listing 8.20: 1-to-16 64-bit Bus Demultiplexer (Chisel: DeMux16.scala)

Listing 8.20 shows a DeMux16 class that defines a 1-to-16 demultiplexer that operates on 64-bit buses. This module is implemented as a subclass of Module, making it a Chisel hardware component. Its interface, specified by the io bundle, includes a 64-bit input bus (io.input), a 4-bit selection signal (io.select) to determine the active output, and a vector of 16 output buses (io.outputs), each 64 bits wide. The selection signal uses a 4-bit width because $\log_2(16) = 4$, allowing indexing of the 16 outputs.

The DeMux16 class instantiates a parameterized demultiplexer from the Demultiplexer companion object. The input and selection signals (io.input and io.select) are passed as arguments to the Demultiplexer factory method, which returns a vector of 16 outputs. These outputs are directly assigned to io.outputs, ensuring that only the selected output is driven by the input signal, while the others remain zero.

8.7.6 Seven-Segment Display

A 7-segment display is a common example of a combinational circuit that illustrates the concept of many-input logic gates. It consists of seven individual segments, labeled a through g, which can be illuminated to form numerical digits (0–9). The control logic for a 7-segment display is designed as a combinational circuit that takes a binary-coded decimal (BCD) input, typically 4 bits, and determines which segments should be activated to display the corresponding digit. Each segment is controlled by a Boolean expression derived from the binary input, effectively making the display a complex many-input gate. The circuit highlights how combinational logic can efficiently decode binary inputs into human-readable visual representations.

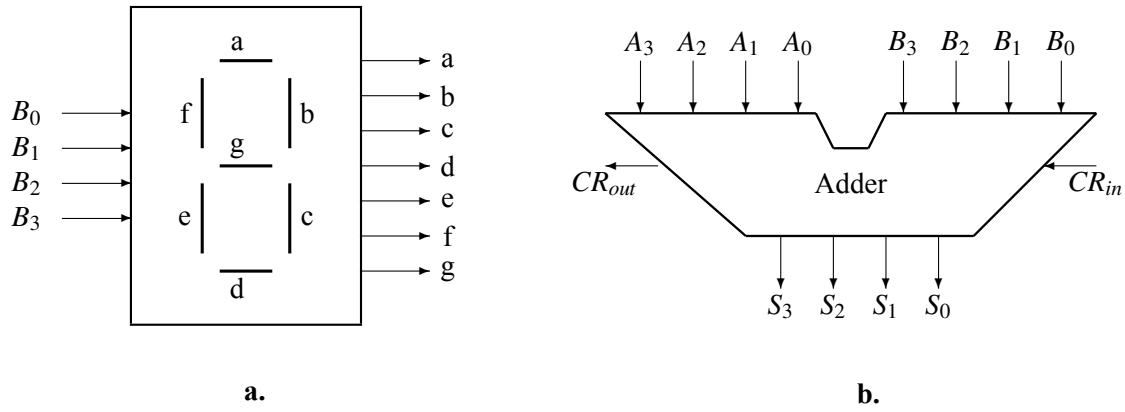


Figure 8.16: The version of digital circuits. **a.** Logic circuit: trans-coder for seven-segment display. **b.** Numeric circuit: four-bit numbers adder.

Figure 8.16 shows the transcoder for a 7-segment display. It receives four-bit coded decimal numbers, from 0 to 9, and convert them in 7-bit codes, a to g, indicating the segments to be activated, according to Table 8.5.

The logic expression for the output a is:

$$a = B'_3 \cdot B'_2 \cdot B'_1 \cdot B'_0 + B'_3 \cdot B'_2 \cdot B_1 \cdot B'_0 + B'_3 \cdot B'_2 \cdot B_1 \cdot B_0 + B'_3 \cdot B_2 \cdot B'_1 \cdot B_0 + B'_3 \cdot B_2 \cdot B_1 \cdot B'_0 + B'_3 \cdot B_2 \cdot B_1 \cdot B_0 + B_3 \cdot B'_2 \cdot B'_1 \cdot B'_0 + B_3 \cdot B'_2 \cdot B'_1 \cdot B_0$$

or a simpler version by negatin the negated function:

$$a = (a')' = (B'_3 \cdot B'_2 \cdot B'_1 \cdot B_0 + B'_3 \cdot B_2 \cdot B'_1 \cdot B'_0)' = (B'_3 \cdot B'_1 \cdot (B_2 \oplus B_0))' = B_3 + B_1 + (B_2 \oplus B_0)'$$

Table 8.5: Seven-segment display truth table.

	B_3	B_2	B_1	B_0	a	b	c	d	e	f	g
0	0	0	0	0	1	1	1	1	1	1	
1	0	0	0	1		1	1				
2	0	0	1	0	1	1		1	1		1
3	0	0	1	1	1	1	1	1			1
4	0	1	0	0		1	1			1	1
5	0	1	0	1	1		1	1		1	1
6	0	1	1	0	1		1	1	1	1	1
7	0	1	1	1	1	1	1				
8	1	0	0	0	1	1	1	1	1	1	1
9	1	0	0	1	1	1	1	1		1	1

8.7.6.1 Seven Segment Display (SystemVerilog)

```

7  module sevenSegmDys(output logic [6:0] seg      ,
8                      input  logic [3:0] number );
9
10     always_comb begin
11         case(number)
12             4'b0000: seg = 7'b1111110; // 0: a, b, c, d, e, f
13             4'b0001: seg = 7'b0110000; // 1: b, c
14             4'b0010: seg = 7'b1101101; // 2: a, b, g, e, d
15             4'b0011: seg = 7'b1111001; // 3: a, b, g, c, d
16             4'b0100: seg = 7'b0110011; // 4: f, g, b, c
17             4'b0101: seg = 7'b1011011; // 5: a, f, g, c, d
18             4'b0110: seg = 7'b1011111; // 6: a, f, g, e, d, c
19             4'b0111: seg = 7'b1110000; // 7: a, b, c
20             4'b1000: seg = 7'b1111111; // 8: All segments on
21             4'b1001: seg = 7'b1111011; // 9: a, f, b, g, c, d
22             default: seg = 7'b0000000; // Default: All segments off
23         endcase
24     end
25 endmodule

```

Listing 8.21: Seven Segment Display (SystemVerilog: SevenSegmentDisplay.sv)

Listing 8.21 shows SystemVerilog code that implements a seven-segment display decoder. The module, named `sevenSegmDys`, has a 4-bit binary input, `number`, and a 7-bit output, `seg`, which controls the state of the seven segments (labeled a through g) of the display.

The `always_comb` block defines a combinational block that describes the behavior of the output of the circuit as a combination based on the value of the input variables. The blocking assignment, `=`, is used to assure the combinational behavior.

The case statement maps each possible input value (0 through 9) to a corresponding 7-bit output pattern that activates the appropriate segments to display the digit. For example, the binary input `4'b0000` (decimal 0) results in the output `7'b1111110`, which lights up all segments except g, forming the number 0 on the display.

The module includes a `default` case that sets all segments to off (`7'b0000000`) for invalid inputs outside the range 0–9. By mapping each input to a fixed segment configuration, this module can be integrated into larger digital systems that require visual representation of numeric values, such as timers, calculators, or embedded control systems.

8.7.6.2 Seven Segment Display (Chisel)

```

8  class SevenSegmentDisplay extends Module {
9      val io = IO(new Bundle {
10         val binIn = Input(UInt(4.W)) // 4-bit binary input
11         val segOut = Output(UInt(7.W)) // 7-bit output for seven-segment
12         display
13     })
14     // Assign names to the binary I/O bits

```

```

15     val B0 = WireDefault(io.binIn(0)).suggestName("B0")
16     val B1 = WireDefault(io.binIn(1)).suggestName("B1")
17     val B2 = WireDefault(io.binIn(2)).suggestName("B2")
18     val B3 = WireDefault(io.binIn(3)).suggestName("B3")
19
20     // Assign names to the 7-segment output bits
21     val a = WireDefault(io.segOut(6)).suggestName("a")
22     val b = WireDefault(io.segOut(5)).suggestName("b")
23     val c = WireDefault(io.segOut(4)).suggestName("c")
24     val d = WireDefault(io.segOut(3)).suggestName("d")
25     val e = WireDefault(io.segOut(2)).suggestName("e")
26     val f = WireDefault(io.segOut(1)).suggestName("f")
27     val g = WireDefault(io.segOut(0)).suggestName("g")
28
29     // Output decoding
30     // 0: a, b, c, d, e, f
31     // 1: b, c
32     // 2: a, b, g, e, d
33     // 3: a, b, g, c, d
34     // 4: f, g, b, c
35     // 5: a, f, g, c, d
36     // 6: a, f, g, e, d, c
37     // 7: a, b, c
38     // 8: All segments on
39     // 9: a, f, b, g, c, d
40     // Default: All segments off
41
42     io.segOut := "b0000000".U(7.W) // Default: All segments off
43
44     when(      io.binIn === 0.U) { io.segOut := "b1111110".U(7.W)
45     } .elsewhen(io.binIn === 1.U) { io.segOut := "b0110000".U(7.W)
46     } .elsewhen(io.binIn === 2.U) { io.segOut := "b1101101".U(7.W)
47     } .elsewhen(io.binIn === 3.U) { io.segOut := "b1111001".U(7.W)
48     } .elsewhen(io.binIn === 4.U) { io.segOut := "b0110011".U(7.W)
49     } .elsewhen(io.binIn === 5.U) { io.segOut := "b1011011".U(7.W)
50     } .elsewhen(io.binIn === 6.U) { io.segOut := "b1011111".U(7.W)
51     } .elsewhen(io.binIn === 7.U) { io.segOut := "b1110000".U(7.W)
52     } .elsewhen(io.binIn === 8.U) { io.segOut := "b1111111".U(7.W)
53     } .elsewhen(io.binIn === 9.U) { io.segOut := "b1111011".U(7.W)
54     }
55 }

```

Listing 8.22: Seven Segment Display (Chisel: SevenSegmentDisplay.scala)

Listing 8.22 shows the `SevenSegmentDisplay` class is a Chisel module that decodes a 4-bit binary input (`binIn`) into a 7-bit output (`segOut`) representing the activation of segments on a seven-segment display. The input, `binIn`, can take values from 0 to 9, and the module determines which segments (labeled a through g) should be illuminated to form the corresponding numeric digit on the display.

The `suggestName` method in Chisel provides a mechanism to assign meaningful, user-defined names to intermediate signals or wires within a hardware design. By default, Chisel generates arbitrary names for these components, which can make debugging and analyzing generated hardware descriptions (e.g.,

Verilog or FIRRTL) challenging. Using `suggestName`, designers can explicitly label signals with descriptive names that convey their purpose or function within the design, making the generated hardware more readable and easier to trace.

The decoding logic is implemented using a series of `when` and `elsewhen` constructs. Each condition compares `binIn` with a specific value and assigns a 7-bit binary pattern to `segOut`. Each bit in the pattern corresponds to one of the seven segments, where a 1 turns the segment on, and a 0 turns it off. For example, when `binIn` is 0, the output is 1111110, which activates all segments except `g` to display the digit 0. Similarly, for `binIn = 8`, all segments are activated (1111111). A default value of 0000000 is set for `segOut` to ensure all segments are off in case of invalid input.

In Chisel 6.6, the use of the `switch` function is discouraged because it often leads to less modular and harder-to-read code, especially in large designs. The `switch` function can introduce an implicit and tightly coupled relationship between the control signal and the cases, reducing flexibility and increasing the risk of errors during refactoring. Instead, Chisel promotes the use of more functional constructs like pattern matching or chained `when` statements which makes the design less prone to subtle bugs.

```
122     assign a = _a_T;  
123     assign b = _b_T;  
124     assign c = _c_T;  
125     assign d = _d_T;  
126     assign e = _e_T;  
127     assign f = _f_T;  
128     assign g = _g_T;
```

Listing 8.23: Propagation of `suggestName()` after `yosys` Elaboration (Generated SystemVerilog: `yosys/SevenSegmentDisplay.v`)

The snippet in Listing 8.23 shows that the signal names are propagated through to the synthesized design.

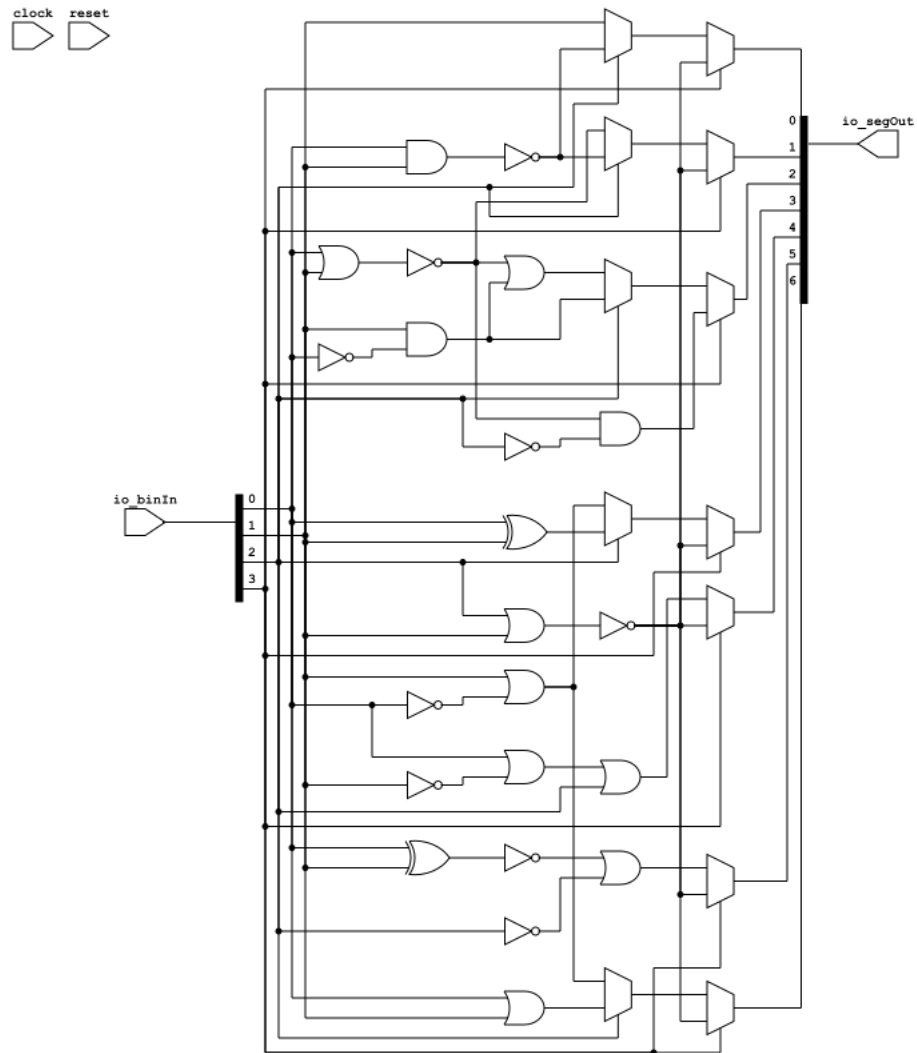


Figure 8.17: Generated Schematic of SevenSegmentDisplay Using when statements.

Figure 8.17 shows the schematic after yosys elaboration. Unfortunately, netlistsvg does not retain the names even though they are in the generated verilog file.

8.7.6.3 Seven Segment Display Using MuxSelect

In Chisel, there are several constructs that can produce hardware in various forms. See Chapter 15 Section 15.1 for details of using `when`, `vs. switch`, `vs. MuxCase`. From a programmer's point of view, `when` is likely the most familiar. However, *when will priority encode complex signals*. In the case of the Seven Segment Display, where all the conditions are independent, it will generate the signals in parallel.

```

43   io.segOut := MuxCase("b0000000".U(7.W), Seq(
44     (io.binIn === 0.U) -> "b1111110".U(7.W),

```



```

45     (io.binIn === 1.U) -> "b0110000".U(7.W),
46     (io.binIn === 2.U) -> "b1101101".U(7.W),
47     (io.binIn === 3.U) -> "b1111001".U(7.W),
48     (io.binIn === 4.U) -> "b0110011".U(7.W),
49     (io.binIn === 5.U) -> "b1011011".U(7.W),
50     (io.binIn === 6.U) -> "b1011111".U(7.W),
51     (io.binIn === 7.U) -> "b1110000".U(7.W),
52     (io.binIn === 8.U) -> "b1111111".U(7.W),
53     (io.binIn === 9.U) -> "b1111011".U(7.W)
54   ))

```

Listing 8.24: Seven Segment Display Using MuxCase (Chisel: SevenSegmentDisplayMuxCase.scala)

When the designer wants *to guarantee parallel generation of hardware, MuxCase is preferred*. Listing 8.24 shows the Chisel code using MuxCase instead of when.

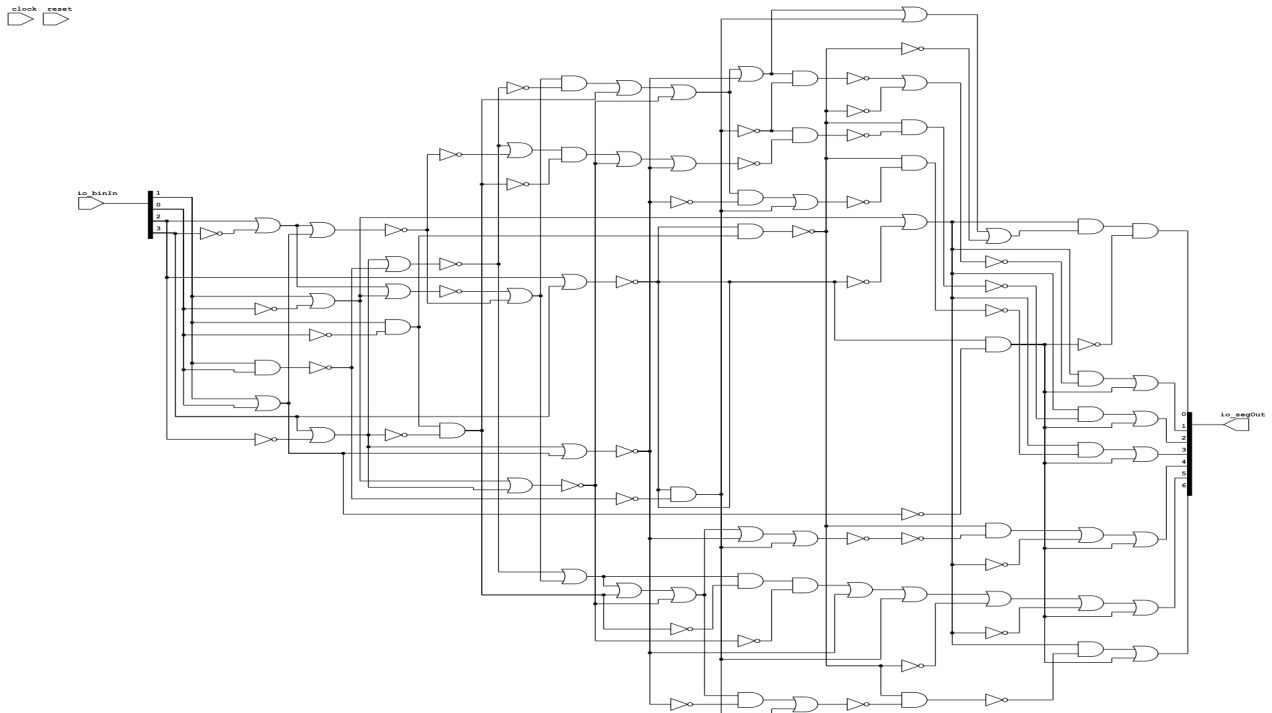


Figure 8.18: Generated Schematic of SevenSegmentDisplay Using MuxCase.

Figure 8.18 shows the schematic generated after yosys synthesis of the MuxCase Chisel code. Note that all multiplexers are reduced to parallel logic. However, it is also a lot of logic.

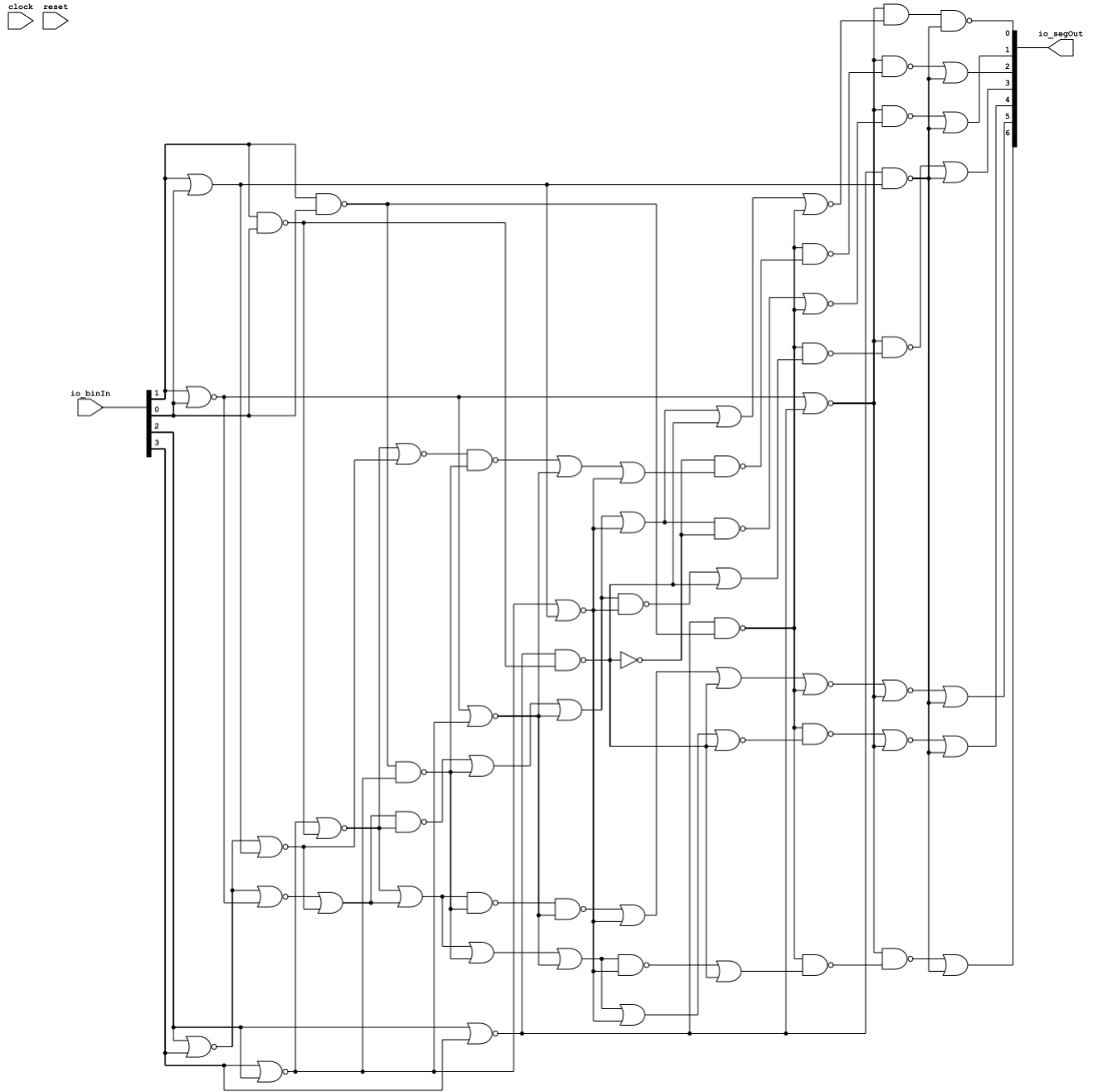


Figure 8.19: Generated Schematic of SevenSegmentDisplay Using MuxCase and Skywater 130nm ASIC Technology files.

Figure 8.19 shows the schematic generated after yosys synthesis of the MuxCase Chisel code with optimization guided by the Skywater CMOS 130nm technology library. Note that most of the gates are now NAND/NOR which are more efficient in CMOS implementations.

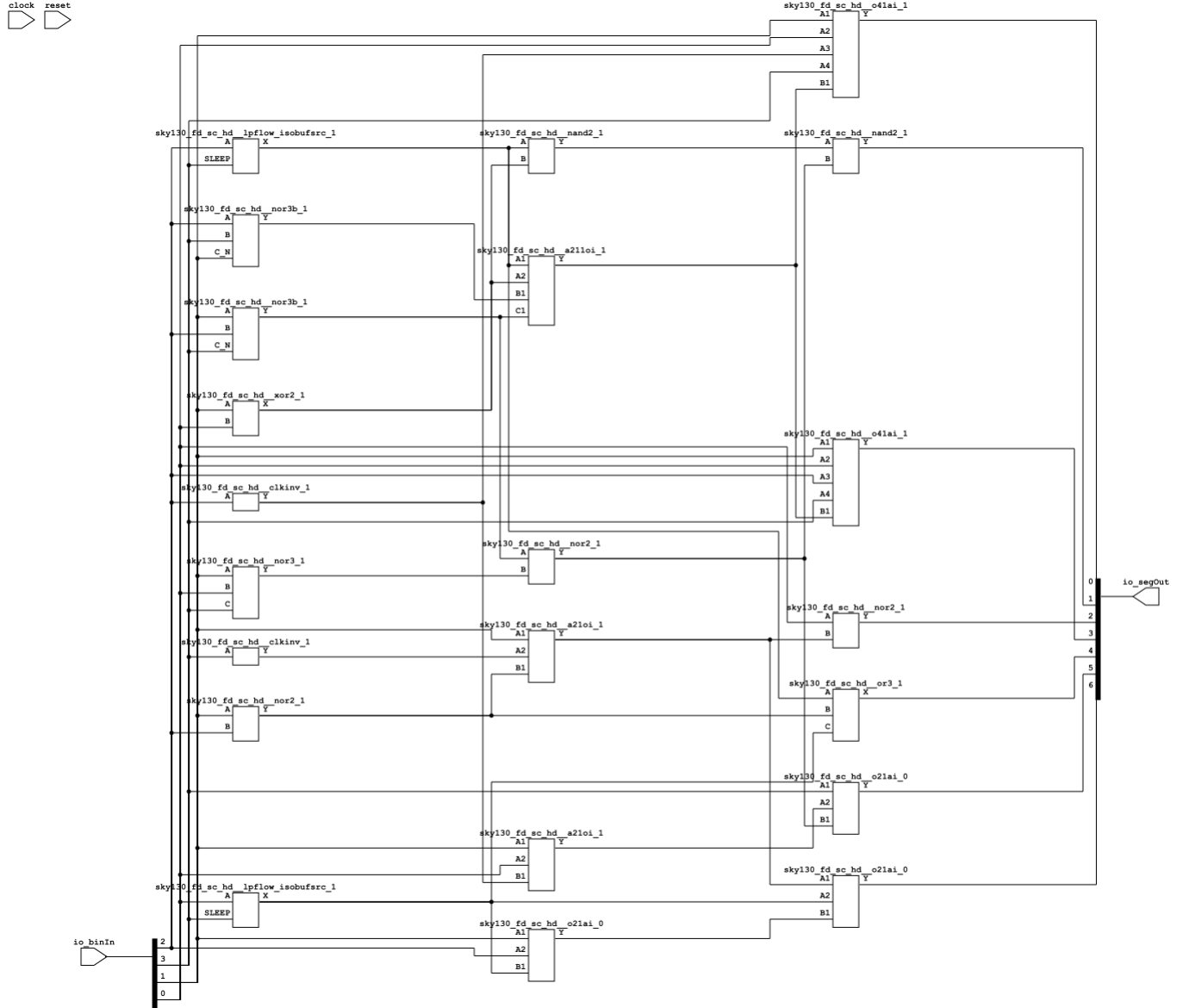


Figure 8.20: Generated Schematic of SevenSegmentDisplay Using MuxCase Showing Exact Standard Cell Skywater 130nm ASIC Cells.

Figure 8.20 shows the schematic generated after yosys synthesis of the MuxCase Chisel code with optimization guided by the Skywater CMOS 130nm technology library and the exact library cells selected from the Skywater library. The library has efficient implementations of 3-input gates. That selection is reflected in the synthesized design.

8.7.6.4 Seven Segment Display Language Comparison

The Chisel and SystemVerilog implementations both utilize conditional logic. Chisel uses `when` statements, while SystemVerilog uses a `case` statement within an `always_comb` block. The primary decoding

functions are both about the same number of lines of code.

SystemVerilog's `always_comb` guarantees that the logic produced is purely combinational, making the designer's intent explicit. In contrast, Chisel does not inherently enforce combinational logic; it depends on how constructs like `when` are used. For instance, `when` can describe sequential behavior if used with registers, but in the given example, the direct use of `:=` ensures combinational behavior.

8.8 Behavioral vs. Structural Descriptions

Digital circuits can be described using two primary approaches: by defining their functional behavior or by specifying their internal structure. A behavioral description focuses on the circuit's input-output relationships, detailing what the circuit does without concern for how it is implemented internally. Conversely, a structural description specifies the arrangement and interconnection of the components that make up the circuit, providing a blueprint for how the desired functionality is achieved.

The design of digital systems is inherently a **hierarchical process**. It begins with a top-level module that represents the overall system and is populated by submodules. Each submodule is defined similarly, with the process continuing until the simplest modules are reached. These basic modules are described directly, either through functional descriptions or by mapping to primitive hardware elements. This hierarchical organization allows for clear abstraction and separation of concerns, enabling designers to manage complexity effectively and focus on smaller, more manageable parts of the system.

One of the defining characteristics of digital design is **modularity**. In this approach, a complex problem is decomposed into smaller, simpler problems that are solved independently. This process is repeated recursively until the simplest possible components are identified and implemented. Modularity facilitates the reuse of identical modules throughout the design, enabling replication of the same component multiple times. This not only reduces development effort but also ensures consistency and reliability across the system. Furthermore, modular design supports scalability and maintainability, as individual modules can be optimized or replaced without affecting the rest of the system.

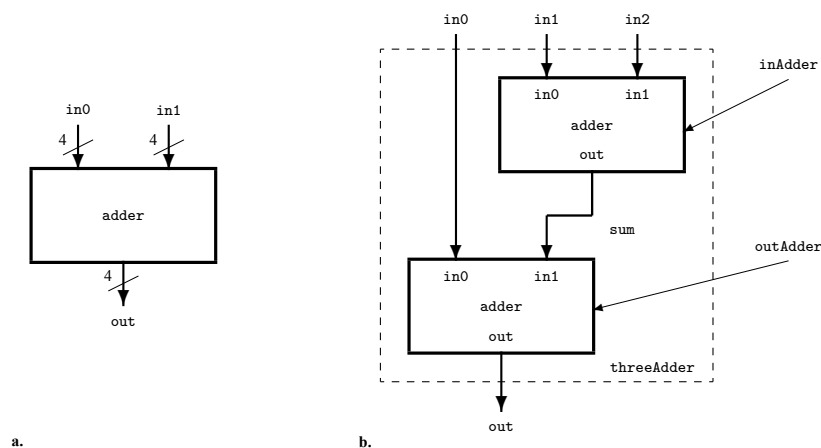


Figure 8.21: **Hierarchical Decomposition Adder Modules.** **a.** Top-level 2-input 4-bit adder. **b.** The structure of a 3-input 4-bit adder.

Figure 8.21a shows an adder module. No indication of how this module is implemented is provided.

We infer from its name that it adds two 4-bit numbers. Figure 8.21b shows a 3-input 4-bit adder composed of two 2-input 4-bit adders. While the module `adder` is a *behavioral* description, the module `threeAdder` is a *structural* one. The first tells us *what* is the function of the module, and the second tells us *how* its functionality is performed by using a structure containing two instantiation of a previously defined subsystems, and an internal connection.

8.9 Adders

Adders are fundamental components in digital computing systems. At the core of processors, adders enable basic addition and form the building blocks for more complex arithmetic units, such as subtractors, multipliers, and dividers. An adder's efficiency directly impacts a processor's overall performance, as addition is one of the most frequently executed operations in computing.

8.9.1 Behavioral Adders

Behavioral adders describe what the circuit should do without specifying how it is implemented at the gate level. In contrast, specific adders, such as a carry-lookahead adder or a Kogge-Stone adder, describe the precise gate-level implementation of the addition operation. When synthesized using a tool like CIRCT, the `BehavioralAdder` would be converted into a hardware implementation that depends on the synthesis tool's optimization strategies. Typically, the synthesis tool would select an efficient adder architecture based on the target hardware platform and constraints. This could include ripple-carry, carry-lookahead, or other optimized implementations. The use of behavioral descriptions provides flexibility, as the specific implementation details are deferred to the synthesis process, enabling the designer to focus on functionality rather than low-level hardware details.

8.9.1.1 Parameterized Abstract Adder (Chisel)

```

8  abstract class Adder(width: Int) extends Module {
9    require(width > 0, "Width must be greater than 0")
10
11   val io = IO(new Bundle {
12     val a = Input(UInt(width.W)) // Input A
13     val b = Input(UInt(width.W)) // Input B
14     val sum = Output(UInt(width.W)) // Output sum
15   })
16 }
```

Listing 8.25: Parameterized Abstract Adder (Chisel: `Adder.scala`)

Listing 8.25 shows code that defines an abstract class `Adder`, which provides a framework for creating parameterized adder modules in Chisel. The class requires a `width` parameter, which specifies the bit-width of the inputs and outputs. The `Adder` class requires a `width` greater than zero to ensure valid hardware design, and this is enforced by the `require` statement. The `io` interface is defined using a `Bundle` that encapsulates three signals: `a`, `b`, and `sum`. The `a` and `b` signals are inputs of type `UInt` with the specified `width`, representing the operands of the adder. The `sum` signal is an output of type `UInt` with the specified `width`, representing the computed sum of the inputs.

The class is marked as `abstract`, which means it cannot be instantiated directly. Instead, specific implementations of the adder, such as ripple-carry or carry-lookahead adders, can extend this base class to reuse the IO definitions and width constraints. Using an abstract base class like `Adder` provides several benefits. It ensures consistency in IO definitions, as all derived adder implementations inherit the same `io` interface, guaranteeing a uniform input-output structure across different adder designs. It promotes reusability by defining the `io` interface and width validation logic once in the base class, reducing code duplication and improving maintainability. The abstract base class introduces a level of abstraction by encapsulating common properties and behaviors, allowing derived classes to focus solely on implementing specific adder designs without redefining shared elements.

8.9.1.2 Precision in Addition

A (bin)	A (dec)	B (bin)	B (dec)	Sum (bin)	Sum (dec)	Overflow
0000	0	0000	0	0000	0	0
0001	1	0010	2	0011	3	0
0111	7	0001	1	1000	8	0
1111	15	0001	1	0000	0	1
1010	10	0110	6	0000	0	1
1110	14	1111	15	1101	13	1
1111	15	1111	15	1110	14	1

Figure 8.22: 4-bit Behavioral Adder Partial Truth Table

Figure 8.22 shows the partial truth table for a 4-bit behavioral adder. The column labeled overflow shows that n -bits is not sufficient to add two n -bit numbers. In general, adding two n -bit binary numbers requires $n + 1$ bits to store the result.

Consider two n -bit binary numbers, A and B , where each can be represented in the form:

$$A = \sum_{i=0}^{n-1} a_i \cdot 2^i, \quad B = \sum_{i=0}^{n-1} b_i \cdot 2^i$$

where $a_i, b_i \in \{0, 1\}$ are the binary digits (bits) of A and B , respectively.

The range of each n -bit number can take any value from 0 to $2^n - 1$. Therefore:

$$0 \leq A \leq 2^n - 1, \quad 0 \leq B \leq 2^n - 1$$

The maximum value of $A + B$ occurs when both A and B are at their maximum value, i.e., $A = 2^n - 1$ and $B = 2^n - 1$. In this case:

$$A + B = (2^n - 1) + (2^n - 1) = 2^n + 2^n - 2 = 2^{n+1} - 2$$

This shows that the result of $A + B$ can range from 0 (minimum) to $2^{n+1} - 2$ (maximum). To store a result as large as $2^{n+1} - 2$, at least $n + 1$ bits are required.

In the case of 4-bit unsigned numbers, observe that each operand, A and B , can represent values from 0 to $2^4 - 1 = 15$. When adding the two largest 4-bit numbers, $A = 15_{base10}$ and $B = 15_{base10}$, the result is $A + B = 30_{base10}$. To represent 30_{base10} in binary, at least 5 bits are required, as the binary representation is 11110_2 .

8.9.1.3 2-input 4-bit Behavioral Adder (SystemVerilog)

```

9  module adder(    output logic [3:0] out ,      // 4-bit output
10                 input  logic [3:0] in0 ,      // 4-bit input
11                 input  logic [3:0] in1 );      // 4-bit input
12
13     assign out = in0 + in1;
14 endmodule

```

Listing 8.26: Behavioral 2-input 4-bit Adder (SystemVerilog: adder.sv)

Listing 8.26 shows a 2-input 4-bit adder implemented in SystemVerilog. It is designed to perform 4-bit unsigned addition. It takes two 4-bit inputs, `in0` and `in1`, and produces a 4-bit output, `out`, representing the sum of the two inputs. The addition operation is performed using the `assign` statement, which continuously evaluates the sum of `in0` and `in1` and assigns the result to `out`.

This module does not include provisions for detecting overflow, which occurs when the result of the addition exceeds the 4-bit range (0 to 15). In such cases, the sum wraps around, effectively discarding the carry bit that exceeds the 4-bit width making it a modulo 16 adder.

8.9.1.4 Parameterized Behavioral Adder (Chisel)

```

8  class BehavioralAdder(width: Int) extends Module {
9      val io = IO(new Bundle {
10         val a = Input(UInt(width.W))    // Input A (UInt)
11         val b = Input(UInt(width.W))    // Input B (UInt)
12         val sum = Output(UInt(width.W)) // Fixed-width sum output
13     })
14
15     // Perform addition and truncate to width
16     io.sum := (io.a + io.b)(width - 1, 0)
17 }

```

Listing 8.27: Parameterized Behavioral Adder (Chisel)

Listing 8.27 shows code that defines a class `BehavioralAdder`, which extends the abstract `Adder` class to implement a simple, behavioral adder in Chisel. All interface IO are of type `UInt`. The class is parameterized by the width of the adder `width`. The addition operation is specified behaviorally using the expression `io.a + io.b`, which computes the sum of the two input signals and assigns it to the output signal `io.sum`.

An important point is that Chisel will expand the result to keep full precision. In simulation, this may be the desired behavior. However, hardware cannot dynamically expand wires. Chisel leaves this decision to the programmer but synthesis requires fixed widths. Our behavioral adder implementation truncates the sum result to the width of the output wires using `(width - 1, 0)` on line 16.

8.9.2 4-bit Behavioral Adder

```

8 class BehavioralAdder4 extends Module {
9   val io = IO(new Bundle {
10     val a = Input(UInt(4.W)) // 4-bit unsigned input
11     val b = Input(UInt(4.W)) // 4-bit unsigned input
12     val sum = Output(UInt(4.W)) // 4-bit unsigned output
13   })
14
15   // Directly use the BehavioralAdder module with UInt
16   io.sum := BehavioralAdder(io.a, io.b, 4)
17 }

```

Listing 8.28: 4-bit Behavioral Adder (Chisel: BehavioralAdder4.scala)

Listing 8.28 shows a 4-bit behavioral adder that instantiates the class BehavioralAdder.

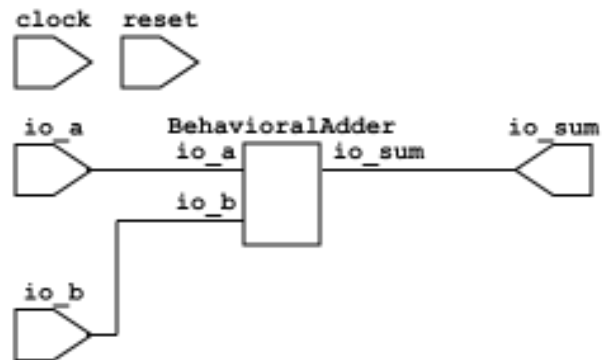


Figure 8.23: High-level Schematic of a Behavioral 4-bit Adder.

After yosys elaboration, both a high-level and flattened circuit diagram can be generated. Figure 8.23 shows the schematic for a high-level schematic. Figure 8.24 shows the schematic for a yosys synthesized and flattened 4-bit adder.

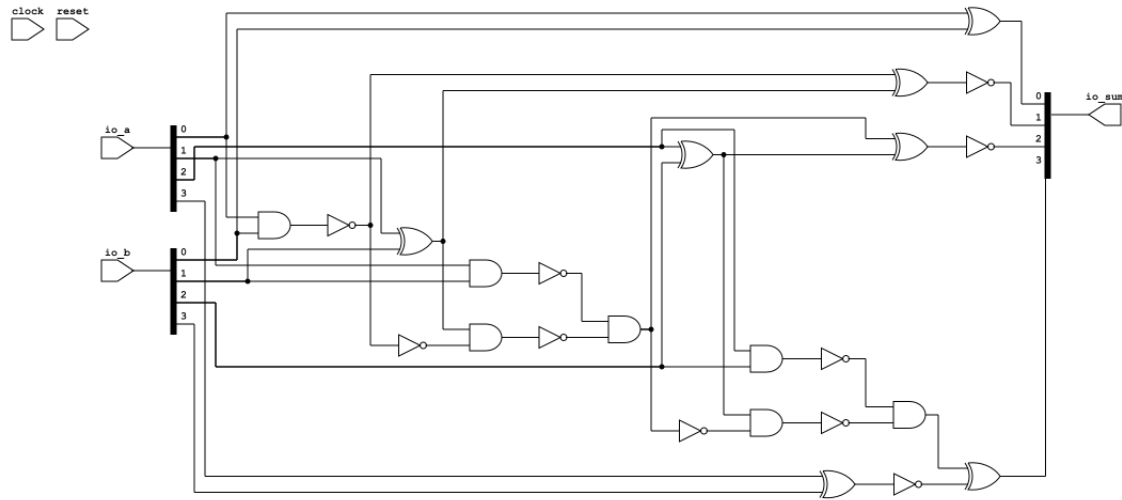


Figure 8.24: Yosys Synthesized and Flattened Schematic of a Behavioral 4-bit Adder.

8.9.3 Structural Adders

Computers use structural adders to perform arithmetic operations efficiently at the hardware level. A structural adder is a circuit composed of interconnected logic gates designed to compute the sum of binary numbers. These adders allow computations to occur in a predictable, deterministic manner, with a fixed propagation delay determined by the design. Unlike behavioral implementations, structural adders operate at the level of individual bits, ensuring that addition is performed in accordance with the system's clock cycle.

The concept of carry bits is central to structural adders. In binary arithmetic, when the sum of two bits exceeds the value that a single bit can represent, the excess is propagated as a carry bit to the next more significant bit. This mechanism ensures accuracy in multi-bit addition, but it also introduces challenges, such as carry propagation delays.

In this Section, we only provide an overview of the simplest structural adders. Practical implementations with higher performance (and complexity) are covered in Chapter 15.

8.9.3.1 Half Adder

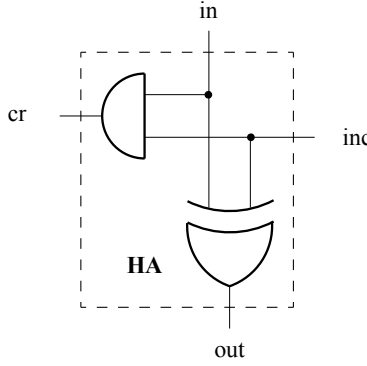


Figure 8.25: Half adder (HA). The XOR circuit compute the *modulo-2* sum while the AND gate compute the carry value.

Figure 8.25 shows the simplest adder - the *half adder*. It adds two binary digits and produces a sum and a carry bit. To build a half adder we use an XOR circuit. The XOR circuit calculates the sum modulo 2 but does not differentiate between 0+0 and 1+1. This differentiation involves the calculation of the arithmetical overshoot, which we denote by carry (to the next binary order). The Carry signal is activated when both inputs are 1. Therefore, in addition to the XOR gate we must add an AND gate. The resulting circuit is called a *half adder* because it does not take into account the carry signal provided by the previous binary order.

8.9.3.2 Increment

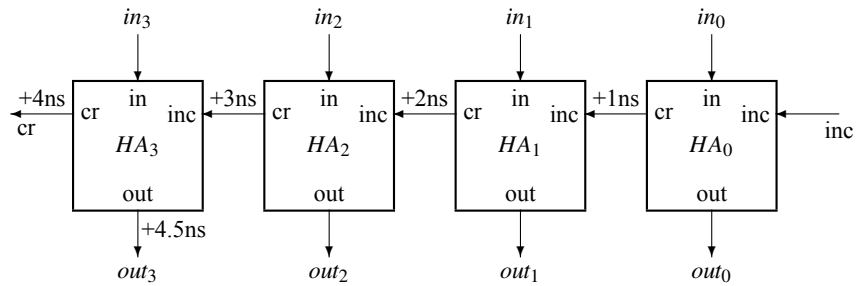


Figure 8.26: Increment circuit for 4-bit numbers (INC4) implemented by serially connected 4 HAs.

Figure 8.26 shows the schematic for a 4-bit incrementer that uses only half adders. Incrementing means adding 1 to a number. If $inc = 1$ the output $out_0 \leq in'_0$ and cr of HA_0 if it is activated command the increment for HA_1 , and so on.

If each 2-input AND from HA_i has, for example, a propagation time, $t_{pAND2} = 1ns = 10^{-9}sec$, and the 2-input XOR has the propagation time, $t_{pXOR2} = 1.5ns$, then the total execution time for the INC4

circuit is:

$$t_{pINC4} = 3 \times t_{pAND2} + t_{pXOR2} = 4.5ns$$

Thus, the size of the n -bit number increment circuit, S_{INCn} , is proportional with the input size. However, the execution time, i.e., depth, D_{INCn} , is in the same order of magnitude:

$$S_{INCn} \in O(n)$$

$$D_{INCn} \in O(n)$$

The depth of the circuit can be reduced by increasing the size (but in the limits of the same magnitude order) using a prefix network such as a Kogge-Stone adder (Kogge and Stone, 1973). See Section 15.5.

8.9.3.3 Full Adder (FA)

Half adders cannot handle carry inputs, leading to the development of the *full adder*, which incorporates a carry input along with two operand inputs. The Full Adder (FA) adds two one-bit numbers with the carry provided from the previous binary position. It generates the sum and the carry value for the next binary position. In Figure 8.27a, a FA is composed of two HAs and an OR circuit used to propagate the carry generated by the first HA or by the second HA. The first FA adds the two input bits, A and B, while the second FA adds the value of the carry, C, generated by the previous binary order.

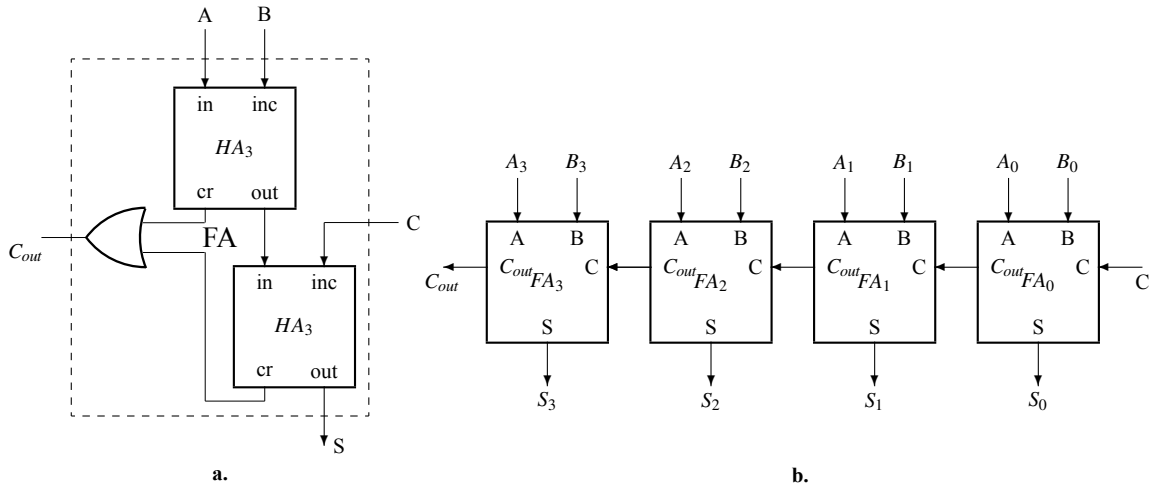


Figure 8.27: Adder. **a.** Full Adder (FA). **b.** 4-bit adder (ADD4).

8.9.3.4 Ripple-Carry Adder

Figure 8.27b shows a schematic for a *ripple carry adder* (RCA). Ripple-carry adders are one of the simplest and most fundamental types of binary adders. They work by connecting a series of full adders, each responsible for processing a single bit of the operands. The carry-out from each adder is propagated to the carry-in of the next, creating a "ripple" effect that moves from the least significant bit (LSB) to the most significant bit (MSB).

While the design of ripple-carry adders is straightforward and easy to implement in hardware, their performance is limited by the sequential nature of carry propagation. The execution time, i.e., the maximum time interval from when an input changes until all the outputs of the circuit reach the correct final value, can be long. This time interval is due to propagation through logic gates on the longest path. In this case, the longest path is the one from input C to output C_{out} . On this path there are 8 logic gates, one AND and one OR for each binary order. The delay for an addition operation increases linearly with the number of bits, as each bit must wait for the carry result from the preceding bit to be resolved. In the general case, the time is proportional to n (it is in $O(n)$).

8.10 Multipliers

8.10.1 Behavioral Multiplier

Similar to behavioral adders, behavioral multipliers describe what the circuit should do without specifying how it is implemented at the gate level.

8.10.1.1 Parameterized Abstract Multiplier (Chisel)

```

8  abstract class Multiplier(width: Int) extends Module {
9      require(width > 0, "Width must be greater than 0")
10
11     val io = IO(new Bundle {
12         val a = Input(UInt(width.W))           // Input A
13         val b = Input(UInt(width.W))           // Input B
14         val isSigned = Input(UInt(1.W))        // 1 for signed, 0 for unsigned
15         val result = Output(UInt((2 * width).W)) // Full precision result
16     })
17 }

```

Listing 8.29: Parameterized Abstract Multiplier (Chisel: Multiplier.scala)

Listing 8.29 shows an abstract multiplier. The `Multiplier` class is an abstract module in Chisel, designed to serve as a base for implementing various multiplier designs. The module defines the basic input-output (I/O) interface and enforces the requirement that the bit-width of the operands, specified by the `width` parameter, must be greater than zero. This ensures that any concrete multiplier inheriting from this class adheres to a consistent interface and valid configuration.

The I/O interface comprises two unsigned integer inputs, `a` and `b`, both of the specified width, a control input `isSigned`, and a single output, `result`, which provides the full precision product of the multiplication. The `isSigned` input determines whether the multiplication is interpreted as signed or unsigned, allowing the module to handle both arithmetic modes dynamically. The `result` output has a width of $2 * \text{width}$, reflecting the theoretical maximum number of bits required to store the result of multiplying two width-bit numbers, regardless of whether the operation is signed or unsigned.

8.10.1.2 Parameterized Behavioral Multiplier (Chisel)

```

8 class BehavioralMultiplier(width: Int) extends Multiplier(width) {
9   when(io.isSigned === 1.U) {
10     io.result := (io.a.asSInt * io.b.asSInt).asUInt // Signed
11               multiplication
12   }.otherwise {
13     io.result := io.a * io.b // Unsigned multiplication
14   }
15 }

```

Listing 8.30: Parameterized Behavioral Multiplier (Chisel: BehavioralMultiplier.scala)

Listing 8.30 shows a behavioral multiplier. The `BehavioralMultiplier` class is a concrete implementation of the abstract `Multiplier` class designed to perform both signed and unsigned multiplications dynamically, based on the `isSigned` input signal. This parameterized module accepts operands `a` and `b` of a specified bit width, with the result of the multiplication output on the `result` port. The `isSigned` input determines whether the multiplication is interpreted as signed or unsigned. When `isSigned` is asserted (equal to 1.U), the operands are cast to signed integers (SInt) for multiplication, and the resulting signed product is cast back to an unsigned integer (UInt) for output. Conversely, if `isSigned` is deasserted (equal to 0.U), the operands are treated as unsigned integers for multiplication.

The output `result` has a width of $2 * \text{width}$, ensuring sufficient precision to accommodate the maximum possible product of two width-bit numbers, whether signed or unsigned.

8.10.2 Precision in Multiplication

A (bin)	A (dec)	B (bin)	B (dec)	Product (bin)	Product (dec)
0001	1	0001	1	00000001	1
0011	3	0010	2	00000110	6
0111	7	0101	5	00101111	35
1111	15	1111	15	11100001	225
1000	8	1001	9	10010000	144
1111	15	0001	1	00001111	15
0110	6	0011	3	00010010	18

Figure 8.28: 4-bit Behavioral Multiplier Partial Truth Table for Unsigned Numbers

Figure 8.28 shows the partial truth table for a 4-bit behavioral multiplier using unsigned numbers. The results demonstrate that multiplying two n -bit numbers may require up to $2n$ bits to represent the full product.

Consider two n -bit binary numbers, A and B , where each can be represented in the form:

$$A = \sum_{i=0}^{n-1} a_i \cdot 2^i, \quad B = \sum_{i=0}^{n-1} b_i \cdot 2^i$$

where $a_i, b_i \in \{0, 1\}$ are the binary digits (bits) of A and B , respectively.

The maximum product occurs when both A and B are at their maximum value, i.e., $A = 2^n - 1$ and $B = 2^n - 1$. In this case:

$$A \times B = (2^n - 1) \times (2^n - 1) = 2^{2n} - 2^n - 2^n + 1 = 2^{2n} - 2^{n+1} + 1$$

This shows that the result of $A \times B$ can range from 0 (minimum) to approximately 2^{2n} (maximum). To store a result as large as $2^{2n} - 1$, $2n$ bits are required.

In the case of 4-bit unsigned numbers, each operand, A and B , can represent values from 0 to $2^4 - 1 = 15$. When multiplying the two largest 4-bit numbers, $A = 15_{base10}$ and $B = 15_{base10}$, the result is $A \times B = 225_{base10}$. To represent 225_{base10} in binary, at least 8 bits are required, as the binary representation is 11100001_2 .

A (bin)	A (dec)	B (bin)	B (dec)	Product (bin)	Product (dec)
0111	7	0101	5	00101111	35
1001	-7	0011	3	11011101	-21
0110	6	1100	-4	11101100	-24
1010	-6	1010	-6	00110000	36
1111	-1	1111	-1	00000001	1
1000	-8	0001	1	11111000	-8
0111	7	1111	-1	11111001	-7

Figure 8.29: 4-bit Behavioral Multiplier Partial Truth Table for Signed Numbers

Figure 8.29 shows the partial truth table for signed numbers. Here, the results indicate that $2n$ bits are also required to store the signed product.

For signed numbers, represented using two's complement, the analysis differs slightly due to the sign extension for negative operands. In two's complement, the most significant bit (MSB) represents the sign, with 0 for positive and 1 for negative numbers. The value of an n -bit signed number A is given by:

$$A = -a_{n-1} \cdot 2^{n-1} + \sum_{i=0}^{n-2} a_i \cdot 2^i$$

Similarly, for B :

$$B = -b_{n-1} \cdot 2^{n-1} + \sum_{i=0}^{n-2} b_i \cdot 2^i$$

The maximum and minimum values of A and B are $2^{n-1} - 1$ (maximum positive) and -2^{n-1} (minimum negative), respectively. The maximum magnitude of the product occurs when one operand is at its maximum positive value and the other is at its minimum negative value:

$$\text{Max Magnitude: } |A \times B| = (2^{n-1} - 1) \cdot 2^{n-1} = 2^{2n-1} - 2^{n-1}$$

In this case, the signed product can range from:

$$-2^{2n-2} \leq A \times B \leq 2^{2n-2} - 1$$

Thus, $2n$ bits are still required to store the full result of signed multiplication to account for both the sign and the magnitude.

8.10.3 Two's Complement Multiplication and Intermediate Overflows

In two's complement multiplication, intermediate overflows that occur during the computation process generally do not affect the correctness of the final result, provided that the final result fits within the target bit range. This behavior arises due to the nature of two's complement arithmetic, which wraps values and preserves correctness when results are truncated to the appropriate bit width.

During multiplication in hardware implementations, intermediate results may produce higher-order bits that "overflow" beyond the required range. For example, multiplying two 4-bit numbers in two's complement may generate intermediate results that temporarily require more than 4 bits. These overflows occur due to the nature of the multiplication operation but are irrelevant to the correctness of the final result, provided the final result fits in 4 bits.

For a signed n -bit number in two's complement, the range of representable values is -2^{n-1} to $2^{n-1} - 1$. As long as the final product lies within this range, any intermediate overflows are inconsequential. For example, multiplying -7 (1001) and 3 (0011) in 4-bit two's complement results in 11101101 in 8 bits. When truncated back to 4 bits, the result is 1101, which correctly represents -21 .

Consider multiplying 4 (0100) and 2 (0010) and -1 1111 in 4-bit two's complement arithmetic:

$$4 \times 2 = 8$$

$$0100 \times 0010 = 0000_1000 = 1000$$

8 is not representable in 2's complement 4-bits but ignoring the overflow and multiplying by -1 gives

$$1000 \times 1111 = 0111_1000 = 1000$$

When truncated to 4 bits, the result becomes:

$$1000$$

This correctly represents -8 in 4-bit two's complement arithmetic, demonstrating that intermediate overflows do not affect the correctness of the final result.

8.10.4 Handling Extended Results

In arithmetic and logical operations, the handling of the extended result, which refers to the extra bits generated during operations such as multiplication or addition, depends on the operation context, the target application, and the architecture. For example, multiplication of two n -bit numbers produces a $2n$ -bit result. In many cases, the result is stored in its entirety, especially in applications that require high precision, such as digital signal processing (DSP) or scientific computing. Architectures like x86 provide instructions that store the low and high halves of the result in separate registers, enabling seamless access to the full $2n$ -bit result.

Modern instruction set architectures (ISAs) often provide distinct opcodes for accessing either the low or high parts of the extended result. For example, in RISC-V, instructions like MUL produce the lower bits of a multiplication result, while MULH and MULHU are used to extract the higher bits for signed and unsigned operations, respectively. Alternatively, some architectures provide combined instructions that store the full-width result across multiple registers. The choice between retaining or discarding the extended result ultimately depends on application requirements, hardware resource constraints, and the overall design philosophy of the ISA. In many ALU designs, retaining the low bits is the default behavior, with access to the high bits provided through additional instructions or status flags for more specialized use cases.

8.11 Multi-Function Arithmetic Units

A multifunction arithmetic unit performs a variety of arithmetic operations such as addition, subtraction, and sometimes multiplication or division, on the same hardware unit. These units are typically designed to handle both signed and unsigned numbers and often support different operand sizes, such as 8-bit, 16-bit, 32-bit, or 64-bit operations. By sharing hardware resources among multiple operations, multifunction arithmetic units reduce the overall complexity of the system.

A multifunction arithmetic unit includes configurable control signals, sometimes called an operation code or opcode, that dictate the operation to be performed. For example, an opcode may specify whether the operation is addition or subtraction, whether the operands are signed or unsigned, and the size of the operands.

Adding a sign/unsigned control bit to an arithmetic unit is a simple example of a multifunction arithmetic unit that enable the same hardware to handle both signed and unsigned operations. In the context of binary arithmetic, signed and unsigned numbers differ primarily in their interpretation of the most significant bit (MSB). For unsigned numbers, the MSB contributes solely to the magnitude of the value, while for signed numbers in two's complement representation, the MSB serves a dual purpose: it indicates the sign of the number (0 for positive, 1 for negative) and also contributes to its magnitude in determining the overall value.

Adding a control signal for selecting between addition and subtraction further enhances the functionality of the unit. Subtraction can be implemented using the same hardware as addition by negating one of the operands, which involves flipping the bits and adding one. With two control signals—one for signed/unsigned selection and another for addition/subtraction—the unit becomes a multifunction adder/subtractor, capable of performing basic arithmetic operations across a wide range of use cases.

When additional operations such as bitwise logical operations (AND, OR, XOR) and shifting (logical, arithmetic, and rotate) are incorporated into the same hardware unit, this is typically called an arithmetic logic unit (ALU). The control signals that specify a particular operation are referred to as an opcode. An ALU is the computational heart of a processor.

8.11.1 Subtraction

8.11.1.1 Behavioral Adder/Subtractor

Similar to behavioral addition described in Section 8.9.1.4, we can also provide for behavioral subtraction. A subtractor can share hardware with an adder making an adder/subtractor the simplest multifunction arithmetic unit.

```

8  class BehavioralAdderSubtractor(width: Int) extends Module {
9      // Define the I/O for the module
10     val io = IO(new Bundle {
11         val a = Input(UInt(width.W))
12         val b = Input(UInt(width.W))
13         val subtract = Input(UInt(1.W))    // 0: add, 1: subtract
14         val result = Output(UInt(width.W))
15     })
16
17     // Compute addition or subtraction
18     val fullResult = Mux(io.subtract.asBool, io.a - io.b, io.a + io.b)

```



```

19
20 // Truncate result to the specified width
21 io.result := fullResult(width - 1, 0)
22 }

```

Listing 8.31: Behavioral Adder/Subtractor (Chisel: BehavioralAdderSubtractor.scala)

Listing 8.31 shows a behavioral adder/subtractor. The `BehavioralAdderSubtractor` class defines a parameterized hardware module in Chisel for performing addition or subtraction of unsigned integers. The module is parameterized by a `width`, allowing it to handle operands of varying bit widths. The input-output interface (`io`) is defined as a `Bundle` that includes two inputs (`a` and `b`), each of type `UInt` with the specified bit width, a control signal (`subtract`) to select the operation, and an output (`result`) that provides the computed value.

The core computation is performed using a `Mux` construct, which selects between addition (`io.a + io.b`) and subtraction (`io.a - io.b`) based on the value of the `subtract` signal. If `subtract` is 0, addition is performed; otherwise, subtraction is performed.

The result of the operation is assigned to the `io.result` output. To ensure that the output adheres to the specified bit width, the `fullResult` is truncated using `(width - 1, 0)`, which extracts only the least significant `width` bits of the result. This ensures that the output does not exceed the desired width, even in cases of overflow or underflow.

```

17 io.sum := BehavioralAdderSubtractor(io.a, io.b, io.subtract, 4)

```

Listing 8.32: 4-bit Behavioral Adder/Subtractor (Chisel: BehavioralAdderSubtractor4.scala)

Listing 8.32 shows an instantiation of a Chisel 4-bit behavioral adder/subtractor. The `BehavioralAdderSubtractor4` module defines an input-output interface (`io`) consisting of two 4-bit unsigned inputs (`a` and `b`), a 1-bit control signal (`subtract`), and a 4-bit unsigned output (`sum`).

The arithmetic operation itself is delegated to a reusable module called `BehavioralAdderSubtractor`, which is invoked with the necessary parameters, including the two input values (`a` and `b`), the `subtract` control signal, and the bit width of the operation (4 bits in this case). The result of the operation is assigned to the `sum` output.

8.11.1.2 Structural Subtractor

Two's complement subtraction can be performed by taking the 2's complement of the subtrahend and then performing addition.

```

17 // Two's complement for subtraction
18 val bAdjusted = Mux(io.subtract.asBool, ~io.b + 1.U, io.b)
19 val fullResult = io.a + bAdjusted
20
21 // Truncate result to the specified width
22 io.result := fullResult(width - 1, 0)

```

Listing 8.33: Parameterized Two's Complement Adder/Subtractor (Chisel: BehavioralAdderSubtractorHS.scala)

Listing 8.33 lines 17 through 22 show subtraction by first negating the subtrahend.

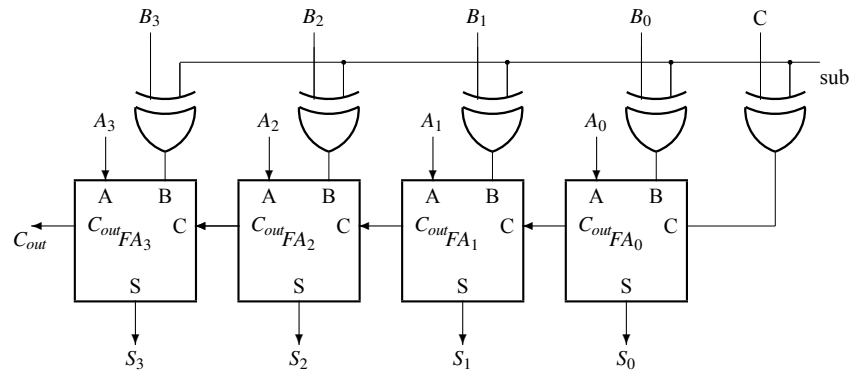


Figure 8.30: Adder/Subtractor

Figure 8.30 shows how subtraction can be performed using the same adder hardware shown in Figure 8.27b. XOR gates are used to negate the subtrahend. Setting the initial carry to 1 provides the 2's complement of the subtrahend.

For example:

$$\begin{aligned}
 +5 &= 0_0101 \\
 -5 &= 1_1010 + 1 = 1_1011 \\
 +5 &= 0_0100 + 1 = 0_0101 \\
 \\
 -5 + 2 &= 1_1011 + \\
 &\quad 0_0010 = \\
 &\quad 1_1101 = -3, \text{ because } 0_0010 + 1 = 0_0011 = 3
 \end{aligned}$$

Each B_i is complemented and the increment with 1 is generated by inverting the carry input. All the inversions are performed under the command `sub` applied to 5 XORs.

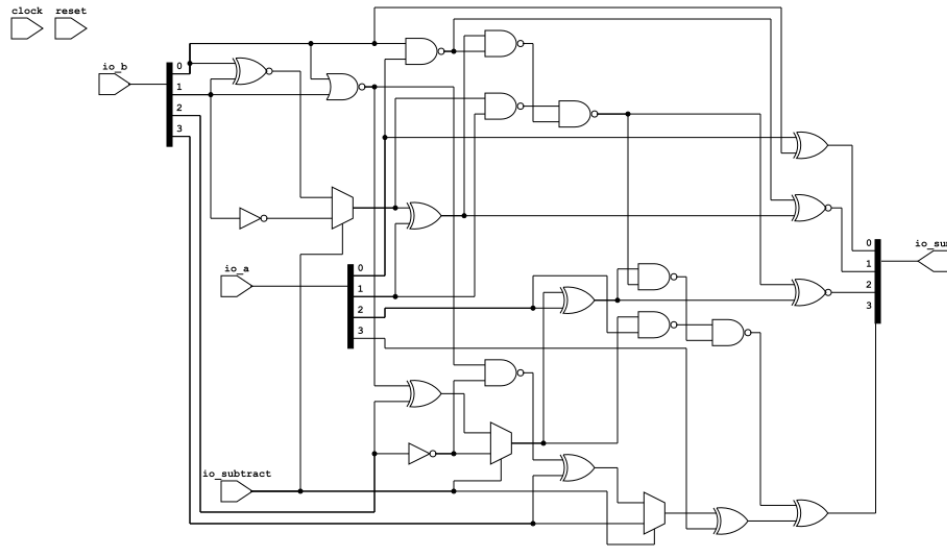


Figure 8.31: Yosys ASIC Synthesized and Flattened Schematic of a Two's Complement 4-bit Adder/Subtractor.

As synthesis design tools become more mature, it is interesting to note that both the Behavioral adder/subtractor and the Two's complement adder/subtractor synthesize to the same netlist shown in Figure 8.31

8.11.1.3 Multifunction 8/16/32/64-bit Signed/Unsigned Behavioral Adder/Subtractor (Chisel)

We now present a multifunction adder/subtractor that is closer to what might be found in a commercial design.

```

10
11  val io = IO(new Bundle {
12    val a = Input(UInt(width.W))           // First operand
13    val b = Input(UInt(width.W))           // Second operand
14    val opcode = Input(UInt(4.W))           // Operation code
15    val carryIn = Input(UInt(1.W))          // Optional carry-in (default 0)
16
17    val result = Output(UInt(width.W))      // Computed result
18
19    //Flags
20    val carryOut = Output(UInt(1.W))        // carry-out
21    val overflowFlag = Output(UInt(1.W))    // Overflow flag
22    val zeroFlag = Output(UInt(1.W))        // Zero flag
23    val negativeFlag = Output(UInt(1.W))    // Negative flag
24  })
25
26  // Default implementations for optional signals
27  io.carryOut := DontCare
28  io.overflowFlag := DontCare
29  io.zeroFlag := DontCare

```

```

30   io.negativeFlag := DontCare
31 }

```

Listing 8.34: Abstract Multifunction Adder/Subtractor (Chisel: MultifunctionAdderSubtractor.scala)

Listing 8.34 serves as an abstract base class for constructing modules that implement multi-functional arithmetic operations. The class is declared as `abstract`, meaning it is not directly instantiated but instead serves as a blueprint for more specific implementations. It extends the `Module` class, the fundamental building block for Chisel designs. The constructor accepts a single parameter `width` that specifies the bit-width of the operands and results. A `require` statement ensures this value is greater than zero, providing an early runtime check for invalid parameters.

The `io` field defines the input-output interface of the module using a Chisel `Bundle`. The bundle includes the following signals:

- `a`: An input signal representing the first operand, of type `UInt` with the specified width.
- `b`: An input signal representing the second operand, also a `UInt` with the same width.
- `opcode`: A 4-bit input signal that specifies the operation to be performed (e.g., addition, subtraction).
- `carryIn`: An optional 1-bit input signal for an optional carry-in, defaulting to 0.
- `result`: An output signal representing the computed result, with the same width as the operands.
- `carryOut`: An optional 1-bit output signal that indicates a carry-out from unsigned arithmetic.
- `overflowFlag`: An optional 1-bit output signal that indicates an overflow in signed arithmetic.
- `zeroFlag`: An optional 1-bit output signal that indicates whether the result is zero.
- `negativeFlag`: An optional 1-bit output signal that indicates whether the result is negative in signed arithmetic.

The base class provides default assignments for the optional output signals `carryOut`, `overflowFlag`, `zeroFlag`, and `negativeFlag`. All are initialized to `DontCare`. This indicates that these signals are not actively driven by the base class, leaving their implementation to derived classes.

This base class is designed for flexibility and modularity:

- The parameterized `width` ensures that the design can accommodate operands of any bit-width, making it reusable across different use cases.
- The abstract nature of the class allows specific implementations (e.g., addition-subtraction for specific operand sizes or opcode handling) to extend and customize its behavior.
- By defining a consistent interface through the `io` bundle, the class simplifies integration into larger designs and promotes compatibility.

We now proceed to an implementation. While the arithmetic is still behavioral, the inclusion of flags moves it close to a structural implementation.

```

10 // Opcodes
11 // b3: Add/Sub
12 // b2: Signed/Unsigned
13 // b1b0: 8/16/32/64 bits
14 object MultifunctionAdderSubtractor64 {
15
16     // Define Opcodes within the companion object
17     object Opcode {
18         val ADD_U8 = "b0000".U
19         val ADD_U16 = "b0001".U
20         val ADD_U32 = "b0010".U
21         val ADD_U64 = "b0011".U
22         val SUB_U8 = "b1000".U
23         val SUB_U16 = "b1001".U
24         val SUB_U32 = "b1010".U
25         val SUB_U64 = "b1011".U
26         val ADD_S8 = "b0100".U
27         val ADD_S16 = "b0101".U
28         val ADD_S32 = "b0110".U
29         val ADD_S64 = "b0111".U
30         val SUB_S8 = "b1100".U
31         val SUB_S16 = "b1101".U
32         val SUB_S32 = "b1110".U
33         val SUB_S64 = "b1111".U
34     }
35
36     // Companion object
37     def apply(
38         a: UInt,
39         b: UInt,
40         opcode: UInt,
41         carryIn: Option[UInt] = None // optional
42     ): MultifunctionAdderSubtractor64 = {
43         val module = Module(new MultifunctionAdderSubtractor64)
44         module.io.a := a
45         module.io.b := b
46         module.io.opcode := opcode
47         carryIn match {
48             case Some(value) => module.io.carryIn := value // Connect when
49                 // provided
50             case None => module.io.carryIn := 0.U // Default to 0 if
51                 // not provided
52         }
53     }
54
55     class MultifunctionAdderSubtractor64 extends
56         MultifunctionAdderSubtractor(64) {

```

```

57     val printDebugInfo = false
58
59     // Decode control signals
60     // b3: Add/Sub
61     // b2: Signed/Unsigned
62     // b1b0: 8/16/32/64 bits
63     val isSub = io.opcode(3) === 1.U // b3: 1 for subtraction
64     val isSigned = io.opcode(2) // b2: 1 for signed
65     val operandSize = io.opcode(1, 0) // b1b0: 00 for 8-bit, 01 for 16-
        bit, 10 for 32-bit, 11 for 64-bit
66
67     // Determine effective width based on operand size
68     val width = MuxCase(64.U, Seq(
69         (operandSize === "b00".U) -> 8.U,
70         (operandSize === "b01".U) -> 16.U,
71         (operandSize === "b10".U) -> 32.U,
72         (operandSize === "b11".U) -> 64.U
73     ))
74
75     val mask = MuxCase("ffffffffffffffff".U, Seq(
76         (operandSize === "b00".U) -> "h00000000000000ff".U,
77         (operandSize === "b01".U) -> "h000000000000ffff".U,
78         (operandSize === "b10".U) -> "h00000000ffffffff".U,
79         (operandSize === "b11".U) -> "ffffffffffffffff".U
80     ))
81
82     // Mask inputs to the appropriate width
83     val aEffective = io.a & mask
84     val bEffective = io.b & mask
85     val bAdjusted = Mux(isSub, (~bEffective + 1.U), bEffective)
86
87     if(printDebugInfo){
88         printf(p"    width = ${width}, isSigned = ${isSigned}, isSub = ${
            isSub}\n")
89         printf(p"    mask =          0x${Hexadecimal(mask)}\n")
90         printf(p"    a =          0x${Hexadecimal(io.a)}\n")
91         printf(p"    b =          0x${Hexadecimal(io.b)}\n")
92         printf(p"    aEffective = 0x${Hexadecimal(aEffective)},\n")
93         printf(p"    bEffective = 0x${Hexadecimal(bEffective)},\n")
94         printf(p"    bAdjusted = 0x${Hexadecimal(bAdjusted)},\n")
95     }
96
97     //*****
98     // Arithmetic operations
99     //*****
100    val fullArithmeticResult = Mux(isSigned,
101        (aEffective.asSInt +& bAdjusted.asSInt).asUInt, // Signed operation
102        (aEffective +& bAdjusted) // Unsigned
103        operation
104    )
105    val truncatedResult = fullArithmeticResult & mask

```

```

105
106   if(printDebugInfo) {
107       printf(p"    fullArithmeticResult = 0x${Hexadecimal(
108           fullArithmeticResult)}\n")
109       printf(p"    truncatedResult =      0x${Hexadecimal(truncatedResult)}\n")
110   }
111
112   //*****
113   // Final result
114   //*****
115   io.result := truncatedResult
116   if(printDebugInfo) printf(p"    io.result =      0x${Hexadecimal(io.
117       result)}\n")
118
119   //*****
120   // Compute Flags
121   //*****
122
123   // Carry Flag: Unsigned Arithmetic
124   // Chisel considers width a dynamic construct
125   // Unfortunately, every width must be explicit
126
127   val isCarry = MuxCase(false.B, Seq(
128       (width === 8.U)   -> fullArithmeticResult(8),
129       (width === 16.U)  -> fullArithmeticResult(16),
130       (width === 32.U)  -> fullArithmeticResult(32),
131       (width === 64.U)  -> fullArithmeticResult(64)
132   ))
133
134   // Borrow
135   val isBorrow = Mux(isSub && !isSigned, aEffective < bEffective, false.B)
136
137   if(printDebugInfo) printf(p"    isCarry =      0x${Hexadecimal(isCarry)}\n")
138   if(printDebugInfo) printf(p"    isBorrow =     0x${Hexadecimal(isBorrow)}\n")
139
140   io.carryOut := MuxCase(0.U, Seq(
141       (!isSigned && !isSub) -> isCarry,
142       (!isSigned && isSub)  -> isBorrow
143   ))
144
145   if(printDebugInfo) printf(p"    io.carryOut =     0x${Hexadecimal(io.
146       carryOut)}\n")
147
148   // Overflow: Signed Arithmetic
149   // Adding with different signs can't cause overflow

```

```

149 // Overflow = both operand signs the same, result sign different
150 val aSign = WireDefault(false.B)
151 val bSign = WireDefault(false.B)
152 val sumSign = WireDefault(false.B)
153
154 aSign := MuxCase(false.B, Seq(
155   (width === 8.U)  -> aEffective(7),
156   (width === 16.U) -> aEffective(15),
157   (width === 32.U) -> aEffective(31),
158   (width === 64.U) -> aEffective(63)
159 ))
160
161 bSign := MuxCase(false.B, Seq(
162   (width === 8.U)  -> bAdjusted(7),
163   (width === 16.U) -> bAdjusted(15),
164   (width === 32.U) -> bAdjusted(31),
165   (width === 64.U) -> bAdjusted(63)
166 ))
167
168 sumSign := MuxCase(false.B, Seq(
169   (width === 8.U)  -> fullArithmeticResult(7),
170   (width === 16.U) -> fullArithmeticResult(15),
171   (width === 32.U) -> fullArithmeticResult(31),
172   (width === 64.U) -> fullArithmeticResult(63)
173 ))
174
175 val isOverflow = (aSign === bSign) && (aSign != sumSign)
176 if (printDebugInfo) printf(p"  overflowFlag =      0x${Hexadecimal(
177   isOverflow)}\n")
178 io.overflowFlag := isOverflow
179
180 val isZero = !truncatedResult.orR // Reduction OR to check if any bit
181   in `io.result` is 1
182 if (printDebugInfo) printf(p"  zeroFlag =      0x${Hexadecimal(isZero)}\n")
183 io.zeroFlag := isZero
184
185 val isNegative = Mux(isSigned, sumSign, false.B)
186 io.negativeFlag := isNegative
187 }

```

Listing 8.35: Multifunction 64-bit Adder/Subtractor (Chisel)

Listing 8.35 shows a 64-bit multifunction adder-subtractor module. It is part of the package `scabook.addersubtractors` and is designed to perform a variety of arithmetic operations, including addition and subtraction, over signed and unsigned integers of varying bit widths (8, 16, 32, and 64). The implementation is parameterized by an opcode, which encodes the operation type and operand characteristics.

The module uses a companion object named `MultifunctionAdderSubtractor64` to define operation-specific opcodes and to facilitate the instantiation of the module. The opcode is a 4-bit value where:

- The most significant bit (b3) determines whether the operation is addition (0) or subtraction (1).
- The second bit (b2) specifies signed (1) or unsigned (0) arithmetic.
- The two least significant bits (b1b0) determine the operand size: 8-bit, 16-bit, 32-bit, or 64-bit.

The companion object also includes an `apply` method that simplifies module instantiation and manages **optional input signals** like `carryIn`.

The class `MultifunctionAdderSubtractor64` extends a generic `MultiFunctionAdderSubtractor` module, parameterized for 64-bit operations.

Its internal implementation uses the opcode to decode control signals and manage input operands.

The code first decodes the control signals. The bit selections in the opcode are not randomly chosen. They ensure minimal decoding is required to determine which operation to perform and the types associated with the operation.

- `isSub`: A Boolean flag derived from b3 to indicate subtraction.
- `isSigned`: A flag derived from b2 to indicate signed arithmetic.
- `operandSize`: The operand size determined by the bits b1b0.

The effective operand width is determined using a multiplexer (`MuxCase`) and is used to create a mask that truncates inputs to their specified width. Operands `a` and `b` are masked accordingly, and `b` is adjusted for subtraction using two's complement logic ($b + 1$).

The code performs the arithmetic operation (addition or subtraction) based on the `isSub` and `isSigned` flags. For signed arithmetic, the operands are cast to `SInt` types before addition. The result is then masked to fit within the specified operand size. The `+` operator is used to perform addition while preserving the carry-out bit. Unlike the standard addition operator (`+`), which only computes the sum of two operands within a fixed width, `+` ensures that the result includes an extra bit to accommodate any overflow from the addition. The result of the operation is then truncated to the selected width using a bitwise `&` operation with the mask. For output consistency, the result is extended back to 64 bits using either sign extension or zero extension, depending on whether the operation is signed or unsigned.

The module computes several flags to indicate the outcome of the operation:

- **Carry/Borrow Flag**: Indicates whether there was a carry or borrow in unsigned arithmetic. The flag depends on the operand size.
- **Overflow Flag**: Detects overflow in signed arithmetic when operands with the same sign produce a result with a different sign.
- **Zero Flag**: Indicates if the result is zero by performing a reduction OR operation.
- **Negative Flag**: Identifies whether the result is negative in signed arithmetic.

The code includes optional debugging functionality, controlled by the `printDebugInfo` flag. When enabled, intermediate values, such as operand adjustments, flags, and results, are printed using the `printf` method.

The final result of the arithmetic operation is assigned to `io.result`. Flags, including carry, over-

flow, zero, and negative, are also output through the corresponding IO ports.

8.11.1.4 Multifunction Parameterized Signed/Unsigned Ripple-Carry Adder/Subtractor (Chisel)

```

9  class RippleCarryAdder(width: Int) extends Module {
10    val io = IO(new Bundle {
11      val a = Input(UInt(width.W))      // Input A
12      val b = Input(UInt(width.W))      // Input B
13      val carryIn = Input(UInt(1.W))    // Carry-in
14      val isSigned = Input(UInt(1.W))   // Control signal: 1 for signed,
                                         // 0 for unsigned
15      val sum = Output(UInt(width.W))   // Sum output
16      val carryOut = Output(UInt(1.W))  // Carry-out for unsigned
                                         // addition
17      val overflow = Output(UInt(1.W))  // Overflow for signed addition
18    })
19
20    val carries = Wire(Vec(width + 1, Bool()))
21    val sumBits = Wire(Vec(width, Bool()))
22
23    carries(0) := io.carryIn // Initial carry-in
24
25    for (i <- 0 until width) {
26      val aBit = io.a(i) // Access individual bits of `a`
27      val bBit = io.b(i) // Access individual bits of `b`
28      val sum = aBit ^ bBit ^ carries(i) // Sum bit
29      val carry = (aBit & bBit) | (aBit & carries(i)) | (bBit & carries(i))
                                         // Carry bit
30      sumBits(i) := sum
31      carries(i + 1) := carry
32    }
33
34    io.sum := sumBits.asUInt // Sum output
35    io.carryOut := carries(width).asUInt // Carry-out for unsigned
                                         // addition
36    io.overflow := (io.isSigned.asBool && (carries(width - 1) ^ carries(
                                         // Overflow for signed addition
                                         width))).asUInt
37  }

```

Listing 8.36: Parameterized Ripple-Carry Adder (Chisel)

Listing 8.36 shows code that defines a `RippleCarryAdder` class in Chisel, which implements the ripple-carry adder (RCA) architecture for addition.

The `RippleCarryAdder` is a hardware module implemented in Chisel, designed to perform binary addition for numbers of a configurable bit width. The module takes two input operands `a` and `b`, each of size `width` bits, along with a `carryIn` input to handle any carry from a previous operation. The addition can be performed in either signed or unsigned mode, controlled by the `isSigned` input. The module outputs the sum of the inputs, a `carryOut` signal indicating carry propagation in unsigned addition, and an `overflow` signal to detect overflow in signed addition.

Internally, the `RippleCarryAdder` uses a ripple-carry mechanism, where the carry bit propagates

sequentially through each bit of the operands. To achieve this, the design employs a loop to iterate over all bits of the operands, calculating the sum and carry for each bit. The sum for a bit is computed using the XOR operation, while the carry is determined based on AND and OR operations between the bits of the operands and the carry from the previous bit. The result is stored in a vector of `Bool` wires, which is eventually converted into a `UInt` output.

The `carryOut` signal is derived from the carry bit generated at the most significant bit (MSB) of the operands. This signal is used to indicate whether the result of an unsigned addition exceeds the maximum representable value for the given width. The `overflow` signal, on the other hand, is specific to signed arithmetic and checks whether the addition resulted in an overflow. This is done by comparing the carries into and out of the MSB using an XOR operation, which indicates overflow if the signs of the operands and the result are inconsistent.

A companion object (not shown) for the `RippleCarryAdder` provides a factory method to create instances of the module. This method ensures that the width of the adder is positive and constructs the module with the specified parameters.

The primary difference between a signed and an unsigned ripple-carry adder lies in how they interpret the binary numbers being added and how they handle overflow conditions. Both types of adders use the same basic ripple-carry mechanism for propagating carry bits through a series of full adders, but their behavior and application diverge due to differences in number representation.

An **unsigned ripple-carry adder** is designed to add two unsigned binary numbers, which represent only non-negative integers. The range of values for an n -bit unsigned number is 0 to $2^n - 1$. In this case, the carry-out from the most significant bit (MSB) indicates whether the sum exceeds the maximum representable value ($2^n - 1$) for the given width. This behavior is straightforward, as unsigned addition follows normal binary arithmetic rules. An overflow condition is typically not flagged explicitly in an unsigned ripple-carry adder, as the carry-out already provides sufficient information about the result exceeding the valid range.

A **signed ripple-carry adder** works with signed binary numbers, represented in two's complement format. The range of values for an n -bit signed number is -2^{n-1} to $2^{n-1} - 1$. Two's complement representation allows the same ripple-carry logic to be used, but special care is needed to detect overflow. Signed overflow occurs when the result of addition is outside the representable range. For example, adding two positive numbers should not yield a negative result, and adding two negative numbers should not yield a positive result. This is detected by comparing the carry into and out of the MSB using an XOR operation: $(\text{Carry-In} \oplus \text{Carry-Out})$. In signed addition, the carry-out from the MSB is not relevant, as it does not necessarily indicate overflow in two's complement representation.

Consider a 4-bit signed ripple-carry adder. The range of representable numbers is -8 to 7 . Let the first operand A be 5, represented as 0101, and the second operand B be -3 , represented as 1101 in two's complement format. Adding these two values produces the expected result of $A - |B| = 5 - 3 = 2$.

Operand A (5):	0101
Operand B (-3):	1101
Addition:	$0101 + 1101 = 10010$
Result (truncated to 4 bits):	$0010 = 2$

In the example, the adder computes $0101 + 1101 = 10010$, and the result is truncated to 4 bits, yielding 0010, which corresponds to 2 in signed representation. Note that the carry-out from the MSB is discarded as it is not relevant in two's complement arithmetic.

8.12 Arithmetic & Logic Unit

An Arithmetic & Logic Unit (ALU) has two kinds of connections: data inputs and outputs, for the n -bit operands, and command inputs for the operations applied to operands (see Fig 8.32). In Figure 8.33, is represented the generic form of an ALU with 8 functions each performed in a distinct module, `func0`, ..., `func7`, whose outputs are selected by a multiplexer as the result by the `func` code. The two operands, `leftOp[n-1:0]`, `rightOp[n-1:0]`, are applied to all the 8 modules.

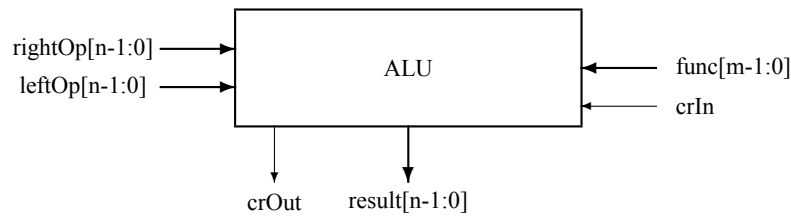


Figure 8.32: ALU.

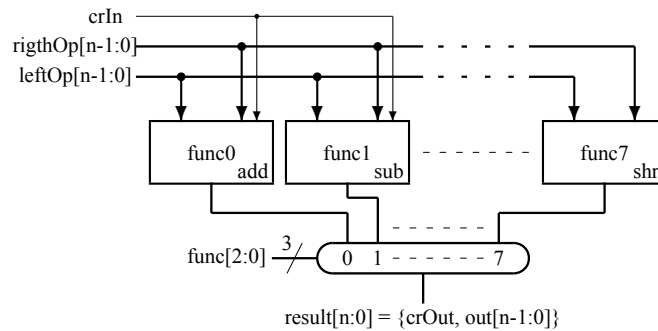


Figure 8.33: The generic form of a 8-function Arithmetic & Logic Unit for n -bit words (ALUn).

```

1  module ALU(input      logic      crIn      ,
2             input      logic [2:0]  func      ,
3             input      logic [31:0] left, right ,
4             output     logic      crOut     ,
5             output     logic [31:0] out      );
6
7  always_comb case(func)
8      3'b000: {crOut, out} = left + right + crIn;    //add
9      3'b001: {crOut, out} = left - right - crIn;    //sub
10     3'b010: {crOut, out} = {1'b0, left & right};    //and
11     3'b011: {crOut, out} = {1'b0, left | right};    //or
12     3'b100: {crOut, out} = {1'b0, left ^ right};    //xor

```

```

13             3'b101: {crOut, out} = {1'b0, ~left};           //not
14             3'b110: {crOut, out} = {1'b0, left};           //left
15             3'b111: {crOut, out} = {1'b0, left >> 1};      //shr
16         endcase
17     endmodule

```

Listing 8.37: Arithmetic and Logic Unit. File: A08_alu.sv (SystemVerilog)

SystemVerilogSummary 1 :

{a,b} : concatenates the binary word b to the binary word a

- : is the operator minus

& : is the operator bitwise AND

| : is the operator bitwise OR

^ : is the operator bitwise XOR

>> : is the operator shift right logical

Actual implementations are sometimes optimized by implementing two or more modules sharing partially the same circuits. A small and simple example is represented in Figure 8.34, where:

- func[2:0] to select the function:

func = 000 add: {crOut, result} = leftOp + rightOp + crIn

func = 100 sub: {crOut, result} = leftOp - rightOp - crIn

func = 010 and: {crOut, result} = {1'b0, leftOp & rightOp} (bitwise AND)

func = 001 xor: {crOut, result} = {1'b0, leftOp $\oplus$ rightOp} (bitwise XOR)

func = 011 shr: {crOut, result} = {leftOp[0], serialIn, leftOp[n-1:1]}

- leftOp[n-1:0]

- rightOp[n-1:0]

- crIn

- crOut

- serialIn: the value loaded in the most significant position in the shift right operation, shr, allowing the following types of shifts:

◇ shr: {crOut, result} = {leftOp[0], 1'b0, leftOp[n-1:1]}

◇ ash: {crOut, result} = {leftOp[0], leftOp[n-1], leftOp[n-1:1]}

◇ rot: {crOut, result} = {leftOp[0], leftOp[0], leftOp[n-1:1]}

◇ csh: {crOut, result} = {leftOp[0], crIn, leftOp[n-1:1]}

• `result[n-1:0]`

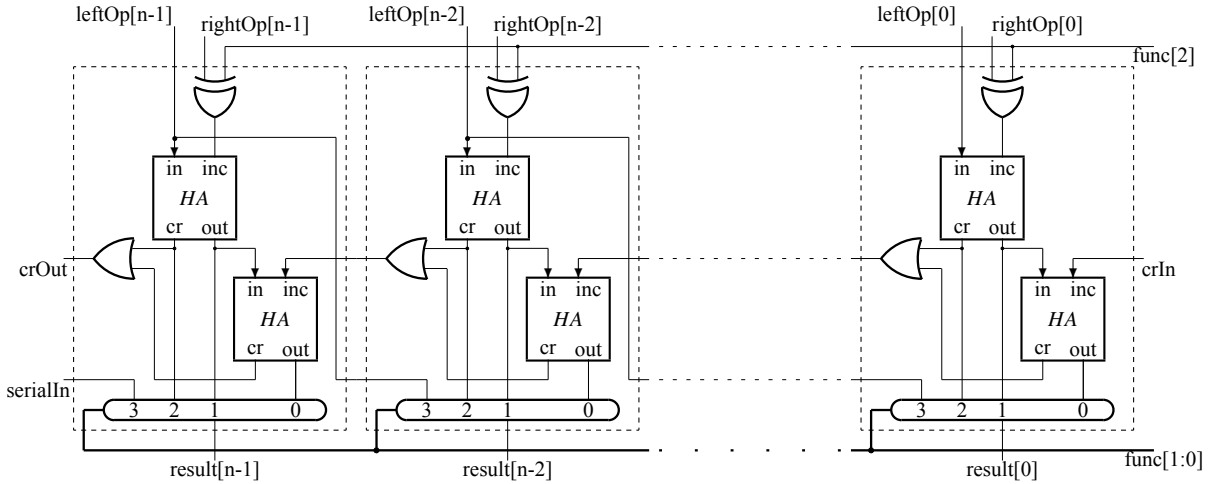


Figure 8.34: A simple Arithmetic & Logic Unit for n -bit words (ALUn).

The implementation takes into account the fact that a HA is implemented using an AND gate and a XOR gate (see Figure ??). Thus, in each of the n slices of ALU only an adder/subtractor is implemented, a XOR is added for the subtract function and for the two logic functions the outputs of the first HA are directly used.

```

9  import scabook.addersubtractors.MultifunctionAdderSubtractor64
10
11  // Opcodes
12  // b5: arithmetic/logic
13  // b1b0: 8/16/32/64 bits
14  object ALU64 {
15      object Opcode {
16          // Arithmetic Operations
17          // b4: Unused
18          // b3: Add/Sub
19          // b2: Unsigned/Signed
20          val ADD_U8  = "b000000".U
21          val ADD_U16 = "b000001".U
22          val ADD_U32 = "b000010".U
23          val ADD_U64 = "b000011".U
24          val SUB_U8  = "b001000".U
25          val SUB_U16 = "b001001".U
26          val SUB_U32 = "b001010".U
27          val SUB_U64 = "b001011".U
28          val ADD_S8  = "b000100".U
29          val ADD_S16 = "b000101".U

```

```

30     val ADD_S32 = "b000110".U
31     val ADD_S64 = "b000111".U
32     val SUB_S8  = "b001100".U
33     val SUB_S16 = "b001101".U
34     val SUB_S32 = "b001110".U
35     val SUB_S64 = "b001111".U
36
37     // Logical Operations
38     // b4b3b2: operation
39     val AND_U8  = "b100000".U
40     val AND_U16 = "b100001".U
41     val AND_U32 = "b100010".U
42     val AND_U64 = "b100011".U
43     val OR_U8   = "b100100".U
44     val OR_U16  = "b100101".U
45     val OR_U32  = "b100110".U
46     val OR_U64  = "b100111".U
47     val XOR_U8  = "b101000".U
48     val XOR_U16 = "b101001".U
49     val XOR_U32 = "b101010".U
50     val XOR_U64 = "b101011".U
51     val SLL_U8  = "b101100".U
52     val SLL_U16 = "b101101".U
53     val SLL_U32 = "b101110".U
54     val SLL_U64 = "b101111".U
55     val SRL_U8  = "b110000".U
56     val SRL_U16 = "b110001".U
57     val SRL_U32 = "b110010".U
58     val SRL_U64 = "b110011".U
59     val SRA_U8  = "b110000".U
60     val SRA_U16 = "b110001".U
61     val SRA_U32 = "b110010".U
62     val SRA_U64 = "b110011".U
63 }
64 }
65
66 class ALU64 extends Module {
67     val io = IO(new Bundle {
68         val a = Input(UInt(64.W))
69         val b = Input(UInt(64.W))
70         val result = Output(UInt(64.W))
71         val opcode = Input(UInt(6.W))
72         val carryOutFlag = Output(UInt(1.W)) //Carry and Borrow
73         val overflowFlag = Output(UInt(1.W)) //V
74         val zeroFlag = Output(UInt(1.W)) //Z
75         val negativeFlag = Output(UInt(1.W)) //N
76     })
77
78     val printDebugInfo = false
79
80     // Debug internals of ALU

```

```

81     if(printDebugInfo) printf(p"[Inside ALU64] --\n")
82
83     // Decode control signals
84     // b5: arithmetic / logic
85     // b4b3b2: logic ope
86     // b3: if arith: Add/Sub
87     // b2: if arith: Signed/Unsigned
88     // b1b0: 8/16/32/64 bits
89     // Decode control signals
90     val isArithmetic = io.opcode(5) === 0.U // MSB: 0 for arithmetic
91     val isLogical = io.opcode(5) === 1.U // MSB: 1 for logical operations
92     val isSub = io.opcode(3) === 1.U && isArithmetic // b3: 1 for
        subtraction
93     val isSigned = io.opcode(2) && isArithmetic // b2: 1 for signed
94     val operandSize = io.opcode(1, 0) // b1b0: 00 for 8-bit, 01 for 16-
        bit, 10 for 32-bit, 11 for 64-bit
95
96     // Determine effective width based on operand size
97     val width = WireDefault(64.U)
98     val mask = WireDefault("ffffffffffffffff".U)
99     switch(operandSize) {
100         is("b00".U) { width := 8.U; mask := "h00000000000000ff".U }
101         is("b01".U) { width := 16.U; mask := "h000000000000ffff".U }
102         is("b10".U) { width := 32.U; mask := "h00000000ffffffff".U }
103         is("b11".U) { width := 64.U; mask := "ffffffffffffffff".U }
104     }
105
106     // Mask inputs to the appropriate width
107     val aEffective = io.a & mask
108     val bEffective = io.b & mask
109     val bAdjusted = Mux(isSub, (~bEffective + 1.U), bEffective)
110
111     if(printDebugInfo){
112         printf(p"    width = ${width}, isSigned = ${isSigned}, isSub = ${
            isSub}\n")
113         printf(p"    mask =          0x${Hexadecimal(mask)}\n")
114         printf(p"    a =          0x${Hexadecimal(io.a)}\n")
115         printf(p"    b =          0x${Hexadecimal(io.b)}\n")
116         printf(p"    aEffective = 0x${Hexadecimal(aEffective)},\n")
117         printf(p"    bEffective = 0x${Hexadecimal(bEffective)},\n")
118         printf(p"    bAdjusted = 0x${Hexadecimal(bAdjusted)},\n")
119     }
120
121     //*****
122     // Arithmetic operations
123     //*****
124     val adderSubtractor = Module(new MultifunctionAdderSubtractor64)
125
126     // Connect inputs
127     adderSubtractor.io.a := aEffective
128     adderSubtractor.io.b := bEffective

```



```

129  adderSubtractor.io.carryIn := 0.U(1.W)
130  adderSubtractor.io.opcode := io.opcode(3, 0)
131
132  // Connect outputs to the ALU64 outputs
133  io.result := adderSubtractor.io.result
134  io.carryOutFlag := adderSubtractor.io.carryOut
135  io.overflowFlag := adderSubtractor.io.overflowFlag
136  io.zeroFlag := adderSubtractor.io.zeroFlag
137  io.negativeFlag := adderSubtractor.io.negativeFlag
138
139
140  //*****
141  // Logical operations
142  //*****
143  // b4b3b2: logical operation
144  val logicalResult = MuxCase(0.U(64.W), Seq(
145      (io.opcode(4, 2) === "b000".U) -> (aEffective & bEffective),
146                                     // AND
147      (io.opcode(4, 2) === "b001".U) -> (aEffective | bEffective),
148                                     // OR
149      (io.opcode(4, 2) === "b010".U) -> (aEffective ^ bEffective),
150                                     // XOR
151      (io.opcode(4, 2) === "b011".U) -> ((aEffective << (bEffective(5, 0)
152          & (width - 1.U))).asUInt & mask), // SLL
153      (io.opcode(4, 2) === "b100".U) -> ((aEffective >> (bEffective(5, 0)
154          & (width - 1.U))).asUInt & mask), // SRL
155      (io.opcode(4, 2) === "b101".U) -> ((aEffective.asSInt >> (
156          bEffective(5, 0) & (width - 1.U))).asUInt & mask) // SRA
157  ))
158
159  if(printDebugInfo) printf(p"  logicalResult =      0x${Hexadecimal(
160      logicalResult)}\n")
161
162  //*****
163  // Assign results to outputs
164  //*****
165  io.result := Mux(isArithmetic, adderSubtractor.io.result,
166      logicalResult)
167  io.carryOutFlag := Mux(isArithmetic, adderSubtractor.io.carryOut, 0.U)
168  io.overflowFlag := Mux(isArithmetic, adderSubtractor.io.overflowFlag,
169      0.U)
170  io.zeroFlag := Mux(isArithmetic, adderSubtractor.io.zeroFlag, 0.U)
171  io.negativeFlag := Mux(isArithmetic, adderSubtractor.io.negativeFlag,
172      0.U)
173
174  }

```

Listing 8.38: 64-bit ALU (Chisel: ALU64.scala)

Listing 8.38 shows the ALU64 class implements a 64-bit Arithmetic Logic Unit (ALU) using Chisel, designed to perform both arithmetic and logical operations. The operations are controlled by a 6-bit

opcode, with modularity and reuse of the `MultifunctionAdderSubtractor64` module to handle arithmetic functionality.

The opcode is a 6-bit signal that specifies the operation type and operand size. The most significant bit (b5) distinguishes between arithmetic (b5 = 0) and logical (b5 = 1) operations. For arithmetic operations, bits b4, b3, and b2 specify addition or subtraction, signed or unsigned computation, and unused functionality. Logical operations are further distinguished by a combination of bits b4, b3, and b2. The operand size is encoded in the two least significant bits (b1b0), allowing operations on 8, 16, 32, or 64-bit data.

The module defines an input-output interface through a `Chisel Bundle`. It accepts two 64-bit operands (a and b), the 6-bit opcode, and produces a 64-bit `result`. Additionally, it outputs flags for carry or borrow (`carryOutFlag`), overflow (`overflowFlag`), zero result (`zeroFlag`), and negative result (`negativeFlag`).

The ALU decodes the opcode to distinguish between arithmetic and logical operations. For arithmetic operations, it passes bits b3b2b1b0 to the `MultifunctionAdderSubtractor`. No pre-decoding of the opcode is required. The ALU also identifies whether the operation is addition or subtraction (`isSub`) and whether it is signed or unsigned (`isSigned`). The operand size is extracted from the opcode (`operandSize`), which determines the effective width of the operation and the corresponding mask to limit the operands.

Operands a and b are masked to their effective width, ensuring the operation respects the specified bit size. For subtraction, operand b is adjusted using two's complement logic ($\sim b + 1$).

Arithmetic operations are implemented by instantiating the `MultifunctionAdderSubtractor64` module. This modular design allows the ALU to reuse the functionality of the adder-subtractor module, which includes handling various operand sizes and signed or unsigned computations. Inputs such as a, b, and opcode are connected to the corresponding inputs of the `MultifunctionAdderSubtractor64`, and its outputs are mapped to the ALU outputs.

Logical operations, such as AND, OR, XOR, SLL (shift left logical), SRL (shift right logical), and SRA (shift right arithmetic), are implemented using a multiplexer (`MuxCase`). The operation is selected based on the opcode bits b4, b3, and b2, and the result is masked to the effective operand width.

The ALU combines arithmetic and logical functionalities by selecting the result based on the operation type (`isArithmetic` or `isLogical`). For arithmetic operations, the result and flags are directly obtained from the `MultifunctionAdderSubtractor64`. Logical operations, on the other hand, do not produce flags such as carry or overflow; these outputs are set to zero when a logical operation is performed.

The ALU includes optional debugging functionality, controlled by the `printDebugInfo` flag. When enabled, it prints intermediate values such as operand adjustments, masks, and results, aiding in testing and verification.

8.13 Problems

8.13.1 Waveforms

Problem 8.1 *Generate the following waveforms:*

$$W = a^3 c^4 b a^2$$

where: $a = 2'b01$, $b = 2'b10$, $c = 2'b11$. and each value is maintained 2 time units (#2).

◇

Solution

Because each symbol is coded on 2 bits, results 2 wave forms: w_1 and w_0 , as follows:

$$W = \begin{bmatrix} w_1 \\ w_0 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 0 & 1 & 1 & 1 & 1 & 1 & 0 & 0 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 0 & 1 & 1 \end{bmatrix}$$

```

1  module waveform1;
2      logic [1:0] W;
3      parameter    a = 2'b01,
4                   b = 2'b10,
5                   c = 2'b11;
6
7      initial begin          W = a    ;
8                             #6 W = c    ;
9                             #8 W = b    ;
10                            #2 W = a    ;
11                            #4 $stop    ;
12
13     end
14 endmodule

```

Listing 8.39: Waveforms. File: A08_waveform1.sv (SystemVerilog)

The resulting waveforms are captured in Figure 8.35.

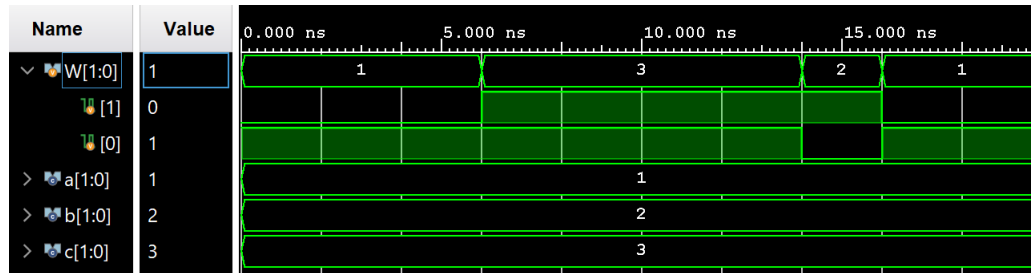


Figure 8.35: Output for waveform1.sv

Problem 8.2 Generate the following waveforms:

$$(ab^2c)^3$$

where: $W = a = 2'b11$, $b = 2'b10$, $c = 2'b01$, and each value is maintained 2 time units (#4).

◇

Problem 8.3 Generate a clock signal with 2 time units period and the following waveforms:

$$(a^3b^2c)^2$$

where: $W = a = 2'b11$, $b = 2'b10$, $c = 2'b01$, and each value is changed synchronously with the negative edge of the clock signal.

◇

8.13.2 Combinatorial Circuits

Problem 8.4 Design in System Verilog the structural solution at the gate level for a half adder.

◇

Solution

A half adder is defined by the following Boolean expressions:

$$sum = a \oplus b$$

$$cr = a \cdot b$$

```

1 module halfAdder(    input  logic a, b,
2                      output logic sum, cr);
3
4     xor myXor(sum, a, b);
5     and myAnd(cr, a, b);
6 endmodule

```

Listing 8.40: Half adder. File: A08_halfAdder.sv (SystemVerilog)

Using Vivado tool the elaborated design represented in Figure 8.36 is provided.

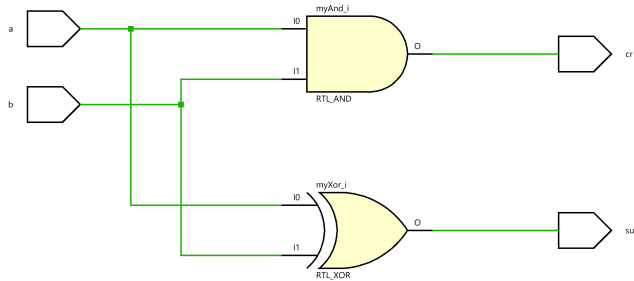


Figure 8.36:

Problem 8.5 Design in System Verilog the structural solution at the gate level for a full-half adder using two solutions:

- describe at the gate level
- a hierarchical approach based on the solution of the previous problem using a top module where two half-adders are instantiated.

Provide the simulation of the circuit to test exhaustively its behavior.

◇

Solution

For the first solution we have the following design:

```

1  module fullAdder(    input    logic a, b, crIn,
2                      output    logic sum, crOut);
3      logic sum0, cr0, cr1;
4
5      xor myXor(sum0, a, b);
6      xor outXor(sum, sum0, crIn);
7      and myAnd1(cr0, a, b);
8      and myAnd2(cr1, sum0, crIn);
9      or myOr(crOut, cr0, cr1);
10 endmodule

```

Listing 8.41: Full adder. File: A08_fullAdder.sv (SystemVerilog)

Using Vivado tool the elaborated design represented in Figure 8.37 is provided.

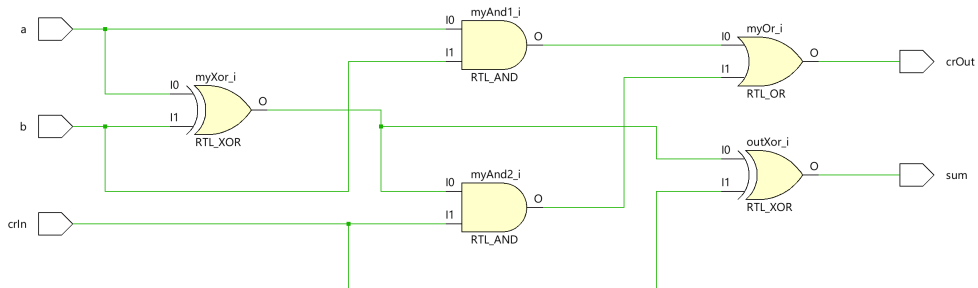


Figure 8.37:

For the second solution we have the following design:

```

1  module hFullAdder(  input    logic a, b, crIn,
2                      output    logic sum, crOut);
3      logic sum0, cr0, cr1;
4
5      halfAdder  ha0(.a (a      ),
6                    .b (b      ),
7                    .sum(sum0   ),
8                    .cr (cr0    )),
9      halfAdder  ha1(.a (sum0   ),
10                    .b (crIn    ),
11                    .sum(sum    ),
12                    .cr (cr1    ));
13      or outOr(crOut, cr0, cr1);
14 endmodule

```

Listing 8.42: Hierarchical Full-Adder. File: A08_hFullAdder.sv (SystemVerilog)

Using Vivado tool the elaborated design represented in Figure 8.38 is provided.

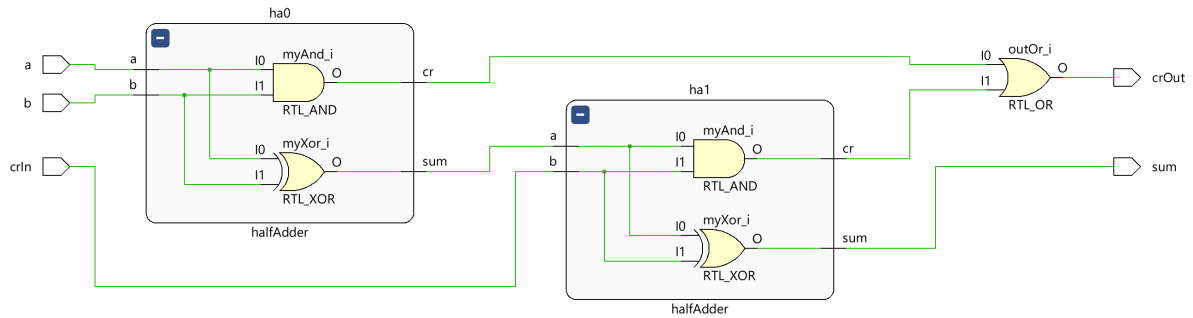


Figure 8.38:

For testing the functionality of the full-adder the following module is designed:

```

1  module testFullAdder
2      logic a, b, crIn, sum, crOut;
3
4      hFullAdder dut(a, b, crIn, sum, crOut);
5
6      initial begin          {a,b,crIn} = 3'b000 ;
7                          #1 {a,b,crIn} = 3'b001 ;
8                          #1 {a,b,crIn} = 3'b010 ;
9                          #1 {a,b,crIn} = 3'b011 ;
10                         #1 {a,b,crIn} = 3'b100 ;
11                         #1 {a,b,crIn} = 3'b101 ;
12                         #1 {a,b,crIn} = 3'b110 ;
13                         #1 {a,b,crIn} = 3'b111 ;
14                         #1 $stop
15                     end
16 endmodule

```

Listing 8.43: Hierarchical Full-Adder. File: A08_testFullAdder.sv (SystemVerilog)

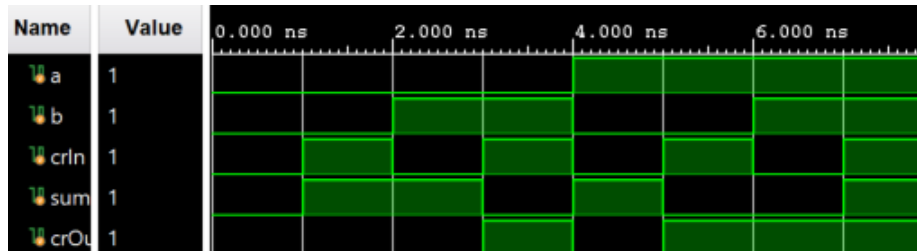


Figure 8.39:

8.14 Exercises

Problem 8.6 Design a 4-bit number adder with carry input and output.

◇

Problem 8.7 Design a 4-bit number subtractor with carry input and output.

◇

Problem 8.8 Design a 4-bit number adder/subtractor with carry input and output. The input `op` designates addition for `op = 0`, and subtract for `op = 1`.

◇

Problem 8.9 Fill in the missing lines in Table ?? and complete the project partially defined by `sevenSegDys.sv`. Test the correct operation.

◇

Problem 8.10 Write a full tester for `dcd.sv` and run it.

◇

Problem 8.11 Design in System Verilog a 4-input decoder using the recursive definition provided in Figure ?. Test its correct functioning.

◇

Problem 8.12 Design in System Verilog a 4-input demultiplexer using `DCD4` designed as solution for Problem 8.11. Test its correct functioning.

◇

Problem 8.13 Design in System Verilog a 4-input multiplexer using `DCD4` designed as solution for Problem 8.11. Test its correct functioning

◇

Problem 8.14 Consider a decoder with 4 inputs, denoted `A[3:0]`, whose outputs are connected to the selected inputs of a multiplexer with output `F` and with 4 selection inputs, denoted `B[3:0]`. What is the transfer function, $F = f(A, B)$, of the system with two 4-bit inputs, `A` and `B` and a one-bit output `F` ?

◇

Problem 8.15 Provide an optimal solution for a circuit which receives 2 8-bit words, `A[7:0]` and `B[7:0]`, and provide one bit output, `EQ`, telling if the two words are identical or not:

$$EQ = \begin{cases} 1 & \text{if } A == B \\ 0 & \text{if } A != B \end{cases} \quad (8.1)$$

◇

Problem 8.16 Test the behavior of the ALU defined by `alu.sv` in Section 8.12.

◇

Problem 8.17 Modify the module `alu.sv` from Section 8.12 to save the bit `left[0]`, when the right shift is performed, to the output `crOut`.

◇

Problem 8.18 Add to the module `alu.sv` from Section 8.12 the following functionality and test it:

- right shift with `crIn` on the most significant position
- arithmetic (right) shift which maintains the sign of the signed integer
- rotate left
- rotate right
- equality comparison: `left == right`
- inequality comparison: `left >= right`
- select the maximum: `out = (left <= right) ? left : right`
- select minimum: `out = (left > right) ? left : right`

◇

Problem 8.19 Design structurally at the gate level in System Verilog the slice of the simple ALU defined in Figure 8.34.

◇

Chapter 9

Memory Circuits: One-Loop, First-Order Systems (1-OS)

9.1	Latch	170
9.1.1	Closing the first loop	170
9.1.2	Clocked latch	172
9.2	Serial extension: Master-Slave Principle	173
9.3	Serial-parallel extension: Register	175
9.3.1	Structure	175
9.3.2	Applications	176
9.4	Parallel extension: Random-Access Memory	177
9.4.1	Generic structure	177
9.4.2	Synchronous RAM	180
9.4.3	Synchronous pipelined RAM	181
9.4.4	Register file	181
9.4.5	Representation of B_n in MSB and LSB Forms with Endianness	183
9.5	Problems	185
9.5.1	Registers	185
9.5.2	Memories	187

In the second lesson we will introduce the memory circuits that are obtained through a first loop that brings to the input of a circuit the effect of applying a signal to another input. This reverse connection allows the circuit to maintain a state according to a command. Thus the memory function is obtained.

We will not go into details because we have a precise target: the internal structure of a computing system that includes a processor, with its internal storage resources, and the memories in which data and software are stored.

9.1 Latch

The simplest circuit will allow the latching of the simplest event: the temporary transition of a digital signal from 1 to 0, or the temporary transition of some circuit from 0 to 1.

9.1.1 Closing the first loop

Closing a feedback loop requires an output of a logic circuit to be connected to one of its inputs. In the Figure 9.1a, the output of an AND gate is connected to one of the inputs, and the (Reset)' pulses are applied to the other input. If the transition to zero lasts long enough so that the signal propagates through the circuit to the second input, then the output is fixed at the zero value and the signal (Reset)' from the input can disappear. The transition event from the input is fixed to the output forever.

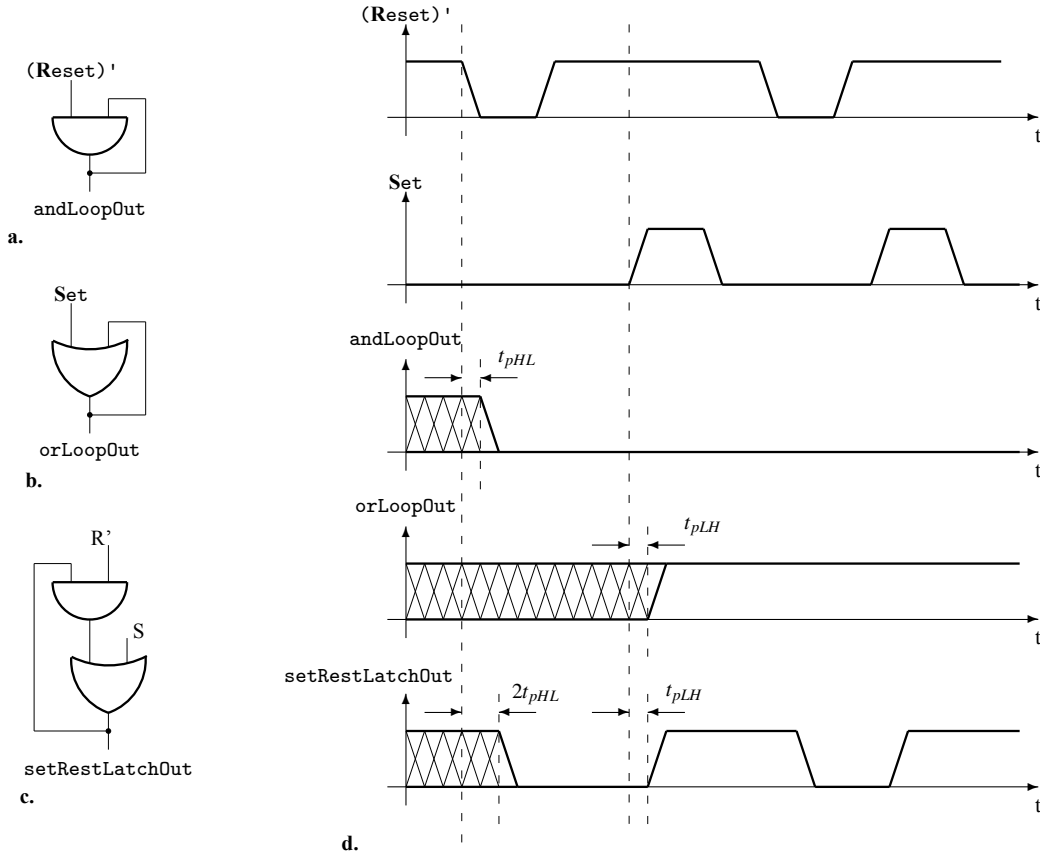


Figure 9.1: **The elementary latches.** Using the loop, closed from the output to one input, elementary storage elements are built. **a.** *AND loop* provides a *reset-only* latch. **b.** *OR loop* provides the *set-only* version of a storage element. **c.** The heterogeneous elementary *set-reset* latch results combining the *reset-only* latch with the *set-only* latch. **d.** The wave forms describing the behavior of the previous three latch circuits.

Now let's take an OR gate and connect its output to one of the inputs, and on the other, apply a transition from 0 to 1. If the duration of the input signal allows the propagation of its effect on the reaction loop, the output of the circuit will be permanently fixed on the value 1.

The good news is that we can capture temporary events indefinitely. We say that the two circuits memorize, one the temporary transition to 0, the other the temporary transition to 1. The bad news is that we cannot make these circuits "forget" what they have memorized. But, by combining the two circuits, we manage to obtain a fundamental memory structure that can change its state according to the set command, S , to 1 or according to the reset command, R' , to 0. The set command is active on 1, in while the reset command is active on 0.

We obtained a circuit with two states between which it can switch according to asymmetrical commands, one active on zero and the other active on one. For reasons of use, it is preferable that both commands are of the same type. We will use the two forms of Morgan's law to transform the circuit in Figure 9.1c and we will obtain two symmetrical structures ordered with signals of the same type. Using

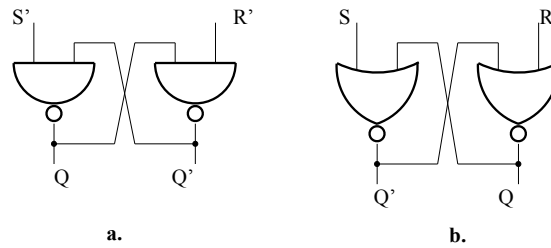


Figure 9.2: **Symmetric elementary latches.** **a.** Symmetric elementary *NAND* latch with low-active commands S' and R' . **b.** Symmetric elementary *NOR* latch with high-active commands S and R .

the de Morgan rule $A + B = (A'B')'$, the latch from Figure 9.1c becomes the latch from Figure 9.2a, and using the de Morgan rule $AB = (A'+B')'$, the latch from Figure 9.1c becomes the latch from Figure 9.2b.

9.1.2 Clocked latch

For the clocked latch two NAND gate are added (see Figure 9.3) to allow the R and R inputs conditioned by clock>

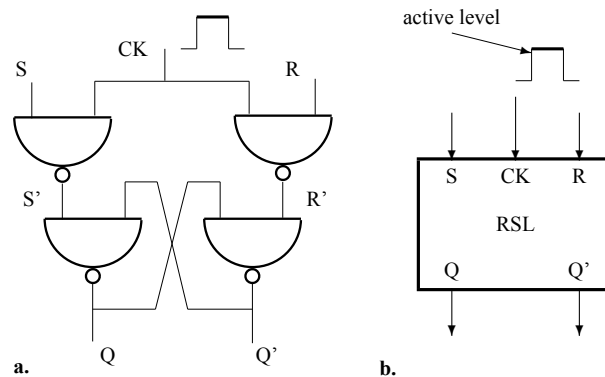


Figure 9.3: **Elementary clocked latch.** The transparent RS clocked latch is sensitive (transparent) to the input signals during the active level of the clock (the high level in this example). **a.** The internal structure. **b.** The logic symbol.

The first latch problem: the inputs for indicating *how* the latch switches are the same as the inputs for indicating *when* the latch switches; we must find a solution for declutching the two actions building a version with distinct inputs for specifying “how” and “when”.

The second latch problem: if we apply synchronously $S'=0$ and $R'=0$ on the inputs of NAND latch (or $S=1$ and $R=1$ on the inputs of OR latch), i.e., the latch is commanded “to switch in both states simultaneously”, then we can not predict what is the state of the latch after the ending of these two active signals.

The first problem is solved in Figure 9.3a by conditioning the application of signals S' and R' to the

latch in Figure 9.2a by an additional signal that we will call clock, CK. Thus, we will use the S and R inputs to specify how the circuit switches, and the CK input to specify when the circuit switches. In this way, the decoupling of "when" from "how" is obtained.

For the second problem, we have a more "brutal" solution achieved through a restriction. We will connect between the two inputs a NOT that will not allow the simultaneous application of setting and resetting the circuit (see Figure 9.4a). The problem is not solved. We avoid doing a wrong use.

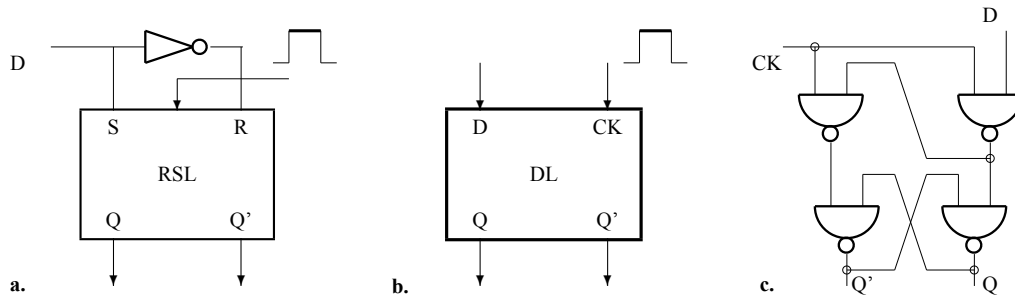


Figure 9.4: **The data latch.** Imposing the restriction $R = S'$ to an RS latch results the **D latch** without non-predictable transitions ($R = S = 1$ is not anymore possible). **a.** The structure. **b.** The logic symbol. **c.** An improved version for the data latch internal structure.

For the time interval in which the clock is active, $CK = 1$, it says that the latch is transparent to the control signal S and R. The latch becomes non-transparent in the time interval in which $CK = 0$. Indeed, in the period in which the latch is transparent, the decoupling between "when" and "how" is not achieved, and the circuit switches conditioned by the evolution of the S and R inputs. A correct use of the latch assumes that during the transparency period the S and R inputs are stable.

We have to admit that we did not separate the "when" from the "how" rigorously enough. We will do it in the following.

9.2 Serial extension: Master-Slave Principle

The clocked latch, in the SR version as well as in the D version (data latch), allows switching at any time during the transparency. This fact limits the applications of this circuit. Many applications require switching precisely determined by the active, positive or negative transition of the clock signal. Applying the *master-slave principle* will allow this behavior.

Figure 9.5a shows a structure where transparency is blocked by the fact that the two RS latches have the clock applied in antiphase. On the active level of the clock, the positive one, the first latch, the master, is transparent, a fact that allows it to be switched according to the S and R inputs without affecting the output of the circuit because the second latch is not transparent. On the inactive level of the clock, the second latch, the slave, copies the state of the master which can no longer be changed. In this way, the entire circuit switches as a consequence of the negative transition of the clock signal. So, the active edge of the clock applied to the master-slave structure is the negative one. In Figure 9.5b, the logical symbol of the structure in Figure 9.5a is represented.

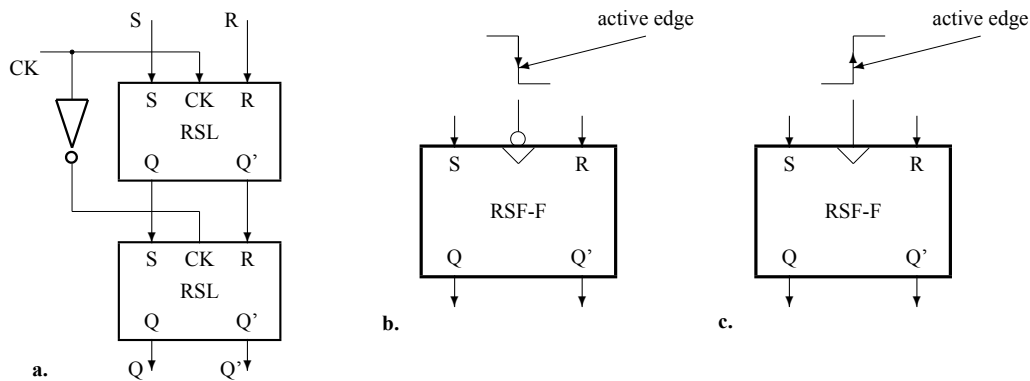


Figure 9.5: **The master-slave principle.** Serially connecting two RS latches, activated with different levels of the clock signal, results a non-transparent storage element. **a.** The structure of a RS master-slave flip-flop, active on the falling edge of the clock signal. **b.** The logic symbol of the RS flip-flop triggered by the *negative edge* of clock. **c.** The logic symbol of the RS flip-flop triggered by the *positive edge* of clock.

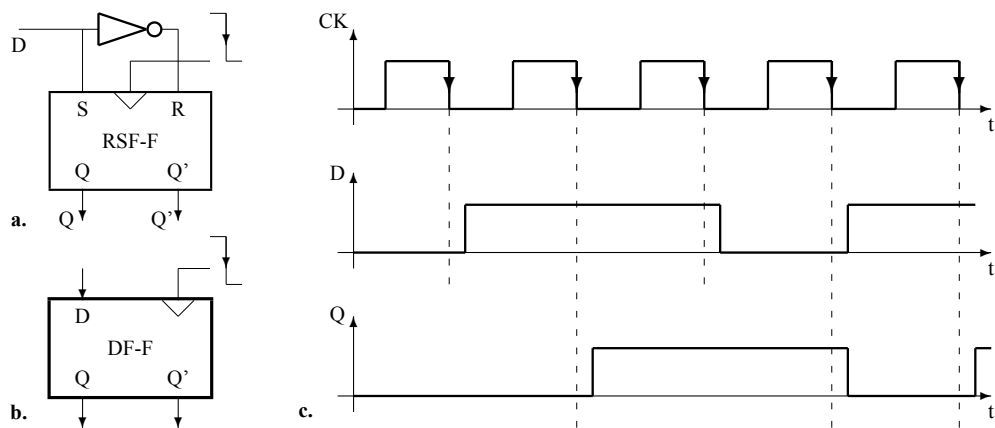


Figure 9.6: **The delay (D) flip-flop.** Restricting the two inputs of an RS flip-flop to $D = S = R'$, results an FF with predictable transitions. **a.** The structure. **b.** The logic symbol. **c.** The wave forms proving the delay effect of the D flip-flop.

The second latch problem also propagates at the level of the master-slave structure. The solution we can come up with at this level is the one that introduces the limitation by connecting the inputs through an inverter circuit (see Figure 9.6a). The result is the D-FF (delay flip-flop) circuit. The name is justified by the waveforms represented in Figure 9.6c where the output of the circuit follows the evolution of the input with a delay equal to the period of the clock signal.

9.3 Serial-parallel extension: Register

9.3.1 Structure

The serial-parallel extension in the field of first-order systems (1-OS) is represented by the register. By connecting in parallel n serially extended structures (the delay flip-flop type master-slave circuit) is obtained a **register** by applying the same clock signal on each D-FF (see Figure 9.7a). A register stores an n -bit configuration synchronously with the active edge of the clock.

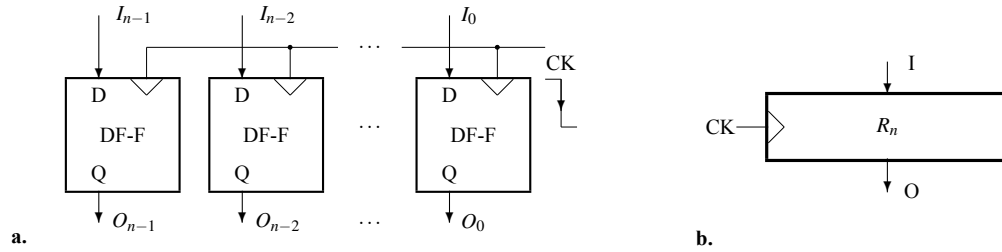


Figure 9.7: **The n -bit register.** **a.** The structure: a bunch of DF-F connected in parallel. **b.** The logic symbol.

The Figure 9.8 shows the effect of the transfer through a register.

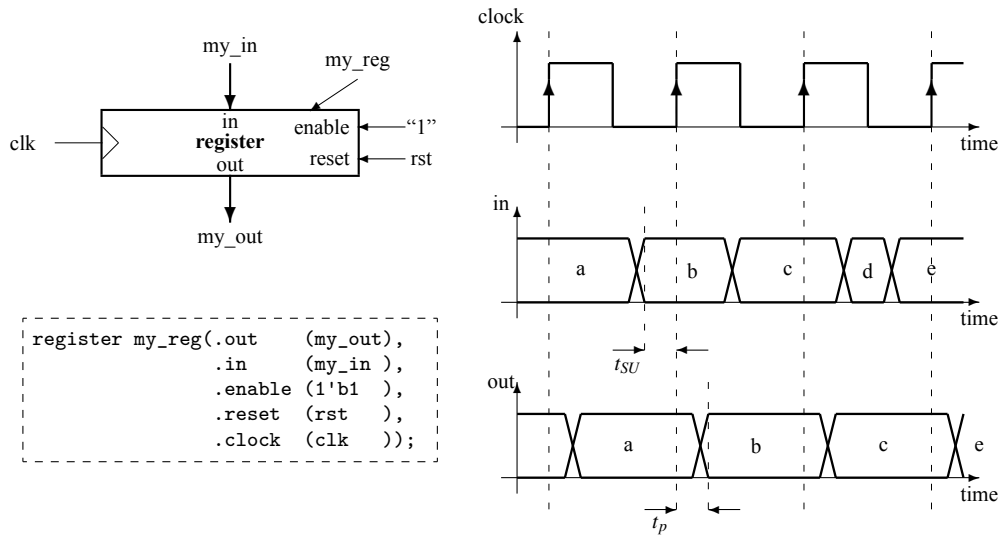


Figure 9.8: **Register operation.** At each active edge of clock (in this example it is the positive edge) the register's output takes the value applied on its inputs if $reset = 0$ and $enable = 1$.

The time behavior is mainly characterized by two times interval:

minimal set-up time : $t_{SU_{min}}$ is the minimum time interval that the input must be kept stable before the active edge of the clock so that the transition of the output corresponds to the input

maximal propagation time : t_p is the propagation time to the output related to the active edge of clock.

9.3.2 Applications

Follows the list of the main applications of the register in digital systems.

9.3.2.1 Storing

The `enable` input allows us to determine when (i.e., in what clock cycle) the input is loaded into a register. If `enable = 0`, the registers *stores* the data loaded in the last clock cycle when the condition `enable = 1` was fulfilled. This means we can keep the content once stored into the register as much time as it is needed.

9.3.2.2 Buffering

The registers can be used to *buffer* (to isolate, to separate) two distinct blocks so as some behaviors are not transmitted through the register. For example, in Figure 9.8 the transitions from `c` to `d` and from `d` to `e` at the input of the register are not transmitted to the output.

9.3.2.3 Synchronizing

For various reasons the digital signals are generated “unaligned in time” to the inputs of a system, but they are needed to be received very well controlled in time. We say usually, the signals are applied *asynchronously* but they must be received *synchronously*. For example, in Figure 9.8 the input of the register changes somehow chaotically related to the active edge of the clock, but the output of the register switches with a constant delay after the positive edge of clock. We say the inputs are synchronized to the output of the register. Their behavior is “time tempered”.

9.3.2.4 Delaying

The input value applied in the clock cycle `n` to the input of a register is generated to the output of the register in the clock cycle `n+1`. In other words, the input of a register is delayed one clock cycle to its output. See in Figure 9.8 how the occurrence of a value in one clock cycle to the register’s input is followed in the next clock cycle by the occurrence of the same value to the register’s output.

9.3.2.5 Looping

Structuring a digital system means to make different kind of connections. One of the most special, *as we see in what follows*, is a connection from some outputs to certain inputs in a digital subsystem. This kind of connections are called **loops**. The register is an important structural element in closing controllable loops inside a complex system.

9.3.2.6 Pipelining

Example 9.1 *The previously exemplified `threeAdder.sv` module performs the addition of three numbers in twice the time of the sum of two numbers. We can reduce the execution time through a pipeline solution that uses two registers. The solution is presented in the following module:*

```
1 module pipelinedThreeAdder( output logic [3:0] out      ,
```

```

2          input  logic [3:0] in0      ,
3          input  logic [3:0] in1      ,
4          input  logic [3:0] in2      ,
5          input  logic      clock     );
6  logic [3:0] syncReg ;
7  logic [3:0] pipeReg ;
8  logic [3:0] sum     ;
9
10 adder  inAdder(    .out(sum      ),
11                  .in0(in0      ),
12                  .in1(in1      )),
13          outAdder( .out(out      ),
14                  .in0(syncReg),
15                  .in1(pipeReg));
16  always_ff @(posedge clock) begin  syncReg <= in2  ;
17                                     pipeReg <= sum  ;
18                                     end
19 endmodule

```

Listing 9.1: The module 'threeAdder' has 3 4-bit inputs and one 4-bit output. The circuit adds modulo 16 three numbers; do not provide carry output. File: A09_pipelinedThreeAdder.sv (SystemVerilog)

Two modules of adder defined Example ?? are instantiated as inAdder and outAdder, they are interconnected using the pipeline register pipeReg. The register subcReg is used to synchronize the in2 input with the pipelined result provided by inAdder.

◇

9.4 Parallel extension: Random-Access Memory

The parallel extension in first-order systems (1-OS) is represented by random access memory (RAM).

9.4.1 Generic structure

The generic structure of a memory with 2^p one-bit locations is represented in the Figure 9.9.

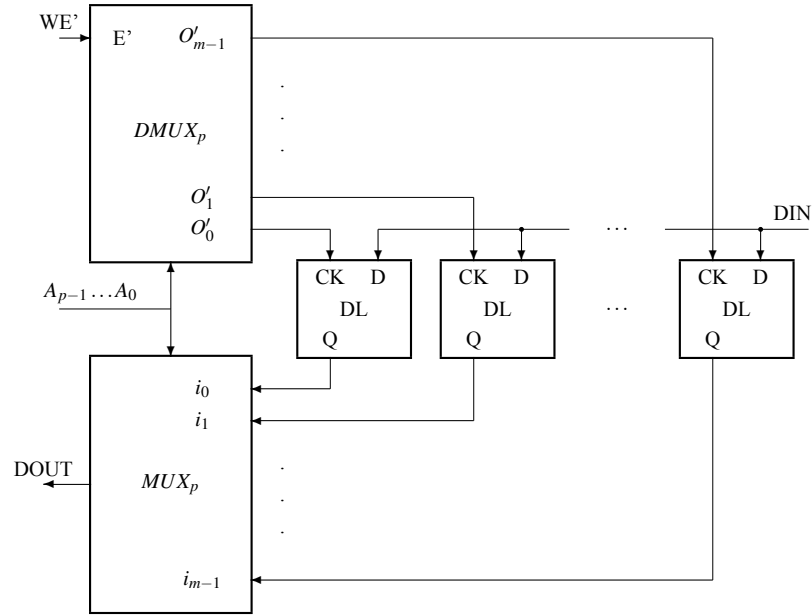


Figure 9.9: **The principle of the random access memory (RAM).** The clock is distributed by a DMUX to one of $m = 2^p$ DLs, and the data is selected by a MUX from one of the m DLs. Both, DMUX and MUX use as selection code a p -bit address. The one-bit data DIN can be stored in the clocked DL.

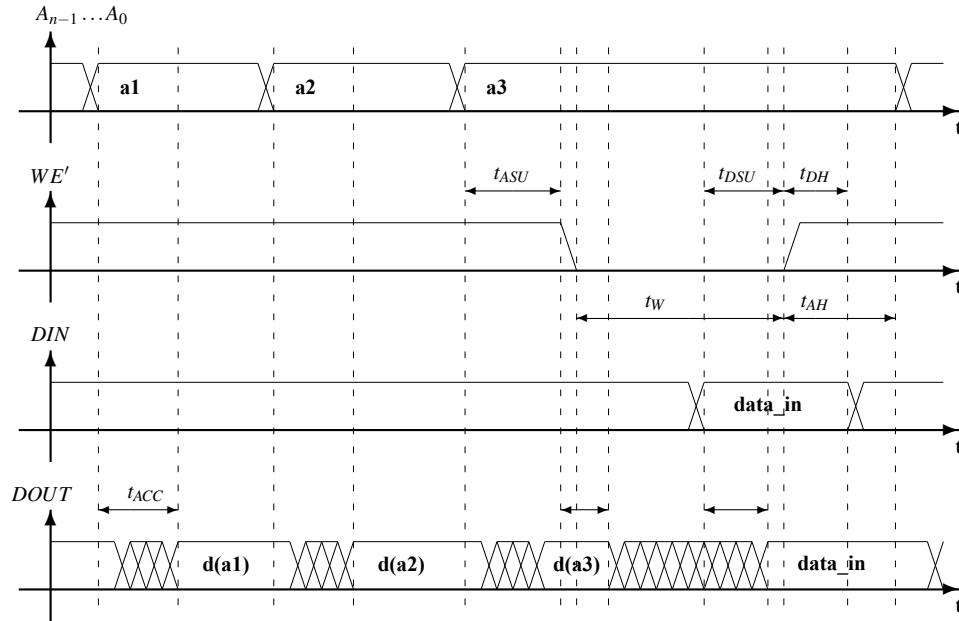


Figure 9.10: **Read and write cycles for an asynchronous RAM.** Reading is a combinational process of selecting. The access time, t_{ACC} , is given by the propagation through a big MUX. The write enable signal must be strictly included in the time interval when the address is stable (see t_{ASU} and t_{AH}). Data must be stable related to the positive transition of WE' (see t_{DSU} and t_{DH}).

The waveforms that define the operation of the structure in Figure 9.9 are represented in Figure 9.10 where the main restrictions are represented by:

t_{ASU} : address set-up time represents the time interval in which the address must be stable before the activation of the WE signal to allow the decoder in the DMUX to do its work.

t_{DSU} : data set-up time represents the time interval in which DIN must be stable before de-activating the WE signal to allow the correct closing of the reaction loop in the selected latch.

t_{DH} : data hold represents the time interval in which DIN must be stable after deactivating the WE signal to allow the correct closing of the reaction loop in the selected latch.

t_W : width of the write enable signal which ensures correct writing in the selected latch

t_{AH} : address hold time is the time interval in which the address must be maintained after the WE signal is provided

t_{ACC} : access time is the time interval after which the content of the selected cell is accessible at the output

All the previously listed times are minimum values.

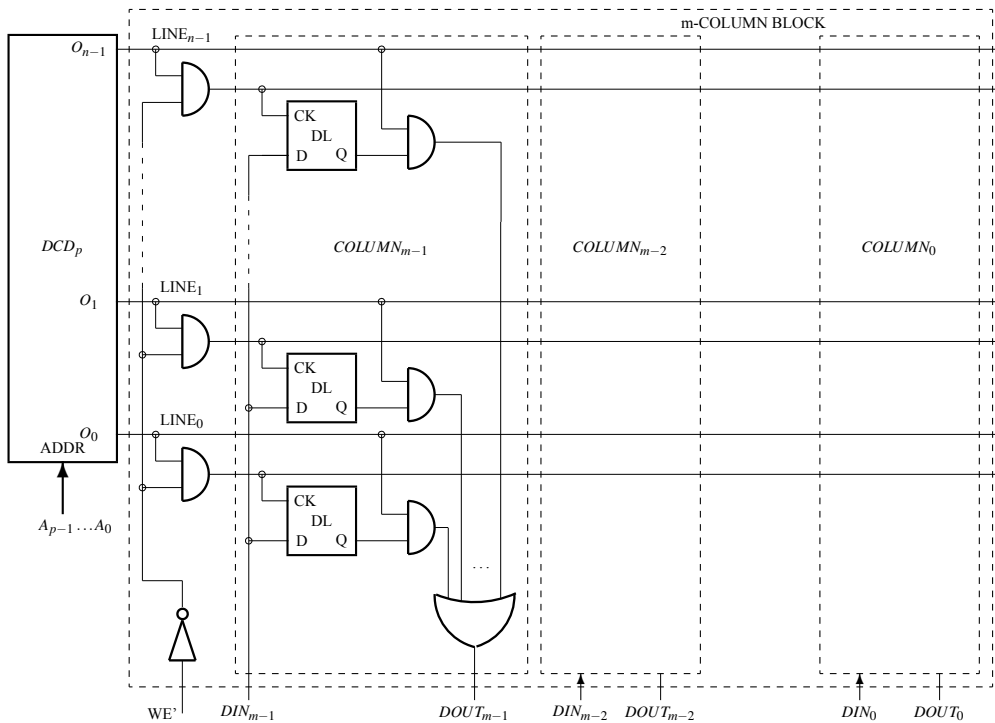


Figure 9.11: **The asynchronous m -bit word RAM.** Expanding the number of bits per word means to connect in parallel one-bit word memories which share the same decoder. Each COLUMN contains storing latches and AND-OR circuits for one bit.

How can you build a RAM memory that stores words of m bits? The generic solution is presented in Figure 9.11 where for each bit a column containing 2^p latches is defined.

In Figure 9.11, DCD_p is shared between the DMUX and the MUX in Figure 9.9. The ANDs associated with the demultiplexing function are selected with the WE signal. The AND-OR structure is repeated m times, once in each $COLUMN_i$ column.

9.4.2 Synchronous RAM

At very high speeds, on the order of GHz, the time restrictions illustrated in Figure 9.10 are very difficult to fulfill in the already described asynchronous version. To increase the degree of controllability of our projects, synchronous RAM memories are used. In Figure 9.12, the behavior of a synchronous memory (SRAM) is illustrated in the sense that all the time intervals that characterize the behavior of the memory are related to the active edge of a clock signal. In this case, the designer of a system that includes a synchronous memory can more rigorously control the time behavior of the system.

For the memories used by the system designer, *memory libraries* are provided by specialized companies in the form of IPs (intellectual property), guaranteeing the correctness of the code used in the description.

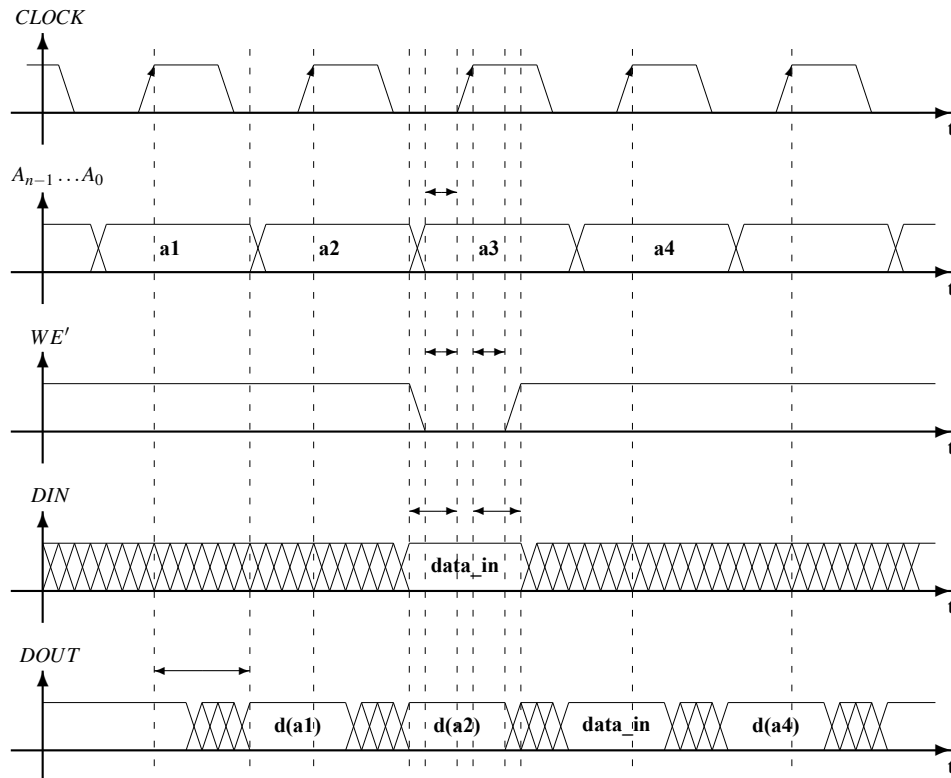


Figure 9.12: **Read and write cycles for Synchronous RAM (SRAM).** For the *flow-through* version of a SRAM the time behavior is similar to a register. The set-up and hold time are defined related to the active edge of clock for all the input connections: *data*, *write-enable*, and *address*. The data output is also related to the same edge.

```

1  module syncRAM( output logic [31:0] out ,
2                  input logic [31:0] in ,
3                  input logic [12:0] addr ,
4                  input logic we ,
5                  input logic clock );
6      logic [31:0] mem [0:8191] ;
7
8      always_ff @(posedge clock) if (we) mem[addr] <= in ;
9      assign out = mem[addr] ;
10 endmodule

```

Listing 9.2: 4096 32-bit words synchronous RAM; asynchronous read. File: A09_syncRAM.sv (SystemVerilog)

9.4.3 Synchronous pipelined RAM

The time t_{ACC} offered by a RAM can sometimes be too high for the speed requirements of the system. To increase the clock frequency, a register is used at the memory output. Thus, the data is obtained at the new output very quickly: t_{ACC} is the propagation time through the register. But there is a price: the **latency** of one clock cycle (we remember that the register consists of *delay* flip-flops).

A skilled designer will almost always be able to hide the effect of latency and take advantage of the increase in system frequency obtained through the pipeline register from the RAM output.

```

1  module syncPipeRAM( output logic [31:0] out ,
2                     input logic [31:0] in ,
3                     input logic [12:0] addr ,
4                     input logic we ,
5                     input logic clock );
6      logic [31:0] mem [0:8191] ;
7
8      always_ff @(posedge clock) begin if (we) mem[addr] <= in ;
9                                   out <= mem[addr] ;
10                               end
11 endmodule

```

Listing 9.3: 4096 32-bit words synchronous RAM; asynchronous read. File: A09_syncPipeRAM.sv (SystemVerilog)

9.4.4 Register file

A register file is a battery of registers from which, in each clock cycle, any two registers can be accessed for reading and any register for writing. The implementation of such a circuit is done with a synchronous memory with three ports: two for reading and one for writing (see Figure 9.13), where:

`leftAddr[n-1:0]`: is the address used to select data to the `leftOut[m-1:0]` output

`rightAddr[n-1:0]`: is the address used to select data to the `rightOut[m-1:0]` output

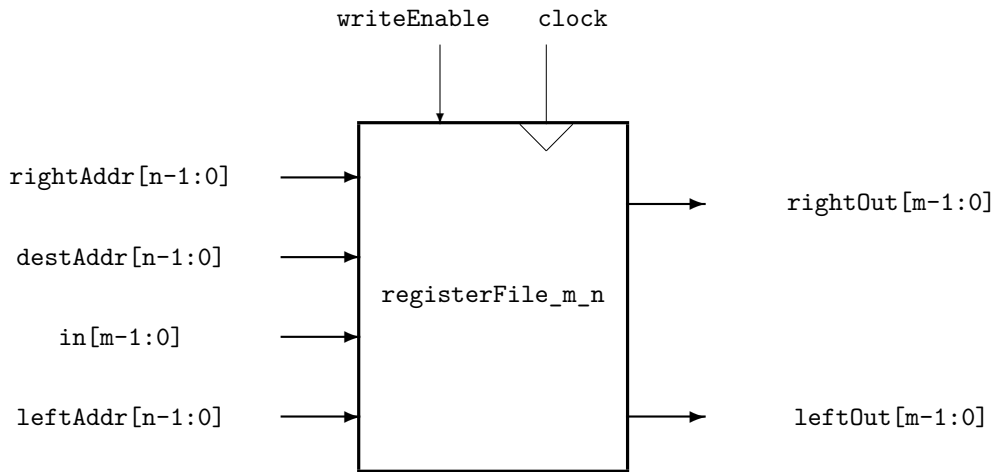


Figure 9.13: **Register file.** In this example it contains 2^n m -bit registers. In each clock cycle any two registers can be read and writing can be performed in anyone.

`destAddr[n-1:0]`: is the address used to select the location where the value applied on input `in[m-1:0]` is stored at the positive edge of the `clock` signal

`writeEnable`: is the write enable command.

The internal structure of a file register is characterized by the two multiplexers that ensure access to any pair of stored values.

```

1  module regFile( output logic [31:0] leftOut    ,
2                 output logic [31:0] righttOut   ,
3                 input  logic [31:0] in          ,
4                 input  logic [4:0] leftAddr     ,
5                 input  logic [4:0] rightAddr    ,
6                 input  logic [4:0] destAddr     ,
7                 input  logic we                ,
8                 input  logic clock              );
9
10  logic [31:0] mem[0:31] ;
11
12  always_ff @(posedge clock) if (we) mem[destAddr] <= in ;
13
14  assign leftOut  = mem[leftAddr] ;
15  assign rightOut = mem[rightAddr];
16 endmodule

```

Listing 9.4: Register file. File: A09_regFile.sv (SystemVerilog)

logic [m-1:0] mem[0:n-1]: defines a block of memory of n m -bit words

always_ff : defines a block which describes the behavior of register-like sequential circuits; it is followed always at least by **@(posedge clock)** or **@(negedge clock)** specifying the clock signal and its active edge. The non-blocking assignment, **<=**, is used to assure the sequential behavior. While **always_comb** describes the behavior of a combinational circuit, **always_ff** (**ff** comes from flip-flop) describes the behavior of the sequential, clock triggered, circuit called **register**. Now **logic** represents a register instead of an output of a combinational circuit like in the case of **always_comb**.

9.4.5 Representation of B_n in MSB and LSB Forms with Endianness

The set B_n , representing n -bit unsigned binary numbers, can be expressed using two common conventions for bit ordering: Most Significant Bit (MSB) first and Least Significant Bit (LSB) first. These conventions are closely tied to the concept of endianness in computer architecture, which determines how binary numbers are stored or transmitted in memory.

9.4.5.1 MSB Representation (Big-Endian Order)

In the Most Significant Bit (MSB) representation, the leftmost bit (b_{n-1}) corresponds to the highest positional value (2^{n-1}), and the rightmost bit (b_0) corresponds to the lowest positional value (2^0). This is the standard positional notation for binary numbers.

For B_n , the value of x is calculated as:

$$x = b_{n-1} \cdot 2^{n-1} + b_{n-2} \cdot 2^{n-2} + \cdots + b_1 \cdot 2^1 + b_0 \cdot 2^0, \quad b_i \in \{0, 1\}.$$

9.4.5.2 LSB Representation (Little-Endian Order)

In the Least Significant Bit (LSB) representation, the least significant bit (b_0) is stored or transmitted first. However, the numerical value of the number remains the same, as the binary digits are still interpreted using the same positional notation. The formula remains:

$$x = b_{n-1} \cdot 2^{n-1} + b_{n-2} \cdot 2^{n-2} + \cdots + b_1 \cdot 2^1 + b_0 \cdot 2^0, \quad b_i \in \{0, 1\}.$$

The difference lies in the physical storage or transmission order: - In big-endian systems, bits are stored or transmitted starting with the most significant bit (b_{n-1}). - In little-endian systems, bits are stored or transmitted starting with the least significant bit (b_0).

Example for $n = 3$

For $B_3 = \{0, 1, 2, 3, 4, 5, 6, 7\}$, the logical representation (big-endian) is:

Logical Representation (Big-Endian):

$$\begin{aligned}
x = 0 &\Rightarrow 000, \\
x = 1 &\Rightarrow 001, \\
x = 2 &\Rightarrow 010, \\
x = 3 &\Rightarrow 011, \\
x = 4 &\Rightarrow 100, \\
x = 5 &\Rightarrow 101, \\
x = 6 &\Rightarrow 110, \\
x = 7 &\Rightarrow 111.
\end{aligned}$$

In a little-endian system, the storage or transmission order of bits is reversed. For instance: - $x = 1$ (big-endian: 001) is stored as 100 (little-endian). - $x = 6$ (big-endian: 110) is stored as 011 (little-endian).

Physical Representation in Little-Endian (Stored Order):

$$\begin{aligned}
x = 0 &\Rightarrow 000, \\
x = 1 &\Rightarrow 100, \\
x = 2 &\Rightarrow 010, \\
x = 3 &\Rightarrow 110, \\
x = 4 &\Rightarrow 001, \\
x = 5 &\Rightarrow 101, \\
x = 6 &\Rightarrow 011, \\
x = 7 &\Rightarrow 111.
\end{aligned}$$

9.4.5.3 Commentary on Endianness

- **Big-Endian Systems**: - Big-endian systems store numbers in memory starting with the most significant bit. This is often the default in higher-level representations of binary numbers and is consistent with standard positional notation.

- **Little-Endian Systems**: - Little-endian systems store numbers in memory starting with the least significant bit. This convention simplifies certain hardware operations, such as incrementing binary counters, and is widely used in modern processors like x86 architectures.

- **Practical Implications**: - While the logical representation of binary numbers is the same regardless of endianness, understanding the storage or transmission order is crucial for tasks like data serialization, memory addressing, and communication protocols.

The binary expansions of x differ in the ordering of bits depending on whether the **Most Significant Bit (MSB)** or **Least Significant Bit (LSB)** form is used. These differences arise from practical applications in engineering versus conventions in mathematics.

Mathematical Preference: MSB Form

Mathematicians universally use the **MSB expansion** to represent binary numbers. The MSB form is consistent with the positional arithmetic taught in grade school, where each digit corresponds to a specific positional value. For n -bit unsigned binary numbers, the MSB expansion is:

$$x = b_{n-1} \cdot 2^{n-1} + b_{n-2} \cdot 2^{n-2} + \cdots + b_1 \cdot 2^1 + b_0 \cdot 2^0, \quad b_i \in \{0, 1\}.$$

This form aligns naturally with:

- **Grade school arithmetic**: Positional notation, where the leftmost digit has the highest weight, and each subsequent digit corresponds to decreasing powers of the base.
- **Decimal analogy**: For example, the decimal number 345 is interpreted as:

$$345 = 3 \cdot 10^2 + 4 \cdot 10^1 + 5 \cdot 10^0.$$

Similarly, in binary, 110 is interpreted as:

$$110 = 1 \cdot 2^2 + 1 \cdot 2^1 + 0 \cdot 2^0.$$

Thus, the MSB form is preferred in mathematics for its clarity and compatibility with standard positional arithmetic.

Use of LSB Form in Engineering and Computing

In contrast, the **LSB expansion** reverses the storage or transmission order of bits, prioritizing the least significant bit (b_0). The formula remains the same, but the physical ordering differs, reflecting how the bits are handled in memory or transmitted serially. For example:

$$x = b_0 \cdot 2^0 + b_1 \cdot 2^1 + \cdots + b_{n-2} \cdot 2^{n-2} + b_{n-1} \cdot 2^{n-1}.$$

The LSB form is not used in pure mathematics but is important in:

- **Computer systems**: - Little-endian architectures (e.g., x86) use LSB-first order for storing and transmitting binary numbers. - This simplifies certain hardware operations, such as incrementing binary counters or performing addition bit-by-bit starting from the least significant bit.
- **Data communication**: - Serial communication protocols often transmit the least significant bit first due to hardware design constraints.

Commentary on MSB as Positional Arithmetic

The MSB expansion closely resembles the positional arithmetic taught in elementary education. Each digit (or bit) contributes to the total value based on its position, with weights corresponding to powers of the base (in this case, 2). This positional system is universally understood and forms the foundation for number systems across different bases (e.g., decimal, binary, hexadecimal).

In contrast, the LSB form prioritizes computational efficiency in specific applications rather than logical clarity. While it has practical importance in engineering and computing, it is rarely used for manual or theoretical calculations.

Conclusion

Mathematicians exclusively use the MSB form due to its consistency with positional arithmetic and its alignment with grade school notation. The LSB form is primarily a practical convention in computer architecture and data communication, where hardware efficiency and specific operational needs dictate its use.

9.5 Problems

9.5.1 Registers

Problem 9.1 *Design a system that receives a signed integer in each clock cycle and outputs the sum of the last four received numbers. When the system is reset, it is considered that the last four received*

numbers had the value 0.

If $t_{su_{min}} = \#1$, $t_p = \#5$, $t_{sum} = \#50$, then what is the minimum period of the clock at which the system can present a correct result when entering a register.

◇

Solution

The system consists in 4 registers, `reg0`, ..., `reg3`, serially connected and a tree of three adders used to sum the content of the registers. The first register, `reg0`, receives in each clock cycle the input value, `in`. Register `reg1` receives the content of `reg0`, `reg2` receives the content of `reg1`, and `reg3` the content of `reg2`. The first two and the last two registers are added using two adders whose outputs are added using the output adder. The division by 4 is simply performed selecting the 32 most significant bits from the output of the last adder.

```

1  module mean(output logic [31:0] out ,
2             input logic [31:0] in ,
3             input logic reset ,
4             input logic clock );
5      logic [31:0] reg0;
6      logic [31:0] reg1;
7      logic [31:0] reg2;
8      logic [31:0] reg3;
9      logic [32:0] sum0;
10     logic [32:0] sum1;
11     logic [33:0] sum;
12
13     always_ff @(posedge clock) if( reset) begin    reg0 <= 0 ;
14                                           reg1 <= 0 ;
15                                           reg2 <= 0 ;
16                                           reg3 <= 0 ;
17
18                                           end
19     else begin    reg0 <= in ;
20                                           reg1 <= reg0;
21                                           reg2 <= reg1;
22                                           reg3 <= reg2;
23
24                                           end
25
26     assign #1 sum0 = reg0 + reg1;
27     assign #1 sum1 = reg2 + reg3;
28     assign #1 sum = sum0 + sum1;
29     assign out = sum[33:2] ;
30 endmodule

```

Listing 9.5: Circuit that compute the mean value from the last component of a received stream of numbers. File: A09_mean.sv (SystemVerilog)

```

1  module testMean;
2      logic [31:0] out ;
3      logic [31:0] in ;

```

```

4      logic          reset          ;
5      logic          clock          ;
6
7      initial begin                clock = 0          ;
8                                  forever #3 clock = ~clock ;
9      end
10
11     initial begin                reset = 1          ;
12                                  in      = 32'b1000;
13                                  #6 reset = 0          ;
14                                  #60 $finish          ;
15     end
16
17     mean dut( out      ,
18              in       ,
19              reset    ,
20              clock    );
21 endmodule

```

Listing 9.6: Tester for circuit mean. File: A09_testMean.sv (SystemVerilog)

The minimal period of clock is: $T_{clock} = t_p + 2 \times t_{sum} + t_{SU_{min}} = \#106$

Problem 9.2 Revisit the Problem 9.1 and insert the first two adders and the output adder pipe registers.

- 1) What is the resulting minimal period of clock
- 2) Design this pipelined version
- 3) Simulate the resulting design.

◇

Problem 9.3 Design a fully buffered ALU and evaluate the minimal period of the clock signal if $t_{SU_{min}} = \#1$, $t_p = \#5$, $t_{ALU} = \#60$. Fully buffered means inputs on ALU are provided from registers and the output is send through a register.

◇

Problem 9.4 Design a system that receives a string of bytes synchronized with the clock and transfers it synchronously with a selectable delay. The selection is made with `sel[1:0]` indicating a delay of 1, 2, 3, or 4 clock cycles.

◇

9.5.2 Memories

Problem 9.5 Design a synchronous pipelined RAM for $2^{addressSize}$ (`wordSize`)-bit words.

◇

Solution

```

1  `define wordSize      32
2  `define addressSize  16

```

Listing 9.7: Define the parameterized synchronous RAM. File: A09_defineParamSyncPipeRAM.vh (SystemVerilog)

```

1  `include "defineParamSyncPipeRAM.vh"
2  module syncPipeRAM( output logic [`wordSize-1:0] out ,
3                     input logic  [`wordSize-1:0] in  ,
4                     input logic  [`addressSize-1:0] addr ,
5                     input logic                                     we ,
6                     input logic                                     clock );
7      logic [`wordSize-1:0] mem[0:2**{`addressSize}-1] ;
8
9      always_ff @(posedge clock) begin if (we) mem[addr] <= in ;
10                                     out <= mem[addr] ;
11                                     end
12  endmodule

```

Listing 9.8: Parameterized synchronous RAM. File: A09_paramSyncPipeRAM.sv (SystemVerilog)

Problem 9.6 Consider memory modules of 1KB and design with them a memory of 4K 32-bit word.

◇

Problem 9.7 Simulate the behavior of the register file described in Section C.2.2 to prove the read-during-write operation.

◇

Chapter 10

Automata: Two-Loop, Second-Order Systems (2-OS)

10.1	Definitions	190
10.1.1	Generic Definition	190
10.1.2	Size vs. complexity	191
10.1.3	Taxonomy	193
10.2	Finite (complex) Automata	195
10.2.1	Recognizing automata	195
10.2.2	Control automata	201
10.3	Simple Automata	202
10.3.1	Counters	202
10.3.2	<i>Program Counter</i>	205
10.3.3	<i>Registers with Arithmetic & Logic Unit (RALU)</i>	206
10.4	Problems	211
10.4.1	Finite Complex Automata	211
10.4.2	Simple Functional Automata	213

In this chapter we will close a second feedback loop in digital systems. This loop will increase the autonomy of the system. If in 0-OS, because we had no loop, the output of the combinational systems is a combination that derives strictly from the current value applied to the input, then in 1-OS a partial autonomy was obtained: the autonomy of the state that could be maintained outside the range of during which the signal that determined it acted. The second loop, which defines 2-OS, will allow the autonomy of the behavior of the system in which it closes, that is, we will obtain the possibility of an evolution of the output even in the absence of a variation of the input signal.

Our final target in this lesson is RALU: **Registers with Arithmetic & Logic Unit**.

10.1 Definitions

The typical circuit in 2-OS is the automaton. We will highlight two categories of automata: small & complex finite automata and large & simple functional automata.

10.1.1 Generic Definition

We will start with a generic definition because it applies to all kinds of automata introduced in this lecture.

Definition 10.1 *An automaton, A , is defined by the following 5-uple:*

$$A = (X, Y, Q, f, g)$$

where:

X : the finite set of input variables

Y : the finite set of output variables

Q : the set of state variables

f : the state transition function, described by $f : X \times Q \rightarrow Q$

g : the output transition function, with one of the following definitions:

- $g : X \times Q \rightarrow Y$ for the Mealy type automaton
- $g : Q \rightarrow Y$ for the Moore type automaton
- $g(q) = q$ for $Y \equiv Q$, where $q \in Q$ for the half-automaton, symbolized with $A_{1/2}$.

At each clock cycle the state of the automaton switches and the output takes the value according to the new state (and the current input, in Mealy's approach).

◇

A strict initial automaton is defined by:

$$A = (X, Y, Q, f, g; q_0)$$

and has a special input, called **reset**, used to led the automaton in the initial state q_0 . If the automaton is initial, the input reset switches the automaton in one, specially selected, initial state.

10.1.2 Size vs. complexity

The huge *size* of the actual circuits implemented on a single chip imposes a more precise distinction between *simple* circuits and *complex* circuits. When we can integrate on a single chip more than 10^9 components, the size of the circuits becomes less important than their complexity. Unfortunately we don't make a clear distinction between *size* and *complexity*. We say usually: "the complexity of a computation is given by the size of memory and by the CPU time". But, if we have to design a circuit of 100 million transistors it is very important to distinguish between a circuit having an uniform structure and a randomly structured ones. In the first case the circuit can be easily specified, easily described in an HDL, easily tested and so on. Otherwise, if the structure is completely random, without any repetitive substructure inside, it can be described using only a description having a similar dimension with the circuit size. When the circuit is small, it is not a problem, but for million of components the problem has no solution. Therefore, if the circuit is very big, it is not enough to deal only with its size, the most important becomes also the degree of uniformity of the circuit. This degree of uniformity, the degree of order inside the circuit can be specified by its **complexity**.

As a consequence we must distinguish more carefully the concept of *size* by the concept of *complexity*. Follow the definitions of these terms with the meanings we will use in this book.

Definition 10.2 The **size** of a digital circuit, $S_{\text{digital circuit}}$, is given by the dimension of the physical resources used to implement it.

◇

In order to provide a numerical expression for size we need a more detailed definition which takes into account technological aspects. In the '40s we counted electronic bulbs, in the '50s we counted transistors, in the '60s we counted SSI¹ and MSI² packages. In the '70s we started to use two measures: sometimes the number of transistors or the number of 2-input gates on the Silicon die and other times the Silicon die area. Thus, we propose two numerical measures for the size.

Definition 10.3 The **gate size** of a digital circuit, $GS_{\text{digital circuit}}$, is given by the total number of CMOS pairs of transistors used for building the gates used to implement it³.

◇

This definition of size offers an almost accurate image about the silicon area used to implement the circuit, but the effects of lay-out, of fan-out and of speed are not caught by this definition.

Definition 10.4 The **area size** of a digital circuit, $AS_{\text{digital circuit}}$, is given by the dimension of the area on silicon used to implement it.

◇

The area size is useful to compute the price of the implementation because when a circuit is produced we pay for the number of wafers. If the circuit has a big area, the number of the circuits per wafer is small and the yield is low⁴.

¹Small Size Integrated circuits

²Medium Size Integrated circuits

³Sometimes gate size is expressed in the total number of 2-input gates necessary to implement the circuit. We prefer to count CMOS pairs of transistors (almost identical with the number of inputs) instead of equivalent 2-input gates because is simplest. Anyway, both ways are partially inaccurate because, for various reasons, the transistors used in implementing a gate have different areas.

⁴The same number of errors make useless a bigger area of the wafer containing large circuits.

Definition 10.5 The **algorithmic complexity** of a digital circuit, simply the **complexity**, $C_{\text{digital circuit}}$, has the magnitude order given by the minimal number of symbols needed to express its definition.

◇

Definition 10.5 is inspired by (Chaitin, 1977)’s definition for the algorithmic complexity of a string of symbols. The *algorithmic complexity* of a string is related to the dimension of the smallest program that generates it. Our $C_{\text{digital circuit}}$ can be associated to the shortest unambiguous circuit description in a certain HDL (in the most of cases it is about a behavioral description).

Definition 10.6 A **simple circuit** is a circuit having the complexity much smaller than its size:

$$C_{\text{simple circuit}} \ll S_{\text{simple circuit}}.$$

Usually the complexity of a simple circuit is constant: $C_{\text{simple circuit}} \in O(1)$.

◇

Definition 10.7 A **complex circuit** is a circuit having the complexity in the same magnitude order with its size:

$$C_{\text{complex circuit}} \sim S_{\text{complex circuit}}$$

◇

Example 10.1 The following Verilog program describes a complex circuit, because the size of its definition (the program) is

$$S_{\text{def. of random_circ}} = k_1 + k_2 \times S_{\text{random_circ}} \in O(S_{\text{random_circ}}).$$

```

1  module A10_randomCirc(output    f, g,
2                        input     a, b, c, d, e);
3      wire      w1, w2;
4      and and1(w1, a, b),
5          and2(w2, c, d);
6      or  or1(f, w1, c),
7          or2(g, e, w2);
8  endmodule

```

Listing 10.1: Example of a complex circuit. File: A10_randomCircuit.sv (SystemVerilog)

◇

Example 10.2 The following Verilog program describes a simple circuit, because the program that define completely the circuit is the same for any value of n .

```

1  module orPrefixes #(parameter n = 256)(output logic [0:n-1] out,
2                                           input  logic [0:n-1] in);
3      integer k;
4      always_comb begin out[0] = in[0];
5                          for (k=1; k<n; k=k+1) out[k] = in[k] | out[k-1];

```

```

6           end
7   endmodule

```

Listing 10.2: Example of a simple circuit. File: A10_orPrefixes.sv (SystemVerilog)

The prefixes of OR circuit consists in n OR_2 gates connected in a very regular form. The definition is the same for any value of n .⁵

◇

SystemVerilogSummary 3 :

and, or, xor, not : are reserved names for predefined circuits which can be instantiated under specific names

parameter : used to define a local parameter

integer k : defined a positive integer

for : defined the well known loop running under the index k

Composing circuits generate not only biggest structures, but also *deepest* ones. The depth of the circuit is related with the associated propagation time.

Definition 10.8 The depth of a combinational circuit is equal with the total number of serially connected constant input gates (usually 2-input gates) on the longest path from inputs to the outputs of the circuit.

◇

At the current technological level the size becomes less important than the complexity, because we can *produce* circuits having an increasing number of components, but we can *describe* only circuits having the range of complexity limited by our mental capacity to deal efficiently with complex representations. The first step to have a circuit is to express what it must do in a behavioral description written in a certain HDL. If this "definition" is too large, having the magnitude order of a huge multi-billion-transistor circuit, we don't have the possibility to write the program expressing our desire.

In the domain of circuit design we passed long ago beyond the stage of *minimizing* the number of gates in a few gates circuit. Now, the most important thing, in the multi-billion-transistor circuit era, is the *ability to describe* simple (because we can't write huge programs), big (because we can produce more circuits on the same area) sized circuits. We must take into consideration that the Moore's Law applies to size not to complexity.

10.1.3 Taxonomy

Starting from the generic definition, in Figure 10.1 are represented the generic structures of the automata used in digital design.

The structures found in current practice are:

⁵A short discussion occurs when the dimension of the input is specified. To be extremely rigorous, the parameter n is expressed using a string of symbols in $O(\log n)$. But usually this aspect can be ignored.

half automaton : is an automaton whose output is identical to the internal state (the combinational circuit that transforms the state into the output is missing); the latency of the Y output compared to the X input is one clock cycle

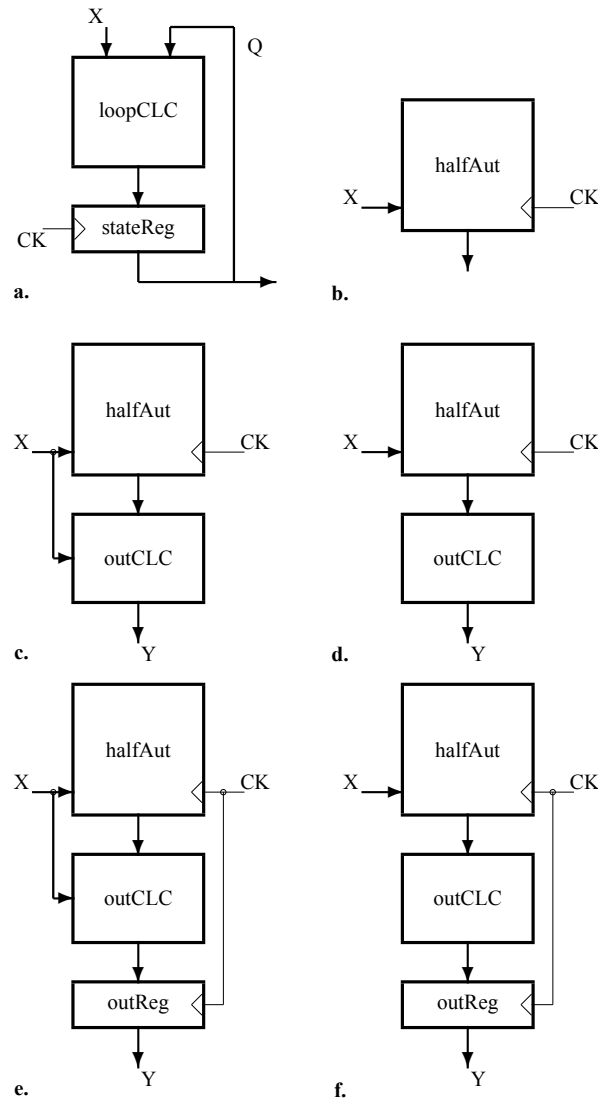


Figure 10.1: **Automata types.** **a.** The structure of the half-automaton ($A_{1/2}$), the no-output function automaton: the state is generated by the previous state and the previous input. **b.** The logic symbol of half-automaton. **c.** Immediate Mealy automaton: the output is generated by the current state and the current input. **d.** Immediate Moore automaton: the output is generated by the current state. **e.** Delayed Mealy automaton: the output is generated by the previous state and the previous input. **f.** Delayed Moore automaton: the output is generated by the previous state.

Mealy immediate automaton : is an automaton whose output is calculated by a combinational circuit depending on state and input; the latency of the Y output compared to the X input is zero

Moore immediate automaton : is an automaton whose output is calculated by a combinational circuit depending only by the state; the latency of the Y output compared to the X input is one clock cycle

Mealy delayed automaton : is a Mealy immediate automaton with the output delayed through a pipeline register; the latency of the Y output compared to the X input is one clock cycle

Moore delayed automaton : is a Moore immediate automaton with the output delayed through a pipeline register; the latency of the Y output compared to the X input is two clock cycles.

Depending on the way the machine is integrated into the system we are designing, we will choose the most suitable version. Latency can sometimes be advantageous.

We will make another important distinction: that between complex automata and simple ones.

Definition 10.9 *A complex automaton has a description proportional with the number of state it has.*

◇

Definition 10.10 *A simple automaton has a description of constant size, independent of the number of its states.*

◇

10.2 Finite (complex) Automata

Definition 10.11 *A finite automaton is characterized by $|Q| \in O(1)$, i.e., the size of the set Q is constant and independent on the length of the string of symbols it receives.*

◇

We will exemplify with two typical cases: a recognizing automaton and a control automaton.

10.2.1 Recognizing automata

Example 10.3 *The binary strings $1^n 0^m$, for $n \geq 1$ and $m \geq 1$, are recognized by a finite automaton. Let's define and design it. The transition diagram defining the behavior of the automaton is presented in Figure 10.2, where the state set Q contains:*

- q_0 - is the initial state in which:
 - ◇ if 1 is received, the output is `rec1` and the automaton switches in the state q_1
 - ◇ if 0 is received the automaton generate on output `fail` and switches in q_3
- q_1 - in this state at least one 1 was received and
 - ◇ if 1 is received, the output is `rec1` and the state remains the same
 - ◇ if 0 is received, the output is `rec0` the the state becomes q_2
- q_2 - this state acknowledges a well formed string:
 - ◇ if 1 is received, the output is `fail` and the state switches in q_3
 - ◇ if 0 is received, the output is `rec0` and the state remains the same

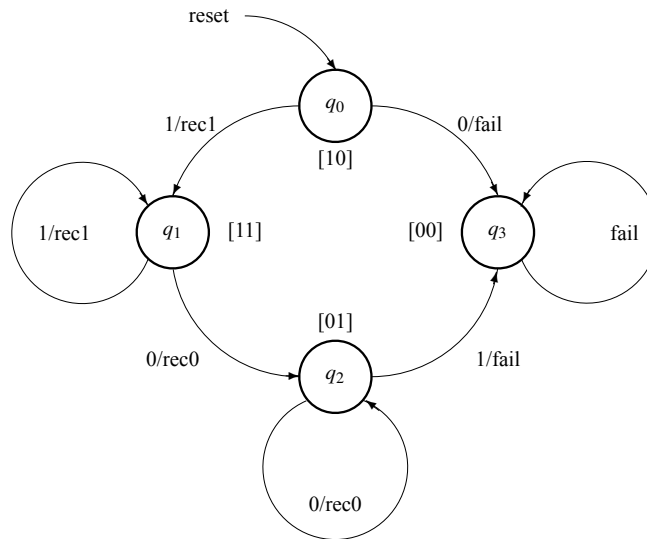


Figure 10.2: **Transition diagram.** The transition diagram for the half-automaton which recognizes strings of form $1^n 0^m$, for $n \geq 1$ and $m \geq 1$. Each circle represent a state, each (marked) arrow represent a (conditioned) transition.

- q_3 - is the error state because an incorrect string was; the state remains unconditionally the same until a new **reset** is applied. received.

The first step in implementing the structure of the just defined automaton is to **assign binary codes** to each state. In this stage we have the absolute freedom. Any assignment can be used. The only difference will be in the resulting structure but not in the resulting behavior.

Table 10.1: Transition state table.

Q_1	Q_0	X	Q_1^+	Q_0^+	Y_1	Y_0
0	0	0	0	0	1	1
0	0	1	0	0	1	1
0	1	0	0	1	0	1
0	1	1	0	0	1	1
1	0	0	0	0	1	1
1	0	1	1	1	1	0
1	1	0	0	1	0	1
1	1	1	1	1	1	0

Let be the codes, symbolized by Q_1 and Q_2 , assigned in square brackets in Figure 10.2. For the outputs, the symbols Y_1 and Y_2 are used with the following meanings:

rec0: 01

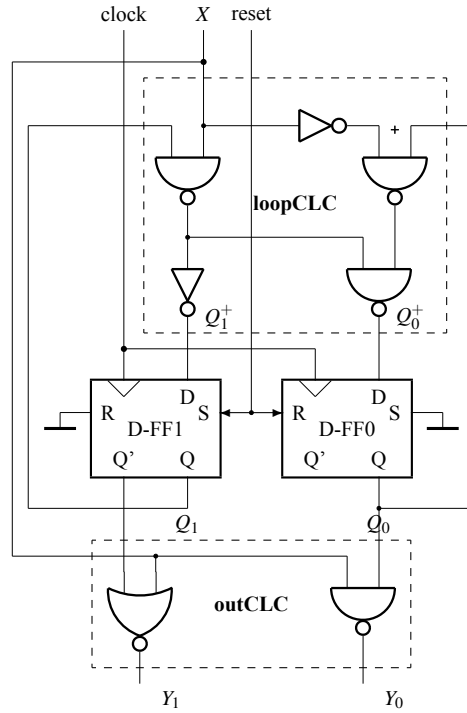


Figure 10.3: **A 4-state finite automaton.** The structure of the finite automaton used to recognize binary string belonging to the 1^n0^m set of strings.

rec1: 10
fail: 11

The input signal is symbolized by X .

Results the transition table, for the functions f and g , presented in Table 10.1. The resulting transition functions the resulting immediate Mealy automaton are:

$$\begin{aligned} Q_1^+ &= Q_1 \cdot X = ((Q_1 \cdot X)')' \\ Q_0^+ &= Q_1 \cdot X + Q_0 \cdot X' = ((Q_1 \cdot X)' \cdot (Q_0 \cdot X'))' \\ Y_1 &= (Q_0 \cdot X)' \\ Y_0 &= Q_1 \cdot X' = (Q_1' + X)' \end{aligned}$$

The circuit is represented in Figure 10.3 in a version using inverted gated only. The 2-bit state register is designed by 2 D flip-flops. The **reset** input is applied on the set input of D-FF1 and on the reset input of D-FF0.

The recognition process begins with the application of the **reset** signal through which the automaton goes to the initial state, q_0 , coded by $Q_1, Q_0 = 10$. A correct string must start with 1, otherwise the automaton goes to the final state, q_3 , which signals **fail**. In state q_1 , the automaton receives 1s, until the first 0 is applied to the input and the automaton passes to state q_2 in which it signals, through **rec0**, that

the string received is correct. If in state q_2 it is received in 1, then the string is cataloged as not belonging to those recognized by blocking the automaton in state q_3 .

◇

Designing a finite automaton using a description in System Verilog HDL involves writing the following files:

- `defines.vh` in which the binary values that the state, input and output variables take are defined
- `topAutomaton.sv` in which the module that describes the automaton defining the state register, instantiating the combinational calculation module of the loop, `loopCLC`, and the module `outCLC` that describes the output circuit is defined
- `loopCLC.sv` which describes the state transition function
- `outCLC.sv` which describes the output transition function.

Example 10.4 Let's revisit the previous example, recognizing the language $L = (a^n b^m | n, m > 0)$, and solve the design using a System Verilog description.

```

1  // input codes
2      `define a 1'b1
3      `define b 1'b0
4  // output codes
5      `define recA    2'b10 // automaton receives a
6      `define recB    2'b01 // automaton receives b
7      `define fail    2'b00 // automaton failed to receive correct string
8  // internal states
9      `define init     2'b10 // initial state
10     `define runA     2'b11 // automaton received at least one b
11     `define runB     2'b01 // automaton received at least one a
12     `define eror     2'b00 // fail state

```

Listing 10.3: Defines binary codes for input, output and state variables. File: A10_defines.vh (SystemVerilog)

```

1  `include "defines.vh"
2  module regLang( input  logic    in      ,
3                  output logic [1:0] out  ,
4                  input  logic    reset   ,
5                  clock      clock  );
6      logic [1:0] state, nextState ;
7
8      always_ff @(posedge clock) begin: state_register
9          if (reset) state <= `init ;
10         else      state <= nextState;
11     end: state_register
12     loopCLC lc( in      ,

```

```

13         state      ,
14         nextState   );
15     outCLC oC( state ,
16               in    ,
17               out    );
18 endmodule

```

Listing 10.4: Structural description of the immediate Mealy automaton designed to recognize the regular language $L = (a^n b^m | n, m > 0)$. File: A10_regLang.sv (SystemVerilog)

```

1  module loopCLC( input  logic      in      ,
2                  input  logic [1:0] state  ,
3                  output logic [1:0] nextState );
4
5      always_comb begin: loop_clc
6          case(state)
7              `init    : nextState = (in == `a) ? `runA : `error ;
8              `runA    : nextState = (in == `a) ? `runA : `runB ;
9              `runB    : nextState = (in == `b) ? `runB : `error ;
10             `error   : nextState = `error ;
11          endcase
12      end: loop_clc
13 endmodule

```

Listing 10.5: Combinational circuit used to compute the loop for the immediate Mealy automaton used to recognize the language $L = (a^n b^m | n, m > 0)$. File: A10_loopCLC.sv (SystemVerilog)

```

1  module outCLC( input      [1:0] state ,
2                 input      in      ,
3                 output bit [1:0] out   );
4
5      always_comb begin: out_clc
6          case(state)
7              `init    : out = (in == `a) ? `recA : `fail ;
8              `runA    : out = (in == `a) ? `recA : `recB ;
9              `runB    : out = (in == `a) ? `fail : `recB ;
10             `error   : out = `fail ;
11          endcase
12      end: out_clc
13 endmodule

```

Listing 10.6: Combinational circuit used to compute the output for immediate Mealy automaton which recognizes the language $L = (a^n b^m | n, m > 0)$. File: A10_outCLC.sv (SystemVerilog)


```

1  module testRegLang;
2      logic      in      ;
3      logic [1:0] out    ;
4      logic      reset   ;
5      logic      clock   ;
6
7      initial begin          clock = 0      ;
8          forever #1 clock = ~clock ;
9      end
10
11     initial begin          reset   = 1 ;
12         in      = `a;
13         #2 reset = 0 ;
14         #10 in   = `b;
15         #6 in    = `a;
16         #8 $finish ;
17     end
18
19     regLang dut(in      ,
20                 out     ,
21                 reset   ,
22                 clock   );
23
24     initial begin
25         $monitor("t=%0d  clock=%0d  reset=%0d  in=%0d  out=%0d",
26                 $time, clock, reset, in, out);
27     end
28
29     initial begin
30         $dumpfile("dump.vcd");

```

Listing 10.7: Test module for the immediate Mealy automaton which recognizes the language $L = (a^n b^m | n, m > 0)$. File: A10_testRegLang.sv (SystemVerilog)

◇

SystemVerilogSummary 4 :

‘define : used to define global variable

‘include "fileName.vh" : used to include code already defined

‘name : used to assert a name already defined using `~define`.

The main point: for any values for m and n , the number of states of the finite automaton is constant, equal with 4. But the description of the of the automaton’s behavior is proportional with the number of states (see the number of lines of the truth table defining the state transition) and independent by values m and n .

10.2.2 Control automata

Another use of a finite automaton is to control. Controlling means to send a sequence of commands to a digital systems and to determine the evolution of this sequence according to signals receiving back from the controlled system. Thus, the evolution in the state space depends: (1) on the internal loop of the automat and (2) on the flags received form the controlled system.

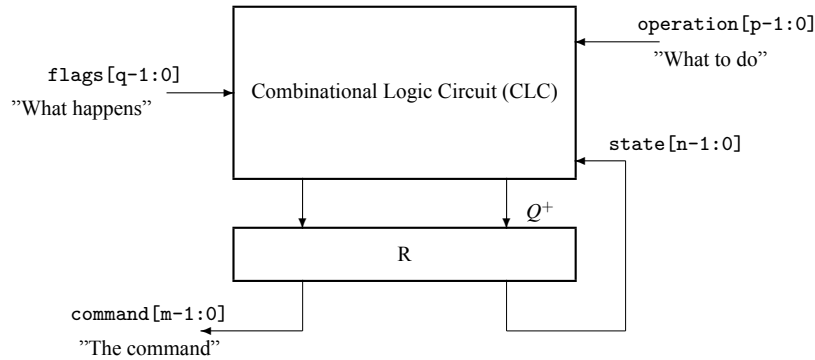


Figure 10.4: **Control Automaton.** The functional definition of control automaton. Control means to issue commands and to receive back signals (flags) characterizing the effect of the command.

In Figure 10.4, a Mealy delayed version is presented. It works as follows:

- the input $operation[p-1:0]$ is a binary code used to initialize the automaton in 2^p different initial states, each associated to a control sequence of commands to be issued to the controlled system
- the inputs $flags[q-1:0]$ represent independent bits used to characterise the behavior of the controlled system (for example if an arithmetic operation provided carry)
- the output $command[m-1:0]$ represent the command issued in each clock cycle toward the control system; it can be organized in one ore few fields to control different part of the system
- $state[n-1:0]$ represent the inner loop of the control automaton

The size of the CLC of this generic version is, in most of cases, to big to be optimally implementable. Indeed, because in the general case a CLC with N inputs is proportional with 2^N , our CLC could have a size proportional with 2^{p+q+n} .

But, to our luck, real applications have certain characteristics that allow a drastic simplification of the generic solution. We present it in the Figure 10.5.

We arrived at the structure in Figure 10.5 starting from the following observations:

- 1) the entry operation is taken into account in the calculation of the transition only in certain states, those of initializing a new command sequence
- 2) the flags entries are not all significant in every state
- 3) the command sequences contain subsequences that are chained unconditionally, a fact that

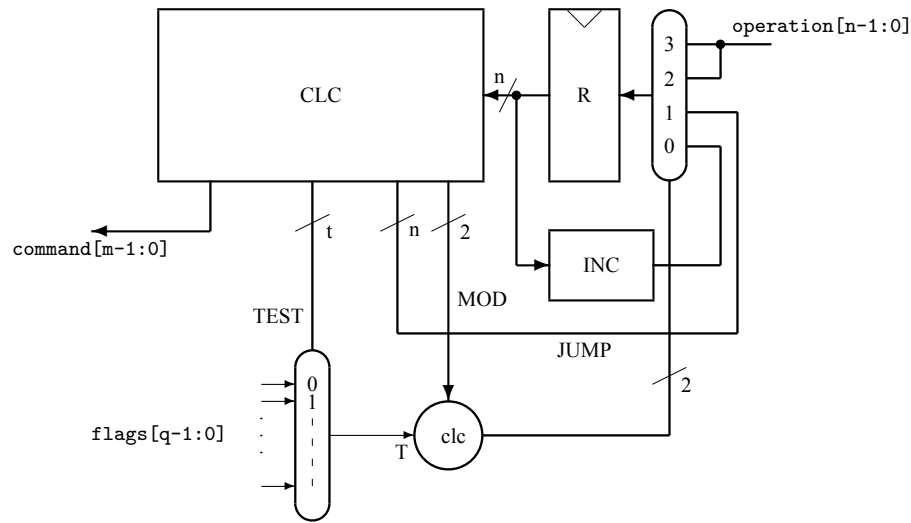


Figure 10.5: **The simplest Controller.** The Moore version of a control automaton is optimized by: (1) using a multiplexor to select, using MOD and T combined by *clc*, the next state from different sources, (2) using an increment circuit (INC) to compute the most frequent next address for CLC, and (3) using a multiplexor to select, using TEST, for each cycle the appropriate flag.

allows their encoding by incrementing

10.3 Simple Automata

A simple automaton has the loop closed through a simple combinatorial circuit. We will provide two examples on our way to build a computing engine.

10.3.1 Counters

10.3.1.1 T Flip-Flop

The smallest and the simplest automaton is a half-automaton with 2 states and one input (see Figure 10.6a). What could be the behavior of an automaton with only two internal states (Q and Q') and a single input bit T ? The only solution:

$$Q \leftarrow T ? Q' : Q$$

Let us remember on of the XOR's function: commanded inverter. Then the actual structure of the smallest and simplest automaton is represented in Figure 10.6b.

The numerical function of the TF-F is 2 modulo counter under the command T . If $T=1$ then the circuit counts, else it preserves the last state.

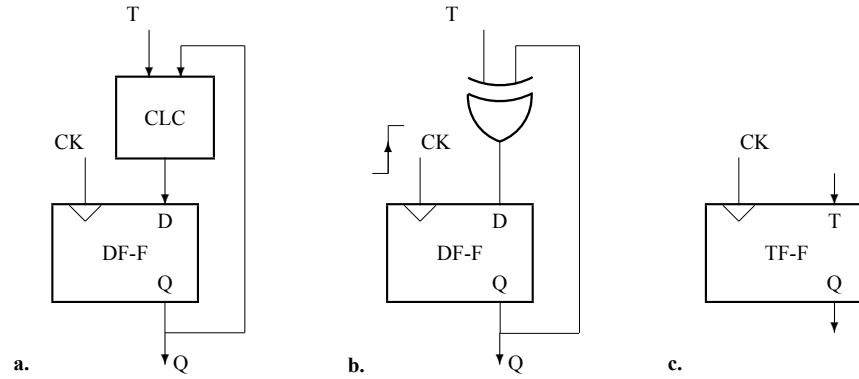


Figure 10.6: **The T flip-flop.** **a.** It is the simplest automaton because: has 1-bit state register (a DF-F), a 2-input loop circuit (one as automaton input and another to close the loop), and direct output from the state register. **b.** The structure of the T flip-flop: the XOR_2 circuits complements the state if $T = 1$. **c.** The logic symbol.

10.3.1.2 Generic Counter

The counter is basically a simple half-automaton that has an increment circuit on the reverse connection. Each active edge of the clock loads the incremented state register value into the state register. In real application, the functionality is extended to the following set of operations:

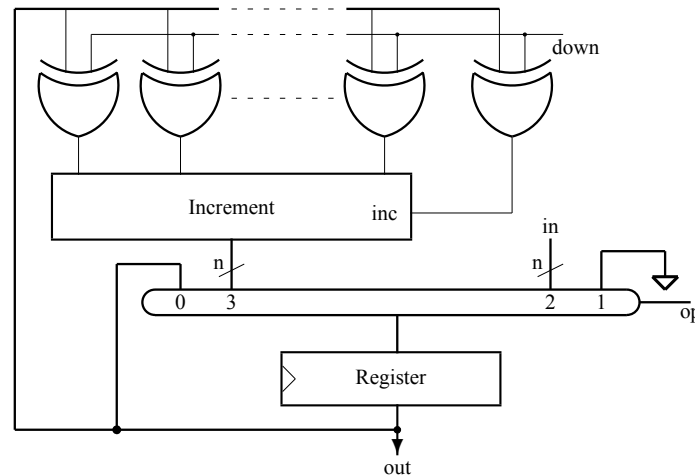


Figure 10.7: Counter.

- $op = 00 = nop: out \leq out$
- $op = 01 = reset: out \leq 0$
- $op = 10 = load: out \leq in$
- $op = 11 = count: out \leq down ? out - 1 : out + 1$

- down: 2's complement of out to the increment circuit's input

The structure of the generic counter is represented in Figure 10.7. If the size of the register is of n bits, then the circuit counts modulo 2^n .

Example 10.5 *Four-bit presetable and reversible counter.*

```

1      `define nop      3'b000
2      `define reset    3'b001
3      `define load     3'b010
4      `define up       3'b011
5      `define down     3'b111

```

Listing 10.8: Defines the command for the counter circuit. File: A10_counterDefines.vh (SystemVerilog)

```

1  `include "counterDefines.vh"
2  module counter( output logic [3:0] out ,
3                  input  logic [2:0] op  ,
4                  input  logic [3:0] in  ,
5                  input  logic          clk );
6
7      always_ff @(posedge clk) case(op)
8          `reset    : out <= 0          ;
9          `load     : out <= in         ;
10         `up       : out <= out + 1;
11         `down     : out <= out - 1;
12         default   : out <= out       ;
13     endcase
14 endmodule

```

Listing 10.9: A presetable up-down counter. File: A10_counter.sv (SystemVerilog)

```

1  `include "counterDefines.vh"
2  module testCounter;
3      logic [3:0] out ;
4      logic [2:0] op  ;
5      logic [3:0] in  ;
6      logic          clk ;
7
8      initial begin          clk = 0          ;
9          forever #1 clk = ~clk ;
10         end
11
12     initial begin          op = `reset ;
13         in = 4'b1010;
14         #2 op = `up      ;

```

```

15             #6  op = `load  ;
16             #2  op = `up    ;
17             #10 op = `down  ;
18             #16 $finish    ;
19         end
20
21     counter dut(out ,
22                op  ,
23                in  ,
24                clk );
25 endmodule

```

Listing 10.10: Test module for the counter. File: A10_testCounter.sv (SystemVerilog)

◇

10.3.2 Program Counter

A specific application of the counter simple automaton is the system used to compute the next address in the process of running a program (see Figure 10.8). The set of functions, specified on the input next, are:

- `nop`: `address <= address for halt instruction`
- `rst`: `address <= 0` for system reset
- `inc`: `address <= address + 1` for program counter increment
- `rjmp`: `address <= address + relValue` for relative jump
- `crjmp`: `address <= (absValue = 0) ? address + relValue : address + 1` for conditioned branch
- `ajmp`: `address <= absValue` for absolute jump

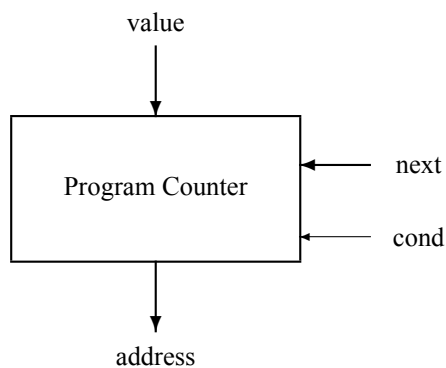


Figure 10.8: Program Counter.

```

1  module programCounter #(parameter n=32)
2      (   output logic [31:0]   progAddr,
3          input  logic [2:0]    sel      ,
4          input  logic [31:0]   relValue,
5          input  logic [31:0]   absValue,
6          input  logic          reset   ,
7          input  logic          clock   );
8      logic [31:0] pc      ;
9      logic [31:0] nextPC  ;
10
11     always_ff @(posedge clock) if (reset) pc <= 0      ;
12                                     else pc <= nextPC;
13
14     always_comb
15     case(sel)
16         3'b000: nextPC = pc                                ;
17         3'b001: nextPC = pc + relValue                      ;
18         3'b010: nextPC = (absValue == 0) ? pc + relValue : pc + 1 ;
19         3'b011: nextPC = (absValue == 0) ? pc + 1 : pc + relValue ;
20         3'b100: nextPC = absValue                          ;
21         default: nextPC = pc + 1                          ;
22     endcase
23
24     assign progAddr = pc      ; // for pipelined version
25     //assign progAddr = nextPC ; // for non-pipelined version
26 endmodule

```

Listing 10.11: Program counter. File: A10_programCounter.sv (SystemVerilog)

10.3.3 Registers with Arithmetic & Logic Unit (RALU)

If we connect in a loop a register file with a logical-arithmetic unit we will obtain the simplest form of an executive core of a processor (see Figure 10.9). The external connections of a Register with Arithmetic & Logic Unit (RALU) type module are:

`func[3:0]` : selects one of the maximum 8 functions performed by ALU

`carryIn` : carry input for arithmetic functions (is borrow for subtract)

`carryOut` : carry output for arithmetic function (is borrow for subtract)

`in[31:0]` : data input

`load` : selects data input as left operand for ALU

`leftAddr[4:0]` : selects left output or left operand for ALU when `load = 0`

`rightAddr[4:0]` : selects right output or right operand for ALU

`clock` : is the clock signal active on the positive edge

`we` : write enable for the register file

`destAddr[4:0]` : destination address for the value out provided by ALU

`leftOut[31:0]` : left output of RALU

`rightOut[31:0]` : right output of RALU

According to the connection list, RALU performs no more than 8 functions, on 32-bit words stored in a 32-word register file.

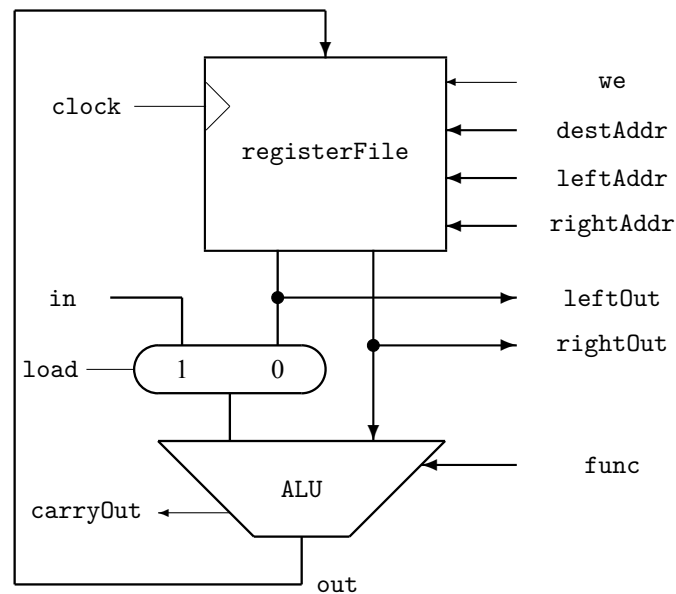


Figure 10.9: 32-bit RALU.

In each clock cycle, the function `func` is applied on two operands selected from the register file and the result, `out`, is loaded back in the register file if `writeEnable = 1`. If `load = 1`, then `leftOp = in`. If `writeEnable = 0`, the content of the register file remains unchanged and the only purpose of the operation is to send toward the external systems the outputs: `carryOut`, `leftOut`, `rightOut`.

The code `func` selects the functions defined in the file `define.vh`:

```

1  `define bigv 4'b0001 // rf[dest] = {rf[left][15:0], rf[right][15:0]}
2  `define add 4'b0010 // rf[dest] = rf[left] + rf[right]
3  `define sub 4'b0011 // rf[dest] = rf[left] - rf[right]
4  `define addcr 4'b0100 // rf[dest] = (rf[left] + rf[right])[32]
5  `define subcr 4'b0101 // rf[dest] = (rf[left] - rf[right])[32]
6  `define lsh 4'b1000 // rf[dest] = rf[left] >> 1
7  `define ash 4'b1001 // rf[dest] = {rf[left][31], rf[left][31:1]}
8  `define move 4'b1010 // rf[dest] = rf[left]
9  `define mult 4'b1011 // rf[dest] = rf[left] * rf[right]
10 `define bwand 4'b1101 // rf[dest] = rf[left] & rf[right]

```



```

11 `define bwor 4'b1110 // rf[dest] = rf[left] | rf[right]
12 `define bwxor 4'b1111 // rf[dest] = rf[left] ^ rf[right]

```

Listing 10.12: Defining the functions for a RALU. File: A10_defineRALU.vh (SystemVerilog)

```

1  module RALU(output logic [31:0] leftOut ,
2             output logic [31:0] righttOut ,
3             output logic crOut ,
4             input logic [4:0] leftAddr ,
5             input logic [4:0] rightAddr ,
6             input logic [4:0] destAddr ,
7             input logic [31:0] in ,
8             input logic load ,
9             input logic we ,
10            input logic [3:0] func ,
11            input logic clock );
12      logic [31:0] out ;
13
14      regFile rf( .leftOut (leftOut ),
15                .righttOut (righttOut ),
16                .in (out ),
17                .leftAddr (leftAddr ),
18                .rightAddr (rightAddr ),
19                .destAddr (destAddr ),
20                .we (we ),
21                .clock (clock ));
22
23      ALU alu(.func (func ),
24             .left (load ? in : leftOut),
25             .right (righttOut ),
26             .crOut (crOut ),
27             .out (out ));
28  endmodule

```

Listing 10.13: RALU. File: A10_RALU.sv (SystemVerilog)

For module regFile see 9.4.4 in the previous chapter.

```

1  `include "define.vh"
2  module ALU( input logic [3:0] func ,
3             input logic [31:0] left ,
4             input logic [31:0] right ,
5             output logic crOut ,
6             output logic [31:0] out );
7      logic [32:0] sum ;
8      logic [32:0] dif ;
9
10     assign sum = left + right ;

```

```

11     assign dif = left - right    ;
12     always_comb
13     case(func)
14         `bigv : {crOut, out} <= {1'b0, left[15:0], right[15:0]};
15         `add  : {crOut, out} <= sum      ;
16         `sub  : {crOut, out} <= dif      ;
17         `addcr : {crOut, out} <= {32'b0, sum[32]} ;
18         `subcr : {crOut, out} <= {32'b0, dif[32]} ;
19         `lsh  : {crOut, out} <= {left[0], left[31:1]} ;
20         `ash  : {crOut, out} <= {left[31], left[31:1]} ;
21         `move : {crOut, out} <= {1'b0, left} ;
22         `mult : {crOut, out} <= {1'b0, left * right} ;
23         `bwand : {crOut, out} <= {1'b0, left & right} ;
24         `bwor  : {crOut, out} <= {1'b0, left | right} ;
25         `bwxor : {crOut, out} <= {1'b0, left ^ right} ;
26         default {crOut, out} <= 33'b0 - 1'b1 ;
27     endcase
28 endmodule

```

Listing 10.14: ALU. File: A10_alu.sv (SystemVerilog)

Let us use the RALU unit we just defined to exemplify simple calculations.

Example 10.6 *The following sequence of operations: load two numbers (12 and 5) in register file, add them, divide the result by 4, add with 23, and send out the result on the right output.*

Table 10.2: Sequence of commands applied to RALU in Example 10.6

func	in	load	leftAddr	rightAddr	we	destAddr	COMMENT
1111	xx	0	00000	00000	1	00000	R0 <= 0
0010	12	1	xxxxx	00000	1	00001	R1 <= 12+R0
0010	5	1	xxxxx	00000	1	00010	R2 <= 5+R0
0010	xx	0	00001	00010	1	00011	R3 <= R1+R2
1001	xx	0	00011	xxxxx	1	00011	R3 <= R3/2
1001	xx	0	00011	xxxxx	1	00011	R3 <= R3/2
0010	23	1	xxxxx	00011	1	00011	R3 <= R3+23
xxxx	xx	x	xxxxx	00011	0	xxxxx	rightOut = R3

◇

A more flexible way to test the correctness of the design is to make a simulation.

```

1  `include "defineRALU.vh"
2  module testRALU;
3      logic [31:0]    leftOut    ;
4      logic [31:0]    righttOut  ;
5      logic           crOut      ;

```

```

6    logic [4:0]    leftAddr    ;
7    logic [4:0]    rightAddr   ;
8    logic [4:0]    destAddr    ;
9    logic [31:0]   in          ;
10   logic          load        ;
11   logic          we          ;
12   logic [3:0]    func        ;
13   logic          clock       ;
14   logic [51:0]   mInstr      ;
15
16   initial begin                clock = 0          ;
17                               forever #1 clock = ~clock ;
18   end
19
20   RALU dut( leftOut    ,
21             righttOut  ,
22             crOut      ,
23             leftAddr   ,
24             rightAddr  ,
25             destAddr   ,
26             in         ,
27             load       ,
28             we         ,
29             func       ,
30             clock      );
31
32   // Define MICRO-INSTRUCTION
33   assign {func, destAddr, leftAddr, rightAddr, load, in} = mInstr ;
34
35   initial begin
36       we = 1'b1 ;
37       mInstr = {`move, 5'b00000,5'b00110,5'b00111,1'b1,32'b1000} ;
38       #2 mInstr = {`move, 5'b00001,5'b00010,5'b00010,1'b1,32'b1001} ;
39       #2 mInstr = {`move, 5'b00010,5'b00110,5'b00111,1'b1,32'b1010} ;
40       #2 mInstr = {`move, 5'b00011,5'b00010,5'b00010,1'b1,32'b1011} ;
41       #2 mInstr = {`add, 5'b00011,5'b00010,5'b00001,1'b0,32'b111} ;
42       #2 mInstr = {`sub, 5'b00011,5'b00010,5'b00001,1'b0,32'b111} ;
43       #2 mInstr = {`bwand,5'b00011,5'b00010,5'b00001,1'b0,32'b111} ;
44       #2 $finish;
45   end
46
47   initial begin
48       $monitor("t=%0d clock=%b func=%b destAddr=%0d leftAddr=%0d
49               rightAddr=%0d load=%0d in=%0d RF=[%0d, %0d, %0d, %0d]",
50               $time, clock, func, destAddr, leftAddr, rightAddr, load
51               , in,
52               dut.rf.mem[0],
53               dut.rf.mem[1],
54               dut.rf.mem[2],
55               dut.rf.mem[3]);
56   end

```

```
56 endmodule
```

Listing 10.15: Testing RALU. File: A10_testRALU.sv (SystemVerilog)

The result of simulation is printed by the monitor defined in the previous test module:

```
t=0 clock=0 func=1010 destAddr=0 leftAddr=6 rightAddr=7 load=1 in=8 RF=[x, x, x, x]
t=1 clock=1 func=1010 destAddr=0 leftAddr=6 rightAddr=7 load=1 in=8 RF=[8, x, x, x]
t=2 clock=0 func=1010 destAddr=1 leftAddr=2 rightAddr=2 load=1 in=9 RF=[8, x, x, x]
t=3 clock=1 func=1010 destAddr=1 leftAddr=2 rightAddr=2 load=1 in=9 RF=[8, 9, x, x]
t=4 clock=0 func=1010 destAddr=2 leftAddr=6 rightAddr=7 load=1 in=10 RF=[8, 9, x, x]
t=5 clock=1 func=1010 destAddr=2 leftAddr=6 rightAddr=7 load=1 in=10 RF=[8, 9, 10, x]
t=6 clock=0 func=1010 destAddr=3 leftAddr=2 rightAddr=2 load=1 in=11 RF=[8, 9, 10, x]
t=7 clock=1 func=1010 destAddr=3 leftAddr=2 rightAddr=2 load=1 in=11 RF=[8, 9, 10, 11]
t=8 clock=0 func=0010 destAddr=3 leftAddr=2 rightAddr=1 load=0 in=7 RF=[8, 9, 10, 11]
t=9 clock=1 func=0010 destAddr=3 leftAddr=2 rightAddr=1 load=0 in=7 RF=[8, 9, 10, 19]
t=10 clock=0 func=0011 destAddr=3 leftAddr=2 rightAddr=1 load=0 in=7 RF=[8, 9, 10, 19]
t=11 clock=1 func=0011 destAddr=3 leftAddr=2 rightAddr=1 load=0 in=7 RF=[8, 9, 10, 1]
t=12 clock=0 func=1101 destAddr=3 leftAddr=2 rightAddr=1 load=0 in=7 RF=[8, 9, 10, 1]
t=13 clock=1 func=1101 destAddr=3 leftAddr=2 rightAddr=1 load=0 in=7 RF=[8, 9, 10, 8]
```

10.4 Problems

10.4.1 Finite Complex Automata

Problem 10.1 Design the finite automaton which recognise the binary strings $a^n b^2 c^m$, for $n \geq 1$ and $m \geq 1$. The input and the output signal are specified in the following file (where the state assignment must be added):

```
1 // input codes
2 `define a 2'b01
3 `define b 2'b00
4 `define c 2'b10
5 // output codes
6 `define recA 2'b10 // automaton receives a
7 `define recB 2'b01 // automaton receives b
8 `define recC 2'b11 // automaton receives c
9 `define fail 2'b00 // automaton failed to receive correct string
10 // internal states
11 // ...
```

Listing 10.16: Binary codes for input, output and state variables. File: A10_defines1.vh (SystemVerilog)

◇

Problem 10.2 Design the interrupt automaton which manage the interrupt process in a processor. The synchronous interaction of a processor with an external event, in its simplest form, is achieved by accepting an interrupt signal, `int`, to which it responds with `inta` ("interrupt acknowledgement"). Acceptance of the interruption is conditioned by the state `enableState` state in which the interrupt management automaton can be. At reset, the automaton goes into the state `disableState`. The `eint` instruction

puts the automaton in the `enableState` state, a state in which `inta` can be issued if `int` is received. After issuing `inta`, the automaton generates the signal `flush` which blocks the update of the processor state for a number of cycles, number that depends on the depth of the execution pipe (3, in the considered case). At the end of `flush` signal the automaton switches in the `disableState`. Instruction `dint` sends the automaton in the `disableState` state.

```

1  // input independent signals
2  `define eint    2'b01 // enable interrupt instruction
3  `define dint    2'b10 // disable interrupt instruction
4  `define int     1'b1  // external interrupt signal
5  // output codes
6  `define idle    2'b00 // idle state enabled or disabled
7  `define inta    2'b01 // interrupt acknowledge
8  `define flush   2'b10 // flush the pipe command
9  // internal states
10 `define enableState ...
11 `define disableState ...
12 // ...

```

Listing 10.17: Binary codes for input, output and state variables. File: A10_defines2.vh (SystemVerilog)

- 1) Draw the graph of the automaton whose external connections are partially defined as follows:
Drawing the graph will allow completing the previous definitions file.
- 2) Provide the System Verilog description of the automaton
- 3) Use the Vivado environment to draw the schematic of the resulting circuit
- 4) Simulate the automaton to test its behavior.

◇

Problem 10.3 Let be the automaton defined by the graph represented in Figure 10.10.

```

1  // input independent signals
2  `define A      1'b1
3  // output codes
4  `define X      2'b01
5  `define Y      2'b10
6  `define Z      2'b11
7  // internal states
8  // ...

```

Listing 10.18: Binary codes for input, output and state variables. File: A10_defines3.vh (SystemVerilog)

Implement the automaton in two versions assigning for the 5 states the binary codes in different versions. Compare the size of the two solutions.

◇

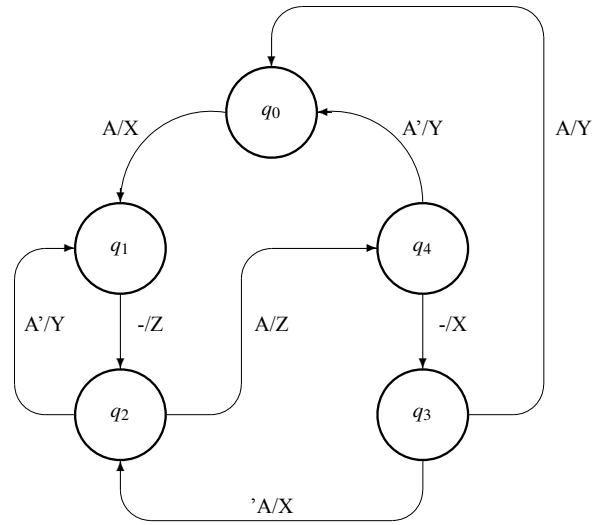


Figure 10.10:

10.4.2 Simple Functional Automata

Problem 10.4 Consider an 8-bit counter whose counting modes are selected by three bits as follows:

- 1) no operation
- 2) increment modulo 2^8 by one unit at each clock cycles
- 3) increment modulo 2^8 by 3 at each clock cycles
- 4) increment modulo 2^8 by 5 at each clock cycles
- 5) increment modulo 2^8 by 7 at each clock cycles
- 6) increment modulo 2^8 by 11 at each clock cycles
- 7) increment modulo 2^8 by 13 at each clock cycles
- 8) increment modulo 2^8 by 17 at each clock cycles

Required:

- 1) Draw the block diagram of the solution you propose using known circuits
- 2) Write the structural System Verilog description of the circuit
- 3) Simulate the design to prove the functionality

◇

Problem 10.5 Simulate the circuit described in Section 10.3.2.

◇

Problem 10.6 Consider the program counter described in Section 10.3.2.

1. Draw the logic diagram of the circuit described in the `programCounter.sv` module using a register, and studied combinational circuits.
2. If $t_{pReg} = \#30$, $t_{SU} = \#5$, $t_{selectionMUX} = \#15$, $t_{selectedMUX} = \#5$, $t_{inc} = \#100$, $t_{add} = \#110$, then what is the minimum period of the clock signal.

◇

Problem 10.7 Simulate the circuit described in Section 10.3.3 running the sequence outlined in Table 10.2.

◇

Problem 10.8 Upgrade the RALU described in Section 10.3.3 with ALU upgraded in Problem 8.18.

◇

Problem 10.9 Add to the RALU described in Section 10.3.3 the possibility that the right operand can come from a second selected input with a distinct associated signal. Instead of `in[31:0]` and `load`, the design will contain `inLeft[31:0]`, `loadLeft`, `inRight[31:0]` and `loadRight`.

◇

Problem 10.10 Consider the RALU described in Section 10.3.3, where: $t_{accessRegFile} = \#50$, $t_{rfSU} = \#5$, $t_{selectionMUX} = \#15$, $t_{selectedMUX} = \#5$, $t_{pALU} = \#150$. What is the minimum period of the clock signal.

◇

Problem 10.11 Consider the RALU described in Section 10.3.3 and fill-up the first 7 columns in Table 10.3 according to operations specified in the last column.

Table 10.3: Sequence of commands applied to RALU described in Section 10.3.3

func	in	load	leftAddr	rightAddr	we	destAddr	COMMENT
							R2 <= 5
							R5 <= 23
							R12 <= R2 + R3
							R14 <= R12
							R1 <= R14
							R2 <= R5 + R13
							R7 <= R2/2
							R8 <= R7 +12

◇

Chapter 11

Processors: Three-Loop, Third-Order Systems (3-OS)

11.1	Counter-Expanded Automata	216
11.2	Processor	218
11.2.1	Architecture vs. Organization	218
11.2.2	Interpreting Processor (CISC processor)	219
11.2.3	Executing Processor (RISC processor)	220
11.3	<i>ToyRISC Processor</i>	221
11.3.1	Organization	221
11.3.2	Instruction Set Architecture	223
11.3.3	Assembly Code	225
11.3.4	Time performance	232
11.4	How is Designed an Instruction Set Architecture	233
11.5	Problems	234

In this chapter we will close the third loop. It can be closed through a combinational circuit, through a memory circuit, or through an automaton. For our purpose, we are interested only on this last case, that of closing the third loop over an automaton through another automaton. We will focus, but not limited on the third loop that allows us to introduce the concept of the processor exemplified with a simple engine called *toyRISC*. We will practice the processor concept in the context provided by the fifth loop, the one that defines the computer concept in the currently practiced understanding, that of Reduced Instruction Set Computer (RISC) type computing systems. Only for those interested and familiar with System Verilog HDL, at the end of this text there is an appendix that allows the simulation of the system described in this lesson.

At the level of the third loop, we are at a *turning point* where the physical structure of the circuits will meet the informational structure of the programs. The functionality will be determined starting from this point by the circuit-information combination. The interface between structural and informational aspects is provided by the concept of *architecture* borrowed from the field of construction.

11.1 Counter-Expanded Automata

Let's remember how we solved the problem of recognizing strings of the form $1^n 0^m$ using a finite automaton. What should we do if we try to recognize strings of the form $1^n 0^n$, strings that have a higher complexity because we have an additional restriction: the number of 1s is the same as the number of 0s? A finite automaton, with a number of states independent of the value of n , cannot solve the problem. The solution is to add a reversible counter in the loop of a finite automaton. In Figure 11.1 a counter-expanded automaton is represented that returns to the automaton that controls it a flag indicating its zero state.

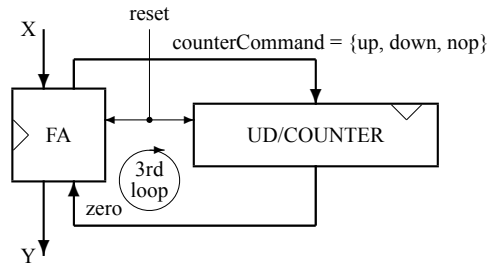


Figure 11.1: Counter-expanded automaton.

Because both FA and UD/COUNTER are 2-OS, connecting them in loop results in having a 3-OS.

Example 11.1 The (context-free) language $(a^n b^n | n > 0)$ is recognized by the counter-expanded automaton based on the following FA:

$$\begin{aligned}
 FA &= ((X \times flag), (Y \times com), Q, f, g) = \\
 &= ((\{a, b, e\} \times \{zero\}), (\{idle, run, yes, no\} \times \{nop, up, down\}), Q, f, g)
 \end{aligned}$$

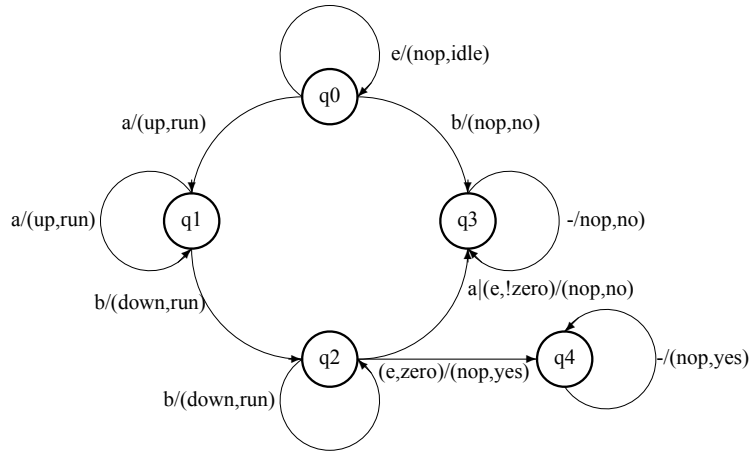


Figure 11.2: The graph describing the behavior of the finite automaton used to build the counter-expanded automaton for the $(a^n b^n | n > 0)$ language.

where e is a delimiter symbol (a well-formed string is, for example, of the form $\dots e e a a b b b e e \dots$), and the set of states, Q , and the functions f and g are given by the graph in Figure 11.2. Establishing the graph of the automaton allows us to also define the set of states: $Q = \{q_0, q_1, q_2, q_3, q_4\}$.

The top module for the System Verilog design is:

```

1  `define nop      2'b00
2  `define up       2'b01
3  `define down     2'b10
4  module CEA(input  logic [1:0] X          ,
5             output logic [1:0] Y          ,
6             output logic      reset, clock);
7
8      FA fa(  X          ,
9             Y          ,
10            conterCommand ,
11            zero         ,
12            reset        ,
13            clock         );
14     logic [31:0] UD/COUNTER ;
15     always_ff @(posedge clock)
16         if(reset) UD/COUNTER <= 0 ;
17         else if(up) UD/COUNTER <= UD/COUNTER + 1;
18         else if(down) UD/COUNTER <= UD/COUNTER - 1;
19         else UD/COUNTER <= UD/COUNTER ;
20 endmodule

```

Listing 11.1: Counter expanded automaton with a 32-bit counter for recognizing the language $a^n b^n$.
File: 0_CEA.vh (SystemVerilog)

while for the finite automaton FA the implementation is similar for the design in Example 10.4.

◇

11.2 Processor

11.2.1 Architecture vs. Organization

Architecture: describes what the computer does.

Organization: describes how it does it.

In the beginning it was only the hardware and the software. At some point there was a need for an interface that would provide the software with a more friendly development environment. What it is? If in a suite of hardware implementations the set of instructions changes radically with each new version, then the programs written for previous versions are thrown into the trash every time a new hardware version is instantiated. This bad habit is cured with the design, at the beginning of the 1960s, of the IBM System/360. At that moment it was decided that:

... to describe the attributes of a system as seen by the programmer, i.e., the conceptual structure and functional behavior; as distinct from the organization of the data flow and controls, the logical design, and the physical implementation. Amdahl et al. (1964)

And so the Hardware – Software duet turned into the Hardware – Architecture – Software triplet. Architecture being the short form for Instruction Set Architecture.

Table 11.1: Architecture vs. Organization.

Architecture	Organization
describes what the computer does	describes how it does it
deals with the functional behavior	deals with a structural relationship
architecture is fixed first	organization is decided after
comprises instruction sets, registers, data types, and addressing modes	consists of multiplexors, adders, multipliers, peripherals, memories, ...
the software developer is aware of it	it escapes the software programmer's detection

There are several ways to close the third loop over a generic 2-OS represented by an automaton in order to obtain the generic structure of a **processor**. On the closed loop over an automaton we can connect a 0-OS (a combinational circuit), a 1-OS (a memory), or a 2-OS (an automaton). The last version is the most significant (see Figure 11.3). It is the main subject of this chapter.

The two machines that configure a processor have distinct functions:

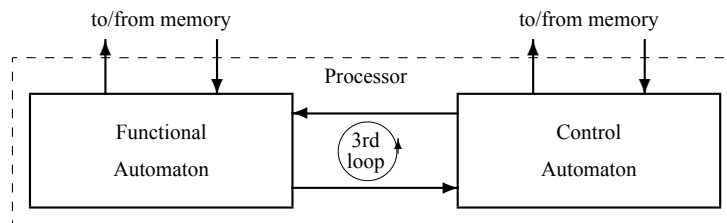


Figure 11.3: Generic Processor

- functional automaton that performs data processing functions; it is a simple automaton
- control automaton that ensures the sequential operation of the processor implemented in two possible versions:
 - ◊ a complex controller of the type presented in 10.2.2, when, in addition to fetching the instruction from the program memory, it is also necessary to break it down into a sequence of micro-instructions executed by the functional automaton
 - ◊ a simple controller of the counter type presented in 10.3.2, when it is sufficient to bring the instruction from the program memory because the instruction is simple enough to be executed directly by the functional automaton

The two types of control generate two types of processors, as follows in the next section.

11.2.2 Interpreting Processor (CISC processor)

The first type of processor, the one that interprets the instruction by decomposing it into a sequence of microinstructions, is represented in Figure 11.4, where the control is performed with a complex automaton. An instruction can involve complex operations implemented sequentially through a series of micro-instructions sent by the controller to the functional automaton that can send back to the controller flags that can characterize the result of the application of each micro-instruction. For this reason, this processor version is called CISC (Complex Instruction Set Computer).

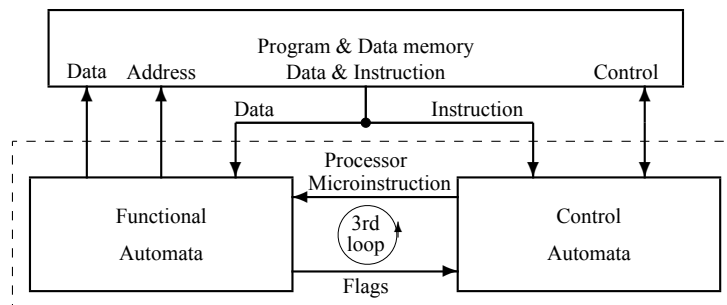


Figure 11.4: Interpreting Processor

For each instruction the CISC processor must perform:

- 1) instruction fetch
- 2) instruction execute
- 3) compute the address for the next instruction

so that an instruction is executed in a variable number of at least several clock cycles.

11.2.3 Executing Processor (RISC processor)

The second type of processor is designed in such a way that it executes each instruction in constant time, usually one clock cycle, but no more than two clock cycles. Because each instruction requires fetching it from memory, and some instructions also require writing or reading data from memory, it is necessary to structure the memory into two sub-memories, one for programs and another for data. This results in the structure in Figure 11.5, where the processor has two access paths to memory resources. In this case, the control automaton can only take care of fetching the instructions from the program memory and the functional automaton will execute sufficiently simple instructions to require only one clock cycle to be operated.

How did you get to this way of designing a processor that executes only simple, executable instructions in one clock cycle? Around 1980, the following facts were evident:

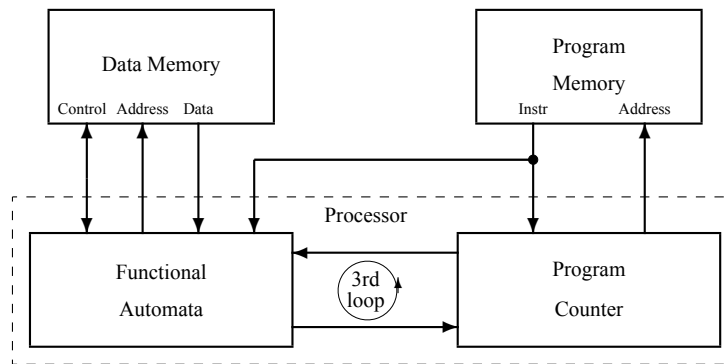


Figure 11.5: Executing Processor

- the frequency with which the instructions of a CISC processor appeared in the programs was very variable: a small number of instructions had a very high frequency of use, and the majority were used with low frequencies
- if the instructions were listed in descending order of frequency of use, then a line could be drawn in this list delimiting a set of frequently used instructions that also had the quality of being able to implement the instructions below the line through a suitable sequencing of them
- the pleasant surprise was that only simple functions were found above the line
- thus, for frequent operations, the effector structure could be imagined to work quickly, and for the less frequent ones, the possibility of their performance is ensured

Thus, a RISC processor executes instead of interpreting. Programs represent sequences of simple instructions, which are executed in parallel with the control of their evolution. Indeed, the functional automaton operates on the data while the control automaton calculates the address from which the next instruction will be read. Both types of operations fall into the category of simple ones, the fact that it allows us to consider a RISC processor as simple to distinguish it from a CISC one which is complex due to the control automaton.

11.3 ToyRISC Processor

The executing processor is simpler than an interpreting processor. The complexity of computation moves almost completely from the physical structure of the processor into the programs executed by the processor, because this kind of processor (called Reduced Instruction Set Computer, RISC) has an organization containing mainly simple, recursively defined circuits.

11.3.1 Organization

The Harvard abstract model of a RISC executing machine determines the internal structure of the processor to have mechanisms allowing in each clock cycle to address both, the program memory and the data memory. Thus, the RALU-type functional automaton, directly interfaced with the data memory, is loop-connected with a control automaton designed to fetch in each clock cycle a new instruction from the program memory. The control automaton does not “know” the function to be performed, it “knows” how to “fetch the function” from an external storage support, the program memory.

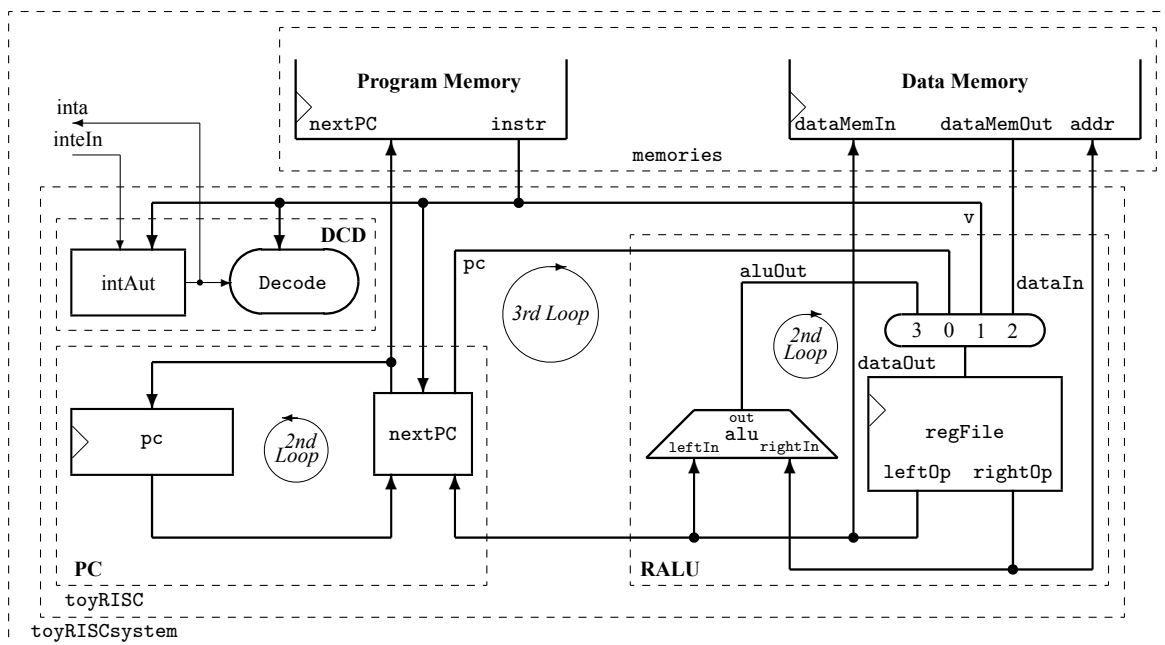


Figure 11.6: The organization of the *toyRISC* processor in its simulation environment where the program memory and data memory are included.

Important note: the organization presented in the following is designed only from the circuit point of view with emphasis on the *competence* of the system. In order to maximize the performance the design must apply specific techniques to increase the frequency of the clock cycle. This concern goes beyond the aims of this lecture. We will partially address these issues in the next lesson. Books on the organization and architecture of computing systems deal extensively with issues of maximizing performance. Books on the architecture and organization of computers written by John Hennessy and David Patterson are

recommended Hennessy and Patterson (2019); Patterson and Hennessy (2019).

In Figure 11.6, the organization of the processor *toyRISC* is represented. It contains two simple loop-connected automata, PC and RALU, serially connected with a complex and small section, DCD.

11.3.1.1 PC: a control automaton

The Control section is a simple functional automaton whose state, stored in the register called Program Counter, (*pc*), is used to compute in each clock cycle the address in Program Memory from where the next instruction is fetched. There are two modes to compute the next address: (1) incrementing, with 1 or a signed number, the current address, or (2) independently from the current value of Program Counter, using a value fetched from an internal register or a value generated by the currently executed instruction. More, the current $pc+1$ can be stored in an internal register when the control of the program *calls* a function and a return is needed. For all the previously described behaviors the combinational circuit **nextPC** (described in `nextPc.v` module of the design presented in Appendix 17.4) is designed to close the loop over the Program Counter register.

11.3.1.2 RALU: a functional automaton

The RALU section is also a simple functional automaton whose state is structured as an array of variables and is stored in a Register File (a small memory with two output ports and one input port). In each clock cycle two values are fetched from this memory and are submitted to be operated in the ALU unit. The result is write back in a location selected by a destination address. The section accepts data coming from the data memory, from the currently executed instruction, or from the **PC** automaton, thus closing the 3rd loop.

11.3.1.3 DCD: interrupt automaton & decoder

The DCD section contains an automaton, the interrupt automaton, and a combinational decoder.

In computers, an interrupt is a signal for the processor to interrupt currently executing code. If the interrupt is accepted, the processor will suspend its current activities, save its state, and execute the function requested by the interrupt. This interruption is temporary, allowing the software to resume normal activities after the program associated to the interrupt finishes.

There are two types of interrupts:

- Hardware interrupts when the interrupt signal generated from external devices (for example, in a keyboard if we press a key to do some action this pressing of the keyboard generates a signal that is given to the processor to do action). They are classified into two types:
 - ◊ Maskable Interrupt: interrupts that can be delayed when a highest priority interrupt has occurred to the processor.
 - ◊ Non Maskable Interrupt: that cannot be delayed and immediately be serviced by the processor.
- Software interrupts generated from internal devices divided into two types:
 - ◊ Normal Interrupts: are caused by the software instructions are called software instructions.

- ◇ Exception: an unplanned interruption while executing a program (for example, while executing a program if we got a value that is divided by zero).

In our very simple example we illustrated a maskable hardware interrupt. The circuit `intSect` consists of a finite automaton with two states, `enableInterrupt` and `disableInterrupt`, controlled with the instructions `ei` and `di`. When the system is reset, the machine switches to the `disableInterrupt` state. If the automaton is in the `enableInterrupt` state and the interrupt signal is activated, then `inta` (interrupt acknowledge) is generated, the `pc+1` value is saved in `regFile`, to be able to return the program to the point where it was interrupted, and the instruction from `intAddr` where the program associated with the interruption treatment is located is read. This program ends with an unconditional jump to the address saved in `regFile`.

Both, the **PC** automaton and the **RALU** automaton are simple, recursively defined automata. The computational complexity is completely moved in the code stored inside the program memory.

11.3.2 Instruction Set Architecture

The storage resources on which the ISA definition is based are the following:

```
reg    [31:0]  programMemory[0:65535]  ;
reg    [31:0]  dataMemory[0:1023]      ;
reg    [15:0]  pc                      ; // program counter
reg    [31:0]  registers[0:31]         ; // register file
reg                      intEnable      ; // interrupt enable FF
```

The structure of the arithmetic & logic instructions have two forms:

- `destinationRegister <= leftOperamd OP rightOperand`
- `destinationRegister <= leftOperamd OP immediateValue`

with the following two formats:

```
instruction[31:0] = {opCode[5:0],          // operation code
                    dest[4:0],             // destination register
                    left[4:0],             // left operand
                    right[4:0],            // right operand
                    noUse[10:0]}
|
{opCode[5:0],
 dest[4:0],
 left[4:0],
 immediate[15:0]} // value
```

Instruction Set Architecture, ISA, of the *toyRISC* processor is described in the Figure ???. ISA, in short the architecture of a computing system includes at least the following types of instructions:

- 1) control instructions

- 2) arithmetic and logic instructions
- 3) memory access instructions
- 4) interrupt control instructions

```

1  /*****
2  File name: DEFINES.vh
3          MICROARCHITECTURE
4  *****/
5  // CONTROL
6  `define nop      6'b00_0000 // no operation: pc<=pc+1;
7  `define rjmp     6'b00_0001 // relative jump: pc<=pc+v;
8  `define zbr      6'b00_0010 // pc<=(rf[l]=0) ? pc+v:pc+1
9  `define nzbr     6'b00_0011 // pc<!=(rf[l]=0) ? pc+v:pc+1
10 `define ret      6'b00_0101 // return: pc<=rf[l][15:0];
11 `define halt     6'b00_0110 // halt until interrupt
12 `define eint     6'b00_1000 // set enable interrupt
13 `define dint     6'b00_1001 // set disable interrupt
14 // ARITHMETIC & LOGIC, for these instructions: pc<=pc+1;
15 `define add       6'b11_0000 // rf[d]<=rf[l]+rf[r];
16 `define sub       6'b11_0001 // rf[d]<=rf[l]-rf[r];
17 `define addv      6'b11_0010 // rf[d]<=rf[l]+v;
18 `define mult      6'b11_0011 // rf[d]<=rf[l]*rf[r];
19 `define multv     6'b11_0100 // rf[d]<=rf[l]*v;
20 `define addc      6'b11_0101 // rf[d]<=(rf[l]+rf[r])[32];
21 `define subc      6'b11_0110 // rf[d]<=(rf[l]-rf[r])[32];
22 `define addvc     6'b11_0111 // rf[d]<=(rf[l]+v)[32];
23 `define lsh       6'b11_1000 // rf[d]<=rf[l] >> 1;
24 `define ash       6'b11_1001 // rf[d]<=
25                     // <={rf[l][31], rf[l][31:1]};
26 `define move      6'b11_1010 // rf[d]<=rf[l];
27 `define swap      6'b11_1011 // rf[d]<=
28                     // <={rf[l][15:0], rf[l][31:16]};
29 `define bwnot     6'b11_1100 // rf[d]<=~rf[l];
30 `define bwand     6'b11_1101 // rf[d]<=rf[l]&rf[r];
31 `define bwor      6'b11_1110 // rf[d]<=rf[l]|rf[r];
32 `define bwxor     6'b11_1111 // rf[d]<=rf[l]^rf[r];
33 // MEMORY, for these instructions: pc=pc+1;
34 `define read      6'b10_0000 // read from dataMemory[rf[l]];
35 `define load      6'b10_0111 // rf[d]<=dataOut;
36 `define store     6'b10_1000 // dataMemory[rf[l]]<=rf[r];
37 `define val       6'b01_0111 // rf[d]<={{16*v[15]}, v};

```

Listing 11.2: toyRISC ISA. File: 0_DEFINES.vh (SystemVerilog)

The first field of the instruction, opCode, contains the information used to determine what is the the action performed by the current instruction. Each instruction is executed in one clock cycle. In Appendix 17.4 the entire design is listed. This design is meant to illustrate mainly the competence of the circuits without aiming to reach some performances in execution. In order to maximize the performance, it is

necessary to apply special techniques (usually in the category of pipelines) related to the field of quantitative optimization of the organization of the computer system. See more on the quantitative approach in Hennessy and Patterson (2019).

11.3.3 Assembly Code

In order to generate the code executable by the toyRISC processor, a code generator translates the mnemonics into binary code. In Table ??, the mnemonics used to write assembly programs are listed. In the second column the action performed specified and in the last column the binary codes are listed. The binary code is organized in 5 or 4 fields. The first field represents the operation code, the second the destination address of the result in the register file (dddd is the binary code of the actual value when it matters), the third field represents the address for the left operand (lllll is the binary code of the actual value when it matters). The least significant 16 bits are organized in 2 fields or in one field. In the first case 5 bits (rrrrr is the binary code of the actual value when it matters) represents the address of the right operand and the rest 11 bits have no meaning, while in the second case all the 16 bits represents a value (vvvvvvvvvvvvvvvv is the binary code of the actual value when it matters).

Table 11.2: Assembly language mnemonics, their meaning and binary form.

Mnemonics	Description	Binary form
NOP	$rf[0] \leq rf[0] + 0$	110010_00000_00000_00000_000000000000
RJMP(label)	$pc \leq pc + \text{value}$	000001_00000_00000_vvvvvvvvvvvvvvv
BRZ(left,label)	$pc \leq (\text{left} = 0) ? pc + \text{immediate} : pc + 1$	000010_00000_lllll_vvvvvvvvvvvvvvv
BRNZ(left,label)	$pc \leq (\text{left} = 0) ? pc + 1 : pc + \text{immediate}$	000011_00000_lllll_vvvvvvvvvvvvvvv
RET	$pc \leq rf[\text{left}]$	000101_00000_lllll_00000_000000000000
HALT	$pc \leq pc$	000110_00000_00000_00000_000000000000
EINT	$\text{intEnable} \leq 1$ (enable interrupt)	001000_00000_00000_00000_000000000000
DINT	$\text{intEnable} \leq 0$ (disable interrupt)	001001_00000_00000_00000_000000000000
ADD(dest,left,right)	$rf[\text{dest}] \leq rf[\text{left}] + rf[\text{right}]$	110000_ddddd_lllll_rrrrr_000000000000
SUB(dest,left,right)	$rf[\text{dest}] \leq rf[\text{left}] - rf[\text{right}]$	110001_ddddd_lllll_rrrrr_000000000000
ADDV(dest,left,value)	$rf[\text{dest}] \leq rf[\text{left}] + \text{value}$	110010_ddddd_lllll_vvvvvvvvvvvvvvv
MULT(dest,left,right)	$rf[\text{dest}] \leq rf[\text{left}] * rf[\text{right}]$	110011_ddddd_lllll_rrrrr_000000000000
MULTV(dest,left,value)	$rf[\text{dest}] \leq rf[\text{left}] * \text{value}$	110100_ddddd_lllll_vvvvvvvvvvvvvvv
ADDC(dest,left,right)	$rf[\text{dest}] \leq (rf[\text{left}] + rf[\text{right}])[32]$	110101_ddddd_lllll_rrrrr_000000000000
SUBC(dest,left,right)	$rf[\text{dest}] \leq (rf[\text{left}] - rf[\text{right}])[32]$	110110_ddddd_lllll_rrrrr_000000000000
ADDVC(dest,left,value)	$rf[\text{dest}] \leq (rf[\text{left}] + \text{value})[32]$	110111_ddddd_lllll_vvvvvvvvvvvvvvv
LSH(dest,left)	$rf[\text{dest}] \leq \{0, rf[\text{left}][31:1]\}$	111000_ddddd_lllll_00000_000000000000
ASH(dest,left)	$rf[\text{dest}] \leq \{rf[\text{left}][31], rf[\text{left}][31:1]\}$	111001_ddddd_lllll_00000_000000000000
MOVE(dest,left)	$rf[\text{dest}] \leq rf[\text{left}]$	111010_ddddd_lllll_00000_000000000000
SWAP(dest,left)	$rf[\text{dest}] \leq rf[\text{left}][15:0], rf[\text{left}][31:16]$	111011_ddddd_lllll_00000_000000000000
NOT(dest,left)	$rf[\text{dest}] \leq \neg rf[\text{left}]$	111100_ddddd_lllll_00000_000000000000
AND(dest,left,right)	$rf[\text{dest}] \leq rf[\text{left}] \& rf[\text{right}]$	111101_ddddd_lllll_rrrrr_000000000000
OR(dest,left,right)	$rf[\text{dest}] \leq rf[\text{left}] rf[\text{right}]$	111110_ddddd_lllll_rrrrr_000000000000
XOR(dest,left,right)	$rf[\text{dest}] \leq rf[\text{left}] \oplus rf[\text{right}]$	111111_ddddd_lllll_rrrrr_000000000000
READ(left)	read from dataMemory[left]	100000_00000_lllll_00000_000000000000
LOAD(dest)	$rf[\text{dest}] \leq \text{dataOut}$	100111_ddddd_00000_00000_000000000000
STORE(left,right)	dataMemory[left] $\leq rf[\text{right}]$	101000_00000_lllll_rrrrr_000000000000
VAL(dest,val)	$rf[\text{dest}] \leq \{11 \text{val}[20], \text{value}\}$	010111_ddddd_00000_vvvvvvvvvvvvvvv

11.3.3.1 Toy Assembler

The binary codes in the 11.2 table are generated, for reasons of maintaining a simple design and simulation environment, using a program also written in System Verilog. This program, `RISCcodeGenerator.sv`, receives a sequence of instructions in the file `program.sv` and generates the executable binary code in the program memory `progMemory`. A portion of this generator is listed below to illustrate the idea of two-pass encoding; the whole is attached in the 17.4.2 section. In the first pass, the absolute positions of the labels are identified through the LB task in the `labelTab` table, and in the second pass through the ULB task, the relative jump is calculated, which is correctly completed in the code binary. The two tasks are necessary because some relative jumps are at advanced locations compared to the position of the jump instruction.

```

1    reg [5:0]    opCode        ;
2    reg [4:0]    d              ;
3    reg [4:0]    l              ;
4    reg [4:0]    r              ;
5    reg [15:0]   v              ;
6    reg [9:0]    addrCounter    ;
7    reg [9:0]    labelTab[0:1023];
8
9    `include "DEFINES.vh"
10
11    task endLine;
12    begin
13        progMemory[addrCounter][31:0] =
14            {    opCode    ,
15                d          ,
16                l          ,
17                v          } ;
18        addrCounter = addrCounter + 1 ;
19    end
20    endtask
21
22    // sets labelTab in the first pass
23    // associating 'counter' with 'labelIndex'
24    task LB ;
25        input [5:0] labelIndex;
26
27        labelTab[labelIndex] = addrCounter;
28    endtask
29    // uses the content of labelTab in the second pass
30    task ULB;
31        input [5:0] labelIndex;
32
33        v = labelTab[labelIndex] - addrCounter;
34    endtask
35
36    // CONTROL INSTRUCTIONS
37    task NOP; // no operation

```

```

38         begin    opCode = `addv ;
39                 d      = 5'b0 ;
40                 l      = 5'b0 ;
41                 v      = 16'b0 ;
42                 endLine ;
43     end
44 endtask
45
46 task RJMP; // relative jump
47     input [15:0] label ;
48
49     begin    opCode = `rjmp ;
50             d      = 5'b0 ;
51             l      = 5'b0 ;
52             ULB(label) ;
53             endLine ;
54     end
55 endtask
56
57 task BRZ; // branch if zero
58     input [4:0] left ;
59     input [9:0] label ;
60
61     begin    opCode = `zbr ;
62             d      = 5'b0 ;
63             l      = left ;
64             ULB(label) ;
65             endLine ;
66     end
67 endtask
68
69 // ...
70
71 // ARITHMETIC & LOGIC INSTRUCTIONS
72 task ADD; // addition
73     input [4:0] dest ;
74     input [4:0] left ;
75     input [4:0] right ;
76
77     begin    opCode = `add ;
78             d      = dest ;
79             l      = left ;
80             v      = {right, 11'b0};
81             endLine ;
82     end
83 endtask
84
85 // ...
86 task LSH; // logic shift with one position
87     input [4:0] dest ;
88     input [4:0] left ;

```

```

89
90     begin    opCode = `lsh  ;
91             d      = dest  ;
92             l      = left  ;
93             v      = 16'b0 ;
94             endLine
95     end
96 endtask
97
98 // ...
99
100 task AND; // bitwise AND
101     input  [4:0]  dest    ;
102     input  [4:0]  left    ;
103     input  [4:0]  right   ;
104
105     begin    opCode = `bwand      ;
106             d      = dest        ;
107             l      = left        ;
108             v      = {right, 11'b0};
109             endLine
110     end
111 endtask
112
113 // ...
114
115 // DATA TRANSFER INSTRUCTIONS
116 task READ; // data read
117     input  [4:0]  left    ;
118
119     begin    opCode = `read  ;
120             d      = 5'b0   ;
121             l      = left   ;
122             v      = 16'b0  ;
123             endLine
124     end
125 endtask
126
127 task LOAD; // data load
128     input  [4:0]  dest    ;
129
130     begin    opCode = `load  ;
131             d      = dest    ;
132             l      = 5'b0    ;
133             v      = 16'b0   ;
134             endLine
135     end
136 endtask
137
138 task STORE; // data store
139     input  [4:0]  left    ;

```

```

140         input    [4:0]    right    ;
141
142         begin      opCode   = `store          ;
143                   d        = 5'b0           ;
144                   l        = left           ;
145                   v        = {right, 11'b0};
146                   endLine   ;
147         end
148     endtask
149
150     // ...
151
152     // RUNNING
153     initial begin      addrCounter = 0;
154                       `include "program.sv" // first pass
155                       addrCounter = 0;
156                       `include "program.sv" // second pass
157     end

```

Listing 11.3: A sample of the binary code generator for toyRISC processor. File: RISCcodeGenerator.sv (SystemVerilog)

SystemVerilogSummary 5 :

task taskName; **input** [...] inputName; ... **begin** taskBody **end endtask** : describes the task taskName of parameters inputName ... which performs the actions defined by taskBody.

11.3.3.2 Simulator

The simulator used to validate the processor design is described in detail in the 17.4.3 section, where the processor was instantiated and the data, dataMemory, and program, ProgMemory memories were added. The interrupt signal intIn has been activated, which will be taken into account immediately by the instruction DI will allow it.

11.3.3.3 Assembly Programs

Example 11.2 Let by a following simple program:

```

1          VAL(0,1)      ;
2          VAL(1,2)      ;
3          VAL(2,3)      ;
4          VAL(3,4)      ;
5          VAL(4,5)      ;
6          ADD(0,0,1)     ;
7          ADD(0,0,2)     ;
8          ADD(0,0,3)     ;
9          ADD(0,0,4)     ;

```

```
10          HALT          ;
```

Listing 11.4: toyRISC program: initializes the first 5 registers and adds them in register 0. File: A11_RISCprogram1.sv (SystemVerilog)

The code generator loads in the program memory the following binary form of the program:

```
progMemory[0] = 01011100000000000000000000000001
progMemory[1] = 01011100001000000000000000000010
progMemory[2] = 01011100010000000000000000000011
progMemory[3] = 010111000110000000000000000000100
progMemory[4] = 010111001000000000000000000000101
progMemory[5] = 1100000000000000000001000000000000
progMemory[6] = 110000000000000000001000000000000
progMemory[7] = 110000000000000000001100000000000
progMemory[8] = 110000000000000000010000000000000
```

The clock period is equal with 2 time units. The program starts at the time unit $t=5$ after the 4 time units action of the reset signal. The value of the program counter and of the first 2 registers in the register file evolve as follows:

```
pc= 0 RF=[1, x, x, x]  ei=0 inta=0
pc= 1 RF=[1, x, x, x]  ei=0 inta=0
pc= 2 RF=[1, 2, x, x]  ei=0 inta=0
pc= 3 RF=[1, 2, 3, x]  ei=0 inta=0
pc= 4 RF=[1, 2, 3, 4]  ei=0 inta=0
pc= 5 RF=[1, 2, 3, 4]  ei=0 inta=0
pc= 6 RF=[3, 2, 3, 4]  ei=0 inta=0
pc= 7 RF=[6, 2, 3, 4]  ei=0 inta=0
pc= 8 RF=[10, 2, 3, 4] ei=0 inta=0
pc= 9 RF=[15, 2, 3, 4] ei=0 inta=0
```

◇

Example 11.3 *Let us take an example to show how the interrupt acts. The interrupt is applied from the beginning of the test, but it is enabled only after the instruction EI runs.*

```
1          VAL(31,10)    ; // loads in register 31 the subroutine address
2          VAL(2,23)     ;
3          VAL(0,13)     ;
4          EI            ; // enable the action of the interrupt
5          ADDV(0,0,2)   ;
6          NOP           ;
7          ADDV(0,0,4)   ;
8          HALT          ;
9          NOP           ;
10         NOP           ;
11 // subroutine triggered by interrupt: loads 44 in register 3
12         DI            ; // disable the action of the interrupt
13         VAL(3,44)     ;
14         RET(30)       ; // jump to the interrupted instruction
```

Listing 11.5: toyRISC program: tests the interrupt action. File: A11_RISCprogram2.sv

(SystemVerilog)

The toy assembler generates in progMemory the following executable code:

```

progMemory[0]    = 0101111111000000000000000001010
progMemory[1]    = 01011100010000000000000000010111
progMemory[2]    = 010111000000000000000000000001101
progMemory[3]    = 001000000000000000000000000000000
progMemory[4]    = 11001000000000000000000000000010
progMemory[5]    = 110010000000000000000000000000000
progMemory[6]    = 110010000000000000000000000000100
progMemory[7]    = 000110000000000000000000000000000
progMemory[8]    = 110010000000000000000000000000000
progMemory[9]    = 110010000000000000000000000000000
progMemory[10]   = 001001000000000000000000000000000
progMemory[11]   = 010111000110000000000000000101100
progMemory[12]   = 000101000001111000000000000000000
progMemory[13]   = xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx

```

The simulation provides:

```

pc= 0  RF=[x, x, x, x]    ei=0 inta=0
pc= 1  RF=[x, x, x, x]    ei=0 inta=0
pc= 2  RF=[x, x, 23, x]   ei=0 inta=0
pc= 3  RF=[13, x, 23, x]  ei=0 inta=0
pc= 4  RF=[13, x, 23, x]  ei=1 inta=1
pc= 10 RF=[13, x, 23, x]  ei=0 inta=0
pc= 11 RF=[13, x, 23, x]  ei=0 inta=0
pc= 12 RF=[13, x, 23, 44] ei=0 inta=0
pc= 4   RF=[13, x, 23, 44] ei=0 inta=0
pc= 5   RF=[15, x, 23, 44] ei=0 inta=0
pc= 6   RF=[15, x, 23, 44] ei=0 inta=0
pc= 7   RF=[19, x, 23, 44] ei=0 inta=0

```

◇

Example 11.4 *Let be a wrong program which runs forever because of an unconditional jump.*

```

1      VAL(0,3)           ;
2      LB(1); ADDV(0,0,-1);
3      NOP                ;
4      RJMP(1)            ; // unconditional jump to the address labeled
                           1
5      HALT               ;

```

Listing 11.6: toyRISC program: an unending running program. File: A11_RISCprogram3.sv (SystemVerilog)

The simulation provides:


```

pc= 0  RF=[3, x, x, x]      ei=0 inta=0
pc= 1  RF=[3, x, x, x]      ei=0 inta=0
pc= 2  RF=[2, x, x, x]      ei=0 inta=0
pc= 3  RF=[2, x, x, x]      ei=0 inta=0
pc= 1  RF=[2, x, x, x]      ei=0 inta=0
pc= 2  RF=[1, x, x, x]      ei=0 inta=0
pc= 3  RF=[1, x, x, x]      ei=0 inta=0
pc= 1  RF=[1, x, x, x]      ei=0 inta=0
pc= 2  RF=[0, x, x, x]      ei=0 inta=0
pc= 3  RF=[0, x, x, x]      ei=0 inta=0
pc= 1  RF=[0, x, x, x]      ei=0 inta=0
pc= 2  RF=[4294967295, x, x, x] ei=0 inta=0
pc= 3  RF=[4294967295, x, x, x] ei=0 inta=0
pc= 1  RF=[4294967295, x, x, x] ei=0 inta=0
pc= 2  RF=[4294967294, x, x, x] ei=0 inta=0
pc= 3  RF=[4294967294, x, x, x] ei=0 inta=0
pc= 1  RF=[4294967294, x, x, x] ei=0 inta=0
pc= 2  RF=[4294967293, x, x, x] ei=0 inta=0
pc= 3  RF=[4294967293, x, x, x] ei=0 inta=0
pc= 1  RF=[4294967293, x, x, x] ei=0 inta=0

```

◇

Example 11.5 *Let be a program which works with dataMemory.*

```

1          VAL(0,1)      ; // loads the address 1 in register 0
2          VAL(1,55)     ; // loads the value 55 in register 1
3          STORE(0,1)    ; // store at the address 1 the value 55
4          READ(0)       ; // reads from address stored in registe 0
5          LOAD(2)       ; // load 55, read for address 1, in register 2
6          HALT          ;

```

Listing 11.7: toyRISC program: tests the access to the data memory. File: A11_RISCprogram4.sv (SystemVerilog)

The simulation provides:

```

pc= 0  RF=[1, x, x, x]      ei=0 inta=0
pc= 1  RF=[1, x, x, x]      ei=0 inta=0
pc= 2  RF=[1, 55, x, x]     ei=0 inta=0
pc= 3  RF=[1, 55, x, x]     ei=0 inta=0
pc= 4  RF=[1, 55, x, x]     ei=0 inta=0
pc= 5  RF=[1, 55, 55, x]    ei=0 inta=0

```

◇

11.3.4 Time performance

The longest combinational path in a system using our *toyRISC*, which imposes the minimum clock period, is:

$$T_{clock} = t_{clock_to_instruction} + t_{leftAddr_to_leftOp} + t_{throughALU} + t_{throughMUX} + t_{fileRegSU}$$

Because the system is not buffered the clock frequency depends also by the time behavior of the system directly connected with *toyRISC*. In this case $t_{clock_to_instruction}$ – the access time of the program memory, related to the active edge of the clock – is an extra-system parameter limiting the speed of our design. The internal propagation time to be considered are: the read time from the file register ($t_{leftAddr_to_leftOp}$ or $t_{rightAddr_to_rightOp}$), the maximum propagation time through ALU (dominated by the time for an 32-bit arithmetic operation), the propagation time through a 4-way 32-bit multiplexer, and the *set-up time* on the file register's data inputs. The way from the output of the file register through **Next PC** circuit is “shorter” because it contains a 16-bit adder, comparing with the 32-bit one of the ALU.

The increase in speed performance will be illustrated in the next lesson by using pipeline registers.

11.4 How is Designed an Instruction Set Architecture

As we already said, we use the term ISA to refer to the actual programmer-visible instruction set. The ISA serves as the boundary between the software engineer and hardware engineer. We will list and comment further on the main characteristics of an ISA

- Class of ISA
 - ◊ register-memory ISAs such as x86, with complex memory access instructions
 - ◊ load-store ISAs such as ARM, RISC V, with only simple load and store instructions for memory access
- Memory addressing is byte addressing with two versions:
 - ◊ byte aligned (ex.: ARM) if the object accessed at address A has s bytes, then the $A \bmod s = 0$
 - ◊ with no alignment (ex.: x86 and RISC V) which produces slower access
- Addressing modes
 - ◊ few simple: register, immediate, displacement, ... (RISC V, ARM)
 - ◊ many complex: the previous and more (x86)
- Types and sizes of operands
 - ◊ integer: 8 bit (ASCII), 16 bit (Unicode character), 32-bit (word), 64-bit (double word)
 - ◊ floats: standard IEEE 754 32-bit floating point and 64-bit floating point
- Operations:
 - ◊ data transfer
 - ◊ arithmetic

- * integer: add, sub, mult, div, rem, rightShift, ...
- * floats: add, sub, mult, div, rem
- ◇ logic: minimally AND and XOR
- ◇ control: jumps, conditional branches, calls, ret, ...
- Control flow instructions:
 - ◇ RISC V tests the value of a register for conditional branches
 - ◇ x86 and ARM test flags in a state register
 - ◇ x86 save the return address from subroutine on a stack memory organized in the main mamory
- Encoding an ISA:
 - ◇ variable length for x86
 - ◇ fix length for RISC V and ARM

11.5 Problems

Problem 11.1 *Instal on your computer the design of the toyRISC processor using modules listed in Section 17.4, and run the simple program which loads in the first 4 registers the numbers 11, 22, 33, 44.*

Solution

The program is:

```

1          VAL (0 ,11)      ;
2          VAL (1 ,22)      ;
3          VAL (2 ,33)      ;
4          VAL (3 ,44)      ;
5          HALT              ;

```

Listing 11.8: toyRISC program used to validate the simulator. File: A11_RISCprogram5.sv (SystemVerilog)

The program generated by `RISCcodeGenerator.sv` is:

```
progMemory[0] = 010111000000000000000000000001011
progMemory[1] = 0101110000100000000000000000010110
progMemory[2] = 01011100010000000000000000000100001
progMemory[3] = 01011100011000000000000000000101100
progMemory[4] = 000110000000000000000000000000000000000
progMemory[5] = xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
```

The result of simulation is:

```
pc= 0  RF=[11, x, x, x]
pc= 1  RF=[11, x, x, x]
pc= 2  RF=[11, 22, x, x]
pc= 3  RF=[11, 22, 33, x]
pc= 4  RF=[11, 22, 33, 44]
```

Problem 11.2 Add to the list of monitoring signals the last two register in the register file in order to verify the way the interrupt signal is managed. Write a program which contains the subroutine of 5 instruction at the location 20 in the program memory. Enable the interrupt action in the sixth step of the main program. Make also all the necessary adjustment in module `testRISC.sv` to allow verifying the interrupt action.

◇

Problem 11.3 The add-prefix function receives the sequence of numbers $[n_0, n_1, \dots, n_{m-1}]$ and returns

$$[n_0, n_0 + n_1, n_0 + n_1 + n_2, \dots, \sum_{i=0}^{m-1} n_i]$$

Write the program that load in the first 16 registers a sequence of different numbers and compute the add-prefix sequence in the last 16 registers.

◇

Problem 11.4 In registers 0 and 1 are stored the positive integers A and B . Write in assembly the program that compute in register 2 the quotient A/B and 3 the remainder.

Hint: subtract as many times as possible B from A counting in 2 successes. In register 3 leave the rest.

◇

What instruction or instructions should be added to the architecture to simplify the program in Problem 11.4?

Problem 11.5 Load in data memory the first 32 Fibonacci numbers and then add them in register 3.

◇

Part III

COMPUTING

Chapter 12

Computers: Four-Loop, Fourth-Order Systems (4-OS)

12.1	The mathematical model of the Turing Machine	240
12.1.1	The Decision Problem	240
12.1.2	Turing Machine	241
12.1.3	Theoretical Foundations of Number Representations in Turing's Work	241
12.1.4	Representation of Numbers Using Symbols and States	242
12.1.5	Summary of Key Theoretical Points in Turing's Paper	243
12.2	von Neumann Abstract Model	244
12.3	System Organization	244
12.4	Memory Management	246
12.4.1	Memory Gap	246
12.4.2	Memory Hierarchy	247
12.4.3	Virtual Memory Mechanism	248
12.4.4	Associative Memory-Based Page Translator	249
12.5	I/O	253
12.5.1	Bus	253
12.5.2	FIFO	254
12.5.3	DMA	257
12.5.4	I/O Devices	262

After the first part of this textbook described the symbolic structures involved in computing, and the second part described the physical structures that can be used to implement computing systems, the time has come to involve the unifying entity represented by *information*. Indeed, information is the one that takes control once the serendipitous meeting between representations, physical structures, and an acting symbolic structure occurs. The symbolic structure we are referring to is the information that manifests itself in the form of computer programs. Computing systems represent the synthesis between the physical structure of circuits and the symbolic structure of programs. This was made possible by a conceptual clarification – the emergence of mathematical models of computation – and a technological evolution – the structuring of digital systems by increasing their autonomy through the impact of included loops.

12.1 The mathematical model of the Turing Machine

In current language we use the term computation without a rigorous definition. When we have to substantiate the concept of computability and computation, rigor is absolutely necessary. We will continue with not only a historical but also a theoretical introduction to the concept of computation in the sense in which it is conceived in computational science.

12.1.1 The Decision Problem

Despite appearances, computers are not numerical instruments but logical structures that have their theoretical origin in solving decision-making problems. To the question: *where and when does the history of computers begin?*, the answer, however strange it may seem, is: *somewhere on the island of Crete about two and a half millennia ago*. Indeed, Epimenides of Knossos (the capital of the island of Crete), a semi-mythical Greek seer from the 7th or 6th century BC, caught himself saying that *"all Cretans are liars"*. Surprised, because in the next second he could not **decide** whether the assertion he had just made was true or false. Thus was born, if the story is true, a problem that occupied the minds of seekers for millennia until it was rigorously reformulated by David Hilbert (1862–1943) and definitively settled by Kurt Gödel (1906–1978).

Hilbert's address of 1900 to the International Congress of Mathematicians in Paris is perhaps the most influential speech ever given about mathematics (Hilbert, 1900). In it, Hilbert outlined 23 major mathematical problems to be studied in the coming century. One of them, the 10th, raises the issue of decision in a particular case. But, in a definitive form the problem is exposed in the book that David Hilbert writes together with Wilhelm Ackermann (Hilbert and Ackermann, 1928). This book included a clear statement of the *Entscheidungsproblem* (decision problem): *is there an algorithm that takes a statement formulated in a formal system and answers with "yes" or "no" whether the statement is true or false in the given system?* The first answer comes in 1931 from the logician Kurt Gödel: ***in a formal system one can correctly construct true structures whose truth cannot be proven or disproven*** (Goedel, 1931) (Goedel, 1986).

Gödel's spectacular result triggered within five years four mathematicians Alonzo Church (Church, 1936), Stephen Kleene (Kleene, 1936), Emil Post (Post, 1936), and Alan Turing (Turing, 1936, 1937) (listed alphabetically) to independently provide works that founded computer science. We focus in this chapter on Turing's work.

12.1.2 Turing Machine

In his paper, about "computable numbers" and Entscheidungsproblem, Alan Turing describes in words what he called a *computing machine*. We call it now Turing Machine (TM) and we describe it as a structure with three components (see Figure 12.1a):

- an infinite **tape**¹ on/from which symbols belonging to a finite set of symbols – the machine's alphabet – can be write or read using
- an accessing write/read **head** which can move in each clock cycle one position to left or right under the control of
- a **finite automaton** (FA) whose state is updated in each clock cycle according to its state and the symbol accessed from the tape.

The translation into a representation containing digital components is shown in Figure 12.1b, where the infinite tape is substituted by an infinite RAM addressed by an up-down counter.

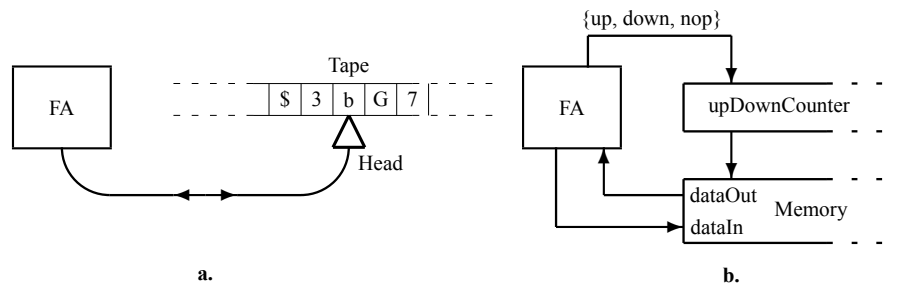


Figure 12.1: Turing Machine. **a.** Original description: Tape & Head & FA. **b.** Digital representation: Memory & upDownCounter & FA.

TM is a mathematical concept because of its unlimited tape. This "infinity" is due to the fact that at the beginning of the computation the size of the space required on the tape is not known. This ignorance is dealt with by the mathematician Turing with the convenient concept of infinity. But it is beyond any doubt that any real useful computation must be done in a finite number of cycles, so we never traverse an infinite memory.

12.1.3 Theoretical Foundations of Number Representations in Turing's Work

Alan Turing's 1937 paper, "On Computable Numbers, with an Application to the Entscheidungsproblem," primarily focuses on the concept of computability rather than explicit number representations used in modern computer systems. However, the theoretical groundwork Turing established lays the foundation for how numbers (and more broadly, data) can be represented and manipulated by machines. Here are the key theoretical points from Turing's paper that relate to number representations:

¹It is most likely that Turing was inspired by the telegraph communication systems of the 1930s that used a paper tape to receive messages.

12.1.3.1 Definition of Computable Numbers

Turing defines a **computable number** as a real number that can be calculated to any desired precision by a mechanical process. Formally, a number x is **computable** if there exists an algorithm (what Turing termed a **computing machine**, now known as a Turing Machine) that, given an integer n , can compute the n -th digit of x in a finite number of steps.

In modern terms:

$$x = \sum_{i=1}^{\infty} d_i \cdot b^{-i}$$

where d_i are digits (either binary, decimal, or other bases) and b is the base. Turing's framework implies that, to represent x as computable, there must be a recursive procedure for determining d_i for all i .

12.1.4 Representation of Numbers Using Symbols and States

Turing's model introduced the concept of a machine that reads, writes, and moves over a tape. The tape holds symbols from a finite alphabet, which can be used to represent numbers in various bases, including binary. Each number can thus be represented by a sequence of symbols on the tape, with the **state of the machine** directing operations on these symbols.

Formally:

- Let $\Sigma = \{0, 1\}$ represent the binary alphabet.
- A **string** $w \in \Sigma^*$ on the tape can represent a binary number, where the position and value of each symbol denote its magnitude.

12.1.4.1 Finite State Automaton for Number Manipulation

Turing proposed that any computable function, including arithmetic operations on numbers, could be executed by a sequence using a finite set of states. Each pair

$$\{\text{state}, \text{symbol_currently_accessed_on_tape}\}$$

directs the machine to either: 1. Write or erase a symbol, 2. Move left or right along the tape. 3. Transition to another state.

This **finite state control** is the precursor to binary arithmetic and logical operations on numbers. It implies that any number representation (binary, decimal, or otherwise) can be manipulated using a finite sequence of state transitions based on the machine's rules.

Definition 12.1 *Formally, for a Turing Machine $M(T)$, we have:*

$$M = (Q, \Sigma, \delta, q_0, F)$$

where:

- Q is a finite set of states,
- Σ is the finite tape alphabet,
- $\delta : Q \times \Sigma \rightarrow Q \times \Sigma \times \{L, R, -\}$ is the transition function, where L and R moves the access head one position to left or right,

- $q_0 \in Q$ is the initial state,
- $F \subseteq Q$ is the set of final (or halting) states.

and a tape T whose content is initially, in the state q_0 , accessed with the read/write head positioned on the first symbol.

◇

This structure allows $M(T)$ to handle number representations in binary or any base by processing symbol strings according to their state rules in a finite number of cycles. In other words, for any function $F : X \rightarrow Y$ there exists a TM whose finite automaton, starting from the initial state q_0 and having on the tape $x \in X$, reaches a final state, q_f , and generates on the tape, after a finite number of steps, $y \in Y$ if $F(x) = y$.

Important note: for each computable sequence another TM is needed.

12.1.4.2 Concept of Universal Representation

In Turing's words:

It is possible to invent a single machine which can be used to compute any computable sequence. If this machine $\mathcal{U}$ is supplied with a tape on the beginning of which is written the S.D of some computing machine $\mathcal{M}$ then $\mathcal{U}$ will compute the same sequence as $\mathcal{M}$. (Turing, 1936)

Turing's concept of a **Universal Turing Machine**, UTM, suggested that any computable number or function could be represented and processed by a single machine capable of interpreting encoded instructions for any specific computation. This idea led to the realization that numbers can be encoded as **data** or **instructions**, providing a basis for multiple forms of number representation and manipulation in modern computers.

If p is a program (or representation) for a number x , then $U(p) = x$, where U is a UTM capable of simulating p . Given that a finite automaton has, by definition, a complex structure, theoretical research has carefully evaluated various versions for the FA associated with the UTM. We mention two limiting cases. One is that of the work of Claude Shannon who publishes a paper describing a UTM with an automaton having only 2 states (Shannon, 1956). The consequence of this limitation is a combinational circuit on the loop of the 2-state finite automaton of very large size. Another occurs after 50 years. It is a paper in which the finite automaton has zero states, that is, it is no longer necessary (Stefan, 2006). This establishes the possibility of simple processors, the RISC type, processors built only from simple recursively defined circuits.

Formally: for any computation done by a TM $M(T)$ can be used the UTM

$$\mathcal{U}(<M, T>)$$

where the stream of symbols M describes the automaton of the TM $M(T)$, while the stream of symbols T is the tape of the same TM. In contemporary terms, M is the program, while T is data, both stored in the same memory.

12.1.5 Summary of Key Theoretical Points in Turing's Paper

- 1) **Computable Numbers:** Defined as numbers that can be calculated to any precision using a finite, mechanical process.

- 2) **Finite State Control:** Established that a finite set of states could operate on symbols to represent numbers in various bases.
- 3) **Machine-Based Representation:** Showed that numbers could be represented as sequences of symbols on a tape, manipulated by a machine's states and transitions.
- 4) **Universal Representation:** Introduced the idea that numbers (or their representations) could be treated as data or instructions, leading to flexible encoding schemes for number manipulation.

Turing's formal model set the stage for understanding how numbers can be systematically represented, stored, and manipulated by computational systems, laying foundational principles that influence number representations in modern computing.

12.2 von Neumann Abstract Model

TM is a mathematical model. The actual computing machines need an *intermediate model*: an **abstract** one, able to solve the problem of computation using real entities. In this "moment" comes the serendipitous meeting between math and tech, between UTM and digital circuits. In the abstract model proposed by the mathematician John von Neumann the finite automaton and the accessing head are integrated in what we call a processor which communicate with the memory which contains programs and data (see Figure 12.2).

The two types of processors defined in the previous section were used to define two abstract computer models.

Historical note: in the 1940s, the first electronic computers were made in two versions that generated two models for the subsequent evolution. Traditionally, but erroneously, these models are called *architectures*. The concept of architecture appeared in the 1960s. For this reason, we will call these concepts *abstract models* instead of architectures.

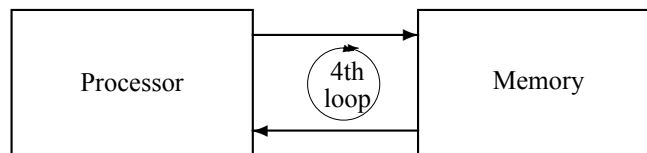


Figure 12.2: The abstract model of computer proposed by John von Neumann.

The interpretive processor is used in the definition of the *von Neumann abstract model*, in which the processor is loop connected to a single memory in which both programs and data are stored (see Figure 12.2). Thus, a 4-OS is obtained by connecting a 3-OS (Processor) with a 1-OS (Memory).

12.3 System Organization

The processor is the core of a computing system that contains also a memory subsystem and an input-output subsystem. The physical resources involved are tightly interleaved in various form of actual orga-

nization. The memory associated with the processor contains programs and data, while the input-output system has two main functions: expanding the memory capacity and ensuring the interaction of the system with the outside world.

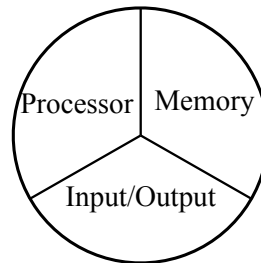


Figure 12.3: The three-partite functionality of a computing system.

The way in which the tripartite functionality is distributed in a given system shows significant variations from one implementation to another. In the following, we will limit ourselves to a simple version, but not simpler than one that allows us to introduce the main concepts and problems. But, first we will show the serendipitous way in which the evolution of logic circuits meets a fundamental mathematical model for what can be rigorously defined as computation.

How looks the actual structure of a computing system? In Figure 12.4 is exemplified a system where are emphasized two buses: the internal bus and the input-output bus. They are interconnected by two units: Bus Bridge and DMA (Direct Memory Access).

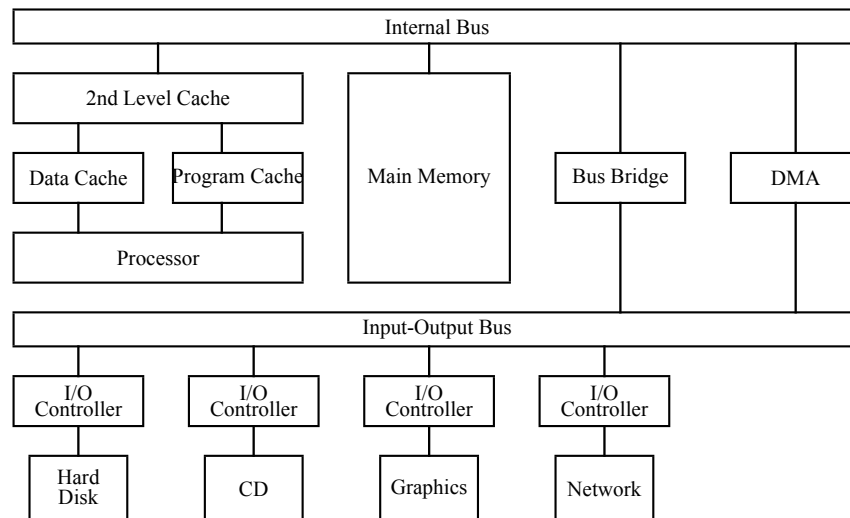


Figure 12.4: The organization of a computing system.

On Internal Bus are connected fast devices while on the Input-Output Bus are connected devices involving a lot of data transfer.

The memory hierarchy is distributed in the version represented in Figure 12.4 as follows:

- 1) Register File in the processor, having a capacity of $32W \div 1024W$, accessed for reading or writing in a single clock cycle

2) the two 1st level cache memories, each having a capacity of $128\text{KB} \div 1\text{MB}$, accessed for reading or writing in $1 \div 3$ clock cycles:

- Program Cache
- Data Cache

so that at this level the system is structured according to the Harvard abstract model

3) 2nd Level Cache memory, having a capacity of $1\text{MB} \div 8\text{MB}$, so that at this level the system is structured according to the von Neumann abstract model

4) Hard Disk connected through:

- Bus Bridge
- DMA

12.4 Memory Management

12.4.1 Memory Gap

The evolution of the performance of the three components of a computer system (see Figure 12.3) occurred at a very different rate due to strictly technological aspects. The most disturbing difference, with major structural consequences, occurred in the case of processor and memory speed performances. The frequency of the processor clock increased much faster than the memory access time (see Figure 12.5. Processor clock frequency increased with 60% per year, while the access time for memories with only 10% per year.

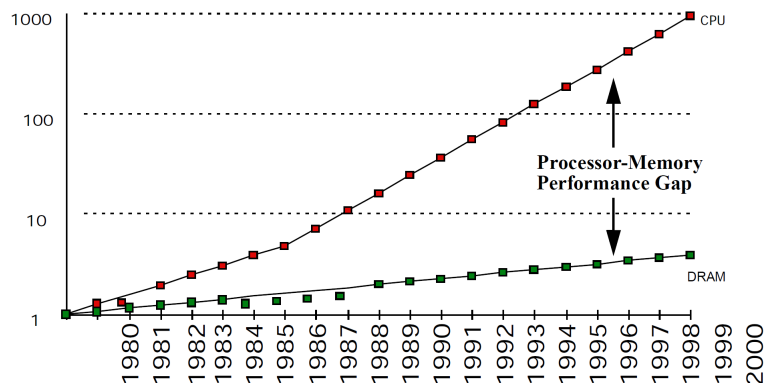


Figure 12.5: Processor-Memory Performance Gap Patterson et al. (1997).

Due to this difference, what we now call a memory gap occurred. The fast processor cannot wait for

the slow memory to deliver or receive its data. And the memory gap imposed the memory hierarchy of the computing system based on the locality principle.

Locality principle says that if a certain location is accessed in the data or program memory, then the next accesses will be made with a high probability in a reasonably small vicinity.

Applying the locality principle allowed the definition of the strategies that govern the conception of a hierarchy of effective memories.

12.4.2 Memory Hierarchy

The hierarchy of memory is based on the fact that when we access a location that is in a memory that is too slow in relation to the speed of processing in the processor, a block or a page of bytes that contains the requested bytes will be transferred to a faster intermediate memory. This transfer is done with the hope that the following accesses will be made in the intermediate memory if the updated page or block in it was large enough.

Figure 12.6 shows the current way in which the memory hierarchy is implemented. At the top of the hierarchy there are registers from register files whose content is accessible at the level of the clock signal cycle. Next comes system memory in the form of the closest level as cache memory. The name comes from the French *caché* which means hidden. Indeed, this level of memory/memories is transparent to the architecture of the computing system in most cases.

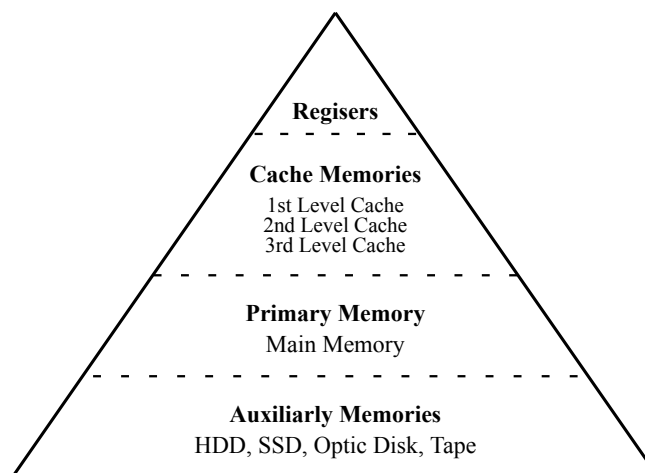


Figure 12.6: Memory hierarchy from fast and small register files to slow and large auxiliary storage memories such as HDD (Hard Disk Drive), SSD (Solid-State Drive), optical disk, tape.

Registers are managed by compilers, their size is $< 4KB$ (in most of cases $\sim 128B$, i.e., 32 32-bit words), the access time is $< 0.5ns$.

Cache memories are managed by hardware, their size is $< 16MB$, the maximum access time is $\sim 0.5ns$.

The main memory is managed by the operating system, its size is in the range of $1 \div 16GB$, the access time is $\sim 100ns$.

The disk memory is managed by the operating system or human operator, its size is $> 128GB$, the access time is $\sim 5ms$.

12.4.3 Virtual Memory Mechanism

How relate two successive levels in the hierarchy? Let us consider two levels in the memory hierarchy, $Level_i$ containing m pages and $Level_{i+1}$ containing n pages, with $n \gg m$. Each page in $Level_i$, called *physical level* is a copy of a page from $Level_{i+1}$ called *virtual level*. In Figure 12.7a, pages 0, 2, and $m-2$ from the level i were brought from pages 3, 4, and 0 of level $i+1$. Any change in the physical level must be actualized in the associated virtual level. The "user" accesses the virtual level while the virtual mechanism accesses the physical level. It is the job of the virtual memory mechanism to perform transparently the access and to keep coherent the content of the virtual level with the physical level.

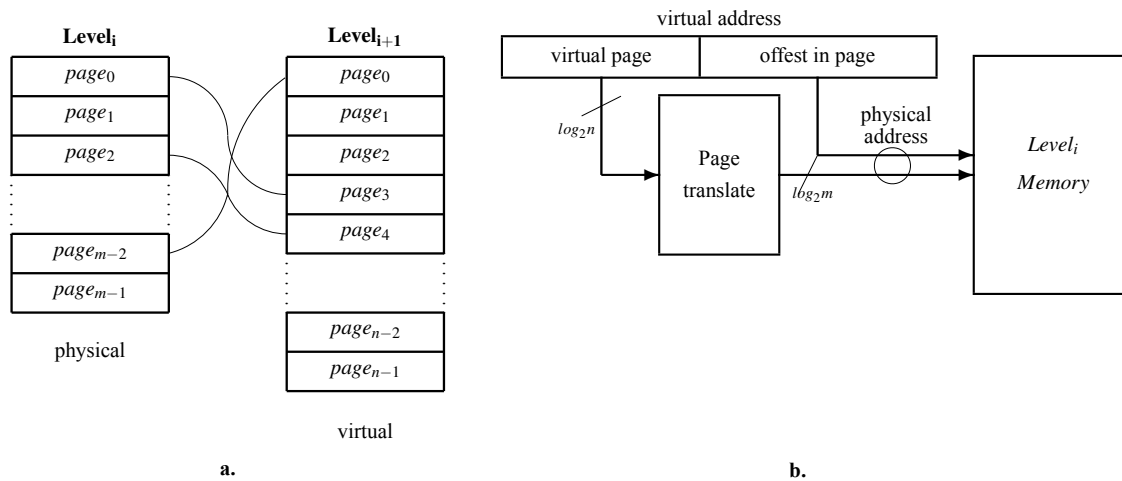


Figure 12.7: Virtual memory mechanism. **a.** Page correspondence. **b.** Translation mechanism.

The addressing mechanism is represented in Figure 12.7b, where the access to the physical memory is done mediated by a translation mechanism performed by the *Page translate* block. The "user" issues a virtual address having two parts:

- virtual page address
- offset in the page

When a new page is loaded into physical memory, the content of this translator is updated according to the correspondence shown in the figure 12.7a.

Page translation mechanism is implemented in various form.

The simplest one is a RAM memory containing $n \log_2 m$ -bit words. The RAM memory is in this case almost empty, because only m locations from n are used, because usually $n \gg m$.

A more efficient solution is to use an associative memory to make the translation.

12.4.4 Associative Memory-Based Page Translator

12.4.4.1 Content-Addressable Memory

A normal way to “question” a memory circuit is to ask for:

Q1: *what is the value of the property A of the object B*

For example: *How old is George?* The *age* is the property and the *object* is George. The first step to design an appropriate device to be questioned as previously is exemplified is to define a circuit able to answer the question:

Q2: *where is the object B?*

with two possible answers:

- 1) *the object B is not in the searched space*
- 2) *the object B is stored in the cell indexed by X.*

The circuit for answering Q2-type questions is called *Content Addressable Memory*, shortly: *CAM*. (About the question Q1 in the next subsection.)

The basic cell of a CAM consists of:

- the storage elements for binary objects
- the “questioning” circuits for searching the value applied to the input of the cell.

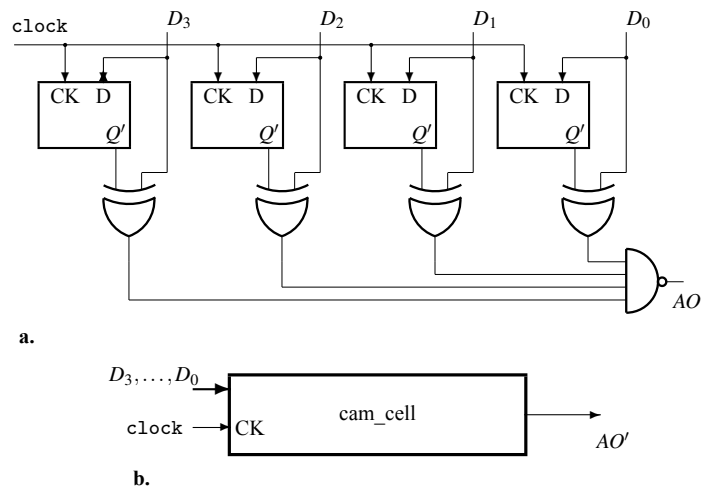


Figure 12.8: **Content Addressable Cell.** **a.** The structure: data latches whose content is compared against the input data using 4 XORs and one NAND. Write is performed applying the clock with stable data input. **b.** The logic symbol.

In Figure 12.8 there are 4 D latches as storage elements and four XORs connected to a 4-input NAND used as comparator. The cell has two functions:

- **to store:** the active level of the clock modify the content of the cell storing the 4-bit input data into the four D latches
- **to search:** the input data is continuously compared with the content of the cell generating the signal $AO' = 0$ if the input matches the content.

The cell is *written* as an m -bit latch and is continuously *interrogated* using a combinational circuit as comparator. The resulting circuit is an 1-OS because results serially connecting a memory, one-loop circuit with a combinational, no-loop circuit. No additional loop is involved.

An n -word CAM contains n CAM cells and some additional combinational circuits for distributing the clock to the selected cell and for generating the global signal M , activated for signaling a successful match between the input value and one or more cell contents. In Figure 12.9a a 4-word of 4 bits each is represented. The write enable, WE , signal is demultiplexed as clock to the appropriate cell, according to the address coded by A_1A_0 . The 4-input NAND generate the signal M . If, at least one address output, AO'_i is zero, indicating match in the corresponding cell, then $M = 1$ indicating a successful search.

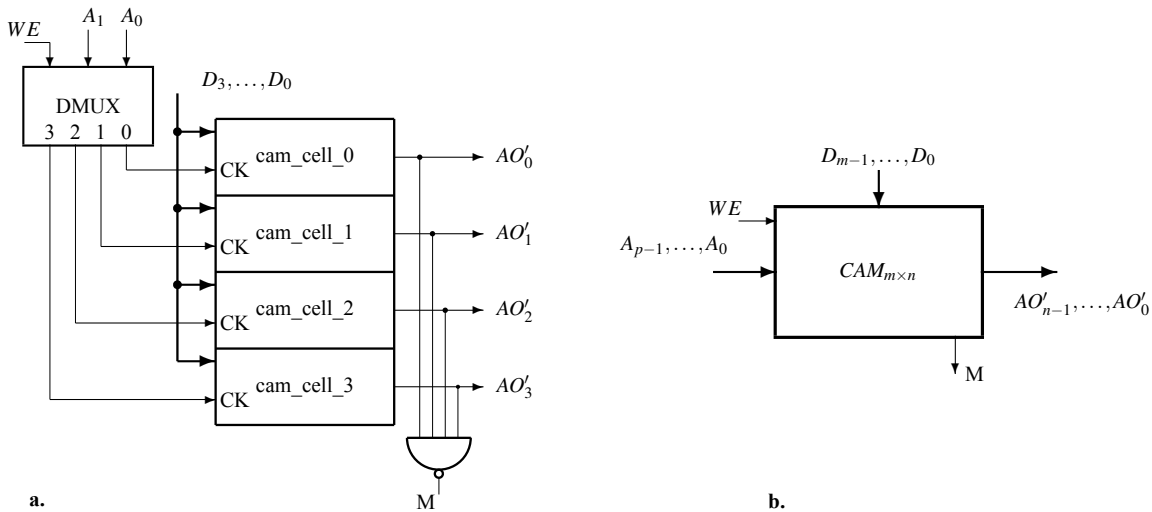


Figure 12.9: **The Content Addressable Memory (CAM).** **a.** A 4-word CAM is built using 4 content addressable cells, a demultiplexer to distribute the write enable (WE) signal, and a $NAND_4$ to generate the match signal (M). **b.** The logic symbol.

The *input address* $A_{p-1}, \dots, A_0$ is binary coded on $p = \log_2 n$ bits. The *output address* $AO'_{n-1}, \dots, AO'_0$ is a *unary code* indicating the place or the places where the data input $D_{m-1}, \dots, D_0$ matches the content of the cell. The output address must be unary coded because there is the possibility of match in more than one cell.

Figure 12.9b represents the logic symbol for a CAM with n m -bit words. The input WE indicate the function performed by CAM. Be very careful with the set-up time and hold time of data related to the WE signal!

The CAM device is used to locate an object (to answer the question **Q2**). Dealing with the properties of an object (answering **Q1**-type questions) means to use one or more complex devices which *associate* one

or more properties to an object. Thus, the *associative memory* will be introduced adding some circuits to CAM.

12.4.4.2 Associative Memory

A partially used RAM can be an associative memory, but a very inefficient one. Indeed, let be a RAM addressed by $A_{n-1} \dots A_0$ containing 2-field words $\{V, D_{m-1} \dots D_0\}$. The *objects* are coded using the address, the *values* of the unique property P are coded by the data field $D_{m-1} \dots D_0$. The one-bit field V is used as a validation flag. If $V = 1$ in a certain location, then there is a match between the object designated by the corresponding address and the value of property P designated by the associated data field.

Example 12.1 *Let be the 1Mword RAM addressed by $A_{19} \dots A_0$ containing 2-field 17-bit words $\{V, D_{15} \dots D_0\}$. The set of objects, OBJ , are coded using 20-bit words, the property P associated to OBJ is coded using 16-bit words. If*

$$RAM[11110000111100001111] = 1_0011001111110000$$

$$RAM[11110000111100001010] = 0_0011001111110000$$

then:

- *for the object 11110000111100001111 the property P is defined ($V = 1$) and has the value 0011001111110000*
- *for the object 11110000111100001010 the property P is not defined ($V = 0$) and the data field is meaningless.*

Now, let us consider the 20-bit address codes four-letter names using for each letter a 5-bit code. How many locations in this memory will contain the field V instantiated to 1? Unfortunately, only extremely few of them, because:

- *only 24 from 32 binary configurations of 5 bits will be used to code the 24 letters of Latin alphabet ($24^4 < 2^{20}$)*
- *but more important: how many different name expressed by 4 letters can be involved in a real application? Usually no more than few hundred, meaning almost nothing related to 2^{20} .*

◇

The previous example teaches us that a RAM used as associative memory is a very inefficient solution. In real applications are used names coded very inefficiently:

$$number_of_possible_names \gg \gg number_of_actual_names.$$

In fact, the natural memory function means almost the same: to remember about something immersed in a huge set of possibilities.

One way to implement an efficient associative memory is to take a CAM and to use it as a *programmable decoder* for a RAM. The (extremely) limited subset of the actual objects are stored into a CAM, and the address outputs of the CAM are used instead of the output of a combinational decoder to select the accessed location of a RAM containing the value of the property P . In Figure 12.10 this version

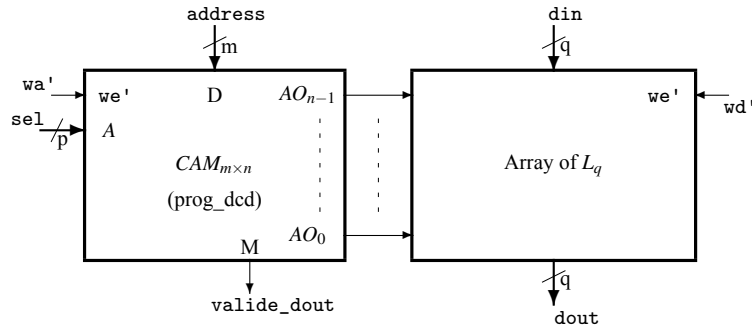


Figure 12.10: **An associative memory (AM).** The structure of an AM can be seen as a RAM with a programmable decoder implemented with a CAM. The decoder is programmed loading CAM with the considered addresses.

of an associative memory is presented. $CAM_{m \times n}$ is usually dimensioned with $2^m \gg n$ working as a decoder programmed to decode **any** very small subset of n addresses expressed by m bits.

Here are the three working mode of the previously described associative memory:

define_object : write the name of an object to the selected location in CAM

$wa' = 0$, $address = name_of_object$, $sel = cam_address$
 $wd' = 1$, $din = don't_care$

associate_value : write the associated value in the randomly accessed array to the location selected by the active address output of CAM

$wa' = 1$, $address = name_of_object$, $sel = don't_care$
 $wd' = 0$, $din = value$

search : search for the value associated with the name of the object applied to the address input

$wa' = 1$, $address = name_of_object$, $sel = don't_care$
 $wd' = 1$, $din = don't_care$
 $dout$ is valid only if $valide_dout = 1$.

This associative memory will be dimensioned according to the dimension of the actual subset of names, which is significantly smaller than the virtual set of the possible names ($2^p \lll 2^m$). Thus, for a searching space with the size in $O(2^m)$ a device having the size in $O(2^p)$ is used.

12.4.4.3 Translation Lookaside Buffer (TLB)

The associative memory will allow us to design a translation lookaside buffer (TLB) with a size given by the number of pages in the physical memory, unlike the one based on a RAM type memory which has a size given by the much larger number of pages in the virtual memory.

With a minimal number of differences, the described virtualization mechanism is applied in managing the relationship between all the hierarchical levels in the memory of the computer system.

In what follows, we will consider the simple example in which the hierarchy contains a single cache level, the main memory and the hard disk. We will compare the main parameters that characterize the cache-main memory relationship and the main memory-hard disk relationship.

- the typical amount of data moved when physical memory is actualized from the virtual memory
 - ◊ block size for cache: 128B
 - ◊ page size main memory: 4KB
- the typical access time when the addressing is hit
 - ◊ 1 clock cycle (very rarely 2 or 3) for cache
 - ◊ ~ 100 clock cycles because of board technology and silicon technology for memories (the first is reduced by multi-chip packaging) for main memory
- the typical miss penalty:
 - ◊ 100 clock cycles for cache memory
 - ◊ 10^6 clock cycles for main memory
- the typical miss rate:
 - ◊ 1% for cache memory
 - ◊ $10^{-4}\%$ for main memory

12.5 I/O

The input-output system includes certain specific mechanisms that we will briefly describe first. Then we will briefly describe the most important input-output devices.

12.5.1 Bus

A **Bus** is a communication system that transfers data between components inside a computer, or between computers. There are two types of buses: serial (those that transfer data bit by bit) and parallel (those that transfer data word by word of n bits). The main problem that arises is that of the synchronization of the transfer. We will discuss it under its essential aspects for the synchronous transfer of n -bit words.

In Figure 12.11, the waveforms for data transfer on the Internal Bus (see Figure 12.4) between Main Memory and 2nd Level Cache are presented. Three important moments are marked in the mentioned waveforms:

- t_1 : the active edge of clock takes a read command (Read = 0) from the address Read Address. Both Read and Read Address must be stable at least minimum **set-up time** before the active edge of clock (the positive one) and **hold time** after the positive edge of clock
- t_2 : the active edge of clock can take the data from the output of Main Memory only if the signal Wait is not active (Wait = 0) thus completing the read cycle. The signal Wait is necessary when the Main Memory is not a fast enough memory, or the clock frequency is too high.

t_3 : the active edge of clock takes Valid Input Data to write it at Write Address because Write signal is active (Write' = 0).

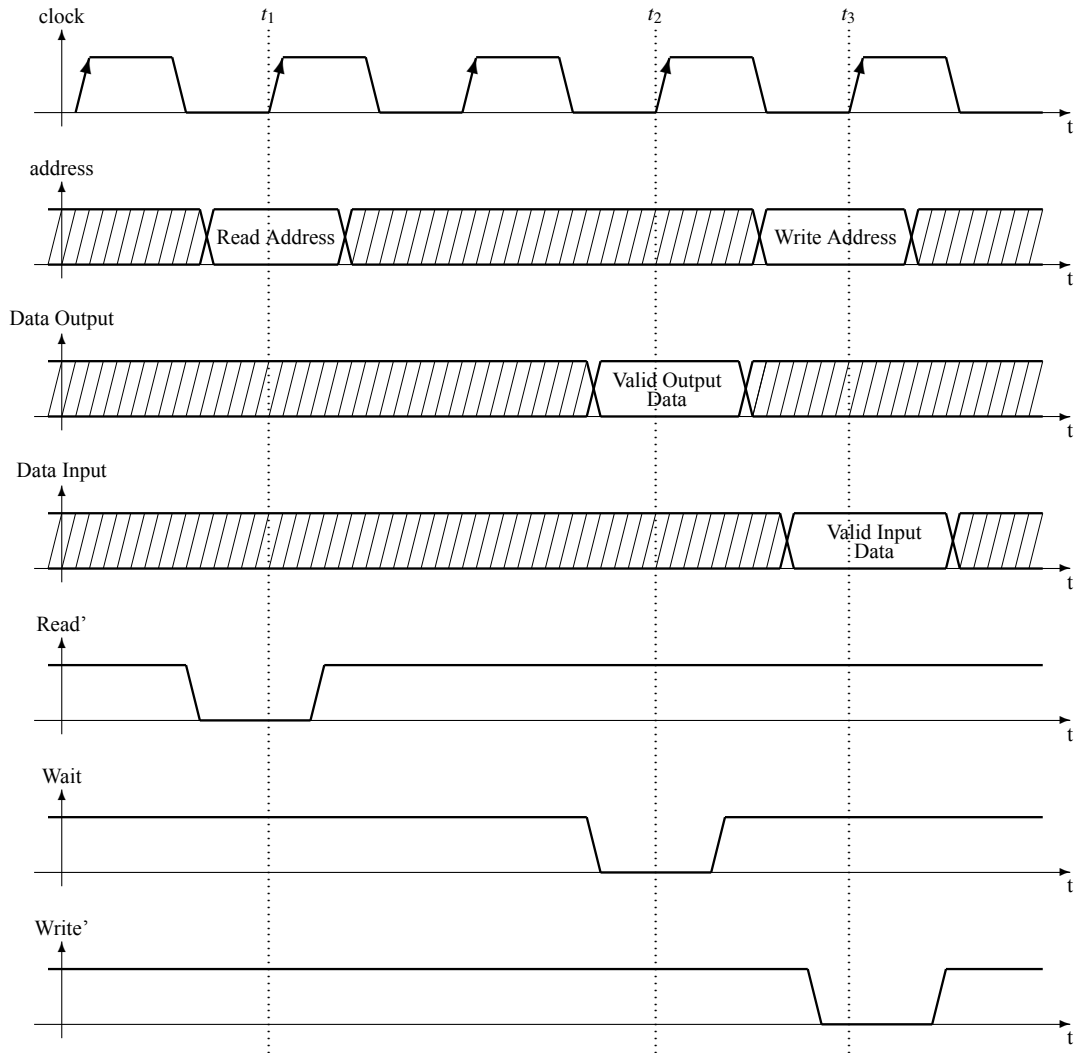


Figure 12.11: The waveforms for a read cycle followed by a write cycle.

The Wait signal is deactivated (Wait = 0) at t_2 introducing a latency of 2 clock cycles in the example illustrated by Figure 12.11. Usually, this latency is very high, reaching hundreds of cycles.

12.5.2 FIFO

12.5.2.1 Definition

First-In-First-Out (FIFO) memory is a special memory device represented in Figure 12.12. It has the following connections (see Figure 12.12a):

dataIn : the data entry of n bits to which the word that will be recorded in the rightmost unoccupied location is applied.

dataOut : the n bit data output from which the oldest unextracted record is extracted

full : it signals the fact that the memory is full and will not accept a new write before executing at least one read

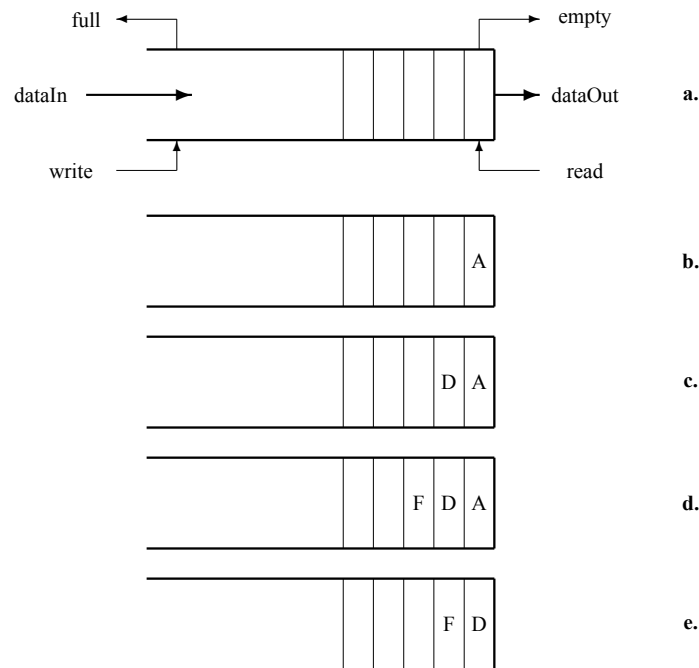


Figure 12.12: **a.** Block schematic of FIFO. **b.** write A operation. **c.** write D. **d.** write F. **e.** read.

write : if activated when $full = 0$, then **dataIn** is queued

empty : it signals the fact that the memory is empty and will not accept a new read before executing at least one write

read : if activated when $empty = 0$, then **dataOut** is extracted from the queue

In Figures from 12.12b to 12.12e is exemplified the way FIFO works, as follows:

write(A) : in the empty FIFO A is stored in the rightmost position (see Figure 12.12b)

write(D) : B is stored in the rightmost free position, immediately after A (see Figure 12.12c)

write(F) : F is stored in the rightmost free position, immediately after B (see Figure 12.12d)

read : A is extracted from FIFO and the queue "advances" one step (see Figure 12.12e)

There are two main types of FIFO:

- synchronous FIFO: the sending system and the receiving system run with the same clock signal (for those who are interested can consult Appendix D)
- asynchronous FIFO: the sending system and the receiving system run with different clock signals

Asynchronous FIFO are used to solve the problem of crossing data between systems running in different clock domain. The module BUS Bridge (see Figure 12.4) contains almost sure an asynchronous FIFO.

12.5.2.2 Structure

In Figure D.1 is presented a solution for the synchronous version, where:

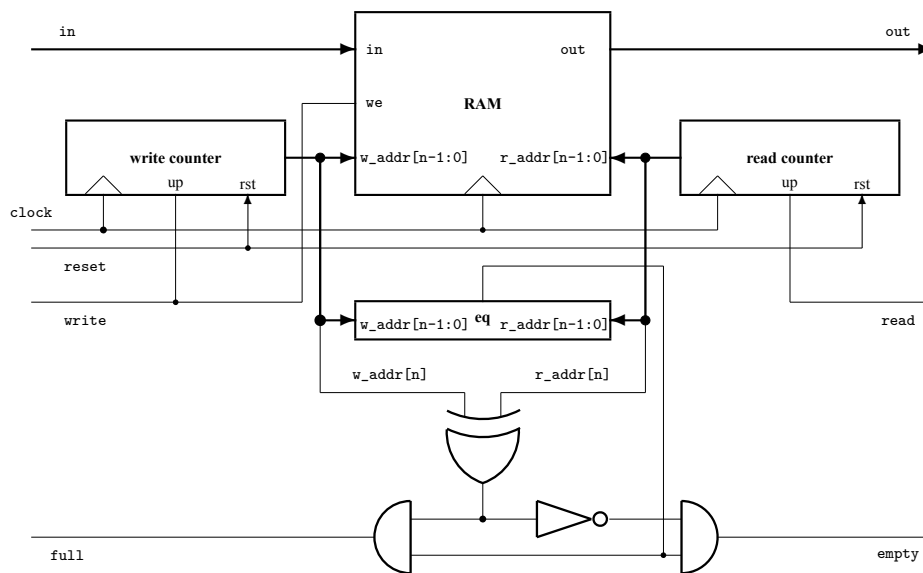


Figure 12.13: **FIFO memory**. Two pointers, evolving in the same direction, and a two-port RAM implement a LIFO (queue) memory. The limit flags are computed combinational from the addresses used to write and to read the memory.

- **RAM** is a 2^n m -bit words two-port asynchronous random access memory, one port for write to the address w_addr and another for read from the address r_addr
- **write counter** is an $(n+1)$ -bit resettable counter incremented each time a write is executed; its output is $w_addr[n:0]$, initially it is reset
- **read counter** is an $(n+1)$ -bit resettable counter incremented each time a read is executed; its output is $r_addr[n:0]$, initially it is reset
- **eq** is a comparator activating its output when the least significant n bits of the two counters are identical.

FIFO works like a circular memory addressed by two pointers ($w_addr[n-1:0]$ and $r_addr[n-1:0]$) running on the same direction. If the write pointer *after* a write operation becomes equal with the read pointer, then the memory is full and the `full` signal is 1. If the read pointer *after* a read operation becomes equal with the write pointer, then the memory is empty and the `empty` signal is 1. The $n+1$ -th bit in each counter is used to differentiate between empty and full when $w_addr[n-1:0]$ and $r_addr[n-1:0]$ are the same. If $w_addr[n]$ and $r_addr[n]$ are different, then $w_addr[n-1:0] = r_addr[n-1:0]$ means full, else it means empty.

The circuit used to compare the two addresses is a combinational one. Therefore, its output has a hazardous behavior which affects the outputs `full` and `empty`. These two outputs must be used carefully in designing the system which includes this FIFO memory. The problem can be managed because the system works in the same clock domain (`clock` is the same for both ends of FIFO and for the entire system). We call this kind of FIFO *synchronous FIFO*.

12.5.3 DMA

12.5.3.1 Definition

Direct memory access (DMA) (see Figure 12.4) is an important feature of computer systems. It allows to access the main memory system independently of the central processor.

When the processor is running an input/output program, it is occupied for the entire duration of the read or write operation. Thus the processor is unavailable to perform other work. With DMA, the processor initiates the transfer only, then it does other operations while the transfer is performed. When the transfer is completed the processor receives an interrupt from the DMA (about interrupt see 11.3.1.3 in this lecture notes).

An example of using the DMA unit is for transferring a page of 4KB from Hard Dist to Main Memory. The CPU initializes the DMA by sending the given information through the data bus:

- the starting address of the memory block where to be read or write.
- the number of words in the memory block
- control bit to define the mode of transfer, read or write.
- control bit to begin the DMA transfer

The DMA controller transfers the data in three modes:

- burst mode:
 - ◇ entire data or burst of block containing data is transferred before CPU takes control of the buses back from DMA
 - ◇ is the quickest mode of DMA Transfer since at once a huge amount of data is being transferred
 - ◇ since at once only the huge amount of data is being transferred so time will be saved in huge amount
 - ◇ **pros**: fastest mode of DMA Transfer

- ◇ **cons:** less user friendly because during the DMA transfer CPU will be blocked.
- cycle stealing mode for each transfer:
 - ◇ slows I/O device because will take some time to prepare data and within that time CPU keeps the control of the buses.
 - ◇ once the data is ready CPU give back control of system buses to DMA for 1-cycle in which the prepared word is transferred to/from memory.
 - ◇ as compared to burst mode this mode is little bit slowest since it requires little bit of time which is actually consumed by I/O device while preparing the data.
 - ◇ **pros:** most Efficient way for DMA Transfer.
 - ◇ **cons:** CPU won't be blocked entire time.
- transparent or interleaving mode:
 - ◇ whenever CPU does not require the system buses, then the control of buses will be given to DMA
 - ◇ CPU will not be blocked due to DMA at all
 - ◇ it is the slowest mode of DMA transfer since DMAC has to wait might be for so long time to just even get the access of system buses from the CPU itself
 - ◇ less amount of data will be transferred
 - ◇ **pros:** CPU will not be blocked at all
 - ◇ **cons:** Slowest DMA transfer rate.

The transfer between an I/O device and MEMORY under the control of the DMA is initiated by the next sequence mode:

- 1) I/O sends request DMA, **DRQ**, and DMA sends to CPU the hold request, **HLD**
- 2) CPU acknowledge to DMA with **HLDA**
- 3) DMA acknowledge to I/O with **DACK**

and when data transfer is done, DMA sends an *interrupt*, INT, to PROCESSOR (see Figure 12.14).

12.5.3.2 Structure

The structure of a minimal DMA connected between two buses, BusA and BusB, contains three main modules:

- a buffer for the data being transferred sometimes between BusA and BusB and other times between BusB and BusA

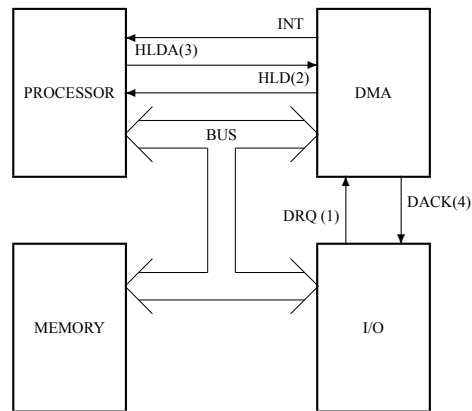


Figure 12.14: Initiating the transfer between I/O and Memory.

- a presetable counter for the memory address from which or to which data is transferred
- a counter extended automaton to control the transfer process.

The resulting block schematic is represented in Figure 12.15, where:

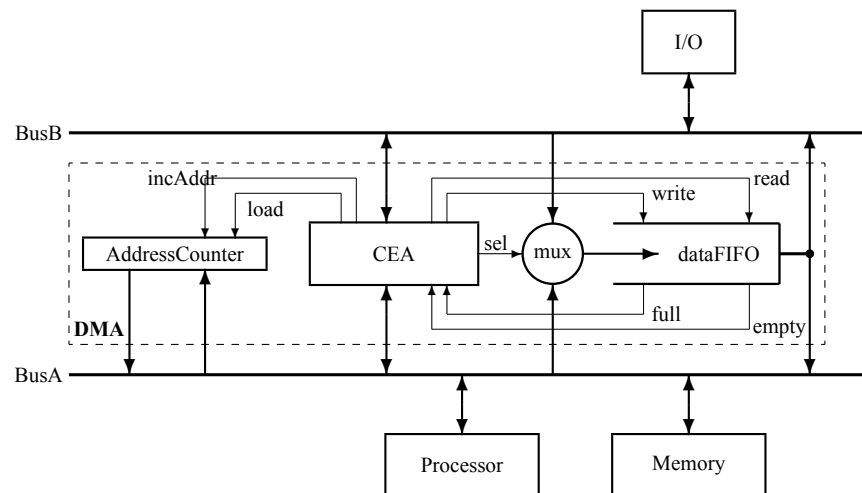


Figure 12.15: Block schematic for a minimal DMA.

- CEA is the counter extended automaton whose counter is initiated with data from BusA under the Processor command; the automaton of CEA receives and sends signal from/on both buses and generates commands for the multiplexer and FIFO
- FIFO is used as a buffer to optimize the transfer time between the two busses where the speed

of transfer are very different; the block of data on BusA is transferred faster than the same block of data on BusB because Memory is much faster than the I/O device

- the multiplexor, mux, allows to use the same FIFO for the transfers in both directions, from I/O to Memory or from Memory to I/O
- AddressCounter is used to generate the addresses for Memory; it is loaded by Processor in the initialization process of a transfer

Representation from Figure 12.15 is a functional schematic and does not contain the interface circuits between DMA and the two buses.

The critical timing in the transfer is on the bus BusA where DMA competes with Processor in using the bus for communicating with Memory in the computing process. For this reason, the transfer mode on BusA is burst mode facilitated by the FIFO memory. When I/O transfers a block in Memory, first the FIFO memory is loaded with a block of data at the low speed of the I/O device, then the FIFO memory is emptied in a burst performed at the high speed of the Memory module. Symmetrical, the FIFO memory is fast loaded from Memory in burst mode, then the data is transferred at the low speed imposed by the I/O device on the BusB. The overall time is imposed by the slowest device, the I/O system, but the time during which the processor is deprived of the possibility of using the BusA bus is minimized by the appropriate use of the FIFO memory.

In addition to the signals (read, write, sel, load, increment) and flags (empty, full) that the CEA generates inside the DMA, the control signals (commands and flags) on the two buses are also managed by the same circuit, as follows:

- on BusA:
 - ◊ commands: HLD, INT, readA, writeA, startReadA, startWriteA
 - ◊ flags: HLDA, waitA
- on BusB:
 - ◊ commands: DACK, readB, writeB, startReadB, startWriteB
 - ◊ flags: DRQ, waitB

In addition to these signals, the CEA finite state machine also has signals related to the associated pre-settable counter. These are: the zero flag (which indicates that the counter has reached zero by decrementing), and the preset (loads the transfer size minus 1 in the pre-settable counter) and dec (decrements the counter value on each write or read).

Example 12.2 *The process of transferring a block of data from or to a I/O device using a minimal DMA structure is presented. It is considered that the system works with two buses, an internal bus, BusA, and an input-output bus, BusB. On the internal bus we need to protect the computing process from too intensive use of the bus by DMA. For this we will use the BusA transfer in burst mode. This type of transfer negotiates the takeover of the bus control only once, and the transparent mode increases the transfer time too much.*

```

/*****
The transfer procedure has only two modes: (1) read from I/O to memory
a block of maximum 4096 words or (2) write from memory to I/O a block
of maximum 4096 words.
*****/
state0: idle state, waiting for starting a read/write transfer
        {nextState,com} = startReadA ? {state1,load} :
                          (startWriteA ? {state2,load} : (state0, preset))

state1: Processor loads in FIFO the source block address in I/O
        {nextState,com} = {state3, write}

state3: Processor loads in AddressCounter the starting address in Memory
        {nextState,com} = {state4, load}

state4: I/O receives from FIFO the pointer to the block
        (nextState,com) = {state5, read, writeB}

state5: the block to be transferred is loaded in FIFO
        {nextState,com} = waitB ? {state5,-} :
                          (zero ? {state11, readB, write, dec, preset} :
                           {state5, readB, write, dec})

state11: DMA request control on BusA
        {nextState,com} = HLDA ? {state6, HLD} : {state11, HLD}

state6: the bloc is transferred in burst mode from FIFO to Memory
        {com,nextState} = {writeA,(waitA ? state6 :
                          (zero ? {read, incAddr, dec, state0} :
                           {read, incAddr, dec, state6})))}

state2: Processor loads in FIFO the destination block address in I/O
        {nextState,com} = {state7, write}

state7: Processor loads in AddressCounter the starting address in Memory
        {nextState,com} = {state8, load}

state8: I/O receives from FIFO the pointer to the block
        (nextState,com) = {state9, read, writeB}

state9: the block is transferred in burst mode from Memory to FIFO
        {com,nextState} = {readA,(waitA ? state9 :
                          (zero ? {write, incAddr, dec, state10} :
                           {write, incAddr, dec, state9})))}

state10: the block is transferred from FIFO to I/O
        {nextState,com} = waitB ? {state10,-} :
                          (zero ? {state0, writeB, write, dec, preset} :
                           {state10, writeB, write, dec})

```

◇

12.5.4 I/O Devices

There are two types of I/O devices:

- Storage devices, mainly represented by hard disk, flash memory, optical disk, tape.
- Functional devices, mainly represented by display system, network interface, ...

We will briefly present the storage devices that extend the memory hierarchy outside the central computing unit.

12.5.4.1 Hard Disk

The most common auxiliary memory is the hard disk drive. Its internal structure is represented in Figure 12.16.

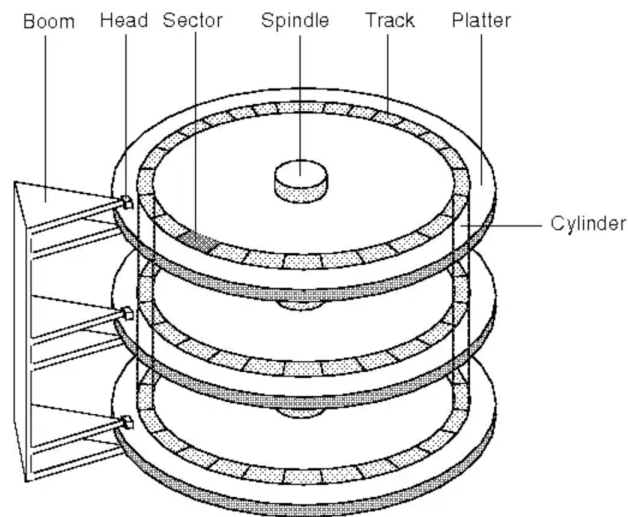


Figure 12.16: Hard Disk Drive Structure [Source: Google-Images]

Main parameters:

- storage capacity: $10 \div 30$ TB
- average access time: $2.5 \div 10$ ms
- MTBF: ~ 285 years (MTBF stands for mean time between failures; it is the predicted elapsed time between inherent failures of a mechanical or electronic system, during normal system operation.)
- bytes per sector: $512 \div 4096$

- shock tolerance
 - ◊ operating: $10 \div 100$ g (g stands for gravitational acceleration: $9.8m/s^2$.)
 - ◊ nonoperating: $100 \div 100$ g

12.5.4.2 Optical Disks

Optical disks are read-only mediums. There are two versions:

- CD: optical compact disk (0.65 GB)
- DVD: digital video disk (4.7 GB); sometimes written on both sides to double capacity

12.5.4.3 Flash Memory

Flash memory is a type of EEPROM (electronically erasable programmable readonly memory), which is normally read-only but can be erased. Solid-state devices (SSDs) based on flash memory technology do not have moving parts.

The capacity per flash memory chip increased by about 50%–60% per year.

Typical SSDs will have a the time taken for a disk drive to locate the area on the disk where the data to be read is stored between 0.08 and 0.16 ms.

Time to **read** 2 KB from a flash memory takes about $75 \mu S$, while DDR SDRAM takes less than 500 ns. Then flash memory is about 150 times slower. But compared to HDD is ~ 400 times faster. We conclude: the flash memory can not replace DRAM for main memory, but is a good candidate to replace HDD.

Time to **write** for flash memory is about 1500 times slower then SDRAM, and about 8–15 times faster than HDD.

12.5.4.4 Tapes

Magnetic-tape data storage is a system for storing information on magnetic tape using digital recording.

Typically 9-track tape are used to store bytes with CRC. Magnetic tape packaged in cartridges and cassettes. The device that performs the writing or reading of data is called a tape drive.

Tape data storages are now used mainly for system backup, data archive or data exchange.

Uncompressed/Native capacity: $\sim 20TB$.

Compressed capacity: $\sim 50TB$.

Chapter 13

Computers: Five-Loop, Fifth-Order Systems (5-OS)

13.1 Harvard Computer Version	266
13.2 Cache Memories	266

13.1 Harvard Computer Version

The Harvard abstract model assumes a fifth loop. The processor together with the program memory forms a 4-OS, and by closing another loop through the data memory, a 5-OS is obtained (see Figure 13.1).

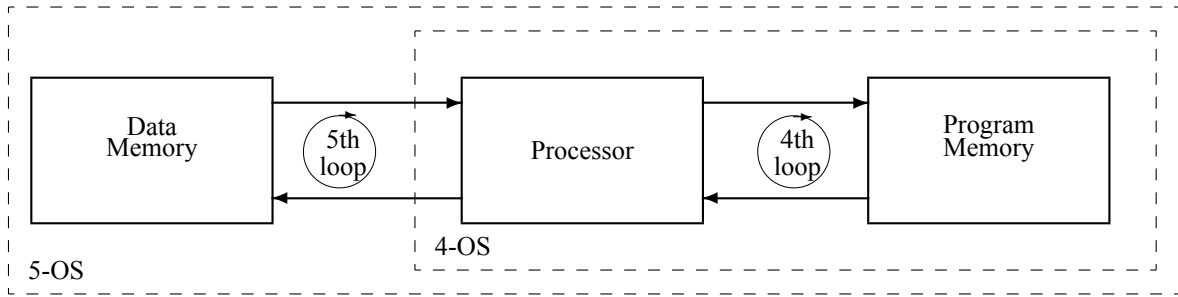


Figure 13.1: The Harvard abstract model of computer.

We will continue to focus, for obvious reasons, on RISC-type processors that allow the implementation of the Harvard abstract model.

13.2 Cache Memories

Chapter 14

Parallel Computation: N-Loop, N-Order systems (N-OS)

14.1 Cellular Automata (CA)	268
14.2 Controlled Cellular Automaton	270
14.2.1 Structure	270
14.2.2 Mapping	271
14.2.3 Architecture	272
14.3 First Global Loop in CA	274
14.3.1 Structure	274
14.3.2 Architectural additions	276
14.4 Second Global Loop in CA	277
14.4.1 Structure	277
14.4.2 Architectural additions	279
14.5 The mathematical model of partially recursive functions	279
14.6 Heterogeneous Computing	281
14.6.1 Complexity vs. intensity in computation	281
14.6.2 Heterogeneous system	282
14.6.3 Programming a heterogeneous system	283
14.7 Parallel Computing Patterns	288
14.7.1 Patterns with Direct Correspondence	289
14.7.2 Patterns Executable on Map-Only	290
14.7.3 Pattern Executable on MapReduce Only	293
14.7.4 Patterns Executable on MapScanReduce	293
14.7.5 Super-scalar Sequence Pattern	295

As we saw in the second part of this book, we were able to achieve the requirements of a computing system by successively introducing feedback loops starting from combinational circuits, circuits without loops. Can we continue the process of closing new loops? Could we have an additional gain from these additional loops? We will try to answer these questions in the following.

14.1 Cellular Automata (CA)

Each new loop increases, as we have seen in the previous chapters, the complexity and the autonomy of the system. When the number of loops is N , for any large N , we cannot keep the increase in complexity at a level that corresponds to the number of loops. For this, the loops must close over identical systems to keep the complexity independent of N . Thus, even once the size becomes proportional to N , the complexity (the size of the shortest description) remains independent of N and we can afford to keep the complexity independent of size. Moore's law ensures exponential growth only for size, but not for complexity. In other words, we can exponentially increase the size of the hardware only if it has a repetitive pattern that keeps the description at a controllable size.

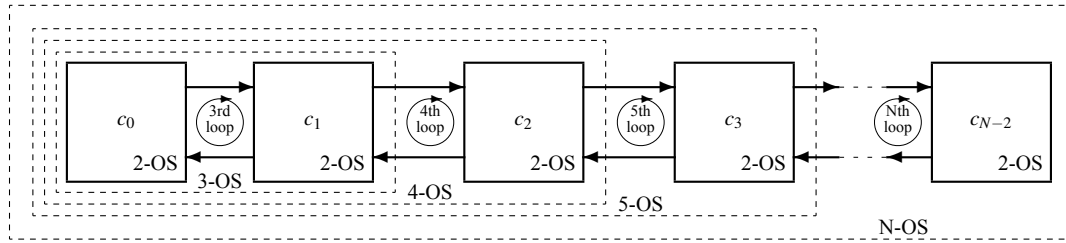


Figure 14.1: An example of cellular structure as a Nth order system, N-OS. In this example, each cell is a second order system, 2-OS.

In Figure 14.1 a system of order N , N-OS, is represented. The size of this system is in $O(N)$, while the complexity can also be in $O(N)$ or in $O(1)$. In the first case the system is complex, and in the second case the system is simple. The first case corresponds to the situation in which the cells $c_0, \dots, c_1, \dots$ are different, requiring, consequently, independent descriptions. The second case corresponds to the situation in which the cells are identical, in which case the description of the system involves the description of the cell and the regular way of interconnection of $n - 1$ cells. Only the second case can be of interest. The first case involves a complexity that, for a large N , is impossible to control, and we are only interested in the case in which N is as large as possible.

Digital systems theory introduces, for the case we are interested in, the concept of *cellular automaton*.



Figure 14.2: One-dimensional cellular automaton.

The theoretical concept of a cellular automaton is represented in Figure 14.2 in its simplest form. It consists of an unlimited number of *identical linearly connected cells*. Each cell is a automaton. All cells switch synchronously according to their internal states and to the signals received from the left cell and the right cell.

Example 14.1 *Let us consider the simplest cellular automaton described by the following modules:*

```

/*****
File names:      cellAut.sv
Circuit name:    One-bit cell cellular automaton
Description:      The cell is a two-state, two one-bit inputs finite
                  automaton
*****/
`define n 1024
module cellAut
(
    output logic ['n-1:0] out ,
    input  logic [7:0] func ,
    input  logic ['n-1:0] init ,
    input  logic      rst , clk );
    genvar i;
    generate for (i=0; i<'n; i=i+1) begin: C
        eCell eCell(.out      (out[i]
                        .func    (func
                        .init     (init[i]
                        .in0      ((i==0) ? out['n-1] : out[i-1] ),
                        .in1      ((i==(n-1)) ? out[0] : out[i+1]),
                        .rst      (rst
                        .clk      (clk
                                ));
    end
    endgenerate
endmodule

```

```

/*****
File names:      eCell.sv
Circuit name:    One-bit cell for the cellular automaton cellAut.sv
Description:      The cell of the cellular automaton cellAut.sv is a
                  two-state, two one-bit inputs finite automaton defined
                  in the most general form using a 8-bit word applied on
                  the selected inputs of a multiplexor
*****/
module eCell(
    output logic out ,
    input  logic [7:0] func,
    input  logic init, in0, in1, rst, clk );
    always_ff @(posedge clk) if (rst) out <= init ;
    else out <= func[{in1, out, in0}];
endmodule

```

The size of the definition is constant, which means it does not depend by the number of cells n , i.e., the cellular automaton is a simple circuit.

◇

14.2 Controlled Cellular Automaton

14.2.1 Structure

The theoretical model of the cellular automaton is not practically usable to perform computation efficiently. Real problems cannot be solved efficiently with a cellular automaton. It is necessary to move from the theoretical model to an abstract model of a parallel computing system. We will do this in several steps. The first is to define a cellular automaton in which the space required for computation can be balanced with the time required for computation. The space-time trade-off is usually achieved by a suitable sequencing of the computation steps.

The theoretical model of a cellular automaton assumes a vector whose components are the states of the N cells. In each cycle, all components of the vector are involved in the computation. Two difficulties arise in such a theoretical model of computation:

- real problems cannot be effectively reduced to such a completely unstructured mechanism
- the size of the physical structure involved becomes inefficiently large.

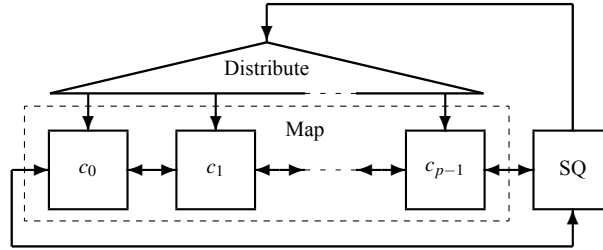


Figure 14.3: Controlled Cellular Automaton (CCA).

The solution that allows the increasing of the flexibility of the computing process and the reducing of the size of the hardware structure involves:

- structuring the state space of the automaton in each cell by replacing the state register, Q , with a register file where the state has a structure of a Cartesian product $Q = \{Q_0 \times Q_1 \times \dots \times Q_{m-1}\}$ (see the RALU circuit in subsection 10.3.3)
- adding a controller, SQ, that sequences the evolution of the internal state in each cell.

Results the system represented in Figure 14.3 where:

- SQ is a controller (processor + data memory + program memory) which issues in each clock cycle a command which the same command for each cell
- Distribute is a $\log$ -depth network (see Figure 14.4) used to send to each cell, with a $\log$ -latency,

the command issued by SQ; it is necessary because a connection from SQ to **many** cells distributed on a **big** surface is electrically impossible at a high frequency of the clock signal, reasons why we use a distribution network that is loaded at each node with only two pipeline registers connected at a reasonably large distance.

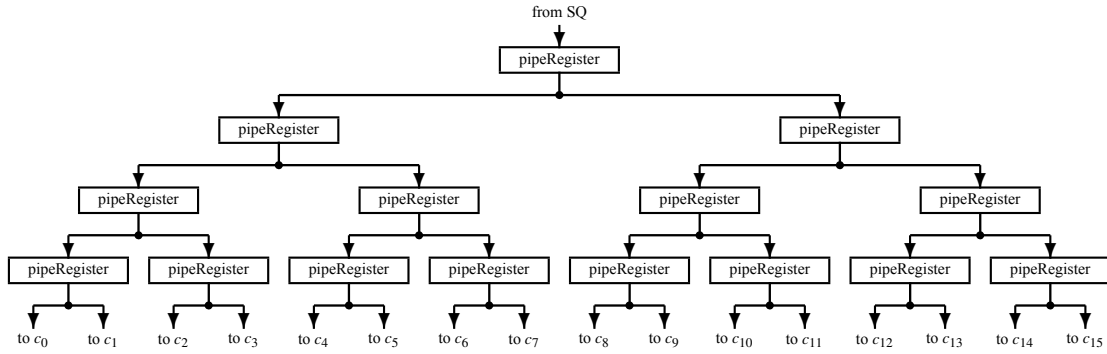


Figure 14.4: Distribution network for a circuit with $p = 16$.

- $c_0, c_1, \dots, c_{p-1}$ which form a cellular automaton, Map, with a finite number of cells, each equipped with a RALU type execution unit.

14.2.2 Mapping

Mapping an operation involves applying a function to each element in a collection (e.g., a list, array). This can be achieved using constructs like the `map(...)` function in many programming languages.

Example 14.2 *If you have a vector of numbers $[1, 2, 3, 4]$ and want to double each number, the code in Python is:*

```

/*****
File name:      vectorDouble.py
Description:    The operation: multiply by 2
                The mapping: [1, 2, 3, 4] → [2, 4, 6, 8]
*****/
numbers = [1, 2, 3, 4]
doubled = map(lambda x: x * 2, numbers)
print(list(doubled)) # Output: [2, 4, 6, 8]

```

The operation multiply, $$, is mapped, in the previous Python program, to the vector $[1, 2, 3, 4]$.*

◇

Our CCA contains an array of cells which can be named Map because it allows applying the same function to the elements of a vector distributed along the cells it is made of.

14.2.3 Architecture

The command issued by SQ is organized in the form of an instruction that contains logical-arithmetic operations, addressing modes of local memory resources in each cell, addresses and constants. The cells are serially connected to the SQ for data transfer between the SQ's memory and the local memory resource distributed in the cells. If in a cellular automaton its global state is represented by an unlimited vector, in a controlled cellular automaton the internal state is represented by the matrix:

$$\mathbf{M}_{mp} = \begin{bmatrix} s_{00} & s_{01} & \dots & s_{0\ p-1} \\ s_{10} & s_{11} & \dots & s_{1\ p-1} \\ \vdots & \vdots & \ddots & \vdots \\ s_{m-1\ 0} & s_{m-1\ 1} & \dots & s_{m-1\ p-1} \end{bmatrix}$$

whose columns represent the local memory (register file) in each cell. SQ issues operations applied to the line of the matrix $\mathbf{M}$ seen as a vector distributed along cells

$$V_i = [s_{i0} \ s_{i1} \ \dots \ s_{i(p-1)}]$$

for $i = 0, \dots, m-1$ so that an operation of the form:

$$V_i \Leftarrow V_j + V_k$$

which involves p additions is executed in constant time, i.e., is executed in *parallel*.

If a cellular automaton with n cells, with state

$$Q = [s_0, s_1, \dots, s_{n-1}]$$

modifies its internal state in constant time, belonging to $O(1)$, a controlled cellular automaton, with the state defined by the matrix M_{mp} , for $m \times p = n$, performs the transformation of its internal state in a time belonging to $O(m)$. This is how the space/time relationship can be negotiated to obtain a computationally efficient structure. The memory gap problem (see subsection 12.4.1) can find a partial solution through a good negotiation between m and p in the controlled cellular automaton structure.

The state of SQ is represented by an array of scalar values: $\text{MSQ} = [v_0 \ v_1 \ \dots]$, i.e., the data memory of the controller SQ.

There are the following subsets of operations performed, under the command of SQ, in the Map section.

14.2.3.1 Spatial selection

The selection of a subset of cells for predicate execution is facilitated, but not exclusively, by a constant vector containing cell indexes, as follows: $IX = \langle 0 \ 1 \ \dots \ p-1 \rangle$

Instead of the conditional jump operations performed in a single-core processing structure, in the many-core structure represented by the Map array, the conditional operations are performed by a selection of active cells according to a condition that is tested in each cell. The conditional selection allows for predicated execution by modifying the contents of the Boolean vector $B = \langle b_0 \ b_1 \ \dots \ b_{p-1} \rangle$ which indicate the state of each cell: *if* ($b_i == 1$) the cell is active, *else* it is inactive. A minimal set used to control the vector B is:

ACTIVATE: activate all cells

$$b_i \Leftarrow 1 \text{ for } i = 0, \dots, p-1$$

WHERE(*cond*): keep active only the active cells where the condition *cond* is fulfilled

$$b_i \Leftarrow (b_i \& \textit{cond}) ? b_i : 0, \text{ for } i = 0, \dots, p-1$$

ENDWHERE: the effect on *B* of the acting WHERE is restored

14.2.3.2 Arithmetic & Logic vector operations

Arithmetic & Logic vector operations are performed on active components of vectors belonging to the active cells where $b_i = 1$. Generic forms:

BOP(*Vl*, *Vr*): defines a binary operation *mapped* on vectors distributed along the cells of the Map section $b_i ? bOp(s_{li}, s_{ri})$, for $i = 0, \dots, p-1$;

Example of use:

$V2 \Leftarrow \text{ADD}(V0, V5)$: in each active cell, $b_i = 1$, the components of the vectors *V0* and *V5* are summed and stored as corresponding components in vector *V2*;

For:

$$B = [1 \ 1 \ 1 \ 0 \ 0 \ 1 \ 0 \ 0]$$

$$V0 = [2 \ 3 \ 1 \ 4 \ 3 \ 5 \ 2 \ 0]$$

$$V5 = [3 \ 5 \ 4 \ 1 \ 2 \ 2 \ 6 \ 8]$$

$$V2 = [9 \ 8 \ 9 \ 8 \ 9 \ 8 \ 9 \ 8]$$

applying $V2 \Leftarrow \text{ADD}(V0, V5)$ results:

$$V2 = [5 \ 8 \ 5 \ 8 \ 9 \ 7 \ 9 \ 8]$$

UOP(*Vl*): defines a unary operation

$$b_i ? uOp(s_{li}), \text{ for } i = 0, \dots, p-1;$$

Example of use: $V0 \Leftarrow \text{INC}(V1)$ means increment in active cells the component of *V1* and send the result in *V0*

For:

$$B = [1 \ 1 \ 1 \ 0 \ 0 \ 1 \ 0 \ 0]$$

$$V1 = [2 \ 3 \ 1 \ 4 \ 3 \ 5 \ 2 \ 0]$$

$$V0 = [8 \ 8 \ 8 \ 8 \ 8 \ 8 \ 8 \ 8]$$

applying $\{\texttt{V0} \Leftarrow \text{INC}(V1)\}$ results:

$$V0 = [3 \ 4 \ 2 \ 8 \ 8 \ 6 \ 8 \ 8]$$

MAPOP(*Va*, *Vb*, *Vc*, ...): sequence of BOPs and UOPs on vectors *Va*, *Vb*, *Vc*, ... in Map section.

Example of use: $V3 \Leftarrow \text{MAPOP}(V0, V2, V5) = \text{MULT}(\text{ADD}(V2, V5), V0)$

For:

$$B = [1 \ 1 \ 1 \ 0 \ 0 \ 1 \ 0 \ 0]$$

$$V0 = [2 \ 3 \ 1 \ 4 \ 3 \ 5 \ 2 \ 0]$$

$$V5 = [3 \ 5 \ 4 \ 1 \ 2 \ 2 \ 6 \ 8]$$

$$V2 = [9 \ 8 \ 9 \ 8 \ 9 \ 8 \ 9 \ 8]$$

```

V3 = [0 0 0 0 0 0 0 0]
applying V3 <= MAPOP(V0,V2,V5) results:
V3 = [24 39 13 0 0 50 0 0]

```

14.2.3.3 Global shift/rotate operations

Global shift/rotate operations are performed on all components of vectors distributed along the cells in Map section ignoring the content of the Boolean vector B. For example:

```

SHIFTRIGHT(V1,leftIn): :
  (i = 0) ? leftIn : sl(i-1), for i = 0, ..., p - 1
Example of use: V0 <= SHIFTRIGHT(V1,23)

```

```

For:
  B = [1 1 1 0 0 1 0 0]
  V1 = [2 3 1 4 3 5 2 0]
results:
  V0 = [23 2 3 1 4 3 5 2]

```

```

ROTATERIGHT(V1): :
  (i = 0) ? sp-1 : sl(i-1), for i = 0, ..., p - 1
Example of use: V0 <= ROTATERIGHT(V1)

```

```

For:
  B = [1 1 1 0 0 1 0 0]
  V1 = [2 3 1 4 3 5 2 0]
results:
  V0 = [0 2 3 1 4 3 5 2]

```

14.3 First Global Loop in CA

14.3.1 Structure

Just as we did in the case of the structural growth process, by successively closing reaction loops, until reaching the level required by mono-core computing, we will also do in the case of controlled cellular automation by successively closing global loops.

By a global loop we mean a circuit that takes a scalar from each cell of a controlled cellular automaton, thus forming a vector. The result of processing this vector is reapplied to the cells, thus closing a loop.

The first global loop that we close is the *reduction* type, which reduces the vector received from the controlled cellular automaton to a scalar. In Figure 14.5 the resulting structure, called MapReduced, is represented. It consists of:

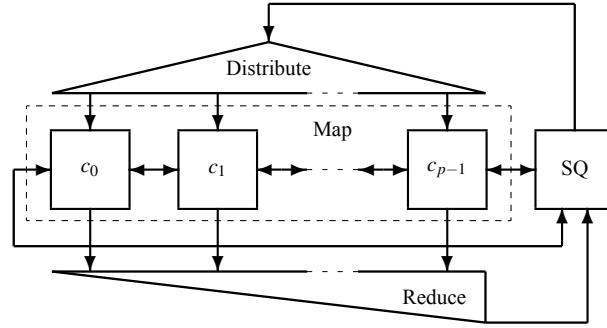


Figure 14.5: The **MapReduce** structure obtained by closing the first global loop over a controlled cellular automaton.

- MAP: a cellular automaton with p components whose cells are execution units equipped, mainly, with an arithmetic-logic unit and a register file with m locations
- SQ: a controller that issues commands to the Map network and can receive scalars from the Reduce network that closes the first global loop
- Distribute: a pipelined $\log$ -depth network used to distribute the instruction issued by the controller SQ
- Reduce: a pipelined $\log$ -depth network (see Figure 14.6 which reduce the vector received from Map to a scalar sent to the register accumulator in SQ, acc , with a $\log$ latency

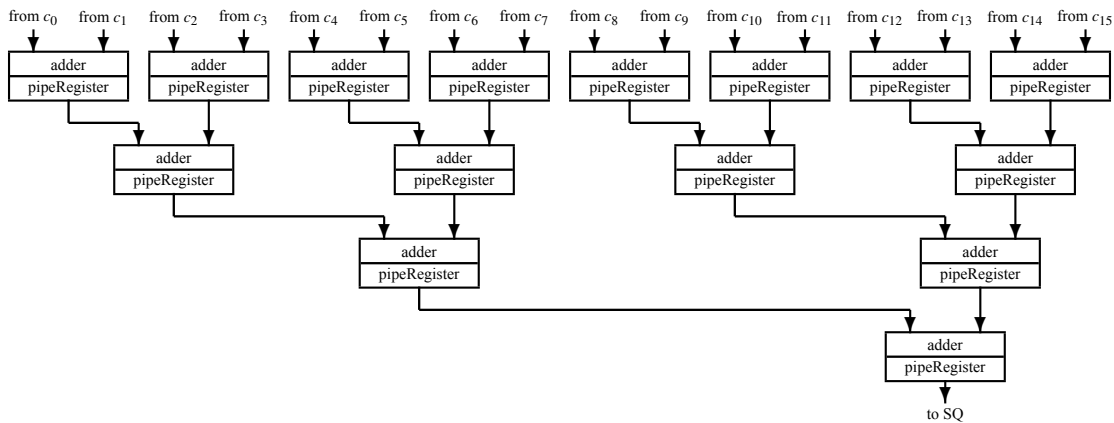


Figure 14.6: Reduce network for $p = 16$, with the size in $O(p)$ and the depth in $O(\log p)$ exemplified for the add function.

The hardware structure of the Reduce network is a binary tree, so it contains $p/2 + p/4 + \dots + 1$ modules. For p a power of 2, the usual case, the network contains $p - 1$ modules. So the size of the

MapReduce system is in $O(p)$.

14.3.2 Architectural additions

The system architecture is completed by adding the first global loop, with the following specific functions:

ADDRED(Vi): reduction add

$$\sum_{j=0}^{p-1} (b_j ? s_{ij} : 0)$$

Example of use: $v0 \leq \text{ADDRED}(V1)$

For:

B = [1 1 1 0 0 1 0 0]

V1 = [2 3 1 4 3 5 2 0]

results:

v0 = 11

MINRED(Vi): reduction minimum

$$\text{MIN}_{j=0}^{p-1} (b_j ? s_{ij} : \infty)$$

Example of use: $v0 \leq \text{MINRED}(V1)$

For:

B = [1 1 1 0 0 1 0 0]

V1 = [2 3 1 4 3 5 2 0]

results:

v0 = 1

MAXRED(Vi): reduction maximum

$$\text{MAX}_{j=0}^{p-1} (b_j ? s_{ij} : 0)$$

Example of use: $v0 \leq \text{MAXRED}(V1)$

For:

B = [1 1 1 0 0 1 0 0]

V1 = [2 3 1 4 3 5 2 0]

results:

v0 = 5

ORRED(Vi): reduction or

$$\text{OR}_{j=0}^{p-1} (b_j ? s_{ij} : 0)$$

Example of use: $v0 \leq \text{ORRED}(V1)$

```

For:
  B  = [1 1 1 0 0 1 0 0]
  V1 = [2 3 1 9 3 5 2 0]
results:
  v0 = 7

```

Example 14.3 To calculate the scalar product of two vectors, V_i and V_j , in such a system, the following is performed, under the SQ command:

```
v1 <= ADDRED(MULT(Vi,Vj))
```

where the vector product, $V_i \times V_j$, is performed in Map in constant time and is transmitted to the Reduce network which performs the sum in time $\log_2 p$ and sends it to SQ in v1. Thus, $2p - 1$ operations – p multiplications and $p - 1$ additions – are performed in $1 + \log_2 p$ clock cycles.

◇

In the previous example, $2p - 1$ computational results – p multipliers from Map and $p - 1$ adders from Reduce – are used in $1 + \log_2 p$ cycles to perform $2p - 1$ operations. This results, for $p = 1024$, in an average of 186 operations/cycle. So, we must be careful! We need algorithms that allow for the most intensive use of hardware resources (Antonescu et al., 2023).

14.4 Second Global Loop in CA

14.4.1 Structure

The second global loop is a scan loop that takes a vector from Map, processes it, and returns the resulting vector back to Map. Figure 14.7 shows, MapScanReduce structure, the structure that results from introducing a scan loop into a MapReduce system. In current implementations, the reduction network can be included in the scan network (see Figure 14.8).

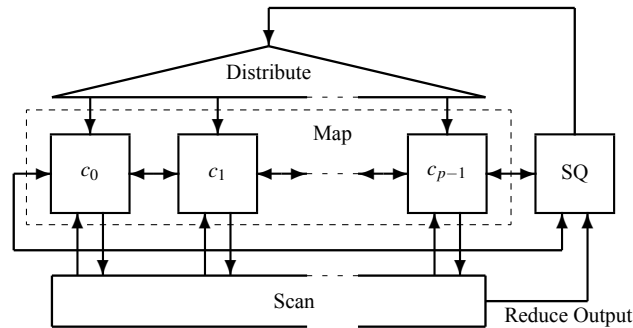


Figure 14.7: MapScanReduce structure obtained by closing the second global loop over a controlled cellular automaton.

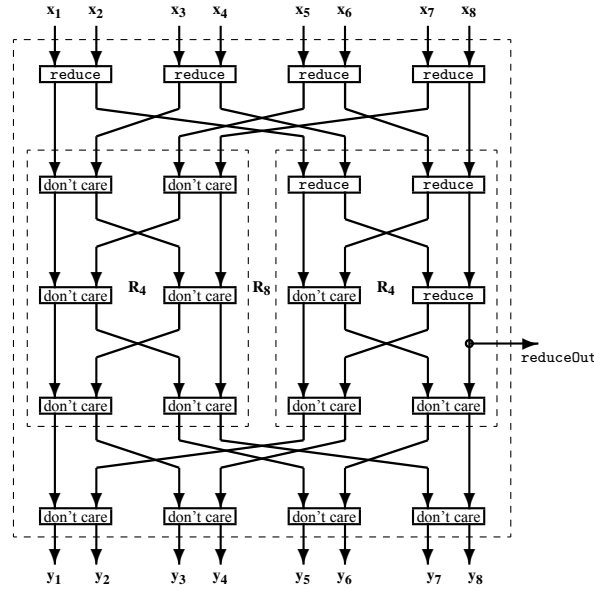


Figure 14.8: An 8-input scan circuit adapted for an 8-input prefix operation (Antonescu and Stefan, 2020) (Antonescu and Stefan, 2024).

We use the term *scan* by extension, starting from the already established use for prefix-type networks.

Definition 14.1 Let OP be an associative and commutative operation. *OP-prefix computation* is defined on the scalar vectors, $[s_0 \ s_1 \ \dots \ s_{n-1}]$ and returns the vector $[p_0 \ p_1 \ \dots \ p_{n-1}]$, where:

$$p_i = OP_0^i s_i$$

◇

Example 14.4 For *OP* arithmetic sum we have *addPrefix*:

$$p_0 = s_0$$

$$p_1 = p_0 + s_1 = s_0 + s_1$$

$$p_2 = p_1 + s_2 = s_0 + s_1 + s_2$$

...

$$p_{n-1} = p_{n-2} + s_{n-1} = s_0 + s_1 + \dots + s_{n-1}$$

◇

Note: the last output of *addPrefix* is the output of *sumReduce*, thus justifying the simplified representation of the two loops in Figure 14.7.

The size of the pipeline implemented Scan network is in $O(p \times \log p)$, and the depth is in $O(\log p)$. Therefore, the size of the MapScanReduce structure is in $O(p \times \log p)$.

14.4.2 Architectural additions

The system architecture is completed by adding the second global loop, with specific functions exemplified as follows:

ADDPX(Vi): add prefix returns a vector whose components are:

$$addPx_j \Leftarrow \sum_{i=0}^j (b_i ? s_{ij} : 0) \text{ for } j = 0, \dots, p-1$$

Example of use: $V2 \Leftarrow \text{ADDPX}(V1)$

For:

$$B = [1 \ 1 \ 1 \ 0 \ 0 \ 1 \ 0 \ 0]$$

$$V1 = [2 \ 3 \ 1 \ 4 \ 3 \ 5 \ 2 \ 0]$$

results:

$$V2 = [2 \ 5 \ 6 \ 6 \ 6 \ 11 \ 11 \ 11]$$

SPLIT(Vi): return a vector with the selected components left aligned and the unselected component right aligned

Example of use: $V2 \Leftarrow \text{SPLIT}(V1)$

For:

$$B = [0 \ 1 \ 1 \ 0 \ 0 \ 1 \ 0 \ 0]$$

$$V1 = [2 \ 3 \ 1 \ 4 \ 3 \ 5 \ 2 \ 0]$$

results:

$$V2 = [3 \ 1 \ 5 \ 2 \ 4 \ 3 \ 2 \ 0]$$

14.5 The mathematical model of partially recursive functions

When we closed the fourth loop, starting from combinational circuits, we had the pleasant encounter with the mathematical model of the Turing Machine. Does a similar encounter await us with the closing of the two loops over a cellular automaton? Yes. It is a mathematical model proposed independently in the same year as the Turing Machine: the model of *partially recursive functions* proposed by Stephen Kleene (Kleene, 1936).

Definition 14.2 Let be the positive integers $x, y, i \in \mathbb{N}$ and the vector $\vec{x} = \langle x_0, x_1, \dots, x_{n-1} \rangle \in \mathbb{N}^n$. Any partial recursive function $f : \mathbb{N}^n \rightarrow \mathbb{N}$ can be computed using three **initial functions**:

- $\text{ZERO}(x) = 0$: the variable x takes the value **zero**
- $\text{INC}(x) = x + 1$: **increments** the variable $x \in \mathbb{N}$
- $\text{SEL}(i, \vec{x}) = x_i$: i **selects** the value of x_i from the vector of positive integers $\vec{x}$

and the application of the following three **rules**:

- **Composition:** $f(\vec{x}) = g(h_0(\vec{x}), \dots, h_{p-1}(\vec{x}))$, where: $f : \mathbb{N}^n \rightarrow \mathbb{N}$ is a total function if $g : \mathbb{N}^p \rightarrow \mathbb{N}$ and $h_i : \mathbb{N}^n \rightarrow \mathbb{N}$, for $i = 0, 1, \dots, p-1$, are total functions

- **Primitive recursion:** $f(\vec{x}, y) = g(\vec{x}, f(\vec{x}, (y - 1)))$, with $f(\vec{x}, 0) = h(\vec{x})$ where: $f : \mathbb{N}^{n+1} \rightarrow \mathbb{N}$ is a total function if $g : \mathbb{N}^{n+1} \rightarrow \mathbb{N}$ and $h : \mathbb{N}^n \rightarrow \mathbb{N}$ are total functions.
- **Minimization:** $f(\vec{x}) = \mu_y[g(\vec{x}, y) = 0]$, which means: the value of the function $f : \mathbb{N}^n \rightarrow \mathbb{N}$ is the smallest y , **if any**, for which the function $g : \mathbb{N}^{n+1} \rightarrow \mathbb{N}$ takes the value $g(\vec{x}, y) = 0$.

◇

If we follow the previous definition, we will identify the initial functions with those directly realized by simple combinational circuits:

- ZERO: implemented by generating a constant value
- INC: implemented as the increment circuit
- SEL: implemented as the multiplexer circuit

while the composition rule is implemented by the circuit represented in Figure 14.9.

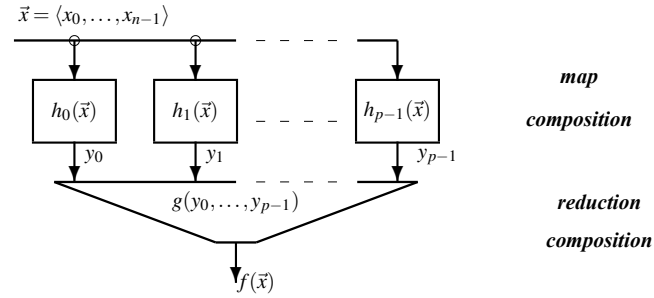


Figure 14.9: **The circuit version of composition.** It is a two-layer construct: the parallel expanded *map* layer serially connected with the *log-dept reduction* layer.

The similarity between Figures 14.5 and 14.9 encourages us to hope that both primitive recursion and minimization can find implementations in structures that we can physically realize with cellular constructions. Indeed, by composing *map composition* functions (see Figure 14.9) one can obtain the purely theoretical construction in Figure 14.10 (for details see (Stefan, 2000)). It is a theoretical one because is extended indefinitely to right for two reasons:

- 1) we not know for how big y a primitive recursion computation is requested
- 2) we not know for how big y , if any, the computation of a partial recursive function ends.

While for the primitive recursion functions the value of y is known at the beginning of computation, for the partial recursive functions the value of y , if any, is the result of the computation. For computing primitive recursive functions only the map composition with the reduction network are sufficient, while for partial functions adding the scan network is mandatory. The simplest form of the reduction network is one which provide the Boolean bitwise OR of the input components. For the scan network the minimal function receives a Boolean vector and provides the Boolean vector FIRST which highlights only the first 1.

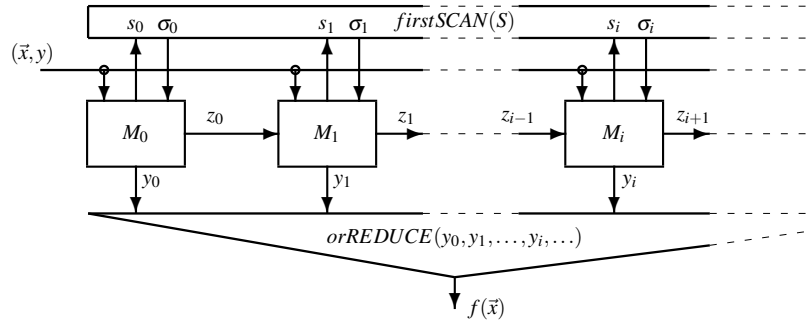


Figure 14.10: **The circuit version of the enhanced composition.** In the map composition right connections are added considering specific map compositions. The scan circuit, a *log*-layer map compositions, is connected with the generic map composition.

The theoretical structure in Figure 14.10 can be transformed into an implementable one by adding a sequencer that converts the size of the structure into execution time. This becomes possible due to the local memories in each cell M_i . The result is a structure like the one in Figure 14.7, a structure that can be considered an *abstract model for a parallel computing system*.

14.6 Heterogeneous Computing

The main application of the MapScanReduce structure is in heterogeneous computing where the distinction between complex computing and intense computing is meaningful.

14.6.1 Complexity vs. intensity in computation

let's define the two types of computations. In this respect, we use the *algorithmic complexity* (Chaitin, 1970) which helps us make the necessary distinction between size and complexity.

Definition 14.3 The *algorithmic complexity* of an entity is given by the length of its shortest description.

◇

Definition 14.4 An entity is *complex* when its size is in the same magnitude order with its algorithmic complexity, and is *simple* when with a size in $O(f(N))$ its complexity remains in $O(1)$.

◇

Now, regarding computation the following two definitions can be accepted.

Definition 14.5 Complex computation is characterized by an execution time proportional to the number of lines of executable code by which it is described.

◇

Definition 14.6 The intensive, simple computation is described by a constant number, in $O(1)$, of lines of executable code, independent of the execution time which is in $O(f(N))$, where N is used to define the size of the preprocessed data.

◇

The previous definitions follow the algorithmic complexity (Chaitin, 1970) definition stating mainly that the complexity of a computation is given by the length of its shortest description. While for complex computation the Turing tax can be reduced using *instruction-level parallelism*, for simple, intense computation a promising solution is *cellular parallelism*. A typical example for intense, simple computation is the multiplication of $N \times N$ matrices performed on a mono-core engine in $O(N^3)$ time. An accelerator with p execution units (cells) is expected to reduce execution times by at least $O(p/\log p)$ times. It runs a constant (independent of N) code size.

14.6.2 Heterogeneous system

Heterogeneous computing takes into account the distinction between complex and intensive computing. This segregation is possible using a system in which a single-core computer, instantiated as a HOST, uses a library of functions implemented in hardware in a MapScanReduce structure to perform complex calculations. In Figure 14.11, such a heterogeneous computing system is represented.

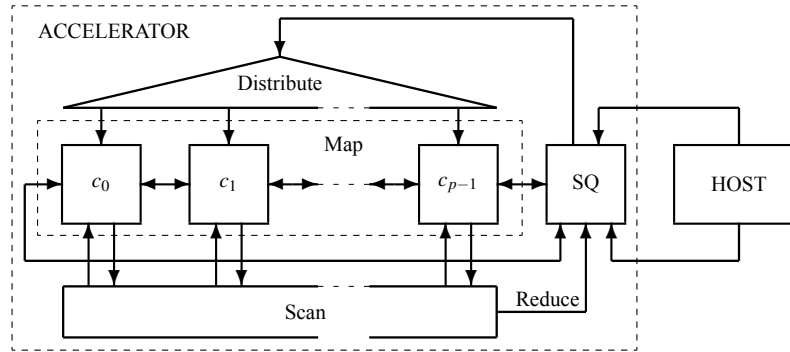


Figure 14.11:

The two types of computation, complex and intensive, can be performed efficiently if we have adequate hardware for each. This is what we achieve with a heterogeneous system. The problem is the acceleration that we obtain for an application in which we have a mixture of intensive and complex computation. Evaluating the performance of the accelerator is not enough because what is important is the total performance that is obtained by taking into account the execution time associated with the complex computation, which cannot be accelerated in the host, and the execution time associated with the intensive computation subject to acceleration in the accelerator.

Since the 1960s, with the advent of the first parallel computers, the problem of limited acceleration due to the weight of complex calculations has arisen. The limitation was expressed by Amdahl's Law.

Amdahl's law, named after Gene Amdahl, was introduced at a conference in 1967 (Amdahl, 1967), and can be used in parallel computing to predict the limit of the theoretical speedup when using heterogeneous computation. It is formulated in the following way:

$$A_{host+accelerator} = \frac{1}{(1 - w_{intenseComp}) + \frac{w_{accelerator}}{A_{accelerator}}}$$

where:

$A_{host+accelerator}$: represents the overall acceleration of the heterogeneous system running an application

$A_{accelerator}$: represents the acceleration obtained by the accelerator running the intense part of the application

$w_{intenseComp}$: the weight of the intense computation in the application expressed by a subunit number

We can highlight two limiting cases:

- 1) computation is dominated by the complex computation: with $w_{intenseComp} = 0.05$ and $A_{accelerator} = 1000$ results $A_{host+accelerator} = 1.05$
- 2) computation is dominated by the intense computation: $w_{intenseComp} = 0.95$ and $A_{accelerator} = 1000$ results $A_{host+accelerator} = 19.62$

In the limiting cases, for $w_{accelerator} = 1$ we have $A_{host+accelerator} = A_{accelerator}$, while for $w_{accelerator} = 0$ we have $A_{host+accelerator} = 1$.

14.6.3 Programming a heterogeneous system

Concerning the programming of multicellular processors, which have seen a rapid proliferation primarily driven by corporate considerations, the scientific community has yet to universally embrace effective solutions. The challenge arises from the prioritization of hardware criteria that promoted multi- and many-cell solutions without sufficient regard for their programmability. This is due to:

"the semiconductor industry threw the equivalent of a Hail Mary pass when it switched from making microprocessors run faster to putting more of them on a chip—doing so without any clear notion of how such devices would in general be programmed. The hope is that someone will be able to figure out how to do that, but at the moment, the ball is still in the air." (Patterson, 2010)

It appears that the solution does not entail specific languages tailored for parallel structures:

"The software span connecting applications to hardware relies more on parallel software architectures than on parallel programming languages. Instead of traditional optimizing compilers, we depend on autotuners, using a combination of empirical search and performance modeling to create highly optimized libraries tailored to specific machines." (Asanovic et al., 2009)

As a result, our heterogeneous approach imposes, for efficiency, a three level programming system:

- 1) ACCELERATOR-level programming is done in assembly language providing a highly optimized library of primitives.
- 2) HOST-level libraries developed in a high-level programming language based on primitives provided by ACCELERATOR.
- 3) HOST-level programming running in a high-level programming language based on "highly optimized libraries tailored to specific machines".

14.6.3.1 Accelerator-level programming

In Table 14.1 is listed partially the set of operations performed at the accelerator-level.

Table 14.1: Accelerator-level operations

Operation	Description
ACTIVATE	activate all cells: $b_i = 1$ for $i = 0, 1, \dots, p1 -$
WHERE($V_i=0$)	where scalar in V_i is zero, cell remains active
ELSEWHERE	cells inactivated by the last WHERE become active
ENDWHERE	vector B takes the value before the most recent WHERE
ADD(V_j, V_k)	add vectors V_j and V_k in each active position
MULT(V_j, V_k)	multiply vectors V_j and V_k in each active position
AND(V_j, V_k)	bitwise AND of vectors V_j and V_k in each active position
OR(V_j, V_k)	bitwise OR of vectors V_j and V_k in each active position
XOR(V_j, V_k)	bitwise XOR of vectors V_j and V_k in each active position
VAL(s)	generate a vector with value s in each active position
ADDRED(V_i)	active components of V_i are submitted to reduction add
MINRED(V_i)	active components of V_i are submitted to reduction minimum
MAXRED(V_i)	active components of V_i are submitted to reduction maximum
ORRED(V_i)	active components of V_i are submitted to reduction logic OR
ADDPX(V_i)	active components of V_i are submitted to prefix add function
SHIFTRIGHT(V_i , leftIn)	shift right the component of V_i and insert leftIn
SHIFTLEFT(V_i , rightIn)	shift left the component of V_i and insert rightIn
ROTATERIGHT(V_i)	rotate right the component of V_i
ROTATELEFT(V_i)	shift left the component of V_i

14.6.3.2 Library of primitives: an example

One of the most used library of function contains linear algebra functions. In the following few of the most used linear algebra functions are described to exemplify the intense functions appropriate to implemented on a MapScanReduce architecture.

Identity matrix is a matrix working as the scalar 1 in the arithmetic multiplication and division; it has 1s on the main diagonal as follows:

$$I = \begin{bmatrix} 1 & 0 & 0 & \dots & 0 \\ 0 & 1 & 0 & \dots & 0 \\ \dots & \dots & \dots & \dots & 0 \\ 0 & 0 & 0 & \dots & 1 \end{bmatrix}$$

Inner (dot or scalar) product of two vector provides a scalar as the sum of matching members products.

Example 14.5 Let be the vectors $V1 = [1 \ 2 \ 3 \ 4]$ and $V2 = [5 \ 6 \ 7 \ 8]$. Their inner, dot or scalar product is:

$$V1 \cdot V2 = 1 \times 5 + 2 \times 6 + 3 \times 7 + 4 \times 8 = 70$$

◇

Matrix-vector multiply provide a vector of the dot products between the lines of the matrix and the vector.

Example 14.6 Let be the matrix

$$M = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

and the vector

$$V = \begin{bmatrix} 10 \\ 11 \\ 12 \end{bmatrix}$$

their product is:

$$M \times V = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix} \times \begin{bmatrix} 10 \\ 11 \\ 12 \end{bmatrix} = \begin{bmatrix} 1 \times 10 + 2 \times 11 + 3 \times 12 \\ 4 \times 10 + 5 \times 11 + 6 \times 12 \\ 7 \times 10 + 8 \times 11 + 9 \times 12 \end{bmatrix} = \begin{bmatrix} 68 \\ 167 \\ 326 \end{bmatrix}$$

◇

Matrix multiplication of two matrices M1 and M2 provides a matrix M whose lines a contains the dot products of a column, starting with the first column, form M2with each lines from M1 starting with the first line in M1. The number of lines in M1 must be the same with the number of columns in M2.

Example 14.7 Let be two matrices:

$$M1 = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$$

and

$$M2 = \begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{bmatrix}$$

Their product is computed as follow:

$$M = M1 \times M2 = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} \times \begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{bmatrix} = \begin{bmatrix} 1 \times 1 + 2 \times 3 + 3 \times 5 & 1 \times 2 + 2 \times 4 + 3 \times 6 \\ 4 \times 1 + 5 \times 3 + 6 \times 5 & 4 \times 2 + 5 \times 4 + 6 \times 6 \end{bmatrix} = \begin{bmatrix} 22 & 28 \\ 49 & 64 \end{bmatrix}$$

◇

Matrix transpose applied to the matrix M is M^T with the lines equal with the columns of M .

Example 14.8 Let be the matrix:

$$M = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

Its transpose is:

$$M^T = \begin{bmatrix} 1 & 4 & 7 \\ 2 & 5 & 8 \\ 3 & 6 & 9 \end{bmatrix}$$

◇

Let us consider the simplest, most intense computation of multiplying matrices.

```

/*****
File name:      matrix_multiplication.py
Description:
*****/
def matrix_multiplication(A, B):
    # Get the number of rows and columns for matrices A and B
    rows_A = len(A)
    cols_A = len(A[0])
    rows_B = len(B)
    cols_B = len(B[0])

    # Check if multiplication is possible
    if cols_A != rows_B:
        raise ValueError("Number of columns in A must equal of rows in B")

    # Initialize the result matrix with zeros
    result = [[0 for _ in range(cols_B)] for _ in range(rows_A)]

    # Perform the matrix multiplication
    for i in range(rows_A):
        for j in range(cols_B):
            for k in range(cols_A):
                result[i][j] += A[i][k] * B[k][j]

    return result

```

The execution time of this program on a mono-core processor is proportional with the number of clock cycles:

$$\text{range}(\text{cols_B}) \times \text{range}(\text{cols_B}) \times \text{range}(\text{cols_A})$$

which means, for example: if $\text{range}(\text{cols_B}) = \text{range}(\text{cols_B}) = \text{range}(\text{cols_A}) = 1024$, the execution time is proportional with $1024^3 \simeq 10^9$.

However, in high-level languages like Python there are function libraries for frequently used functions like matrix multiplication. In this case the program takes the simplified form below:

```

/*****
File name:      matrix_multiplication.py
Description:
*****/
# ...
import numpy as np

A = np.array(...)
B = np.array(...)

```

```
# Perform matrix multiplication
result = np.dot(A, B)

print("Resultant Matrix:")
print(result)
```

The line `result = np.dot(A,B)` access, when the program runs on a mono-core processor, a highly optimized program (usually written in assembly language) for the matrix multiplication operation. In a heterogeneous system, instead of calling a program from library, is accessed a function performed hardware accelerated by the associated accelerator.

Example 14.9 Let see how looks the algorithm for multiplying two $n \times n$ matrices, A and B. The transposed matrix of B is T. Each matrix is represented by vectors in a Map with $p \geq n$.

```
/******
File name:      matrix_multiplication
Description:
*****/
A = [V[0] V[1] ... V[n-1]]
T = [V[n] V[n+1] ... V[2n-1]]

i, j      integers

for (j=0; j<n; j=j+1)
    for (i=0; i<n; i=i+1)
        SHIFTRIGHT(V[2N+J],ADDRED(MULT(V[n+j],V[i])))
```

Because the previous algorithm works with vectors, instead of the three embedded loops in the Python program `matrix_multiplication.py`, in the `matrix_multiplication` algorithm **there are only two embedded loops**. Besides this good news, there is also a bad news: the operation repeated n^2 times is executed in time belonging to $O(\log n)$ cycles. Because the line `result[i][j] += A[i][k] * B[k][j]` in the Python program is executed in constant time, the acceleration provided by the MapScanReduce accelerator is only of $n/\log_2 n$. (A well-designed algorithm can avoid this bad news, but this topic goes beyond the level at which we approach parallelism problems.)

◇

In Table 14.2

Table 14.2: Library of primitives

Function	Description
vLoad(V_i)	load vector V_i from the HOST's memory
vStore(V_i)	store vector V_i to the HOST's memory
mLoad(V_i, s)	load from the HOST's memory the s -line matrix starting with V_i
mStore(V_i, s)	store to the HOST's memory the s -line matrix starting with V_i
vvMult(V_i, V_j, s, V_k)	inner product in V_i between V_j and V_k
mvMult(V_i, V_j, s, V_k)	multiply in V_i the s -line matrix starting with V_j , with vector V_k
mmMult()	
mTrans()	

14.7 Parallel Computing Patterns

Multiplying matrices is a promising example for the capacity to perform in parallel with MapScanReduce accelerator intense computation. It would be interesting to investigate the acceleration capacity of the MapScanReduce framework for a wider range of applications. A good idea could be formed by looking at the computational patterns that have been highlighted in applications designed and run in the last decades. Let us start with the study carried out in (McCool et al., 2012) from which we use Figure 14.12 where 17 parallel computational patterns are presented. We will see that almost all of them can be performed using the accelerator MapScanReduce. We will not succeed in the case of those patterns that do not represent an intensive computation but a complex one.

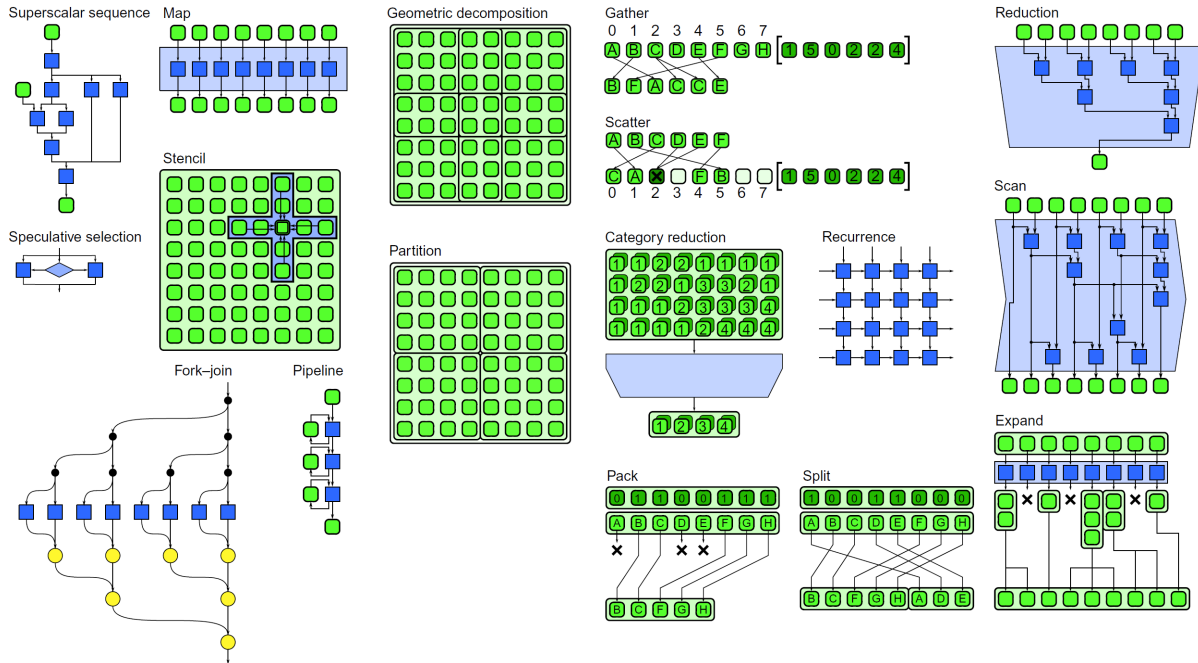


Figure 14.12: Parallel patterns (McCool et al., 2012).

We will analyze in turn how the 17 patterns revealed by McCool and his collaborators can be accelerated using the MapScanReduce structure.

14.7.1 Patterns with Direct Correspondence

A first subset of patterns is the one that is directly implementable with the MapScanReduce accelerator.

14.7.1.1 Map Pattern

From the list proposed by McCool & Co., the Map pattern finds a direct counterpart in the accelerator we are evaluating. In Figure 17.14 the Map pattern is represented using green boxes, which represent data, and blue boxes, which represent hardware structure performing sequences of *bOps* and *uOps*. Overall the array of blue boxes represents a Map structure performing a $\text{MAPOP}(V_a, V_b, V_c, \dots)$ on the vectors stored along the cells of the array. Thus, the Map pattern is has a direct correspondent in we can do with arithmetic & logic vector operations.

Example 14.10 *Let be an 8-cell Map with*

```
V10 = <x x x x x x x x>
V12 = <2 4 7 1 8 4 5 2>
V15 = <4 2 2 8 5 1 3 6>
```

and we have to generate in the even indexed cells of V10 the sum of the even indexed cell from V12 and V15 and in odd cells of V10 the difference between the odd cells of V12 and V15. The sequence of operations is:

```
ACTIVATE          | B = <1 1 1 1 1 1 1 1>: set all cells active
V2 <= VAL(1)      | V2 = <1 1 1 1 1 1 1 1>: load 1 in all positions of V2
V3 <= AND(IX,V2)   | V3 = <0 1 0 ... 1>: bwAND between IX and V2
WHERE(V3==0)      | B = < 1 0 1 ... 0>
V10 <= ADD(V12,V15) | V10 = <6 x 9 x 13 x 8 x>
ELSEWHERE         | B = <0 1 0 ... 1>
V10 <= SUB(V12,V15) | V10 = <6 2 9 -7 13 3 8 -4>
ENDWHERE          | B = <1 1 1 1 1 1 1 1>
```

The execution time of the previous sequence of operations does not depend on the number of cells, i.e., the execution time is in $O(1)$. The acceleration provided by this execution, compared with one performed by a mono-core engine, is in $O(p)$. The degree of parallelism is near 1, i.e., almost in each clock cycle all cells are active.

◇

There are real premises for the acceleration obtained by using a Map array to be in $O(p)$, but more than that there are many cases when the acceleration is **superlinear** compared to a single-core structure, due to the control that is performed by SQ, control that in a single-core implementation is also the responsibility of the processor (Antonescu and Malita, 2024).

14.7.1.2 Reduction Pattern

The reduction pattern is also performed directly by the proposed accelerator due to the Reduce network which is part of its hardware. The usual reduction functions are exemplified in subsection 14.3.2. They are executed directly by the hardware.

14.7.1.3 Scan Pattern

The scan pattern is also performed directly by the proposed accelerator due to the Reduce network which is part of its hardware. In subsection 14.4.2 two functions are exemplified. They are executed directly by the hardware.

14.7.2 Patterns Executable on Map-Only

Many of the patterns we consider are executed exclusively involving the Map section. For them to be executed, a sequence of Map operations is required each time.

14.7.2.1 Stencil Pattern

The stencil pattern (depicted in Figure 14.12) is not directly implemented in the structure of the proposed accelerator, but can be efficiently programmed taking into account the fact that the input data structure is a matrix that can be loaded as part of the $\mathbf{M}_{\text{mp}}$ matrix. The finite neighborhood of each cell can be involved in the calculation of a new value in each cell very efficiently, because the values from the vertical vectors are in the same cell and a global shift operations allows access to the values from the horizontal vector. In fact, any finite neighborhood can be involved in updating any value in $\mathbf{M}_{\text{mp}}$.

For example, let us consider the matrix stored in vectors from V_0 to V_{n-1} . Let us consider a stencil computation in each cell of vector V_i using as operands values from a type of neighborhood emphasized in Figure ??: s_{ij} as a central value, $s_{i(j-1)}, s_{i(j-2)}$ as values from the same vector positioned left, and $s_{i(j+1)}, s_{i(j+2)}$ positioned right, $s_{(i+1)j}, s_{(i+2)j}, s_{(i-1)j}, s_{(i-2)j}$ positioned in the same cell up two positions and down two positions. The values to be used as operands are collected in each cell and then submitted to a pure map sequence of operations which update the horizontal vector V_i as follows:

```

Vn      <= SHIFTRIGHT(Vi, 0)
V(n+1) <= SHIFTRIGHT(Vn, 0)
V(n+2) <= SHHIFTLEFT(Vi, 0)
V(n+3) <= SHHIFTLEFT(V(n+2), 0)
MAPOP(Vi, Vn, V(n+1), V(n+2), V(n+3), ...)

```

Because the update computation is done in parallel for all the components of the horizontal vector V_i , the acceleration of the computation is in $O(p)$.

14.7.2.2 Geometric Decomposition Pattern

The geometric decomposition pattern works on configurable (overlapping) sub-collections of data. In Figure 14.12, four 5×5 scalars sub-collections are defined. If data is loaded in the accelerator's memory as a matrix of scalars, a sub-collection, $S(i, j, k, q)$, is a matrix loaded in $\mathbf{M}_{\text{mp}}$ between two horizontal vectors, V_i and V_j , and two columns (vertical vectors), $C_k = [s_{0k} \ s_{1k} \ \dots \ s_{m-1k}]$ and $C_q = [s_{0q} \ s_{1q} \ \dots \ s_{m-1q}]$. The values for i and j are controlled by the program executed on the sequencer SQ, while the values of k and q are used by the *spatial selection* mechanism in Map. The IX vector is used to delimit the index of the vertical vectors applying spatial selection functions:

```

WHERE(IX >= k)
WHERE(IX <= q)
MAPOP(Vi, V(i+1), ... V(j-1), V(j))

```

Thus, any sub-collection is delimited in the matrix $\mathbf{M}_{mp}$, if $i, j, k, q \leq m, p$, in constant time. The degree of parallelism depends on the size of the sub-collections.

14.7.2.3 Partition Pattern

The partition pattern corresponds to a geometric decomposition pattern without overlapping regions. See in Figure 14.12 a partition in four 4×4 scalars sub-collections. When dealing with sub-collections of varying sizes, it involves *segmentation*. The same mechanisms for defining sections (sub-collections) used for geometric decomposition are considered for splitting, with the additional care to avoid overlap. We can take into account for the horizontal delimitations, in addition to those stated for the geometric decomposition, conditions imposed on the content of the horizontal vectors. The spatial selection functions, specific to the MAP structure, represent a particularly useful feature, specific to MapScanReduce architecture.

14.7.2.4 Speculative Execution Pattern

Within a many-cell system, the speculative selection pattern is an optimization technique which performs both computations and multiple-condition evaluations, in order to combine condition evaluation with processing. On our machine it is executed as a map pattern involving a $\text{MAPOP}(\dots)$ followed by various spatial selections.

Example 14.11 *Let there be a Boolean expression of $\log_2 p$ binary variables and we are required to determine the binary configurations for which the expression takes the logical value 1. The binary form of the index of each cell in the Map represents the p binary combinations for which the expression must be evaluated. The cells evaluate in parallel the same expression under the SQ command and finally using spatial selection the cell in which the evaluation result is 1 remains active.*

◇

The acceleration obtained for this pattern is in $O(p)$.

14.7.2.5 Recurrence Pattern

The recurrence pattern represented in the expanded form represented in Figure 14.13) is characterized by nested loops with data dependencies. These structures can be parallelized by obtaining a dependency graph and creating tasks that can be run in parallel for each layer of the graph (separating hyperplane).

Since the recurrence pattern appears in loops, where operations are repeated, these can be run on our machine as all cells will execute the same instructions. Moreover, the fixed and predetermined data access pattern further facilitates this process.

Let us consider, as example the $p \times p$ recurrence pattern depicted in Figure 14.13 for $p = 4$ (see in (McCool et al., 2012) at pag. 206). The computation consists of p cycles. In each cycle, a scalar, s_i , for $i = 0, 1, \dots, p - 1$ is inserted in a vector in Map and the resulting vector is involved in a MAPOP processing. In each cycle the MAPOP sequence can be different. The sequence of operations for a certain computation is, in our MapScanReduce accelerator, the following:

```
V1 <= SHIFTRIGHT(V1, s1)
MAPOP(V1, ...)
V2 <= SHIFTRIGHT(V2, s2)
MAPOP(V2, ...)
```

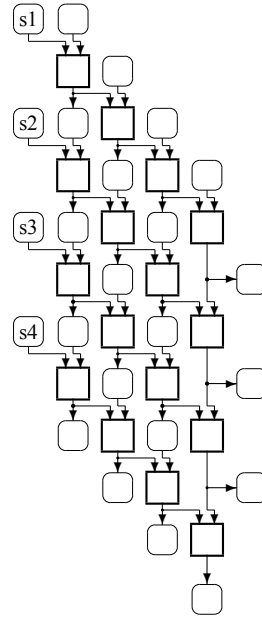


Figure 14.13: Recurrence pattern implementation (McCool et al., 2012).

```

V3 <= SHIFTRIGHT(V3,s3)
MAPOP(V3,...)
V4 <= SHIFTRIGHT(V4,s4)
MAPOP(V4,...)
...
Vp <= SHIFTRIGHT(Vp,s(p))
MAPOP(Vp,...)

```

In time in $O(p)$ a number of operations in $O(p^2)$ are performed, thus obtaining an acceleration in $O(p)$.

14.7.2.6 Pipeline Pattern

The pipeline pattern refers to having a sequence of stages that flow into one another sequentially. Parallel speedup is achieved by ensuring that all stages carry out their respective operations independently of each other. In a departure from the traditional approach, this architecture implements the Pipeline principle as all major components (SQ, cells in Map, Scan/Reduce network, Data transfer mechanism) work in parallel to one another with data flowing between them.

The MapScanReduce architecture is suited for the pipeline pattern only when all computational cells must execute the same instructions, for situations where each pipeline stage executes the same operations, i.e., the pipeline of stages describes a simple, intense computation.

A typical example is the FIR filter computation involving the vectors V1 (to insert the input data in), V2 (to store the result of multiplications), V3 (to accumulate the results of multiplications), 4 (to store tap constants).

```

loop{   V1 <= SHIFTRIGHT(V1,in)

```

```

V2 <= MULT(V1,V4)
V3 <= ADD(V3,V2)
V3 <= SHIFTRIGHT(V3,0)  }

```

The output of the computation is the right end of the vector V3.

A possible improvement is to combine the pipeline pattern with the reduce pattern. The loop will be shorted because the multiplication and the addition will be done in parallel in Map section and Reduce section of the MapReduce accelerator. The acceleration provided is in $O(p)$.

14.7.3 Pattern Executable on MapReduce Only

In the previous pattern, using the reduction network was optional. But there are applications where it is mandatory.

14.7.3.1 Fork-Join Pattern

Fork-join is a control pattern that divides (fork) work and subsequently combines (join) the results (see Figure 14.12). This pattern is among the more general and loosely defined ones, making a comprehensive discussion challenging. As a starting point, two approaches can be brought to attention:

- 1) simple symmetric patterns, a case of *divide et impera*)
- 2) complex, unpredictable, independent, thread spawning.

Our accelerator can only be used in the first case, which represents an intense, but simple computation. The acceleration that can be obtained is in $O(p/\log p)$ due to the latencies imposed by distribution, involved in fork, and reduction used to join the results.

14.7.4 Patterns Executable on MapScanReduce

There follow patterns that assume the use of the entire structure of the accelerator MapScanReduce.

14.7.4.1 Category Reduction Pattern

Category reduction pattern selects the set of elements that make up the string stored in a number of horizontal vectors, is not directly implemented in the structure of the proposed accelerator, but it can be efficiently programmed. We consider that the $\mathbf{M}_{mp}$ matrix distributed within the Map section has the capability to store the input stream S , which has the of length $|S|$ in the form of $N = \lceil |S|/p \rceil$ p -component vectors.

Spatial selections are utilized to keep active only cells where the vector V_i , for $i = 1, 2, \dots, N$, has no x_s , a value not included in S . The same operation allows the selection of all cells where the remained active calls contain F the first value different from X which is and substitute it with X . The result is the stream R in D , initially empty. The sequence of operations is:

```

R = <>
for(i=1; i<=N; i=i+1) {
  do on Vi  WHERE(!= x)
            WHERE(first)
            v0 <= REDADD(Vi)
}

```

```

R <= <R v0>
for(j=i; j<N; j=j+1) {
  do on Vj    WHERE(v0)
              VAL(x)
}
until(Vi = <X,X,...,X>)
}

```

Because the mono-core execution of the pattern has the time in $O(|S| \times |D|)$, and the execution time in our accelerator is in $O(|S|/p \times |D| \times \log p)$, the acceleration obtained for this pattern is in $O(p/\log p)$. The $\log$ part in our evaluation appears due to the latencies of Scan to determine the first occurrence of a symbol different from X in a vector V_j (see line `WHERE(first)`), and due to the latency of Reduce network to read in Controller the value of the first occurrence different from x in the vector V_i (see line `v0 <= REDADD(Vi)`).

Important note: the scan type network used for this pattern processes a Boolean vector, which is why we can approximate that the size of the system remains in $O(p)$.

14.7.4.2 Pack Pattern

The pack pattern is facilitated by the Scan section. The selection of the components to be packed is done using the Boolean vector B , which is managed by the spatial selection functions. The pattern is defined by $B = \langle 0\ 1\ 1\ 0\ 0\ 1\ 1\ 1 \rangle$. The positions in the destination vector of the components to be packed are computed using the `ADDPX(B)`. The algorithm is:

```

initially:  B  = <1 1 1 1 1 1 1 1>
            V0 = <0 1 1 0 0 1 1 1>
            V1 = <A B C D E F G H>

WHERE(V0==1)      | B  = <0 1 1 0 0 1 1 1>
V2 <= VAL(1)      | V2 = <x 1 1 x x 1 1 1>
V2 <= ADDPX(V2)   | V2 = <x 1 2 x x 3 4 5>
V2 <= ADD(V2,(-1)) | V2 = <x 0 1 x x 2 3 4>
V3 <= PERM(V1,2)  | V3 = <B C F G H x x x>
ENDWHERE

```

The pack pattern is an operation accelerated in $O(p/\log p)$ because of the latency introduced by the scan operations (`ADDPX`, `PERM`).

14.7.4.3 Gather Pattern

The gather pattern specifies in the source vector the source for each position in the destination vector (see Figure 14.12). The gather pattern is a permutation defined using the order vector as its source and the index vector IX as its destination. We will denote by (i) the content of the source or destination vector at position i . In Figure ??, the gather operation is defined by the operation `permute` with the order vector $\langle (1)\ (5)\ (0)\ (2)\ (2)\ (4) \rangle$. Because the scalar C has two destinations, in the positions 2 and 3, the operation is performed in two steps using successively two order vectors: $\langle (1)\ (5)\ (0)\ (2)\ (3)\ (4) \rangle$ and $\langle (1)\ (5)\ (0)\ (2)\ (2)\ (4) \rangle$. Only if each destination has its distinct source, the operation is performed using one permutation operation. The acceleration for this pattern is in $O(p/\log p)$, if the permutation is known in advance and control bits are precomputed.

14.7.4.4 Scatter Pattern

The scatter pattern specifies the destination for each position. The scatter pattern is a permutation defined using the index vector IX as its source and the order vector as its destination. In Fig. 14.12:

$$\langle (0) (1) (2) (3) (4) (5) \rangle \Rightarrow \langle (1) (5) (0) (2) (2) (4) \rangle$$

is represented as a deficient scatter because a destination, 2, has two distinct sources, 3 and 4. It must be considered a sort of “overflow” and the system must generate an error (exception).

If the definition of scatter is not deficient, the scatter operation is a permute operation operated in the SCAN section of our accelerator with an acceleration in $O(p/\log p)$, if the permutation is known in advance and control bits can be precomputed.

14.7.4.5 Split Pattern

The split pattern can be conceptualized as two sequential applications of the pack pattern. The first involves a standard pack with selected values going to the left, while the second necessitates computing the $NOT(B)$ vector such that previously unselected values are now selected and can then be packed to the right using $COMPRIGHT(d, l, r)$. In sequence of operations for pack pattern the vector $v0$ is complemented, instead $VSUB(2, 2, 1)$ is executed $VADD(2, 2, (p-REDMAX(V2)-1))$ followed by $COMPRIGHT(V2, V1, V2)$.

As with pack, the split pattern is accelerated in $O(p/\log p)$.

14.7.4.6 Expand Pattern

For this pattern each cell stores a self-delimited vertical vector of size 0 to p (refer to Figure 14.12).

The expand pattern is implemented in three steps:

- 1) the matrix M_{mp} is transposed (an efficient algorithm uses permutation)
- 2) the resulting empty lines are removed
- 3) the resulting self-delimited horizontal vectors are transferred serially to the sequencer’s memory.

Due to the serial transfer of data to the sequencer’s memory, this pattern cannot be accelerated on this architecture. This time the complexity is given by the data structure, even if the algorithm is in the category of intense, simple ones.

14.7.5 Super-scalar Sequence Pattern

The super-scalar pattern describes a complex computation, which is why it requires a special discussion. The content of the square boxes in Figure 14.12 can be:

- 1) a simple instruction or a complex sequence of instructions
- 2) a complex computation performed on a data structure with size in $O(f(N))$.

In the first case, the solution is instruction-level parallelism competence and the implementation is done with a super-scalar pipeline processor. In the second case, the solution can be provided by the recently introduced concept of accelerator-level parallelism (ALP) (?).

A way of implementing ALP can be a MapScanReduce system with cells as MapScanReduce structures. Each cell accelerates an intense function that can be different from the others. We can say that a *super-tensorial* computation is ensured. We are dealing with a complex calculation that involves intensive calculations.

Part IV

PRACTICAL DESIGNS

Chapter 15

Performance Optimized Digital Logic

15.1	Comparison of <code>when</code> and <code>MuxCase</code> in Chisel	300
15.1.1	Behavioral and Structural Implications	300
15.1.2	Synthesis Implications	300
15.1.3	Specific Considerations	300
15.1.4	Switch Synthesis in Chisel	301
15.1.5	Comparison with <code>MuxCase</code> and <code>when</code>	301
15.2	Carry Lookahead Adder	301
15.3	Carry Select Adder	301
15.4	Carry Save Adder	302
15.5	Kogge-Stone Adder	302
15.6	<i>Multipliers</i>	302

15.1 Comparison of when and MuxCase in Chisel

The choice between when and MuxCase in Chisel depends on the specific design scenario and synthesis goals.

15.1.1 Behavioral and Structural Implications

when is more natural and intuitive for conditions that resemble software-like if-else logic. It typically generates cascaded conditional logic (e.g., if-else structures) in the hardware, where each condition is checked sequentially. On the other hand, MuxCase represents a priority-free selection logic, a multiplexer, where all conditions are evaluated simultaneously. This structure often synthesizes directly into a multiplexer tree, which can result in better performance and reduced latency for certain designs.

15.1.2 Synthesis Implications

Timing and Performance: For timing, when introduces propagation delays due to the cascading nature of sequential conditions. In contrast, MuxCase generates parallel logic, potentially yielding better timing performance as conditions are evaluated independently.

Area: The when construct can produce more compact hardware for sequential conditions since unused conditions are pruned early. However, MuxCase may generate larger hardware if many mutually exclusive conditions are evaluated together because all conditions are checked in parallel.

Readability: when is preferable for sequential, human-readable conditions with natural priority, while MuxCase is better suited for a concise representation of mutually exclusive cases.

15.1.3 Specific Considerations

Mutually Exclusive Conditions: If conditions are naturally exclusive, MuxCase is often a better fit. For example:

```
val result = MuxCase(0.U, Seq(
    (opcode === ADD → (a + b),
    (opcode === SUB → (a - b))
```

)) This translates to a 3-to-1 multiplexer in hardware.

Priority Logic: When conditions have a natural priority (e.g., an else case that catches all unhandled conditions), when is preferable. For example:

```
when(opcode === ADD) {
    result := a + b
}.elsewhen(opcode === SUB) {
    result := a - b
}.otherwise {
    result := 0.U
```

} This logic represents a chain of conditional checks in hardware.

15.1.4 Switch Synthesis in Chisel

In Chisel, a `switch` statement is a construct used for multi-way branching based on a condition or value. The hardware synthesized from a `switch` statement depends on the conditions and the synthesis toolchain. Generally, it can result in one of the following hardware structures:

A **multiplexer tree** is generated if the `switch` conditions are mutually exclusive, with each case corresponding to a distinct value of the selector. For instance:

```
val result = Wire(UInt(8.W))switch(selector) { is("b00".U) { result := a } is("b01".U) { r
```

This should synthesize into a 4-to-1 multiplexer controlled by the value of `selector`.

When the `switch` includes overlapping or non-mutually-exclusive conditions, **priority logic** is typically generated. Priority logic evaluates cases sequentially, giving precedence to earlier conditions. For wide selectors or more complex logic, synthesis tools may generate **decoder logic** to translate the selector into one-hot or prioritized enable signals that control the corresponding cases.

15.1.5 Comparison with `MuxCase` and `when`

`MuxCase` is more explicit and directly creates a multiplexer, making it compact and efficient for mutually exclusive conditions. `when` constructs explicitly generate priority logic. In contrast, `switch` can represent either multiplexers or priority logic depending on the use case, providing a flexible but less predictable synthesis.

15.1.5.1 Limitations of `switch`

While `switch` provides a clean and readable way to handle multi-way branching, it is not always recommended for large or complex selectors. For wide selectors, the generated decoder logic can result in significant area and timing overhead. Additionally, overlapping or non-mutually-exclusive cases can inadvertently lead to priority chains that increase latency and complicate timing analysis.

15.2 Carry Lookahead Adder

Carry Look-Ahead Adders (CLAs) are designed to overcome the performance bottlenecks of ripple-carry adders by reducing the dependency on sequential carry propagation. Instead of waiting for each bit's carry to be computed, the CLA computes carries in parallel using the generate (G) and propagate (P) signals for each bit. These signals allow the adder to determine whether a carry will be generated or propagated at each stage. Using these signals, the carry for each bit position can be calculated directly through a set of Boolean equations, significantly reducing the delay to logarithmic time relative to the adder's width. This design is particularly effective for high-speed arithmetic operations in processors and digital signal processing, where minimizing latency is important. However, the parallel computation and additional logic circuitry result in increased hardware complexity compared to simpler adders.

15.3 Carry Select Adder

Carry Select Adders (CSAs) improve upon the delay of ripple-carry adders by precomputing the sum and carry outputs for both possible carry-in values (0 and 1) in parallel. Each stage of the adder computes two possible results based on these assumptions, and a multiplexer is used to select the correct result

once the carry-in is determined. This approach significantly reduces the propagation delay associated with sequential carry computation while maintaining a relatively straightforward design. Although CSAs require additional hardware resources for the parallel computations and multiplexers, they achieve a good trade-off between speed and area, making them well-suited for medium-sized arithmetic units where both latency and complexity are important considerations.

15.4 Carry Save Adder

Carry-save adders are specialized adders used primarily in multiplication. Unlike conventional adders, which propagate carries through the stages immediately, CSAs represent the result as two separate numbers: a partial sum and a carry vector. These two outputs are then combined in a subsequent addition step, often using a more efficient adder such as a carry-lookahead adder. By deferring the carry propagation, CSAs significantly reduce the delay for intermediate steps, making them highly efficient for scenarios where multiple additions must be performed simultaneously. Examples include the Wallace tree (Wallace, 1964) and Dadda tree (Dadda, 1965). This efficiency is achieved at the cost of additional logic to manage the separate sum and carry outputs.

15.5 Kogge-Stone Adder

Parallel prefix adders are a class of high-performance adders designed to optimize the computation of carry signals in binary addition, significantly reducing delay compared to simpler adder designs like ripple-carry adders. They achieve this by dividing the addition process into three distinct stages: generating initial propagate and generate signals for each bit, computing intermediate carries through a tree-like prefix structure, and finally calculating the sum based on these carry signals. The tree structure allows the carry signals to be computed in parallel, with a logarithmic time complexity relative to the number of bits, making these adders highly efficient for large bit-width operations. These adders are widely used in modern high-speed computing systems and processors.

The Kogge-Stone Adder is a parallel prefix adder known for its high performance and suitability for large bit-width arithmetic operations. It minimizes the propagation delay using a tree-like structure to compute carry signals in a logarithmic number of stages relative to the bit width. The design uses generate and propagate signals to recursively compute carries in parallel, ensuring that the carry for any given bit is available after just $\log_2(n)$ stages, where n is the bit width. While this structure significantly enhances speed, it requires substantial wiring and additional resources, making it resource-intensive compared to simpler designs like ripple-carry adders. Due to its efficiency in reducing delay, the Kogge-Stone Adder is frequently employed in high-performance processors and digital systems requiring fast arithmetic computations (Kogge and Stone, 1973).

15.6 Multipliers

Chapter 16

Instruction-Level Parallelism

16.1	Pipelining	304
16.1.1	Pipeline Acceleration	304
16.1.2	<i>Pipelined Version of toyRISC</i>	305
16.1.3	Latency	309
16.2	Hazards Generated by Dependencies	309
16.2.1	Data dependency	310
16.2.2	Control Dependency	315
16.3	Programming Pipelined Processors	321
16.4	Superscalar Processor	323
16.4.1	Register renaming	323
16.4.2	Out-of-Order Execution	324
16.5	Problems	327
16.5.1		327
16.5.2		327

There are various form of instruction-level parallelism. We will consider in this short course only the parallelism introduced by the pipeline mechanism and the multiple execution units. Thus, the 5th lesson deals with the pipeline mechanism, used to accelerate the processor's operation, and the implications of its use. But as we will see any gain obtained from the application of an improvement technique comes with a price.

16.1 Pipelining

Combinational logic circuits (CLCs) that have a very high depth cause the system clock frequency to be reduced. The solution that allows increasing the clock frequency is the introduction of pipeline registers.

16.1.1 Pipeline Acceleration

In Figure 16.1 the pipeline technique is illustrated. In Figure 16.1a, the frequency at which the system clock can work is given by the propagation time through the CLC_{fog} to which is added the time associated with the propagation through the registers.

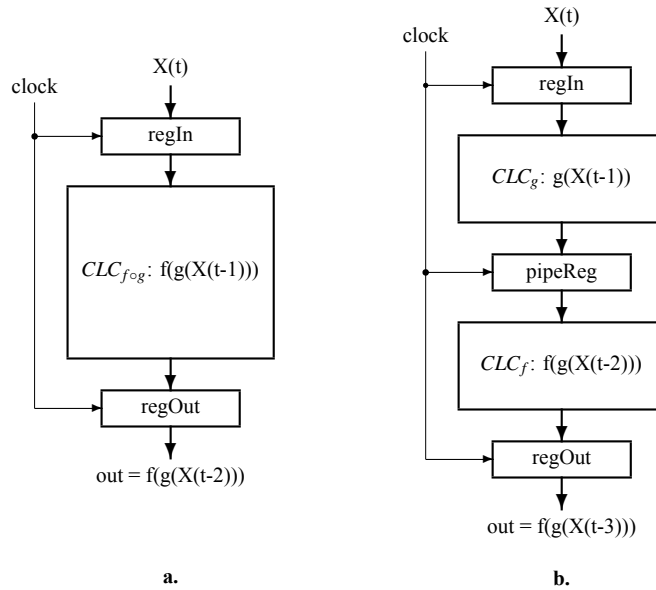


Figure 16.1: Pipelining. **a.** The initial circuit, without pipeline register. **b.** The pipelined version of the circuit.

$$T_{clock} = t_{regProp} + t_{CLC_{fog}} + t_{set-up} \simeq t_{CLC_{fog}}$$

In Figure 16.1b, the frequency at which the system clock can work will be increased because the period of the clock is given by

$$T_{clock} = t_{regProp} + \max(t_{CLC_f}, t_{CLC_g}) + t_{set-up}$$

The resulting acceleration is:

$$\alpha \simeq \frac{t_{CLC_{f \circ g}}}{\max(t_{CLC_f}, t_{CLC_g})}$$

The maximum efficiency, $\alpha \simeq 2$, is obtained when

$$t_{CLC_f} \simeq t_{CLC_g}$$

If the resulting frequency is not high enough, the technique is applied to the deepest circuit or both. The process can continue until the requested speed is reached.

16.1.2 *Pipelined Version of toyRISC*

16.1.2.1 Structure

The operating frequency of the toyRISC processor presented in the previous lesson can be increased by inserting some pipeline registers on the critical path of the combinatorial propagation. A version, adapted to be accelerated, is shown in Figure 16.2 where the two memories are not represented. Instead, only the connections to those memories are represented. The program memory is a synchronous memory (see 9.4.2) connected in our system in combinational mode through the output multiplexers. The data memory is a synchronous pipelined memory (see 9.4.3) which responds with one cycle latency to the read command.

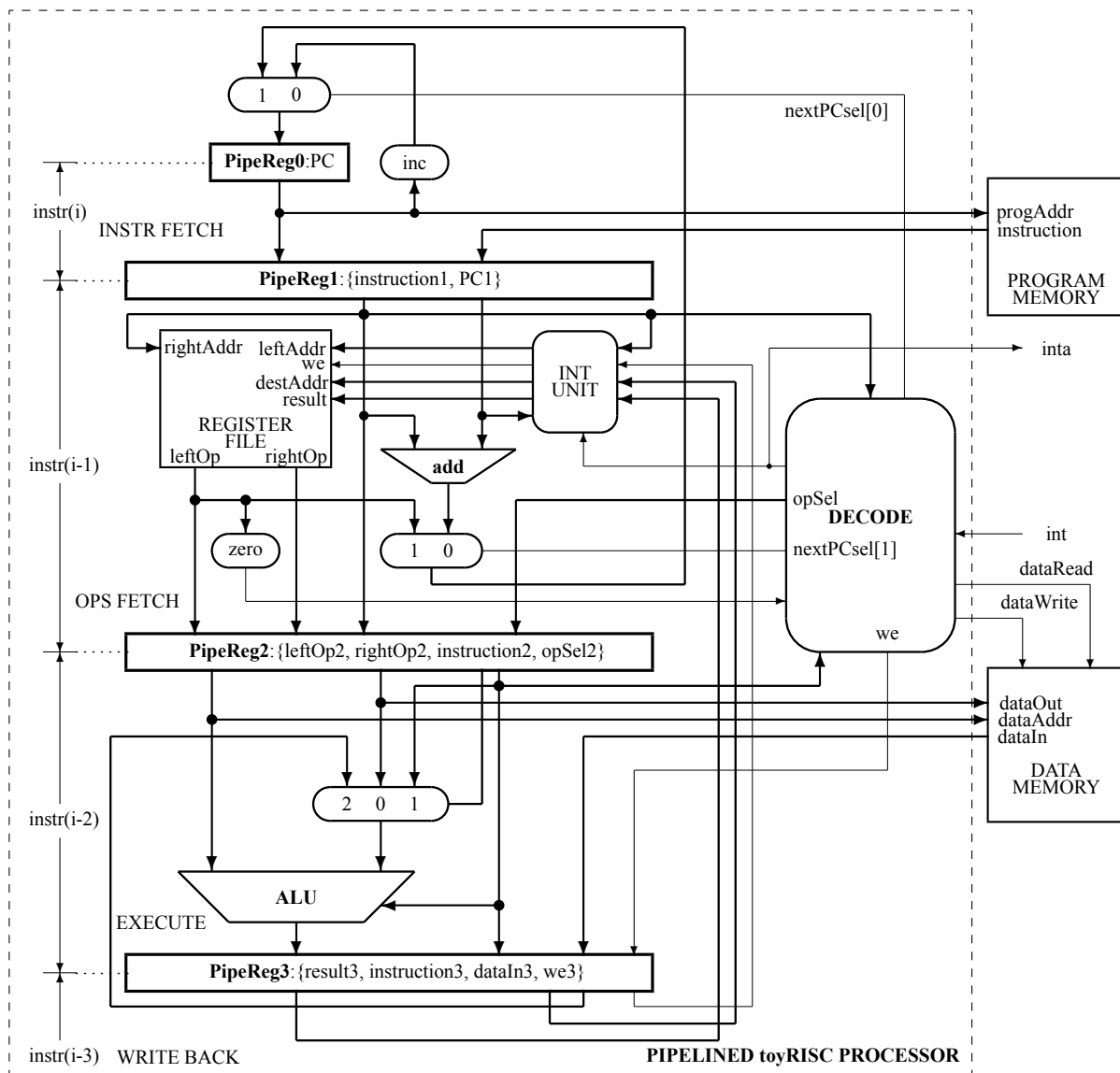


Figure 16.2: The pipelined version of toyRISC.

16.1.2.2 Micro-architecture

The micro-architecture is:

```

/*****
File name: DEFINES.vh
          PIPELINED TOY-RISC MICROARCHITECTURE
*****/

```

```

// CONTROL OPERATIONS
`define nop      6'b00_0000 // no operation: pc<=pc+1;
`define rjmp     6'b00_0001 // relative jump: pc<=pc+v;
`define zbr      6'b00_0010 // pc<=(rf[l]=0) ? pc+v:pc+1
`define nzbr     6'b00_0011 // pc<=! (rf[l]=0) ? pc+v:pc+1
`define ret      6'b00_0101 // return: pc<=rf[l][15:0];
`define halt     6'b00_0110 // halt until interrupt
`define eint     6'b00_1000 // set enable interrupt
`define dint     6'b00_1001 // set disable interrupt
// ARITHMETIC & LOGIC OPERATIONS, for these instructions: pc<=pc+1;
`define add      6'b11_0000 // rf[d]<=rf[l]+rf[r];
`define sub      6'b11_0001 // rf[d]<=rf[l]-rf[r];
`define addv     6'b11_0010 // rf[d]<=rf[l]+v;
`define mult     6'b11_0011 // rf[d]<=rf[l]*rf[r];
`define multv    6'b11_0100 // rf[d]<=rf[l]*v;
`define addc     6'b11_0101 // rf[d]<=(rf[l]+rf[r])[32];
`define subc     6'b11_0110 // rf[d]<=(rf[l]-rf[r])[32];
`define addvc    6'b11_0111 // rf[d]<=(rf[l]+v)[32];
`define lsh      6'b11_1000 // rf[d]<=rf[l] >> 1;
`define ash      6'b11_1001 // rf[d]<=
                        // <={rf[l][31], rf[l][31:1]};
`define move     6'b11_1010 // rf[d]<=rf[l];
`define swap     6'b11_1011 // rf[d]<=
                        // <={rf[l][15:0], rf[l][31:16]};
`define bwnot    6'b11_1100 // rf[d]<=~rf[l];
`define bwand    6'b11_1101 // rf[d]<=rf[l]&rf[r];
`define bwor     6'b11_1110 // rf[d]<=rf[l]|rf[r];
`define bwxor    6'b11_1111 // rf[d]<=rf[l]^rf[r];
// DATA LOAD-STORE OPERATIONS, for these instructions: pc=pc+1;
`define read     6'b10_0000 // read from dataMemory[rf[l]];
`define load     6'b10_0111 // rf[d]<=dataOut;
`define store    6'b10_1000 // dataMemory[rf[l]]<=rf[r];
`define val      6'b01_0111 // rf[d]<={16*{v[15]}}, v};

```

The three pipeline registers introduced in design divide the execution of an instruction in four stages:

INSTRUCTION FETCH (IF) : the content of the register pc (program counter) addresses in the program memory the binary code of an instruction; in the same time the module next computes the value for the next pc. The execution time for this stage is:

$$t_{IF} = t_{pc} + \max(t_{next}, t_{ACCtoProgramMemory}) + t_{suPipeReg1}$$

In the pipeline register **PipeReg_1** is loaded the information requested to finish the execution of the fetched instruction

OPERANDS FETCH (OF) : from the register are fetched the two operands of the instruction using the fields leftAddr and rightAddr fetched in the previous cycle from the program memory. In **pipeReg_2** are loaded the two fetched operands and the information needed in the next cycles to end the execution of the instruction. The execution time for this stage is:

$$t_{OF} = t_{PipeReg1} + t_{regFile} + t_{suPipeReg2}$$

EXECUTION (EX) : ALU performs the operation according to the code opCode, propagated to this stage through the two previous pipeline registers, generate result and the predicate zero = (result == 0); the add module adds to the program counter the signed value of value to perform the branch operation. The execution time for this stage is:

$$t_{EX} = t_{PipeReg2} + \max((t_{mux} + t_{alu}), (t_{add} + t_{mux}), t_{dataMemPipeReg}) + t_{suPipeReg3}$$

WRITE BACK (WB) : acts on the first two levels in the pipeline: it allows the modification of the pc to perform jumps and writes the result of the current operation in the register file. The execution time for this stage is:

$$t_{WB} = t_{PipeReg3} + \max(t_{suRegFile}, (t_{next} + t_{suPC}))$$

The maximum clock frequency is limited by:

$$T_{clock} = \max(t_{IF}, t_{OF}, t_{EX}, t_{WB})$$

16.1.2.3 Architecture

The Instruction Set Architecture is:

```

/*****
toyRISC'S ARCHITECTURE
*****/
NOP          // no operation
RJMP(lb)     // relative jumpto label 'lb'
BRZ(l,lb)    // branch if rf[l]=zero at label 'lb'
BRNZ(l,lb)   // branch if rf[l]!=zero at label 'lb'
RET(l)       // return from subroutine: pc<=rf[l]
HALT        // halt until interrupt is received, pc = pc
// for the following instructions: pc<=pc+1;
EINT        // set enable interrupt
DINT        // set disable interrupt
ADD(d,l,r)   // rf[d]<=rf[l]+rf[r];
SUB(d,l,r)   // rf[d]<=rf[l]-rf[r];
ADDV(d,l,v)  // rf[d]<=rf[l]+v;
MULT(d,l,r)  // rf[d]<=rf[l]*rf[r];
MULTV(d,l,v) // rf[d]<=rf[l]*v;
ADDC(d,l,r)  // rf[d]<=(rf[l]+rf[r])[32];
SUBC(d,l,r)  // rf[d]<=(rf[l]-rf[r])[32];
ADDVC(d,l,v) // rf[d]<=(rf[l]+v)[32];
LSH(d,l)     // rf[d]<=rf[l] >> 1;
ASH(d,l)     // rf[d]<={rf[l][31],rf[l][31:1]};
MOVE(d,l)    // rf[d]<=rf[l];
SWAP(d,l)    // rf[d]<={rf[l][15:0],rf[l][31:16]};
NOT(d,l)     // rf[d]<=~rf[l];
AND(d,l,r)   // rf[d]<=rf[l]&rf[r];
OR(d,l,r)    // rf[d]<=rf[l]|rf[r];

```

```

XOR(d,1,r)  // rf[d]<=rf[1]^rf[r];
READ(1)     // read from dataMemory[rf[1]];
LOAD(d)     // rf[d]<=dataOut;
STORE(1,r)  // dataMemory[rf[1]]<=rf[r];
VAL(d,v)    // rf[d]<={{16*{v[15]}}},v};

```

16.1.3 Latency

In each clock cycle the execution of one instruction ends with a latency of three clock cycles. The entire structure performs in parallel 4 successive instructions, each stage being involved in the execution of one instruction.

Example 16.1 *Let us see how is executed the following sequence of instructions:*

```

XOR(5,0,0) ;
VAL(1,12)  ;
SUB(2,3,4) ;
ADD(0,0,3) ;
AND(1,3,2) ;

```

In Table 16.1 the distribution along the pipe of the execution is represented. Each line in the table represents a pipeline level. The execution of the first instruction, XOR, starts in the i -th clock cycle, with XOR0, and ends in the $(i+3)$ -th cycle with XOR3. Then in each clock cycle one of the following instruction ends.

Table 16.1: Pipelined execution.

Pipe Stage \ Time	t=i	t=i+1	t=i+2	t=i+3	t=i+4	t=i+5	t=i+6	t=i+7
Pipe0:IF	XOR0	VAL0	SUB0	ADD0	AND0			
Pipe1:OF		XOR1	VAL 1	SUB1	ADD1	AND1		
Pipe2:EX			XOR2	VAL2	SUB2	ADD2	AND2	
Pipe3:WB				XOR3	VAL3	SUB3	ADD3	AND3

◇

In each clock cycle, an instruction completes with a latency of 3 clock cycles. The good news: the clock frequency increases significantly due to pipeline organization. The bad news: the latency introduced by the pipeline organization creates unwanted dependencies.

16.2 Hazards Generated by Dependencies

There are three types of dependencies:

- 1) Structural Dependency
- 2) Data Dependency
- 3) Control Dependency

These dependencies generate hazardous behaviors of the pipelined processor due to the parallel execution of instructions which partially overlap. In the following some of them will be illustrated. Due to the fact that we used the Harvard abstract model for our toyRISC processor, main structural dependency are avoided. Therefore we concentrate only to the next two type of dependencies.

Structural dependencies are typically represented by those related to processor memory. In the case of our processor, toyRISC, they do not appear because we have an abstract model of the Harvard type with two memories, one for programs and another for data. Thus, in what follows, we will focus on the other two types of dependencies.

16.2.1 Data dependency

When the current instruction uses values generated by a previous instruction that has not been completed, the effect of data dependency appears. There are 3 types of data dependencies that we've been talking about:

- RAW: Read after Write
- WAR: Write after Read
- WAW: Write after Write

We will focus on the first, RAW, which is typical for our processor version. For the other two, they are illustrated in Example 16.14.

Example 16.2 *Let us see how is executed the following sequence of instructions:*

```
XOR(5,0,0) ;
VAL(1,12)  ;
SUB(2,3,4) ;
ADD(0,2,1) ;
AND(1,1,2) ;
```

In Table 16.2 the distribution along the pipe of the execution is represented.

Table 16.2: Data dependency in the pipelined execution .

Pipe Stage \ Time	t=i	t=i+1	t=i+2	t=i+3	t=i+4	t=i+5	t=i+6	t=i+7
Pipe0:IF	XOR0	VAL0	SUB0	ADD0	AND0			
Pipe1:OF		XOR1	VAL1	SUB1	ADD1	AND1		
Pipe2:EX			XOR2	VAL2	SUB2	ADD2	AND2	
Pipe3:WB				XOR3	VAL3	SUB3	ADD3	AND3

The WB cycle for SUB, SUB3, is executed at i+5, too late to have the content of rf2 actualized with the result of SUB(2,3,4) for the instruction ADD(0,2,1) which is supposed to have rf2 actualized by the previous instruction in i+4. ADD2 must be preceded by SUB3. SUB3 is delayed with one clock cycle for a proper execution of the code. The instruction ADD(0,2,1) uses the value in rf2 before the execution of SUB2,3,4.

◇

Example 16.3 *Data dependency is illustrated also by running on our simulator the program run in Example 11.2:*

```

NOP          ;
VAL(0,1)     ;
VAL(1,2)     ;
VAL(2,3)     ;
VAL(3,4)     ;
VAL(4,5)     ;
NOP          ;
NOP          ;
ADD(5,4,3)   ;
HALT         ;
HALT         ;

```

The result is correct because of the two NOPs inserted in the program from Example 11.2 before the ADD instruction:

```

t=0 pc=  x RF=[x, x, x, x, x, x, x, x] leftOp2=x rightOp2=x result3=x
t=1 pc=1023 RF=[x, x, x, x, x, x, x, x] leftOp2=x rightOp2=x result3=x
t=5 pc=  0 RF=[x, x, x, x, x, x, x, x] leftOp2=x rightOp2=x result3=x
t=7 pc=  1 RF=[x, x, x, x, x, x, x, x] leftOp2=x rightOp2=x result3=x
t=9 pc=  2 RF=[x, x, x, x, x, x, x, x] leftOp2=x rightOp2=x result3=x
t=11 pc= 3 RF=[x, x, x, x, x, x, x, x] leftOp2=x rightOp2=x result3=x
t=13 pc= 4 RF=[x, x, x, x, x, x, x, x] leftOp2=x rightOp2=x result3=1
t=15 pc= 5 RF=[1, x, x, x, x, x, x, x] leftOp2=x rightOp2=x result3=2
t=17 pc= 6 RF=[1, 2, x, x, x, x, x, x] leftOp2=1 rightOp2=1 result3=3
t=19 pc= 7 RF=[1, 2, 3, x, x, x, x, x] leftOp2=1 rightOp2=1 result3=4
t=21 pc= 8 RF=[1, 2, 3, 4, x, x, x, x] leftOp2=1 rightOp2=1 result3=5
t=23 pc= 9 RF=[1, 2, 3, 4, 5, x, x, x] leftOp2=1 rightOp2=1 result3=1
t=25 pc=10 RF=[1, 2, 3, 4, 5, x, x, x] leftOp2=5 rightOp2=4 result3=1
t=27 pc=10 RF=[1, 2, 3, 4, 5, x, x, x] leftOp2=1 rightOp2=1 result3=9
t=29 pc=10 RF=[1, 2, 3, 4, 5, 9, x, x] leftOp2=1 rightOp2=1 result3=1

```

If the two NOPs are omitted,

```

NOP          ;
VAL(0,1)     ;
VAL(1,2)     ;
VAL(2,3)     ;
VAL(3,4)     ;
VAL(4,5)     ;
ADD(5,4,3)   ;
HALT         ;
HALT         ;

```

then the processor behaves incorrectly, as follows:

```

t=0 pc=  x RF=[x, x, x, x, x, x, x, x] leftOp2=x rightOp2=x result3=x
t=1 pc=1023 RF=[x, x, x, x, x, x, x, x] leftOp2=x rightOp2=x result3=x
t=5 pc=  0 RF=[x, x, x, x, x, x, x, x] leftOp2=x rightOp2=x result3=x

```



```

t=7  pc= 1  RF=[x, x, x, x, x, x, x, x] leftOp2=x rightOp2=x result3=x
t=9  pc= 2  RF=[x, x, x, x, x, x, x, x] leftOp2=x rightOp2=x result3=x
t=11 pc= 3  RF=[x, x, x, x, x, x, x, x] leftOp2=x rightOp2=x result3=x
t=13 pc= 4  RF=[x, x, x, x, x, x, x, x] leftOp2=x rightOp2=x result3=1
t=15 pc= 5  RF=[1, x, x, x, x, x, x, x] leftOp2=x rightOp2=x result3=2
t=17 pc= 6  RF=[1, 2, x, x, x, x, x, x] leftOp2=1 rightOp2=1 result3=3
t=19 pc= 7  RF=[1, 2, 3, x, x, x, x, x] leftOp2=1 rightOp2=1 result3=4
t=21 pc= 8  RF=[1, 2, 3, 4, x, x, x, x] leftOp2=x rightOp2=x result3=5
t=23 pc= 8  RF=[1, 2, 3, 4, 5, x, x, x] leftOp2=1 rightOp2=1 result3=x
t=25 pc= 8  RF=[1, 2, 3, 4, 5, x, x, x] leftOp2=1 rightOp2=1 result3=1

```

because the operands for the arithmetic operations are not yet in place being delayed on the execution pipe.

◇

To solve the problem of data dependency just emphasized there are used two solutions: stalling and forwarding.

16.2.1.1 Stalling

An inefficient solution, for the execution time, is to stall by introducing a NOP instruction between SUB(2,3,4) and ADD(0,2,1), thus delaying the use of the content of rf2 with one clock cycle. Result the following sequence of instructions:

```

XOR(5,0,0) ;
VAL(1,12)  ;
SUB(2,3,4) ;
NOP        ;
ADD(0,2,1) ;
AND(1,1,2) ;

```

whose execution is illustrated in Table 16.3.

Table 16.3: Stalling

Instr \ Time	t=i	t=i+1	t=i+2	t=i+3	t=i+4	t=i+5	t=i+6	t=i+7	t=i+8
IF	XOR0	VAL0	SUB0	NOP0	ADD0	AND0			
OF		XOR1	VAL1	SUB1	NOP1	ADD1	AND1		
EX			XOR2	VAL2	SUB2	NOP2	ADD2	AND2	
WB				XOR3	VAL3	SUB3	NOP3	ADD3	AND3

Now, ADD2 is preceded by SUB3 and addition is performed taking into account the result of the subtract operation.

16.2.1.2 Reordering

Sometimes, **but only sometimes**, there is a very simple and efficient solution for data dependency: re-ordering the instructions in the sequence of instructions. In Example 16.2, the instruction XOR(5,0,0)

can be moved, without affecting the result of running the sequence of instructions, between the instructions SUB and ADD. as follows:

```
VAL(1,12)    ;
SUB(2,3,4)   ;
XOR(5,0,0)   ;
ADD(0,2,1)   ;
AND(1,1,2)   ;
```

Thus, the XOR instruction is used for stalling instead of the NOP instruction, but now the execution time is not affected.

16.2.1.3 Forwarding

By adding hardware, the data dependency can be resolved, **in all cases**, with no time penalty. The solution is to connect `result` not only to the register file input, but also directly to the ALU input when the value just computed is needed for the next instruction. The mechanism is called *forwarding* and requires the addition of one input to the ALU input as left operand or as right operand. But, because there are binary operations (operations with two operands) sometimes both operands arrive too late in the register file to be fetched for the current instruction.

Thus, forwarding control is done by a circuit which analyses the binary code of three successive instructions. Three situations can occur:

```
xxxxxx_00001_xxx...x      // identified in PipeReg_3
xxxxxx_yyyyy_00001_xxx...x // identified in PipeReg_2
```

or

```
xxxxxx_00001_xxx...x      // identified in PipeReg_3
xxxxxx_yyyyy_zzzzz_00001_xxx...x // identified in PipeReg_2
```

or

```
xxxxxx_00001_xxx...x      // identified in PipeReg_3
xxxxxx_yyyyy_00001_00001_xxx...x // identified in PipeReg_2
```

```
xxxxxx_00001_xxx...x      // identified in PipeReg_4
xxxxxx_00011_xxx...x      // identified in PipeReg_3
xxxxxx_yyyyy_00011_00001_xxx...x // identified in PipeReg_2
```

```
xxxxxx_00001_xxx...x      // identified in PipeReg_4
xxxxxx_00011_xxx...x      // identified in PipeReg_3
xxxxxx_yyyyy_00001_00011_xxx...x // identified in PipeReg_2
```

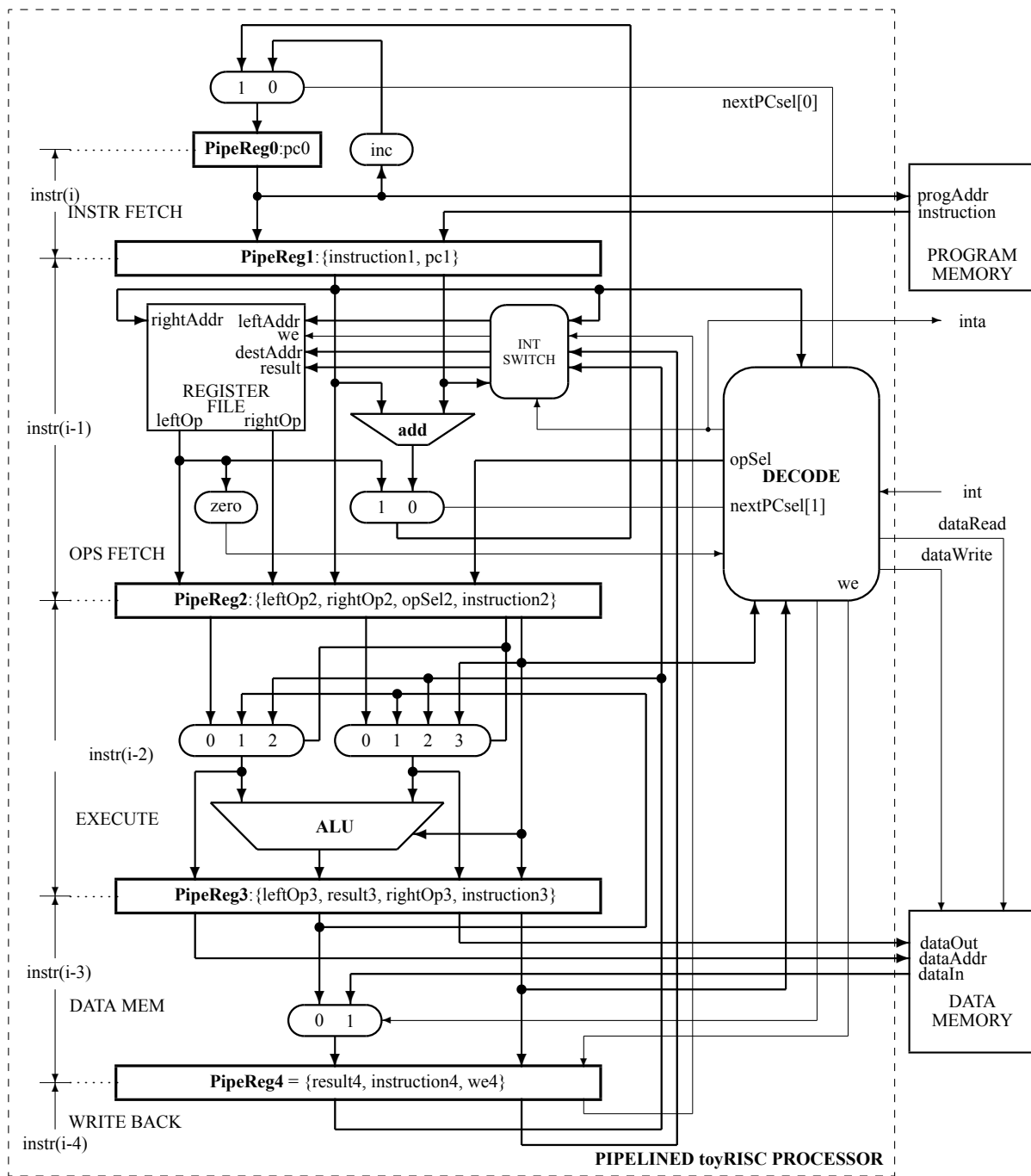


Figure 16.3: The change used to reduce hazards imposed by data dependencies.

The structure of the processor is modified as it is shown in Figure 16.3, where a new level in pipe is added and the operands at ALU inputs are selected using multiplexors with additional inputs. The

selection for the augmented multiplexors are computed combinational as `opSel[3:0]` in DECODE and synchronized in **PipeReg2** as `opSel2[3:0]` to avoid an additional delay in selecting ALU operands. Thus, `result` is used as operand in the EXECUTE state of the pipeline, if the forwarding mechanism decides, as `result3` connected the inputs 1 or as `result4` to the inputs 2 to the selection multiplexors at ALU operand inputs. The value of `result3` is used to be forwarded when the dependency is identified with the result of the instructions issued in one clock cycle before the current one. The value of `result4` is used to be forwarded when the dependency is identified with the result of the instructions issued in two clock cycles before the current one.

Example 16.4 *Data dependency solved by using forwarding is exemplified starting from Example 16.4:*

```

NOP          ;
VAL(0,1)     ;
VAL(1,2)     ;
VAL(2,3)     ;
VAL(3,4)     ;
VAL(4,5)     ;
ADD(5,4,3)   ;
HALT         ;
HALT         ;

```

Now the processor behaves incorrectly without the additional NOPs, as follows:

```

t=0  pc=  x  RF=[x, x, x, x, x, x, x, x]
t=1  pc=1023 RF=[x, x, x, x, x, x, x, x]
t=5  pc=  0  RF=[x, x, x, x, x, x, x, x]
t=7  pc=  1  RF=[x, x, x, x, x, x, x, x]
t=9  pc=  2  RF=[x, x, x, x, x, x, x, x]
t=11 pc=  3  RF=[x, x, x, x, x, x, x, x]
t=13 pc=  4  RF=[x, x, x, x, x, x, x, x]
t=15 pc=  5  RF=[x, x, x, x, x, x, x, x]
t=17 pc=  6  RF=[1, x, x, x, x, x, x, x]
t=19 pc=  7  RF=[1, 2, x, x, x, x, x, x]
t=21 pc=  8  RF=[1, 2, 3, x, x, x, x, x]
t=23 pc=  8  RF=[1, 2, 3, 4, x, x, x, x]
t=25 pc=  8  RF=[1, 2, 3, 4, 5, x, x, x]
t=27 pc=  8  RF=[1, 2, 3, 4, 5, 9, x, x]

```

because the operands for the arithmetic operations are now forwarded using the two multiplexors to the ALU's inputs.

◇

16.2.2 Control Dependency

The pipelined processor always fetches the instruction immediately after any taken branch.

Example 16.5 *Let be the following sequence of instructions which contains a unconditioned (relative) jump:*

```

i:      XXX      ;
i+1:    RJMP(12);
i+2: LB(2)  YYY    ;
i+3:    UUU      ;
i+4:    VVV      ;
...     ...
i+j: LB(12) ZZZ    ;

```

Its execution is supposed to be:

```

XXX      ;
RJMP(12);
ZZZ      ;

```

but in our processor it will be:

```

XXX      ;
RJMP(12);
YYY      ;
ZZZ      ;

```

because the instruction YYY are inserted into the pipe before the execution of the jump instruction completed in $t=i+2$ when the jump address is available (see Table 16.4).

Table 16.4: Hazard generated by the control dependency

Instr \ Time	t=i	t=i+1	t=i+2	t=i+3	t=i+4	t=i+5	t=i+6	t=i+7
IF	XXX0	RJMP0	YYY0	ZZZ0				
OF		XXX1	RJMP1	YYY1	ZZZ1			
EX			XXX2	----	YYY2	ZZZ2		
DM				XXX3	----	YYY3	ZZZ3	
WB					XXX4	----	YYY4	ZZZ4

◇

Example 16.6 *In the following example the effect of interrupt is illustrated together with the effect of the control dependency due to the unwanted execution of VAL(4,33) positioned after the RET30 instruction.*

```

VAL(31,13)      ;
VAL(2,23)        ;
VAL(0,13)        ;
VAL(1,13)        ;
EI               ;
ADDV(0,0,1)      ;
VAL(1,1)         ;
VAL(2,222)       ;
NOP              ;
ADDV(0,0,4)      ;

```

```

        HALT          ;
        HALT          ;
        NOP           ;
// subroutine triggered by interrupt
        ADDV(30,30,-1) ;
        NOP           ;
        NOP           ;
        NOP           ;
        RET(30)       ;
        VAL(4,33)     ;

```

The result of simulation is:

t=0	pc=	x	RF=[x, x, x, x, x, x, x, x]	intState=xxx	inta=x
t=1	pc=	1023	RF=[x, x, x, x, x, x, x, x]	intState=000	inta=0
t=5	pc=	0	RF=[x, x, x, x, x, x, x, x]	intState=000	inta=0
t=7	pc=	1	RF=[x, x, x, x, x, x, x, x]	intState=000	inta=0
t=9	pc=	2	RF=[x, x, x, x, x, x, x, x]	intState=000	inta=0
t=11	pc=	3	RF=[x, x, x, x, x, x, x, x]	intState=000	inta=0
t=13	pc=	4	RF=[x, x, x, x, x, x, x, 13]	intState=000	inta=0
t=15	pc=	5	RF=[x, x, 23, x, x, x, x, 13]	intState=000	inta=0
t=17	pc=	6	RF=[13, x, 23, x, x, x, x, 13]	intState=001	inta=0
t=19	pc=	7	RF=[13, 13, 23, x, x, x, x, 13]	intState=010	inta=1
t=21	pc=	13	RF=[13, 13, 23, x, x, x, 6, 13]	intState=011	inta=0
t=23	pc=	13	RF=[13, 13, 23, x, x, x, 6, 13]	intState=100	inta=0
t=25	pc=	13	RF=[13, 13, 23, x, x, x, 6, 13]	intState=000	inta=0
t=27	pc=	14	RF=[13, 13, 23, x, x, x, 6, 13]	intState=000	inta=0
t=29	pc=	15	RF=[13, 13, 23, x, x, x, 5, 13]	intState=000	inta=0
t=31	pc=	16	RF=[13, 13, 23, x, x, x, 5, 13]	intState=000	inta=0
t=33	pc=	17	RF=[13, 13, 23, x, x, x, 5, 13]	intState=000	inta=0
t=35	pc=	18	RF=[13, 13, 23, x, x, x, 5, 13]	intState=000	inta=0
t=37	pc=	5	RF=[13, 13, 23, x, x, x, 5, 13]	intState=000	inta=0
t=39	pc=	6	RF=[13, 13, 23, x, x, x, 5, 13]	intState=000	inta=0
t=41	pc=	7	RF=[13, 13, 23, x, x, x, 5, 13]	intState=000	inta=0
t=43	pc=	8	RF=[13, 13, 23, x, 33, x, 5, 13]	intState=000	inta=0
t=45	pc=	9	RF=[14, 13, 23, x, 33, x, 5, 13]	intState=000	inta=0
t=47	pc=	10	RF=[14, 1, 23, x, 33, x, 5, 13]	intState=000	inta=0
t=49	pc=	11	RF=[14, 1, 222, x, 33, x, 5, 13]	intState=000	inta=0
t=51	pc=	11	RF=[14, 1, 222, x, 33, x, 5, 13]	intState=000	inta=0
t=53	pc=	11	RF=[18, 1, 222, x, 33, x, 5, 13]	intState=000	inta=0
t=55	pc=	11	RF=[18, 1, 222, x, 33, x, 5, 13]	intState=000	inta=0
t=57	pc=	11	RF=[18, 1, 222, x, 33, x, 5, 13]	intState=000	inta=0

The execution of VAL(4,33) is obvious at t=43. The program syntax does not assume this.

◇

There are few solutions for solving the previously emphasized hazard.

16.2.2.1 Stalling

By introducing a stall using a NOP instruction the sequence of instruction becomes:

```

i:      XXX      ;
i+1:    RJMP(12);
i+2:    NOP      ;
i+3: LB(2)  YYY   ;
...     ...     ;
i+j: LB(12) ZZZ   ;

```

the execution will be:

```

XXX      ;
RJMP(12);
NOP      ;
ZZZ      ;

```

This solution results in a branch penalty (the number of stalls introduced during the branch operations in the pipelined processor) of 1 cycle (the NOP introduced after RJMP(12)).

16.2.2.2 Reordering

Sometimes the branch penalty can be reduced by reordering.

Because, in the previous example, the jump is unconditional, which means the execution of the instruction XXX does not have implications on the jump instruction, the sequence of instruction can be reordered, as follows:

```

i:      RJMP(12);
i+1:    XXX      ;
i+2: LB(2)  YYY   ;
...     ...     ;
i+j: LB(12) ZZZ   ;

```

thus, executing the instruction XXX after jump no penalty is introduced.

Example 16.7 *Reordering is exemplified by the following code:*

```

NOP      ;
VAL(0,-3) ;
NOP      ;
NOP      ;
LB(1); NOP ;
NOP      ;
BRNZ(0,1) ;
ADDV(0,0,1) ;
HALT     ;
HALT     ;

```

where the increment of the register 0 is specified out of loop, but is executed associated with each branch. The result of simulation is:

```

t=0  pc=  x  RF=[x, x, x, x, x, x, x, x]
t=1  pc=1023 RF=[x, x, x, x, x, x, x, x]
t=5  pc=  0  RF=[x, x, x, x, x, x, x, x]
t=7  pc=  1  RF=[x, x, x, x, x, x, x, x]
t=9  pc=  2  RF=[x, x, x, x, x, x, x, x]
t=11 pc=  3  RF=[x, x, x, x, x, x, x, x]
t=13 pc=  4  RF=[x, x, x, x, x, x, x, x]
t=15 pc=  5  RF=[4294967293, x, x, x, x, x, x, x]
t=17 pc=  6  RF=[4294967293, x, x, x, x, x, x, x]
t=19 pc=  7  RF=[4294967293, x, x, x, x, x, x, x]
t=21 pc=  4  RF=[4294967293, x, x, x, x, x, x, x]
t=23 pc=  5  RF=[4294967293, x, x, x, x, x, x, x]
t=25 pc=  6  RF=[4294967293, x, x, x, x, x, x, x]
t=27 pc=  7  RF=[4294967294, x, x, x, x, x, x, x]
t=29 pc=  4  RF=[4294967294, x, x, x, x, x, x, x]
t=31 pc=  5  RF=[4294967294, x, x, x, x, x, x, x]
t=33 pc=  6  RF=[4294967294, x, x, x, x, x, x, x]
t=35 pc=  7  RF=[4294967295, x, x, x, x, x, x, x]
t=37 pc=  4  RF=[4294967295, x, x, x, x, x, x, x]
t=39 pc=  5  RF=[4294967295, x, x, x, x, x, x, x]
t=41 pc=  6  RF=[4294967295, x, x, x, x, x, x, x]
t=43 pc=  7  RF=[0, x, x, x, x, x, x, x]
t=45 pc=  8  RF=[0, x, x, x, x, x, x, x]
t=47 pc=  9  RF=[0, x, x, x, x, x, x, x]
t=49 pc=  9  RF=[0, x, x, x, x, x, x, x]
t=51 pc=  9  RF=[1, x, x, x, x, x, x, x]
t=53 pc=  9  RF=[1, x, x, x, x, x, x, x]
t=55 pc=  9  RF=[1, x, x, x, x, x, x, x]

```

◇

16.2.2.3 Static Branch Prediction

Let us see now how the conditioned jumps, the branches, are managed to be performed efficiently. There are two cases: backward branches or forward branches. Static prediction presumes that backward branches will be taken and that forward branches will not.

In this case, most of the predictions will be correct. Indeed, if we have a jump back to execute a loop that repeats n times, then once in $n + 1$ situations the prediction will have to be corrected.

Example 16.8 *A backward branch is one that has a target address that is lower than its own address. For example, it refers to the following type of loop:*

```

i:      VAL(1,23)    ;
i+1:    VAL(2,1)     ;
i+2:    LB(3) SUB(1,1,2) ;
i+3:    NZJMP(1,3)   ;
i+4:    NOP          ;
i+5:    NOP          ;
i+6:    ...

```


This technique can help with prediction accuracy of loops, which are usually backward-pointing branches, and are taken more often than not taken.

◇

Example 16.9 A forward branch is one that has a target address that is higher than its own address. For example, it refers to the following type of loop:

```

i:      VAL(1,23)  ;
i+1:    VAL(2,1)   ;
i+2:    SUB(1,1,2) ;
i+3:    ZJMP(1,2)  ;
i+4:    XXX        ;
...
i+j:    LB(2)  YYY ;

```

◇

In static prediction, all decisions are made at compile time, before the execution of the program

Note: In order to reduce the branch penalty, in the case of unconditional jumps, certain changes can be made in the hardware. For example, the calculation of unconditional jump addresses can be moved to the OP FETCH (OF) level. The execution latency of unconditional jumps is thus reduced by one unit.

16.2.2.4 Dynamic Branch Prediction

Dynamic branch prediction predicts branches based on dynamic information collected at run-time. It requires additional hardware. As examples, I have chosen two of the simplest prediction mechanisms.

Last-time, one-bit predictor is the simplest solution which uses a two state automaton whose behavior is described in Figure 16.4. We will code the state `prediction not taken` with 0, and the state `prediction taken` with 1. The automaton is a *saturated* 1-bit counter. If is incremented from 0 goes to 1, if is decremented from 1 goes to 0. But if it is incremented from 1, stays in 1, while decremented from 0 stays in 0. This 2-state automaton says whether the branch was recently taken or not. Based on this, the processor fetches the next instruction from the target address or sequential address. If the prediction is wrong, flushes the pipeline and also flips prediction. So, every time a wrong prediction is made, the prediction bit is flipped.

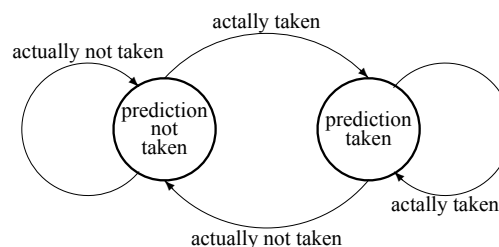


Figure 16.4: The saturated one-bit up/down counter as last-time branch predictor.

This mechanism always mispredicts the last iteration and the first iteration of a loop branch, thus the accuracy for a loop with N iterations is $(100 \times (N - 2)/N)\%$. For large values of N the accuracy of the prediction increases, while for small values this mechanism is not very efficient.

Two-Bit Counter Based Predictor is a two-state finite automaton. Its behavior is described in Figure 16.5. The advantage of the two-bit predictor over a one-bit predictor is that a conditional jump has to deviate twice from what it has done most in the past before the prediction changes. For example, a loop-closing conditional jump is mispredicted once rather than twice.

Each state of the automaton is interpreted as follows:

- 00: strongly not taken
- 01: weakly not taken
- 10: weakly taken
- 11: strongly taken

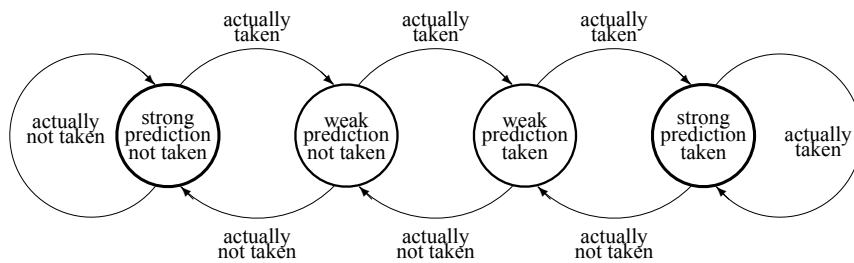


Figure 16.5: Two-bit saturated counter based predictor.

The automaton is a *saturated* 2-bit counter. It increments for *actually taken* and decrements for *actually not taken*. This predictor changes prediction only on two successive mispredictions.

16.3 Programming Pipelined Processors

Example 16.10 *The assembly program, written for the generic version of toyRISC, changes the sign of the number stored in `rf[2]` provides in `rf[3]` -7 and then in `rf[1]` back the value of 7.*

```

VAL(2, 7);
CHSGN(3,2);
CHSGN(1,3);
HALT;

```

The perform the same operation in the pipelined version without the forwarding mechanism, the program must take into account the data dependency, is:

```

VAL(2, 7);
NOP;
NOP;

```

```

CHSGN(3,2);
NOP;
NOP;
CHSGN(1,3);
HALT;
HALT;

```

Adding the forwarding mechanism, the program becomes:

```

VAL(2, 7);
CHSGN(3,2);
CHSGN(1,3);
HALT;
HALT;

```

◇

Example 16.11 *The program which establish by $\text{rf}[3] = 1$ that the content of $\text{rf}[0]$ is divisible by 4, else $\text{rf}[3] = 0$, is:*

```

VAL(0,8);
VAL(1,3);
AND(2,0,1);
NOP;
NOP;
BRZ(2,22);
NOP;
RJMP(33);
VAL(3,0);
LB(22); VAL(3,1);
LB(33); HALT;
HALT;

```

If it runs preceded by:

```
VAL(0,9);
```

then the result is $\text{rf}[3] = 0$, else if it is preceded by:

```
VAL(0,8);
```

then the result is $\text{rf}[3] = 1$.

The first two NOPs in the program are introduced because the forwarding mechanism is implemented only for the operands of the ALU, not for testing the value at the output of the register file by the circuit zero. The third NOP is due to the control dependency.

◇

Example 16.12 *The program which compute in $\text{rf}[4]$ the mean values stored in $\text{rf}[0]$, $\text{rf}[1]$, $\text{rf}[2]$, $\text{rf}[3]$ is:*

```

ADD(4,0,1);
ADD(4,4,2);
ADD(4,4,3);
ASH(4,4);
ASH(4,4);
HALT;
HALT;

```

If it runs preceded by:

```

VAL(0,10);
VAL(1,12);
VAL(2,3);
VAL(3,44);

```

then the result if `rf[4] = 17`.

◇

16.4 Superscalar Processor

If the processor is designed with several execution units (for example: one integer ALU, two floating-point units, two load/store units), the potential parallelism thus obtained should be activated at the highest possible level. This means that in each clock cycle a maximum number of resources should be active, if possible all of them.

In this case, the processor, called *superscalar*, can execute instructions in an order imposed by the availability of data and execution units, rather than by their initial sequence in the program. For example, the *PowerPC 970* processor fetches and decodes up to eight instructions, dispatch up to five to reserve stations (register files), issue up to eight to the execution units and retire up to five per cycle.

16.4.1 Register renaming

The sequence in which the instructions are executed must be strictly respected when there are data dependencies. But when certain independent sequences can be identified, strictly sequential execution is no longer mandatory, especially in the case of a superscalar processor that allows parallelism at the level of instructions. We can apply in such cases the *register renaming* technique.

Example 16.13 Consider this sequence of code running on an out-of-order processor:

```

READ(0,4)    ;    // read in rf[0] from the address in rf[4]
ADD(0,0,12)   ;    // rf[0] <= rf[0] + rf[12]
STORE(0,7)    ;    // dataMemory[rf[7]] <= rf[0]
READ(0,5)     ;
ADD(0,0,6)    ;
STORE(0,21)   ;

```

The instructions in the last three lines are **independent** of the first three lines. The processor cannot execute `STORE(0,21)` until `STORE(0,7)` is done. We can eliminate this restriction involving in our code a supplementary register, as follows:

```

READ(0,4)    ;    // read in rf[0] from the address in rf[4]
ADD(0,0,12)  ;    // rf[0] <= rf[0] + rf[12]
STORE(0,7)   ;    // dataMemory[rf[7]] <= rf[0]
READ(2,5)    ;
ADD(2,2,6)   ;
STORE(2,21)  ;

```

The first three instructions involves `rf[0]`, while the last three instructions work on the register `rf[2]`, allowing the last three instructions to be executed in parallel with the first three.

◇

16.4.2 Out-of-Order Execution

Out-of-order execution is a mechanism used in high-performance processing units to make use efficiently of instruction cycles. Using this mechanism, a processor executes instructions in an order governed by the availability of input data and execution units, rather than by their original order in a program. Thus the processor avoids being idle while waiting for the preceding instruction to complete and can, in the meantime, proceed to execution of the next instructions that are able to run immediately and independently. The mechanism is efficient for a processor with multiple execution units **with speed difference between instructions**. It is the case of superscalar processor with ISA performing integer and floating-point arithmetic operations. There are also speed difference between arithmetic and logic instructions and memory access instructions.

While in a pipelined scalar processor an instruction is executed in the following steps:

- 1) Instruction fetch.
- 2) Operands fetch, if input operands are available in processor's register file, else the processor stalls until they are available.
- 3) The instruction is executed.
- 4) The results is write back to the register file.

in a superscalar processor the out-of-order execution is requested. It consists of the following steps:

- 1) Instruction fetch.
- 2) Instruction dispatch to an instruction queue (also called instruction buffer or reservation stations).
- 3) The instruction waits in the queue until its input operands are available. The instruction can leave the queue before older instructions.
- 4) The instruction is issued to the appropriate idle functional unit.
- 5) The results are queued.
- 6) Only after all older instructions have their results written back to the register file, then this result is written back to the register file.

The benefit of out-of-order execution processing grows as the instruction pipeline deepens and the speed difference between main memory or cache memory and the processor widens. On modern machines, the processor runs few times faster than the cache memory and many times faster than system memory, so during the time an in-order processor waits for data, a large number of instructions could be executed.

Example 16.14 *In the following sequence of instructions (see Patterson and Hennessy (2005), p. 185):*

```
DIV(0,2,4) ;
ADD(6,0,8) ;
STR(6,1)   ; // store
SUB(8,9,7) ;
MUL(6,9,8) ;
```

here are the following dependencies:

- 1) *between ADD and SUB if SUB finishes before ADD starts (a WAR hazard) is an antidependence*
- 2) *between ADD and MUL if ADD finishes later than MUL (WAW hazard) is an output dependence*
- 3) *between DIV and ADD a true data dependency (a RAW hazard)*
- 4) *between SUB and MUL a true data dependency (a RAW hazard)*
- 5) *between ADD and STORE a true data dependency (a RAW hazard)*

First, let us apply register renaming technique involving two additional registers, rf(10) and rf(11):

```
DIV(0, 2, 4) ;
ADD(10,0, 8) ;
STR(10,1)   ;
SUB(11,9, 7) ;
MUL(6, 9, 11) ;
```

Any WAR and WAW dependencies are now removed statically by the compiler. For RAW dependency the forwarding mechanism works but only in a processor with one integer ALU. What can be done for a superscalar processor with floating point arithmetic?

◇

The hardware added for out-of-order execution is big sized and complex. With the increase in the number of pipeline levels, the costs (area plus complexity in use) increase. Multiprocessing and multi-threading will allow more efficient alternative solutions.

16.4.2.1 Floating-point representation of real numbers

To expand the range in which the numbers are represented, the following form is used:

$$N = \text{sgn}(1 + \text{frac}) \times 2^{\text{exp}-127}$$

where:

$$sgn \in \{-, +\} = \{0, 1\}$$

$$frac \in [0, 1)$$

$$exp \in [0, 255]$$

The binary representation is:

$$\langle sgn|exp|frac \rangle$$

For example the 32-bit version:

0_10000001_100000000000000000000000

represents the number: $+(1 + 0.5) \times 2^{129-127} = 6$

More at: <https://www.geeksforgeeks.org/ieee-standard-754-floating-point-numbers/>

For the purpose we are pursuing, it is enough to understand that the operation with represented floating-point numbers involves a pipeline or a sequence of elementary operations of one clock cycle. The number of cycles will be different depending on the operation, minimum for addition and maximum for division.

16.4.2.2 Tomasulo's Algorithm

The execution unit of a scalar processor has, in addition to the ALU for integers that we discussed, several units that perform operations with floating-point numbers. For out-of-order execution, it is not enough to have a register file. Next to each execution unit, for whole or real, complex storage units called Reservation Stations (RS) will be added. Each recording in a RS has the following fields Patterson and Hennessy (2019):

- Op—The operation to perform on source operands S1 and S2.
- Qj, Qk—The reservation stations that will produce the corresponding source operand; a value of zero indicates that the source operand is already available in Vj or Vk, or is unnecessary.
- Vj, Vk—The value of the source operands. Note that only one of the V fields or the Q field is valid for each operand. For loads, the Vk field is used to hold the offset field.
- A—Used to hold information for the memory address calculation for a load or store. Initially, the immediate field of the instruction is stored here; after the address calculation, the effective address is stored here.
- Busy—Indicates that this reservation station and its accompanying functional unit are occupied.

while the register file has a field, Qi:

- Qi—The number of the reservation station that contains the operation whose result should be stored into this register. If the value of Qi is blank (or 0), no currently active instruction is computing a result destined for this register, meaning that the value is simply the register contents.

16.5 Problems

16.5.1

16.5.2

Chapter 17

ToyRISC Design

The organization of the simple executive processor *toyRISC* (described in `toyRISC.v` module of the design listed in Figure 17.2) is given in Figure 11.6, where two simple automata, the **RALU** (**R**egisters with **A**rithmetic-**L**ogic **U**nit) subsystem and the **Control** subsystem are loop connected thus closing the 3rd loop which allow the emergence of the processing function. The only complex subsystems are two small modules: the decode circuit (described in `Decode.v` and the interrupt automaton, `intSect.v`.

17.1 Code

The design has a multi-level organization

17.1.1 First Level

The top level

```

/*****
File name: 01_theSystem.v
Description: a RISC processor and its memories
*****/
module theSystem ( input      reset      ,
                   input      interrupt  ,
                   input  [15:0] intAddr  ,
                   output     inta       ,
                   input      clk        );

    wire      procWr      ;
    wire  [31:0] memOut    ;
    wire  [31:0] procOut   ;
    wire  [31:0] dataAddr  ;
    wire  [31:0] instr     ;
    wire  [15:0] instrAddr ;

    toyRISC PROCESSOR( instrAddr , // to program memory
                      instr      , // from program memory
                      dataAddr   , // data memory address
                      procOut     , // data to data memory
                      memOut      , // data from data memory
                      procWr      , // write enable
                      reset       ,
                      interrupt    ,
                      intAddr      ,
                      inta         ,
                      clk          );

    memories MEMORIES( instr      ,
                      instrAddr   ,
                      memOut      ,
                      procOut     ,
                      dataAddr     ,
                      procWr       ,
                      clk          );

endmodule

```

Figure 17.1: 01_theSystem

17.1.2 Second Level

```

/*****
File name: 02_toyRISC.v
Description: a RISC processor
*****/
module toyRISC(output  [15:0] instrAddr  , // to program memory
               input   [31:0] instr     , // from program memory
               output  [31:0] dataAddr   , // data memory address
               output  [31:0] procOut    , // data to data memory
               input   [31:0] memOut     , // data from data memory
               output  procWr           , // write enable
               input   reset            ,
               input   interrupt        ,
               input   [15:0] intAddr    ,
               output  inta             ,
               input   clk              );

    wire writeEnable ;
    wire [15:0] address ;
    wire [31:0] leftOp ;

    Decode Decode( .procWr      (procWr      ),
                  .writeEnable(writeEnable),
                  .instr        (instr        ),
                  .inta         (inta         ));

    Control Control(instrAddr ,
                   instr      ,
                   address    ,
                   leftOp     ,
                   reset      ,
                   interrupt   ,
                   intAddr     ,
                   inta        ,
                   clk         );

    RALU RALU( instr      ,
              dataAddr    ,
              procOut      ,
              memOut       ,
              address      ,
              leftOp       ,
              writeEnable  ,
              inta         ,
              clk          );

endmodule

```

Figure 17.2: 02_toyRISC

```

/*****
File name: 02_memories.v
Description: data memory and program memory used to simulate the
            behavior of toyRISC
*****/
module memories(output [31:0] instr      ,
               input   [15:0] instrAddr ,
               output [31:0] memOut     ,
               input   [31:0] procOut   ,
               input   [31:0] dataAddr  ,
               input   [31:0] procWr    ,
               input   clk             );

    programMemory  PROG_MEM( .instr      (instr      ),
                             .instrAddr  (instrAddr   ),
                             .clk        (clk         ));

    memory DATA_MEM(memOut  ,
                     procOut ,
                     dataAddr,
                     procWr  ,
                     clk     );

endmodule

```

Figure 17.3: 02_memories

17.1.3 Third Level

```

/*****
File name: 03_Control.v
Description:
*****/
#include "0_ISAdefine.vh"
module Control(output [15:0] instrAddr ,
               input  [31:0] instruction ,
               output [15:0] address   ,
               input  [31:0] leftOp    ,
               input          reset     ,
               input          interrupt ,
               input  [15:0] intAddr   ,
               output          inta     ,
               input          clk       );

    // Interrupt section
    reg intEnable ;

    always @(posedge clk)
        if (reset)                intEnable <= 0 ;
        else if (instruction[31:26] == 'ei) intEnable <= 1 ;
        else if (instruction[31:26] == 'di) intEnable <= 0 ;
        else if (interrupt)        intEnable <= 0 ;

    assign inta = intEnable & interrupt ;

    // Program counter section
    reg [15:0] pc ;

    always @(posedge clk) if (reset) pc <= -1 ;
                          else      pc <= instrAddr ;

    nextPc nextPc( .addr    (instrAddr      ),
                   .address (address        ),
                   .pc       (pc             ),
                   .jmpVal   (instruction[15:0]),
                   .leftOp   (leftOp         ),
                   .opCode   (instruction[31:26]),
                   .inta     (inta           ),
                   .intAddr   (intAddr        ));
endmodule

```

Figure 17.4:

```

/*****
File name: 03_RALU.v
Description:
*****/
#include "0_ISAdefine.vh"
module RALU(
    input  [31:0]  instr      ,
    output [31:0]  dataAddr   ,
    output [31:0]  procOut    ,
    input  [31:0]  memOut     ,
    input  [15:0]  address    ,
    output [31:0]  leftOp     ,
    input         writeEnable ,
    input         inta        ,
    input         clk         );

    wire [31:0] aluOut  ;
    wire [31:0] rightOp ;
    wire [31:0] regFileIn;

    assign dataAddr = rightOp ;
    assign procOut  = leftOp  ;
    assign callIns  = instr[31:26] == 'call ;

    regFile
    regFile(.leftOut  (leftOp
                        ),
            .rightOut (rightOp
                        ),
            .in       (regFileIn
                        ),
            .leftAddr (instr[20:16]
                        ),
            .rightAddr(instr[15:11]
                        ),
            .destAddr ((inta || callIns) ? 5'b11111 : instr[25:21]),
            .writeEnable(writeEnable | inta
                        ),
            .clk         (clk
                        ));

    mux4_32 mux(.out(regFileIn
                    ),
               .in0({16'b0, address}
                    ),
               .in1({16{instr[15]}}, instr[15:0]
                    ),
               .in2(memOut
                    ),
               .in3(aluOut
                    ),
               .sel(instr[31:30] & {!inta, !inta}
                    ));

    alu alu(.out      (aluOut
                    ),
            .leftIn   (leftOp
                    ),
            .rightIn  (rightOp
                    ),
            .func      (instr[31:26]
                    ));
endmodule

```

Figure 17.5:

```

/*****
File name: 03_Decode.v
Description:
*****/
#include "0_ISAdefine.vh"
module Decode( output      procWr      , // data memory write enable
               output      writeEnable , // register file write enable
               input  [31:0] instr      ,
               input      inta          );

    assign procWr      = instr[31:26] == 'store      ;
    assign writeEnable = &instr[31:30] | &instr[28:26] | inta ;
endmodule

```

Figure 17.6:

```

/*****
File name: 03_programMemory.v
Description:
*****/
module programMemory( output reg [31:0] instr      ,
                     input      [15:0] instrAddr    ,
                     input      clk                 );
    reg [31:0] progMem[0:1023] ;

    always @(posedge clk) instr <= progMem[instrAddr[9:0]] ;
endmodule

```

Figure 17.7:


```
/******  
File name: 03_memory.v  
Description:  
*****/  
module memory(  output  [31:0]  memOut  ,  
                input   [31:0]  procOut ,  
                input   [31:0]  dataAddr ,  
                input           procWr  ,  
                input           clk    );  
    reg [31:0]  mem[0:1023] ;  
  
    assign memOut = mem[dataAddr[9:0]] ;  
    always @(posedge clk) if (procWr)  
        mem[dataAddr[9:0]] <= procOut ;  
endmodule
```

Figure 17.8:

17.1.4 Fourth Level

```

/*****
File name: 04_nextPc.v
Description:
*****/
#include "0_ISAdefine.vh"
module nextPc( output reg [15:0] addr ,
               output reg [15:0] address ,
               input  [15:0] pc ,
               input  [15:0] jmpVal ,
               input  [31:0] leftOp ,
               input  [5:0] opCode ,
               input      inta ,
               input  [15:0] intAddr );

    always @(*)
        if (inta)
            begin
                addr = intAddr ;
                address = pc + 1 ;
            end
        else case(opCode)
            'nop :      addr = pc + 1 ;
            'rjmp :      addr = pc + jmpVal ;
            'zjmp :      addr = (leftOp == 0) ? pc + jmpVal : pc + 1 ;
            'nzjmp :     addr = (leftOp == 0) ? pc + 1 : pc + jmpVal ;
            'ret :       addr = leftOp ;
            'ajmp :      addr = jmpVal ;
            'call :      begin
                            addr = jmpVal ;
                            address = pc + 1 ;
                        end
            default :    addr = pc + 1 ;
        endcase
    endmodule

```

Figure 17.9:

```

/*****
File name: 04_alu.v
Description:
*****/
#include "0_ISAdefine.vh"
module alu(output reg [31:0] out ,
           input [31:0] leftIn ,
           input [31:0] rightIn ,
           input [5:0] func );

    wire [32:0] sum ;
    wire [32:0] dif ;
    wire [31:0] mult;

    assign sum = leftIn + rightIn ;
    assign dif = leftIn - rightIn ;
    assign mult = leftIn[15:0] * rightIn[15:0] ;

    always @(*)
        case(func)
            'bigv : out = {leftIn[15:0], rightIn[15:0]};
            'add : out = sum[31:0] ;
            'sub : out = dif[31:0] ;
            'addcr : out = sum[32] ;
            'subcr : out = dif[32] ;
            'lsh : out = leftIn >> 1 ;
            'ash : out = {leftIn[31], leftIn[31:1]} ;
            'move : out = leftIn ;
            'mult : out = mult ;
            'bwand : out = leftIn & rightIn ;
            'bwor : out = leftIn | rightIn ;
            'bwxor : out = leftIn ^ rightIn ;
            default : out = leftIn ;
        endcase
endmodule

```

Figure 17.10:

```

/*****
File name: 04_regFile.v
Description:
*****/
module regFile( output [31:0] leftOut      ,
                output [31:0] rightOut     ,
                input  [31:0] in           ,
                input  [4:0] leftAddr      ,
                input  [4:0] rightAddr     ,
                input  [4:0] destAddr      ,
                input                writeEnable ,
                input                clk      );

    reg [31:0] registers[0:31] ;

    always @(posedge clk)
        if (writeEnable) registers[destAddr] <= in ;

    assign leftOut  = registers[leftAddr] ;
    assign rightOut = registers[rightAddr] ;
endmodule

```

Figure 17.11:

```

/*****
File name: 04_mux4_32.v
Description:
*****/
module mux4_32( output reg [31:0] out ,
                input      [31:0] in0 ,
                input      [31:0] in1 ,
                input      [31:0] in2 ,
                input      [31:0] in3 ,
                input      [1:0] sel );

    always @(*) case(sel)
        2'b00: out = in0 ;
        2'b01: out = in1 ;
        2'b10: out = in2 ;
        2'b11: out = in3 ;
    endcase
endmodule

```

Figure 17.12:

17.2 Assembly

```

/*****
File name: 00_toyRISCcodeGenerator.v
Description:
*****/
reg [5:0]   RISCopCode      ;
reg [4:0]   RISCdest        ;
reg [4:0]   RISCleft        ;
reg [4:0]   RISCright       ;
reg [10:0]  RISCvalue       ;
reg [6:0]   RISCaddrCounter ;
reg [6:0]   RISClableTab[0:128] ;

`include "0_ISAdefine.vh"

task hendLine;
begin
    dut.MEMORIES.PROG_MEM.progMem[ RISCaddrCounter ] =
        {
            RISCopCode ,
            RISCdest ,
            RISCleft ,
            RISCright ,
            RISCvalue } ;
    RISCaddrCounter = RISCaddrCounter + 1 ;
end
endtask

// sets RISClableTab in the first pass associating
// 'RISCaddrCounter' with 'labelIndex'
task LB ;
    input [4:0] labelIndex ;

    RISClableTab[labelIndex] = RISCaddrCounter;
endtask
// uses the content of RISClableTab in the second pass for rjmp
task ULB;
    input [4:0] labelIndex ;

    {RISCright, RISCvalue} =
        RISClableTab[labelIndex] - RISCaddrCounter;
endtask
// uses the content of RISClableTab in the second pass for call
task CLB;
    input [4:0] labelIndex ;

    {RISCright, RISCvalue} =
        RISClableTab[labelIndex];
endtask

task HALT;
begin
    RISCopCode = 'rjmp ;
    RISCdest   = 0 ;
    RISCleft   = 0 ;

```

```

        RISCright = 0 ;
        RISCvalue = 0 ;
        hendLine ;
    end
endtask

task NOP;
    begin
        RISCopCode = 'nop ;
        RISCdest = 0 ;
        RISCleft = 0 ;
        RISCright = 0 ;
        RISCvalue = 0 ;
        hendLine ;
    end
endtask

task RJMP;
    input [5:0] label ;

    begin
        RISCopCode = 'rjmp ;
        RISCdest = 0 ;
        RISCleft = 0 ;
        ULB(label) ;
        hendLine ;
    end
endtask

task ZJMP;
    input [4:0] leftAddr;
    input [5:0] label ;

    begin
        RISCopCode = 'zjmp ;
        RISCdest = 0 ;
        RISCleft = leftAddr ;
        ULB(label) ;
        hendLine ;
    end
endtask

task NZJMP;
    input [4:0] leftAddr;
    input [5:0] label ;

    begin
        RISCopCode = 'nzjmp ;
        RISCdest = 0 ;
        RISCleft = leftAddr ;
        ULB(label) ;
        hendLine ;
    end
endtask

task RET;
    input [4:0] leftAddr;

    begin
        RISCopCode = 'ret ;

```

```

        RISCdest    = 0          ;
        RISCleft    = 5'b11111  ;
        RISCright   = 0          ;
        RISCvalue   = 0          ;
        hendLine    ;
    end
endtask

task AJMP;
    input [10:0] value ;

    begin
        RISCopCode    = 'ajmp;
        RISCdest      = 0      ;
        RISCleft       = 0      ;
        {RISCright, RISCvalue} = value ;
        hendLine      ;
    end
endtask

task CALL;
    input [5:0] label ;

    begin
        RISCopCode    = 'call;
        RISCdest      = 0      ;
        RISCleft       = 0      ;
        CLB(label)    ;
        hendLine      ;
    end
endtask

task BIGV;
    input [4:0] destAddr ;
    input [4:0] leftAddr ;
    input [4:0] rightAddr ;

    begin
        RISCopCode    = 'bigv ;
        RISCdest      = destAddr ;
        RISCleft       = leftAddr ;
        RISCright      = rightAddr ;
        RISCvalue      = 11'b0 ;
        hendLine      ;
    end
endtask

task ADD;
    input [4:0] destAddr ;
    input [4:0] leftAddr ;
    input [4:0] rightAddr ;

    begin
        RISCopCode    = 'add ;
        RISCdest      = destAddr ;
        RISCleft       = leftAddr ;
        RISCright      = rightAddr ;
        RISCvalue      = 11'b0 ;
        hendLine      ;
    end
endtask

```

```

    end
endtask

task SUB;
    input [4:0] destAddr    ;
    input [4:0] leftAddr    ;
    input [4:0] rightAddr   ;

    begin
        RISCopCode = 'sub    ;
        RISCdest   = destAddr ;
        RISCleft   = leftAddr  ;
        RISCright  = rightAddr ;
        RISCvalue  = 11'b0    ;
        hendLine   ;
    end
endtask

task CRADD;
    input [4:0] destAddr    ;
    input [4:0] leftAddr    ;
    input [4:0] rightAddr   ;

    begin
        RISCopCode = 'addcr   ;
        RISCdest   = destAddr ;
        RISCleft   = leftAddr  ;
        RISCright  = rightAddr ;
        RISCvalue  = 11'b0    ;
        hendLine   ;
    end
endtask

task CRSUB;
    input [4:0] destAddr    ;
    input [4:0] leftAddr    ;
    input [4:0] rightAddr   ;

    begin
        RISCopCode = 'subcr   ;
        RISCdest   = destAddr ;
        RISCleft   = leftAddr  ;
        RISCright  = rightAddr ;
        RISCvalue  = 11'b0    ;
        hendLine   ;
    end
endtask

task LEFTSH; // logic left shift
    input [4:0] destAddr    ;
    input [4:0] leftAddr    ;

    begin
        RISCopCode = 'lsh     ;
        RISCdest   = destAddr ;
        RISCleft   = leftAddr  ;
        RISCright  = 5'b0     ;
        RISCvalue  = 11'b0    ;
        hendLine   ;
    end

```



```

    end
endtask

task ARITHSH; // arithmetic shift
    input [4:0] destAddr;
    input [4:0] leftAddr;

    begin
        RISCopCode = 'ash      ;
        RISCdest   = destAddr ;
        RISCleft   = leftAddr  ;
        RISCright  = 0         ;
        RISCvalue  = 0         ;
        hendLine   ;
    end
endtask

task MOVE;
    input [4:0] destAddr;
    input [4:0] leftAddr;

    begin
        RISCopCode = 'move ;
        RISCdest   = destAddr ;
        RISCleft   = leftAddr  ;
        RISCright  = 0         ;
        RISCvalue  = 0         ;
        hendLine   ;
    end
endtask

task MULT;
    input [4:0] destAddr ;
    input [4:0] leftAddr  ;
    input [4:0] rightAddr ;

    begin
        RISCopCode = 'mult ;
        RISCdest   = destAddr ;
        RISCleft   = leftAddr  ;
        RISCright  = rightAddr ;
        RISCvalue  = 0         ;
        hendLine   ;
    end
endtask

task AND;
    input [4:0] destAddr ;
    input [4:0] leftAddr  ;
    input [4:0] rightAddr ;

    begin
        RISCopCode = 'bwand ;
        RISCdest   = destAddr ;
        RISCleft   = leftAddr  ;
        RISCright  = rightAddr ;
        RISCvalue  = 0         ;
        hendLine   ;
    end
end

```

```

endtask

task OR;
    input [4:0] destAddr    ;
    input [4:0] leftAddr   ;
    input [4:0] rightAddr  ;

    begin    RISCopCode = 'bwor ;
             RISCdest  = destAddr ;
             RISCleft  = leftAddr ;
             RISCright = rightAddr ;
             RISCvalue = 0        ;
             hendLine   ;

    end
endtask

task XOR;
    input [4:0] destAddr    ;
    input [4:0] leftAddr   ;
    input [4:0] rightAddr  ;

    begin    RISCopCode = 'bwxor ;
             RISCdest  = destAddr ;
             RISCleft  = leftAddr ;
             RISCright = rightAddr ;
             RISCvalue = 11'b0    ;
             hendLine   ;

    end
endtask

task READ;
    input [4:0] destAddr;
    input [4:0] rightAddr;

    begin    RISCopCode = 'read ;
             RISCdest  = destAddr ;
             RISCleft  = 5'b0     ;
             RISCright = rightAddr ;
             RISCvalue = 11'b0    ;
             hendLine   ;

    end
endtask

task STORE;
    input [4:0] leftAddr    ;
    input [4:0] rightAddr  ;

    begin    RISCopCode = 'store ;
             RISCdest  = 5'b0    ;
             RISCleft  = leftAddr ;
             RISCright = rightAddr ;
             RISCvalue = 11'b0    ;
             hendLine   ;

    end
endtask

```

```

task VAL;
  input [4:0]    destAddr;
  input [15:0]   value   ;

  begin   RISCopCode = 'val           ;
          RISCdest   = destAddr       ;
          RISCleft   = 5'b0           ;
          {RISCright, RISCvalue} = value ;
          hendLine   ;

  end
endtask

task EINT; // enable interrupt
  begin   RISCopCode = 'ei   ;
          RISCdest   = 0     ;
          RISCleft   = 0     ;
          RISCright  = 0     ;
          RISCvalue  = 0     ;
          hendLine   ;

  end
endtask

task DINT; // disable interrupt
  begin   RISCopCode = 'di   ;
          RISCdest   = 0     ;
          RISCleft   = 0     ;
          RISCright  = 0     ;
          RISCvalue  = 0     ;
          hendLine   ;

  end
endtask

// RUNNING
initial begin   RISCaddrCounter = 0;
                'include "00_toyRISCprogram.v" // first pass
                RISCaddrCounter = 0;
                'include "00_toyRISCprogram.v" // second pass

end

```

17.3 Simulation

```

/*****
File name: 00_toyRiscSimulator.v
Description:
*****/
// 'include "DEFINES.vh"
module toyRiscSimulator;
  reg      reset, interrupt, clk   ;
  reg [15:0] intAddr              ;

```

```
wire          inta      ;

integer i;

initial begin          clk = 0        ;
                    forever #1    clk = ~clk   ;
end

`include "00_toyRISCcodeGenerator.v"

initial for (i=0; i<32; i=i+1)
$display("programMemory[%0d]\t\t=\t%b",
         i , dut.MEMORIES.PROG_MEM.progMem[i]);

initial
begin
reset           = 1            ;
intAddr         = 16'b1111     ;
interrupt       = 0            ;
#4              reset         = 0            ;
#20             interrupt     = 1            ;
#150            begin
// DISPLAY THE CONTENT OF THE REGISTER FILE
for (i=0; i<8; i=i+1)
$display("RF[%0d]\t\t=\t%b",
         i , dut.PROCESSOR.RALU.regFile.registers[i]);
// DISPLAY THE DATA MEMORY OF HOST
for (i=0; i<8; i=i+1)
$display("dataMemory[%0d]\t\t=\t%b",
         i , dut.MEMORIES.DATA_MEM.mem[i]);
end
#2 $stop ;
end

theSystem dut(reset , interrupt , intAddr , inta , clk);

initial begin
$monitor
("t=%0d\t\t\tpc=%0d\t\t\tinta=%0d\t\t\t
XXXXXXXXXX RF[0]=%0d\t\tRF[1]=%0d\t\tRF[2]=%0d\t\tRF[3]=%0d\t\t...
XXXXXXXXXX \t\tRF[31]=%0d",
$time ,
dut.PROCESSOR.Control.pc ,
dut.PROCESSOR.Control.address ,
dut.PROCESSOR.inta ,
dut.PROCESSOR.RALU.regFileIn ,
dut.PROCESSOR.RALU.regFile.registers[0] ,
dut.PROCESSOR.RALU.regFile.registers[1] ,
dut.PROCESSOR.RALU.regFile.registers[2] ,
dut.PROCESSOR.RALU.regFile.registers[3] ,
dut.PROCESSOR.RALU.regFile.registers[31]);
end
endmodule
```

```

/*****
File name: 00_toyRISCprogram.v
Description: at #20 the simulator generate an interrupt
*****/
        VAL(0,11);
        VAL(1,1);
        EINT;
LB(0);   SUB(0,0,1);
        NZJMP(0,0);
        CALL(1);
        NOP;
        NOP;
        VAL(2,33);
        HALT;
LB(1);   VAL(0,66);
        RET;
        NOP;
        NOP;
        NOP;
        NOP;
        VAL(3,13);
        NOP;
        RET;

```

Figure 17.13:

```

*****
The binary form of program provided by 00_toyRISCcodeGenerator.v
*****
programMemory[0]   = 010111000000000000000000000001011
programMemory[1]   = 01011100001000000000000000000001
programMemory[2]   = 01010100000000000000000000000000
programMemory[3]   = 1100110000000000000000010000000000
programMemory[4]   = 00001100000000001111111111111111
programMemory[5]   = 000111000000000000000000000001010
programMemory[6]   = 00000000000000000000000000000000
programMemory[7]   = 00000000000000000000000000000000
programMemory[8]   = 010111000100000000000000000100001
programMemory[9]   = 00000100000000000000000000000000
programMemory[10]  = 01011100000000000000000001000010
programMemory[11]  = 00010100000111110000000000000000
programMemory[12]  = 00000000000000000000000000000000
programMemory[13]  = 00000000000000000000000000000000
programMemory[14]  = 00000000000000000000000000000000
programMemory[15]  = 00000000000000000000000000000000
programMemory[16]  = 010111000110000000000000000001101
programMemory[17]  = 00000000000000000000000000000000
programMemory[18]  = 00010100000111110000000000000000

*****
The result of simulation provided by the monitor run by 00_toyRiscSimulator.v
*****

```

t=0	pc=x	inta=0	RF[0]=x	RF[1]=x	RF[2]=x	RF[3]=x	...	RF[31]=x
t=1	pc=65535	inta=0	RF[0]=x	RF[1]=x	RF[2]=x	RF[3]=x	...	RF[31]=x
t=5	pc=0	inta=0	RF[0]=11	RF[1]=x	RF[2]=x	RF[3]=x	...	RF[31]=x
t=7	pc=1	inta=0	RF[0]=11	RF[1]=x	RF[2]=x	RF[3]=x	...	RF[31]=x
t=9	pc=2	inta=0	RF[0]=11	RF[1]=1	RF[2]=x	RF[3]=x	...	RF[31]=x
t=11	pc=3	inta=0	RF[0]=11	RF[1]=1	RF[2]=x	RF[3]=x	...	RF[31]=x
t=13	pc=4	inta=0	RF[0]=10	RF[1]=1	RF[2]=x	RF[3]=x	...	RF[31]=x
t=15	pc=3	inta=0	RF[0]=10	RF[1]=1	RF[2]=x	RF[3]=x	...	RF[31]=x
t=17	pc=4	inta=0	RF[0]=9	RF[1]=1	RF[2]=x	RF[3]=x	...	RF[31]=x
t=19	pc=3	inta=0	RF[0]=9	RF[1]=1	RF[2]=x	RF[3]=x	...	RF[31]=x
t=21	pc=4	inta=0	RF[0]=8	RF[1]=1	RF[2]=x	RF[3]=x	...	RF[31]=x
t=23	pc=3	inta=0	RF[0]=8	RF[1]=1	RF[2]=x	RF[3]=x	...	RF[31]=x
t=24	pc=3	inta=1	RF[0]=8	RF[1]=1	RF[2]=x	RF[3]=x	...	RF[31]=x
t=25	pc=15	inta=0	RF[0]=8	RF[1]=1	RF[2]=x	RF[3]=x	...	RF[31]=4
t=27	pc=16	inta=0	RF[0]=8	RF[1]=1	RF[2]=x	RF[3]=x	...	RF[31]=4
t=29	pc=17	inta=0	RF[0]=8	RF[1]=1	RF[2]=x	RF[3]=13	...	RF[31]=4
t=31	pc=18	inta=0	RF[0]=8	RF[1]=1	RF[2]=x	RF[3]=13	...	RF[31]=4
t=33	pc=4	inta=0	RF[0]=8	RF[1]=1	RF[2]=x	RF[3]=13	...	RF[31]=4
t=35	pc=3	inta=0	RF[0]=8	RF[1]=1	RF[2]=x	RF[3]=13	...	RF[31]=4
t=37	pc=4	inta=0	RF[0]=7	RF[1]=1	RF[2]=x	RF[3]=13	...	RF[31]=4
t=39	pc=3	inta=0	RF[0]=7	RF[1]=1	RF[2]=x	RF[3]=13	...	RF[31]=4
t=41	pc=4	inta=0	RF[0]=6	RF[1]=1	RF[2]=x	RF[3]=13	...	RF[31]=4
t=43	pc=3	inta=0	RF[0]=6	RF[1]=1	RF[2]=x	RF[3]=13	...	RF[31]=4
t=45	pc=4	inta=0	RF[0]=5	RF[1]=1	RF[2]=x	RF[3]=13	...	RF[31]=4
t=47	pc=3	inta=0	RF[0]=5	RF[1]=1	RF[2]=x	RF[3]=13	...	RF[31]=4
t=49	pc=4	inta=0	RF[0]=4	RF[1]=1	RF[2]=x	RF[3]=13	...	RF[31]=4
t=51	pc=3	inta=0	RF[0]=4	RF[1]=1	RF[2]=x	RF[3]=13	...	RF[31]=4
t=53	pc=4	inta=0	RF[0]=3	RF[1]=1	RF[2]=x	RF[3]=13	...	RF[31]=4
t=55	pc=3	inta=0	RF[0]=3	RF[1]=1	RF[2]=x	RF[3]=13	...	RF[31]=4
t=57	pc=4	inta=0	RF[0]=2	RF[1]=1	RF[2]=x	RF[3]=13	...	RF[31]=4
t=59	pc=3	inta=0	RF[0]=2	RF[1]=1	RF[2]=x	RF[3]=13	...	RF[31]=4
t=61	pc=4	inta=0	RF[0]=1	RF[1]=1	RF[2]=x	RF[3]=13	...	RF[31]=4
t=63	pc=3	inta=0	RF[0]=1	RF[1]=1	RF[2]=x	RF[3]=13	...	RF[31]=4
t=65	pc=4	inta=0	RF[0]=0	RF[1]=1	RF[2]=x	RF[3]=13	...	RF[31]=4
t=67	pc=5	inta=0	RF[0]=0	RF[1]=1	RF[2]=x	RF[3]=13	...	RF[31]=4
t=69	pc=10	inta=0	RF[0]=0	RF[1]=1	RF[2]=x	RF[3]=13	...	RF[31]=6
t=71	pc=11	inta=0	RF[0]=66	RF[1]=1	RF[2]=x	RF[3]=13	...	RF[31]=6
t=73	pc=6	inta=0	RF[0]=66	RF[1]=1	RF[2]=x	RF[3]=13	...	RF[31]=6
t=75	pc=7	inta=0	RF[0]=66	RF[1]=1	RF[2]=x	RF[3]=13	...	RF[31]=6
t=77	pc=8	inta=0	RF[0]=66	RF[1]=1	RF[2]=x	RF[3]=13	...	RF[31]=6
t=79	pc=9	inta=0	RF[0]=66	RF[1]=1	RF[2]=33	RF[3]=13	...	RF[31]=6

17.4 toyRISC Structural Implementation

17.4.1 Structure

```

/*****
File name: toyRISC.sv

                                toyRISC

*****/
`include "DEFINES.vh"
module toyRISC( input    logic [31:0] instr    ,

```

```

        output logic [9:0] nextPC      ,
        input  logic      intIn       ,
        output logic      inta        ,
        input  logic [31:0] dataIn     ,
        output logic [31:0] dataOut    ,
        output logic [9:0] addr        ,
        output logic      dataRead     ,
        output logic      dataWrite    ,
        input  logic      reset        ,
        input  logic      clk          );

logic [5:0] opCode ;
logic [4:0] d, l, r ; // dest, left, right addresses for rf
logic [31:0] v ; // immediate value
logic [31:0] leftOp ; a
logic [9:0] pc ;
logic we ;
logic [1:0] sel ;

assign opCode = instr[31:26] ;
assign d = instr[25:21] ;
assign l = instr[20:16] ;
assign r = instr[15:11] ;
assign v = {{16{instr[15]}} , instr[15:0]};

DCDtoyRISC DCD(opCode      ,
               intIn       ,
               inta        ,
               dataRead     ,
               dataWrite    ,
               we           ,
               sel          ,
               reset        ,
               clk          );

PCtoyRISC PC( nextPC      ,
              pc          ,
              leftOp      ,
              v[9:0]      ,
              opCode      ,
              inta        ,
              reset       ,
              clk          );

RALUtoyRISC RALU( dataOut  ,
                  leftOp  ,
                  pc      ,
                  opCode  ,
                  dataIn  ,
                  v        ,
                  l        ,

```

```

        r        ,
        d        ,
        sel      ,
        we       ,
        inta     ,
        clk      );

    assign addr = leftOp[15:0] ;
endmodule // Synthesis results: #LUT=455, #FF=11, #DSP=6

```

```

/*****
File name: DCDtoyRISC.sv
          toyRISC's DCD
*****/
#include "DEFINES.vh"
module DCDtoyRISC( input  logic [5:0]  opCode      ,
                  input  logic      intIn       ,
                  output logic      inta        ,
                  dataRead ,
                  dataWrite ,
                  we        ,
                  output logic [1:0] sel        ,
                  input  logic      reset      ,
                  clk       );

    // Interrupt automaton
    logic ei ; // enable interrupt = state register
    always_ff @(posedge clk) if (reset) ei <= 0;
                                else begin if (opCode == 'eint) ei <= 1;
                                           if (opCode == 'dint) ei <= 0;
                                           if (intIn & ei)    ei <= 0;
                                end

    assign inta = intIn & ei;

    // Decoding
    assign dataRead = (opCode == 'read) ? 1'b1 : 1'b0 ;
    assign dataWrite = (opCode == 'store) ? 1'b1 : 1'b0 ;
    assign we = inta | (opCode[5:4] == 2'b11) |
                (opCode[5:4] == 2'b01) |
                (opCode == 6'b10_0111) ;
    assign sel = inta ? 2'b00 : opCode[5:4] ;
endmodule

```

```

/*****
File name: PCtoyRISC.sv
          toyRISC's PC
*****/

```



```

*****
#include "DEFINES.vh"
module PCtoyRISC(
    output logic [9:0] nextPC ,
    output logic [9:0] pc ,
    input logic [31:0] leftOp ,
    input logic [9:0] v ,
    input logic [5:0] opCode ,
    input logic inta ,
    reset ,
    clk );

    always_ff @(posedge clk) if (reset) pc <= -1 ;
                                else if (inta) pc <= leftOp[9:0] ;
                                else pc <= nextPC ;

    always_comb case(opCode)
        'rjmp : nextPC = pc + v ;
        'zbr : nextPC = (leftOp == 0) ? pc + v : pc + 1;
        'nzbr : nextPC = (leftOp != 0) ? pc + v : pc + 1;
        'ret : nextPC = leftOp ;
        'halt : nextPC = pc ;
        default : nextPC = pc + 1 ;
    endcase
endmodule

```

```

/*****
File name: RALUtoyRISC.sv
toyRISC's RALU
*****/
#include "DEFINES.vh"
module RALUtoyRISC(
    output logic [31:0] dataOut ,
    output logic [31:0] leftOp ,
    input logic [9:0] pc ,
    input logic [5:0] opCode ,
    input logic [31:0] dataIn ,
    v ,
    input logic [4:0] l ,
    r ,
    d ,
    input logic [1:0] sel ,
    input logic we ,
    inta ,
    clk );

    logic [31:0] leftIn ;
    logic [31:0] rightIn ;
    logic [31:0] muxOut ;
    logic [31:0] aluOut ;

    registerFile regFile(.leftOp (leftIn ),

```

```

        .rightOp    (rightIn ),
        .in         (muxOut  ),
        .leftAddr   (l       ),
        .rightAddr  (r       ),
        .destAddr   (d       ),
        .we         (we      ),
        .inta       (inta    ),
        .opCode     (opCode  ),
        .clk        (clk     ));

    assign dataOut = rightIn ;
    assign leftOp  = leftIn  ;

    ALU alu(.out      (aluOut),
            .leftIn   (leftIn ),
            .rightIn  (rightIn),
            .value    (v      ),
            .opCode   (opCode ));

    bigMux bigMux( .aluOut (aluOut),
                  .dataIn  (dataIn),
                  .value   (v      ),
                  .pc      (pc     ),
                  .sel     (sel    ),
                  .muxOut  (muxOut ));

endmodule

```

```

/*****
File name: registerFile.sv
Circuit name:
Description:
*****/
module registerFile(output logic [31:0] leftOp,
                   rightOp,
                   input logic [31:0] in,
                   input logic [4:0] leftAddr,
                   rightAddr,
                   destAddr,
                   input logic we,
                   input logic inta,
                   input logic [5:0] opCode,
                   input logic clk);
    logic [31:0] rf[0:31];

    always_ff @(posedge clk)
        if (we) rf[inta ? 5'b11110 : destAddr] <= in ;

    assign leftOp = rf[inta ? 5'b11111 : leftAddr];

```

```

    assign rightOp = rf[rightAddr]
endmodule

```

```

/*****
File name: ALU.sv
Circuit name:
Description:
*****/
`include "DEFINES.vh"
module ALU( output logic [31:0] out ,
            input  logic [31:0] leftIn ,
            input  logic [31:0] rightIn ,
            input  logic [5:0] value ,
            input  logic [5:0] opCode );

    always_comb
    case(opCode)
        'add : out = leftIn + rightIn ;
        'sub : out = leftIn - rightIn ;
        'addv : out = leftIn + value ;
        'mult : out = leftIn * rightIn ;
        'multv : out = leftIn * value ;
        'addc : out = (leftIn + rightIn) > 2**32 - 1;
        'subc : out = (leftIn - rightIn) > 2**32 - 1;
        'addvc : out = (leftIn + value) > 2**32 - 1 ;
        'lsh : out = leftIn >> 1 ;
        'ash : out = {leftIn[31], leftIn[31:1]} ;
        'move : out = leftIn ;
        'swap : out = {leftIn[15:0], leftIn[31:16]} ;
        'bwnot : out = ~leftIn ;
        'bwand : out = leftIn & rightIn ;
        'bwor : out = leftIn | rightIn ;
        'bwxor : out = leftIn ^ rightIn ;
        default : out = leftIn ;
    endcase
endmodule

```

```

/*****
File name: bigMux.sv
Circuit name:
Description:
*****/
module bigMux ( input logic [31:0] aluOut ,
                dataIn ,
                value ,

```

```

        input  logic [9:0]  pc      ,
        input  logic [1:0]  sel     ,
        output logic [31:0] muxOut );

    always_comb case(sel)
        2'b00: muxOut = pc      ;
        2'b01: muxOut = value   ;
        2'b10: muxOut = dataIn  ;
        2'b11: muxOut = aluOut  ;
    endcase

endmodule

```

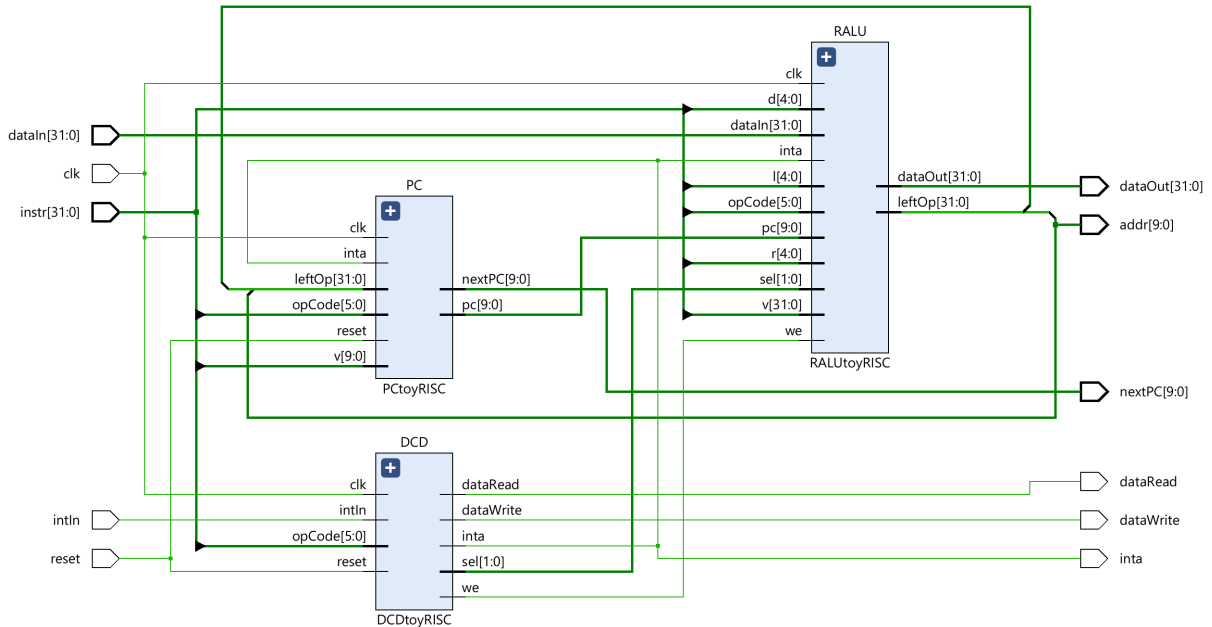


Figure 17.14:

17.4.2 Code generator

```

/*****
File name: RISCcodeGenerator.sv
Description: ISA

NOP                : rf[0] <= rf[0] + 0
RJMP(label)       : pc <= pc + value
BRZ(left, label)  : pc <= (left = 0) ? pc+immediate : pc+1
BRNZ(left, label) : pc <= (left = 0) ? pc+1 : pc+immediate
RET               : pc <= rf[left]

```

```

HALT                : pc <= pc
EINT                : intEnable <= 1 (enable interrupt)
DINT                : intEnable <= 0 (disable interrupt)
ADD(dest, left, right) : rf[dest] <= rf[left] + rf[right]
SUB(dest, left, right) : rf[dest] <= rf[left] - rf[right]
ADDV(dest, left, value) : rf[dest] <= rf[left] + value
MULT(dest, left, right) : rf[dest] <= rf[left] * rf[right]
MULTV(dest, left, value) : rf[dest] <= rf[left] * rf[right]
ADDC(dest, left, right) : rf[dest] <= (rf[left] + rf[right])[32]
SUBC(dest, left, right) : rf[dest] <= (rf[left] - rf[right])[32]
ADDVC(dest, left, value) : rf[dest] <= (rf[left] + value)[32]
LSH(dest, left)      : rf[dest] <= {0, rf[left][31:1]}
ASH(dest, left)      : rf[dest] <= {rf[left][31], rf[left][31:1]}
MOVE(dest, left)     : rf[dest] <= rf[left]
SWAP(dest, left)     : rf[dest] <= {rf[left][15:0], rf[left][31:16]}
NOT(dest, left)      : rf[dest] <= ~rf[left]
AND(dest, left, right) : rf[dest] <= rf[left] & rf[right]
OR(dest, left, right)  : rf[dest] <= rf[left] | rf[right]
XOR(dest, left, right) : rf[dest] <= rf[left] ^ rf[right]
READ(left)           : read from dataMemory[left]
LOAD(dest)           : rf[dest] <= dataOut
STORE(left, right)    : dataMemory[left] <= rf[right]
VAL(dest, val)       : rf[dest] <= {{11{val[20]}}, value}
******/

reg [5:0]   opCode      ;
reg [4:0]   d            ;
reg [4:0]   l            ;
reg [4:0]   r            ;
reg [15:0]  v            ;
reg [9:0]   addrCounter  ;
reg [9:0]   labelTab[0:1023];

‘include ”DEFINES.vh”

task endLine;
begin
    progMemory[addrCounter][31:0] =
        {
            opCode ,
            d      ,
            l      ,
            v      }
        ;
    addrCounter = addrCounter + 1 ;
end
endtask

// sets labelTab in the first pass
// associating 'counter' with 'labelIndex'
task LB ;
    input [5:0] labelIndex;

```

```

        labelTab[labelIndex] = addrCounter;
    endtask
    // uses the content of labelTab in the second pass
    task ULB;
        input [5:0] labelIndex;

        v = labelTab[labelIndex] - addrCounter;
    endtask

// CONTROL INSTRUCTIONS
    task NOP; // no operation
        begin
            opCode = 'addv ;
            d      = 5'b0 ;
            l      = 5'b0 ;
            v      = 16'b0 ;
            endLine
        end
    endtask

    task RJMP; // relative jump
        input [15:0] label ;

        begin
            opCode = 'rjmp ;
            d      = 5'b0 ;
            l      = 5'b0 ;
            ULB(label) ;
            endLine
        end
    endtask

    task BRZ; // branch if zero
        input [4:0] left ;
        input [9:0] label ;

        begin
            opCode = 'zbr ;
            d      = 5'b0 ;
            l      = left ;
            ULB(label) ;
            endLine
        end
    endtask

    task BRNZ; // branch if not zero
        input [4:0] left ;
        input [9:0] label ;

        begin
            opCode = 'nzbr ;
            d      = 5'b0 ;
            l      = left ;
            ULB(label) ;

```

```

        endLine      ;
    end
endtask

task RET; // return from subroutine
    input  [4:0] left ;

    begin    opCode = 'ret ;
             d      = 5'b0 ;
             l      = left ;
             v      = 16'b0 ;
             endLine ;

    end
endtask

task HALT; // halt running
    begin    opCode = 'halt ;
             d      = 5'b0 ;
             l      = 5'b0 ;
             v      = 16'b0 ;
             endLine ;

    end
endtask

task EI; // enable interrupt
    begin    opCode = 'eint ;
             d      = 5'b0 ;
             l      = 5'b0 ;
             v      = 16'b0 ;
             endLine ;

    end
endtask

task DI; // disable interrupt
    begin    opCode = 'dint ;
             d      = 5'b0 ;
             l      = 5'b0 ;
             v      = 16'b0 ;
             endLine ;

    end
endtask

// ARITHMETIC & LOGIC INSTRUCTIONS
task ADD; // addition
    input  [4:0] dest ;
    input  [4:0] left  ;
    input  [4:0] right ;

    begin    opCode = 'add ;
             d      = dest ;

```

```

        l      = left      ;
        v      = {right , 11'b0};
        endLine      ;
    end
endtask

task SUB; // subtract
    input [4:0] dest      ;
    input [4:0] left      ;
    input [4:0] right     ;

    begin
        opCode = 'sub      ;
        d      = dest      ;
        l      = left      ;
        v      = {right , 11'b0};
        endLine      ;
    end
endtask

task ADDV; // addition with value
    input [4:0] dest      ;
    input [4:0] left      ;
    input [15:0] value     ;

    begin
        opCode = 'adv      ;
        d      = dest      ;
        l      = left      ;
        v      = value      ;
        endLine      ;
    end
endtask

task MULT; // multiplication
    input [4:0] dest      ;
    input [4:0] left      ;
    input [4:0] right     ;

    begin
        opCode = 'mult     ;
        d      = dest      ;
        l      = left      ;
        v      = {right , 11'b0};
        endLine      ;
    end
endtask

task MULTV; // multiplication with value
    input [4:0] dest      ;
    input [4:0] left      ;
    input [15:0] value     ;

```



```

        begin    opCode = 'multv;
                d      = dest  ;
                l      = left  ;
                v      = value ;
                endLine      ;
        end
    endtask

    task ADDC; // carry from addition
        input [4:0] dest  ;
        input [4:0] left  ;
        input [4:0] right ;

        begin    opCode = 'addc      ;
                d      = dest      ;
                l      = left      ;
                v      = {right, 11'b0};
                endLine      ;
        end
    endtask

    task SUBC; // carry from subtract
        input [4:0] dest  ;
        input [4:0] left  ;
        input [4:0] right ;

        begin    opCode = 'subc      ;
                d      = dest      ;
                l      = left      ;
                v      = {right, 11'b0};
                endLine      ;
        end
    endtask

    task ADDVC; // carry from addition with value
        input [4:0] dest  ;
        input [4:0] left  ;
        input [15:0] value ;

        begin    opCode = 'addvc;
                d      = dest  ;
                l      = left  ;
                v      = value ;
                endLine      ;
        end
    endtask

    task LSH; // logic shift with one position
        input [4:0] dest  ;
        input [4:0] left  ;

```

```

        begin    opCode = 'lsh  ;
                d      = dest  ;
                l      = left  ;
                v      = 16'b0 ;
                endLine      ;
    end
endtask

task ASH; // arithmetic shift with one position
input [4:0] dest ;
input [4:0] left  ;

    begin    opCode = 'ash  ;
            d      = dest  ;
            l      = left  ;
            v      = 16'b0 ;
            endLine      ;
    end
endtask

task MOVE; // data move inside register file
input [4:0] dest ;
input [4:0] left  ;

    begin    opCode = 'move ;
            d      = dest  ;
            l      = left  ;
            v      = 16'b0 ;
            endLine      ;
    end
endtask

task SWAP; // swap in register
input [4:0] dest ;
input [4:0] left  ;

    begin    opCode = 'swap ;
            d      = dest  ;
            l      = left  ;
            v      = 16'b0 ;
            endLine      ;
    end
endtask

task NOT; // bitwise NOT
input [4:0] dest ;
input [4:0] left  ;

    begin    opCode = 'bwnot;

```

```

        d      = dest  ;
        l      = left  ;
        v      = 16'b0 ;
        endLine ;
    end
endtask

task AND; // bitwise AND
    input [4:0] dest  ;
    input [4:0] left  ;
    input [4:0] right ;

    begin
        opCode = 'bwand ;
        d      = dest  ;
        l      = left  ;
        v      = {right, 11'b0};
        endLine ;
    end
endtask

task OR; // bitwise OR
    input [4:0] dest  ;
    input [4:0] left  ;
    input [4:0] right ;

    begin
        opCode = 'bwor ;
        d      = dest  ;
        l      = left  ;
        v      = {right, 11'b0};
        endLine ;
    end
endtask

task XOR; // bitwise XOR
    input [4:0] dest  ;
    input [4:0] left  ;
    input [4:0] right ;

    begin
        opCode = 'bwxor ;
        d      = dest  ;
        l      = left  ;
        v      = {right, 11'b0};
        endLine ;
    end
endtask

// DATA TRANSFER INSTRUCTIONS
task READ; // data read
    input [4:0] left  ;

```

```

        begin    opCode = 'read ;
                d      = 5'b0 ;
                l      = left  ;
                v      = 16'b0 ;
                endLine
        end
    endtask

    task LOAD; // data load
        input  [4:0] dest ;

        begin    opCode = 'load ;
                d      = dest ;
                l      = 5'b0 ;
                v      = 16'b0 ;
                endLine
        end
    endtask

    task STORE; // data store
        input  [4:0] left  ;
        input  [4:0] right ;

        begin    opCode = 'store ;
                d      = 5'b0 ;
                l      = left ;
                v      = {right, 11'b0};
                endLine
        end
    endtask

    task VAL; // value load
        input  [4:0] dest ;
        input  [15:0] value ;

        begin    opCode = 'val ;
                d      = dest ;
                l      = 5'b0 ;
                v      = value ;
                endLine
        end
    endtask

    // RUNNING
    initial begin    addrCounter = 0;
                    'include "program.sv" // first pass
                    addrCounter = 0;
                    'include "program.sv" // second pass
        end

```

17.4.3 Simulator

```

/*****
File name: testRISC.sv
Circuit name:
Description:
*****/
`include "DEFINES.vh"
module testRISC;
    logic          inta      ;
    logic          intIn     ;
    logic          reset     ;
    logic          clk       ;

    integer i      ;

    `include "RISCcodeGenerator.sv"

    initial begin
        clk = 0      ;
        forever #1   clk = ~clk ;
    end

    initial begin
        intIn      = 1 ;
        reset      = 1 ;
        // DISPLAY THE CONTENT OF PROGRAM MEMORY
        for (i=0; i<8; i=i+1)
            $display("progMemory[%0d]\ t=\b", i ,
                    progMemory[i]) ;
        #4 reset      = 0 ;
        #40 $finish    ;
    end

    logic [31:0] instr      ;
    logic [9:0]  nextPC     ;
    logic [31:0] dataIn     ;
    logic [31:0] dataOut    ;
    logic [9:0]  addr       ;
    logic        dataRead   ;
    logic        dataWrite  ;

    logic [31:0] dataMemory[0:1023] ;
    logic [31:0] progMemory[0:1023] ;
    logic [31:0] dataMemOut      ;

    always_ff @(posedge clk) begin
        if (dataRead) dataIn <= dataMemory[addr] ;
        if (dataWrite) dataMemory[addr] <= dataOut ;
        instr <= progMemory[nextPC] ;
    end
end

```

```

toyRISC dut( instr      ,
              nextPC    ,
              intIn     ,
              inta      ,
              dataIn    ,
              dataOut   ,
              addr      ,
              dataRead  ,
              dataWrite ,
              reset     ,
              clk        );

// MONITOR FOR PROGRAM LAOD & CONTROLLER
initial begin
    $monitor("t=%0d pc=%0d RF=[%0d,%0d,%0d,%0d] ei=%b inta=%b",
             $time,
             dut.PC.pc,
             dut.RALU.regFile.rf[0],
             dut.RALU.regFile.rf[1],
             dut.RALU.regFile.rf[2],
             dut.RALU.regFile.rf[3],
             dut.DCD.ei,
             dut.inta);
end
endmodule

```

17.4.4 Testing

```

/*****
File name: program.sv
Description: programs to validate the main types of instructions
*****/
// Testing instantiating registers & arithmetic operation
/*
    VAL(0,2) ;
    VAL(1,4) ;
    VAL(2,8) ;
    MULT(3,1,2) ;
    HALT ;
// */
// Testing interrupt mechanism
/*
    VAL(31,10) ;
    VAL(2,23) ;
    VAL(0,13) ;
    EI ;

```

```

        ADDV(0,0,2) ;
        NOP        ;
        ADDV(0,0,4) ;
        HALT       ;
        NOP        ;
        NOP        ;
    // subroutine triggered by interrupt
        DI         ;
        VAL(3,44)  ;
        RET(30)    ;
// */
// Testing jump & branch
/*
        VAL(0,3)   ;
    LB(1); ADDV(0,0,-1);
        NOP        ;
        RJMP(1)     ; // BRNZ(0,1)
        HALT       ;
// */
// Testing data memory
/*
        VAL(0,1)    ;
        VAL(1,55)   ;
        STORE(0,1)  ;
        READ(0)     ;
        LOAD(2)     ;
        HALT       ;
// */

```

```

progMemory[0] = 010111111110000000000000000001010
progMemory[1] = 0101110001000000000000000000010111
progMemory[2] = 01011100000000000000000000000001101
progMemory[3] = 00100000000000000000000000000000000
progMemory[4] = 11001000000000000000000000000000010
progMemory[5] = 11001000000000000000000000000000000
progMemory[6] = 110010000000000000000000000000000100
progMemory[7] = 000110000000000000000000000000000000
progMemory[8] = 110010000000000000000000000000000000
progMemory[9] = 110010000000000000000000000000000000
progMemory[10] = 001001000000000000000000000000000000
progMemory[11] = 01011100011000000000000000000101100
progMemory[12] = 00010100000111100000000000000000000
progMemory[13] = xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
progMemory[14] = xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
progMemory[15] = xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx

```

```

t=0 pc=  x RF=[x, x, x, x] out=x leftIn=x rightIn=x value=x opCode=x ei=x inta=x
t=1 pc=1023 RF=[x, x, x, x] out=x leftIn=x rightIn=x value=x opCode=x ei=0 inta=0
t=3 pc=1023 RF=[x, x, x, x] out=x leftIn=x rightIn=x value=10 opCode=23 ei=0 inta=0
t=5 pc=  0 RF=[x, x, x, x] out=x leftIn=x rightIn=x value=10 opCode=23 ei=0 inta=0
t=7 pc=  1 RF=[x, x, x, x] out=x leftIn=x rightIn=x value=23 opCode=23 ei=0 inta=0
t=9 pc=  2 RF=[x, x, 23, x] out=x leftIn=x rightIn=x value=13 opCode=23 ei=0 inta=0

```

```

t=11 pc= 3 RF=[13, x, 23, x] out=13 leftIn=13 rightIn=13 value=0 opCode=8 ei=0 inta=0
t=13 pc= 4 RF=[13, x, 23, x] out=12 leftIn=10 rightIn=13 value=2 opCode=50 ei=1 inta=1
t=15 pc= 10 RF=[13, x, 23, x] out=13 leftIn=13 rightIn=13 value=0 opCode=50 ei=0 inta=0
t=17 pc= 11 RF=[13, x, 23, x] out=13 leftIn=13 rightIn=13 value=44 opCode=23 ei=0 inta=0
t=19 pc= 12 RF=[13, x, 23, 44] out=4 leftIn=4 rightIn=13 value=0 opCode=5 ei=0 inta=0
t=21 pc= 4 RF=[13, x, 23, 44] out=15 leftIn=13 rightIn=13 value=2 opCode=50 ei=0 inta=0
t=23 pc= 5 RF=[15, x, 23, 44] out=15 leftIn=15 rightIn=15 value=0 opCode=50 ei=0 inta=0
t=25 pc= 6 RF=[15, x, 23, 44] out=19 leftIn=15 rightIn=15 value=4 opCode=50 ei=0 inta=0
t=27 pc= 7 RF=[19, x, 23, 44] out=19 leftIn=19 rightIn=19 value=0 opCode=6 ei=0 inta=0

```

17.5 Pipelined toyRISC

17.5.1 Structure

```

/*****
File name: toyRISCsystem.sv
Circuit name:
Description:
*****/
`include "0_DEFINES.vh"
module toyRISCsystem(      output logic   inta      ,
                          input  logic   intIn     ,
                              reset      ,
                              clk        );

    logic [31:0] instruction ;
    logic [9:0]  PC          ;
    logic [31:0] dataIn      ;
    logic [31:0] dataOut     ;
    logic [9:0]  dataAddr    ;
    logic        dataRead    ;
    logic        dataWrite   ;

    toyRISC processor( instruction ,
                        PC          ,
                        intIn       ,
                        inta        ,
                        dataIn      ,
                        dataOut     ,
                        dataAddr    ,
                        dataRead    ,
                        dataWrite   ,
                        reset       ,
                        clk        );

    memorySystem memSys(instruction ,
                        PC          ,
                        dataIn      ,
                        dataOut     ,
                        dataAddr    ,
                        dataRead    ,
                        dataWrite   ,

```



```

                                clk
endmodule

```

```

/*****
File name: memorySystem.sv
Circuit name:
Description:
*****/
module memorySystem(output logic [31:0] instruction ,
                   input  logic [9:0] PC ,
                   output logic [31:0] dataIn ,
                   input  logic [31:0] dataOut ,
                   input  logic [9:0] dataAddr ,
                   input  logic dataRead ,
                   input  logic dataWrite ,
                   input  logic clk );

    logic [31:0] dataMemory[0:1023] ;
    logic [31:0] progMemory[0:1023] ;

    assign instruction = progMemory[PC] ;

    always_ff @(posedge clk) begin
        if (dataWrite) dataMemory[dataAddr] <= dataOut ;
    end

    assign dataIn = dataMemory[dataAddr];

endmodule

```

```

/*****
File name: toyRISC.sv
Circuit name:
Description:
*****/
#include "0_DEFINES.vh"
module toyRISC( input  logic [31:0] instruction ,
               output logic [9:0] PC ,
               input  logic intIn ,
               output logic inta ,
               input  logic [31:0] dataIn ,
               output logic [31:0] dataOut ,
               output logic [9:0] dataAddr ,
               output logic dataRead ,
               output logic dataWrite ,

```

```

        input  logic      reset      ,
        input  logic      clk        );

    logic [31:0] instruction1;
    logic [9:0]  PC1                ;
    logic [31:0] leftOp              ;
    logic [1:0]  nextPCsel           ;
    logic [31:0] results             ;
    logic [5:0]  opCode1             ;
    logic [31:0] leftOp2             ;
    logic [31:0] rightOp2           ;
    logic [31:0] value2              ;
    logic [5:0]  opCode2             ;
    logic [4:0]  destAddr2           ;
    logic        we3                 ;
    logic [4:0]  destAddr3           ;
    logic [31:0] result3             ;
    logic [5:0]  decode              ;
    logic        we                   ;
    logic [1:0]  opSel               ;
    logic [1:0]  opSel2              ;
    logic        zero                ;
    logic [5:0]  opCode3             ;

    DECODE dcd( .opCode1    (instruction1[31:26]),
                .zero       (zero),
                .opCode2    (opCode2),
                .intIn      (intIn),
                .inta       (inta),
                .nextPCsel  (nextPCsel),
                .opSel      (opSel),
                .we         (we),
                .dataRead   (dataRead),
                .dataWrite  (dataWrite),
                .reset      (reset),
                .clk        (clk)
    );

    INSTRFETCH progCount( instruction ,
                          PC          ,
                          instruction1 ,
                          PC1         ,
                          leftOp      ,
                          nextPCsel   ,
                          reset       ,
                          clk         );

    OPSFETCH regFile( instruction1 ,
                     PC1           ,
                     inta          ,
                     we3           ,
                     destAddr3     ,

```

```

        result3      ,
        opSel        ,
        opSel2       ,
        zero         ,
        leftOp       ,
        leftOp2      ,
        rightOp2     ,
        value2       ,
        opCode2      ,
        destAddr2    ,
        clk          );

EXECUTE execute(leftOp2      ,
               rightOp2     ,
               value2       ,
               opCode2      ,
               destAddr2    ,
               dataIn       ,
               we           ,
               opSel2       ,
               dataAddr     ,
               dataOut      ,
               we3          ,
               destAddr3    ,
               result3      ,
               opCode3      ,
               clk          );
endmodule // Synthesis results: #LUT=335, #FF=204, #DSP=3

```

```

/*****
File name: DECODE.sv
Circuit name:
Description:
*****/
`include "0_DEFINES.vh"
module DECODE(  input  logic [5:0] opCode1      ,
               input  logic      zero          ,
               input  logic [5:0] opCode2      ,
               input  logic      intIn         ,
               output logic      inta           ,
               output logic [1:0] nextPCsel     ,
               output logic [1:0] opSel        ,
               output logic      we            ,
               output logic      dataRead       ,
               output logic      dataWrite     ,
               input  logic      reset         ,
               input  logic      clk           );
    logic [2:0] intState ;

```

```

/*
    intState = 000: intDisable
    intState = 001: intEnable
    intState = 010: intExec1
    intState = 011: intExec2
    intState = 100: intExec3
*/
// Interrupt automaton
always_ff @(posedge clk)
    if (reset)                                intState <= 3'b000 ;
    else begin
        if (opCode1 == 'eint)                intState <= 3'b001 ;
        if (opCode1 == 'dint)                intState <= 3'b000 ;
        if (intIn && (intState == 3'b001))    intState <= 3'b010 ;
        if (intState == 3'b010)              intState <= 3'b011 ;
        if (intState == 3'b011)              intState <= 3'b100 ;
        if (intState == 3'b100)              intState <= 3'b000 ;
    end
    //assign intr = intState == 2'b10 ;
    assign inta = intState == 3'b010 ;

    logic [5:0] jmpSel ;

    assign jmpSel = (intState > 3'b001) ? 'halt : opCode1 ;

    always_comb
        if (inta)                            nextPCsel = 2'b11 ;
        else case(jmpSel)
            'halt : nextPCsel = 2'b00 ;
            'rjmp : nextPCsel = 2'b10 ;
            'zbr : nextPCsel = zero ? 2'b10 : 2'b01 ;
            'nzbr : nextPCsel = zero ? 2'b01 : 2'b10 ;
            'ret : nextPCsel = 2'b11 ;
            default : nextPCsel = 2'b01 ;
        endcase

    always_comb case(opCode1)
        'load : opSel = 2'b10 ;
        'addv : opSel = 2'b01 ;
        'multv : opSel = 2'b01 ;
        'addvc : opSel = 2'b01 ;
        'val : opSel = 2'b01 ;
        default : opSel = 2'b00 ;
    endcase

    assign dataWrite = opCode2 == 'store ;
    assign dataRead = opCode2 == 'read ;
    assign we = ((opCode2[5:4] == 2'b11) |

```

```

                                (opCode2 == 'load)      |
                                (opCode2 == 'val)) & !(intState > 3'b001) ;
endmodule

```

```

/*****
File name: INSTRFETCH.sv
Circuit name:
Description:
*****/
module INSTRFETCH(  input    logic [31:0]    instruction ,
                   output  logic [9:0]      PC          ,
                   output  logic [31:0]    instruction1 ,
                   output  logic [9:0]      PC1         ,
                   input    logic [31:0]    leftOut      ,
                   input    logic [1:0]     nextPCsel    ,
                   input    logic          reset       ,
                   input    logic          clk          );

    // PipeReg0
    always @(posedge clk)
        if (reset) begin
            PC <= -1 ;
            instruction1 <= 0 ;
            //PC1 <= 0 ;
        end
        else begin case(nextPCsel)
            2'b00: PC <= PC ;
            2'b01: PC <= PC + 1 ;
            2'b10: PC <= instruction1[9:0] + PC1 ;
            2'b11: PC <= leftOut ;
        endcase

    // PipeReg1
    instruction1 <= instruction ;
    PC1 <= PC ;

end
endmodule

```

```

/*****
File name: OPSFETCH.sv
Circuit name:
Description:
*****/
module OPSFETCH(input    logic [31:0]    instruction1 ,
                input    logic [9:0]      PC1          ,
                input    logic          inta          ,
                input    logic          we3           ,

```

```

        input  logic [4:0]    destAddr3    ,
        input  logic [31:0]   result3      ,
        input  logic [1:0]    opSel        ,
        output logic [1:0]    opSel2       ,
        output logic          zero         ,
        output logic [31:0]   leftOp       ,
        output logic [31:0]   leftOp2     ,
        output logic [31:0]   rightOp2    ,
        output logic [31:0]   value2       ,
        output logic [5:0]    opCode2     ,
        output logic [4:0]    destAddr2   ,
        input  logic          clk          );
    logic [31:0] rf[0:31]    ;
    logic [31:0] rightOp     ;
    logic          we        ;
    logic [4:0]    destAddr  ;
    logic [31:0]   result    ;
    logic [4:0]    leftAddr  ;
    logic [4:0]    rightAddr ;

    always_ff @(posedge clk) if (we) rf[destAddr] <= result ;

    assign leftOp      = rf[leftAddr]      ;
    assign rightOp     = rf[rightAddr]     ;
    assign zero        = leftOp == 0       ;
    assign rightAddr   = instruction1[15:11] ;
// PipeReg2:
    always_ff @(posedge clk) begin
        leftOp2    <= leftOp                ;
        rightOp2   <= rightOp               ;
        value2     <= {{16{instruction1[15]}} , instruction1[15:0]};
        opSel2     <= opSel                 ;
        opCode2    <= instruction1[31:26]   ;
        destAddr2  <= instruction1[25:21]   ;
    end

    intUnit intunit(.we          (we          ),
                    .destAddr    (destAddr    ),
                    .result      (result      ),
                    .leftAddr    (leftAddr    ),
                    .we3         (we3         ),
                    .destAddr3   (destAddr3   ),
                    .result3     (result3     ),
                    .leftAddr1   (instruction1[20:16] ),
                    .PC1        (PC1        ),
                    .inta        (inta        ));
endmodule

```

```

/*****
File name: EXECUTE.sv
Circuit name:
Description:
*****/
`include "0_DEFINES.vh"
module EXECUTE( input logic [31:0] leftOp2 ,
               input logic [31:0] rightOp2 ,
               input logic [31:0] value2 ,
               input logic [5:0] opCode2 ,
               input logic [4:0] destAddr2 ,
               input logic [31:0] dataIn ,
               input logic we ,
               input logic [1:0] opSel2 ,
               output logic [9:0] dataAddr ,
               output logic [31:0] dataOut ,
               output logic we3 ,
               output logic [4:0] destAddr3 ,
               output logic [31:0] result3 ,
               output logic [5:0] opCode3 ,
               input logic clk );

    logic [31:0] rightIn ;
    logic [31:0] out ;
    logic [31:0] dataIn3 ;

    assign dataAddr = leftOp2[9:0] ;
    assign dataOut = rightOp2 ;

    always_comb case(opSel2)
        2'b00: rightIn = rightOp2 ;
        2'b01: rightIn = value2 ;
        2'b10: rightIn = dataIn3 ;
        2'b11: rightIn = 31'b0 ;
    endcase

    ALU alu(.out (out),
            .leftIn (leftOp2),
            .rightIn(rightIn),
            .opCode (opCode2));

    // PipeReg3
    always_ff @(posedge clk) begin we3 <= we ;
                                destAddr3 <= destAddr2 ;
                                result3 <= out ;
                                opCode3 <= opCode2 ;
                                dataIn3 <= dataIn ;
    end

endmodule

```

```

/*****
File name: ALU.sv
Circuit name:
Description:
*****/
`include "0_DEFINES.vh"
module ALU( output logic [31:0] out ,
            input logic [31:0] leftIn ,
            input logic [5:0] opCode );

    logic [32:0] sum ;
    logic [32:0] dif ;

    assign sum = leftIn + rightIn ;
    assign dif = leftIn - rightIn ;

    always_comb case(opCode)
        `add : out = sum[31:0] ;
        `sub : out = dif[31:0] ;
        `addv : out = sum[31:0] ;
        `mult : out = leftIn * rightIn ;
        `multv : out = leftIn * rightIn ;
        `addc : out = sum[32] ;
        `subc : out = dif[32] ;
        `addvc : out = sum[32] ;
        `lsh : out = leftIn >> 1 ;
        `ash : out = {leftIn[31], leftIn[31:1]} ;
        `move : out = leftIn ;
        `swap : out = {leftIn[15:0], leftIn[31:16]} ;
        `bwnot : out = ~leftIn ;
        `bwand : out = leftIn & rightIn ;
        `bwor : out = leftIn | rightIn ;
        `bwxor : out = leftIn ^ rightIn ;
        `load : out = rightIn ;
        `val : out = rightIn ;
        default : out = leftIn ;
    endcase
endmodule

```

```

/*****
File name: intUnit.sv
Circuit name:
Description:
*****/
module intUnit( output logic we ,
                output logic [4:0] destAddr ,

```



```

        output logic [31:0] result      ,
        output logic [4:0]  leftAddr   ,
        input  logic        we3        ,
        input  logic [4:0]  destAddr3   ,
        input  logic [31:0] result3     ,
        input  logic [4:0]  leftAddr1   ,
        input  logic [9:0]  PC1         ,
        input  logic        inta        );

    assign we      = inta ? 1'b1        : we3      ;
    assign destAddr = inta ? 5'b11110   : destAddr3 ;
    assign result  = inta ? {21'b0, PC1} : result3  ;
    assign leftAddr = inta ? 5'b11111   : leftAddr1 ;
endmodule

```

17.5.2 Code generator

```

/*****
File name: RISCcodeGenerator.sv
Circuit name:
Description:
*****/
    reg [5:0]    opCode      ;
    reg [4:0]    d           ;
    reg [4:0]    l           ;
    reg [4:0]    r           ;
    reg [15:0]   v           ;
    reg [9:0]    addrCounter  ;
    reg [9:0]    labelTab[0:1023];

    `include "0_DEFINES.vh"

    task endLine;
    begin
        dut.memSys.progMemory[addrCounter][31:0] =
            {
                opCode ,
                d       ,
                l       ,
                v       }
            ;
        addrCounter = addrCounter + 1 ;
    end
endtask

// sets labelTab in the first pass
// associating 'counter' with 'labelIndex'
task LB ;
    input [5:0] labelIndex ;

```

```

        labelTab[labelIndex] = addrCounter;
    endtask
    // uses the content of labelTab in the second pass
    task ULB;
        input [5:0] labelIndex;

        v = labelTab[labelIndex] - addrCounter ;
    endtask

// CONTROL INSTRUCTIONS
    task NOP; // no operation
        begin    opCode = 'nop ;
                  {d,l,v} = 26'b0 ;
                  endLine ;
        end
    endtask

    task RJMP; // relative jump
        input [15:0] label ;

        begin    opCode = 'rjmp ;
                  d      = 5'b0 ;
                  l      = 5'b0 ;
                  ULB(label) ;
                  endLine ;
        end
    endtask

    task BRZ; // branch if zero
        input [4:0] left ;
        input [15:0] label ;

        begin    opCode = 'zbr ;
                  d      = 5'b0 ;
                  l      = left ;
                  ULB(label) ;
                  endLine ;
        end
    endtask

    task BRNZ; // branch if not zero
        input [4:0] left ;
        input [15:0] label ;

        begin    opCode = 'nzbr ;
                  d      = 5'b0 ;
                  l      = left ;
                  ULB(label) ;
                  endLine ;

```

```

    end
endtask

task RET; // return from subroutine
    input  [4:0] left ;

    begin
        opCode = 'ret ;
        d      = 5'b0 ;
        l      = left ;
        v      = 16'b0 ;
        endLine
    end
endtask

task HALT; // halt running
    begin
        opCode = 'halt ;
        {d,l,v} = 26'b0 ;
        endLine
    end
endtask

task EI; // enable interrupt
    begin
        opCode = 'eint ;
        {d,l,v} = 26'b0 ;
        endLine
    end
endtask

task DI; // disable interrupt
    begin
        opCode = 'dint ;
        {d,l,v} = 26'b0 ;
        endLine
    end
endtask

// ARITHMETIC & LOGIC INSTRUCTIONS
task ADD; // addition
    input  [4:0] dest ;
    input  [4:0] left  ;
    input  [4:0] right ;

    begin
        opCode = 'add ;
        d      = dest ;
        l      = left  ;
        v      = {right, 11'b0};
        endLine
    end
endtask

task SUB; // subtract

```

```

    input  [4:0]  dest    ;
    input  [4:0]  left    ;
    input  [4:0]  right   ;

    begin    opCode = 'sub          ;
            d      = dest          ;
            l      = left          ;
            v      = {right, 11'b0};
            endLine          ;
    end
endtask

task ADDV; // addition with value
    input  [4:0]  dest    ;
    input  [4:0]  left    ;
    input  [15:0] value    ;

    begin    opCode = 'adv          ;
            d      = dest          ;
            l      = left          ;
            v      = value         ;
            endLine          ;
    end
endtask

task MULT; // multiplication
    input  [4:0]  dest    ;
    input  [4:0]  left    ;
    input  [4:0]  right   ;

    begin    opCode = 'mult         ;
            d      = dest          ;
            l      = left          ;
            v      = {right, 11'b0};
            endLine          ;
    end
endtask

task MULTV; // multiplication with value
    input  [4:0]  dest    ;
    input  [4:0]  left    ;
    input  [15:0] value    ;

    begin    opCode = 'multv        ;
            d      = dest          ;
            l      = left          ;
            v      = value         ;
            endLine          ;
    end
endtask

```

```

task ADDC; // carry from addition
    input    [4:0]    dest    ;
    input    [4:0]    left    ;
    input    [4:0]    right   ;

    begin     opCode   = 'addc      ;
              d       = dest      ;
              l       = left      ;
              v       = {right, 11'b0};
              endLine  ;

    end
endtask

task SUBC; // carry from subtract
    input    [4:0]    dest    ;
    input    [4:0]    left    ;
    input    [4:0]    right   ;

    begin     opCode   = 'subc      ;
              d       = dest      ;
              l       = left      ;
              v       = {right, 11'b0};
              endLine  ;

    end
endtask

task ADDVC; // carry from addition with value
    input    [4:0]    dest    ;
    input    [4:0]    left    ;
    input    [15:0]   value    ;

    begin     opCode   = 'addvc;
              d       = dest      ;
              l       = left      ;
              v       = value     ;
              endLine  ;

    end
endtask

task LSH; // logic shift with one position
    input    [4:0]    dest    ;
    input    [4:0]    left    ;

    begin     opCode   = 'lsh      ;
              d       = dest      ;
              l       = left      ;
              v       = 16'b0     ;
              endLine  ;

    end

```

```

endtask

task ASH; // arithmetic shift with one position
    input    [4:0]    dest    ;
    input    [4:0]    left    ;

    begin    opCode    = 'ash    ;
            d          = dest    ;
            l          = left    ;
            v          = 16'b0    ;
            endLine    ;

    end
endtask

task MOVE; // data move inside register file
    input    [4:0]    dest    ;
    input    [4:0]    left    ;

    begin    opCode    = 'move    ;
            d          = dest    ;
            l          = left    ;
            v          = 16'b0    ;
            endLine    ;

    end
endtask

task SWAP; // swap in register
    input    [4:0]    dest    ;
    input    [4:0]    left    ;

    begin    opCode    = 'swap    ;
            d          = dest    ;
            l          = left    ;
            v          = 16'b0    ;
            endLine    ;

    end
endtask

task NOT; // bitwise NOT
    input    [4:0]    dest    ;
    input    [4:0]    left    ;

    begin    opCode    = 'bwnot;
            d          = dest    ;
            l          = left    ;
            v          = 16'b0    ;
            endLine    ;

    end
endtask

```

```

task AND; // bitwise AND
    input    [4:0]    dest    ;
    input    [4:0]    left    ;
    input    [4:0]    right   ;

    begin    opCode    = 'bwand        ;
            d          = dest          ;
            l          = left          ;
            v          = {right, 11'b0};
            endLine    ;

    end
endtask

task OR; // bitwise OR
    input    [4:0]    dest    ;
    input    [4:0]    left    ;
    input    [4:0]    right   ;

    begin    opCode    = 'bwor        ;
            d          = dest          ;
            l          = left          ;
            v          = {right, 11'b0};
            endLine    ;

    end
endtask

task XOR; // bitwise XOR
    input    [4:0]    dest    ;
    input    [4:0]    left    ;
    input    [4:0]    right   ;

    begin    opCode    = 'bwxor        ;
            d          = dest          ;
            l          = left          ;
            v          = {right, 11'b0};
            endLine    ;

    end
endtask

// DATA TRANSFER INSTRUCTIONS
task READ; // data read
    input    [4:0]    left    ;

    begin    opCode    = 'read    ;
            d          = 5'b0    ;
            l          = left    ;
            v          = 16'b0    ;
            endLine    ;

    end
endtask

```

```

task LOAD; // data load
    input    [4:0]    dest    ;

    begin    opCode    = 'load  ;
            d          = dest  ;
            l          = 5'b0  ;
            v          = 16'b0 ;
            endLine    ;

    end
endtask

task STORE; // data store
    input    [4:0]    left    ;
    input    [4:0]    right   ;

    begin    opCode    = 'store ;
            d          = 5'b0   ;
            l          = left   ;
            v          = {right, 11'b0};
            endLine    ;

    end
endtask

task VAL; // value load
    input    [4:0]    dest    ;
    input    [15:0]   value   ;

    begin    opCode    = 'val   ;
            d          = dest  ;
            l          = 5'b0  ;
            v          = value ;
            endLine    ;

    end
endtask

// RUNNING
initial begin    addrCounter = 0;
                'include "0_program.sv" // first pass
                addrCounter = 0;
                'include "0_program.sv" // second pass

end

```

17.5.3 Simulator

```

/*****
File name: testRISC.sv

```


Circuit name:

Description:

*****/

'include "0_DEFINES.vh"

module testRISC;

logic inta ;

logic intIn ;

logic reset ;

logic clk ;

integer i ;

'include "0_RISCcodeGenerator.sv"

initial **begin** clk = 0 ;

forever #1 clk = ~clk ;

end

initial

begin intIn = 1 ;

 reset = 1 ;

 // DISPLAY THE CONTENT OF PROGRAM MEMORY

for (i=0; i<16; i=i+1)

\$display("progMemory[%0d]\ t= %b", i ,
 dut.memSys.progMemory[i]) ;

 #4 reset = 0 ;

 #100 **\$finish** ;

end

toyRISCsystem dut(inta ,
 intIn ,
 reset ,
 clk);

 // MONITOR FOR PROGRAM LAOD & CONTROLLER

initial **begin**

\$monitor("t=%0d\pc=%0d\RF=[%0d,%0d,%0d,%0d,%0d,%0d,%0d,%0d]\
 leftOp2=%0d\rightOp2=%0d\result3=%0d\intState=%b\inta=%b",

 \$time ,

 dut.processor.PC,

 dut.processor.regFile.rf[0],

 dut.processor.regFile.rf[1],

 dut.processor.regFile.rf[2],

 dut.processor.regFile.rf[3],

 dut.processor.regFile.rf[4],

 dut.processor.regFile.rf[5],

 dut.processor.regFile.rf[30],

 dut.processor.regFile.rf[31],

 dut.processor.leftOp2 ,

 dut.processor.rightOp2 ,

```

        dut.processor.result3 ,
        dut.processor.dcd.intState ,
        dut.processor.dcd.inta );

    end
endmodule

```

17.5.4 Testing

```

/*****
File name: program.sv
Circuit name:
Description:
*****/
/*
    NOP          ;
    VAL(0,1)     ;
    VAL(1,2)     ;
    VAL(2,3)     ;
    VAL(3,4)     ;
    VAL(4,5)     ;
    NOP          ;
    NOP          ;
    ADD(5,4,3)   ;
    HALT         ;
    HALT         ;
// */

/*
    VAL(0,2)     ;
    VAL(1,4)     ;
    VAL(2,8)     ;
    NOP          ;
    MULT(3,1,2)  ;
    NOP          ;
    HALT         ;
// */
/*
    VAL(31,13)   ;
    VAL(2,23)    ;
    VAL(0,13)    ;
    VAL(1,13)    ;
    EI           ;
    ADDV(0,0,1)  ;
    VAL(1,1)     ;
    VAL(2,222)   ;
    NOP          ;

```

```

        ADDV(0,0,4) ;
        HALT      ;
        HALT      ;
        NOP       ;
        // subroutine triggered by interrupt
        ADDV(30,30,-1) ;
        NOP       ;
        NOP       ;
        NOP       ;
        RET(30)    ;
// */
/*
        NOP       ;
        VAL(0,-3)  ;
        NOP       ;
        NOP       ;
        LB(1);    ADDV(0,0,1);
        NOP       ;
        BRNZ(0,1)  ;
        NOP       ;
        HALT      ;
        HALT      ;
// */
/*
        VAL(0,1)   ;
        VAL(1,55)  ;
        STORE(0,1) ;
        READ(0)    ;
        LOAD(2)    ;
        HALT       ;
// */

```

17.6 Forwarding toyRISC

17.6.1 Structure

```

/*****
File name: toyRISCsystem.sv
Circuit name:
Description:
*****/
`include "0_DEFINES.vh"
module toyRISCsystem(    output logic   inta      ,
                        input  logic   intIn     ,
                                reset      ,
                                clk       );

    logic [31:0] instruction ;

```

```

logic    [9:0]    PC          ;
logic    [31:0]   dataIn      ;
logic    [31:0]   dataOut     ;
logic    [9:0]    dataAddr    ;
logic          dataRead      ;
logic          dataWrite     ;

toyRISC processor( instruction ,
                  PC          ,
                  intIn       ,
                  inta        ,
                  dataIn       ,
                  dataOut      ,
                  dataAddr     ,
                  dataRead     ,
                  dataWrite    ,
                  reset        ,
                  clk          );
memorySystem memSys(instruction ,
                  PC          ,
                  dataIn       ,
                  dataOut      ,
                  dataAddr     ,
                  dataRead     ,
                  dataWrite    ,
                  clk          );

endmodule

```

```

/*****
File name: memorySystem.sv
Circuit name:
Description:
*****/
module memorySystem(output logic [31:0] instruction ,
                  input logic [9:0] PC ,
                  output logic [31:0] dataIn ,
                  input logic [31:0] dataOut ,
                  input logic [9:0] dataAddr ,
                  input logic dataRead ,
                  input logic dataWrite ,
                  input logic clk );

logic [31:0] dataMemory[0:1023] ;
logic [31:0] progMemory[0:1023] ;

assign instruction = progMemory[PC] ;

always_ff @(posedge clk) begin

```

```

        if (dataWrite) dataMemory[dataAddr] <= dataOut ;
    end

    assign dataIn    = dataMemory[dataAddr];

endmodule

```

```

/*****
File name: .sv
Circuit name:
Description:
*****/
/*****
File name: toyRISC.sv
                                toyRISC
*****/
`include "0_DEFINES.vh"
module toyRISC( input    logic [31:0] instruction ,
                output   logic [9:0]  pc          ,
                input    logic         intIn       ,
                output   logic         inta        ,
                input    logic [31:0]  dataIn      ,
                output   logic [31:0]  dataOut     ,
                output   logic [9:0]  dataAddr    ,
                output   logic         dataRead    ,
                output   logic         dataWrite   ,
                input    logic         reset      ,
                input    logic         clk        );

    logic [31:0] instruction1;
    logic [31:0] instruction2;
    logic [31:0] instruction3;
    logic [31:0] instruction4;
    logic [9:0]  pc1         ;
    logic [31:0] leftOp      ;
    logic [1:0]  nextPCsel   ;
    logic        we4         ;
    logic [31:0] result4     ;
    logic [3:0]  opSel       ;
    logic [3:0]  opSel2      ;
    logic        zero        ;
    logic [31:0] leftOp2     ;
    logic [31:0] rightOp2    ;
    logic [31:0] leftOp3     ;
    logic [31:0] rightOp3    ;
    logic [31:0] result3     ;
    logic        we          ;
    logic        memSel      ;

```

```

DECODE dcd( .instruction1  (instruction1  ),
            .instruction2  (instruction2  ),
            .instruction3  (instruction3  ),
            .zero          (zero         ),
            .intIn         (intIn        ),

            .inta          (inta         ),
            .nextPCsel     (nextPCsel    ),
            .opSel         (opSel        ),
            .memSel        (memSel       ),
            .we            (we           ),
            .dataRead      (dataRead     ),
            .dataWrite     (dataWrite    ),

            .reset         (reset        ),
            .clk           (clk          ));

INSTRFETCH progCount(  instruction ,
                      leftOp      ,
                      nextPCsel   ,

                      pc          ,
                      instruction1 ,
                      pc1         ,

                      reset       ,
                      clk         );

OPSFETCH regFile(  instruction1 ,
                  instruction4 ,
                  pc1          ,
                  inta         ,
                  we4          ,
                  result4      ,
                  opSel        ,

                  zero         ,
                  leftOp       ,
                  leftOp2      ,
                  rightOp2     ,
                  opSel2       ,
                  instruction2 ,

                  clk          );

EXECUTE execute(leftOp2      ,
               rightOp2     ,
               opSel2       ,
               instruction2 ,
               result4      ,

```

```

        leftOp3      ,
        result3      ,
        rightOp3     ,
        instruction3  ,

        clk           );

    DATAMEM datamem( result3      ,
                    instruction3 ,
                    we           ,
                    memSel      ,
                    dataIn      ,

                    result4      ,
                    instruction4 ,
                    we4          ,

                    clk           );

    assign dataAddr = leftOp3 ;
    assign dataOut  = rightOp3 ;
endmodule // Synthesis results: #LUT=335, #FF=204, #DSP=3

```

```

/*****
File name: DECODE.sv
Circuit name:
Description:
*****/
`include "0_DEFINES.vh"
module DECODE(  input  logic [31:0]  instruction1 ,
                input  logic [31:0]  instruction2 ,
                input  logic [31:0]  instruction3 ,
                input  logic          zero        ,
                input  logic          intIn       ,

                output logic          inta        ,
                output logic [1:0]    nextPCsel  ,
                output logic [3:0]    opSel      ,
                output logic          memSel     ,
                output logic          we         ,
                output logic          dataRead   ,
                output logic          dataWrite  ,

                input  logic          reset      ,
                input  logic          clk        );

    logic [5:0] opCode1 ;

```

```

    logic [4:0] destAddr1    ;
    logic [4:0] leftAddr1    ;
    logic [4:0] rightAddr1   ;
    logic [4:0] destAddr2    ;
    logic [5:0] opCode3      ;
    logic [4:0] destAddr3    ;
    logic      leftFwd       ;
    logic      rightFwd      ;

    assign opCode1      = instruction1[31:26] ;
    assign leftAddr1    = instruction1[20:16] ;
    assign rightAddr1   = instruction1[15:11] ;
    assign destAddr2    = instruction2[25:21] ;
    assign destAddr3    = instruction3[25:21] ;
    assign opCode3      = instruction3[31:26] ;

// Interrupt automaton
    logic [2:0] intState    ;
/*
    intState = 000: intDisable
    intState = 001: intEnable
    intState = 010: intExec1
    intState = 011: intExec2
    intState = 100: intExec3
    intState = 101: intExec4
*/
    always_ff @(posedge clk)
        if (reset) intState <= 3'b000 ;
        else
            begin
                if (opCode1 == 'eint) intState <= 3'b001 ;
                if (opCode1 == 'dint) intState <= 3'b000 ;
                if (intIn && (intState == 3'b001)) intState <= 3'b010 ;
                if (intState == 3'b010) intState <= 3'b011 ;
                if (intState == 3'b011) intState <= 3'b100 ;
                if (intState == 3'b100) intState <= 3'b101 ;
                if (intState == 3'b101) intState <= 3'b000 ;
            end
        assign inta = intState == 3'b011 ;
// end Interrupt automaton

// Program control
    logic [5:0] jmpSel    ;

    assign jmpSel = (intState > 3'b011) ? 'nop : opCode1 ;

    always_comb if (inta) nextPCsel = 2'b11 ;
                  else
                      case(jmpSel)

```



```

        'halt      : nextPCsel = 2'b00          ;
        'rjmp      : nextPCsel = 2'b10          ;
        'zbr       : nextPCsel = zero ? 2'b10 : 2'b01 ;
        'nzbr      : nextPCsel = zero ? 2'b01 : 2'b10 ;
        'ret       : nextPCsel = 2'b11          ;
        default    : nextPCsel = 2'b01          ;
    endcase
// end program control

// Forwarding section
    assign leftFwdY =
        (leftAddr1 == destAddr2) && (opCode1[5] == 1'b1);
    assign rightFwdY =
        (rightAddr1 == destAddr2) && (opCode1[5] == 1'b1);
    assign leftFwdO =
        (leftAddr1 == destAddr3) && (opCode1[5] == 1'b1);
    assign rightFwdO =
        (rightAddr1 == destAddr3) && (opCode1[5] == 1'b1);
    always_comb
        case (opCode1)
            'addv    : opSel[1:0] = 2'b11 ;
            'multv   : opSel[1:0] = 2'b11 ;
            'addvc   : opSel[1:0] = 2'b11 ;
            'val     : opSel[1:0] = 2'b11 ;
            default  : if (rightFwdY)      opSel[1:0] = 2'b01 ;
                      else if (rightFwdO) opSel[1:0] = 2'b10 ;
                      else                opSel[1:0] = 2'b00 ;

        endcase
    always_comb
        if (leftFwdY)      opSel[3:2] = 2'b01 ;
        else if (leftFwdO) opSel[3:2] = 2'b10 ;
        else               opSel[3:2] = 2'b00 ;
// end forwarding section

    assign dataWrite = opCode3 == 'store ;
    assign dataRead  = opCode3 == 'read  ;
    assign memSel    = opCode3 == 'read  ;
    assign we        = ((opCode3[5:4] == 2'b11) |
                        (opCode3 == 'val) |
                        (opCode3 == 'read)) & !(intState > 3'b001);
endmodule

```

```

/*****
File name: INSTRFETCH.sv
Circuit name:
Description:
*****/
module INSTRFETCH ( input  logic [31:0]  instruction ,

```

```

        input  logic [31:0] leftOp      ,
        input  logic [1:0]  nextPCsel   ,

        output logic [9:0]   pc          ,
        output logic [31:0]  instruction1 ,
        output logic [9:0]   pc1         ,

        input  logic        reset       ,
        input  logic        clk         );

// PipeReg0
always_ff @(posedge clk)
    if (reset) begin
        pc <= -1 ;
        instruction1 <= 0 ;
    end
    else begin case(nextPCsel)
        2'b00: pc <= pc ;
        2'b01: pc <= pc + 1 ;
        2'b10: pc <= instruction1[9:0] + pc1 ;
        2'b11: pc <= leftOp ;
    endcase

// PipeReg1
        instruction1 <= instruction ;
        pc1 <= pc ;

    end

endmodule

```

```

/*****
File name: OPSFETCH.sv
Circuit name:
Description:
*****/
module OPSFETCH(
    input  logic [31:0] instruction1 ,
    input  logic [31:0] instruction4 ,
    input  logic [9:0]  pc1          ,
    input  logic        inta         ,
    input  logic        we4         ,
    input  logic [31:0] result4      ,
    input  logic [3:0]  opSel        ,

    output logic        zero         ,
    output logic [31:0] leftOp       ,
    output logic [31:0] leftOp2      ,
    output logic [31:0] rightOp2     ,
    output logic [3:0]  opSel2       ,
    output logic [31:0] instruction2 ,

    input  logic        clk         );

```

```

logic    [31:0]  rf[0:31]      ;
logic    [31:0]  rightOp      ;
logic    we      ;
logic    [4:0]   destAddr     ;
logic    [31:0]  result       ;
logic    [4:0]   leftAddr      ;
logic    [4:0]   rightAddr     ;

always_ff @(posedge clk) if (we) rf[destAddr] <= result ;

assign leftOp      =
    ((destAddr == leftAddr) && we) ? result : rf[leftAddr]    ;
assign rightOp     =
    ((destAddr == rightAddr) && we) ? result : rf[rightAddr]  ;
assign zero        = leftOp == 0                             ;
assign rightAddr   = instruction1[15:11]                      ;

// PipeReg2:
always_ff @(posedge clk)
    begin
        leftOp2      <= leftOp          ;
        rightOp2     <= rightOp         ;
        opSel2       <= opSel           ;
        instruction2  <= instruction1    ;
    end

    intUnit intunit(.we      (we          ),
                    .destAddr (destAddr    ),
                    .result   (result      ),
                    .leftAddr (leftAddr    ),

                    .we4      (we4         ),
                    .destAddr4 (instruction4[25:21] ),
                    .result4   (result4     ),
                    .leftAddr1 (instruction1[20:16] ),
                    .pc1       (pc1         ),
                    .inta      (inta        ));
endmodule

/*****
File name: EXECUTE.sv
Circuit name:
Description:
*****/
'include "0_DEFINES.vh"
module EXECUTE( input    logic [31:0]    leftOp2      ,

```

```

        input  logic [31:0]    rightOp2    ,
        input  logic [3:0]     opSel2      ,
        input  logic [31:0]    instruction2 ,
        input  logic [31:0]    result4     ,

        output logic [31:0]    leftOp3     ,
        output logic [31:0]    result3     ,
        output logic [31:0]    rightOp3    ,
        output logic [31:0]    instruction3 ,

        input                                clk          );

    logic [31:0]    rightIn  ;
    logic [31:0]    leftIn   ;
    logic [31:0]    out      ;
    logic [31:0]    value    ;

    assign value = {{16{instruction2[15]}} , instruction2[15:0]} ;

    always_comb case (opSel2[1:0])
        2'b00: rightIn = rightOp2 ;
        2'b01: rightIn = result3  ;
        2'b10: rightIn = result4  ;
        2'b11: rightIn = value    ;
    endcase

    always_comb case (opSel2[3:2])
        2'b00: leftIn = leftOp2 ;
        2'b01: leftIn = result3 ;
        2'b10: leftIn = result4 ;
        2'b11: leftIn = 31'b0   ;
    endcase

    ALU alu (.out      (out          ),
             .leftIn   (leftIn       ),
             .rightIn  (rightIn      ),
             .opCode   (instruction2[31:26]));

    // PipeReg3
    always_ff @(posedge clk)
        begin
            leftOp3    <= leftIn      ;
            result3     <= out        ;
            rightOp3    <= rightIn     ;
            instruction3 <= instruction2 ;
        end
endmodule

```

```

/*****
File name: DATAMEM.sv

```

Circuit name:

Description:

```

*****/
module DATAMEM( input  logic [31:0]    result3    ,
                input  logic [31:0]    instruction3 ,
                input  logic          we          ,
                input  logic          memSel       ,
                input  logic [31:0]    dataIn     ,

                output logic [31:0]    result4     ,
                output logic [31:0]    instruction4 ,
                output logic          we4          ,

                input  logic          clk          );

    always_ff @(posedge clk)
        begin
            result4      <= memSel ? dataIn : result3;
            instruction4 <= instruction3          ;
            we4          <= we                    ;
        end
endmodule

```

/******

File name: ALU.sv

Circuit name:

Description:

*****/

'include "0_DEFINES.vh"

```

module ALU( output logic [31:0] out    ,
            input logic [31:0] leftIn  ,
            input logic [31:0] rightIn ,
            input logic [5:0]  opCode );

    logic [32:0] sum ;
    logic [32:0] dif ;

    assign sum = leftIn + rightIn ;
    assign dif = leftIn - rightIn ;

    always_comb case(opCode)
        'add    : out = sum[31:0] ;
        'sub    : out = dif[31:0] ;
        'addv   : out = sum[31:0] ;
        'mult   : out = leftIn * rightIn ;
        'multv  : out = leftIn * rightIn ;
        'adde   : out = sum[32] ;
        'subc   : out = dif[32] ;
        'addvc  : out = sum[32] ;
    endcase

```

```

        'lsh      : out = leftIn >> 1                      ;
        'ash      : out = {leftIn[31], leftIn[31:1]}       ;
        'move     : out = leftIn                          ;
        'swap     : out = {leftIn[15:0], leftIn[31:16]}    ;
        'bwnot    : out = ~leftIn                         ;
        'bwand    : out = leftIn & rightIn                ;
        'bwor     : out = leftIn | rightIn                ;
        'bwxor    : out = leftIn ^ rightIn                ;
        // 'load  : out = rightIn                        ;
        'val      : out = rightIn                         ;
        default   : out = leftIn                          ;
    endcase
endmodule

```

```

/*****
File name: intUnit.sv
Circuit name:
Description:
*****/
module intUnit( output logic we,
                output logic [4:0] destAddr,
                output logic [31:0] result,
                output logic [4:0] leftAddr,

                input logic we4,
                input logic [4:0] destAddr4,
                input logic [31:0] result4,
                input logic [4:0] leftAddr1,
                input logic [9:0] pc1,
                input logic inta
            );

    assign we = inta ? 1'b1 : we4 ;
    assign destAddr = inta ? 5'b11110 : destAddr4 ;
    assign result = inta ? {21'b0, pc1} : result4 ;
    assign leftAddr = inta ? 5'b11111 : leftAddr1 ;
endmodule

```

17.6.2 Code generator

```

/*****
File name: RISCcodeGenerator.sv
Circuit name:
Description:
*****/

```

```

reg [5:0]    opCode      ;
reg [4:0]    d           ;
reg [4:0]    l           ;
reg [4:0]    r           ;
reg [15:0]   v           ;
reg [9:0]    addrCounter ;
reg [9:0]    labelTab[0:1023];

‘include ”0_DEFINES.vh”

task endLine;
  begin
    dut.memSys.progMemory[addrCounter][31:0] =
      {
        opCode ,
        d      ,
        l      ,
        v      }
    addrCounter = addrCounter + 1 ;
  end
endtask

// sets labelTab in the first pass
// associating 'counter' with 'labelIndex'
task LB ;
  input [5:0] labelIndex;

  labelTab[labelIndex] = addrCounter;
endtask
// uses the content of labelTab in the second pass
task ULB;
  input [5:0] labelIndex;

  v = labelTab[labelIndex] - addrCounter ;
endtask

// CONTROL INSTRUCTIONS
task NOP; // no operation
  begin
    opCode = ‘nop ;
    {d,l,v} = 26’b0 ;
    endLine ;
  end
endtask

task RJMP; // relative jump
  input [15:0] label ;

  begin
    opCode = ‘rjmp ;
    d      = 5’b0 ;
    l      = 5’b0 ;
    ULB(label) ;
  end

```

```

        endLine      ;
    end
endtask

task BRZ; // branch if zero
    input  [4:0] left ;
    input  [15:0] label ;

    begin
        opCode = 'zbr ;
        d      = 5'b0 ;
        l      = left ;
        ULB(label) ;
        endLine ;
    end
endtask

task BRNZ; // branch if not zero
    input  [4:0] left ;
    input  [15:0] label ;

    begin
        opCode = 'nzbr ;
        d      = 5'b0 ;
        l      = left ;
        ULB(label) ;
        endLine ;
    end
endtask

task RET; // return from subroutine
    input  [4:0] left ;

    begin
        opCode = 'ret ;
        d      = 5'b0 ;
        l      = left ;
        v      = 16'b0 ;
        endLine ;
    end
endtask

task HALT; // halt running
    begin
        opCode = 'halt ;
        {d,l,v} = 26'b0 ;
        endLine ;
    end
endtask

task EI; // enable interrupt
    begin
        opCode = 'eint ;
        {d,l,v} = 26'b0 ;
        endLine ;
    end
endtask

```



```

        end
    endtask

    task DI; // disable interrupt
        begin
            opCode = 'dint ;
            {d,l,v} = 26'b0 ;
            endLine
        ;
        end
    endtask

// ARITHMETIC & LOGIC INSTRUCTIONS
    task ADD; // addition
        input [4:0] dest ;
        input [4:0] left ;
        input [4:0] right ;

        begin
            opCode = 'add ;
            d = dest ;
            l = left ;
            v = {right, 11'b0};
            endLine
        ;
        end
    endtask

    task SUB; // subtract
        input [4:0] dest ;
        input [4:0] left ;
        input [4:0] right ;

        begin
            opCode = 'sub ;
            d = dest ;
            l = left ;
            v = {right, 11'b0};
            endLine
        ;
        end
    endtask

    task ADDV; // addition with value
        input [4:0] dest ;
        input [4:0] left ;
        input [15:0] value ;

        begin
            opCode = 'addv ;
            d = dest ;
            l = left ;
            v = value ;
            endLine
        ;
        end
    endtask

```

```

task MULT; // multiplication
  input    [4:0]    dest    ;
  input    [4:0]    left    ;
  input    [4:0]    right   ;

  begin    opCode    = 'mult          ;
           d         = dest          ;
           l         = left          ;
           v         = {right, 11'b0};
           endLine   ;

  end
endtask

task MULTV; // multiplication with value
  input    [4:0]    dest    ;
  input    [4:0]    left    ;
  input    [15:0]   value   ;

  begin    opCode    = 'multv;
           d         = dest          ;
           l         = left          ;
           v         = value         ;
           endLine   ;

  end
endtask

task ADDC; // carry from addition
  input    [4:0]    dest    ;
  input    [4:0]    left    ;
  input    [4:0]    right   ;

  begin    opCode    = 'addc          ;
           d         = dest          ;
           l         = left          ;
           v         = {right, 11'b0};
           endLine   ;

  end
endtask

task SUBC; // carry from subtract
  input    [4:0]    dest    ;
  input    [4:0]    left    ;
  input    [4:0]    right   ;

  begin    opCode    = 'subc          ;
           d         = dest          ;
           l         = left          ;
           v         = {right, 11'b0};
           endLine   ;

  end

```

```

endtask

task ADDVC; // carry from addition with value
  input    [4:0]  dest    ;
  input    [4:0]  left    ;
  input    [15:0] value    ;

  begin    opCode   = 'addvc;
           d        = dest    ;
           l        = left    ;
           v        = value    ;
           endLine   ;
  end
endtask

task LSH; // logic shift with one position
  input    [4:0]  dest    ;
  input    [4:0]  left    ;

  begin    opCode   = 'lsh   ;
           d        = dest    ;
           l        = left    ;
           v        = 16'b0   ;
           endLine   ;
  end
endtask

task ASH; // arithmetic shift with one position
  input    [4:0]  dest    ;
  input    [4:0]  left    ;

  begin    opCode   = 'ash   ;
           d        = dest    ;
           l        = left    ;
           v        = 16'b0   ;
           endLine   ;
  end
endtask

task MOVE; // data move inside register file
  input    [4:0]  dest    ;
  input    [4:0]  left    ;

  begin    opCode   = 'move  ;
           d        = dest    ;
           l        = left    ;
           v        = 16'b0   ;
           endLine   ;
  end
endtask

```

```

task SWAP; // swap in register
    input    [4:0]    dest    ;
    input    [4:0]    left    ;

    begin    opCode    = 'swap    ;
            d          = dest    ;
            l          = left    ;
            v          = 16'b0    ;
            endLine    ;

    end
endtask

task NOT; // bitwise NOT
    input    [4:0]    dest    ;
    input    [4:0]    left    ;

    begin    opCode    = 'bwnot;
            d          = dest    ;
            l          = left    ;
            v          = 16'b0    ;
            endLine    ;

    end
endtask

task AND; // bitwise AND
    input    [4:0]    dest    ;
    input    [4:0]    left    ;
    input    [4:0]    right   ;

    begin    opCode    = 'bwand    ;
            d          = dest    ;
            l          = left    ;
            v          = {right, 11'b0};
            endLine    ;

    end
endtask

task OR; // bitwise OR
    input    [4:0]    dest    ;
    input    [4:0]    left    ;
    input    [4:0]    right   ;

    begin    opCode    = 'bwor    ;
            d          = dest    ;
            l          = left    ;
            v          = {right, 11'b0};
            endLine    ;

    end
endtask

```

```

task XOR; // bitwise XOR
    input    [4:0]    dest    ;
    input    [4:0]    left    ;
    input    [4:0]    right   ;

    begin    opCode    = 'bwxor    ;
            d          = dest      ;
            l          = left      ;
            v          = {right, 11'b0};
            endLine
    end
endtask

// DATA TRANSFER INSTRUCTIONS
task READ; // data read
    input    [4:0]    dest    ;
    input    [4:0]    left    ;

    begin    opCode    = 'read    ;
            d          = dest      ;
            l          = left      ;
            v          = 16'b0    ;
            endLine
    end
endtask

task STORE; // data store
    input    [4:0]    left    ;
    input    [4:0]    right   ;

    begin    opCode    = 'store    ;
            d          = 5'b0      ;
            l          = left      ;
            v          = {right, 11'b0};
            endLine
    end
endtask

task VAL; // value load
    input    [4:0]    dest    ;
    input    [15:0]   value    ;

    begin    opCode    = 'val     ;
            d          = dest      ;
            l          = 5'b0      ;
            v          = value     ;
            endLine
    end
endtask

```

```

// RUNNING
initial begin
    addrCounter = 0;
    'include "0_program.sv" // first pass
    addrCounter = 0;
    'include "0_program.sv" // second pass
end

```

17.6.3 Simulator

```

/*****
File name: testRISC.sv
Circuit name:
Description:
*****/
'include "0_DEFINES.vh"
module testRISC;
    logic    inta    ;
    logic    intIn   ;
    logic    reset   ;
    logic    clk     ;

    integer i    ;

    'include "0_RISCcodeGenerator.sv"

    initial begin
        clk = 0 ;
        forever #1 clk = ~clk ;
    end

    initial
        begin
            intIn      = 1 ;
            reset      = 1 ;
            // DISPLAY THE CONTENT OF PROGRAM MEMORY
            for (i=0; i<16; i=i+1)
                $display("progMemory[%0d]\t\t=\t\t%b", i ,
                        dut.memSys.progMemory[i]) ;
            #4      reset      = 0 ;
            #100    $finish    ;
        end

    toyRISCsystem dut( inta    ,
                       intIn   ,
                       reset   ,
                       clk     );

    // MONITOR FOR PROGRAM LAOD & CONTROLLER

```

```

initial begin
  $monitor("t=%0d pc=%0d RF=[%0d,%0d,%0d,%0d,%0d,%0d,%0d,%0d,%0d,%0d]
  dataWrite=%0b DM=[%0d,%0d,%0d,%0d] intState=%b inta=%b",
    $time,
    dut.processor.pc,
    dut.processor.regFile.rf[0],
    dut.processor.regFile.rf[1],
    dut.processor.regFile.rf[2],
    dut.processor.regFile.rf[3],
    dut.processor.regFile.rf[4],
    dut.processor.regFile.rf[5],
    dut.processor.regFile.rf[30],
    dut.processor.regFile.rf[31],
    dut.processor.dataWrite,
    dut.memSys.dataMemory[0],
    dut.memSys.dataMemory[1],
    dut.memSys.dataMemory[2],
    dut.memSys.dataMemory[3],
    dut.processor.dcd.intState,
    dut.processor.dcd.inta);
end
endmodule

```

17.6.4 Testing

```

/*****
File name: program.sv
Circuit name:
Description:
*****/
/*
    NOP          ;
    VAL(0,1)     ;
    VAL(1,2)     ;
    VAL(2,3)     ;
    VAL(3,4)     ;
    VAL(4,5)     ;
    //NOP        ;
    //NOP        ;
    //NOP        ;
    ADD(5,4,3)   ;
    HALT         ;
    HALT         ;
// */

/*

```

```

        VAL(0,2)      ;
        VAL(1,4)      ;
        VAL(2,8)      ;
        MULT(3,1,2)   ;
        ADDV(0,0,5)   ;
        NOP           ;
        HALT          ;
        HALT          ;
// */
/*
        VAL(31,13)    ;
        VAL(2,23)     ;
        VAL(0,24)     ;
        VAL(1,13)     ;
        EI            ;
        ADDV(2,31,1);
        VAL(1,1)      ;
        VAL(2,222)    ;
        NOP           ;
        ADD(0,1,2)    ;
        HALT          ;
        HALT          ;
        NOP           ;
// subroutine triggered by interrupt
        ADDV(30,30,-2) ;
        VAL(3,44)      ;
        NOP            ;
        NOP            ;
        NOP            ;
        RET(30)         ;
        NOP            ;
// */
// *
        NOP           ;
        VAL(0,5)       ;
LB(1);  ADDV(0,0,-1);
        NOP           ;
        NOP           ;
        BRNZ(0,1)      ;
        NOP           ;
        HALT          ;
        HALT          ;
// */
/*
        VAL(1,55)      ;
        VAL(0,1)        ;
        STORE(0,1)     ;
        NOP            ;
        NOP            ;
        READ(2,0)       ;

```



```
    HALT      ;  
    HALT      ;  
// */
```

Chapter 18

RISC-V TBD

Chapter 19

RISC-V Multithreaded TBD

Chapter 20

Using the Heterogeneous System hRISC

20.1	Micro-architectures	417
20.1.1	Host micro-architecture	417
20.1.2	Accelerator micro-architecture	419
20.1.3	Putting all together	424
20.2	Heterogeneous architecture	425
20.2.1	Host architecture	425
20.2.2	Accelerator's dual architecture	427
20.3	Programming heterogeneous system	430
20.3.1	Direct programming using accelerator's assembly	431
20.3.2	FLA 0.1	435
20.3.3	Programming using FLA 0.1	439

This appendix brings together in a heterogeneous configuration (see 14.6.2), under the name *hRISC*, the mono-core toyRISC structure with the many-core pRISC structure. The two structures are assembled into a system in order to simulate the functioning of a heterogeneous computer thus structured. First, the micro-architectures associated with the system components will be described. The architecture of the entire system is then defined. Programming examples are provided at the end when the development of a function library core at the lowest level in the system are also presented.

The top module of the system is listed in Listing 20.1. The listed module contains three modules – HOST, INTERFACES and ACCELERATOR – and their interconnections. Because the description is used only for simulation purpose, the only external connections are: `clock` and `reset`. The interconnections between the three module in top are:

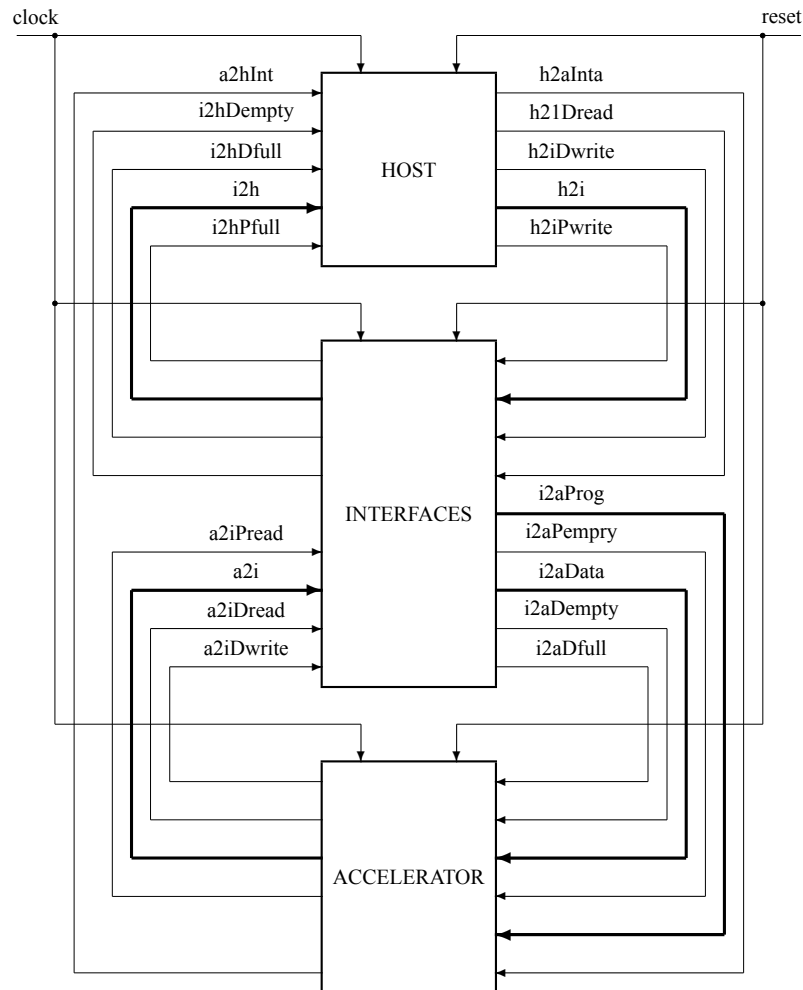


Figure 20.1: Heterogeneous System (`1_hetSys.sv`): HOST computer and many-cell ACCELERATOR interconnected through INTERFACES (three FIFOs: one for commands and programming and two for in and out of data).

from HOST to ACCELERATOR: program & data 64-bit path and the synchronizing signals:

commands: write programs, initialize programs and write data or transfer parameters; each transfer is done on 64-bit words per clock cycle

flags: indicating that interfaces in ACCELERATOR are not ready to accept the current transfer of program or data

from ACCELERATOR to HOST: 64-bit data path and the synchronizing signals:

command: sends to HOST data in 64-bit words

flag: indicating that the interface ACCELERATOR in is not ready to send a 64-bit data word

synchronizing signals: used to signal to HOST the end of a computation generating an *interrupt* answered by HOST.

```

1  `include "O_DEFINES.vh"
2  module hetSys( input logic reset, clock );
3      logic [63:0] h2a ; // program & data
4      logic h2aProgWrite;
5      logic a2hProgFull ;
6      logic h2aDataWrite;
7      logic a2hDataFull ;
8      logic [63:0] a2h ; // data
9      logic h2aRead ;
10     logic a2hEmpty ;
11     logic a2hInt ; // interrupt
12     logic h2aInta ; // interrupt acknowledge
13
14     host HOST( h2a ,
15               h2aProgWrite,
16               a2hProgFull ,
17               h2aDataWrite,
18               a2hDataFull ,
19               a2h ,
20               h2aDataRead ,
21               a2hDataEmpty,
22               a2hInt ,
23               h2aInta ,
24               reset ,
25               clock );
26
27     accelerator ACCELERATOR(h2a ,
28                              h2aProgWrite,
29                              a2hProgFull ,
30                              h2aDataWrite,
31                              a2hDataFull ,
32                              a2h ,
33                              h2aDataRead ,
34                              a2hDataEmpty,
```



```

35         a2hInt      ,
36         h2aInta     ,
37         reset       ,
38         clock        );
39     endmodule

```

Listing 20.1: Heterogeneous system. File: 1_hetSys.sv. (SystemVerilog)

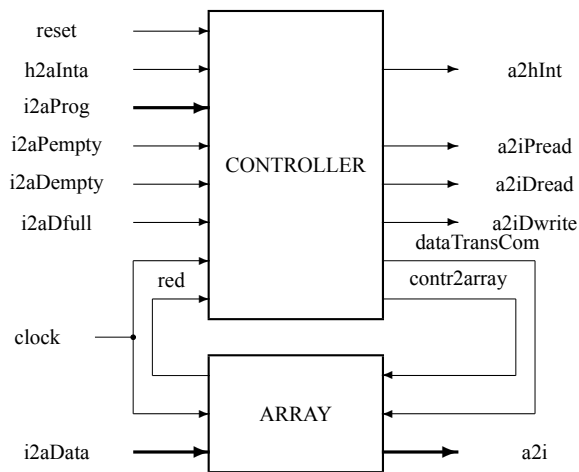


Figure 20.2: Accelerator (2_accelerator.sv): the many-cell engine ARRAY and the sequencer CONTROLLER (a mono-core computer).

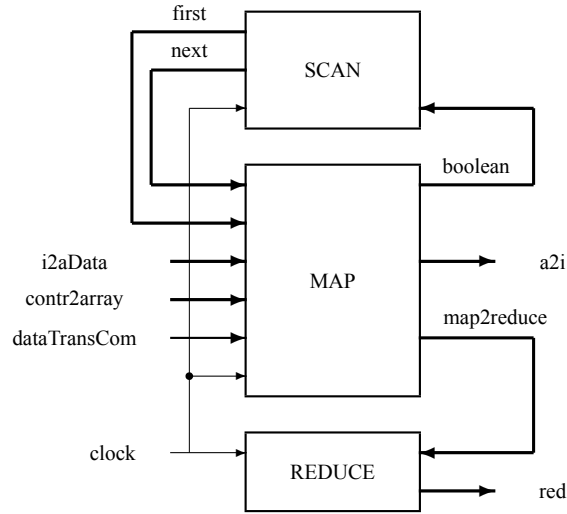


Figure 20.3: Array (3_array.sv): the array of p cells MAP loop connected with two *lor*-depth networks (REDUCE for reduce functions and SCAN for scan functions).

The simulation program ensures the initial loading of the HOST memory with the program it runs as well as with the program that the HOST loads into the ACCELERATOR during the system initialization process. If applicable, the HOST data memory is also initialized by the simulation program using a text file.

This introduction is followed by the description of the micro-architectures, the description of the hardware structure and examples of use in simulation of the designed system.

20.1 Micro-architectures

The heterogeneous structure is defined by three micro-architectures, each describing the micro-operations performed at the lowest level in the system.

20.1.1 Host micro-architecture

The host's 32-bit micro-architecture is defined in Listing 20.2. The associated instruction format is:

```
hInstr={opCode[5:0],dest[4:0],left[4:0],right[4:0],noUse[10:0]} |
       {opCode[5:0],dest[4:0],left[4:0],val[15:0]}
```

The internal state of the host (dimensioned with small memories to support fast simulation) is stored in:

rf[0:31] : two-output port register file

cr : carry bit

pc[9:0] : program counter

hDmem[0:1023] : host data memory

hPmem[0:1023] : host program memory

```

1  // CONTROL INSTRUCTIONS
2  `define hnop      6'b000000 // pc <= pc+1
3  `define hjmp      6'b000001 // pc <= pc+val
4  `define hjmpz     6'b000010 // pc <= (rf[left]==0) ? pc+val:pc+1
5  `define hjmpnz    6'b000011 // pc <= (rf[left]==0) ? pc+1:pc+val
6  `define hpsend    6'b100000 // pc <= (host2int[31:24]==
7                          // {`prun,`ctl}) ? pc+1 : pc; h2iPwrite = 1;
8                          // rf[left] <= rf[left]+!i2hPfull
9  `define hdget     6'b100010 // pc <= (leftOut==rightOut) ? pc+1 : pc;
10                         // i2hDread = 1;
11                         // rf[left] <= rf[left]+!i2hDfull
12 `define hdsend    6'b100001 // pc <= (leftOut==rightOut) ? pc+1 : pc;
13                         // h2iDwrite = 1;
14                         // rf[left] <= rf[left]+!i2hDfull
15 `define hintwait  6'b000111 // pc <= (int) ? pc+1 : pc
16 `define hajmp     6'b001000 // pc <= val
17 `define hhalt     6'b001001 // pc <= pc
18 `define hsttcc    6'b001010 // start host cycle counter
19 `define hstpcc    6'b001011 // stop host cycle counter
20
21 // FUNCTIONAL INSTRUCTIONS
22 `define hadd      6'b010000
23     // {cr,rf[dest]} <= rf[left]+rf[right]
24 `define haddcr    6'b010001
25     // {cr,rf[dest]} <= rf[left]+rf[right]+cr
26 `define hsub      6'b010010
27     // {cr,rf[dest]} <= rf[left]-rf[right]
28 `define hsubcr    6'b010011
29     // {cr,rf[dest]} <= rf[left]-rf[right]-cr
30 `define hmult     6'b010100
31     // {cr,rf[dest]} <= rf[left]*rf[right]
32 `define hbwand    6'b010101
33     // {cr,rf[dest]} <= {cr, rf[left] & rf[right]}
34 `define hbwor     6'b010110
35     // {cr,rf[dest]} <= {cr, rf[left] | rf[right]}
36 `define hbwxor    6'b010111
37     // {cr,rf[dest]} <= {cr, rf[left] ^ rf[right]}
38 `define haddv     6'b011000
39     // {cr,rf[dest]} <= rf[left]+{{16{val[15]}},val}
40 `define haddcrv   6'b011001
41     // {cr,rf[dest]} <= rf[left]+{{16{val[15]}},val}+cr
42 `define hsubv     6'b011010
43     // {cr,rf[dest]} <= rf[left]-{{16{val[15]}},val}
44 `define hsubcrv   6'b011011
45     // {cr,rf[dest]} <= rf[left]-{{16{val[15]}},val}-cr
46 `define hmultv    6'b011100

```

```

47     // {cr,rf[dest]} <= rf[left]*{{16{val[15]}},val}
48 `define hbwandv 6'b011101
49     // {cr,rf[dest]} <= {cr,rf[left]&{{16{val[15]}},val}}
50 `define hbworv 6'b011110
51     // {cr,rf[dest]} <= {cr,rf[left]|{{16{val[15]}},val}}
52 `define hbwxorv 6'b011111
53     // {cr,rf[dest]} <= {cr,rf[left]^{{16{val[15]}},val}}
54
55 // DATA TRANSFER INSTRUCTIONS
56 `define hfsend 6'b100011 // function send
57     // {code[5:0], `prun, `ctl, initAddress}
58 `define hssend 6'b100100 // scalar send
59     // {code[5:0], scalar}
60 `define hvalue 6'b101000
61     // {cr,rf[dest]} <= {cr, {{16{val[15]}},val}}
62 `define hinsval 6'b101001
63     // {cr,rf[dest]} <= {cr, {rf[left][15:0],val}}
64 `define hload 6'b101010
65     // {cr,rf[dest]} <= {cr, hDmem[rf[left]]}
66 `define hstore 6'b111011
67     // hDmem[rf[left]] <= rf[right]

```

Listing 20.2: Host micro-architecture. (SystemVerilog)

20.1.2 Accelerator micro-architecture

The accelerator's micro-architecture is a Cartesian one, i.e., each micro-instruction belongs to a set which is a Cartesian product of two sets of micro-instructions. The two micro-architectures are: controller micro-architecture and map micro-architecture.

20.1.2.1 Controller micro-architecture

The controller micro-architecture is defined in Listing 20.3. It is an accumulator-based one, starting from the idea that for intense computation the complexity of the computing engine can and should be kept low. The associated instruction format is:

$$\text{sInstr} = \{ \text{cOpcode}[4:0], \text{cMode}[2:0], \text{val}[23:0] \} \mid \{ 8'b11111_000, \text{cOpcode}[4:0], \text{val}[18:0] \}$$

The first instruction format refers to instructions using as left operand the accumulator, *acc*, and the right operand, *op*, the value, provided by *value* from instruction, or the local memory content addressed by *value* and, sometimes, a relative pointer *addr*. The co-operand, *coOp*, is the output of the Reduce network.

The internal state of the 32-bit word controller (dimensioned with small memories to support fast simulation) is stored in:

cMem[0:1023] : one output port register file

acc : accumulator register

cAddr : relative address register

cr : carry bit

pc[9:0] : program counter

pMem[0:1023] : 64-bit word program memory

```

1  // CONTROLLER'S AUTOMATON STATES
2  `define idle      2'b00
3  `define loading  2'b01
4  `define running  2'b10
5
6  // ADDRESSING MODES DEFINING THE RIGHT OPERAND OP
7  `define imm 3'b000 // op: {{8{val[23]}}}, val}
8  `define dir 3'b001 // op: cMem[val]
9  `define rel 3'b010 // op: cMem[cAddr+val]
10 `define rei 3'b011 // op: cMem[cAddr+val]; cAddr = cAddr+val
11 `define cim 3'b100 // op: coOp
12
13 // FUNCTIONAL INSTRUCTIONS
14 `define cadd      5'b00000 // {cr,acc} <= acc + op
15 `define caddcr    5'b00001 // {cr,acc} <= acc + op + cr
16 `define csub      5'b00010 // {cr,acc} <= acc - op
17 `define csubcr    5'b00011 // {cr,acc} <= acc - op - cr
18 `define cmult     5'b00100 // {cr,acc} <= {cr, acc * op}
19 `define cbwand    5'b00101 // {cr,acc} <= {cr, acc & op}
20 `define cbwor     5'b00110 // {cr,acc} <= {cr, acc | op}
21 `define cbwxor    5'b00111 // {cr,acc} <= {cr, acc ^ op}
22 // DATA MOVE INSTRUCTIONS
23 `define cload     5'b01000 // {cr,acc} <= {cr, op}
24 `define cstore    5'b01001 // cMem <= acc
25 `define cinsval   5'b01010 // {cr,acc} <= {cr, (acc[23:0],val[7:0])}
26
27 // SELECTS NO-OP INSTRUCTIONS
28
29 `define contr     5'b11111 // instruction: instr[23:19]
30
31 // CONTROL & GLOBAL INSTRUCTIONS for instr[31:24]= 11111_000
32 `define nop       5'b00000 // pc <= pc+1
33 `define jmp       5'b00001 // pc <= pc+val
34 `define brz       5'b00010 // pc <= (acc = 0) ? pc+val : pc+1
35 `define brnz      5'b00011 // pc <= (acc = 0) ? pc+1 : pc+val
36 `define brzdec    5'b00100 // pc <= (acc = 0) ? pc+val : pc+1
37 `define brnzdec   5'b00101 // pc <= (acc = 0) ? pc+1 : pc+val
38 `define ajmp      5'b00110 // pc <= val
39 `define halt      5'b00111 // pc <= pc
40
41 `define setint     5'b01000 // set interrupt
42
43 `define datains    5'b01001 // insert data in array
44 `define dataext    5'b01010 // extract data from array

```

```

44
45 `define param    5'b01011 // load parameter
46 `define pload    5'b01100 // load program starting to val
47 `define prun     5'b01101 // run the program at val
48
49 `define start     5'b01110 // start cycle counter
50 `define stop      5'b01111 // stop cycle coounter
51
52 `define gshift    5'b11101 // global_shift(val[2:0])
53
54 `define crela     5'b11110 // cAddr <= acc; load relative address
55 `define cshift    5'b11111 // local_shift(val[2:0])

```

Listing 20.3: Controller micro-architecture. (SystemVerilog)

Right operand select is specified by the 3-bit field, *cMode* used to select the right operand *op* in SEQUENCER where:

$$\text{acc} \leftarrow \text{acc} \quad \text{cOpcode} \quad \text{op}$$

The 5 modes of selecting the right operand are:

imm - the right operand, *op*, is **immediate**, given by the scalar provided in instruction:

$$\text{op} = \text{val} = \{\{8\{\text{val}[23]\}\}, \text{value}\}$$

dir - the right operand is read at the **direct** address in data memory:

$$\text{op} = \text{mem}[\text{value}]$$

rel - right operand is read from the **relative** to *addr* address in data memory:

$$\text{op} = \text{mem}[\text{addr} + \text{value}]$$

rei - right operand is read from the **relative** to *addr* address in data memory and the value of *addr* is actualized (**incremented**) to the current use:

$$\text{op} = \text{mem}; \quad \text{addr} \leftarrow \text{addr} + \text{value}$$

cim - right operand is the **co-operand** used **immediately**:

$$\text{op} = \text{ReduceOut}$$

20.1.2.2 Map micro-architecture

The sequencer micro-architecture is defined in Listing 20.4. It is an accumulator-based one, starting from the idea that for intense computation the complexity of the computing engine can and should be kept low. In addition, when we have a multicellular structure, it is good to keep the cells at a level of complexity and size as low as possible. The associated instruction format is:

```
sInstr={mOpcode[4:0], mMode[2:0], val[23:0]} |
      {8'b11111_000, mOpcode[4:0], val[18:0]}
```

The first instruction format refers to instructions using as left operand the accumulator, *acc*, and the right operand, *op*, the value, provided by *value* from instruction, or the local memory content addressed by *value* and, sometimes, a relative pointer *addr*. The co-operand, *coOp*, is the accumulator of the sequencer, *acc*.

The internal state of the Map array is stored in:

distributed memory : $\mathbf{M}_{mp}$ of form

$$\begin{bmatrix} s[00] & s[01] & \dots & s[0p-1] \\ s[10] & s[11] & \dots & s[1p-1] \\ \vdots & \vdots & \ddots & \vdots \\ s[m-10] & s[m-11] & \dots & s[m-1p-1] \end{bmatrix} = \begin{bmatrix} V[0] \\ V[1] \\ \vdots \\ V[m-1] \end{bmatrix} = [W[0] \ W[1] \ \dots \ W[p-1]]$$

where: $V[i]$ are vector distributed along the Map array of cells, and $W[j]$ are "vertical" vectors stored in the local memories (register files) in each cell.

accumulator vector :

$$\mathbf{ACC}_p = [acc[0] \ acc[1] \ \dots \ acc[p-1]]$$

where: $acc[i]$ is the accumulator register in each cell

relative address vector :

$$\mathbf{ADDR}_p = [addr[0] \ addr[1] \ \dots \ addr[p-1]]$$

where: $addr[i]$ is the relative address used in each cell to address the local memory

Boolean vector :

$$\mathbf{B}_p = [b[0] \ b[1] \ \dots \ b[p-1]]$$

where: $b[i]$ is the bit indicating by value 1 the active cell

carry vector :

$$\mathbf{CR}_p = [cr[0] \ cr[1] \ \dots \ cr[p-1]]$$

where: $cr[i]$ is the carry bit in each cell

index vector :

$$\mathbf{IX}_p = [0 \ ix_1 \ 1 \ p-1]$$

is used to identify the cell according to its position in the one-dimension array Map.

```
1
2 // ADDRESSING MODE DEFINING THE RIGHT OPERAND OP
3 `define imm 3'b000 // op: {{8{val[23]}}}, val}
4 `define dir 3'b001 // op: mem[val]
5 `define rel 3'b010 // op: mem[addr+val]
6 `define rei 3'b011 // op: mem[addr+val]; addr = addr+val
7 `define cim 3'b100 // op: coOp
8 `define cdr 3'b101 // op: mem[coOp]
```

```

9  `define crl 3'b110 // op: mem[addr+coOp]
10 `define cri 3'b111 // op: mem[addr+coOp]; addr = addr+coOp
11
12 // FUNCTIONAL INSTRUCTIONS
13 `define add      5'b00000 // {cr[i],acc[i]} <= acc[i] + op
14 `define addcr    5'b00001 // {cr[i],acc[i]} <= acc[i] + op + cr
15 `define sub      5'b00010 // {cr[i],acc[i]} <= acc[i] - op
16 `define subcr    5'b00011 // {cr[i],acc[i]} <= acc[i] - op - cr
17 `define mult     5'b00100 // {cr[i],acc[i]} <= {cr[i], acc[i] * op}
18 `define bwand    5'b00101 // {cr[i],acc[i]} <= {cr[i], acc[i] & op}
19 `define bwor     5'b00110 // {cr[i],acc[i]} <= {cr[i], acc[i] | op}
20 `define bwxor    5'b00111 // {cr[i],acc[i]} <= {cr[i], acc[i] ^ op}
21 // DATA MOVE INSTRUCTIONS
22 `define load      5'b01000 // {cr[i],acc[i]} <= {cr[i], op}
23 `define store     5'b01001 // aMem[address[i]] <= aAcc
24 `define insval    5'b01010 // acc[i] <= {acc[i][23:0], val[7:0]}
25
26 `define sendio    5'b01011 // ioReg[i] <= op; ioSet
27 `define getio     5'b01100 // mem[(addr[i])+val] <= ioReg[i]; ioRst
28
29 // SEARCH INSTRUCTIONS
30 `define search    5'b01101 // act[i]<=(b[i]&acc[i]=op)?act[i]:act[i]+1
31 `define csearch  5'b01110 // act[i]<=(b[i-1]&acc[i]=op)?0:act[i]+1
32 `define insert    5'b01111 // insert in the first active position in sr
33
34 // SELECTS NO-OP INSTRUCTIONS
35 `define contr     5'b11111 // instruction: instr[23:19]
36
37 // CONTROL & GLOBAL INSTRUCTIONS for instr[31:24]= 11111_000
38 `define allact    5'b00000 // act[i] <= 0
39 `define where     5'b00001 // act[i] <= (b[i]&cond) ? act[i] : [i]+1
40 `define elsew     5'b00010 // act[i] <= (act[i]=0)?1:(act[i]=1)?0:act[i]
41 `define back      5'b00011 // act[i] <= (act[i]=0)? 0 : act[i]-1
42 `define selsh     5'b00100 // selection global shift right
43
44 `define delete    5'b00110 // delete in first position in sr
45 `define getsr     5'b00111 // acc[i] <= sr[i]
46 `define sendsr    5'b01000 // sr[i] <= acc[i]
47
48 `define ixload    5'b01001 // acc[i] <= i
49
50 `define setred    5'b01010 // set reduce's function
51
52 `define sradd     5'b11011 // acc[i] <= acc[i] + sr[i]
53 `define srmin     5'b11100 // acc[i] <= min(acc[i], sr[i])
54 `define srmax     5'b11101 // acc[i] <= max(acc[i], sr[i])
55
56 `define arela     5'b11110 // addr <= aAcc
57 `define shift     5'b11111 // local_shift(val[2:0])

```

Listing 20.4: Map micro-architecture. (SystemVerilog)

Right operand select is specified by the 3-bit field, `mMode` used to select the right operand `op` in each cell from MAP where:

$$\text{acc}[i] \leq \text{acc}[i] \quad \text{mOpcode} \quad \text{op}$$

The 8 modes of selecting the right operand are:

imm - right operand, `op`, is **immediate**, given by the scalar provided in instruction:

$$\text{op}[i] = \text{val} = \{\{8\{\text{val}[23]\}\}, \text{val}\}$$

dir - right operand is read from the **direct** address in data memory:

$$\text{op}[i] = \text{vectMmem}[\text{val}]$$

rel - right operand is read from the **relative** to `addr` address in data memory:

$$\text{op}[i] = \text{mem}[\text{addr}[i] + \text{val}]$$

rei - right operand is read from the **relative** to `addr` address in data memory and the value of `addr` is actualized (**incremented**) to the current use:

$$\text{op}[i] = \text{mem}[\text{addr}[i] + \text{val}]; \text{addr}[i] \leq \text{addr}[i] + \text{val}$$

cim - right operand is the **co-operand** used **immediately**, i.e., `acc`:

$$\text{op}[i] = \text{acc}$$

cdr - operation is performed similar with `dir` mode but the memory location is selected by the **co-operand** used **direct** as address

$$\text{op}[i] = \text{mem}[\text{acc}]$$

crl - similar with `rel`, but the `addr` value is incremented with the **co-operand** used as **relative** address

$$\text{op}[i] = \text{mem}[\text{addr}[i] + \text{acc}]$$

cri - similar with `rei`, but the `addr` value is incremented with the **co-operand** used as **relative** address whose value is also **incrementad**

$$\text{op}[i] = \text{mem}[\text{addr}[i] + \text{acc}]; \text{addr}[i] \leq \text{addr}[i] + \text{acc}$$

20.1.3 Putting all together

The previously defined three micro-architecture are assembled in a unique file, `O_DEFINES.vh`.

```

1  // GLOBAL PARAMETERS
2  `define n (32)           // internal word
3  `define p (16)           // number of cells
4  `define m (1024)         // memory size (host, sequencer, cell)
5
6  // MICRO-ARCHITECTURES
7  `include "A08_hostDEFINES.vh"
8  `include "A08_sequencerDEFINES.vh"
```

```
9  `include "A08_mapDEFINES.vh"
```

Listing 20.5: Micro-architectures. File: 0_DEFINES.vh. (SystemVerilog)

20.2 Heterogeneous architecture

The general architecture of the hRISC heterogeneous system consists of the toyRISC architecture and the dual pRISC architecture.

20.2.1 Host architecture

The host architecture will be described through the host's instruction set architecture (hISA) and a version 0.1 function library architecture (FLA 0.1) that we propose.

20.2.1.1 Instruction set

Table 20.1: Host's Instruction Set Architecture.

OpCode	Commentaries	Assembly
CONTROL OPERATIONS		
nop	no operation: $pc \leq pc+1$	hNOP
hjmp	relative jump: $pc \leq pc+val$	hJMP(lb)
hjmpz	conditioned jump: $pc \leq (rf[left]==0) ? pc+val : pc+1$	hJMPZ(l, lb)
hjmpnz	conditioned jump: $pc \leq (rf[left]==0) ? pc+1 : pc+val$	hJMPNZ(l, lb)
hajmp	absolute jump: $pc \leq pc + val$	hAJMP(val)
hhalt	halt: $pc \leq pc$	hHALT
hpsend	send to Map a auto-delimited program	hPSEND(d, l)
hdget	get from Map a stream of data	hDGET(d, l, r)
hdsend	send to Map a stream of data	hDSEND(d, l, r)
hintwait	interrupt wait	hINTWAIT
hsttcc	start host's cycle counter	hSTARTCC
hstpcc	stop host's cycle counter	hSTOPCC
FUNCTIONAL OPERATIONS		
hadd	$\{cr, rf[dest]\} \leq rf[left] + rf[right]$	hADD(d, l, r)
haddcr	$\{cr, rf[dest]\} \leq rf[left] + rf[right] + cr$	hADDCR(d, l, r)
hsub	$\{cr, rf[dest]\} \leq rf[left] - rf[right]$	hSUB(d, l, r)
hsubcr	$\{cr, rf[dest]\} \leq rf[left] - rf[right] - cr$	hSUBCR(d, l, r)
hmult	$\{cr, rf[dest]\} \leq \{cr, rf[left] * rf[right]\}$	hMULT(d, l, r)
hbwand	$\{cr, rf[dest]\} \leq \{cr, rf[left] \& rf[right]\}$	hAND(d, l, r)
hbwor	$\{cr, rf[dest]\} \leq \{cr, rf[left] rf[right]\}$	hOR(d, l, r)
hbwxor	$\{cr, rf[dest]\} \leq \{cr, rf[left] \wedge rf[right]\}$	hXOR(d, l, r)
haddv	$\{cr, rf[dest]\} \leq rf[left] + val$	hADDV(d, l, val)
haddcrv	$\{cr, rf[dest]\} \leq rf[left] + val + cr$	hADDCRV(d, l, val)
hsubv	$\{cr, rf[dest]\} \leq rf[left] - val$	hSUBV(d, l, val)
hsubcrv	$\{cr, rf[dest]\} \leq rf[left] - val - cr$	hSUBCRV(d, l, val)
hmultv	$\{cr, rf[dest]\} \leq \{cr, rf[left] * val\}$	hMULTV(d, l, val)
hbwandv	$\{cr, rf[dest]\} \leq \{cr, rf[left] \& val\}$	hANDV(d, l, val)
hbworv	$\{cr, rf[dest]\} \leq \{cr, rf[left] val\}$	hORV(d, l, val)
hbwxorv	$\{cr, rf[dest]\} \leq \{cr, rf[left] \wedge val\}$	hXORV(d, l, val)
DATA TRANSFER INSTRUCTIONS		
hvalue	$\{cr, rf[dest]\} \leq \{cr, \{16\{val[15]\}\}, val\}$	hVALUE(d, val)
hinsval	$\{cr, rf[dest]\} \leq \{cr, \{rf[left][15:0], val\}\}$	hINSVAL(d, val)
hload	$\{cr, rf[dest]\} \leq \{cr, hDmem[rf[left]]\}$	hLOAD(d, l)
hstore	$hDmem[rf[left]] \leq rf[right]$	hSTORE(l, r)

20.2.1.2 Low-level library of functions (FLA 0.1)

An assembly program written for the host computer uses the hISA, and a first version of a library of functions, LFA 0.1 (see Table 20.2), where each function is implemented by a program executed by the accelerator. Thus, the library used by host is a hardware implemented library unlike the usual software-designed libraries.

Table 20.2: Host's Function Library Architecture (FLA) defined in file:
00_theKernel.sv.

Assembly	Commentaries
hSTART	Start controller's cycle counter
hSTOP	Stop controller's cycle counter
hINTRQ	Interrupt request
hVGENX(addr)	$V(addr) \leq IX$
hVGENN(addr,val)	$V(addr) \leq [val \text{ } val \text{ } \dots \text{ } val]$
hSQGENX	Generate square matrix $[V[0]=IX \text{ } V[1]=IX+1 \text{ } \dots \text{ } V[p-1]=IX+p-1]$
hSQGENN(val)	Generate matrix with $V[i] = [val \text{ } val \text{ } \dots \text{ } val]$, for $i = 0, \dots, p-1$
hUNIT	Generate unit matrix starting with $V[addr]$
hMSEND(addr,size)	Send matrix of size starting from $V[addr]$
hMGET(addr,size)	Get matrix of size and load from $V[addr]$
hSQMADD(d,l,r)	$M[d \text{ } d+p-1] \leq M[l \text{ } l+p-1] + M[r \text{ } r+p-1]$
hSQMMULT(d,l,r)	$V[d] \leq M[l \text{ } l+p-1] * V[r]$
hSQMMULT(d,l,r)	$M[d \text{ } d+p-1] \leq M[l \text{ } l+p-1] * M[r \text{ } r+p-1]$
hSQMMAC(d,l,r)	$M[d \text{ } d+p-1] \leq M[d \text{ } d+p-1] + (M[l \text{ } l+p-1] * M[r \text{ } r+p-1])$

20.2.2 Accelerator's dual architecture

20.2.2.1 Controller's architecture

Each assembly-level instruction is made by concatenating the operation micro-code (indicated on the first column of the following tables) with the micro-code that selects the operation mode (indicated on the first line in the following table).

Table 20.3: Instructions with operands from the controller's Instruction Set Architecture.

cOpcode \ cMode	imm	dir	rel	rei	cim
cadd	cVADD(s)	cADD(s)	cRADD(s)	cRIADD(s)	CADD
caddc	cVADDC(s)	cADDC(s)	cRADDC(s)	cRIADDC(s)	CADDC
csub	cVSUB(s)	cSUB(s)	cRSUB(s)	cRISUB(s)	CSUB
crsub	cVRSUB(s)	cRSUB(s)	cRRSUB(s)	cRIRSUB(s)	CRSUB
csubc	cVSUBC(s)	cSUBC(s)	cRSUBC(s)	cRISUBC(s)	CSUBC
crsubc	cVRSUBC(s)	cRSUBC(s)	cRRSUBC(s)	cRIRSUBC(s)	CRSUBC
cmult	cVMULT(s)	cMULT(s)	cRMULT(s)	cRIMULT(s)	CMULT
cbwand	cVAND(s)	cAND(s)	cRAND(s)	cRIAND(s)	CAND
cbwor	cVOR(s)	cOR(s)	cROR(s)	cRIOR(s)	COR
cbwxor	cVXOR(s)	cXOR(s)	cRXOR(s)	cRIXOR(s)	CXOR
cload	cVLOAD(s)	cLOAD(s)	cRLOAD(s)	cRILOAD(s)	CLOAD
cstore		cSTORE(s)	cRSTORE(s)	cRISTORE(s)	CCSTORE

Table 20.4: Sequencer's Instruction Set Architecture with no operand.

{cOpcode, value[2:0]}	Commentaries	Assembly
{cshift,4}	Shift right one bit position	cSHR
{cshift,5}	Shift right arithmetic one bit position	cASHR
{cshift,6}	Shift right one bit position with carry	cSHRC
{cshift,1}	Shift left one bit position	cSHL
{cshift,2}	Shift left one bit position with carry	cSHLC
{cshift,7}	Rotate right one bit position	cROTR
{cshift,3}	Rotate left one bit position	cROTL
{gshift,0}	1-position global right shift	cGRSHIFT
{gshift,1}	1-position global left shift	cGLSHIFT
{gshift,5}	Reduction output insert right	cRREDINS
{gshift,4}	Reduction output insert left	cLREDINS
{gshift,2}	1-position global right rotate	cGRROTATE
{gshift,3}	1-position global left rotate	cGLROTATE
nop	No operation	cNOP
jmp	Relative jump to label s	cJMP(s)
ajmp	Absolute jump to label s	cAJMP(s)
brz	Jump to label s if $acc=0$	cBRZ(s)
brnz	Jump to label s if $acc!=0$	cBRNZ(s)
brzdec	Jump to label s if $acc=0$; $acc \leq acc-1$	cBRZDEC(s)
brnzdec	Jump to label s if $acc!=0$; $acc \leq acc-1$	cBRNZDEC(s)
halt	$pc \leq pc$	cHALT
cinsval	$acc \leq \{(acc \ll 8), value[7:0]\}$	cINSVAL
crela	$addr \leq acc$	cADDRLD
start	Start cycle counter	cSTART
stop	Stop cycle counter	cSTOP

20.2.2.2 Map's architecture

Each assembly-level instruction is made by concatenating the operation micro-code (indicated on the first column of the following tables) with the micro-code that selects the operation mode (indicated on the first line in the following table).

Table 20.5: Instructions with operands from the Instruction Set Architecture.

	imm	dir	rel	rei	cim	cdr	crl	cri
add	VADD(s)	ADD(s)	RADD(s)	RIADD(s)	CADD	CAADD	CRADD	CRIADD
addc	VADDC(s)	ADDC(s)	RADDC(s)	RIADDC(s)	CADDC	CAADDC	CRADDC	CRIADDC
sub	VSUB(s)	SUB(s)	RSUB(s)	RISUB(s)	CSUB	CASUB	CRSUB	CRISUB
rsub	VRSUB(s)	RSUB(s)	RRSUB(s)	RIRSUB(s)	CRSUB	CARSUB	CRRSUB	CIRRSUB
subc	VSUBC(s)	SUBC(s)	RSUBC(s)	RISUBC(s)	CSUBC	CASUBC	CRSUBC	CRISUBC
rsubc	VRSUBC(s)	RSUBC(s)	RRSUBC(s)	RIRSUBC(s)	CRSUBC	CARSUBC	CRRSUBC	CIRRSUBC
mult	VMULT(s)	MULT(s)	RMULT(s)	RIMULT(s)	CMULT	CAMULT	CRMULT	CRIMULT
bwand	VAND(s)	AND(s)	RAND(s)	RIAND(s)	CAND	CAAND	CRAND	CRIAND
bwor	VOR(s)	OR(s)	ROR(s)	RIOR(s)	COR	CAOR	CROR	CRIOR
bwxor	VXOR(s)	XOR(s)	RXOR(s)	RIXOR(s)	CXOR	CAXOR	CRXOR	CRIXOR
load	VLOAD(s)	LOAD(s)	RLOAD(s)	RILOAD(s)	CLOAD	CALOAD	CRLOAD	CRILOAD
store		STORE(s)	RSTORE(s)	RISTORE(s)	CSTORE	CASTORE	CRSTORE	CRISTORE
getio		GETIO(s)		RIGETIO(s)				
sendio		SENDIO(s)		RISENDIO(s)				
search	VSEARCH(s)				SEARCH			
csearch	VCSEARCH(s)				CSEARCH			
insert	VINSERT(s)				INSERT			

Table 20.6: Map's Instruction Set Architecture with no operand.

{cOpcode, value[2:0]}	Commentaries	Assembly
{shift,4}	Shift right one bit position	SHR
{shift,5}	Shift right arithmetic one bit position	ASHR
{shift,6}	Shift right one bit position with carry	SHRC
{shift,1}	Shift left one bit position	SHL
{shift,2}	Shift left one bit position with carry	SHLC
{shift,7}	Rotate right one bit position	ROTR
{shift,3}	Rotate left one bit position	ROTL
ixload	acc[i] <= i	IXLOAD
getsr	acc[i] <= serialReg[i]	GETSR
sendsr	serialReg[i] <= acc[i]	SENDSR
{where,0}	b[i] <= (b[i] & acc[i]=0) ? 1 : 0	WHEREZERO
{where,1}	b[i] <= (b[i] & carry[i]) ? 1 : 0	WHERECARRY
{where,2}	b[i] <= (b[i] & acc[i][31]=1) ? 1 : 0	WERENEGATIVE
{where,3}	b[i] <= i=0 ? 0 : (b[i-1] ? 1 : 0)	WEREPREVACT
{where,4}	b[i] <= (b[i] & first ? 1 : 0)	WEREFIRST
{where,5}	b[i] <= (b[i] & next ? 1 : 0)	WERENEXT
{where,6}	b[i] <= (b[i] & !(first next)) ? 1 : 0	WEREPREV
{where,8}	b[i] <= (b[i] & !(acc[i]=0)) ? 1 : 0	WERENZERO
{where,9}	b[i] <= (b[i] & !carry[i]) ? 1 : 0	WERENCARRY
{where,10}	b[i] <= (b[i] & !(acc[i][31]=1)) ? 1 : 0	WERENNEGATIVE
{where,11}	b[i] <= i=0 ? 1 : (b[i-1] ? 0 : 1)	WERENPREVACT
{where,12}	b[i] <= (b[i] & !first) ? 1 : 0	WERENFIRST
{where,13}	b[i] <= (b[i] & !next) ? 1 : 0	WERENNEXT
{where,14}	b[i] <= (b[i] & (first next)) ? 1 : 0	WERENPREV
elsew	Reapply the last where with the negated condition	ELSEWHERE
back	Redo b[i] affected by the most recently active where	ENDWHERE
selsh	b[i] <= i=0 ? 0 : b[i-1]	SELSHIFT
allact	b[i] <= 1	ACTIVATE
{setred,0}	Set the reduction function on addition	REDADD
{setred,3}	Set the reduction function on minimum	REDMIN
{setred,2}	Set the reduction function on maximum	REDMAX
{setred,1}	Set the reduction function on logical OR	REDOR
delete		DELETE
cinsval	acc[i] <= {(acc[i] << 8), value[7:0]}	INSVAL
crela	addr[i] <= acc[i]	ADDRLD

20.3 Programming heterogeneous system

The heterogeneous system runs two programs simultaneously. They are assembled independently and are launched together. The program on the host is the one that loads the program into the accelerator. In listing 20.6:

- line 1 loads in rf[0] the initial address in the host's data memory from where the program must be sent to the accelerator; it is 0
- line 2 sends the program stored in data memory at the address from rf[0] to the address 0 in

the program memory of the accelerator

```

1          hVALUE(0,0);
2          hPSEND(0,0);
3
4          // here comes the host program
5
6          hHALT;
```

Listing 20.6: The framework in which the host program must be included. File: 0_hProgram.sv (SystemVerilog)

The accelerator’s program consists of pairs of instructions (see Listing 20.7). One instruction executed by the controller of types cXXX, and another issued by the controller for Map array, of form XXX. The accelerator’s program is self-delimited, i.e., it starts with cPload(0) – loads the program in controller’s program memory starting from the address 0 – and ends with cPrun(0) – execute the program loaded in the controller’s program memory starting from the address 0.

```

1          cPLOAD(0);      ACTIVATE;
2          cNOP;           GETIO(1);
3          cNOP;           IXLOAD;
4
5          // here comes the accelerator program
6
7          LB(32); cHALT;    NOP;
8          cPRUN(0);      NOP;
```

Listing 20.7: The framework in which the accelerator program must be included. File: 0_aProgram.sv (SystemVerilog)

Programming hRISC is done in two ways, a simple direct one – inserting directly in 0_aProgram.sv a executable accelerator code – or using a program written using hISA and FLA 0.1 inserted in 0_hProgram.sv.

20.3.1 Direct programming using accelerator’s assembly

Example 20.1 *Let’s consider a 16-cell system. The reduction function add is used to add in the controller’s accumulator the indexes.*

```

1          cPLOAD(0);      ACTIVATE;
2          cNOP;           GETIO(1);
3          cNOP;           IXLOAD;
4          cNOP;           REDADD;
5          cNOP;           NOP;
6          cNOP;           NOP;
7          cNOP;           NOP;
8          cNOP;           NOP;
```



```

9          cNOP;          NOP;
10         cNOP;          NOP;
11         cNOP;          NOP;
12         cNOP;          NOP;
13         cCLOAD;        NOP;
14         LB(32); cHALT;   NOP;
15         cPRUN(0);      NOP;

```

Listing 20.8: Program which adds in acc the indexes from a 16-cell Map. File: 0_aProgram1.sv (SystemVerilog)

The result of simulation is:

```

t=43 pc= 0  cAcc=x  ACC=[x, x, x, x, x, x, x, x, x, x, x, x, x, x, x, x]
t=45 pc= 1  cAcc=x  ACC=[x, x, x, x, x, x, x, x, x, x, x, x, x, x, x, x]
t=47 pc= 2  cAcc=x  ACC=[x, x, x, x, x, x, x, x, x, x, x, x, x, x, x, x]
t=49 pc= 3  cAcc=x  ACC=[x, x, x, x, x, x, x, x, x, x, x, x, x, x, x, x]
t=51 pc= 4  cAcc=x  ACC=[x, x, x, x, x, x, x, x, x, x, x, x, x, x, x, x]
t=53 pc= 5  cAcc=x  ACC=[x, x, x, x, x, x, x, x, x, x, x, x, x, x, x, x]
t=55 pc= 6  cAcc=x  ACC=[x, x, x, x, x, x, x, x, x, x, x, x, x, x, x, x]
t=57 pc= 7  cAcc=x  ACC=[x, x, x, x, x, x, x, x, x, x, x, x, x, x, x, x]
t=59 pc= 8  cAcc=x  ACC=[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15]
t=61 pc= 9  cAcc=x  ACC=[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15]
t=63 pc= 10 cAcc=x  ACC=[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15]
t=65 pc= 11 cAcc=x  ACC=[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15]
t=67 pc= 12 cAcc=x  ACC=[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15]
t=69 pc= 13 cAcc=x  ACC=[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15]
t=71 pc= 13 cAcc=120 ACC=[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15]

```

The 8 lines of code cNOP; NOP; are due to the latencies introduced by the distribution net from controller to Map and to the reduction net from Map to controller.

If we need to write a program to be executed in Map arrays with p cells, then the code is:

```

1          cPLOAD(0);      ACTIVATE;
2          cNOP;           GETIO(1);
3          cNOP;           IXLOAD;
4          // begin add indexes
5          cVLOAD($clog2(`p)-1); REDADD;
6          LB(1); cNOP;     NOP;
7          cBRNZDEC(1);    NOP;
8          cCLOAD;        NOP;
9          // end add indexes
10         LB(32); cHALT;   NOP;
11         cPRUN(0);      NOP;

```

Listing 20.9: Program which adds in acc the indexes from a p -cell Map. File: 0_aProgram2.sv (SystemVerilog)

The result of simulation for $p = 16$ is:

```

t=31 pc= 0  cAcc=x  ACC=[x, x, x, x, x, x, x, x, x, x, x, x, x, x, x, x]

```

```

t=33 pc= 1 cAcc=x ACC=[x, x, x, x, x, x, x, x, x, x, x, x, x, x, x, x]
t=35 pc= 2 cAcc=x ACC=[x, x, x, x, x, x, x, x, x, x, x, x, x, x, x, x]
t=37 pc= 3 cAcc=x ACC=[x, x, x, x, x, x, x, x, x, x, x, x, x, x, x, x]
t=39 pc= 4 cAcc=x ACC=[x, x, x, x, x, x, x, x, x, x, x, x, x, x, x, x]
t=41 pc= 5 cAcc=3 ACC=[x, x, x, x, x, x, x, x, x, x, x, x, x, x, x, x]
t=43 pc= 4 cAcc=3 ACC=[x, x, x, x, x, x, x, x, x, x, x, x, x, x, x, x]
t=45 pc= 5 cAcc=2 ACC=[x, x, x, x, x, x, x, x, x, x, x, x, x, x, x, x]
t=47 pc= 4 cAcc=2 ACC=[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15]
t=49 pc= 5 cAcc=1 ACC=[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15]
t=51 pc= 4 cAcc=1 ACC=[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15]
t=53 pc= 5 cAcc=0 ACC=[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15]
t=55 pc= 6 cAcc=0 ACC=[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15]
t=57 pc= 7 cAcc=-1 ACC=[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15]
t=59 pc= 7 cAcc=120 ACC=[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15]

```

The execution time is the same but the code is shorter and, more important, is general, for any p .

The execution time is $t_{addIndexes} = 2 + 2 \times \log_2 p \in O(\log p)$.

The computation is intense, because the size of the code is constant – four lines – while the execution time increases with p .

We can evaluate the speedup, compared to the execution time of a single-core computing unit, by writing a program for the controller that sums p numbers (see Listing 20.10).

```

1      cPLOAD(0);      ACTIVATE;
2      cNOP;           GETIO(1);
3      cNOP;           IXLOAD;
4      // initialization of values
5      cVLOAD(0);      NOP; // initial value
6      cSTORE(1);      NOP; // cMem[1] <= initial value
7      cVLOAD(`p);     NOP; // number of scalars
8      cSTORE(0);      NOP; // cMem[0] <= number of scalars
9      cVLOAD(1);      NOP; // initial pointer
10     cADDRLD;        NOP; // cAddr <= initial pointer
11     // main loop
12     // addition
13     LB(11); cLOAD(1); NOP;
14     cRIADD(1);      NOP;
15     cSTORE(1);      NOP;
16     // control
17     cLOAD(0);       NOP;
18     cVSUB(1);       NOP;
19     cSTORE(0);      NOP;
20     cBRNZDEC(11);   NOP;

```

Listing 20.10: Program executed by controller which adds in acc p numbers. File: 0_aProgram3.sv (SystemVerilog)

The execution time for adding with the controller p numbers is: $t_{adding_p_numbers} = 7 \times p + 6$. Therefore, the acceleration provided by using the heterogeneous approach is

$$\alpha = \frac{7 \times p + 6}{2 \times (1 + \log_2 p)} > \frac{p}{\log_2 p}$$

The bad news: latency introduced by the reduce network which is responsible for the log at the denominator. Good news: the inequality is an advantage provided by the fact that in the heterogeneous approach the addition is done by Map, and the control is done by controller controller; while in the mono-core approach both the operation and the control are executed by the controller.

◇

Example 20.2 *The program which loads in the even cells the index multiplied by 2 and in the odd cells the index multiplied by 3 is listed in Listing 20.11.*

```

1          cPLOAD(0);      ACTIVATE;
2          cNOP;           GETIO(1);
3          cNOP;           IXLOAD;
4
5          cNOP;           VAND(1);
6          cNOP;           WHEREZERO;
7          cNOP;           IXLOAD;
8          cNOP;           VMULT(2);
9          cNOP;           ELSEWHERE;
10         cNOP;           IXLOAD;
11         cNOP;           VMULT(3);
12         cNOP;           ENDWHERE;
13
14         LB(32); cHALT;    NOP;
15         cPRUN(0);      NOP;

```

Listing 20.11: Program illustrates the use of the spatial selection mechanism. File: 0_aProgram4.sv (SystemVerilog)

The result of the execution is:

t=39 pc= 0	ACC=[x, x, x, x, x, x, x, x, x, x, x, x, x, x, x, x]	B = [xxxxxxxxxxxxxxxx]
t=41 pc= 1	ACC=[x, x, x, x, x, x, x, x, x, x, x, x, x, x, x, x]	B = [xxxxxxxxxxxxxxxx]
t=43 pc= 2	ACC=[x, x, x, x, x, x, x, x, x, x, x, x, x, x, x, x]	B = [xxxxxxxxxxxxxxxx]
t=45 pc= 3	ACC=[x, x, x, x, x, x, x, x, x, x, x, x, x, x, x, x]	B = [xxxxxxxxxxxxxxxx]
t=47 pc= 4	ACC=[x, x, x, x, x, x, x, x, x, x, x, x, x, x, x, x]	B = [xxxxxxxxxxxxxxxx]
t=49 pc= 5	ACC=[x, x, x, x, x, x, x, x, x, x, x, x, x, x, x, x]	B = [xxxxxxxxxxxxxxxx]
t=51 pc= 6	ACC=[x, x, x, x, x, x, x, x, x, x, x, x, x, x, x, x]	B = [xxxxxxxxxxxxxxxx]
t=53 pc= 7	ACC=[x, x, x, x, x, x, x, x, x, x, x, x, x, x, x, x]	B = [1111111111111111]
t=55 pc= 8	ACC=[x, x, x, x, x, x, x, x, x, x, x, x, x, x, x, x]	B = [1111111111111111]
t=57 pc= 9	ACC=[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15]	B = [1111111111111111]
t=59 pc= 10	ACC=[0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1]	B = [1111111111111111]
t=61 pc= 11	ACC=[0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1]	B = [1010101010101010]
t=63 pc= 11	ACC=[0, 1, 2, 1, 4, 1, 6, 1, 8, 1, 10, 1, 12, 1, 14, 1]	B = [1010101010101010]
t=65 pc=1023	ACC=[0, 1, 4, 1, 8, 1, 12, 1, 16, 1, 20, 1, 24, 1, 28, 1]	B = [1010101010101010]
t=67 pc=1023	ACC=[0, 1, 4, 1, 8, 1, 12, 1, 16, 1, 20, 1, 24, 1, 28, 1]	B = [0101010101010101]
t=69 pc=1023	ACC=[0, 1, 4, 3, 8, 5, 12, 7, 16, 9, 20, 11, 24, 13, 28, 15]	B = [0101010101010101]
t=71 pc=1023	ACC=[0, 3, 4, 9, 8, 15, 12, 21, 16, 27, 20, 33, 24, 39, 28, 45]	B = [0101010101010101]
t=73 pc=1023	ACC=[0, 3, 4, 9, 8, 15, 12, 21, 16, 27, 20, 33, 24, 39, 28, 45]	B = [1111111111111111]

◇

Example 20.3 *The program generates a pseudo-random sequence in ADD and selects in acc the biggest value.*

```

1          cPLOAD(0);          ACTIVATE;
2          cNOP;               GETIO(1);
3          cNOP;               IXLOAD;
4
5          cNOP;               VADD(5);
6          cNOP;               VMULT(9875);
7          cNOP;               SHR;
8          cNOP;               SHR;
9          cNOP;               SHR;
10         cNOP;               REDMAX;
11         cVLOAD($clog2(`p)-1); VAND(31);
12     LB(33); cNOP;            NOP;
13         cBRNZDEC(33);        NOP;
14
15         cCLOAD;              NOP;
16         cNOP;                NOP;

```

Listing 20.12: Program generates in acc the biggest value generated pseudo-randomly in ACC. File: 0_aProgram5.sv (SystemVerilog)

The program execution provides:

ACC=[27, 14, 0, 19, 5, 23, 10, 28, 14, 1, 19, 6, 24, 10, 29, 15]

and cAcc = 29.

◇

20.3.2 FLA 0.1

```

1  /*****
2  File name: 00_theKernel.v
3  Description: LINEAR ALGEBRA KERNEL LIBRARY FUNCTIONS
4  *****/
5          cHALT;      NOP;
6          cJMP(1);    NOP; // hSTART
7          cJMP(2);    NOP; // hSTOP
8          cJMP(3);    NOP; // hINTRQ
9          cJMP(4);    NOP; // hSQGENX
10         cJMP(5);    NOP; // hSQGENN
11         cJMP(6);    NOP; // hMSEND(addr-1, size)
12         cJMP(7);    NOP; // hMGET(addr-1, size)
13         cJMP(8);    NOP; // hUNIT(addr-1)
14         cJMP(9);    NOP; // hSQADD(dest, left, right)
15         cJMP(10);   NOP; // hVGENX(addr)
16         cJMP(11);   NOP; // hVGENN(addr, value)
17         cJMP(12);   NOP; // hSQMMULT(dest, left, right)
18         cJMP(13);   NOP; // hSQMMULT(dest, left, right)
19         cJMP(14);   NOP; // hSQMMAC(dest, left, right)
20 //***** START COUNTER *****/
21     LB(1); cSTART;      VLOAD(22);

```

```

22         cJMP(32);      NOP;
23 //***** STOP COUNTER *****
24         LB(2);  cSTOP;      VLOAD(33);
25         cJMP(32);      NOP;
26 //***** INTERRUPT REQUEST *****
27         LB(3);  cSETINT;     VLOAD(44);
28         cJMP(32);      NOP;
29 //***** SQUARE MATRIX X GENERATE *****
30         LB(4);  cVLOAD(`p);   VLOAD(-1);
31         cNOP;      ADDRDL;
32         cNOP;      IXLOAD;
33         LB(17); cNOP;      RISTORE(1);
34         cBRNZDEC(17); VADD(1);
35         cJMP(32);      NOP;
36 //***** SQUARE MATRIX N GENERATE *****
37         LB(5);  cVLOAD(`p-1); VLOAD(-1);
38         cNOP;      ADDRDL;
39         cNOP;      VLOAD(0);
40         LB(18); cNOP;      RISTORE(1);
41         cBRNZDEC(18); VADD(1);
42         cJMP(32);      NOP;
43 //***** SEND MATRIX *****
44         LB(6);  cPARAM;      NOP;
45         cPARAM;      CLOAD;
46         cNOP;      ADDRDL;
47         cNOP;      NOP;
48         NOP;      RISENDIO(1);
49         LB(19); cNOP;      NOP;
50         cBRZDEC(32); NOP;
51         cNOP;      NOP;
52         cNOP;      NOP;
53         cNOP;      NOP;
54         cDATAEXT(`p/2); NOP;
55         cJMP(19);      RISENDIO(1);
56 //***** GET MATRIX *****
57         LB(7);  cPARAM;      GETIO(0);
58         cPARAM;      CLOAD;
59         cNOP;      NOP;
60         cNOP;      NOP;
61         cNOP;      NOP;
62         cNOP;      ADDRDL;
63         LB(20); cDATAINS(`p/2); NOP;
64         cNOP;      RIGETIO(1);
65         cBRZDEC(32); NOP;
66         cNOP;      NOP;
67         cNOP;      NOP;
68         cNOP;      NOP;
69         cJMP(20);      NOP;
70 //***** UNIT MATRIX GENERATE *****
71         LB(8);  cPARAM;      NOP;
72         cNOP;      CLOAD;

```

```

73      cSTORE(1);      ADDRDL;
74      cVLOAD(`p);     VLOAD(0);
75      LB(21); cNOP;     RISTORE(1);
76      cBRNZDEC(21);   NOP;
77      cLOAD(1);       NOP;
78      cVADD(1);       CLOAD;
79      cNOP;           ADDRDL;
80      cNOP;           IXLOAD;
81      cNOP;           WHEREZERO;
82      cNOP;           VLOAD(1); // N on diagonal
83      cNOP;           ELSEWHERE;
84      cNOP;           VLOAD(0);
85      cNOP;           SENDSR;
86      cNOP;           ENDWHERE;
87      cNOP;           VLOAD(23);
88      LB(22); cNOP;     GETSR;
89      cGRSHIFT;       RISTORE(1);
90      cBRNZDEC(22);   NOP;
91      cJMP(32);       NOP;
92      //***** ADD SQUARE MATRICES *****
93      LB(9); cPARAM;    NOP;
94      cSTORE(3);      NOP; // dest at mem[3]
95      cPARAM;         NOP;
96      cSTORE(4);      NOP; // left at mem[4]
97      cPARAM;         NOP;
98      cSTORE(5);      NOP; // right at mem[5]
99      cSUB(4);        NOP;
100     cSTORE(0);      NOP; // right-left at mem[0]
101     cLOAD(3);       NOP;
102     cSUB(5);        NOP;
103     cSTORE(1);      NOP; // dest-right at mem[1]
104     cLOAD(4);       NOP;
105     cSUB(3);        NOP;
106     cVADD(1);       NOP;
107     cSTORE(2);      NOP; // left-dest+1 at mem[2]
108     cVLOAD(`p);     NOP;
109     cSTORE(6);      NOP;
110     LB(23); cLOAD(4); NOP;
111     cADD(0);        CALOAD;
112     cADD(1);        CAADD;
113     cADD(2);        CSTORE;
114     cSTORE(4);      NOP;
115     cLOAD(6);       NOP;
116     cVSUB(1);       NOP;
117     cSTORE(6);      NOP;
118     cBRNZ(23);      NOP;
119     cJMP(32);       NOP;
120     //***** INDEX VECTOR GENERATE *****
121     LB(10); cPARAM;    IXLOAD;
122     cJMP(32);       CSTORE;
123     //***** N VECTOR GENERATE *****

```

```

124     LB(11); cPARAM;          NOP;
125         cPARAM;          CLOAD;
126         cJMP(32);         CSTORE;
127 //***** MATRIX-VECTOR MULTIPLY *****
128     LB(12); cPARAM;          NOP;
129         cPARAM;          CLOAD;
130         cSTORE(0);        VADD(1);    // mem[0] = right addr
131         cPARAM;          ADDRDL;    // matrix end addr + 1
132         cSTORE(1);        REDADD;    // mem[1] = dest addr
133         cVLOAD(`p);       RILOAD(-1);
134     LB(24); cLREDINS;        MULT(`p);
135         cBRNZDEC(24);      RILOAD(-1);
136         cVLOAD($clog2(`p)-2); NOP;
137     LB(27); cBRNZDEC(27);    NOP;
138         cLOAD(1);         GETSR;
139         cJMP(32);         CSTORE;
140 //***** MULTIPLY SQUARE MATRICES *****
141     LB(13); cPARAM;          REDADD;
142         cVSUB(1);         NOP;
143         cSTORE(3);        NOP;    // dest => 3
144         cPARAM;          NOP;
145         cSTORE(0);        NOP;    // left => 0
146         cPARAM;          CLOAD;
147         cSTORE(2);        ADDRDL;    // right => 2
148         cVLOAD(`p);       NOP;
149         cSTORE(1);        NOP;
150         cLOAD(2);         NOP;
151         cVADD(1);         CALOAD;
152         cSTORE(2);        STORE(3*`p);
153         cVLOAD(`p);       RILOAD(0);
154     LB(25); cLREDINS;        MULT(3*`p);
155         cBRNZDEC(25);      RILOAD(-1);
156         cVLOAD($clog2(`p)-3); NOP;
157     LB(28); cBRNZDEC(28);    NOP;
158         cLOAD(3);         NOP;
159         cVADD(1);         NOP;
160         cSTORE(3);        GETSR;
161         cLOAD(2);         CSTORE;
162         cVADD(1);         CALOAD;
163         cSTORE(2);        NOP;
164         cLOAD(0);         STORE(3*`p);
165         cLOAD(1);         CLOAD;
166         cVSUB(1);         NOP;
167         cBRZ(32);         ADDRDL;
168         cSTORE(1);        NOP;
169         cVLOAD(`p);       RILOAD(0);
170         cJMP(25);         NOP;
171 //***** MULTIPLY & ACCUMULATE SQUARE MATRICES *****
172     LB(14); cPARAM;          REDADD;
173         cVSUB(1);         NOP;
174         cSTORE(3);        NOP;    // dest => 3

```

```

175          cPARAM;          NOP;
176          cSTORE(0);        NOP;    // left => 0
177          cPARAM;          CLOAD;
178          cSTORE(2);        ADDRLD; // right => 2
179          cVLOAD(`p);       NOP;
180          cSTORE(1);        NOP;
181          cLOAD(2);         NOP;
182          cVADD(1);         CALOAD;
183          cSTORE(2);        STORE(3*`p);
184          cVLOAD(`p);       RILOAD(0);
185      LB(26); cLREDINS;      MULT(3*`p);
186          cBRNZDEC(26);      RILOAD(-1);
187          cVLOAD($clog2(`p)-3); NOP;
188      LB(29); cBRNZDEC(29);  NOP;
189          cLOAD(3);          NOP;
190          cVADD(1);          NOP;
191          cSTORE(3);         GETSR;
192          NOP;               CAADD;
193          cLOAD(2);          CSTORE;
194          cVADD(1);          CALOAD;
195          cSTORE(2);         NOP;
196          cLOAD(0);          STORE(3*`p);
197          cLOAD(1);          CLOAD;
198          cVSUB(1);          NOP;
199          cBRZ(32);          ADDRLD;
200          cSTORE(1);         NOP;
201          cVLOAD(`p);        RILOAD(0);
202          cJMP(26);          NOP;

```

Listing 20.13: FLA 0.1. File: 00_theKernel.sv (SystemVerilog)

20.3.3 Programming using FLA 0.1

Example 20.4 *The program multiplying a matrix with a vector written on host calls the components from the library FLA 0.1 which is included in the program running on the accelerator under the name: 00_theKernel.sv. The matrix whose last line is in the vector $V[p-1]$ is multiplied with the vector $V[p]$ and the result is stored as $V[p+1]$. The multiplied matrix is generated using the function `hSQGENX` which generate the matrix starting with $V[0] = IX$ and continues with vectors with the components incremented with 1. $V[p] = [3 \ 3 \ \dots \ 3]$ is the multiplying vector.*

```

1      hVALUE(0,0);
2      hPSEND(0,0);
3
4      hSQGENX;
5      hVGENN(3,`p);
6      hSTART;
7      hSQMVMULT(`p-1, `p, `p+1);
8      hSTOP;
9

```



```
10          hHALT;
```

Listing 20.14: The program on host for matrix-vector multiply. File: 0_hProgram3.sv (SystemVerilog)

```
1          cPLOAD(0);      ACTIVATE;
2          cNOP;           GETIO(1);
3          cNOP;           IXLOAD;
4
5          `include "00_theKernel.sv"
6
7          LB(32); cHALT;      NOP;
8          cPRUN(0);         NOP;
```

Listing 20.15: The program on accelerator which includes the kernel library FLA 0.1 used by host. File: 0_aProgram6.sv (SystemVerilog)

Running the program for $p = 16$ results:

vect[0]	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
vect[1]	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
vect[2]	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
vect[3]	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
vect[4]	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
vect[5]	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
vect[6]	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21
vect[7]	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22
vect[8]	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23
vect[9]	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24
vect[10]	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25
vect[11]	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26
vect[12]	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27
vect[13]	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28
vect[14]	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29
vect[15]	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30
vect[16]	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3
vect[17]	360	408	456	504	552	600	648	696	744	792	840	888	936	984	1032	1080

The execution time resulting from the code listed in Listing 20.13 is:

$$t_{mvMult}(p) = 2p + \log_2 p + 8 \in O(p)$$

Table 20.7 summarizes the time performance for matrix-vector multiplication. The constant associated with Big O notation used to indicate execution time tends towards 2 as this table suggests.

Table 20.7: Matrix-vector multiplication evaluation depending on the size of the square matrix.

# of Lines	# of Cycles	# Cycles/Dot Product
16	44	2.75
32	87	2.43
64	152	2.23
128	281	2.11
256	538	2.06
512	1051	2.03

◇

Example 20.5 The program multiplying two matrices written on host calls the components from the library FLA 0.1 which is included in the program running on the accelerator under the name: 00_theKernel.sv. The first two lines in the host program generates the two matrices to be multiplied. The call of multiplication is preceded by the function which start the cycles counter on controller and is followed by the start function.

```

1      hVALUE(0,0);
2      hPSEND(0,0);
3      hSQGENX;                               // generate matrix X
4      hUNIT(`p-1);                           // generate matrix UNIT
5      hSTART;
6      hSQMMULT(2*`p,(2*`p)-1,0); // multiply matrices
7      hSTOP;
8      hHALT;

```

Listing 20.16: The program on host for multiplying matrices. File: 0_hProgram1.sv (SystemVerilog)

The simulation provided, for $p = 16$, the following:

```

vect[0]  0  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15
vect[1]  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16
vect[2]  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17
vect[3]  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18
vect[4]  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19
vect[5]  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20
vect[6]  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20 21
vect[7]  7  8  9 10 11 12 13 14 15 16 17 18 19 20 21 22
vect[8]  8  9 10 11 12 13 14 15 16 17 18 19 20 21 22 23
vect[9]  9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24
vect[10] 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25
vect[11] 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26
vect[12] 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27
vect[13] 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28
vect[14] 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29
vect[15] 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30
vect[16] 1  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0

```

```

vect[17]  0  1  0  0  0  0  0  0  0  0  0  0  0  0  0  0
vect[18]  0  0  1  0  0  0  0  0  0  0  0  0  0  0  0  0
vect[19]  0  0  0  1  0  0  0  0  0  0  0  0  0  0  0  0
vect[20]  0  0  0  0  1  0  0  0  0  0  0  0  0  0  0  0
vect[21]  0  0  0  0  0  1  0  0  0  0  0  0  0  0  0  0
vect[22]  0  0  0  0  0  0  1  0  0  0  0  0  0  0  0  0
vect[23]  0  0  0  0  0  0  0  1  0  0  0  0  0  0  0  0
vect[24]  0  0  0  0  0  0  0  0  1  0  0  0  0  0  0  0
vect[25]  0  0  0  0  0  0  0  0  0  1  0  0  0  0  0  0
vect[26]  0  0  0  0  0  0  0  0  0  0  1  0  0  0  0  0
vect[27]  0  0  0  0  0  0  0  0  0  0  0  1  0  0  0  0
vect[28]  0  0  0  0  0  0  0  0  0  0  0  0  1  0  0  0
vect[29]  0  0  0  0  0  0  0  0  0  0  0  0  0  1  0  0
vect[30]  0  0  0  0  0  0  0  0  0  0  0  0  0  0  1  0
vect[31]  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  1
vect[32]  0  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15
vect[33]  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16
vect[34]  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17
vect[35]  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18
vect[36]  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19
vect[37]  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20
vect[38]  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20 21
vect[39]  7  8  9 10 11 12 13 14 15 16 17 18 19 20 21 22
vect[40]  8  9 10 11 12 13 14 15 16 17 18 19 20 21 22 23
vect[41]  9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24
vect[42] 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25
vect[43] 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26
vect[44] 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27
vect[45] 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28
vect[46] 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29
vect[47] 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30

```

The execution time resulting from the code listed in Listing 20.13 is:

$$t_{mmMult}(p) = 2p^2 + p \times \log_2 p + 12p + 13 \in O(p^2)$$

The execution time is evaluated by the simulator at $aCC = 781$ clock cycles for $p = 16$. In Table 20.8 are displayed the number of clock cycles per dot product for $p \times p$ matrices. The constant associated with Big O notation used to indicate execution time tends towards 2 as this table suggests.

Table 20.8: Multiplication evaluation depending on the size of the square matrices.

# of Lines	# of Cycles	# Cycles/Dot Product
16	781	3.05
32	2605	2.54
64	9457	2.28
128	35213	2.15
256	136205	2.07
512	535053	2.04

As the matrix size increases, the number of clock cycles per dot product converges to 2, the size of the loop that calculates each element of the resulting matrix.

◇

Example 20.6 *The program for multiply-add matrices written on host calls the components from the library FLA 0.1 which is included in the program running on the accelerator under the name: 00_theKernel.sv. The lines 3 and 4 in the host program generates the two matrices to be multiplied. The call of multiplication is followed by the multiply and add matrices which perform the same multiplication but adds the result over the previously result.*

```

1      hVALUE(0,0);
2      hPSEND(0,0);
3      hSQGENX;                               // generate matrix X
4      hUNIT(`p-1);                           // generate matrix UNIT
5      hSTART;
6      hSQMMULT(2*`p,(2*`p)-1,0);             // multiply matrices
7      hSQMMAC(2*`p,2*`p-1,0);               // mult & acc matrices
8      hSTOP;
9      hHALT;
```

Listing 20.17: The program on host for multiply-add matrices. File: 0_hProgram2.sv (SystemVerilog)

The simulation provided, for $p = 16$, the following values for the resulting matrix:

vect[32]	0	2	4	6	8	10	12	14	16	18	20	22	24	26	28	30
vect[33]	2	4	6	8	10	12	14	16	18	20	22	24	26	28	30	32
vect[34]	4	6	8	10	12	14	16	18	20	22	24	26	28	30	32	34
vect[35]	6	8	10	12	14	16	18	20	22	24	26	28	30	32	34	36
vect[36]	8	10	12	14	16	18	20	22	24	26	28	30	32	34	36	38
vect[37]	10	12	14	16	18	20	22	24	26	28	30	32	34	36	38	40
vect[38]	12	14	16	18	20	22	24	26	28	30	32	34	36	38	40	42
vect[39]	14	16	18	20	22	24	26	28	30	32	34	36	38	40	42	44
vect[40]	16	18	20	22	24	26	28	30	32	34	36	38	40	42	44	46
vect[41]	18	20	22	24	26	28	30	32	34	36	38	40	42	44	46	48
vect[42]	20	22	24	26	28	30	32	34	36	38	40	42	44	46	48	50
vect[43]	22	24	26	28	30	32	34	36	38	40	42	44	46	48	50	52
vect[44]	24	26	28	30	32	34	36	38	40	42	44	46	48	50	52	54
vect[45]	26	28	30	32	34	36	38	40	42	44	46	48	50	52	54	56
vect[46]	28	30	32	34	36	38	40	42	44	46	48	50	52	54	56	58
vect[47]	30	32	34	36	38	40	42	44	46	48	50	52	54	56	58	60

The execution time is evaluated by the simulator at aCC = 1570 clock cycles for $p = 16$.

◇

APPENDIXES

Appendix A

Hardware Description Languages

A.1 Chisel

A.1.1 Introduction

Chisel (Constructing Hardware In a Scala Embedded Language) is a hardware design language embedded within Scala. It allows hardware designers to write complex, parameterizable, and reusable hardware components using high-level programming constructs (Bachrach et al., 2012).

This Appendix is a very brief introduction to Chisel primarily for designers to reference features quickly. For processor design, Schoeberl (2019) is a highly recommended free book. The Chisel Cheat Sheet is also helpful (Developers, 2023).

A.1.2 Key Features of Chisel

Chisel is a domain-specific language (DSL) for hardware design, using Scala’s object-oriented and functional programming features (Chisel Contributors, 2023a). It compiles to synthesizable Verilog, integrating with traditional hardware development flows.

Hardware Construction Language. High-level abstractions for hardware components.

Parameterization. Easy creation of generic and reusable modules.

Strong Typing. Reduces errors through rigorous type checking.

Functional Programming. Utilizes Scala’s functional paradigms for concise code.

A.1.3 Syntax

A.1.4 Scala Basics

Understanding basic Scala syntax is needed for Chisel programming (Odersky et al., 2008).

Immutable Variables (`val`). Cannot be reassigned.

Mutable Variables (var). Can be reassigned.

Functions (def). Defined using the `def` keyword.

Classes and Objects. Used to define hardware modules and singletons.

A.1.5 Import Statements

Include necessary Chisel libraries at the beginning of your file:

```
1 import chisel3._
2 import chisel3.util._
```

A.1.6 Data Types

Chisel provides specialized data types for hardware representation (Chisel Contributors, 2023a).

A.1.7 Base Types

Bool. Single-bit boolean (`true/false`). Declaration: `val myBool = Bool()`

UInt. Unsigned integer with specified width. Declaration: `val myUInt = UInt(8.W)`

SInt. Signed integer with specified width. Declaration: `val mySInt = SInt(16.W)`

FixedPoint. Fixed-point number with total and fractional bits. Declaration: `val myFixed = FixedPoint(16.W, 8.BP)`

Clock. Represents a clock signal.

Reset. Represents a reset signal.

A.1.8 Aggregate Types

Bundle. Custom data structures combining multiple fields. Declaration: `class MyBundle extends Bundle { val a = UInt(8.W) }`

Vec. Indexable collection of elements of the same type. Declaration: `val myVec = Vec(4, UInt(8.W))`

A.1.9 Literals

Unsigned Integer. `10.U, 3.U(8.W)`

Signed Integer. `-5.S, -2.S(8.W)`

Boolean. `true.B, false.B`

A.1.10 Operators

Arithmetic (+, -, *, /, %). + (addition), - (subtraction), * (multiplication), / (division), % (modulo operation)

Logical (&, |, ^, !). & (logical AND), | (logical OR), ^ (logical XOR), ! (logical NOT)

Comparison (==, !=, <, <=, >, >=). == (equal to), != (not equal to), < (less than), <= (less than or equal to), > (greater than), >= (greater than or equal to)

Bitwise Reduction (&, |, ^). & (bitwise AND reduction), | (bitwise OR reduction), ^ (bitwise XOR reduction). These operators are applied to a single operand.

Shift (<<, >>). << (left shift), >> (right shift)

A.1.11 Modules and Hierarchy

A.1.12 Defining Modules

Modules are defined by extending the `Module` class (Chisel Contributors, 2023a).

```
1 class MyModule extends Module {
2   // Module implementation
3 }
```

A.1.13 Input/Output Ports

Ports are declared using the `IO` method with a `Bundle`.

```
1 class MyModule extends Module {
2   val io = IO(new Bundle {
3     val in  = Input(UInt(8.W))
4     val out = Output(UInt(8.W))
5   })
6   // Module logic
7 }
```

A.1.14 Instantiating Modules

Modules can be instantiated within other modules.

```
1 val subModule = Module(new SubModule)
2 subModule.io.in := io.in
```

```
3 io.out := subModule.io.out
```

A.1.15 Parameterized Modules

Modules can be parameterized for reusability.

```
1 class MyModule(val width: Int) extends Module {  
2   val io = IO(new Bundle {  
3     val in  = Input(UInt(width.W))  
4     val out = Output(UInt(width.W))  
5   })  
6   // Module logic  
7 }
```

A.1.16 Wires and Registers

Wires represent combinational logic signals (Chisel Contributors, 2023a).

```
1 val tempWire = Wire(UInt(8.W))  
2 tempWire := io.in + 1.U
```

Registers store state across clock cycles.

- Reg: Creates a register without an initial value.
Declaration: `val myReg = Reg(UInt(8.W))`
- RegInit: Creates a register with an initial value.
Declaration: `val myReg = RegInit(0.U(8.W))`

Registers are updated using non-blocking assignment.

```
1 myReg := myReg + 1.U
```

A.1.17 Control Structures

A.1.18 when Statements

Conditional hardware execution.

```
1 when (condition) {  
2   // Actions when condition is true  
3 } .elsewhen (anotherCondition) {  
4   // Actions when anotherCondition is true  
5 } .otherwise {  
6   // Actions when all conditions are false  
7 }
```

A.1.19 switch Statements

Multi-way conditional execution.

```
1  switch(selector) {  
2    is(value1) {  
3      // Actions for value1  
4    }  
5    is(value2) {  
6      // Actions for value2  
7    }  
8  }
```

A.1.20 Mux Function

Multiplexer for conditional selection.

```
1  val result = Mux(condition, trueValue, falseValue)
```

A.1.21 Loops

Used for hardware generation, not for creating sequential logic (Chisel Contributors, 2023b).

```
1  for (i <- 0 until n) {  
2    // Generate n instances of hardware  
3  }
```

A.1.22 Functions and Procedures

A.1.23 Defining Functions

Functions encapsulate reusable logic.

```
1  def add(a: UInt, b: UInt): UInt = {  
2    a + b  
3  }
```

A.1.24 Invoking Functions

```
1  val sum = add(io.in1, io.in2)
```

A.1.25 Parameterized Types

Types can be parameterized for flexibility.

```
1 def myFunction[T <: Data](in: T): T = {  
2   // Function body  
3 }
```

The function `myFunction` is a generic method in Chisel that operates on a parameterized data type `T`. The parameter `T` is constrained to extend the `Data` type, which is the base class for all hardware types in Chisel, such as `UInt`, `SInt`, or `Bundle`. This ensures that `myFunction` can accept any valid Chisel hardware type as its input. The method takes a single input argument `in` of type `T` and returns a value of the same type `T`. While the function body is not specified, this structure is commonly used to define reusable, type-safe operations on hardware constructs, enabling modular and flexible hardware design.

A.1.26 Generic Modules

Modules can be generic over data types.

```
1 class MyGenericModule[T <: Data](genType: T) extends Module {  
2   val io = IO(new Bundle {  
3     val in  = Input(genType)  
4     val out = Output(genType)  
5   })  
6   // Module logic  
7 }
```

The `MyGenericModule` class is a parameterized hardware module in Chisel. It extends the `Module` base class and takes a type parameter `T`, constrained to be a subtype of `Data`, which ensures that it can represent any Chisel hardware type such as `UInt`, `SInt`, or custom `Bundles`. The parameter `genType` is passed to the constructor and defines the specific type of data the module will operate on. Inside the module, the `io` interface is declared using a `Bundle`, containing an input port `in` and an output port `out`, both of which use the generic type `genType`. This design allows the module to be highly reusable and adaptable, as it can handle different data types depending on the parameter provided during instantiation. The comment `// Module logic` indicates where the functional implementation of the module would be added.

A.1.27 Hardware Construction in Chisel

A.1.28 Vectors (Vec)

Indexable collections of elements.

```
1 val myVec = VecInit(Seq.fill(4)(0.U(8.W)))  
2 myVec(0) := io.in
```

A.1.29 Memories (Mem)

Memory elements for storage.

```
1 val myMem = Mem(1024, UInt(8.W))
2 val dataOut = myMem.read(addr)
3 myMem.write(addr, dataIn)
```

A.1.30 Bundles

Custom data structures grouping multiple fields.

```
1 class MyBundle extends Bundle {
2   val field1 = UInt(8.W)
3   val field2 = Bool()
4 }
```

A.1.31 Testing and Verification

A.1.32 ChiselTest Framework

ChiselTest provides tools for testing Chisel modules (Chisel Contributors, 2023c).

```
1 import chiseltest._
2 import org.scalatest.flatspec.AnyFlatSpec
3
4 class MyModuleTest extends AnyFlatSpec with ChiselScalatestTester {
5   "MyModule" should "perform correctly" in {
6     test(new MyModule) { c =>
7       c.io.in.poke(0.U)
8       c.clock.step(1)
9       c.io.out.expect(0.U)
10    }
11  }
12 }
```

A.1.33 Assertions

In-module checks using assert.

```
1 assert(io.out === expectedValue)
```

A.1.34 Peculiarities of Chisel

A.1.35 Chisel is Embedded in Scala

Chisel code is Scala code. This means:

- Full access to Scala’s features.
- Potential confusion between hardware constructs and software constructs.

A.1.36 Delayed Execution Model

Chisel builds a hardware graph during elaboration, which may lead to unexpected behavior if side effects are not managed properly (Chisel Contributors, 2023a).

A.1.37 Assignment Operators

- `:=` Used for updating hardware signals.
- `=` Used for variable assignment in Scala.

A.1.38 Type System

Chisel enforces strict type checking, which may differ from SystemVerilog or VHDL.

A.1.39 Combining Hardware and Software Constructs

Be cautious when mixing Scala control structures (e.g., `if`, `for`) with Chisel constructs (`when`, `switch`) (Chisel Contributors, 2023b).

A.1.40 Hardware vs. Software Mindset

Designers must think in terms of hardware concurrency, even when writing code that resembles sequential software.

A.1.41 Installing Chisel on Ubuntu

A.1.42 Install a Java Development Kit

Chisel is built on Scala, which requires the Java Development Kit (JDK). You have two options to install and manage the JDK.

A.1.42.1 Install the Default JDK

```
1 sudo apt update && sudo apt install default-jdk
```

A.1.42.2 SDKMan

SDKMAN! is a tool for managing parallel versions of Software Development Kits (SDKs) such as Java, Groovy, Kotlin, and more. It simplifies the installation, update, and switching between SDK versions.

Installation To install SDKMAN!, open a terminal and run the following command:

```
1 curl -s "https://get.sdkman.io" | bash
```

Add the following line to your .bashrc configuration file:

```
1 export SDKMAN_DIR="$HOME/.sdkman"
2 source "$SDKMAN_DIR/bin/sdkman-init.sh"
3 source ~/.bashrc
```

After installation, restart your terminal or run:

```
1 source "$HOME/.sdkman/bin/sdkman-init.sh"
```

Verify the installation by running:

```
1 sdk version
```

Using SDKMAN! Here are some common commands:

List All Available Java Versions:

```
1 sdk list java
```

Install a Specific Version

```
1 sdk install java 23-oracle
```

Set a Default Version

```
1 sdk default java 23-oracle
```

Switch Java Versions:

```
1 sdk use java 23-oracle
```


A.1.43 Install Scala Build Tool (sbt)

```
1 echo "deb https://repo.scala-sbt.org/scalasbt/debian all main" | \  
2 sudo tee /etc/apt/sources.list.d/sbt.list && \  
3 curl -sL "https://keyserver.ubuntu.com/pks/lookup? \  
4 op=get&search=0x99E82A75642AC823" | \  
5 sudo apt-key add && \  
6 sudo apt update && \  
7 sudo apt install sbt
```

Clone this book Or use the template at

```
1 https://github.com/chipsalliance/chisel
```

Build and Run Your Chisel Project Navigate to your project's directory and use sbt to compile and run it:

```
1 cd your-chisel-project && sbt test
```

A.1.44 Install Verilator

```
1 sudo apt-get install verilator
```

For any operating system other than Ubuntu, see: <https://verilator.org/guide/latest/install.html>.

A.1.45 Install gtkwave

To view the waveforms created by verilator, instal gtkwave.

```
1 sudo apt-get install gtkwave
```

A.1.46 Install Vivado

A.1.47 Install OpenLane

A.1.48 Install OSS CAD Suite

A.1.49 Install cirtc (firtool)

A.1.50 Example Walkthrough

Assuming you have cloned this book (Structured Computer Architecture), you will find the Chisel source files in the RTL/Chisel directory at the top level of the repository.

A.1.51 Key Directories

RTL/Chisel/src/main/scala/scabook:	Chisel source files
RTL/Chisel/test/scala/scabook:	Chisel test harness files
RTL/Chisel/generated_verilog_clean:	SystemVerilog generated by Chisel
RTL/Chisel/generated_verilog_annotated:	SystemVerilog with debug symbols
RTL/Chisel/verilog_testbench:	Verilator C++ testbench files

A.1.52 Run Chisel Test

The following very simple design generates a waveform

```

1  // Licensed under the BSD 3-Clause License.
2  // See https://opensource.org/licenses/BSD-3-Clause for details.
3  package scabook
4
5  import chisel3._
6  import circt.stage.ChiselStage
7
8  class WaveFormGenerator extends Module {
9    val io = IO(new Bundle {
10      val randomWave = Output(Bool())
11      val clockWave = Output(Bool())
12    })
13
14    // Random wave pattern based on a sequence of values
15    val randomWaveSeq = VecInit(false.B, true.B, false.B, true.B, false.B
16    )
17    val randomIndex = RegInit(0.U(3.W))
18    io.randomWave := randomWaveSeq(randomIndex)
19    randomIndex := Mux(randomIndex === 4.U, 0.U, randomIndex + 1.U)
20
21    // Clock waveform toggles every 2 cycles
22    val clockToggle = RegInit(false.B)
23    clockToggle := ~clockToggle
24    io.clockWave := clockToggle
25  }
26
27  object WaveFormGenerator extends App {
28    ChiselStage.emitSystemVerilogFile(
29      new WaveFormGenerator,
30      Array("--target-dir", "generated_verilog_annotated")
31      //firtoolOpts = Array("-disable-all-randomization", "-strip-debug-
32      info"),
33    )
34  }

```

Listing A.1: Waveform Generator Code (Chisel)

The testbench for the code is:

```

1 // See README.md for license details.
2
3 package scabook
4
5 import chisel3._
6 import chisel3.simulator.EphemeralSimulator._
7 import org.scalatest.freespec.AnyFreeSpec
8 import org.scalatest.matchers.must.Matchers
9
10 class WaveFormGeneratorSpec extends AnyFreeSpec with Matchers {
11
12   "WaveFormGenerator should generate the correct waveforms" in {
13     simulate(new WaveFormGenerator) { dut =>
14       val randomWaveExpected = Seq(false, true, false, true, false)
15       var randomIndex = 0
16       var clockState = false
17
18       for (cycle <- 0 until 10) { // Simulate 10 clock cycles
19         // Check randomWave output
20         dut.io.randomWave.expect(randomWaveExpected(randomIndex).B)
21         randomIndex = (randomIndex + 1) % randomWaveExpected.length
22
23         // Check clockWave toggling every cycle
24         dut.io.clockWave.expect(clockState.B)
25         clockState = !clockState
26
27         // Step the simulation forward
28         dut.clock.step()
29       }
30     }
31   }
32 }

```

Listing A.2: Waveform Generator Testbench Code (Chisel)

A.1.53 Run the Testbench

If you have cloned the textbook, build.sbt will be in your Chisel/ directory.

```

1 ThisBuild / scalaVersion      := "2.13.15"
2 ThisBuild / version           := "0.1.0"
3 ThisBuild / organization      := "%ORGANIZATION%"
4
5 val chiselVersion = "6.6.0"
6
7 lazy val root = (project in file("."))
8   .settings(
9     name := "%NAME%",
10    libraryDependencies ++= Seq(

```

```

11     "org.chipsalliance" %% "chisel" % chiselVersion,
12     "org.scalatest" %% "scalatest" % "3.2.16" % Test,
13     //"ch.epfl.scala" % "sbt-bloop_2.12_1.0" % "2.0.5", //bloop in
        VSC
14     "org.slf4j" % "slf4j-api" % "2.0.9",           // SLF4J API
15     "org.slf4j" % "slf4j-simple" % "2.0.9"         // Simple SLF4J
        backend
16 ),
17
18 // It seems sbt-bloop is not compatible with scala 2.13 yet
19 // https://index.scala-lang.org/scalacenter/bloop/artifacts/sbt-
        bloop
20 //addSbtPlugin("ch.epfl.scala" % "sbt-bloop-core" % "1.4.1"),
21
22 scalacOptions ++= Seq(
23     "-language:reflectiveCalls",
24     "-deprecation",
25     "-feature",
26     "-Xcheckinit",
27     "-Ymacro-annotations"
28 ),
29
30
31 addCompilerPlugin("org.chipsalliance" % "chisel-plugin" %
        chiselVersion cross CrossVersion.full),
32
33 // Add custom source directory for generators
34 Compile / unmanagedSourceDirectories += baseDirectory.value / "
        generators",
35
36 // Enable SBT logging
37 //logLevel := Level.Debug, // Enable verbose logging
38
39 // Enable logging by passing JVM properties to the application
40 Compile / run / fork := true,
41 Compile / run / javaOptions ++= Seq(
42     "-Xmx4G",
43     "-Dchisel.firtool=true", // Ensures CIRCT is the backend. https:
        //github.com/llvm/circt
44     "-Dorg.slf4j.simpleLogger.defaultLogLevel=WARN", // Set log level
        to WARN
45     "-Dorg.slf4j.simpleLogger.showDateTime=true", // Optional:
        Show timestamps
46     "-Dorg.slf4j.simpleLogger.dateTimeFormat=yyyy-MM-dd HH:mm:ss" //
        Optional: Timestamp format
47 ),
48
49 // Compile / runMain / javaOptions ++= Seq(
50 //     "-Xmx4G",
51 //     "-Dchisel.firtool=true",
52 //     "-Dorg.slf4j.simpleLogger.defaultLogLevel=WARN",

```

```

53      //      "-Dorg.slf4j.simpleLogger.showDateTime=true",
54      //      "-Dorg.slf4j.simpleLogger.dateTimeFormat=yyyy-MM-dd HH:mm:ss"
55      // )
56  )

```

Listing A.3: Waveform Generator Testbench Code (Chisel)

The following commands will run the test bench

```

1  cd RTL/Chisel/
2  sbt test

```

The IDE being used is Visual Studio Code with the Scala(Metals) extension installed. Your output should look something like:

```

1  (base) jglossner@jglossner-Alienware-Aurora-R7:~/GitRepos/
    Structured_Computer_Architecture/Chisel/$ sbt test
2  downloading sbt launcher 1.10.6
3  [info] welcome to sbt 1.10.5 (Oracle Corporation Java 23)
4  [info] loading settings for project chisel-build-build from metals.sbt
    ...
5  [info] loading project definition from /home/jglossner/GitRepos/
    Structured_Computer_Architecture/Chisel/project/project
6  [info] loading settings for project chisel-build from metals.sbt ...
7  [info] loading project definition from /home/jglossner/GitRepos/
    Structured_Computer_Architecture/Chisel/project
8  [success] Generated .bloop/chisel-build.json
9  [success] Total time: 1 s, completed Nov 30, 2024, 11:12:37 AM
10 [info] loading settings for project root from build.sbt ...
11 [info] set current project to %NAME% (in build file:/home/jglossner/
    GitRepos/Structured_Computer_Architecture/Chisel/)
12 [info] WaveFormGeneratorSpec:
13 [info] - WaveFormGenerator should generate the correct waveforms
14 [info] GCDSpec:
15 [info] - Gcd should calculate proper greatest common denominator
16 [info] Run completed in 8 seconds, 403 milliseconds.
17 [info] Total number of tests run: 2
18 [info] Suites: completed 2, aborted 0
19 [info] Tests: succeeded 2, failed 0, canceled 0, ignored 0, pending 0
20 [info] All tests passed.
21 [success] Total time: 9 s, completed Nov 30, 2024, 11:12:46 AM

```

A.1.54 Generate Clean System Verilog

The only difference between the clean SystemVerilog and the annotated version is the commented out code.

For clean SystemVerilog:

```

1 object WaveFormGenerator extends App {
2   ChiselStage.emitSystemVerilogFile(
3     new WaveFormGenerator,
4     //Array("--target-dir", "generated")
5     firtoolOpts = Array("-disable-all-randomization", "-strip-debug-
      info"),
6   )
7 }

```

Run the SystemVerilog generator from the Chisel/ directory

```
1 sbt run
```

This will produce

```

1 // Generated by CIRCT firtool-1.99.1
2
3 // Include register initializers in init blocks unless synthesis is set
4 `ifndef RANDOMIZE
5   `ifdef RANDOMIZE_REG_INIT
6     `define RANDOMIZE
7   `endif // RANDOMIZE_REG_INIT
8 `endif // not def RANDOMIZE
9 `ifndef SYNTHESIS
10   `ifndef ENABLE_INITIAL_REG_
11     `define ENABLE_INITIAL_REG_
12   `endif // not def ENABLE_INITIAL_REG_
13 `endif // not def SYNTHESIS
14
15 // Standard header to adapt well known macros for register
16   randomization.
17
18 // RANDOM may be set to an expression that produces a 32-bit random
19   unsigned value.
20 `ifndef RANDOM
21   `define RANDOM $random
22 `endif // not def RANDOM
23
24 // Users can define INIT_RANDOM as general code that gets injected into
25   the
26 // initializer block for modules with registers.
27 `ifndef INIT_RANDOM
28   `define INIT_RANDOM
29 `endif // not def INIT_RANDOM
30
31 // If using random initialization, you can also define RANDOMIZE_DELAY
32   to
33 // customize the delay used, otherwise 0.002 is used.

```

```

30 `ifndef RANDOMIZE_DELAY
31     `define RANDOMIZE_DELAY 0.002
32 `endif // not def RANDOMIZE_DELAY
33
34 // Define INIT_RANDOM_PROLOG_ for use in our modules below.
35 `ifndef INIT_RANDOM_PROLOG_
36     `ifdef RANDOMIZE
37         `ifdef VERILATOR
38             `define INIT_RANDOM_PROLOG_ `INIT_RANDOM
39         `else // VERILATOR
40             `define INIT_RANDOM_PROLOG_ `INIT_RANDOM #`RANDOMIZE_DELAY begin
41                 end
42         `endif // VERILATOR
43     `else // RANDOMIZE
44         `define INIT_RANDOM_PROLOG_
45     `endif // RANDOMIZE
46 `endif // not def INIT_RANDOM_PROLOG_
47 module WaveFormGenerator( // /home/jglossner/GitRepos/
48     Structured_Computer_Architecture/RTL/Chisel/generators/generated/
49     firrtl/WaveFormGenerator.fir.mlir:3:5
50     input  clock, // /home/jglossner/GitRepos/
51     Structured_Computer_Architecture/RTL/Chisel/generators/generated/
52     firrtl/WaveFormGenerator.fir.mlir:3:41
53     reset, // /home/jglossner/GitRepos/
54     Structured_Computer_Architecture/RTL/Chisel/generators/
55     generated/firrtl/WaveFormGenerator.fir.mlir:3:67
56     output io_randomWave, // /home/jglossner/GitRepos/
57     Structured_Computer_Architecture/RTL/Chisel/generators/generated/
58     firrtl/WaveFormGenerator.fir.mlir:3:96
59     io_clockWave // /home/jglossner/GitRepos/
60     Structured_Computer_Architecture/RTL/Chisel/generators/
61     generated/firrtl/WaveFormGenerator.fir.mlir:3:133
62 );
63
64 wire [7:0] _GEN = '{1'h0, 1'h0, 1'h0, 1'h0, 1'h1, 1'h0, 1'h1, 1'h0};
65 reg  [2:0] randomIndex; // /home/jglossner/GitRepos/
66 Structured_Computer_Architecture/RTL/Chisel/generators/generated/
67 firrtl/WaveFormGenerator.fir.mlir:12:22
68 reg      clockToggle; // /home/jglossner/GitRepos/
69 Structured_Computer_Architecture/RTL/Chisel/generators/generated/
70 firrtl/WaveFormGenerator.fir.mlir:20:22
71 always @(posedge clock) begin // /home/jglossner/GitRepos/
72     Structured_Computer_Architecture/RTL/Chisel/generators/generated/
73     firrtl/WaveFormGenerator.fir.mlir:3:41
74     if (reset) begin // /home/jglossner/GitRepos/
75         Structured_Computer_Architecture/RTL/Chisel/generators/generated/
76         /firrtl/WaveFormGenerator.fir.mlir:3:41
77         randomIndex <= 3'h0; // /home/jglossner/GitRepos/
78         Structured_Computer_Architecture/RTL/Chisel/generators/
79         generated/firrtl/WaveFormGenerator.fir.mlir:7:17, :12:22
80         clockToggle <= 1'h0; // /home/jglossner/GitRepos/

```

```

        Structured_Computer_Architecture/RTL/Chisel/generators/
        generated/firrtl/WaveFormGenerator.fir.mlir:3:5, :20:22
60     end
61     else begin // /home/jglossner/GitRepos/
        Structured_Computer_Architecture/RTL/Chisel/generators/generated
        /firrtl/WaveFormGenerator.fir.mlir:3:41
62         randomIndex <= randomIndex == 3'h4 ? 3'h0 : randomIndex + 3'h1;
        // /home/jglossner/GitRepos/
        Structured_Computer_Architecture/RTL/Chisel/generators/
        generated/firrtl/WaveFormGenerator.fir.mlir:6:17, :7:17,
        :12:22, :15:25, :16:27, :18:27
63         clockToggle <= ~clockToggle; // /home/jglossner/GitRepos/
        Structured_Computer_Architecture/RTL/Chisel/generators/
        generated/firrtl/WaveFormGenerator.fir.mlir:20:22, :21:25
64     end
65 end // always @(posedge)
66 `ifdef ENABLE_INITIAL_REG_ // /home/jglossner/GitRepos/
        Structured_Computer_Architecture/RTL/Chisel/generators/generated/
        firrtl/WaveFormGenerator.fir.mlir:3:5
67 `ifdef FIRRTL_BEFORE_INITIAL // /home/jglossner/GitRepos/
        Structured_Computer_Architecture/RTL/Chisel/generators/generated
        /firrtl/WaveFormGenerator.fir.mlir:3:5
68 `FIRRTL_BEFORE_INITIAL // /home/jglossner/GitRepos/
        Structured_Computer_Architecture/RTL/Chisel/generators/
        generated/firrtl/WaveFormGenerator.fir.mlir:3:5
69 `endif // FIRRTL_BEFORE_INITIAL
70 initial begin // /home/jglossner/GitRepos/
        Structured_Computer_Architecture/RTL/Chisel/generators/generated
        /firrtl/WaveFormGenerator.fir.mlir:3:5
71     automatic logic [31:0] _RANDOM[0:0]; // /home/jglossner/
        GitRepos/Structured_Computer_Architecture/RTL/Chisel/
        generators/generated/firrtl/WaveFormGenerator.fir.mlir:3:5
72 `ifdef INIT_RANDOM_PROLOG_ // /home/jglossner/GitRepos/
        Structured_Computer_Architecture/RTL/Chisel/generators/
        generated/firrtl/WaveFormGenerator.fir.mlir:3:5
73 `INIT_RANDOM_PROLOG_ // /home/jglossner/GitRepos/
        Structured_Computer_Architecture/RTL/Chisel/generators/
        generated/firrtl/WaveFormGenerator.fir.mlir:3:5
74 `endif // INIT_RANDOM_PROLOG_
75 `ifdef RANDOMIZE_REG_INIT // /home/jglossner/GitRepos/
        Structured_Computer_Architecture/RTL/Chisel/generators/
        generated/firrtl/WaveFormGenerator.fir.mlir:3:5
76     _RANDOM[/*Zero width*/ 1'b0] = `RANDOM; // /home/jglossner/
        GitRepos/Structured_Computer_Architecture/RTL/Chisel/
        generators/generated/firrtl/WaveFormGenerator.fir.mlir:3:5
77     randomIndex = _RANDOM[/*Zero width*/ 1'b0][2:0]; // /
        home/jglossner/GitRepos/Structured_Computer_Architecture/RTL
        /Chisel/generators/generated/firrtl/WaveFormGenerator.fir.
        mlir:3:5, :12:22
78     clockToggle = _RANDOM[/*Zero width*/ 1'b0][3]; // /home/
        jglossner/GitRepos/Structured_Computer_Architecture/RTL/

```



```

        Chisel/generators/generated/firrtl/WaveFormGenerator.fir.
        mlir:3:5, :12:22, :20:22
79     `endif // RANDOMIZE_REG_INIT
80   end // initial
81   `ifdef FIRRTL_AFTER_INITIAL // /home/jglossner/GitRepos/
        Structured_Computer_Architecture/RTL/Chisel/generators/generated
        /firrtl/WaveFormGenerator.fir.mlir:3:5
82     `FIRRTL_AFTER_INITIAL // /home/jglossner/GitRepos/
        Structured_Computer_Architecture/RTL/Chisel/generators/
        generated/firrtl/WaveFormGenerator.fir.mlir:3:5
83   `endif // FIRRTL_AFTER_INITIAL
84   `endif // ENABLE_INITIAL_REG_
85   assign io_randomWave = _GEN[randomIndex]; // /home/jglossner/
        GitRepos/Structured_Computer_Architecture/RTL/Chisel/generators/
        generated/firrtl/WaveFormGenerator.fir.mlir:3:5, :12:22, :13:12
86   assign io_clockWave = clockToggle; // /home/jglossner/GitRepos/
        Structured_Computer_Architecture/RTL/Chisel/generators/generated/
        firrtl/WaveFormGenerator.fir.mlir:3:5, :20:22
87 endmodule

```

Listing A.4: Clean SystemVerilog Code Produced by Chisel

A.1.55 Generate Annotated/Debug System Verilog

This code will put the SystemVerilog simulation code directly into a generated folder

```

1  object WaveFormGenerator extends App {
2    ChiselStage.emitSystemVerilogFile(
3      new WaveFormGenerator,
4      Array("--target-dir", "generated")
5      //firtoolOpts = Array("-disable-all-randomization", "-strip-debug-
        info"),
6    )
7  }

```

The annotated/debug SystemVerilog generated code should look like:

```

1  // Generated by CIRCT firtool-1.99.1
2
3  // Include register initializers in init blocks unless synthesis is set
4  `ifndef RANDOMIZE
5    `ifdef RANDOMIZE_REG_INIT
6      `define RANDOMIZE
7    `endif // RANDOMIZE_REG_INIT
8  `endif // not def RANDOMIZE
9  `ifndef SYNTHESIS
10   `ifndef ENABLE_INITIAL_REG_
11     `define ENABLE_INITIAL_REG_
12   `endif // not def ENABLE_INITIAL_REG_

```

```

13 `endif // not def SYNTHESIS
14
15 // Standard header to adapt well known macros for register
    randomization.
16
17 // RANDOM may be set to an expression that produces a 32-bit random
    unsigned value.
18 `ifndef RANDOM
19     `define RANDOM $random
20 `endif // not def RANDOM
21
22 // Users can define INIT_RANDOM as general code that gets injected into
    the
23 // initializer block for modules with registers.
24 `ifndef INIT_RANDOM
25     `define INIT_RANDOM
26 `endif // not def INIT_RANDOM
27
28 // If using random initialization, you can also define RANDOMIZE_DELAY
    to
29 // customize the delay used, otherwise 0.002 is used.
30 `ifndef RANDOMIZE_DELAY
31     `define RANDOMIZE_DELAY 0.002
32 `endif // not def RANDOMIZE_DELAY
33
34 // Define INIT_RANDOM_PROLOG_ for use in our modules below.
35 `ifndef INIT_RANDOM_PROLOG_
36     `ifdef RANDOMIZE
37         `ifdef VERILATOR
38             `define INIT_RANDOM_PROLOG_ `INIT_RANDOM
39         `else // VERILATOR
40             `define INIT_RANDOM_PROLOG_ `INIT_RANDOM #`RANDOMIZE_DELAY begin
                end
41         `endif // VERILATOR
42     `else // RANDOMIZE
43         `define INIT_RANDOM_PROLOG_
44     `endif // RANDOMIZE
45 `endif // not def INIT_RANDOM_PROLOG_
46 module WaveFormGenerator( // /home/jglossner/GitRepos/
    Structured_Computer_Architecture/RTL/Chisel/generators/generated/
    firrtl/WaveFormGenerator.fir.mlir:3:5
47 input  clock, // /home/jglossner/GitRepos/
    Structured_Computer_Architecture/RTL/Chisel/generators/generated/
    firrtl/WaveFormGenerator.fir.mlir:3:41
48     reset, // /home/jglossner/GitRepos/
    Structured_Computer_Architecture/RTL/Chisel/generators/
    generated/firrtl/WaveFormGenerator.fir.mlir:3:67
49 output io_randomWave, // /home/jglossner/GitRepos/
    Structured_Computer_Architecture/RTL/Chisel/generators/generated/
    firrtl/WaveFormGenerator.fir.mlir:3:96
50     io_clockWave // /home/jglossner/GitRepos/

```

```

        Structured_Computer_Architecture/RTL/Chisel/generators/
        generated/firrtl/WaveFormGenerator.fir.mlir:3:133
51 );
52
53 wire [7:0] _GEN = '{1'h0, 1'h0, 1'h0, 1'h0, 1'h1, 1'h0, 1'h1, 1'h0};
54 reg [2:0] randomIndex; // /home/jglossner/GitRepos/
    Structured_Computer_Architecture/RTL/Chisel/generators/generated/
    firrtl/WaveFormGenerator.fir.mlir:12:22
55 reg clockToggle; // /home/jglossner/GitRepos/
    Structured_Computer_Architecture/RTL/Chisel/generators/generated/
    firrtl/WaveFormGenerator.fir.mlir:20:22
56 always @(posedge clock) begin // /home/jglossner/GitRepos/
    Structured_Computer_Architecture/RTL/Chisel/generators/generated/
    firrtl/WaveFormGenerator.fir.mlir:3:41
57 if (reset) begin // /home/jglossner/GitRepos/
    Structured_Computer_Architecture/RTL/Chisel/generators/generated
    /firrtl/WaveFormGenerator.fir.mlir:3:41
58 randomIndex <= 3'h0; // /home/jglossner/GitRepos/
    Structured_Computer_Architecture/RTL/Chisel/generators/
    generated/firrtl/WaveFormGenerator.fir.mlir:7:17, :12:22
59 clockToggle <= 1'h0; // /home/jglossner/GitRepos/
    Structured_Computer_Architecture/RTL/Chisel/generators/
    generated/firrtl/WaveFormGenerator.fir.mlir:3:5, :20:22
60 end
61 else begin // /home/jglossner/GitRepos/
    Structured_Computer_Architecture/RTL/Chisel/generators/generated
    /firrtl/WaveFormGenerator.fir.mlir:3:41
62 randomIndex <= randomIndex == 3'h4 ? 3'h0 : randomIndex + 3'h1;
    // /home/jglossner/GitRepos/
    Structured_Computer_Architecture/RTL/Chisel/generators/
    generated/firrtl/WaveFormGenerator.fir.mlir:6:17, :7:17,
    :12:22, :15:25, :16:27, :18:27
63 clockToggle <= ~clockToggle; // /home/jglossner/GitRepos/
    Structured_Computer_Architecture/RTL/Chisel/generators/
    generated/firrtl/WaveFormGenerator.fir.mlir:20:22, :21:25
64 end
65 end // always @(posedge)
66 `ifdef ENABLE_INITIAL_REG_ // /home/jglossner/GitRepos/
    Structured_Computer_Architecture/RTL/Chisel/generators/generated/
    firrtl/WaveFormGenerator.fir.mlir:3:5
67 `ifdef FIRRTL_BEFORE_INITIAL // /home/jglossner/GitRepos/
    Structured_Computer_Architecture/RTL/Chisel/generators/generated
    /firrtl/WaveFormGenerator.fir.mlir:3:5
68 `FIRRTL_BEFORE_INITIAL // /home/jglossner/GitRepos/
    Structured_Computer_Architecture/RTL/Chisel/generators/
    generated/firrtl/WaveFormGenerator.fir.mlir:3:5
69 `endif // FIRRTL_BEFORE_INITIAL
70 initial begin // /home/jglossner/GitRepos/
    Structured_Computer_Architecture/RTL/Chisel/generators/generated
    /firrtl/WaveFormGenerator.fir.mlir:3:5
71 automatic logic [31:0] _RANDOM[0:0]; // /home/jglossner/

```

```

72     GitRepos/Structured_Computer_Architecture/RTL/Chisel/
        generators/generated/firrtl/WaveFormGenerator.fir.mlir:3:5
`ifdef INIT_RANDOM_PROLOG_ // /home/jglossner/GitRepos/
    Structured_Computer_Architecture/RTL/Chisel/generators/
        generated/firrtl/WaveFormGenerator.fir.mlir:3:5
73     `INIT_RANDOM_PROLOG_ // /home/jglossner/GitRepos/
        Structured_Computer_Architecture/RTL/Chisel/generators/
        generated/firrtl/WaveFormGenerator.fir.mlir:3:5
74 `endif // INIT_RANDOM_PROLOG_
75 `ifdef RANDOMIZE_REG_INIT // /home/jglossner/GitRepos/
    Structured_Computer_Architecture/RTL/Chisel/generators/
        generated/firrtl/WaveFormGenerator.fir.mlir:3:5
76     _RANDOM[/*Zero width*/ 1'b0] = `RANDOM; // /home/jglossner/
        GitRepos/Structured_Computer_Architecture/RTL/Chisel/
        generators/generated/firrtl/WaveFormGenerator.fir.mlir:3:5
77     randomIndex = _RANDOM[/*Zero width*/ 1'b0][2:0]; // /
        home/jglossner/GitRepos/Structured_Computer_Architecture/RTL/
        /Chisel/generators/generated/firrtl/WaveFormGenerator.fir.
        mlir:3:5, :12:22
78     clockToggle = _RANDOM[/*Zero width*/ 1'b0][3]; // /home/
        jglossner/GitRepos/Structured_Computer_Architecture/RTL/
        Chisel/generators/generated/firrtl/WaveFormGenerator.fir.
        mlir:3:5, :12:22, :20:22
79 `endif // RANDOMIZE_REG_INIT
80 end // initial
81 `ifdef FIRRTL_AFTER_INITIAL // /home/jglossner/GitRepos/
    Structured_Computer_Architecture/RTL/Chisel/generators/generated
        /firrtl/WaveFormGenerator.fir.mlir:3:5
82     `FIRRTL_AFTER_INITIAL // /home/jglossner/GitRepos/
        Structured_Computer_Architecture/RTL/Chisel/generators/
        generated/firrtl/WaveFormGenerator.fir.mlir:3:5
83 `endif // FIRRTL_AFTER_INITIAL
84 `endif // ENABLE_INITIAL_REG_
85 assign io_randomWave = _GEN[randomIndex]; // /home/jglossner/
    GitRepos/Structured_Computer_Architecture/RTL/Chisel/generators/
        generated/firrtl/WaveFormGenerator.fir.mlir:3:5, :12:22, :13:12
86 assign io_clockWave = clockToggle; // /home/jglossner/GitRepos/
    Structured_Computer_Architecture/RTL/Chisel/generators/generated/
        firrtl/WaveFormGenerator.fir.mlir:3:5, :20:22
87 endmodule

```

Listing A.5: Annotated SystemVerilog Code Produced by Chisel

A.1.56 Verilator C++ Test Driver

Create a Verilator C++ test driver. It is in Chisel/verilator_testbench

```

1  #include "VWaveFormGenerator.h" // Verilated top module
2  #include "verilated.h"
3  #include "verilated_vcd_c.h" // For waveform tracing (optional)

```

```

4
5  int main(int argc, char **argv) {
6      Verilated::commandArgs(argc, argv);
7
8      // Instantiate the DUT (Device Under Test)
9      VWaveFormGenerator* dut = new VWaveFormGenerator;
10
11     // Enable waveform tracing
12     Verilated::traceEverOn(true);
13     VerilatedVcdC* trace = new VerilatedVcdC;
14     dut->trace(trace, 99); // Trace 99 levels of hierarchy
15     trace->open("waveform.vcd");
16
17     // Simulate for 20 clock cycles
18     for (int cycle = 0; cycle < 20; ++cycle) {
19         // Toggle clock
20         dut->clock = !dut->clock;
21
22         // Evaluate the DUT
23         dut->eval();
24
25         // Dump waveform data to the VCD file
26         trace->dump(cycle);
27
28         // Print output signals
29         if (dut->clock) { // Print on the rising edge of the clock
30             printf("Cycle %d: randomWave=%d, clockWave=%d\n", cycle /
31                 2, dut->io_randomWave, dut->io_clockWave);
32         }
33
34         // Clean up
35         trace->close();
36         delete trace;
37         delete dut;
38         return 0;
39     }

```

Listing A.6: Verilator Testbench for WaveFormGenerator

From the Chisel/verilator_testbench, execute:

```

1  verilator --cc ../generated_verilog_annotated/WaveFormGenerator.sv --
   exe WaveFormGenerator_testbench.cpp --trace
2  make -C obj_dir -f VWaveFormGenerator.mk VWaveFormGenerator

```

This will output the executable in Chisel/verilator_testbench/obj_dir
 Execute the WaveFormGenerator program create in the obj_dir

```
1 obj_dir/VWaveFormGenerator
```

This will create a vcd file that can be displayed with gtkwave

```
1 gtkwave waveform.vcd
```

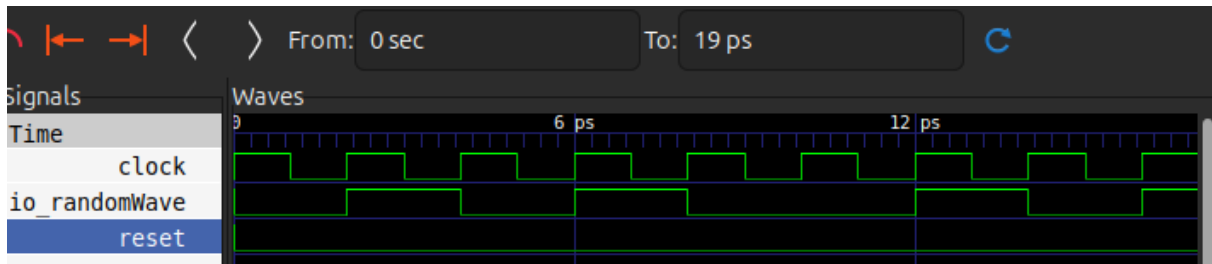


Figure A.1: Waveforms generated for a Verilator SystemVerilog simulation

A.1.57 Vivado Execution

Vivado is a comprehensive FPGA design suite developed by Xilinx. It integrates various tools for hardware design, simulation, synthesis, implementation, and debugging. Vivado supports Verilog, SystemVerilog, and VHDL for both design and simulation.

A.1.58 Steps to Use Vivado

1) Create a Project:

- Open Vivado and create a new project.
- Specify the project type (RTL or post-synthesis).
- Add source files (e.g., Verilog, SystemVerilog, or VHDL).

2) Write or Import a Testbench:

- Add a testbench to simulate your design.
- Set the testbench as the top module for simulation.

3) Simulate the Design:

- Use the Vivado simulator to run behavioral simulations.
- Analyze the results in the waveform viewer.

4) Synthesize and Implement:

- Perform synthesis to generate a netlist.
- Run implementation to map the design to the FPGA.

5) Generate a Bitstream:

- Create a bitstream file to program the FPGA.

A.1.59 Key Features of Vivado

- Supports advanced FPGA features such as high-speed transceivers and DSP blocks.
- Built-in waveform viewer for debugging simulations.
- Comprehensive support for Xilinx devices.

A.1.60 Comparison of Vivado and Verilator

Vivado and Verilator are both widely used in hardware design, but they cater to different needs. Below is a detailed comparison:

When to Use Vivado vs. Verilator

- **Use Vivado:**
 - ◇ When designing and implementing FPGA designs targeting Xilinx devices.
 - ◇ If you require a complete toolchain for synthesis, implementation, and simulation.
 - ◇ For designs involving VHDL.
- **Use Verilator:**
 - ◇ For high-speed simulations of Verilog/SystemVerilog designs.
 - ◇ When testing large designs with long simulation runs.
 - ◇ For open-source or academic projects where licensing costs are a concern.

The following is the SystemVerilog module ‘waveFormGenerator’, which generates a random waveform and a clock signal:

```

1  // Licensed under CERN-OHL-S v2. See https://cern.ch/cern-ohl for
   details.
2  /*****
3  File name: waveFormGenerator.sv

```

Feature	Vivado	Verilator
Purpose	Comprehensive FPGA design suite	High-speed cycle-accurate simulation tool
Languages Supported	Verilog, SystemVerilog, VHDL	Verilog, SystemVerilog
Test Driver Language	HDL (Verilog/SystemVerilog/VHDL)	C++ (or SystemC for advanced simulations)
Simulation Speed	Moderate (event-driven simulator)	Very high (compiled C++ simulation)
Waveform Viewer	Built-in (integrated with simulation)	Requires external tools like GTKWave (VCD)
FPGA Synthesis	Yes	No
FPGA Targeting	Xilinx FPGAs	Not applicable
Debugging Features	Integrated debugging and timing analysis tools	Debugging through C++ or external tools
Open Source	No (proprietary software)	Yes (GPL-licensed)

Table A.1: Comparison of Vivado and Verilator


```

4  Circuit name: no circuit, only wave formes
5  Description: two waveforms are generated, a "random" one and
6    a periodical one: a clock signal
7  *****/
8  module waveFormGenerator();
9      logic randomWave;
10     logic clock      ;
11     initial begin          randomWave = 0  ;
12                             #2 randomWave = 1  ;
13                             #6 randomWave = 0  ;
14                             #4 randomWave = 1  ;
15                             #8 randomWave = 0  ;
16                             #5 $stop          ;
17     end
18     initial begin          clock = 0          ;
19                             forever #2 clock = ~clock  ;
20     end
21 endmodule

```

SystemVerilog Testbench

The testbench for the ‘waveFormGenerator’ module monitors its signals during simulation:

```

1  `timescale 1ns/1ps
2
3  module waveFormGenerator_tb;
4      logic randomWave;
5      logic clock;
6
7      // Instantiate the DUT (Device Under Test)
8      waveFormGenerator dut();
9
10     initial begin
11         $monitor("Time: %0t | randomWave: %b | clock: %b", $time, dut.
12             randomWave, dut.clock);
13         #30 $finish; // End the simulation after 30 time units
14     end
15 endmodule

```

Listing A.7: SystemVerilog Testbench for waveFormGenerator

Explanation

Module Details

The ‘waveFormGenerator’ module generates:

- A ‘randomWave’ signal with values that change at specified time delays.
- A ‘clock’ signal that toggles every 2 time units.

Testbench Details

The testbench:

- Instantiates the ‘waveFormGenerator’ module.
- Monitors the ‘randomWave’ and ‘clock’ signals using ‘\$monitor’.
- Ends the simulation after 30 time units with ‘\$finish’.

A.2 System Verilog

A.2.1 Introduction

SystemVerilog is a hardware description and hardware verification language, which extends the capabilities of Verilog by incorporating advanced modeling features and constructs (IEEE, 2018). It combines design specification and verification features, making it a unified language for hardware design and verification. This chapter serves as a reference guide for hardware designers familiar with Verilog or VHDL, providing syntax and commands of SystemVerilog.

A.2.2 Overview of SystemVerilog

SystemVerilog enhances Verilog by introducing new data types, operators, and constructs for modeling complex hardware designs. It provides advanced features for design abstraction, code reusability, and verification, making it suitable for modern hardware development (Sutherland and Mills, 2010).

A.2.3 Key Enhancements over Verilog

- Richer data types and user-defined types.
- Enhanced procedural blocks and control flow.
- Interfaces and modports for better module communication.
- Object-oriented programming features for verification.
- Assertions and coverage constructs for verification.

A.2.4 Syntax

A.2.5 Data Types

SystemVerilog introduces new data types in addition to those available in Verilog.

A.2.5.1 Built-in Data Types

logic Replaces **reg** and can be driven by continuous assignments.

bit Unsigned, 2-state data type (0 or 1).

byte 8-bit signed integer.

shortint 16-bit signed integer.

int 32-bit signed integer.

longint 64-bit signed integer.

integer Equivalent to `int`.

time 64-bit unsigned integer.

A.2.5.2 User-Defined Types

typedef Used to create new type names. Example: `typedef logic [7:0] byte_t;`

enum Enumerated types. Example: `enum logic [1:0] {IDLE, BUSY, DONE} state;`

struct Composite data types grouping variables. Example: `struct {logic [7:0] addr; logic [15:0] data;} packet;`

union Overlapping data storage. Example: `union {int i; byte b[4];} u;`

A.2.6 Arrays

A.2.6.1 Packed and Unpacked Arrays

Packed Arrays Continuous set of bits. Declaration: `logic [7:0] data_byte;`

Unpacked Arrays Arrays of elements. Declaration: `logic [7:0] memory [0:255];`

A.2.6.2 Dynamic and Associative Arrays

dynamic array Size can change at runtime. Declaration: `int dynamic_array[];`

associative array Indexed by arbitrary data types. Declaration: `int associative_array[string];`

queue Variable-size ordered collection. Declaration: `int queue[$];`

A.2.7 Operators

SystemVerilog extends Verilog operators with new ones.

Streaming Operators `<<, >>` Used for packing and unpacking bits.

Type Casting `$cast`, static casting (`type'(expression)`)

Conditional Operator Ternary operator (`? :`) enhanced for 4-state expressions.

A.2.8 Procedural Blocks and Control Flow

A.2.9 Enhanced Procedural Blocks

`always_comb` Replaces `always @(*)` for combinational logic.

`always_ff` For sequential logic triggered by clock edges.

`always_latch` For latch inference.

A.2.10 Control Flow Statements

SystemVerilog introduces new control flow constructs.

A.2.10.1 Looping Constructs

`foreach` Iterates over arrays. Example: `foreach (array[i]) array[i] = 0;`

`do...while` Post-test loop.

`break, continue` Loop control.

`return` Exits a function or task.

A.2.10.2 Conditional Compilation

``$ifdef`, ``$ifndef`, ``$elsif`, ``$else`, ``$endif`

A.2.11 Interfaces and Modports

Interfaces encapsulate communication between modules.

```

1 interface bus_if(input logic clk);
2     logic [7:0] addr;
3     logic [15:0] data;
4     logic valid;
5     modport master (output addr, data, valid);
6     modport slave (input addr, data, valid);
7 endinterface

```

Modports define the direction of signals for different roles.

- Specify which signals are inputs or outputs for a module.

- Enhance readability and enforce directionality.

A.2.12 Tasks and Functions

Functions return values and execute in zero simulation time.

- Can have input arguments only.
- Cannot consume time (no delays or event controls).
- Declared using `function` keyword.

Tasks can consume time and have input/output/inout arguments.

- Can include delays and event controls.
- Declared using `task` keyword.

A.2.13 Packages

Packages allow the grouping of declarations that can be shared across modules.

```
1 package my_pkg;  
2     typedef logic [7:0] byte_t;  
3     function int increment(int a);  
4         return a + 1;  
5     endfunction  
6 endpackage  
7  
8 // Importing a package  
9 import my_pkg::*;
```

A.2.14 Object-Oriented Programming Features

SystemVerilog introduces classes for verification purposes (Stapko, 2008).

A.2.15 Classes

- Support for inheritance, polymorphism, and encapsulation.
- Used primarily in testbenches.

A.2.16 Randomization

- Constraint random variables using `rand`, `randc`.
- Use `constraint` blocks to specify constraints.
- Randomize using `$urandom`, `randomize()` method.

A.2.17 Assertions and Coverage

A.2.18 Assertions

SystemVerilog assertions (SVA) are used for design verification (Foster, 2003).

- Immediate Assertions: Checked immediately when executed.
Syntax: `assert(expression) else ...`
- Concurrent Assertions: Monitors behaviors over time.
Syntax: `assert property (property_expression);`

A.2.19 Coverage

Coverage metrics help measure how much of the design has been tested.

- `covergroup`: Defines coverage points.
- `coverpoint`: Specifies variables or expressions to be covered.
- `cross`: Cross-coverage between coverpoints.

A.2.20 Peculiarities of SystemVerilog

A.2.21 Multiple Procedural Blocks

SystemVerilog introduces specialized procedural blocks (`always_comb`, `always_ff`, `always_latch`) that help avoid common coding errors by automatically inferring the sensitivity list.

A.2.22 Data Type Confusion

The introduction of new data types like `logic`, `bit`, and `byte` can cause confusion with traditional Verilog types (`reg`, `wire`).

A.2.23 Implicit Net Declarations

In Verilog, undeclared identifiers default to a one-bit wire. SystemVerilog requires all nets and variables to be explicitly declared unless the compiler directive ``default_nettype` is used.

A.2.24 Use of Interfaces

Interfaces can significantly simplify module connections but require a shift in design methodology compared to traditional Verilog module port lists.

A.2.25 Enhanced For Loops

The `foreach` loop in SystemVerilog can only be used with arrays and can lead to less readable code if not used carefully.

A.2.26 Randomization and Constraints

While powerful for verification, the randomization features and constraints are complex and can lead to non-intuitive behaviors if constraints are not properly defined.

A.2.27 Mixing Verification and Design Constructs

SystemVerilog allows mixing design and verification constructs, which can lead to code that is not synthesizable. Designers need to be careful to use synthesizable constructs in design code.

A.3 Chisel Compared with System Verilog

A.3.1 Introduction

SystemVerilog and Chisel are both powerful languages used for hardware design and verification. SystemVerilog is an extension of Verilog, widely used in the industry for designing and verifying digital systems (IEEE, 2018). Chisel, on the other hand, is a hardware description language embedded in Scala, offering advanced features like parameterization and functional programming paradigms (Bachrach et al., 2012). This chapter provides a comparative analysis of SystemVerilog and Chisel, highlighting directly mapped constructs, unique features of each language, and guidance for implementing designs when transitioning between the two languages.

A.3.2 Overview of SystemVerilog and Chisel

A.3.3 SystemVerilog

SystemVerilog enhances Verilog by incorporating advanced modeling features, object-oriented programming concepts, and verification constructs (Sutherland and Mills, 2010). It is standardized as IEEE 1800 and is widely adopted in the semiconductor industry for both design and verification purposes.

A.3.4 Chisel

Chisel is a domain-specific language for hardware design embedded in Scala, enabling hardware designers to use modern programming paradigms (Chisel Contributors, 2023a). It provides powerful abstractions for parameterization and reuse, and it generates synthesizable Verilog code, integrating with traditional hardware development flows.

A.3.5 Directly Mapped Constructs

Despite differences in syntax and paradigms, many hardware constructs have direct equivalents in both SystemVerilog and Chisel. This section presents a comparison of these constructs.

A.3.6 Data Types

A.3.7 Modules and Hierarchy

- *Module Definition:*
 - ◊ **SystemVerilog:** Modules are defined using the `module` keyword.

<i>Concept</i>	<i>SystemVerilog</i>	<i>Chisel</i>
Boolean Signal	<code>logic</code>	<code>Bool</code>
Unsigned Integer	<code>logic [N-1:0]</code>	<code>UInt(N.W)</code>
Signed Integer	<code>signed logic [N-1:0]</code>	<code>SInt(N.W)</code>
Array	<code>logic [N-1:0] array [M-1:0]</code>	<code>Vec(M, UInt(N.W))</code>
Structure	<code>struct {...}</code>	<code>class ... extends Bundle {...}</code>

Table A.2: Directly Mapped Data Types

- ◇ **Chisel:** Modules are classes extending `Module`.
- *Ports:*
 - ◇ **SystemVerilog:** Ports are declared in the module header.
 - ◇ **Chisel:** Ports are defined within an `IO` bundle.
- *Instantiation:*
 - ◇ **SystemVerilog:** Instantiate modules by name and connect ports.
 - ◇ **Chisel:** Instantiate modules by creating new instances and connecting `io` fields.

A.3.8 Assignments

- *Continuous Assignment:*
 - ◇ **SystemVerilog:** `assign out = a & b;`
 - ◇ **Chisel:** `out := a & b`
- *Procedural Assignment:*
 - ◇ **SystemVerilog:** Inside `always` blocks using `=` or `<=`
 - ◇ **Chisel:** Use `:=` for signal assignment.

A.3.9 Control Structures

- *Conditional Statements:*
 - ◇ **SystemVerilog:** `if...else`
 - ◇ **Chisel:** `when, .elsewhen, .otherwise`
- *Case Statements:*
 - ◇ **SystemVerilog:** `case...endcase`

◇ **Chisel:** `switch, is`

A.3.10 Loops

- *For Loops:*

◇ **SystemVerilog:** `for (int i = 0; i < N; i++) begin ... end`

◇ **Chisel:** `for (i <- 0 until N) { ... }`

- *Generate Loops:*

◇ **SystemVerilog:** `generate...endgenerate`

◇ **Chisel:** Use Scala loops during elaboration to generate hardware.

A.3.11 Memory and Arrays

- *Memories:*

◇ **SystemVerilog:** `logic [N-1:0] mem [0:M-1];`

◇ **Chisel:** `Mem(M, UInt(N.W))`

A.3.12 Features Unique to Chisel

Chisel offers several features not available in SystemVerilog.

A.3.13 Parameterization and Generics

Chisel allows advanced parameterization using Scala's type system and functional programming features (Chisel Contributors, 2023a).

- *Parameterized Modules:* Modules can be parameterized with types and values.
Example: `class MyModule(width: Int) extends Module {...}`
- *Higher-Order Functions:* Functions that take other functions as parameters.
- *Functional Programming Constructs:* Map, reduce, and other functional methods.

A.3.14 Object-Oriented Programming

Chisel leverages Scala's object-oriented features.

- *Inheritance:* Modules and Bundles can inherit from base classes.
- *Traits:* Reusable collections of fields and methods.

A.3.15 Type Safety and Strong Typing

Chisel enforces strict type checking, reducing errors due to mismatched widths or types.

A.3.16 Hardware Generation

Chisel allows hardware generation using Scala code.

- *Dynamic Hardware Generation*: Create variable hardware structures based on parameters or conditions.

A.3.17 Testing with ChiselTest

Chisel provides a testing framework integrated with ScalaTest, enabling sophisticated testing methodologies.

A.3.18 Features Unique to SystemVerilog

SystemVerilog includes features not present in Chisel, particularly in verification.

A.3.19 Verification Constructs

SystemVerilog provides extensive verification features (Foster, 2003).

- *Assertions*: Immediate and concurrent assertions for design verification.
- *Coverage*: Functional and code coverage constructs.
- *Randomization*: Random stimulus generation with constraints.

A.3.20 Interfaces and Modports

SystemVerilog interfaces simplify module connections and support complex bus protocols (Guyer, 2005).

A.3.21 Object-Oriented Verification

SystemVerilog supports classes, inheritance, and polymorphism in testbenches.

A.3.22 Time Control and Simulation

SystemVerilog allows precise control over simulation time and events.

A.3.23 Explicit Event Control

SystemVerilog provides constructs like `#delay`, `@(posedge clk)`, which are not directly available in Chisel.

A.3.24 Implementing Designs: A Cross-Language Guide

This section provides guidance for designers transitioning between SystemVerilog and Chisel.

A.3.25 From SystemVerilog to Chisel

- *Modules*: Replace `module` with classes extending `Module`.
- *Ports*: Define an IO bundle instead of module port lists.
- *Signals*: Use `val` and `Wire`, `Reg` instead of `wire`, `reg`.
- *Assignments*: Use `:=` for signal assignments.
- *Control Flow*: Replace `if...else` with `when...elsewhen...otherwise`.
- *Generate Blocks*: Use Scala loops and conditionals for hardware generation.

A.3.26 From Chisel to SystemVerilog

- *Classes and Modules*: Map Chisel modules to `module` definitions.
- *Bundles*: Translate `Bundle` classes to `struct` or module port groups.
- *Functional Constructs*: Implement Scala functional constructs using procedural code or generate statements.
- *Testing*: Replace `ChiselTest` with SystemVerilog testbenches and verification constructs.

A.3.27 Common Pitfalls and Tips

- *Type Differences*: Be mindful of type widths and signedness.
- *Synchronous vs. Combinational Logic*: Ensure that registers and combinational logic are correctly implemented.
- *Initialization*: SystemVerilog allows initial blocks, while Chisel uses `RegInit`.
- *Simulation Time*: Chisel abstracts away simulation time; SystemVerilog allows explicit time control.

Appendix B

Physical Implementations of Logic Gates

B.1 Physical Implementation of NOT Circuit

B.1.1 CMOS switches

A logic gates consists in a network of interconnected switches implemented using the two type of MOS transistors: p-MOS and n-MOS. How behaves the two type of transistors in specific configurations is presented in Figure B.1.

A switch connected to V_{DD} is implemented using a p-MOS transistor. It is represented in Figure B.1a *off* (generating z , which means *Hi-Z*: no signal, neither 0, nor 1) and in Figure B.1b it is represented *on* (generating 1 logic, or *truth*).

A switch connected to *ground* is implemented using a n-MOS transistor. It is represented in Figure B.1c *off* (generating z , which means *Hi-Z*: no signal, neither 0, nor 1) and in Figure B.1e it is represented *on* (generating 0 logic, or *false*).

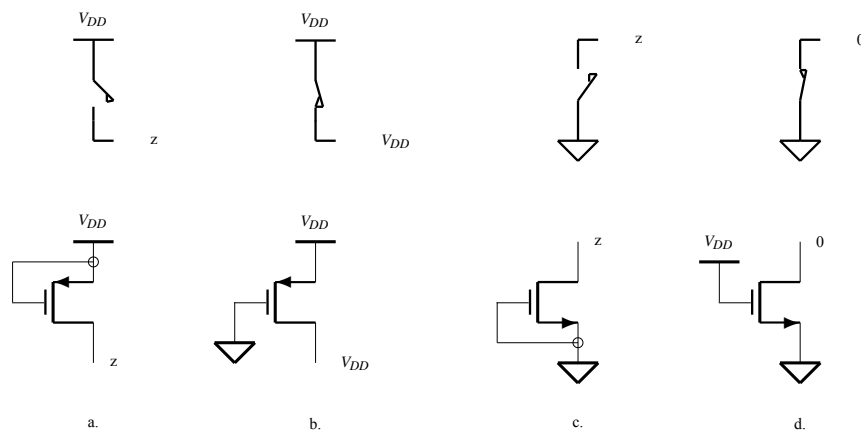


Figure B.1: **Basic switches.** **a.** Open switch connected to V_{DD} . **b.** Closed switch connected to V_{DD} . **c.** Open switch connected to *ground*. **d.** Closed switch connected to *ground*.

A MOS transistor works very well as an *on-off* switch connecting its drain to a certain potential. A

p-MOS transistor can be used to connect its drain to a high potential when its gates is connected to ground, and an n-MOS transistor can connect its drain to ground if its gates is connected to a high potential. This complementary behavior is used to build the elementary logic circuits.

In Figure B.2 is presented the *switch-resistor-capacitor model* (SRC). If $V_{GS} < V_T$ then the transistor is **off**, if $V_{GS} \geq V_T$ then the transistor is **on**. In both cases the input of the transistor behaves like a capacitor, the gate-source capacitor C_{GS} .

When the transistor is on its drain-source resistance is:

$$R_{ON} = R_n \frac{L}{W}$$

where: L is the channel length, W is the channel width, and R_n is the resistance per square. The length L is a constant characterizing a certain technology. For example, if $L = 0.13\mu m$ this means it is about a $0.13\mu m$ process.

The input capacitor has the value:

$$C_{GS} = \frac{\epsilon_{OX} LW}{d}.$$

The value:

$$C_{OX} = \frac{\epsilon_{OX}}{d}$$

where: $\epsilon_{OX} \approx 3.9\epsilon_0$ is the permittivity of the silicon dioxide, is the gate-to-channel capacitance per unit area of the MOSFET gate.

In this conditions the gate input current is:

$$i_G = C_{GS} \frac{dv_{GS}}{dt}$$

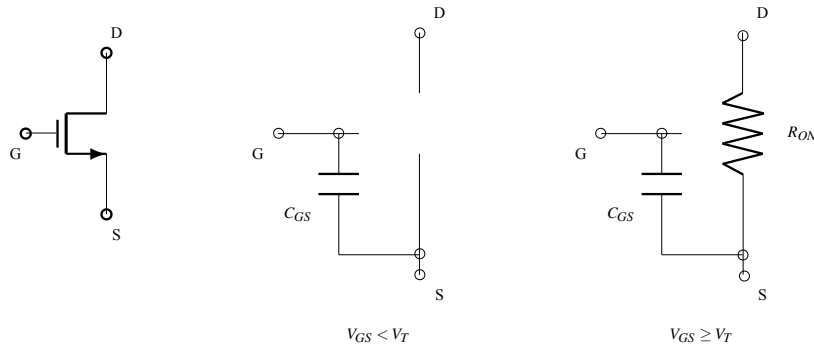


Figure B.2: **The MOSFET switch.** The *switch-resistor-capacitor model* consists in the two states: OF ($V_{GS} < V_T$), and ON ($V_{GS} \geq V_T$). In both states the input is defined by the capacitor C_{GS} .

Example B.1 For an AND gate with low strength, with $W = 1.8\mu m$, in $0.13\mu m$ technology, supposing $C_{OX} = 4fF/\mu m^2$, results the input capacitance:

$$C_{GS} = 4 \times 0.13 \times 1.8fF = 0.936fF$$

Assuming $R_n = 5K\Omega$, results for the same gate:

$$R_{ON} = 5 \times \frac{0.13}{1.8} K\Omega = 361\Omega$$

◇

B.1.2 The Inverter

B.1.2.1 The static behavior

The smallest and simplest logic circuit – the inverter – can be built using a pair of complementary transistors, connecting together the two gates as input and the two drains as output, while the n-MOS source is connected to ground (interpreted as logic 0) and the p-MOS source to V_{DD} (interpreted as logic 1). Results the circuit represented in Figure B.3.

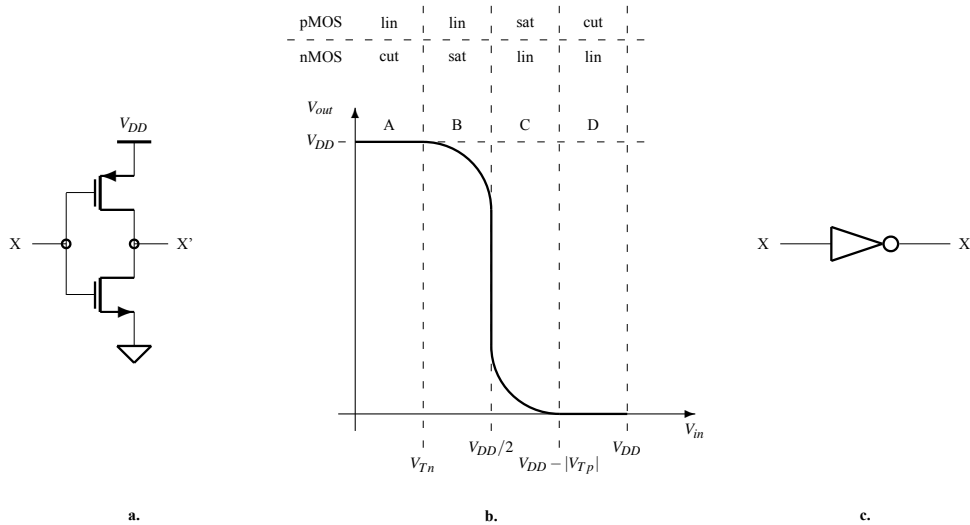


Figure B.3: **Building an inverter.** **a.** The inverter circuit. **b.** The logic symbol for the inverter circuit.

The behavior of the inverter consist in combining the behaviors of the two switches previously defined. For $in = 0$ pMOS is *on* and nMOS is *off* the output generating V_{DD} which means 1. For $in = 1$ pMOS is *off* and nMOS is *on* the output generating 0.

The static behavior of the inverter (or NOT) circuit can be easy explained starting from the switches described in Figure B.1. Connecting together a switch generating z with a switch generating 1 or 0, the connection point will generate 0 or 1.

B.1.2.2 Dynamic behavior

The propagation time of an inverter can be analyzed using the two serially connected inverters represented in Figure B.4. The delay of the first inverter is generated by its capacitive load, C_L , composed by:

- its parasitic drain/bulk capacitance, C_{DB} , is the intrinsic output capacitance of the first inverter

- wiring capacitance, C_{wire} , which depends on the length of the wire (of width W_w and of length L_w) connected between the two invertors:

$$C_{wire} = C_{thickox} W_w L_w$$

- next stage input capacitance, C_G , approximated by summing the gate capacitance for pMOS and nMOS transistors:

$$C_G = C_{Gp} + C_{Gn} = C_{ox}(W_p L_p + W_n L_n)$$

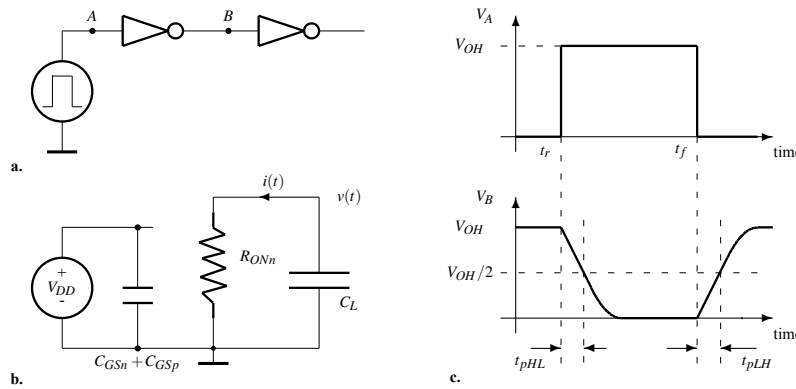


Figure B.4: The propagation time.

The total load capacitance

$$C_L = C_{DB} + C_{wire} + C_G$$

is sometimes dominated by C_{wire} . For short connections C_G dominates, while for big *fan-out* both, C_{wire} and C_G must be considered.

The signal V_A is used to measure the propagation time of the first NOT in Figure B.4a. It is generated by an ideal pulse generator with output impedance 0. Thus, the rising time and the falling time of this signal are considered 0 (the input capacitance of the NOT circuit is charged or discharged in no time).

The two delay times (see Figure B.4c) associated to an invertor (to a gate in the general case) are defined as follows:

- t_{pLH} : the time interval between the moment the input switches in 0 and the output reaches $V_{OH}/2$ coming from 0
- t_{pHL} : the time interval between the moment the input switches in 1 and the output reaches $V_{OH}/2$ coming from V_{OH}

Let us consider the transition of V_A from 0 to V_{OH} at t_r (rise edge). Before transition, at t_r^- , C_L is fully charged and $V_B = V_{OH}$. In Figure B.4b is represented the equivalent circuit at t_r^+ , when pMOS is off and nMOS is on. In this moment starts the process of discharging the capacitance C_L at the constant current

$$I_{Dn(sat)} = \frac{1}{2} \mu_n C_{ox} \frac{W_n}{L_n} (V_{OH} - V_{Tn})^2$$

In Figure B.5, at t_r^- the transistor is cut, $I_{Dn} = 0$. At t_r^+ the nMOS transistor switch in saturation and becomes an ideal *constant current generator* which starts to discharge C_L linearly at the constant current $I_{Dn(sat)}$. The process continue until $V_{OUT} = V_{OH}$, according to the definition of t_{pHL} .

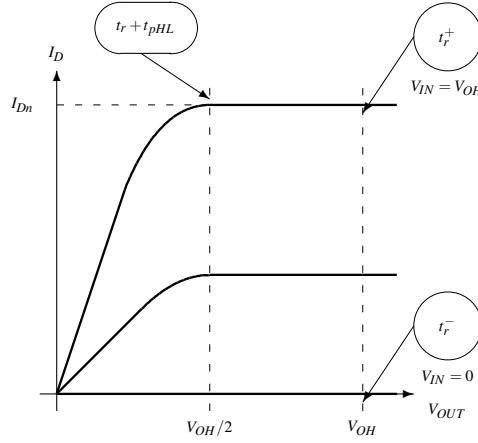


Figure B.5: **The output characteristic of the nMOS transistor.**

In order to compute t_{pHL} we take into consideration the constant value of the discharging current which provide a linear variation of v_{OUT} .

$$\frac{dv_{out}}{dt} = \frac{d}{dt}\left(\frac{q_L}{C_L}\right) = \frac{-I_{Dn(sat)}}{C_L}$$

$$\frac{dv_{out}}{dt} = \frac{\frac{V_{OH}}{2} - V_{OH}}{t_{pHL}}$$

We solve the equations for t_{pHL} :

$$t_{pHL} = C_L \frac{1}{\mu_n C_{ox} \frac{W_n}{L_n} (V_{OH} - V_{Tn})} \frac{V_{OH}}{V_{OH} - V_{Tn}}$$

Because:

$$R_{ONn} = \frac{1}{\mu_n C_{ox} \frac{W_n}{L_n} (V_{OH} - V_{Tn})}$$

results:

$$t_{pHL} = C_L R_{ONn} \frac{1}{1 - \frac{V_{Tn}}{V_{OH}}} = k_n R_{ONn} C_L = k_n \tau_{nL}$$

where:

- τ_{nL} is the *constant time* associated to the H-L transition
- k_n is a constant associated to the technology we use; it goes down when V_{OH} increases or V_T decreases

The speed of a gate depends by its dimension and by the capacitive load it drives. For a big W the value of R_{ON} is small charging or discharging C_L faster.

For t_{pLH} the approach is similar. Results: $t_{pLH} = k_p \tau_{pL}$.

By definition the propagation time associated to a circuit is:

$$t_p = (t_{pLH} + t_{pHL})/2$$

its value being dominated by the value of C_L and the size (width) of the two transistors, W_n and W_p .

B.2 Physical Implementation of NAND & NOR Gates

The 2-input AND circuit, $a \cdot b$, works like a “gate” opened by the signal a for the signal b . Indeed, the gate is “open” for b only if $a = 1$. This is the reason for which the AND circuit was baptised **gate**. Then, the use imposed this alias as the generic name for any logic circuit. Thus, AND, OR, XOR, NAND, ... are all called *gates*.

B.2.1 The static behavior of gates

For 2-input NAND and 2-input NOR gates the same principle will be applied, interconnecting 2 pairs of complementary transistors to obtain the needed behaviors.

There are two kind of interconnecting rules for the same type of transistors, p-MOS or n-MOS. They can be interconnected serially or parallel.

A serial connection will establish an *on* configuration only if both transistors of the same type are *on*, and the connection is *off* if at least one transistor is *off*.

A parallel connection will establish an *on* configuration if at least one is *on*, and the connection is *off* only if both are *off*.

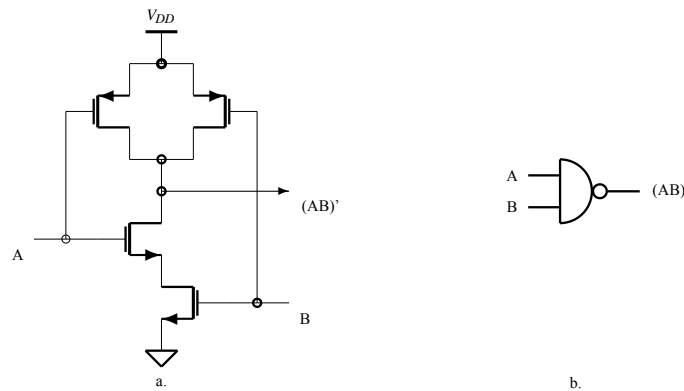


Figure B.6: **The NAND gate.** **a.** The internal structure of a NAND gate: the output is 1 when at least one input is 0. **b.** The logic symbol for NAND.

Applying the previous rules result the circuits presented in Figures B.6 and B.7.

For the NAND gate the output is 0 if both n-MOS transistors are *on*, and the output is one when at least on p-MOS transistor is *on*. Indeed, if $A = B = 1$ both n transistors are *on* and both p transistors are

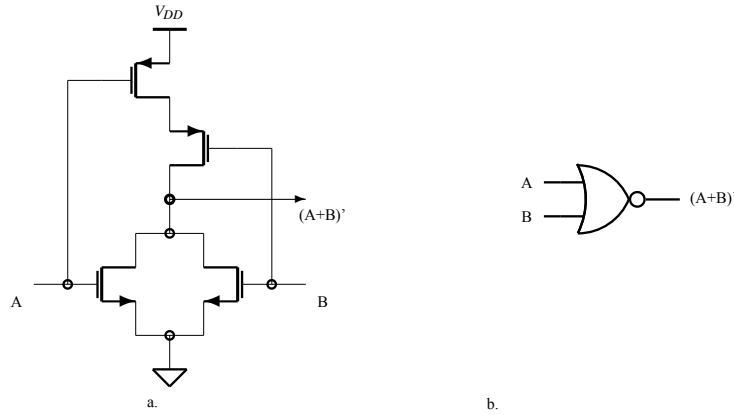


Figure B.7: **The NOR gate.** **a.** The internal structure of a NOR gate: the output is 1 only when both inputs are 0. **b.** The logic symbol for NOR.

off. The output corresponds with the definition, it is 0. If $A = 0$ or $B = 0$ the output is 1, because at least one p transistor is *on* and at least one n transistor is *off*.

A similar explanation works for the NOR gate. The main idea is to design a gate so as to avoid the simultaneous connection of V_{DD} and ground potential to the output of the gate.

For designing an AND or an OR gate we will use an additional NOT connected to the output of an AND or an OR gate. The area will be a little bigger (maybe!), but the strength of the circuit will be increased because the NOT circuit works as a buffer improving the time performance of the non-inverting gate.

The propagation time for the 2-input main gates is computed in a similar way as the propagation for NOT circuit is computed. The only differences are due to the fact that sometimes R_{ON} must be substituted with $2 \times R_{ON}$.

B.2.2 Propagation time

B.2.2.1 Propagation time for NAND gate

Propagation time becomes, in the worst case when only one input switches:

$$t_{HL} = k_n(2R_{ONn})C_L$$

$$t_{LH} = k_p(R_{ONp})C_L$$

because the capacitor C_L is charged through one pMOS transistor and is discharged through two, serially connected, nMOS transistors.

B.2.2.2 Propagation time for NOR gate

Propagation time becomes, in the worst case when only one input switches:

$$t_{HL} = k_n(R_{ONn})C_L$$

$$t_{LH} = k_p(2R_{ONp})C_L$$

because the capacitor C_L is charged through two, serially connected, pMOS transistors and is discharged through one nMOS transistor.

It is obvious that we must prefer, when is is possible, the use of NAND gates instead of NOR gates, because, for the same area, $R_{ONp} > R_{ONn}$.

Appendix C

Simulations

C.1 Combo Simulations

C.1.1 ALU Simulation

The simplest behavioral design of an 8-function ALU is listed in the following module:

```
/******  
File name:      alu.sv  
Circuit name:   arithmetic and logic unit  
Description:    the circuit selects, using the selection code 'func', one  
                of the 8 functions  
******/  
module ALU(input    logic      crIn          ,  
           input    logic [2:0]  func        ,  
           input    logic [31:0] left, right  ,  
           output   logic        crOut       ,  
           output   logic [31:0] out         );  
  
    always_comb case(func)  
        3'b000: {crOut, out} = left + right + crIn;      //add  
        3'b001: {crOut, out} = left - right - crIn;      //sub  
        3'b010: {crOut, out} = {1'b0, left & right};     //and  
        3'b011: {crOut, out} = {1'b0, left | right};     //or  
        3'b100: {crOut, out} = {1'b0, left ^ right};     //xor  
        3'b101: {crOut, out} = {1'b0, ~left};            //not  
        3'b110: {crOut, out} = {1'b0, left};             //left  
        3'b111: {crOut, out} = {1'b0, left >> 1};       //shr  
        default {crOut, out} = 33'b0 - 1'b1;  
    endcase  
endmodule
```

The elaborated design provided by the Vivado tool based on the previous System Verilog is represented in Figure C.1.

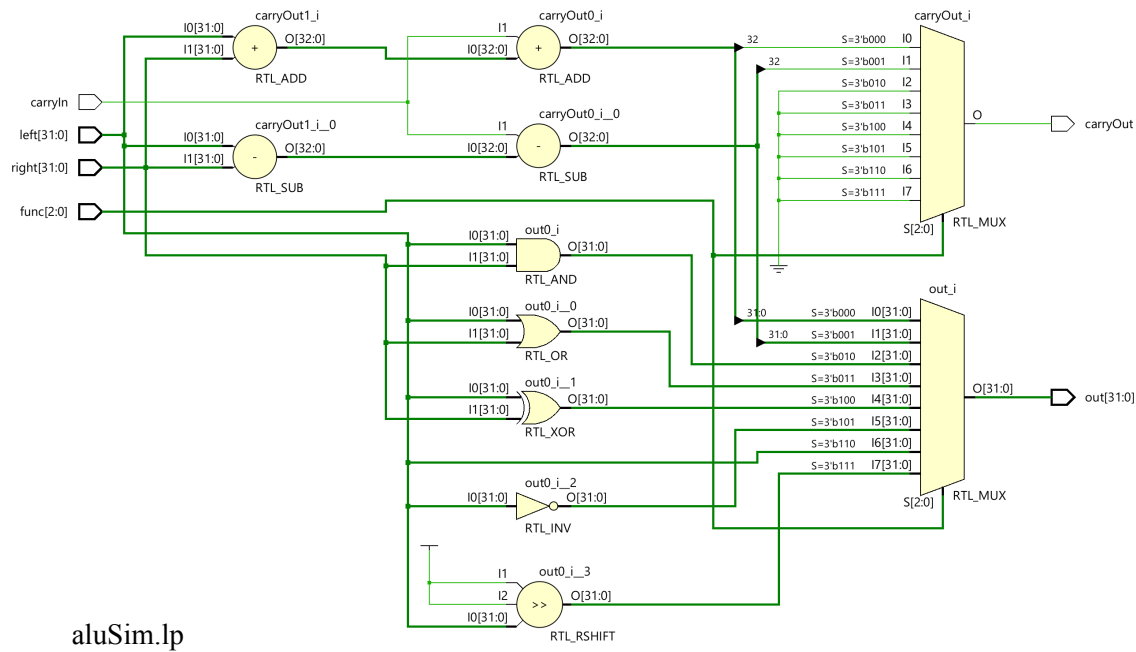


Figure C.1:

C.2 Memory Simulations

C.2.1 Pipelined Three Number Adder

```

/*****
File name:      pipelinedThreeAdder.sv
Circuit name:   pipelined Three Input Adder
Description:    The module 'threeAdder' has 3 4-bit inputs and one 4-bit
                output. The circuit adds modulo 16 three numbers;
                do not provide carry output
*****/
module pipelinedThreeAdder( output logic [3:0] out      ,
                           input  logic [3:0] in0      ,
                           input  logic [3:0] in1      ,
                           input  logic [3:0] in2      ,
                           input  logic          clock  );

    logic [3:0] syncReg ;
    logic [3:0] pipeReg ;
    logic [3:0] sum      ;

    adder    inAdder( .out(sum      ),
                     .in0(in0      ),
                     .in1(in1      ) ),
    outAdder( .out(out      ) );

```

```

        .in0(syncReg),
        .in1(pipeReg));
    always_ff @(posedge clock) begin
        syncReg <= in2 ;
        pipeReg <= sum ;
    end
endmodule

```

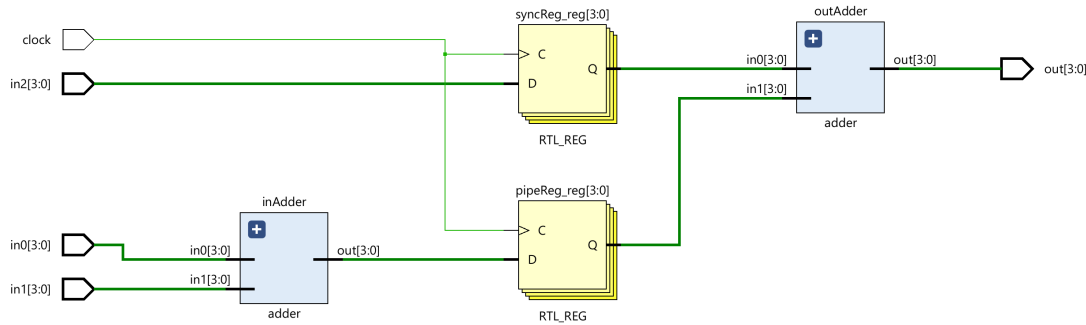


Figure C.2:

C.2.2 Read-During-Write Register File

In the design of the pipeline processor with forwarding mechanism, a register file that provides the currently written value at the output proved useful. The register file version that provides *read-during-write* facility requires an additional structure that performs comparison and multiplexing operations (see Figure C.3 obtained following the project described in the following module).

```

/*****
File name:      fastRegFile.sv
Circuit name:   Register File
Description:    Three-ported, 32 32-bit words implemented with a
                synchronous RAM
*****/
module fastRegFile(
    output logic [31:0] leftOut,
    output logic [31:0] rightOut,
    input logic [31:0] in,
    input logic [4:0] leftAddr,
    input logic [4:0] rightAddr,
    input logic [4:0] destAddr,
    input logic we,
    input logic clock);
    logic [31:0] rf[0:31];

```

```

always_ff @(posedge clock) if (we) rf[destAddr] <= in    ;

assign leftOut  =
    ((destAddr == leftAddr) && we) ? in : rf[leftAddr]  ;
assign rightOut =
    ((destAddr == rightAddr) && we) ? in : rf[rightAddr];
endmodule

```

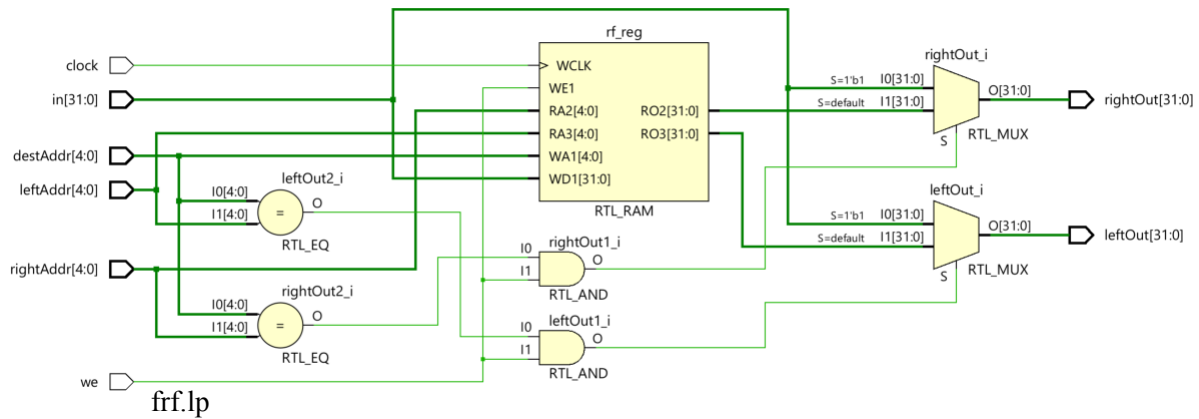


Figure C.3:

Appendix D

FIFO memory

The FIFO memory, or the *queue memory* is used to interconnect subsystems working logical, or both logical and electrical, asynchronously.x

Definition D.1 *FIFO memory implements a data structure which consists in a string of maximum 2^m n -bit recordings accessed for write and read, at its two ends. **Full** and **empty** signals are provided indicating the write operation or the read operation are not allowed.* $\diamond$

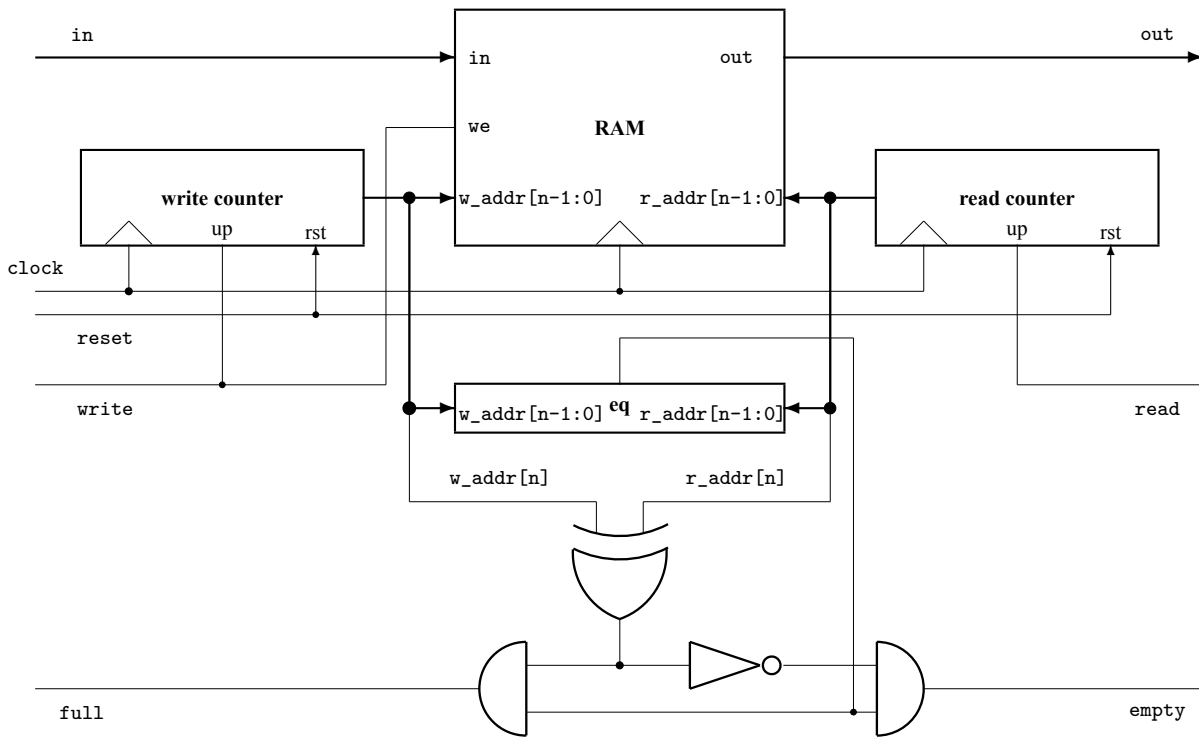
A FIFO is considered synchronous if both *read* and *write* signals are synchronized with the same clock signal. If the two commands, *read* and *write*, are synchronized with different clock signals, then the FIFO memory is called asynchronous.

In Figure D.1 is presented a solution for the synchronous version, where:

- **RAM** is a 2^n m -bit words two-port asynchronous random access memory, one port for write to the address w_addr and another for read from the address r_addr
- **write counter** is an $(n + 1)$ -bit resetable counter incremented each time a write is executed; its output is $w_addr[n:0]$, initially it is reset
- **read counter** is an $(n + 1)$ -bit resetable counter incremented each time a read is executed; its output is $r_addr[n:0]$, initially it is reset
- **eq** is a comparator activating its output when the least significant n bits of the two counters are identical.

FIFO works like a circular memory addressed by two pointers ($w_addr[n-1:0]$ and $r_addr[n-1:0]$) running on the same direction. If the write pointer *after* a write operation becomes equal with the read pointer, then the memory is full and the **full** signal is 1. If the read pointer *after* a read operation becomes equal with the write pointer, then the memory is empty and the **empty** signal is 1. The $n + 1$ -th bit in each counter is used to differentiate between empty and full when $w_addr[n-1:0]$ and $r_addr[n-1:0]$ are the same. If $w_addr[n]$ and $r_addr[n]$ are different, then $w_addr[n-1:0] = r_addr[n-1:0]$ means full, else it means empty.

The circuit used to compare the two addresses is a combinational one. Therefore, its output has a hazardous behavior which affects the outputs **full** and **empty**. These two outputs must be used carefully in designing the system which includes this FIFO memory. The problem can be managed because the



f66.lp

Figure D.1: **FIFO memory.** Two pointers, evolving in the same direction, and a two-port RAM implement a LIFO (queue) memory. The limit flags are computed combinational from the addresses used to write and to read the memory.

system works in the same clock domain (*clock* is the same for both ends of FIFO and for the entire system). We call this kind of FIFO *synchronous FIFO*.

VeriSim D.1 A Verilog synthesisable description of a synchronous FIFO follows:

```

/*****
File name:      simple_fifo.v
Circuit name:   Synchronous First-In First-Out memory
Description:    behavioral description of a synchronous FIFO
*****/
module simple_fifo(output [31:0] out,
                  output empty,
                  output full,
                  input [31:0] in,
                  input write,
                  input read,
                  input reset)

```

```

        input          clock    );
    wire    [9:0]      write_addr, read_addr;

    counter write_counter( .out          (write_addr ),
                           .reset        (reset      ),
                           .count_up     (write       ),
                           .clock        (clock      )),
        read_counter( .out          (read_addr  ),
                      .reset        (reset      ),
                      .count_up     (read        ),
                      .clock        (clock      ));

    dual_ram memory( .out          (out          ),
                    .in           (in           ),
                    .read_addr     (read_addr[8:0] ),
                    .write_addr    (write_addr[8:0]),
                    .we            (write       ),
                    .clock         (clock       ));

    assign eq         = read_addr[8:0] == write_addr[8:0] ,
           phase      = ~(read_addr[9] == write_addr[9]) ,
           empty      = eq & phase ,
           full       = eq & ~phase ;
endmodule

```

```

/*****
File name:      counter.v
Circuit name:   Counter
Description:     behavioral description of the counters used to
                  implement the asynchronous FIFO
*****/
module counter(output reg [9:0] out ,
               input  reset ,
               input  count_up ,
               input  clock );

    always @(posedge clock) if (reset) out <= 0;
                           else if (count_up) out <= out + 1;
endmodule

```

```

/*****
File name:      dual_ram.v
Circuit name:   Dual-Port RAM
Description:    behavioral description of the dual-port RAM used
                to implement the synchronous FIFO
*****/
module dual_ram(    output  [31:0]  out      ,
                   input   [31:0]  in       ,
                   input   [8:0]   read_addr ,
                   input   [8:0]   write_addr,
                   input           we       ,
                   input           clock    );
    reg    [63:0]  mem[511:0];
    assign    out = mem[read_addr] ;
    always @(posedge clock) if (we) mem[write_addr] <= in ;
endmodule

```

◇

An *asynchronous FIFO* uses two independent clocks, one for *write counter* and another for *read counter*. This type of FIFO is used to interconnect subsystems working in different clock domains. The previously described circuit is unable to work as an asynchronous FIFO. The signals *empty* and *full* are meaningless, being generated in two clock domains. Indeed, *write counter* and *read counter* are triggered by different clocks generating the signal *eq* with hazardous transitions related to two different clocks: *write clock* and *read clock*. This signal can not be used neither in the system working with *write clock* nor in the system working with *read clock*. *Read clock* is unable to avoid the hazard generated by *write clock*, and *write clock* is unable to avoid the hazard generated by *read clock*. Special tricks must be used.

Appendix E

Complete Code Listing

E.1	SystemVerilog	500
E.1.1	Waveform Generator	500
E.1.2	Decoder	500
E.1.3	Multiplexer	501
E.1.4	Demultiplexer	502
E.1.5	Seven Segment Display	502
E.1.6	Adders	503
E.2	Chisel	503
E.2.1	Waveform Generator	503
E.2.2	Decoder	504
E.2.3	Multiplexer	505
E.2.4	Demultiplexer	506
E.2.5	Seven Segment Display	508
E.2.6	Mux Version	509
E.2.7	Adders	512
E.2.8	Multi-Function Arithmetic Units	513
E.2.9	ALUs	517

E.1 SystemVerilog

E.1.1 Waveform Generator

```

1  // Licensed under CERN-OHL-S v2. See https://cern.ch/cern-ohl for
   details.
2  /*****
3  File name: waveFormGenerator.sv
4  Circuit name: no circuit, only wave formes
5  Description: two waveforms are generated, a "random" one and
6               a periodical one: a clock signal
7  *****/
8  module waveFormGenerator();
9      logic randomWave;
10     logic clock      ;
11     initial begin          randomWave = 0  ;
12                             #2 randomWave = 1  ;
13                             #6 randomWave = 0  ;
14                             #4 randomWave = 1  ;
15                             #8 randomWave = 0  ;
16                             #5 $stop          ;
17     end
18     initial begin          clock = 0          ;
19                             forever #2 clock = ~clock  ;
20     end
21 endmodule

```

Listing E.1: Waveform Generator Code (Hand Written SystemVerilog)

E.1.2 Decoder

E.1.2.1 Parameterized Decoder

```

1  // Licensed under CERN-OHL-S v2. See https://cern.ch/cern-ohl for
   details.
2  /*
   *****/
3  File name:      dcdParameterized.sv
4  Circuit name:   parameterized decoder
5  Description:
6  *****/
7
8  module Decoder #(
9      parameter INPUT_WIDTH = 4,           // Number of input bits
10     parameter OUTPUT_WIDTH = (1 << INPUT_WIDTH) // Number of output
        bits (2^INPUT_WIDTH)
11 ) (

```

```

12     input  logic [INPUT_WIDTH-1:0] in,           // Binary input
13     output logic [OUTPUT_WIDTH-1:0] out          // Decoded outputs
14 );
15
16     always_comb begin
17         out = 1'b1 << in; // Shift the bit based on the input value
18     end
19
20 endmodule

```

Listing E.2: Parameterized Decoder (SystemVerilog: dcdParameterized.sv)

E.1.2.2 4-to-16 bit Behavioral Decoder

```

1 // Licensed under CERN-OHL-S v2. See https://cern.ch/cern-ohl for
  details.
2 /*
   *****
3 File name:      dcd.sv
4 Circuit name:   4-input decoder
5 Description:
6 *****
   */
7
8 module dcd( output logic [15:0]    dcdOut  ,
9            input  logic [3:0]     dcdIn   );
10
11     assign dcdOut = 1 << dcdIn ;
12 endmodule

```

Listing E.3: 4-to-16 Behavioral Decoder (SystemVerilog:dcd.sv)

E.1.3 Multiplexer

E.1.3.1 16-to-1 Single Bit Multiplexer

```

1 // Licensed under CERN-OHL-S v2. See https://cern.ch/cern-ohl for
  details.
2 /*
   *****
3 File name:      mux.sv
4 Circuit name:   4-input multiplexer
5 Description:
6 *****
   */
7
8 module mux( output logic          muxOut  ,
9            input  logic [15:0]    muxIn   ,
10            input  logic [3:0]     muxSel  );
11

```

```

12     assign muxOut = muxIn[muxSel] ;
13 endmodule

```

Listing E.4: 16-to-1 single bit Multiplexer (SystemVerilog: mux.sv)

E.1.4 Demultiplexer

E.1.4.1 1-to-16 bit Demultiplexer

```

1  // Licensed under CERN-OHL-S v2. See https://cern.ch/cern-ohl for
    details.
2  /*
    *****
3  File name:      dMux.sv
4  Circuit name:   4-input demultiplexer
5  Description:    Demultiplexer is an enabled decoder
6  *****
    */
7
8  module dMux(output logic [15:0] dMuxOut ,
9             input  logic [3:0] dMuxIn ,
10            input  logic dMuxEnable );
11
12     assign dMuxOut = dMuxEnable << dMuxIn ;
13 endmodule

```

Listing E.5: 1-to-16 bit Demultiplexer (SystemVerilog: dMux.sv)

E.1.5 Seven Segment Display

```

1  // Licensed under CERN-OHL-S v2. See https://cern.ch/cern-ohl for
    details.
2  /*
    *****
3  File name:      SevenSegmentDisplay.sv
4  Circuit name:   Seven-Segment Display
5  Description:
6  *****
    */
7  module sevenSegmDys(output logic [6:0] seg ,
8                    input  logic [3:0] number );
9
10     always_comb begin
11         case(number)
12             4'b0000: seg = 7'b1111110; // 0: a, b, c, d, e, f
13             4'b0001: seg = 7'b0110000; // 1: b, c
14             4'b0010: seg = 7'b1101101; // 2: a, b, g, e, d

```

```

15         4'b0011: seg = 7'b1111001; // 3: a, b, g, c, d
16         4'b0100: seg = 7'b0110011; // 4: f, g, b, c
17         4'b0101: seg = 7'b1011011; // 5: a, f, g, c, d
18         4'b0110: seg = 7'b1011111; // 6: a, f, g, e, d, c
19         4'b0111: seg = 7'b1110000; // 7: a, b, c
20         4'b1000: seg = 7'b1111111; // 8: All segments on
21         4'b1001: seg = 7'b1111011; // 9: a, f, b, g, c, d
22         default: seg = 7'b0000000; // Default: All segments off
23     endcase
24 end
25 endmodule

```

Listing E.6: Seven Segment Display (SystemVerilog: SevenSegmentDisplay.sv)

E.1.6 Adders

E.1.6.1 4-bit Behavioral Adder

```

1  // Licensed under CERN-OHL-S v2. See https://cern.ch/cern-ohl for
   details.
2  /*
   *****
3  File name:      adder.sv
4  Circuit name:   Adder
5  Description:    The module 'adder' has 2 4-bit inputs and one 4-bit
   output
6                  The circuit adds modulo 16; do not use or provide carry
   signal
7  *****
8  */
9  module adder(    output logic [3:0] out ,    // 4-bit output
10                 input  logic [3:0] in0 ,    // 4-bit input
11                 input  logic [3:0] in1 );    // 4-bit input
12
13     assign out = in0 + in1;
14 endmodule

```

Listing E.7: Behavioral 2-input 4-bit Adder (SystemVerilog: adder.sv)

E.2 Chisel

E.2.1 Waveform Generator

```

1  // Licensed under the BSD 3-Clause License.
2  // See https://opensource.org/licenses/BSD-3-Clause for details.
3  package scabook
4

```



```

5 import chisel3._
6 import cirt.stage.ChiselStage
7
8 class WaveFormGenerator extends Module {
9   val io = IO(new Bundle {
10     val randomWave = Output(Bool())
11     val clockWave = Output(Bool())
12   })
13
14   // Random wave pattern based on a sequence of values
15   val randomWaveSeq = VecInit(false.B, true.B, false.B, true.B, false.B
16   )
17   val randomIndex = RegInit(0.U(3.W))
18   io.randomWave := randomWaveSeq(randomIndex)
19   randomIndex := Mux(randomIndex === 4.U, 0.U, randomIndex + 1.U)
20
21   // Clock waveform toggles every 2 cycles
22   val clockToggle = RegInit(false.B)
23   clockToggle := ~clockToggle
24   io.clockWave := clockToggle
25 }
26
27 object WaveFormGenerator extends App {
28   ChiselStage.emitSystemVerilogFile(
29     new WaveFormGenerator,
30     Array("--target-dir", "generated_verilog_annotated")
31     //firtoolOpts = Array("-disable-all-randomization", "-strip-debug-
32     info"),
33   )
34 }

```

Listing E.8: Waveform Generator Code (Chisel: WaveFormGenerator.scala)

E.2.2 Decoder

E.2.2.1 Parameterized Decoder

```

1 // Licensed under the BSD 3-Clause License.
2 // See https://opensource.org/licenses/BSD-3-Clause for details.
3
4 package scabook
5
6 import chisel3._
7 import chisel3.util._
8
9 class Decoder(n: Int) extends Module {
10   val io = IO(new Bundle {
11     val input = Input(UInt(log2Ceil(n).W)) // Input signal, width
12     // depends on n
13     val output = Output(UInt(n.W)) // Output is n bits wide
14   })
15 }

```

```

13   })
14
15   // Default: all outputs are 0
16   io.output := 0.U
17
18   // Decode input to one-hot output
19   io.output := 1.U << io.input
20 }

```

Listing E.9: Parameterized Decoder (Chisel: Decoder.scala)

E.2.2.2 2-to-4 Decoder

```

1  // Licensed under the BSD 3-Clause License.
2  // See https://opensource.org/licenses/BSD-3-Clause for details.
3
4  package scabook
5
6  import chisel3._
7
8  class Decoder2to4 extends Decoder(4) {
9  }

```

Listing E.10: Instantiated 2-to-4 Decoder (Chisel: Decoder2to4.scala)

E.2.3 Multiplexer

E.2.3.1 Parameterized Bus Multiplexer

```

1  // Licensed under the BSD 3-Clause License.
2  // See https://opensource.org/licenses/BSD-3-Clause for details.
3
4  package scabook
5
6  import chisel3._
7  import chisel3.util._
8
9  class Multiplexer(val n: Int, val width: Int) extends Module {
10    require(n > 0, "The number of inputs must be greater than zero.")
11    require(isPow2(n), "The number of inputs must be a power of two.")
12
13    val io = IO(new Bundle {
14      val inputs = Input(Vec(n, UInt(width.W))) // `n` inputs of type `UInt`
15      val select = Input(UInt(log2Ceil(n).W)) // Select line
16      val output = Output(UInt(width.W)) // Output of type `UInt`
17    })
18
19    io.output := io.inputs(io.select) // Select which input

```

```

20 }
21
22 // Companion object to simplify instantiation
23 object Multiplexer {
24   def apply(inputs: Vec[UInt], select: UInt): UInt = {
25     require(inputs.nonEmpty, "Inputs cannot be empty.")
26     val mux = Module(new Multiplexer(inputs.length, inputs(0).getWidth)
27                       )
28     mux.io.inputs := inputs
29     mux.io.select := select
30     mux.io.output
31   }
32 }

```

Listing E.11: Parameterized Bus Width Multiplexer (Chisel: Multiplexer.scala)

E.2.3.2 4-to-1 8-bit Bus Multiplexer

```

1 // Licensed under the BSD 3-Clause License.
2 // See https://opensource.org/licenses/BSD-3-Clause for details.
3
4 package scabook
5
6 import chisel3._
7
8 class Mux4 extends Module {
9   val io = IO(new Bundle {
10     val inputs = Input(Vec(4, UInt(8.W))) // Four 8-bit inputs
11     val select = Input(UInt(2.W))         // 2-bit selector
12     val output = Output(UInt(8.W))        // Single 8-bit output
13   })
14
15   // Use the Multiplexer companion object for simplicity
16   io.output := Multiplexer(io.inputs, io.select)
17 }

```

Listing E.12: 4-to-1 8-bit Bus Multiplexer (Chisel: Mux4.scala)

E.2.4 Demultiplexer

E.2.4.1 Parameterized Bus Demultiplexer

```

1 // Licensed under the BSD 3-Clause License.
2 // See https://opensource.org/licenses/BSD-3-Clause for details.
3
4 package scabook
5
6 import chisel3._
7 import chisel3.util._

```

```

8
9 class Demultiplexer(numOutputs: Int, width: Int) extends Module {
10   require(isPow2(numOutputs), "Number of outputs must be a power of 2")
11
12   val io = IO(new Bundle {
13     val input = Input(UInt(width.W))           // Single `
14     val select = Input(UInt(log2Ceil(numOutputs).W)) // Select
15     val outputs = Output(Vec(numOutputs, UInt(width.W))) // Vec of `
16   })
17
18   // Default all outputs to zero
19   io.outputs.foreach(_ := 0.U)
20
21   // Drive the selected output
22   io.outputs(io.select) := io.input
23 }
24
25 // Companion object for easier instantiation
26 object Demultiplexer {
27   def apply(input: UInt, select: UInt, numOutputs: Int): Vec[UInt] = {
28     require(numOutputs > 0, "Number of outputs must be greater than
29       zero.")
30     require(isPow2(numOutputs), "Number of outputs must be a power of
31       2.")
32     val demux = Module(new Demultiplexer(numOutputs, input.getWidth))
33     demux.io.input := input
34     demux.io.select := select
35     demux.io.outputs
36   }
37 }

```

Listing E.13: Parameterized Bus Demultiplexer (Chisel: Demultiplexer.scala)

E.2.4.2 1-to-16 64-bit Bus Demultiplexer

```

1 // Licensed under the BSD 3-Clause License.
2 // See https://opensource.org/licenses/BSD-3-Clause for details.
3
4 package scabook
5
6 import chisel3._
7
8
9 class DeMux16 extends Module {
10   val io = IO(new Bundle {
11     val input = Input(UInt(64.W)) // 64-bit input bus
12     val select = Input(UInt(4.W)) // 4-bit selector for 16 outputs

```

```

13     val outputs = Output(Vec(16, UInt(64.W))) // 16 output buses, each
        64-bit
14 }
15
16 // Instantiate the 1-to-16 demultiplexer
17 io.outputs := Demultiplexer(
18     input = io.input,
19     select = io.select,
20     numOutputs = 16
21 )
22 }

```

Listing E.14: 1-to-16 64-bit Bus Demultiplexer (Chisel: DeMux16.scala)

E.2.5 Seven Segment Display

E.2.5.1 when Version

```

1 // Licensed under the BSD 3-Clause License.
2 // See https://opensource.org/licenses/BSD-3-Clause for details.
3
4 package scabook
5
6 import chisel3._
7
8 class SevenSegmentDisplay extends Module {
9     val io = IO(new Bundle {
10         val binIn = Input(UInt(4.W)) // 4-bit binary input
11         val segOut = Output(UInt(7.W)) // 7-bit output for seven-segment
            display
12     })
13
14     // Assign names to the binary I/O bits
15     val B0 = WireDefault(io.binIn(0)).suggestName("B0")
16     val B1 = WireDefault(io.binIn(1)).suggestName("B1")
17     val B2 = WireDefault(io.binIn(2)).suggestName("B2")
18     val B3 = WireDefault(io.binIn(3)).suggestName("B3")
19
20     // Assign names to the 7-segment output bits
21     val a = WireDefault(io.segOut(6)).suggestName("a")
22     val b = WireDefault(io.segOut(5)).suggestName("b")
23     val c = WireDefault(io.segOut(4)).suggestName("c")
24     val d = WireDefault(io.segOut(3)).suggestName("d")
25     val e = WireDefault(io.segOut(2)).suggestName("e")
26     val f = WireDefault(io.segOut(1)).suggestName("f")
27     val g = WireDefault(io.segOut(0)).suggestName("g")
28
29     // Output decoding
30     // 0: a, b, c, d, e, f
31     // 1: b, c

```

```

32    // 2: a, b, g, e, d
33    // 3: a, b, g, c, d
34    // 4: f, g, b, c
35    // 5: a, f, g, c, d
36    // 6: a, f, g, e, d, c
37    // 7: a, b, c
38    // 8: All segments on
39    // 9: a, f, b, g, c, d
40    // Default: All segments off
41
42    io.segOut := "b0000000".U(7.W) // Default: All segments off
43
44    when(      io.binIn === 0.U) { io.segOut := "b1111110".U(7.W)
45  } .elsewhen(io.binIn === 1.U) { io.segOut := "b0110000".U(7.W)
46  } .elsewhen(io.binIn === 2.U) { io.segOut := "b1101101".U(7.W)
47  } .elsewhen(io.binIn === 3.U) { io.segOut := "b1111001".U(7.W)
48  } .elsewhen(io.binIn === 4.U) { io.segOut := "b0110011".U(7.W)
49  } .elsewhen(io.binIn === 5.U) { io.segOut := "b1011011".U(7.W)
50  } .elsewhen(io.binIn === 6.U) { io.segOut := "b1011111".U(7.W)
51  } .elsewhen(io.binIn === 7.U) { io.segOut := "b1110000".U(7.W)
52  } .elsewhen(io.binIn === 8.U) { io.segOut := "b1111111".U(7.W)
53  } .elsewhen(io.binIn === 9.U) { io.segOut := "b1111011".U(7.W)
54  }
55  }

```

Listing E.15: Seven Segment Display using Bus and when (Chisel: SevenSegmentDisplay.scala)

E.2.6 Mux Version

```

1  // Licensed under the BSD 3-Clause License.
2  // See https://opensource.org/licenses/BSD-3-Clause for details.
3
4  package scabook
5
6  import chisel3._
7
8  class SevenSegmentDisplayMux extends Module {
9    val io = IO(new Bundle {
10      val binIn = Input(UInt(4.W)) // 4-bit binary input
11      val segOut = Output(UInt(7.W)) // 7-bit output for seven-segment
        display
12    })
13
14    // Assign names to the binary I/O bits
15    val B0 = io.binIn(0).suggestName("B0")
16    val B1 = io.binIn(1).suggestName("B1")
17    val B2 = io.binIn(2).suggestName("B2")
18    val B3 = io.binIn(3).suggestName("B3")
19    val a = io.segOut(6).suggestName("a")
20    val b = io.segOut(5).suggestName("b")

```

```

21  val c = io.segOut(4).suggestName("c")
22  val d = io.segOut(3).suggestName("d")
23  val e = io.segOut(2).suggestName("e")
24  val f = io.segOut(1).suggestName("f")
25  val g = io.segOut(0).suggestName("g")
26
27  // Output decoding
28  // 0: a, b, c, d, e, f
29  // 1: b, c
30  // 2: a, b, g, e, d
31  // 3: a, b, g, c, d
32  // 4: f, g, b, c
33  // 5: a, f, g, c, d
34  // 6: a, f, g, e, d, c
35  // 7: a, b, c
36  // 8: All segments on
37  // 9: a, f, b, g, c, d
38  // Default: All segments off
39
40  io.segOut := Mux(io.binIn === 0.U, "b1111110".U(7.W), // 0: a, b, c,
    d, e, f
41      Mux(io.binIn === 1.U, "b0110000".U(7.W), // 1: b, c
42      Mux(io.binIn === 2.U, "b1101101".U(7.W), // 2: a, b, g,
    e, d
43      Mux(io.binIn === 3.U, "b1111001".U(7.W), // 3: a, b, g,
    c, d
44      Mux(io.binIn === 4.U, "b0110011".U(7.W), // 4: f, g, b,
    c
45      Mux(io.binIn === 5.U, "b1011011".U(7.W), // 5: a, f, g,
    c, d
46      Mux(io.binIn === 6.U, "b1011111".U(7.W), // 6: a, f, g,
    e, d, c
47      Mux(io.binIn === 7.U, "b1110000".U(7.W), // 7: a, b, c
48      Mux(io.binIn === 8.U, "b1111111".U(7.W), // 8: All
    segments
49      Mux(io.binIn === 9.U, "b1111011".U(7.W), // 9: a, f, b,
    g, c, d
50      "b0000000".U(7.W) // Default (all segments off)
51      )))))))
52  }

```

Listing E.16: Seven Segment Display using Bus and Mux (Chisel: SevenSegmentDisplay.scala)

E.2.6.1 Flat Version

```

1  package scabook
2
3  import chisel3._
4  import chisel3.util._
5

```

```

6  class SevenSegmentDisplayFlat extends Module {
7      val io = IO(new Bundle {
8          //In
9          val binIn_B0 = Input(Bool())
10         val binIn_B1 = Input(Bool())
11         val binIn_B2 = Input(Bool())
12         val binIn_B3 = Input(Bool())
13
14         // Out
15         val segOut_a = Output(Bool())
16         val segOut_b = Output(Bool())
17         val segOut_c = Output(Bool())
18         val segOut_d = Output(Bool())
19         val segOut_e = Output(Bool())
20         val segOut_f = Output(Bool())
21         val segOut_g = Output(Bool())
22     })
23
24     // Combine input signals into a UInt for internal logic
25     val binIn = Wire(UInt(4.W))
26     binIn := Cat(io.binIn_B3, io.binIn_B2, io.binIn_B1, io.binIn_B0)
27
28     // Create a UInt for output signals and split it back into individual
        signals
29     val segOut = Wire(UInt(7.W))
30     io.segOut_a := segOut(6)
31     io.segOut_b := segOut(5)
32     io.segOut_c := segOut(4)
33     io.segOut_d := segOut(3)
34     io.segOut_e := segOut(2)
35     io.segOut_f := segOut(1)
36     io.segOut_g := segOut(0)
37
38     // Output decoding
39     // 0: a, b, c, d, e, f
40     // 1: b, c
41     // 2: a, b, g, e, d
42     // 3: a, b, g, c, d
43     // 4: f, g, b, c
44     // 5: a, f, g, c, d
45     // 6: a, f, g, e, d, c
46     // 7: a, b, c
47     // 8: All segments on
48     // 9: a, f, b, g, c, d
49     // Default: All segments off
50
51     // Default: All segments off
52     segOut := "b0000000".U
53
54     // Seven-segment display logic
55     when(      binIn == 0.U) { segOut := "b1111110".U }

```



```

56 .elsewhen(binIn === 1.U) { segOut := "b0110000".U }
57 .elsewhen(binIn === 2.U) { segOut := "b1101101".U }
58 .elsewhen(binIn === 3.U) { segOut := "b1111001".U }
59 .elsewhen(binIn === 4.U) { segOut := "b0110011".U }
60 .elsewhen(binIn === 5.U) { segOut := "b1011011".U }
61 .elsewhen(binIn === 6.U) { segOut := "b1011111".U }
62 .elsewhen(binIn === 7.U) { segOut := "b1110000".U }
63 .elsewhen(binIn === 8.U) { segOut := "b1111111".U }
64 .elsewhen(binIn === 9.U) { segOut := "b1111011".U }
65 }

```

Listing E.17: Seven Segment Display using Named Inputs (Chisel: SevenSegmentDisplayFlat.scala)

E.2.7 Adders

E.2.7.1 Parameterized Abstract Adder

```

1 // Licensed under the BSD 3-Clause License.
2 // See https://opensource.org/licenses/BSD-3-Clause for details.
3
4 package scabook.adders
5
6 import chisel3._
7
8 abstract class Adder(width: Int) extends Module {
9   require(width > 0, "Width must be greater than 0")
10
11   val io = IO(new Bundle {
12     val a = Input(UInt(width.W)) // Input A
13     val b = Input(UInt(width.W)) // Input B
14     val sum = Output(UInt(width.W)) // Output sum
15   })
16 }

```

Listing E.18: Parameterized Abstract Adder (Chisel: Adder.scala)

E.2.7.2 Parameterized Behavioral Adder

```

1 // Licensed under the BSD 3-Clause License.
2 // See https://opensource.org/licenses/BSD-3-Clause for details.
3
4 package scabook.adders
5
6 import chisel3._
7
8 class BehavioralAdder(width: Int) extends Module {
9   val io = IO(new Bundle {
10     val a = Input(UInt(width.W)) // Input A (UInt)
11     val b = Input(UInt(width.W)) // Input B (UInt)

```

```

12     val sum = Output(UInt(width.W)) // Fixed-width sum output
13   })
14
15   // Perform addition and truncate to width
16   io.sum := (io.a + io.b)(width - 1, 0)
17 }
18
19 // Companion object for easier instantiation
20 object BehavioralAdder {
21   def apply(a: UInt, b: UInt, width: Int): UInt = {
22     val adder = Module(new BehavioralAdder(width))
23     adder.io.a := a
24     adder.io.b := b
25     adder.io.sum
26   }
27 }

```

Listing E.19: Parameterized Behavioral Adder (Chisel)

E.2.7.3 4-bit Behavioral Adder

```

1 // Licensed under the BSD 3-Clause License.
2 // See https://opensource.org/licenses/BSD-3-Clause for details.
3
4 package scabook.adders
5
6 import chisel3._
7
8 class BehavioralAdder4 extends Module {
9   val io = IO(new Bundle {
10     val a = Input(UInt(4.W)) // 4-bit unsigned input
11     val b = Input(UInt(4.W)) // 4-bit unsigned input
12     val sum = Output(UInt(4.W)) // 4-bit unsigned output
13   })
14
15   // Directly use the BehavioralAdder module with UInt
16   io.sum := BehavioralAdder(io.a, io.b, 4)
17 }

```

Listing E.20: 4-bit Behavioral Adder (Chisel: BehavioralAdder4.scala)

E.2.8 Multi-Function Arithmetic Units

E.2.8.1 Behavioral Subtractor

```

1 // Licensed under the BSD 3-Clause License.
2 // See https://opensource.org/licenses/BSD-3-Clause for details.
3
4 package scabook.addersubtractors

```

```

5
6 import chisel3._
7
8 class BehavioralAdderSubtractor(width: Int) extends Module {
9   // Define the I/O for the module
10  val io = IO(new Bundle {
11    val a = Input(UInt(width.W))
12    val b = Input(UInt(width.W))
13    val subtract = Input(UInt(1.W)) // 0: add, 1: subtract
14    val result = Output(UInt(width.W))
15  })
16
17  // Compute addition or subtraction
18  val fullResult = Mux(io.subtract.asBool, io.a - io.b, io.a + io.b)
19
20  // Truncate result to the specified width
21  io.result := fullResult(width - 1, 0)
22 }
23
24 // Companion object for easier instantiation
25 object BehavioralAdderSubtractor {
26   def apply(a: UInt, b: UInt, subtract: UInt, width: Int): UInt = {
27     val module = Module(new BehavioralAdderSubtractor(width))
28     module.io.a := a
29     module.io.b := b
30     module.io.subtract := subtract
31     module.io.result
32   }
33 }

```

Listing E.21: Behavioral Adder/Subtractor (Chisel: BehavioralAdderSubtractor.scala)

E.2.8.2 4-bit Behavioral Subtractor

```

1 // Licensed under the BSD 3-Clause License.
2 // See https://opensource.org/licenses/BSD-3-Clause for details.
3
4 package scabook.addersubtractors
5
6 import chisel3._
7
8 class BehavioralAdderSubtractor4 extends Module {
9   val io = IO(new Bundle {
10     val a = Input(UInt(4.W))
11     val b = Input(UInt(4.W))
12     val subtract = Input(UInt(1.W))
13     val sum = Output(UInt(4.W))
14   })
15
16   // Directly use the BehavioralAdder module with UInt

```

```

17   io.sum := BehavioralAdderSubtractor(io.a, io.b, io.subtract, 4)
18 }

```

Listing E.22: 4-bit Behavioral Adder/Subtractor (Chisel: BehavioralAdderSubtractor4.scala)

E.2.8.3 Structural Subtractor

```

1  // Licensed under the BSD 3-Clause License.
2  // See https://opensource.org/licenses/BSD-3-Clause for details.
3
4  package scabook.addersubtractors
5
6  import chisel3._
7
8  class BehavioralAdderSubtractorHW(width: Int) extends Module {
9    // Define the I/O for the module
10   val io = IO(new Bundle {
11     val a = Input(UInt(width.W)) // First operand
12     val b = Input(UInt(width.W)) // Second operand
13     val subtract = Input(UInt(1.W)) // Control signal: 1 for
14     // subtraction, 0 for addition
15     val result = Output(UInt(width.W)) // Result of addition or
16     // subtraction
17   })
18
19   // Two's complement for subtraction
20   val bAdjusted = Mux(io.subtract.asBool, ~io.b + 1.U, io.b)
21   val fullResult = io.a + bAdjusted
22
23   // Truncate result to the specified width
24   io.result := fullResult(width - 1, 0)
25 }
26
27 // Companion object for easier instantiation
28 object BehavioralAdderSubtractorHW {
29   def apply(a: UInt, b: UInt, subtract: UInt, width: Int): UInt = {
30     val module = Module(new BehavioralAdderSubtractorHW(width))
31     module.io.a := a
32     module.io.b := b
33     module.io.subtract := subtract
34     module.io.result
35   }
36 }

```

Listing E.23: Parameterized Two's Complement Adder/Subtractor (Chisel: BehavioralAdderSubtractorHS.scala)

E.2.8.4 Multifunction Parameterized Ripple-Carry Adder/Subtractor

```

1  // Licensed under the BSD 3-Clause License.
2  // See https://opensource.org/licenses/BSD-3-Clause for details.
3
4  package scabook.adders
5
6  import chisel3._
7  import chisel3.util._
8
9  class RippleCarryAdder(width: Int) extends Module {
10     val io = IO(new Bundle {
11         val a = Input(UInt(width.W))    // Input A
12         val b = Input(UInt(width.W))    // Input B
13         val carryIn = Input(UInt(1.W))  // Carry-in
14         val isSigned = Input(UInt(1.W)) // Control signal: 1 for signed,
15         0 for unsigned
16         val sum = Output(UInt(width.W))  // Sum output
17         val carryOut = Output(UInt(1.W)) // Carry-out for unsigned
18         addition
19         val overflow = Output(UInt(1.W)) // Overflow for signed addition
20     })
21
22     val carries = Wire(Vec(width + 1, Bool()))
23     val sumBits = Wire(Vec(width, Bool()))
24
25     carries(0) := io.carryIn // Initial carry-in
26
27     for (i <- 0 until width) {
28         val aBit = io.a(i) // Access individual bits of `a`
29         val bBit = io.b(i) // Access individual bits of `b`
30         val sum = aBit ^ bBit ^ carries(i) // Sum bit
31         val carry = (aBit & bBit) | (aBit & carries(i)) | (bBit & carries(i)) // Carry bit
32         sumBits(i) := sum
33         carries(i + 1) := carry
34     }
35
36     io.sum := sumBits.asUInt // Sum output
37     io.carryOut := carries(width).asUInt // Carry-out for unsigned
38     addition
39     io.overflow := (io.isSigned.asBool && (carries(width - 1) ^ carries(
40         width))).asUInt // Overflow for signed addition
41 }
42
43 object RippleCarryAdder {
44     /**
45      * Creates a RippleCarryAdder module with the specified width.
46      *
47      * @param width The width of the adder (number of bits for inputs and
48      * outputs)
49      * @return A new instance of RippleCarryAdder

```

```

45     */
46     def apply(width: Int): RippleCarryAdder = {
47         require(width > 0, "Width must be greater than 0")
48         Module(new RippleCarryAdder(width))
49     }
50 }

```

Listing E.24: Parameterized Ripple-Carry Adder (Chisel)

E.2.9 ALUs

```

1  // Licensed under the BSD 3-Clause License.
2  // See https://opensource.org/licenses/BSD-3-Clause for details.
3
4  package scabook.ALUs
5
6  import chisel3._
7  import chisel3.util._
8
9  import scabook.addersubtractors.MultifunctionAdderSubtractor64
10
11  // Opcodes
12  // b5: arithmetic/logic
13  // b1b0: 8/16/32/64 bits
14  object ALU64 {
15      object Opcode {
16          // Arithmetic Operations
17          // b4: Unused
18          // b3: Add/Sub
19          // b2: Unsigned/Signed
20          val ADD_U8  = "b000000".U
21          val ADD_U16 = "b000001".U
22          val ADD_U32 = "b000010".U
23          val ADD_U64 = "b000011".U
24          val SUB_U8  = "b001000".U
25          val SUB_U16 = "b001001".U
26          val SUB_U32 = "b001010".U
27          val SUB_U64 = "b001011".U
28          val ADD_S8  = "b000100".U
29          val ADD_S16 = "b000101".U
30          val ADD_S32 = "b000110".U
31          val ADD_S64 = "b000111".U
32          val SUB_S8  = "b001100".U
33          val SUB_S16 = "b001101".U
34          val SUB_S32 = "b001110".U
35          val SUB_S64 = "b001111".U
36
37          // Logical Operations
38          // b4b3b2: operation
39          val AND_U8  = "b100000".U

```

```

40     val AND_U16 = "b100001".U
41     val AND_U32 = "b100010".U
42     val AND_U64 = "b100011".U
43     val OR_U8   = "b100100".U
44     val OR_U16  = "b100101".U
45     val OR_U32  = "b100110".U
46     val OR_U64  = "b100111".U
47     val XOR_U8   = "b101000".U
48     val XOR_U16  = "b101001".U
49     val XOR_U32  = "b101010".U
50     val XOR_U64  = "b101011".U
51     val SLL_U8   = "b101100".U
52     val SLL_U16  = "b101101".U
53     val SLL_U32  = "b101110".U
54     val SLL_U64  = "b101111".U
55     val SRL_U8   = "b110000".U
56     val SRL_U16  = "b110001".U
57     val SRL_U32  = "b110010".U
58     val SRL_U64  = "b110011".U
59     val SRA_U8   = "b110000".U
60     val SRA_U16  = "b110001".U
61     val SRA_U32  = "b110010".U
62     val SRA_U64  = "b110011".U
63   }
64 }
65
66 class ALU64 extends Module {
67   val io = IO(new Bundle {
68     val a = Input(UInt(64.W))
69     val b = Input(UInt(64.W))
70     val result = Output(UInt(64.W))
71     val opcode = Input(UInt(6.W))
72     val carryOutFlag = Output(UInt(1.W)) //Carry and Borrow
73     val overflowFlag = Output(UInt(1.W)) //V
74     val zeroFlag = Output(UInt(1.W)) //Z
75     val negativeFlag = Output(UInt(1.W)) //N
76   })
77
78   val printDebugInfo = false
79
80   // Debug internals of ALU
81   if(printDebugInfo) printf(p"[Inside ALU64] --\n")
82
83   // Decode control signals
84   // b5: arithmetic / logic
85   // b4b3b2: logic ope
86   // b3: if arith: Add/Sub
87   // b2: if arith: Signed/Unsigned
88   // b1b0: 8/16/32/64 bits
89   // Decode control signals
90   val isArithmetic = io.opcode(5) === 0.U // MSB: 0 for arithmetic

```

```

91  val isLogical = io.opcode(5) === 1.U // MSB: 1 for logical operations
92  val isSub = io.opcode(3) === 1.U && isArithmetic // b3: 1 for
    subtraction
93  val isSigned = io.opcode(2) && isArithmetic // b2: 1 for signed
94  val operandSize = io.opcode(1, 0) // b1b0: 00 for 8-bit, 01 for 16-
    bit, 10 for 32-bit, 11 for 64-bit
95
96  // Determine effective width based on operand size
97  val width = WireDefault(64.U)
98  val mask = WireDefault("ffffffffffffffff".U)
99  switch(operandSize) {
100    is("b00".U) { width := 8.U; mask := "h00000000000000ff".U }
101    is("b01".U) { width := 16.U; mask := "h000000000000ffff".U }
102    is("b10".U) { width := 32.U; mask := "h00000000ffffffff".U }
103    is("b11".U) { width := 64.U; mask := "ffffffffffffffff".U }
104  }
105
106  // Mask inputs to the appropriate width
107  val aEffective = io.a & mask
108  val bEffective = io.b & mask
109  val bAdjusted = Mux(isSub, (~bEffective + 1.U), bEffective)
110
111  if(printDebugInfo){
112    printf(p" width = ${width}, isSigned = ${isSigned}, isSub = ${
        isSub}\n")
113    printf(p" mask = 0x${Hexadecimal(mask)}\n")
114    printf(p" a = 0x${Hexadecimal(io.a)}\n")
115    printf(p" b = 0x${Hexadecimal(io.b)}\n")
116    printf(p" aEffective = 0x${Hexadecimal(aEffective)},\n")
117    printf(p" bEffective = 0x${Hexadecimal(bEffective)},\n")
118    printf(p" bAdjusted = 0x${Hexadecimal(bAdjusted)},\n")
119  }
120
121  //*****
122  // Arithmetic operations
123  //*****
124  val adderSubtractor = Module(new MultifunctionAdderSubtractor64)
125
126  // Connect inputs
127  adderSubtractor.io.a := aEffective
128  adderSubtractor.io.b := bEffective
129  adderSubtractor.io.carryIn := 0.U(1.W)
130  adderSubtractor.io.opcode := io.opcode(3, 0)
131
132  // Connect outputs to the ALU64 outputs
133  io.result := adderSubtractor.io.result
134  io.carryOutFlag := adderSubtractor.io.carryOut
135  io.overflowFlag := adderSubtractor.io.overflowFlag
136  io.zeroFlag := adderSubtractor.io.zeroFlag
137  io.negativeFlag := adderSubtractor.io.negativeFlag
138

```



```

139
140 //*****
141 // Logical operations
142 //*****
143 // b4b3b2: logical operation
144 val logicalResult = MuxCase(0.U(64.W), Seq(
145     (io.opcode(4, 2) === "b000".U) -> (aEffective & bEffective),
146                                     // AND
147     (io.opcode(4, 2) === "b001".U) -> (aEffective | bEffective),
148                                     // OR
149     (io.opcode(4, 2) === "b010".U) -> (aEffective ^ bEffective),
150                                     // XOR
151     (io.opcode(4, 2) === "b011".U) -> ((aEffective << (bEffective(5, 0)
152                                     & (width - 1.U))).asUInt & mask), // SLL
153     (io.opcode(4, 2) === "b100".U) -> ((aEffective >> (bEffective(5, 0)
154                                     & (width - 1.U))).asUInt & mask), // SRL
155     (io.opcode(4, 2) === "b101".U) -> ((aEffective.asSInt >> (
156                                     bEffective(5, 0) & (width - 1.U))).asUInt & mask) // SRA
157 ))
158
159 if(printDebugInfo) printf(p" logicalResult = 0x${Hexadecimal(
160     logicalResult)}\n")
161
162 //*****
163 // Assign results to outputs
164 //*****
165 io.result := Mux(isArithmetic, adderSubtractor.io.result,
166     logicalResult)
167 io.carryOutFlag := Mux(isArithmetic, adderSubtractor.io.carryOut, 0.U)
168 io.overflowFlag := Mux(isArithmetic, adderSubtractor.io.overflowFlag,
169     0.U)
170 io.zeroFlag := Mux(isArithmetic, adderSubtractor.io.zeroFlag, 0.U)
171 io.negativeFlag := Mux(isArithmetic, adderSubtractor.io.negativeFlag,
172     0.U)
173
174 }

```

Listing E.25: 64-bit ALU (Chisel: ALU64.scala)

Bibliography

- Amdahl, G. M. (1967). Validity of the single processor approach to achieving large scale computing capabilities. In *Proceedings of the April 18-20, 1967, spring joint computer conference on - AFIPS '67 (Spring)*. New York, page 483–485. ACM.
- Amdahl, G. M., Blaauw, G. A., and Jr., F. P. B. (1964). Architecture of the ibm system/360. *IBM Journal of Research and Development*, 8(2):87–101.
- Antonescu, M. and Malita, M. (2024). Fft on a heterogeneous system with a general-purpose map-scan accelerator. *ROMJIST*, 27(2):123–136.
- Antonescu, M., Malita, M., and Stefan, G. M. (2023). Latency hiding of log-depth scan and reduce networks in heterogeneous embedded systems. In *Proceedings of IEEE 29th International Symposium for Design and Technology in Electronic Packaging*, pages 81–86.
- Antonescu, M. and Stefan, G. M. (2020). Multi-function scan circuit. In *Proceedings of the 43rd International Semiconductor Conference CAS 2020*, pages 123–126.
- Antonescu, M. and Stefan, G. M. (2024). Multi-function scan circuit for assisting the parallel computational map pattern. *ROMJIST*, 27(1):3–22.
- Asanovic, K., Bodik, R., Catanzaro, B., Gebis, J., Husbands, P., Keutzer, K., Patterson, D. A., Plishker, W., Shalf, J., Williams, S., and Yelick, K. (2009). A view of the parallel computing landscape. *Communications of the ACM*, 52(10):56–67.
- Ashlock, R. B. (1998). *Error Patterns in Computation: Using Error Patterns to Improve Instruction*. Prentice Hall, Upper Saddle River, NJ, 6th edition.
- Bachrach, J., Vo, H., Richards, B., Lee, Y., Waterman, A., Avizienis, R., Wawrzynek, J., and Asanovic, K. (2012). Chisel: constructing hardware in a scala embedded language. In *Proceedings of the 49th Annual Design Automation Conference*, pages 1216–1225. ACM.
- Bobis, J. (2004). Enhancing mental computation in the classroom. *Educational Studies in Mathematics*, 57:93–115.
- Boole, G. (1854). *An Investigation of the Laws of Thought*. Macmillan, London.
- Boyer, C. B. and Merzbach, U. C. (1991). *A History of Mathematics*. Wiley, 2nd edition.
- Burton, D. M. (2010a). *Elementary Number Theory*. McGraw-Hill Education, 7th edition.
- Burton, D. M. (2010b). *The History of Mathematics: An Introduction*. McGraw-Hill, 7th edition.

- Chaitin, G. (1970). On the difficulty of computation. *IEEE Transactions of Information Theory*.
- Chaitin, G. (1977). Algorithmic information theory. *IBM J. Res. Develop.*
- Chisel Contributors (2023a). Chisel language documentation. <https://www.chisel-lang.org/>.
- Chisel Contributors (2023b). Chisel tutorials. <https://github.com/chipsalliance/chisel-tutorial>.
- Chisel Contributors (2023c). Chiseltest documentation. <https://github.com/ucb-bar/chisel-testers2>.
- Church, A. (1936). An unsolvable problem of elementary number theory. *American Journal of Mathematics*, 58:345–363.
- Consortium, T. U. (2020). *The Unicode Standard, Version 13.0.0*. The Unicode Consortium.
- Dadda, L. (1965). Some schemes for parallel multipliers. *Alta Frequenza*, 34(4):349–356.
- Developers, C. (2023). Chisel cheat sheet. https://github.com/freechipsproject/chisel-cheatsheet/blob/master/chisel_cheatsheet.pdf. Accessed: October 1, 2023.
- Foster, H. D. (2003). *Assertions in SystemVerilog*. Springer.
- Goedel, K. (1931). Über formal unentscheidbare sätze der principia mathematica und verwandter systeme, i. *Monatshefte für Mathematik und Physik*, 38:173–98. Kurt Goedel, 1992. On Formally Undecidable Propositions Of Principia Mathematica And Related Systems, tr. B. Meltzer, with a comprehensive introduction by Richard Braithwaite. Dover reprint of the 1962 Basic Books edition.
- Goedel, K. (1986). On formally decidable propositions of principia mathematica and related systems i. In Feferman, S., Jr., J. W. D., Kleene, S. C., Moore, G. H., Solovay, R. M., and van Heijenoort, J., editors, *Collected Works, Volume I: Publications 1929–1936*, pages 144–195. Oxford University Press, New York. Originally published in 1931 in *Monatshefte für Mathematik und Physik*.
- Goldberg, D. (1991). What every computer scientist should know about floating-point arithmetic. *ACM Computing Surveys*, 23(1):5–48.
- Grattan-Guinness, G. E. (1997). *The Search for Mathematical Roots, 1870–1940: Logics, Set Theories, and the Foundations of Mathematics from Cantor through Russell to Gödel*. Princeton University Press, Princeton. Includes historical discussion on the contributions of De Morgan and Boole.
- Gray, F. (1953). Pulse code communication. *US Patent 2,632,058*. Assigned to Bell Telephone Laboratories.
- Gustafson, J. (2015). *The End of Error: Unum Computing*. Chapman and Hall/CRC.
- Guyer, S. (2005). Introduction to systemverilog. In *Proceedings of the 2005 ACM/SIGDA 13th International Symposium on Field-Programmable Gate Arrays*, page 259. ACM.
- Hamming, R. W. (1980). *Coding and Information Theory*. Prentice Hall, Englewood Cliffs, NJ.

- Han, J. and Orshansky, M. (2013). Approximate computing: An emerging paradigm for energy-efficient design. *Proceedings of the 18th Asia and South Pacific Design Automation Conference (ASP-DAC)*, pages 365–370.
- Hennessy, J. L. and Patterson, D. A. (2019). *Computer Architecture: A Quantitative Approach*. Morgan Kaufmann, sixth edition edition.
- Heydari, M. and Abdel-Hamid, A. (2016). Residue number system: Applications in digital signal processing. *IEEE Signal Processing Magazine*, 33:127–137.
- Hilbert, D. (1900). Mathematical problems. lecture delivered before the international congress of mathematicians at paris in 1900. <https://mathcs.clarku.edu/~djoyce/hilbert/problems.html>.
- Hilbert, D. and Ackermann, W. (1928). *Grundzüge der theoretischen Logik (Principles of Mathematical Logic)*. Springer-Verlag, English translation: David Hilbert and Wilhelm Ackermann. *Principles of Mathematical Logic*. AMS Chelsea Publishing, Providence, Rhode Island, USA, 1950.
- Horowitz, P. and Hill, W. (1993). *The Art of Electronics*. Cambridge University Press, Cambridge, 2 edition.
- IBM (1964). Extended binary coded decimal interchange code (ebcdic). *IBM Systems Journal*, 3.
- IEEE (2008). Ieee standard for floating-point arithmetic.
- IEEE (2018). Ieee standard for systemverilog—unified hardware design, specification, and verification language.
- IEEE Standards Association (1984). *IEEE Std 91-1984: Standard Graphic Symbols for Logic Functions*. IEEE, New York.
- IEEE Standards Association (2011). *IEEE Std 181-2011: Standard for Transitions, Pulses, and Related Waveforms*. IEEE, New York.
- Ifrah, G. (2000a). *The Universal History of Numbers: From Prehistory to the Invention of the Computer*. Wiley.
- Ifrah, G. (2000b). *The Universal History of Numbers: From Prehistory to the Invention of the Computer*. Wiley, New York.
- Karnaugh, M. (1953). The map method for synthesis of combinational logic circuits. *Transactions of the American Institute of Electrical Engineers, Part I: Communication and Electronics*, 72(9):593–599.
- Kleene, S. C. (1936). General recursive functions of natural numbers. *Math. Ann.*, 112.
- Kline, M. (1990). *Mathematics: The Loss of Certainty*. Oxford University Press.
- Knuth, D. E. (1997a). *The Art of Computer Programming, Volume 2: Seminumerical Algorithms*. Addison-Wesley, Boston, 3rd edition.
- Knuth, D. E. (1997b). *The Art of Computer Programming, Volume 2: Seminumerical Algorithms*. Addison-Wesley.

- Koch, H. (1970). *Ancient Methods of Multiplication and Division*. Cambridge University Press, Cambridge.
- Kogge, W. J. and Stone, H. S. (1973). A parallel algorithm for the efficient solution of a general class of recurrence equations. *IEEE Transactions on Computers*, C-22(8):786–793.
- Koren, I. (2002). *Computer Arithmetic Algorithms*. A. K. Peters, Natick, MA, 2nd edition.
- Lathi, B. P. (2005). *Digital Signal Processing: Principles, Algorithms, and Applications*. Oxford University Press.
- Lukyanov, N. P. (1961). The setun computer: A balanced ternary digital machine. *Cybernetics and Systems Analysis*, 1:56–64.
- Mackenzie, C. E. (1980). *Coded Character Sets, History and Development*. Addison-Wesley Publishing Company.
- Maharaj, A. (2010). *Mathematics Education: Numeracy, Algebra, and Geometry*. Pearson, London.
- Mano, M. M. and Ciletti, M. D. (2017). *Digital Design*. Pearson, Boston, 5 edition.
- Maor, E. (2007). *The Pythagorean Theorem: A 4,000-Year History*. Princeton University Press.
- Martin, H. (1992). *European Mathematical Techniques in Primary Education*. Educational Research Books, Bristol.
- McCool, M., Robinson, A. D., and Reinders, J. (2012). *Structured Parallel Programming. Patterns for Efficient Computation*. Elsevier.
- Moore, R. E. and Lodwick, F. W. (2009). *Introduction to Interval Analysis*. SIAM.
- Morgan, A. D. (1847). *Formal Logic: or, The Calculus of Inference, Necessary and Probable*. Taylor and Walton, London.
- Munkres, J. R. (2000). *Topology*. Prentice Hall, Upper Saddle River, NJ, 2nd edition. Widely regarded for its rigorous treatment of metric spaces and topological concepts.
- Naylor, M. and Jones, P. (2013). Applications of logarithmic number systems. *IEEE Transactions on Computers*, 62(6):1154–1165.
- Neill, J. (1993). *Alternative Methods of Arithmetic*. Oxford University Press, Oxford.
- Nelson, D. (2002). *Subtraction Simplified: European and Swedish Methods*. Springer, New York.
- Niven, I. (1991). *Digital Systems and Complementary Arithmetic*. Wiley, New York.
- Odersky, M., Spoon, L., and Venners, B. (2008). *Programming in Scala*. Artima Inc.
- Oppenheim, A. V. and Schaffer, R. W. (2014). *Discrete-Time Signal Processing*. Pearson, Upper Saddle River, NJ, 3rd edition.
- Ore, O. (1990). *Number Theory and Its History*. Dover Publications.

- Parker, R. (2014). *Innovative Arithmetic: Advanced Techniques for Division*. Harvard Educational Press, Cambridge, MA.
- Patterson, D., Anderson, T., Cardwell, N., Fromm, R., Patel, K., Chawathe, Y., Dibble, M., Gibas, G., Keeton, K., Kopf, D. A., Mann, T., Thomas, R. H., and Yelick, K. (1997). A case for intelligent ram. *IEEE Micro*, 17(2):34–44.
- Patterson, D. A. (2010). The trouble with multicore. *IEEE Spectrum*, 47(7):28–32.
- Patterson, D. A. and Hennessy, J. L. (2005). *Computer Organization & Design: The Hardware / Software Interface*. Morgan Kaufmann, third edition edition.
- Patterson, D. A. and Hennessy, J. L. (2019). *Computer Organization & Design: The Hardware / Software Interface*. Morgan Kaufmann, sixth edition edition.
- Peirce, C. S. (1885). On the algebra of logic: A contribution to the philosophy of notation. *American Journal of Mathematics*, 7(2):180–202.
- Post, E. (1936). Finite combinatory processes. formulation i. *The Journal of Symbolic Logic*, 1:103–105.
- Quine, W. V. (1952). The problem of simplifying truth functions. *American Mathematical Monthly*, 59(8):521–531.
- Randell, B. (1982). The origins of digital computers. *Springer*.
- Reys, R. E. (1995). *Helping Children Learn Mathematics*. Allyn and Bacon, Boston, 5th edition.
- Robson, E. (2008). *Mathematics in Ancient Iraq: A Social History*. Princeton University Press, Princeton.
- Rosen, K. H. (2018). *Discrete Mathematics and Its Applications*. McGraw-Hill Education, New York, 8th edition.
- Ross, S. M. (2008). *A First Course in Probability*. Pearson, Boston, 8th edition.
- Rudin, W. (1976). *Principles of Mathematical Analysis*. McGraw-Hill Education, New York, NY, 3rd edition. A classic reference for metric spaces, completeness, and Cauchy sequences.
- Schoeberl, M. (2019). *Digital Design with Chisel*. Kindle Direct Publishing.
- Shannon, C. E. (1938). A symbolic analysis of relay and switching circuits. *Trans. American Institute of Electrical Engineers*, 57(12):713–723. The paper is an abstract of the thesis presented in 1937 at MIT for the degree of master in science.
- Shannon, C. E. (1948). A mathematical theory of communication. *Bell System Technical Journal*, 27:379–423, 623–656.
- Shannon, C. E. (1956). A universal turing machine with two internal states. In *Annals of Mathematics Studies, No. 34: Automata Studies*, pages 157–165. Princeton Univ. Press.
- Sheffer, H. M. (1913). A set of five independent postulates for boolean algebras, with application to logical constants. *Transactions of the American Mathematical Society*, 14(4):481–488.
- Smith, D. H. (2009). *Mathematics for Elementary Teachers*. McGraw-Hill, New York.

- Smith, D. T. and Doe, J. (1997). *The Science of Computer Arithmetic*. MIT Press.
- Smith, J. E. and Sohi, G. S. (1991). *The Microarchitecture of Superscalar Processors*. Springer-Verlag, New York.
- Stallings, W. (2020). *Computer Organization and Architecture: Designing for Performance*. Pearson, 11th edition.
- Stapko, T. (2008). *Advanced Verification Techniques: A SystemC Based Approach for Successful Tapeout*. Newnes.
- Stefan, G. M. (2000). Composition is the only independent rule in kleene’s model of partial recursive functions. <https://users.dcae.pub.ro/~gstefan/2ndLevel/composition.html>.
- Stefan, G. M. (2006). A universal turing machine with zero internal states. *Romanian Journal of Information Science and Technology*, 9(3):227–243.
- Stefan, G. M. (2024). Loops & complexity in digital systems. lecture notes in digital electronics. https://users.dcae.pub.ro/~gstefan/2ndLevel/teachingMaterials/0_CID_VOL1.pdf. Accessed: December 1, 2024.
- Stewart, I. (2008). *Taming the Infinite: The Story of Mathematics*. Quercus.
- Stewart, I. and Tall, D. (2015). *The Foundations of Mathematics*. Oxford University Press, Oxford, 2 edition.
- Sutherland, S. (2002). *Verilog 2001: A Guide to the New Features of the Verilog Hardware Description Language*. Kluwer Academic Publishers.
- Sutherland, S. and Mills, D. (2010). *SystemVerilog for Design: A Guide to Using SystemVerilog for Hardware Design and Modeling*. Springer Science & Business Media, 2nd edition.
- Tanenbaum, A. S. and Austin, T. (2013). *Structured Computer Organization*. Pearson, Boston, 6 edition.
- Tirthaji, B. K. (1992). *Vedic Mathematics*. Motilal Banarsidass, Delhi.
- Turing, A. (1937). On computable numbers, with an application to the entscheidungsproblem. *Proceedings of the London Mathematical Society*, 2(42):230–265.
- Turing, A. M. (1936). On computable numbers with an application to the entscheidungsproblem. *Proceedings of the London Mathematical Society*, 42. Extended in 1937, volume 43.
- Wallace, C. S. (1964). A suggestion for a fast multiplier. *IEEE Transactions on Electronic Computers*, EC-13(1):14–17.
- Wang, Y. and Goyal, P. (2019). Bfloat16: The next step in hardware efficiency for ai workloads. *Google AI Blog*.
- Wells, D. (2010). *The Penguin Dictionary of Curious and Interesting Numbers*. Penguin Books.

Index

Chisel

- dataType, 116

- gen, 116

- Traits, 98

fan-out: the number on input gates connected to the
output of a digital circuit, 486

gate, 488